Spring Boot 3

A Complete Guide

From Zero to Production-Ready REST APIs -- With Java 17+ and Spring Boot 3.4

2025 Edition • 30 Chapters + 3 Appendices

Table of Contents

Chapter 1 — What Is Spring Boot
Chapter 2 — Development Environment
Chapter 3 — First Spring Boot Project
Chapter 4 — Project Structure
Chapter 5 — Auto-Configuration
Chapter 6 — Dependency Injection
Chapter 7 — Beans and Annotations
Chapter 8 — Properties and Profiles
Chapter 9 — First REST Controller
Chapter 10 — Request and Response Handling
Chapter 11 — Input Validation
Chapter 12 — Exception Handling
Chapter 13 — Interceptors and Filters
Chapter 14 — Spring Data JPA and H2
Chapter 15 — Repositories and CRUD
Chapter 16 — Pagination and Sorting
Chapter 17 — Spring Security Introduction
Chapter 18 — JWT Authentication
Chapter 19 — Password Encoding and Roles
Chapter 20 — CORS Configuration
Chapter 21 — File Upload and Download
Chapter 22 — Logging
Chapter 23 — Scheduling
Chapter 24 — Caching
Chapter 25 — Sending Emails
Chapter 26 — Testing
Chapter 27 — API Documentation

Chapter 28 — Actuator and Monitoring
Chapter 29 — Externalized Configuration
Chapter 30 — Docker and Deployment

Appendix A — WebSocket Communication
Appendix B — Complete Project Walkthrough
Appendix C — Glossary

Chapter 1: What Is Spring Boot?

What You Will Learn

- What the Spring Framework is and why it exists.
- What Spring Boot is and how it improves Spring.
- The history of Spring and Spring Boot.
- How Spring Boot removes boilerplate code.
- What "convention over configuration" means.
- How plain Java, Spring, and Spring Boot compare for building web apps.
- The Spring ecosystem and its major projects.
- Real companies that use Spring Boot in production.

Why This Chapter Matters

Imagine you want to build a house. You could start by making your own bricks, mixing your own cement, and cutting your own wood. That would take forever. Or you could buy ready-made materials and focus on designing your dream home.

Spring Boot is like buying ready-made materials. It lets you focus on building your application instead of wasting time on setup and configuration. Before you write a single line of Spring Boot code, you need to understand what it is, where it came from, and why millions of developers choose it every day.

This chapter gives you that foundation. By the end, you will understand what Spring Boot does for you and why it has become the most popular way to build Java applications.

1.1 The Problem: Building Java Web Apps Is Hard

Let us start with a simple question. What does it take to build a web application in plain Java?

Here is what you need to do:

1. Write Java code for your business logic.
2. Set up a web server (like Apache Tomcat).
3. Write XML configuration files.
4. Configure database connections manually.
5. Handle security settings.
6. Package everything into a WAR file.
7. Deploy the WAR file to the server.

That is a lot of work before you can even say "Hello, World!" to a user.

Think of it like cooking a meal from scratch. You need to grow the vegetables, raise the chicken, mill the flour, and churn the butter. By the time you start cooking, you are already exhausted.

There had to be a better way. And there was.

1.2 What Is the Spring Framework?

The Spring Framework is a toolkit for building Java applications. It was created in 2003 by Rod Johnson.

> Toolkit: A collection of pre-built tools and libraries that help you do common tasks without writing everything from scratch.

Think of Spring as a well-organized toolbox. Need to connect to a database? There is a tool for that. Need to handle web requests? There is a tool for that too. Need to send emails? Yep, there is a tool for that as well.

Spring solves several big problems:

Problem	Spring's Solution
Objects depend on each other in messy ways	Dependency Injection (DI)
Database code is repetitive and error-prone	Spring Data
Handling web requests requires lots of setup	Spring MVC

Security is complex to implement	Spring Security
Configuration files are long and confusing	Annotation-based configuration

What "Framework" Means

A framework is a structure that your code fits into. You follow the framework's rules, and it handles the boring parts for you.

Think of a framework like a form you fill out at the doctor's office. The form gives you the structure (name here, date of birth there). You just fill in your specific information. The framework handles the layout and organization.

Spring's Core Idea: Dependency Injection

We will cover this in detail in Chapter 6. For now, here is the simple version.

In plain Java, if class A needs class B, class A creates class B itself:

```java
public class OrderService {
    // OrderService creates its own EmailService
    private EmailService emailService = new EmailService();
}
```

With Spring, you say "I need an EmailService" and Spring gives you one:

```java
public class OrderService {
    private final EmailService emailService;

    // Spring gives OrderService an EmailService automatically
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

This might seem like a small change. But it makes your code much easier to test, change, and maintain. More on this in Chapter 6.

1.3 The Problem with Spring: Too Much Configuration

Spring was a huge improvement over plain Java. But Spring had its own problem: configuration overload.

To build a simple web application with Spring, you needed to:

1. Create a `web.xml` file to configure the web layer.
2. Create Spring XML configuration files to define your beans.
3. Configure a database connection in another XML file.
4. Set up a view resolver for web pages.
5. Download and install a separate web server.
6. Package your app as a WAR file and deploy it.

Here is a taste of what Spring XML configuration looked like:

```xml
<!-- Spring XML configuration — you needed files like this -->
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mydb"/>
        <property name="username" value="root"/>
        <property name="password" value="password"/>
    </bean>

    <bean id="sessionFactory"
          class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
        <property name="dataSource" ref="dataSource"/>
        <property name="packagesToScan" value="com.example.model"/>
        <property name="hibernateProperties">
            <props>
                <prop key="hibernate.dialect">
                    org.hibernate.dialect.MySQLDialect
                </prop>
                <prop key="hibernate.show_sql">true</prop>
            </props>
        </property>
    </bean>
</beans>
```

That is a lot of XML just to connect to a database. And this is only one piece of the puzzle.

Developers joked that Spring stood for "Still Programming, Really? I Need to write more XML? No way, Give me something Simpler!"

Something simpler did come along.

1.4 Enter Spring Boot

Spring Boot was released in 2014. It was created by Pivotal (now part of VMware/Broadcom). The goal was simple: make it easy to create Spring applications with minimal setup.

> Spring Boot: A project built on top of Spring Framework that provides auto-configuration, embedded servers, and opinionated defaults so you can start building applications quickly.

Spring Boot did not replace Spring. It sits on top of Spring and does the configuration work for you.

Think of it this way:

- Plain Java = Building a house from raw materials.
- Spring Framework = Building a house from pre-made parts. You still need to read the blueprints and assemble everything.
- Spring Boot = Moving into a pre-built house. The walls, plumbing, and electricity are already done. You just decorate and arrange the furniture.

What Spring Boot Gives You

Spring Boot provides four key features:

1. Auto-Configuration: Spring Boot looks at your project dependencies and configures things automatically. Added a database driver? Spring Boot sets up the connection for you.
2. Embedded Server: No need to install Tomcat separately. Spring Boot includes a web server inside your application. Just run your app and the server starts.
3. Starter Dependencies: Instead of adding 10 individual libraries, you add one "starter" that includes everything you need.
4. Production-Ready Features: Health checks, metrics, and monitoring built in.

1.5 A Brief History

Here is how we got from plain Java to Spring Boot:

```
+------+    +--------+    +----------+    +-------------+
| 1995 |    |  2003  |    |   2014   |    |    Today    |
| Java |--->| Spring |--->| Spring   |--->| Spring Boot |
| Born |    | 1.0    |    | Boot 1.0 |    | 3.4+        |
+------+    +--------+    +----------+    +-------------+
   |            |              |                |
   v            v              v                v
 Write      Less code,     Minimal         Fastest way
 everything  but lots      config,          to build
 yourself    of XML        embedded         production
                           server           Java apps
```

Key Milestones:

Year	Event
1995	Java released by Sun Microsystems
2003	Spring Framework 1.0 released
2004	Spring 1.1 adds AOP (Aspect-Oriented Programming)
2009	Spring 3.0 adds Java-based configuration (less XML)
2013	Spring 4.0 adds Java 8 support
2014	Spring Boot 1.0 released
2018	Spring Boot 2.0 with Spring Framework 5
2022	Spring Boot 3.0 with Spring Framework 6 and Java 17 baseline
2024	Spring Boot 3.4 with improved auto-configuration and virtual threads support

---

1.6 Comparison: Plain Java vs Spring vs Spring Boot

Let us build a simple web endpoint that returns "Hello, World!" in three different ways. This comparison will show you why Spring Boot is so popular.

Approach 1: Plain Java (Using Servlets)

```java
// File: HelloServlet.java
// Plain Java approach — lots of boilerplate

import jakarta.servlet.http.HttpServlet;            // Line 1
import jakarta.servlet.http.HttpServletRequest;      // Line 2
import jakarta.servlet.http.HttpServletResponse;     // Line 3
import jakarta.servlet.annotation.WebServlet;        // Line 4
import java.io.IOException;                          // Line 5

@WebServlet("/hello")                                // Line 6
public class HelloServlet extends HttpServlet {      // Line 7

    @Override                                         // Line 8
    protected void doGet(HttpServletRequest request,  // Line 9
                         HttpServletResponse response) // Line 10
            throws IOException {                      // Line 11
        response.setContentType("text/plain");        // Line 12
        response.getWriter().write("Hello, World!");  // Line 13
    }                                                 // Line 14
}                                                     // Line 15
```

Line-by-line explanation:

- Lines 1-5: Import statements. You need to import servlet classes manually.
- Line 6: `@WebServlet("/hello")` maps this servlet to the `/hello` URL path.
- Line 7: Your class must extend `HttpServlet`. This is required.
- Lines 8-11: You override the `doGet` method to handle HTTP GET requests.
- Line 12: You manually set the response content type.
- Line 13: You manually write the response body.

But wait, there is more! You also need:

- A `web.xml` configuration file.
- An external Tomcat server installed and configured.
- A WAR file packaging step.
- Deployment to the Tomcat server.

That is a lot of work for "Hello, World!"

Approach 2: Spring Framework (Without Boot)

```java
// File: HelloController.java
// Spring MVC approach — better, but still needs setup

import org.springframework.stereotype.Controller;        // Line 1
import org.springframework.web.bind.annotation.GetMapping; // Line 2
import org.springframework.web.bind.annotation.ResponseBody; // Line 3

@Controller                                               // Line 4
public class HelloController {                            // Line 5

    @GetMapping("/hello")                                 // Line 6
    @ResponseBody                                         // Line 7
    public String hello() {                               // Line 8
        return "Hello, World!";                           // Line 9
    }                                                     // Line 10
}                                                         // Line 11
```

Line-by-line explanation:

- Line 1: `@Controller` tells Spring this class handles web requests.
- Line 6: `@GetMapping("/hello")` maps GET requests to `/hello`.
- Line 7: `@ResponseBody` tells Spring to return the string directly (not a web page name).
- Lines 8-9: A simple method that returns "Hello, World!"

This is cleaner code. But you still need:

- Multiple XML configuration files.
- An external Tomcat server.
- Careful dependency management.
- A WAR file and deployment process.

Approach 3: Spring Boot

```java
// File: HelloController.java
// Spring Boot approach — clean and simple

import org.springframework.web.bind.annotation.GetMapping;   // Line 1
import org.springframework.web.bind.annotation.RestController; // Line 2

@RestController                                               // Line 3
public class HelloController {                                // Line 4

    @GetMapping("/hello")                                     // Line 5
    public String hello() {                                   // Line 6
        return "Hello, World!";                               // Line 7
    }                                                         // Line 8
}                                                             // Line 9
```

Line-by-line explanation:

- Line 2: `@RestController` combines `@Controller` and `@ResponseBody`. One annotation instead of two.
- Line 5: `@GetMapping("/hello")` maps GET requests to `/hello`.
- Lines 6-7: Return "Hello, World!" That is it.

And the main application class:

```java
// File: Application.java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

No XML. No external server. No WAR file. Just run the `main` method and your web app is live.

The Comparison at a Glance

```
+---------------------+------------+--------+-------------+
|                     | Plain Java | Spring | Spring Boot |
+---------------------+------------+--------+-------------+
| XML config files    | Yes        | Yes    | No          |
| External server     | Yes        | Yes    | No          |
| Lines of setup code | 50+        | 30+    | ~5          |
| Time to "Hello"     | 30+ min    | 15 min | 2 min       |
| Dependency mgmt     | Manual     | Manual | Auto        |
+---------------------+------------+--------+-------------+
```

1.7 Convention Over Configuration

You will hear this phrase a lot in the Spring Boot world:

> Convention over configuration: The framework makes smart default choices for you. You only need to change settings when you want something different from the default.

Here is an analogy. When you check into a hotel room, you expect:

- A bed with clean sheets.
- A bathroom with hot water.
- Wi-Fi that works.
- A TV with basic channels.

You did not have to ask for any of that. It is the convention. The hotel assumed you would want these things.

But if you want extra pillows or a room with a view, you can request that. You configure only what you want to change.

Spring Boot works the same way:

Convention (Default)	You Can Configure
Embedded Tomcat server on port 8080	Change to port 9090 in `application.properties`
H2 in-memory database if H2 is on classpath	Switch to MySQL or PostgreSQL
`/static` folder serves static files	Change the static file location
`application.properties` for settings	Use `application.yml` instead

To change the default port, you only need one line:

```properties
# File: application.properties
server.port=9090
```

That is it. No XML. No Java configuration class. One line.

1.8 The Spring Ecosystem

Spring Boot is part of a larger family of projects called the Spring ecosystem. Here is how they fit together:

```
+-------------------------------------------------------------+
|                    SPRING ECOSYSTEM                         |
|                                                             |
|  +-------------------------------------------------+  |
|  |              SPRING BOOT                          |  |
|  |  (Auto-config, embedded server, starters)         |  |
```

```
|  |                                                  |  |
|  |  +-----------------------------------------------+ |  |
|  |  |           SPRING FRAMEWORK                    | |  |
|  |  |  (Core, MVC, Data Access, AOP)                | |  |
|  |  +-----------------------------------------------+ |  |
|  +-----------------------------------------------------+  |
|                                                            |
|  OTHER SPRING PROJECTS:                                    |
|  +----------------+  +----------------+                   |
|  | Spring Data    |  | Spring Security|                   |
|  | (Database      |  | (Login, roles, |                   |
|  |  access)       |  |  permissions)  |                   |
|  +----------------+  +----------------+                   |
|  +----------------+  +----------------+                   |
|  | Spring Cloud   |  | Spring Batch   |                   |
|  | (Microservices,|  | (Process large |                   |
|  |  distributed)  |  |  data sets)    |                   |
|  +----------------+  +----------------+                   |
|  +----------------+  +----------------+                   |
|  | Spring AMQP    |  | Spring GraphQL |                   |
|  | (Message       |  | (GraphQL API   |                   |
|  |  queues)       |  |  support)      |                   |
|  +----------------+  +----------------+                   |
+------------------------------------------------------------+
```

Key Spring Projects Explained:

- Spring Framework: The foundation. Provides core features like dependency injection and web support.
- Spring Boot: Sits on top of Spring Framework. Adds auto-configuration and embedded servers.
- Spring Data: Makes database access easy. Write less code to talk to databases.
- Spring Security: Handles login, passwords, roles, and permissions.
- Spring Cloud: Tools for building microservices (small, independent services that work together).
- Spring Batch: Processes large amounts of data in batches (like reading a million records from a file).

You do not need to learn all of these right now. In this book, we focus on Spring Boot with Spring Web and Spring Data.

1.9 Who Uses Spring Boot?

Spring Boot is not just a learning tool. It powers some of the biggest applications in the world:

Company	How They Use Spring Boot
Netflix	Runs backend microservices that serve millions of users
Amazon	Uses Spring Boot for internal tools and services
LinkedIn	Powers parts of their professional networking platform
Alibaba	Runs e-commerce backend services
JPMorgan Chase	Builds financial applications and trading platforms
Uber	Uses it for backend microservices

According to developer surveys, Spring Boot is consistently the most popular Java framework. It has millions of downloads every month.

Why do companies choose Spring Boot?

1. Fast development: Less setup means faster time to market.
2. Large talent pool: Many Java developers know Spring Boot.
3. Production-ready: Built-in monitoring, health checks, and metrics.
4. Scalable: Works for small apps and large enterprise systems.
5. Strong community: Excellent documentation, tutorials, and Stack Overflow support.

1.10 What Spring Boot Is NOT

Let us clear up some common misconceptions:

- Spring Boot is NOT a new language. It is Java. You write Java code.
- Spring Boot is NOT a replacement for Spring. It uses Spring underneath. It just makes Spring easier to use.
- Spring Boot is NOT only for web apps. You can build command-line tools, batch processors, messaging apps, and more.
- Spring Boot is NOT magic. It has sensible defaults, but you can change everything. Understanding what it does under the hood (which this book will teach you) makes you a better developer.

Common Mistakes

1. Confusing Spring and Spring Boot. Spring is the framework. Spring Boot is the tool that makes Spring easier to use. Spring Boot uses Spring internally.
2. Thinking Spring Boot is only for microservices. Spring Boot works great for monolithic applications too. You can build any type of Java application with it.
3. Skipping the fundamentals. Do not jump into Spring Boot without understanding basic Java concepts like classes, objects, interfaces, and inheritance.
4. Thinking you need to learn all Spring projects at once. Start with Spring Boot and Spring Web. Add more as you need them.

Best Practices

1. Learn Java basics first. You should be comfortable with classes, objects, methods, interfaces, and collections before starting Spring Boot.
2. Start with the latest stable version. This book uses Spring Boot 3.4+. Avoid older versions when starting fresh.
3. Use Java 17 or later. Spring Boot 3.x requires Java 17 as the minimum version. Java 17 is a Long-Term Support (LTS) release.
4. Read the official documentation. The Spring Boot documentation at `docs.spring.io` is excellent and well-organized.
5. Build projects, do not just read. The best way to learn is by typing the code yourself and running it.

Quick Summary

Spring Boot is a tool built on top of the Spring Framework. It removes the tedious configuration work and lets you build Java applications quickly. It provides auto-configuration (smart defaults), an embedded web server (no separate installation needed), and starter dependencies (one dependency instead of many). Spring Boot follows the principle of "convention over configuration" which means it makes sensible choices for you and you only change what you need to. It was released in 2014 and has become the most popular way to build Java applications. Companies like Netflix, Amazon, and LinkedIn use it in production.

Key Points

- Spring Framework (2003) is a toolkit for building Java applications.
- Spring Boot (2014) sits on top of Spring and provides auto-configuration.
- Spring Boot includes an embedded web server. No external Tomcat needed.
- "Convention over configuration" means smart defaults with easy overrides.
- Spring Boot requires Java 17+ (starting from Spring Boot 3.0).
- The Spring ecosystem includes many projects: Data, Security, Cloud, and more.
- Spring Boot is used by major companies in production systems.

Practice Questions

1. What is the main difference between Spring Framework and Spring Boot?
2. Name three things that Spring Boot provides that plain Spring does not.
3. What does "convention over configuration" mean? Give an example.
4. Why does Spring Boot 3.x require Java 17 as a minimum?
5. A colleague says "Spring Boot is only for building microservices." Is this correct? Explain your answer.

Exercises

Exercise 1: Research Spring Boot Versions

Visit the Spring Boot releases page at `https://github.com/spring-projects/spring-boot/releases`. Find the latest stable version. Write down:

- The version number.
- The release date.
- One new feature mentioned in the release notes.

Exercise 2: Identify the Spring Projects

Go to `https://spring.io/projects`. List five Spring projects and write one sentence about what each one does. Which ones do you think you will use most often?

Exercise 3: Compare Frameworks

Research one alternative to Spring Boot (such as Quarkus, Micronaut, or Jakarta EE). Write three similarities and three differences between it and Spring Boot.

What Is Next?

Now that you understand what Spring Boot is and why it matters, it is time to set up your development environment. In Chapter 2, you will install Java, an IDE, and Maven. By the end of that chapter, you will have all the tools you need to start building Spring Boot applications.

Chapter 2: Setting Up Your Development Environment

What You Will Learn

- What Java 17 is and how to install it on your computer
- The difference between JDK and JRE
- How to pick and install an IDE (IntelliJ IDEA or VS Code)
- How to install Maven, the build tool for Java projects
- How to verify that everything is installed correctly
- How to fix the most common setup errors

Why This Chapter Matters

Imagine you want to build a wooden table. Before you cut a single piece of wood, you need a workbench, a saw, a hammer, and a ruler. Without the right tools, you cannot build anything.

Software development works the same way. Before you write a single line of Spring Boot code, you need the right tools installed on your computer. This chapter walks you through every tool, step by step.

If you skip this chapter and jump to coding, you will hit confusing errors. Spend 30 minutes here now. Save hours of frustration later.

2.1 The Tools You Need

Here is the full list of tools. Do not worry. We will install each one together.

```
+-------------------------------------------------------+
|            Your Development Toolbox                   |
+-------------------------------------------------------+
|                                                       |
|   1. Java 17+ (JDK)  --  The language and compiler   |
|   2. IDE              --  The code editor             |
|   3. Maven            --  The build tool              |
|                                                       |
+-------------------------------------------------------+
```

Java 17+ (JDK) is the programming language. Spring Boot 3.4 requires Java 17 or newer.

IDE stands for Integrated Development Environment. It is a smart text editor built for writing code. Think of it as Microsoft Word, but for programmers.

Maven is a build tool. It downloads libraries, compiles your code, and packages your application. Think of it as a kitchen assistant that fetches ingredients, mixes them, and serves the dish.

2.2 Understanding JDK vs JRE

Before you install Java, you need to understand two terms: JDK and JRE.

JRE stands for Java Runtime Environment. It can run Java programs, but it cannot build them. Think of a JRE as a DVD player. It plays movies, but it cannot record them.

JDK stands for Java Development Kit. It includes the JRE plus tools to compile and debug code. Think of a JDK as a movie studio. It can both record and play movies.

```
+-----------------------------------------------+
|                    JDK                        |
|                                               |
|   +---------------------------------------+   |
|   |               JRE                     |   |
|   |                                       |   |
|   |   Java Virtual Machine (JVM)          |   |
|   |   Core Libraries                      |   |
|   |   (runs your programs)                |   |
|   |                                       |   |
|   +---------------------------------------+   |
|                                               |
|   + javac  (compiler - turns code into        |
|             bytecode)                         |
|   + jdb    (debugger - finds bugs)            |
|   + jar    (packager - bundles your app)      |
```

```
|                                                |
+------------------------------------------------+
```

Key takeaway: Always install the JDK, not just the JRE. You are a developer. You need the full toolkit.

2.3 Installing Java 17+

Step 1: Check If Java Is Already Installed

Open a terminal (Mac/Linux) or Command Prompt (Windows). Type this command:

```bash
java -version
```

If Java is installed, you will see output like this:

```
openjdk version "17.0.10" 2024-01-16
OpenJDK Runtime Environment (build 17.0.10+7)
OpenJDK 64-Bit Server VM (build 17.0.10+7, mixed mode)
```

The first number matters. It must be 17 or higher. If you see `17`, `21`, or `22`, you are good. If you see `11`, `8`, or `1.8`, you need to upgrade.

If Java is not installed, you will see an error:

```
'java' is not recognized as an internal or external command
```

or

```
-bash: java: command not found
```

This means Java is not on your computer yet. Follow the next step.

Step 2: Download and Install the JDK

There are several JDK distributions. They all contain the same core Java. Think of them like different brands of bottled water. The water is the same. The label is different.

We recommend Eclipse Temurin (by Adoptium). It is free, open source, and well-maintained.

1. Go to https://adoptium.net
2. Select Java 17 (LTS) or Java 21 (LTS)
3. Choose your operating system (Windows, Mac, or Linux)
4. Download the installer
5. Run the installer. Accept the defaults.

> LTS stands for Long-Term Support. These versions receive security updates for many years. Always pick an LTS version for learning.

Step 3: Verify the Installation

Close your terminal and open a new one. This is important. The old terminal does not know about the new installation.

```bash
java -version
```

Expected output:

```
openjdk version "17.0.10" 2024-01-16
OpenJDK Runtime Environment Temurin-17.0.10+7 (build 17.0.10+7)
OpenJDK 64-Bit Server VM Temurin-17.0.10+7 (build 17.0.10+7, mixed mode)
```

Also check the compiler:

```bash
javac -version
```

Expected output:

```
javac 17.0.10
```

If `java` works but `javac` does not, you installed the JRE instead of the JDK. Go back and install the JDK.

Step 4: Check the JAVA_HOME Variable

Many tools need a variable called `JAVA_HOME`. This variable tells other programs where Java lives on your computer.

On Mac/Linux:

```bash
echo $JAVA_HOME
```

On Windows (Command Prompt):

```cmd
echo %JAVA_HOME%
```

You should see a path like:

```
/usr/lib/jvm/temurin-17-jdk       (Linux)
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home  (Mac)
C:\Program Files\Eclipse Adoptium\jdk-17.0.10.7-hotspot        (Windows)
```

If it is empty, you need to set it. Here is how:

Mac/Linux (add to ~/.bashrc or ~/.zshrc):

```bash
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export PATH=$JAVA_HOME/bin:$PATH
```

Then reload:

```bash
source ~/.zshrc
```

Windows:

1. Open System Properties > Environment Variables
2. Click "New" under System Variables
3. Variable name: `JAVA_HOME`
4. Variable value: the path to your JDK folder (e.g., `C:\Program Files\Eclipse Adoptium\jdk-17.0.10.7-hotspot`)
5. Click OK
6. Edit the `Path` variable and add `%JAVA_HOME%\bin`

2.4 Choosing and Installing an IDE

You can write Java in Notepad. But that is like eating soup with a fork. It works, but it is painful.

An IDE gives you:

- Syntax highlighting -- colors different parts of your code so it is easy to read
- Auto-complete -- suggests code as you type
- Error detection -- underlines mistakes before you even run the code
- Debugging -- lets you pause your program and inspect its state
- Built-in terminal -- run commands without leaving the editor

Option A: IntelliJ IDEA (Recommended)

IntelliJ IDEA is the most popular IDE for Java. The Community Edition is free and has everything you need for this book.

1. Go to https://www.jetbrains.com/idea/download
2. Download the Community Edition (free)
3. Run the installer
4. Open IntelliJ IDEA
5. On the Welcome screen, click "New Project" to verify it detects your JDK

```
+-----------------------------------------+
|         IntelliJ IDEA Editions          |
+-----------------------------------------+
|                                         |
|  Community Edition (Free)               |
|    - Java, Kotlin, Groovy              |
|    - Maven, Gradle                      |
|    - Git integration                    |
|    - Good enough for this book!         |
|                                         |
|  Ultimate Edition (Paid)                |
|    - Everything in Community            |
|    - Spring Boot support                |
|    - Database tools                     |
|    - Web frameworks                     |
|                                         |
+-----------------------------------------+
```

> Tip: The Community Edition does not have built-in Spring Boot support, but it works perfectly fine. You just will not get the Spring-specific auto-complete. For learning, this is not a problem.

Option B: Visual Studio Code

VS Code is a lightweight, free editor made by Microsoft. It works well with Java when you add the right extensions.

1. Go to https://code.visualstudio.com
2. Download and install
3. Open VS Code
4. Install these extensions (click the square icon on the left sidebar):
    - Extension Pack for Java (by Microsoft)
    - Spring Boot Extension Pack (by VMware)

```
+-----------------------------------------+
|     VS Code Extensions to Install       |
+-----------------------------------------+
|                                         |
|  1. Extension Pack for Java             |
|     - Language support                  |
|     - Debugger                          |
|     - Test runner                       |
|     - Maven support                     |
|                                         |
|  2. Spring Boot Extension Pack          |
|     - Spring Initializr integration     |
|     - Spring Boot dashboard             |
|     - Spring Boot property editor       |
|                                         |
+-----------------------------------------+
```

Which IDE Should You Pick?

Feature	IntelliJ Community	VS Code
Java Support	Excellent	Good
Setup Effort	Low	Medium
Memory Usage	Higher	Lower
Spring Support	Basic (free)	Good (plugins)
Learning Curve	Easy	Easy
Best For	Java-focused work	Multi-language

Our recommendation: If you only write Java, pick IntelliJ IDEA. If you work with multiple languages (JavaScript, Python, etc.), pick VS Code.

Both work. Pick one and move on. Do not spend hours deciding.

2.5 Installing Maven

What Is Maven?

Maven is a build tool for Java. It does three main jobs:

1. Downloads libraries -- You tell Maven what libraries you need. It downloads them for you.
2. Compiles code -- It turns your `.java` files into `.class` files (bytecode).
3. Packages your app -- It bundles everything into a single `.jar` file you can run.

Think of Maven as a recipe manager in a kitchen. You write a recipe (the `pom.xml` file). Maven reads the recipe, fetches the ingredients (libraries), and cooks the dish (builds your app).

```
+-----------------------------------------------------+
|              How Maven Works                        |
+-----------------------------------------------------+
|                                                     |
|   You write:   pom.xml  (the recipe)                |
|                  |                                   |
|                  v                                   |
|   Maven reads the recipe                            |
|                  |                                   |
|                  v                                   |
|   Maven downloads libraries from the internet       |
|   (called "dependencies")                           |
|                  |                                   |
|                  v                                   |
|   Maven compiles your Java code                     |
|                  |                                   |
|                  v                                   |
|   Maven packages everything into a .jar file        |
|                                                     |
+-----------------------------------------------------+
```

Do You Need to Install Maven Separately?

Short answer: No, if you use Spring Boot. Spring Boot projects include a file called `mvnw` (Maven Wrapper). This file downloads and uses the correct Maven version automatically.

But it is still good to have Maven installed globally. Some tutorials assume it. Some commands are easier to type with `mvn` instead of `./mvnw`.

Installing Maven

On Mac (using Homebrew):

```bash
brew install maven
```

On Linux (Ubuntu/Debian):

```bash
sudo apt update
sudo apt install maven
```

On Windows:

1. Go to https://maven.apache.org/download.cgi
2. Download the Binary zip archive
3. Extract to `C:\Program Files\Maven`
4. Add `C:\Program Files\Maven\bin` to your `Path` environment variable

Verify Maven

```bash
mvn -version
```

Expected output:

```
Apache Maven 3.9.6 (bc0240f3c744dd6b6ec2920b3cd08dcc295161ae)
Maven home: /usr/local/Cellar/maven/3.9.6/libexec
Java version: 17.0.10, vendor: Eclipse Adoptium
Default locale: en_US, platform encoding: UTF-8
OS name: "mac os x", version: "14.3", arch: "aarch64", family: "mac"
```

Notice that Maven also shows the Java version it is using. Make sure this matches the Java 17+ you installed.

2.6 Verifying Your Complete Setup

Let us run all three checks in one go. Open a terminal and type:

```bash
java -version
javac -version
mvn -version
```

If all three commands produce output (no errors), your setup is complete.

```
+-----------------------------------------------------+
|         Setup Verification Checklist                |
+-----------------------------------------------------+
|                                                     |
|  [x]  java -version    -->  17 or higher            |
|  [x]  javac -version   -->  17 or higher            |
|  [x]  mvn -version     -->  3.8 or higher           |
|  [x]  JAVA_HOME set    -->  points to JDK           |
|  [x]  IDE installed    -->  IntelliJ or VS Code     |
|                                                     |
+-----------------------------------------------------+
```

2.7 Common Setup Errors and Fixes

Error 1: "java is not recognized" or "command not found"

Cause: Java is not in your system PATH.

Fix:

- Make sure you installed the JDK (not just downloaded it).
- Add the JDK `bin` folder to your PATH variable.
- Close and reopen your terminal after making changes.

Error 2: java -version shows Java 8 or 11

Cause: An older version of Java is installed, and your system is using that one.

Fix:

- Install Java 17+ alongside the old version.
- Set `JAVA_HOME` to point to the new JDK.
- On Mac, use `/usr/libexec/java_home -V` to see all installed versions.

- On Windows, move the new JDK path above the old one in the PATH variable.

Error 3: javac works but java does not (or vice versa)

Cause: You have mismatched JDK/JRE installations. The `java` command points to one installation. The `javac` command points to another.

Fix:

- Use `which java` (Mac/Linux) or `where java` (Windows) to find where each command lives.
- Make sure both point to the same JDK folder.

Error 4: Maven uses the wrong Java version

Cause: Maven looks at the `JAVA_HOME` variable. If `JAVA_HOME` points to an old JDK, Maven will use that one.

Fix:

- Set `JAVA_HOME` to point to your Java 17+ JDK.
- Run `mvn -version` again. Check the "Java version" line.

Error 5: IntelliJ does not detect the JDK

Cause: IntelliJ cannot find the JDK on your system.

Fix:

1. Open IntelliJ IDEA
2. Go to File > Project Structure > SDKs
3. Click the "+" button
4. Choose "JDK"
5. Browse to your JDK installation folder
6. Click OK

Error 6: "Permission denied" when running mvnw

Cause: On Mac/Linux, the Maven Wrapper script does not have execute permission.

Fix:

```bash
chmod +x mvnw
```

This command gives the file permission to run as a program.

Common Mistakes

1. Installing JRE instead of JDK. You need the JDK. The JRE cannot compile code.
2. Not restarting the terminal. After installing Java or setting environment variables, you must open a new terminal window.
3. Forgetting JAVA_HOME. Many tools (Maven, IntelliJ, Gradle) depend on this variable. Always set it.
4. Using Java 8 or 11 with Spring Boot 3.4. Spring Boot 3.4 requires Java 17 at minimum. It will not start with older versions.
5. Spending too long choosing an IDE. Pick IntelliJ or VS Code. Either one works. Move on.

Best Practices

1. Use an LTS version of Java. Java 17 and Java 21 are LTS. They receive updates for years. Avoid non-LTS versions like Java 18 or 19.
2. Set JAVA_HOME immediately. Do this right after installing Java. It prevents problems later.
3. Use the Maven Wrapper (mvnw) in projects. It ensures every developer on a team uses the same Maven version.
4. Keep your IDE updated. IDE updates include bug fixes and better Java support.
5. Learn keyboard shortcuts. Speed comes from knowing your IDE. Learn five shortcuts this week. Learn five more next week.

Quick Summary

In this chapter, you installed three essential tools:

- JDK 17+ -- The Java compiler and runtime. You verified it with `java -version` and `javac -version`.

- An IDE -- IntelliJ IDEA Community Edition or Visual Studio Code. Your smart code editor.
- Maven -- The build tool that manages libraries and compiles code. You verified it with `mvn -version`.

You also learned the difference between JDK and JRE. The JDK is the full toolkit for developers. The JRE is just the player.

Key Points

- Spring Boot 3.4 requires Java 17 or higher.
- Always install the JDK, not the JRE. The JDK includes the compiler (`javac`).
- Set the `JAVA_HOME` environment variable. Many tools depend on it.
- Maven is a build tool. It downloads libraries, compiles code, and packages applications.
- Spring Boot projects include `mvnw` (Maven Wrapper) so you do not strictly need Maven installed globally.
- IntelliJ IDEA Community Edition and VS Code are both good choices for Java development.

Practice Questions

1. What is the difference between JDK and JRE? Why do developers need the JDK?
2. You run `java -version` and see `openjdk version "11.0.2"`. Can you use this version with Spring Boot 3.4? What should you do?
3. After installing Java 17, you open a terminal and type `java -version`, but it still shows Java 11. What is the most likely cause? How do you fix it?
4. What does the `JAVA_HOME` environment variable do? Name two tools that depend on it.
5. What is the purpose of the Maven Wrapper (`mvnw`) file in a Spring Boot project?

Exercises

Exercise 1: Verify Your Setup

Open a terminal and run these commands. Write down the output of each one:

```bash
java -version
javac -version
mvn -version
echo $JAVA_HOME    # Mac/Linux
echo %JAVA_HOME%   # Windows
```

Confirm that:

- Java version is 17 or higher
- `javac` version matches the `java` version
- Maven uses the same Java version
- `JAVA_HOME` points to your JDK folder

Exercise 2: Explore the JDK

Navigate to your JDK installation folder. Find the `bin` directory. List the files inside it. You will see tools like `java`, `javac`, `jar`, `jdb`, and many more. Write down what three of these tools do. (Hint: run any tool with `--help` to see its description.)

Exercise 3: Create a Simple Java Program

Create a file called `Hello.java` with this content:

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("My tools are ready!");
    }
}
```

Compile and run it from the terminal:

```bash
javac Hello.java
java Hello
```

Expected output:

```
My tools are ready!
```

If this works, your Java setup is correct.

What Is Next?

Your workbench is ready. Your tools are sharp. In the next chapter, you will use these tools to create your first Spring Boot project. You will visit Spring Initializr, generate a project, import it into your IDE, and see your first web application running in the browser.

Chapter 3: Your First Spring Boot Project

What You Will Learn

- How to use Spring Initializr to generate a project
- What each option on Spring Initializr means
- How to add the Spring Web dependency
- How to import the project into your IDE
- What the `@SpringBootApplication` annotation does
- How to run your application and see it in a browser
- How to create a `/hello` endpoint that returns a message
- How to fix the most common startup errors

Why This Chapter Matters

You have your tools ready. Now it is time to build something real.

In this chapter, you will create a Spring Boot application from scratch. By the end, you will have a web server running on your computer. You will type a URL in your browser and see a response. That response comes from code you wrote.

This is your first taste of Spring Boot. It takes less than 10 minutes.

3.1 What Is Spring Initializr?

Spring Initializr is a website that generates a starter project for you. Think of it as a pizza ordering form. You pick your crust (Java version), your size (packaging), your toppings (dependencies), and it gives you a ready-made project to download.

You can find it at https://start.spring.io.

```
+-----------------------------------------------------+
|              Spring Initializr                      |
|              (start.spring.io)                      |
+-----------------------------------------------------+
|                                                     |
|   You choose:                                       |
|     - Project type (Maven or Gradle)                |
|     - Language (Java, Kotlin, Groovy)               |
|     - Spring Boot version                           |
|     - Project metadata (name, package)              |
|     - Dependencies (the libraries you need)         |
|                                                     |
|   You get:                                          |
|     - A .zip file with a complete project           |
|     - Ready to import into your IDE                 |
|     - Ready to run                                  |
|                                                     |
+-----------------------------------------------------+
```

3.2 Creating the Project Step by Step

Open your browser and go to https://start.spring.io.

You will see a form with several options. Let us go through each one.

Option 1: Project

Choose Maven. Maven is the build tool we installed in the previous chapter.

> Gradle is another build tool. It is also popular. But Maven is easier for beginners and more common in tutorials. We will use Maven throughout this book.

Option 2: Language

Choose Java. This book uses Java.

Option 3: Spring Boot Version

Choose the latest stable version. At the time of writing, that is 3.4.x. Do not pick versions marked with "SNAPSHOT" or "M1". Those are pre-release versions. They may have bugs.

```
+-------------------------------------------+
|     Picking the Right Version             |
+-------------------------------------------+
|                                           |
|   3.4.4          <-- Pick this (stable)   |
|   3.4.5-SNAPSHOT <-- Skip (pre-release)   |
|   3.5.0-M1       <-- Skip (milestone)    |
|   3.5.0-SNAPSHOT <-- Skip (pre-release)   |
|                                           |
+-------------------------------------------+
```

SNAPSHOT means the version is still being built. It changes every day. M1 means "Milestone 1". It is a preview of an upcoming release. Both are unstable.

Option 4: Project Metadata

Fill in these fields:

Field	Value	What It Means
Group	`com.example`	Your organization's domain, reversed
Artifact	`hello-spring-boot`	The name of your project
Name	`hello-spring-boot`	Same as artifact (auto-filled)
Description	`My first Spring Boot app`	A short description
Package	`com.example.hellospringboot`	The base Java package for your code
Packaging	`Jar`	How Maven packages the app
Java	`17`	The Java version

Group is usually your company's domain name in reverse. For example, if your company is `acme.com`, the group is `com.acme`. For personal projects, `com.example` is fine.

Artifact is the name of your project. It becomes the name of the `.jar` file Maven creates.

Package is the Java package where your code lives. Java uses packages to organize code, like folders organize files.

Option 5: Dependencies

Click the "ADD DEPENDENCIES" button. Search for Spring Web. Click it to add it.

Spring Web gives you everything you need to build a web application:

- An embedded web server (Tomcat)
- Tools to handle HTTP requests (GET, POST, etc.)
- Tools to return responses (HTML, JSON, plain text)

```
+-------------------------------------------+
|     What Spring Web Gives You             |
+-------------------------------------------+
|                                           |
|   Embedded Tomcat server                  |
|     (no need to install a server)         |
|                                           |
|   @RestController annotation              |
|     (marks a class as a web controller)   |
|                                           |
|   @GetMapping annotation                  |
|     (maps a URL to a method)              |
|                                           |
|   JSON support (Jackson library)          |
|     (converts Java objects to JSON)       |
|                                           |
+-------------------------------------------+
```

Generate and Download

Click the GENERATE button. A `.zip` file will download. Extract it to a folder you will remember, such as `~/projects/hello-spring-boot`.

3.3 Importing the Project into Your IDE

IntelliJ IDEA

1. Open IntelliJ IDEA
2. Click Open (not "New Project")
3. Navigate to the extracted folder
4. Select the `pom.xml` file
5. IntelliJ will ask: "Open as Project?" Click Yes

6. Wait for IntelliJ to download dependencies (you will see a progress bar at the bottom)

VS Code

1. Open VS Code
2. Click File > Open Folder
3. Navigate to the extracted folder and open it
4. VS Code will detect the Java project and start downloading dependencies
5. Wait for the Java extension to finish loading (check the bottom status bar)

> Tip: The first time you open the project, your IDE will download many files. This is normal. Maven is fetching all the libraries Spring Boot needs. It can take a few minutes on a slow connection.

3.4 Understanding @SpringBootApplication

Open the main class. It is in:

```
src/main/java/com/example/hellospringboot/HelloSpringBootApplication.java
```

Here is what the file looks like:

```java
package com.example.hellospringboot;              // 1

import org.springframework.boot.SpringApplication; // 2
import org.springframework.boot.autoconfigure.SpringBootApplication; // 3

@SpringBootApplication                             // 4
public class HelloSpringBootApplication {          // 5

    public static void main(String[] args) {       // 6
        SpringApplication.run(                     // 7
            HelloSpringBootApplication.class,      // 8
            args                                   // 9
        );
    }
}
```

Line-by-line explanation:

- Line 1: `package com.example.hellospringboot;` -- This declares the package. Packages are like folders for your Java classes. Every class must belong to a package.
- Lines 2-3: `import ...` -- These lines import classes from the Spring Boot library. Java needs to know where each class comes from.
- Line 4: `@SpringBootApplication` -- This is an annotation. An annotation is a label you attach to code. It tells Spring Boot: "This class is the starting point of the application." We will explore what this annotation does internally in Chapter 5.
- Line 5: `public class HelloSpringBootApplication` -- This is the main class of your application. The name matches the file name.
- Line 6: `public static void main(String[] args)` -- This is the entry point. When you run the application, Java looks for a `main` method and starts there. Every Java application has one.
- Lines 7-9: `SpringApplication.run(...)` -- This line starts Spring Boot. It does a lot behind the scenes: it starts the web server, scans for components, loads configuration, and more. Think of it as turning the key in the ignition of a car.

```
+-------------------------------------------------------+
|     What SpringApplication.run() Does                 |
+-------------------------------------------------------+
|                                                       |
|   1. Creates the Spring application context           |
|      (the container that holds all your beans)        |
|                                                       |
|   2. Scans for annotated classes                      |
|      (@Controller, @Service, @Repository, etc.)       |
|                                                       |
|   3. Configures everything automatically              |
|      (database connections, web server, etc.)         |
|                                                       |
|   4. Starts the embedded web server (Tomcat)          |
|      on port 8080 by default                          |
|                                                       |
+-------------------------------------------------------+
```

3.5 Running the Application

From the IDE

IntelliJ IDEA:

- Click the green play button next to the `main` method

- Or right-click the file and choose "Run"

VS Code:

- Click the "Run" link that appears above the `main` method
- Or open the Command Palette (Ctrl+Shift+P) and type "Spring Boot: Run"

From the Terminal

Navigate to your project folder and run:

```bash
./mvnw spring-boot:run
```

On Windows:

```bash
mvnw.cmd spring-boot:run
```

What You Should See

The console will show many lines of output. Look for these important lines:

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.4.4)

2024-01-15T10:30:00.123  INFO --- Starting HelloSpringBootApplication
2024-01-15T10:30:01.456  INFO --- Tomcat initialized with port 8080 (http)
2024-01-15T10:30:02.789  INFO --- Started HelloSpringBootApplication in 2.5 seconds
```

The banner (the Spring logo made of text) tells you Spring Boot is running. The last line says "Started" with a time. Your application is live.

Opening It in the Browser

Open your web browser. Type this URL in the address bar:

```
http://localhost:8080
```

localhost means "this computer." 8080 is the port number. Together, they mean: "Connect to the web server running on this computer, on port 8080."

You will see an error page that says "Whitelabel Error Page." This is normal. We have not created any web pages yet. The error page means the server is running. It just has nothing to show.

```
+-----------------------------------------------------+
|              What Is localhost?                     |
+-----------------------------------------------------+
|                                                     |
|   Your computer has an address: 127.0.0.1           |
|   "localhost" is a shortcut for that address.       |
|                                                     |
|   When you type localhost:8080, your browser        |
|   talks to the web server running on your own       |
|   computer, on port 8080.                           |
|                                                     |
|   Browser ---request---> localhost:8080              |
|   Browser <--response--- Tomcat (Spring Boot)       |
|                                                     |
+-----------------------------------------------------+
```

3.6 Creating Your First Endpoint: /hello

Now let us make the application do something useful. We will create an endpoint that responds with "Hello, Spring Boot!" when you visit `/hello` in your browser.

Create a new file called `HelloController.java` in the same package as the main class:

```
src/main/java/com/example/hellospringboot/HelloController.java
```

Type this code:

```java
package com.example.hellospringboot;                    // 1

import org.springframework.web.bind.annotation.GetMapping;     // 2
import org.springframework.web.bind.annotation.RestController;  // 3

@RestController                                         // 4
public class HelloController {                          // 5
```

```
    @GetMapping("/hello")                                // 6
    public String sayHello() {                           // 7
        return "Hello, Spring Boot!";                    // 8
    }
}
```

Line-by-line explanation:

- Line 1: `package com.example.hellospringboot;` -- Same package as the main class. This is important. Spring Boot scans this package for components.
- Lines 2-3: Import the annotations we need.
- Line 4: `@RestController` -- This annotation tells Spring Boot: "This class handles web requests." The word "Rest" refers to REST, a style of building web APIs. The word "Controller" means this class controls what happens when someone visits a URL.
- Line 5: The class declaration. The name `HelloController` is just a convention. You can name it anything.
- Line 6: `@GetMapping("/hello")` -- This annotation tells Spring Boot: "When someone sends a GET request to `/hello`, run this method." A GET request is what your browser sends when you type a URL and press Enter.
- Line 7: `public String sayHello()` -- This method returns a `String`. Whatever string it returns will be sent back to the browser.
- Line 8: `return "Hello, Spring Boot!";` -- The response. The browser will display this text.

```
+-----------------------------------------------------+
|           How the Request Flows                     |
+-----------------------------------------------------+
|                                                     |
|   Browser types: http://localhost:8080/hello        |
|        |                                            |
|        v                                            |
|   Browser sends GET request to /hello               |
|        |                                            |
|        v                                            |
|   Tomcat receives the request                       |
|        |                                            |
|        v                                            |
|   Spring finds @GetMapping("/hello")                |
|        |                                            |
|        v                                            |
|   Spring calls sayHello() method                    |
|        |                                            |
|        v                                            |
|   Method returns "Hello, Spring Boot!"              |
|        |                                            |
|        v                                            |
|   Tomcat sends the string back to the browser       |
```

```
|        |                                            |
|        v                                            |
|   Browser displays: Hello, Spring Boot!             |
|                                                     |
+-----------------------------------------------------+
```

Run and Test

1. Stop the application if it is still running (press Ctrl+C in the terminal, or click the stop button in the IDE).
2. Start it again (so it picks up the new class).
3. Open your browser and go to:

```
http://localhost:8080/hello
```

Expected output in the browser:

```
Hello, Spring Boot!
```

Congratulations! You just built a web endpoint. Your browser sent a request. Spring Boot handled it. Your method returned a response. The browser displayed it.

3.7 Adding Another Endpoint

Let us add one more endpoint to practice. Add this method inside the `HelloController` class, after the `sayHello()` method:

```java
@GetMapping("/goodbye")                             // 1
public String sayGoodbye() {                        // 2
    return "Goodbye! Thanks for visiting.";         // 3
}
```

The complete `HelloController.java` now looks like this:

```java
package com.example.hellospringboot;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Spring Boot!";
    }

    @GetMapping("/goodbye")
    public String sayGoodbye() {
        return "Goodbye! Thanks for visiting.";
    }
}
```

Restart the application. Visit:

- `http://localhost:8080/hello` -- shows "Hello, Spring Boot!"
- `http://localhost:8080/goodbye` -- shows "Goodbye! Thanks for visiting."

Each `@GetMapping` maps a URL path to a method. You can add as many endpoints as you want.

3.8 Common Startup Errors

Error 1: Port 8080 Already in Use

```
Web server failed to start. Port 8080 was already in use.
```

Cause: Another application is already using port 8080. This could be another Spring Boot app you forgot to stop, or a different program.

Fix (Option A): Find and stop the other application.

On Mac/Linux:

```bash
lsof -i :8080
```

This shows the process using port 8080. Note the PID (process ID) and kill it:

```bash
kill -9 <PID>
```

On Windows:

```cmd
netstat -ano | findstr :8080
taskkill /PID <PID> /F
```

Fix (Option B): Change the port. Open `src/main/resources/application.properties` and add:

```properties
server.port=8081
```

Now the app runs on port 8081. Visit `http://localhost:8081/hello` instead.

Error 2: No Compiler Is Provided

```
No compiler is provided in this environment.
Perhaps you are running on a JRE rather than a JDK?
```

Cause: Maven found a JRE, not a JDK. The JRE cannot compile code.

Fix: Install the JDK and set `JAVA_HOME` to point to the JDK folder. See Chapter 2.

Error 3: Class Not Found or Bean Not Created

Your `/hello` endpoint returns a 404 (not found) error.

Cause: The controller class is in the wrong package. Spring Boot scans the package of the main class and all sub-packages. If your controller is in a different package tree, Spring Boot will not find it.

```
CORRECT:
  com.example.hellospringboot
    ├── HelloSpringBootApplication.java   (main class)
    └── HelloController.java             (same package - found!)

  com.example.hellospringboot.controller
    └── HelloController.java             (sub-package - also found!)

WRONG:
  com.example.hellospringboot
    └── HelloSpringBootApplication.java   (main class)
  com.example.controller
```

```
    └── HelloController.java            (different package tree - NOT found!)
```

Fix: Move the controller class into the same package as the main class, or into a sub-package.

Error 4: Application Starts but Stops Immediately

```
Started HelloSpringBootApplication in 1.2 seconds
```

Then the application exits.

Cause: You did not add the Spring Web dependency. Without it, there is no web server. The application starts, finds nothing to do, and stops.

Fix: Add Spring Web to your `pom.xml`:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Or regenerate the project from Spring Initializr with the Spring Web dependency selected.

Error 5: Whitelabel Error Page (404) at localhost:8080

Cause: The server is running, but there is no endpoint mapped to `/`. This is normal if you only created a `/hello` endpoint.

Fix: Either visit `/hello` instead, or add a mapping for `/`:

```java
@GetMapping("/")
public String home() {
    return "Welcome to my Spring Boot app!";
}
```

Common Mistakes

1. Forgetting to add Spring Web. Without it, the application has no web server. It starts and stops immediately.
2. Putting the controller in the wrong package. The controller must be in the same package as `@SpringBootApplication`, or in a sub-package.
3. Not restarting the application after code changes. Spring Boot does not pick up changes automatically (unless you add DevTools). Stop and restart.
4. Using a SNAPSHOT version of Spring Boot. Snapshots are unstable. Use the latest stable release.
5. Typing `localhost:8080` without `http://`. Some browsers handle this. Others do not. Always use the full URL: `http://localhost:8080`.

Best Practices

1. Start with Spring Initializr. Never create a Spring Boot project from scratch. The Initializr generates the correct structure and configuration.
2. Add only the dependencies you need. Each dependency adds complexity. Start small. Add more later.
3. Keep controllers in the same package tree as the main class. This ensures Spring Boot can find them.
4. Use `@RestController` for API endpoints. It combines `@Controller` and `@ResponseBody`, which means methods return data directly (not view names).
5. Test endpoints in the browser or with curl. Verify each endpoint works before building the next one.

Quick Summary

In this chapter, you:

1. Used Spring Initializr (start.spring.io) to generate a project with Maven, Java 17, and the Spring Web dependency.
2. Imported the project into your IDE.
3. Explored the main class and understood `@SpringBootApplication` and `SpringApplication.run()`.
4. Ran the application and accessed `http://localhost:8080`.

5. Created a `HelloController` with a `@GetMapping("/hello")` endpoint.
6. Saw "Hello, Spring Boot!" in your browser.

Key Points

- Spring Initializr (start.spring.io) generates ready-to-run Spring Boot projects.
- The Spring Web dependency gives you an embedded Tomcat server and web annotations.
- `@SpringBootApplication` marks the main class. It is the entry point of the application.
- `SpringApplication.run()` starts the Spring Boot application, including the web server.
- `@RestController` marks a class as a web request handler.
- `@GetMapping("/path")` maps a URL path to a method.
- The default port is 8080. Change it with `server.port` in `application.properties`.
- Controllers must be in the same package (or a sub-package) as the main class.

Practice Questions

1. What is Spring Initializr and why should you use it instead of creating a project manually?
2. What does the `@RestController` annotation tell Spring Boot about a class?
3. You create a controller in the package `com.other.controllers`, but your main class is in `com.example.hellospringboot`. The endpoint returns a 404 error. Why?
4. You start your application but it stops immediately after printing "Started." What is the most likely cause?
5. What is the difference between `localhost` and a regular website URL? What does port `8080` mean?

Exercises

Exercise 1: Personalized Greeting

Add a new endpoint `/greet` that returns "Hello! Welcome to [your name]'s Spring Boot application!" Replace `[your name]` with your actual name.

Expected result: Visiting `http://localhost:8080/greet` shows your personalized greeting.

Exercise 2: Multiple Endpoints

Create a new controller called `InfoController.java` with these endpoints:

- `/info/app` -- returns "Hello Spring Boot App v1.0"
- `/info/author` -- returns your name
- `/info/java` -- returns the string "Java 17"

Verify all three endpoints work in the browser.

Exercise 3: Change the Port

Modify `application.properties` to run the application on port 9090 instead of 8080. Start the application and verify you can access your endpoints at `http://localhost:9090/hello`.

What Is Next?

You have a running Spring Boot application. But what are all those files and folders that Spring Initializr created? In the next chapter, we will explore the project structure. You will learn what every file and folder does, and how Maven's `pom.xml` works.

Chapter 4: Understanding the Project Structure

What You Will Learn

- The complete folder structure of a Spring Boot project
- What every file and folder does
- How to read `pom.xml` line by line
- What `src/main/java`, `src/main/resources`, and `src/test` contain
- How `application.properties` configures your application
- What `static` and `templates` folders are for
- What the Maven Wrapper (`mvnw`) is
- How starter dependencies work

Why This Chapter Matters

Imagine you move into a new house. Before you start decorating, you need to know which room is the kitchen, which is the bedroom, and where the electrical panel is. If you plug a washing machine into the wrong outlet, things break.

A Spring Boot project is your house. Each folder has a purpose. Each file has a job. If you put code in the wrong place, things break silently. This chapter gives you a complete tour of the house so you always know where everything belongs.

4.1 The Complete Folder Tree

Here is the full structure of the project you created in Chapter 3:

```
hello-spring-boot/
│
├── mvnw                              Maven Wrapper script (Mac/Linux)
├── mvnw.cmd                          Maven Wrapper script (Windows)
├── pom.xml                           Project configuration file
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── example/
│   │   │           └── hellospringboot/
│   │   │               └── HelloSpringBootApplication.java
│   │   │
│   │   └── resources/
│   │       ├── application.properties    App configuration
│   │       ├── static/                   Static files (CSS, JS, images)
│   │       └── templates/                HTML templates
│   │
│   └── test/
│       └── java/
│           └── com/
│               └── example/
│                   └── hellospringboot/
│                       └── HelloSpringBootApplicationTests.java
│
├── .gitignore                        Files Git should ignore
├── .mvn/
│   └── wrapper/
│       └── maven-wrapper.properties  Maven Wrapper config
│
└── target/                           Build output (created after first build)
```

Let us visit each room in this house.

4.2 The Root Directory

The root directory is the top-level folder. It contains the most important configuration files.

pom.xml -- The Heart of the Project

`pom.xml` stands for Project Object Model. It is the most important file in a Maven project. It tells Maven:

- What your project is called
- What version of Java to use

- What libraries (dependencies) to download
- How to build the project

Think of `pom.xml` as the blueprint of a house. The builder (Maven) reads the blueprint to know what materials to buy and how to assemble them.

Here is the `pom.xml` from your project, explained line by line:

```xml
<?xml version="1.0" encoding="UTF-8"?>                          <!-- 1 -->
<project xmlns="http://maven.apache.org/POM/4.0.0"              <!-- 2 -->
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>                          <!-- 3 -->

    <parent>                                                     <!-- 4 -->
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>                               <!-- 5 -->
    <artifactId>hello-spring-boot</artifactId>                   <!-- 6 -->
    <version>0.0.1-SNAPSHOT</version>                            <!-- 7 -->
    <name>hello-spring-boot</name>                               <!-- 8 -->
    <description>My first Spring Boot app</description>          <!-- 9 -->

    <properties>                                                 <!-- 10 -->
        <java.version>17</java.version>
    </properties>

    <dependencies>                                               <!-- 11 -->
        <dependency>
            <groupId>org.springframework.boot</groupId>          <!-- 12 -->
            <artifactId>spring-boot-starter-web</artifactId>     <!-- 13 -->
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>          <!-- 14 -->
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>                                  <!-- 15 -->
        </dependency>
    </dependencies>

    <build>                                                      <!-- 16 -->
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
```

```
        </plugins>
    </build>

</project>
```

Line-by-line explanation:

- Line 1: XML declaration. Every `pom.xml` starts with this. It says "this is an XML file."
- Line 2: The `<project>` tag. Everything in the `pom.xml` lives inside this tag. The `xmlns` attributes tell the XML parser what format to expect.
- Line 3: `<modelVersion>4.0.0</modelVersion>` -- The version of the POM format. It is always 4.0.0. You never change this.
- Lines 4: `<parent>` -- This is the secret sauce. Your project inherits settings from `spring-boot-starter-parent`. This parent POM defines:
  - Default Java version
  - Default plugin versions
  - Default dependency versions
  - Sensible default settings

Think of it as inheriting a house that already has plumbing and electricity. You just need to add furniture.

- Line 5: `<groupId>com.example</groupId>` -- Your organization identifier. Like a last name for the project.
- Line 6: `<artifactId>hello-spring-boot</artifactId>` -- Your project name. Like a first name.
- Line 7: `<version>0.0.1-SNAPSHOT</version>` -- The current version of your project. SNAPSHOT means "work in progress."
- Line 8-9: `<name>` and `<description>` -- Human-readable metadata. They do not affect how the project builds.
- Line 10: `<properties>` -- Variables you can reuse. Here, `java.version` is set to 17. Maven and Spring Boot use this to configure the Java compiler.
- Line 11: `<dependencies>` -- The libraries your project needs. This is where you list everything your code depends on.
- Lines 12-13: `spring-boot-starter-web` -- The web dependency. Notice there is no `<version>` tag. The version comes from the parent POM. This is one benefit of using the parent.
- Lines 14-15: `spring-boot-starter-test` -- Testing libraries (JUnit, Mockito, etc.). The `<scope>test</scope>` means this library is only used when running tests, not in the final application.

- Line 16: `<build>` -- Build configuration. The `spring-boot-maven-plugin` lets you run the app with `mvn spring-boot:run` and creates executable JAR files.

```
+-------------------------------------------------------+
|         How the Parent POM Works                      |
+-------------------------------------------------------+
|                                                        |
|   spring-boot-starter-parent (v3.4.4)                 |
|     |                                                  |
|     ├── Sets Java version defaults                     |
|     ├── Manages 300+ dependency versions               |
|     ├── Configures Maven plugins                       |
|     └── Defines sensible defaults                      |
|           |                                            |
|           v                                            |
|   Your pom.xml inherits all of the above.              |
|   You only need to list WHAT you need,                 |
|   not WHICH VERSION.                                   |
|                                                        |
+-------------------------------------------------------+
```

4.3 The src/main/java Directory

This is where your Java source code lives.

```
src/main/java/
└── com/
    └── example/
        └── hellospringboot/
            ├── HelloSpringBootApplication.java
            └── HelloController.java
```

The folder path `com/example/hellospringboot` matches the Java package `com.example.hellospringboot`. In Java, every folder is a package. Every class must declare which package it belongs to.

Why packages matter:

- They organize your code (like folders organize files on your computer).
- They prevent naming conflicts (two classes can have the same name if they are in different packages).
- Spring Boot scans the package of the main class and its sub-packages for components.

As your project grows, you will create sub-packages:

```
src/main/java/
└── com/
    └── example/
        └── hellospringboot/
            ├── HelloSpringBootApplication.java
            ├── controller/
            │   ├── HelloController.java
            │   └── UserController.java
            ├── service/
            │   ├── UserService.java
            │   └── EmailService.java
            ├── repository/
            │   └── UserRepository.java
            └── model/
                └── User.java
```

This is a common pattern in Spring Boot projects:

Package	Purpose
`controller`	Classes that handle web requests
`service`	Classes that contain business logic
`repository`	Classes that talk to the database
`model`	Classes that represent data (entities)

We will use this pattern in later chapters.

4.4 The src/main/resources Directory

This directory holds non-Java files that your application needs.

```
src/main/resources/
├── application.properties
├── static/
└── templates/
```

application.properties

This is the main configuration file. It controls how your application behaves. It uses a simple `key=value` format.

```properties
# Server configuration
server.port=8080

# Application name
spring.application.name=hello-spring-boot
```

Each line sets one property. Lines starting with `#` are comments. Spring Boot reads this file at startup and applies the settings.

Here are some common properties:

```properties
# Change the port
server.port=9090

# Set the context path (adds a prefix to all URLs)
server.servlet.context-path=/api

# Enable/disable the Spring banner
spring.main.banner-mode=off

# Set logging level
logging.level.root=INFO
logging.level.com.example=DEBUG
```

> Tip: You can also use `application.yml` instead of `application.properties`. YAML is another format that uses indentation instead of dots. Both work. We use `.properties` in this book because it is simpler.

```
+---------------------------------------------------+
|     application.properties vs application.yml    |
+---------------------------------------------------+
|                                                   |
|  Properties format:                               |
|    server.port=8080                               |
|    spring.application.name=hello                  |
|                                                   |
|  YAML format:                                     |
|    server:                                        |
|      port: 8080                                   |
|    spring:                                        |
|      application:                                 |
|        name: hello                                |
|                                                   |
|  Both are equivalent. Pick one style.             |
|                                                   |
```

```
+--------------------------------------------------+
```

static/ Folder

The `static` folder holds files that are served directly to the browser without processing. These are:

- CSS files -- stylesheets for web pages
- JavaScript files -- client-side scripts
- Images -- logos, icons, photos
- HTML files -- static web pages

If you put a file called `logo.png` in `src/main/resources/static/`, you can access it at `http://localhost:8080/logo.png`.

```
src/main/resources/static/
├── css/
│   └── style.css          --> http://localhost:8080/css/style.css
├── js/
│   └── app.js             --> http://localhost:8080/js/app.js
└── images/
    └── logo.png           --> http://localhost:8080/images/logo.png
```

templates/ Folder

The `templates` folder holds server-side HTML templates. These templates are processed by a template engine (like Thymeleaf) before being sent to the browser.

The difference between `static` and `templates`:

- static: Files are served as-is. No processing.
- templates: Files are processed. You can insert dynamic data (like the current user's name) into the HTML.

We will not use templates in this book. We focus on building REST APIs that return JSON, not HTML pages.

4.5 The src/test Directory

```
src/test/
└── java/
    └── com/
        └── example/
            └── hellospringboot/
                └── HelloSpringBootApplicationTests.java
```

This directory mirrors the `src/main/java` structure. It holds your test classes. The generated test class looks like this:

```java
package com.example.hellospringboot;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest                                          // 1
class HelloSpringBootApplicationTests {

    @Test                                                 // 2
    void contextLoads() {                                 // 3
    }
}
```

- Line 1: `@SpringBootTest` -- This annotation tells JUnit to start the full Spring Boot application before running the tests. It tests that the application starts without errors.
- Line 2: `@Test` -- This marks a method as a test case. JUnit will run this method when you run tests.
- Line 3: `contextLoads()` -- This test has an empty body. It passes if the application context loads without throwing an exception. It is a smoke test that verifies your configuration is correct.

Run the tests with:

```bash
./mvnw test
```

Expected output:

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

4.6 The Maven Wrapper (mvnw)

You will notice two files in the root: `mvnw` and `mvnw.cmd`.

```
hello-spring-boot/
├── mvnw          (Mac/Linux)
├── mvnw.cmd      (Windows)
└── .mvn/
    └── wrapper/
        └── maven-wrapper.properties
```

What is the Maven Wrapper?

The Maven Wrapper is a small script that downloads and uses a specific version of Maven. This way, every developer on a team uses the same Maven version, even if they have different versions installed globally.

Think of it as a portable toolbox. Instead of saying "make sure you have wrench size 12," the Wrapper includes its own wrench.

Why does it matter?

- Consistency: Everyone uses the same Maven version.
- No installation required: A new developer can clone the project and build it immediately, without installing Maven first.
- Version control: The Maven version is defined in `maven-wrapper.properties`.

How to use it:

Instead of `mvn`, use `./mvnw` (Mac/Linux) or `mvnw.cmd` (Windows):

```bash
# Instead of:
mvn spring-boot:run

# Use:
./mvnw spring-boot:run
```

```
+-----------------------------------------------------+
|     mvn vs ./mvnw                                   |
+-----------------------------------------------------+
|                                                     |
|   mvn                                               |
|     Uses the globally installed Maven.              |
|     Version depends on what you installed.          |
|                                                     |
|   ./mvnw                                            |
|     Uses the Maven version defined in the           |
|     project. Downloads it if needed.                |
|     Guarantees the same version for everyone.       |
```

```
|                                                    |
+----------------------------------------------------+
```

4.7 The target/ Directory

The `target` folder does not exist when you first create the project. Maven creates it when you build the project.

```bash
./mvnw package
```

After building:

```
target/
├── classes/                      Compiled .class files
│   └── com/
│       └── example/
│           └── hellospringboot/
│               ├── HelloSpringBootApplication.class
│               └── HelloController.class
├── test-classes/                 Compiled test .class files
├── hello-spring-boot-0.0.1-SNAPSHOT.jar   The executable JAR
└── ...                           Other build artifacts
```

Important: Never edit files in `target/`. They are generated. Maven deletes and recreates this folder every time you build. Any changes you make will be lost.

To clean the `target` folder:

```bash
./mvnw clean
```

4.8 The .gitignore File

Git is a version control system. It tracks changes to your code. The `.gitignore` file tells Git which files to not track.

The generated `.gitignore` typically contains:

```
HELP.md
target/
!.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/

### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr

### VS Code ###
.vscode/
```

Why ignore these files?

- `target/` -- Generated files. Every developer can recreate them by running `mvn package`.
- `.idea/`, `.vscode/` -- IDE-specific settings. Each developer's IDE settings are personal.
- `*.iml` -- IntelliJ module files. Generated by the IDE.

Rule of thumb: Track source code and configuration. Ignore generated files and IDE settings.

4.9 Understanding Starter Dependencies

In the `pom.xml`, you added `spring-boot-starter-web`. But what is a "starter"?

A starter is a bundle of related libraries. Instead of listing 10 individual libraries, you add one starter that includes all of them.

Think of it as ordering a combo meal at a restaurant. Instead of ordering a burger, fries, a drink, and a napkin separately, you say "Combo #1" and get everything.

```
+-------------------------------------------------------+
|       What spring-boot-starter-web Includes           |
+-------------------------------------------------------+
|                                                       |
|   spring-boot-starter-web                             |
|     |                                                 |
|     ├── spring-web                                    |
|     |     (core web support)                          |
|     |                                                 |
|     ├── spring-webmvc                                 |
|     |     (@Controller, @GetMapping, etc.)            |
|     |                                                 |
```

```
|     ├── spring-boot-starter-tomcat                    |
|     |     (embedded Tomcat web server)                |
|     |                                                 |
|     ├── spring-boot-starter-json                      |
|     |     (Jackson for JSON processing)               |
|     |                                                 |
|     └── spring-boot-starter                           |
|           (core Spring Boot + logging + YAML)         |
|                                                       |
+-------------------------------------------------------+
```

Here are some common starters:

Starter	What It Provides
`spring-boot-starter-web`	Web application + embedded Tomcat
`spring-boot-starter-data-jpa`	Database access with JPA/Hibernate
`spring-boot-starter-test`	JUnit, Mockito, and testing utilities
`spring-boot-starter-security`	Authentication and authorization
`spring-boot-starter-validation`	Bean validation (@NotNull, @Size, etc.)
`spring-boot-starter-actuator`	Health checks and monitoring endpoints

You never need to memorize versions. The parent POM manages them all. Just add the starter. Maven handles the rest.

Common Mistakes

1. Editing files in the `target/` directory. These files are generated. Maven deletes them on every clean build. Always edit files in `src/`.
2. Putting Java classes outside the base package. If your main class is in `com.example.hellospringboot`, all other classes must be in the same package or a sub-package.
3. Putting configuration files in `src/main/java`. Configuration files like `application.properties` go in `src/main/resources`. Java files go in `src/main/java`.
4. Adding version numbers to starter dependencies. The parent POM manages versions. Adding your own version can cause conflicts.

5. Committing the `target/` folder to Git. Always add `target/` to `.gitignore`. It wastes space and causes merge conflicts.

Best Practices

1. Follow the standard directory layout. Put Java code in `src/main/java`, resources in `src/main/resources`, tests in `src/test/java`. Maven expects this structure.
2. Organize code into sub-packages. Use `controller`, `service`, `repository`, and `model` packages to separate concerns.
3. Use `application.properties` for configuration. Do not hard-code values like port numbers or database URLs in Java code.
4. Use the Maven Wrapper (`mvnw`). It ensures consistent builds across different machines.
5. Keep `pom.xml` clean. Only add dependencies you actually use. Remove unused ones regularly.

Quick Summary

A Spring Boot project has a well-defined structure:

- `pom.xml` is the project blueprint. It lists dependencies, Java version, and build settings.
- `src/main/java` holds your Java source code, organized in packages.
- `src/main/resources` holds configuration files (`application.properties`), static files, and templates.
- `src/test` holds test classes that mirror the main source structure.
- `mvnw` is the Maven Wrapper. It ensures everyone uses the same Maven version.
- `target/` is the build output directory. Never edit files here.
- Starters are bundles of related libraries. They simplify dependency management.

Key Points

- `pom.xml` is the most important file. It controls what Maven downloads and how it builds the project.

- The `<parent>` section inherits settings from `spring-boot-starter-parent`, including dependency versions.
- `src/main/java` is for Java code. `src/main/resources` is for configuration and static files.
- `application.properties` controls application behavior (port, logging, database, etc.).
- The `static/` folder serves files directly. The `templates/` folder uses a template engine.
- Starters are dependency bundles. `spring-boot-starter-web` includes Tomcat, Spring MVC, and Jackson.
- The Maven Wrapper (`mvnw`) guarantees consistent builds without requiring Maven installation.
- Never commit `target/` to version control.

Practice Questions

1. What is the purpose of `pom.xml`? What happens if you delete it?
2. What is the difference between `src/main/resources/static` and `src/main/resources/templates`?
3. Why does the `spring-boot-starter-web` dependency in `pom.xml` not have a `<version>` tag?
4. You put a file called `style.css` in `src/main/resources/static/css/`. What URL would you use to access it in the browser?
5. What is the Maven Wrapper and why should you use `./mvnw` instead of `mvn`?

Exercises

Exercise 1: Explore Your Project

Open your project in a file explorer (not the IDE). Navigate through every folder. For each folder and file, write down its purpose in one sentence. Compare your notes with the descriptions in this chapter.

Exercise 2: Add a Static File

Create a file called `src/main/resources/static/hello.html` with this content:

```html
<!DOCTYPE html>
<html>
<head><title>Hello</title></head>
<body><h1>Hello from a static file!</h1></body>
</html>
```

Start the application and visit `http://localhost:8080/hello.html`. You should see the HTML page. Notice that no controller was needed. Static files are served automatically.

Exercise 3: Read the Dependency Tree

Run this command in your project folder:

```bash
./mvnw dependency:tree
```

This shows all libraries your project uses, including ones pulled in by the starters. Look at the output. Count how many libraries `spring-boot-starter-web` brings in. Write down three libraries you recognize.

What Is Next?

Now you know where everything lives in a Spring Boot project. But how does Spring Boot know which libraries to configure automatically? How does it know to start Tomcat without you writing any server code? In the next chapter, we will uncover the magic of auto-configuration.

Chapter 5: Auto-Configuration -- The Magic Behind Spring Boot

What You Will Learn

- What auto-configuration is and why Spring Boot uses it
- How `@SpringBootApplication` breaks down into three annotations
- What `@EnableAutoConfiguration` does behind the scenes
- How `@ConditionalOnClass` works (configure only if a library is present)
- How to use the `--debug` flag to see what Spring Boot configured
- How to override auto-configuration with your own properties

Why This Chapter Matters

In the last chapter, you saw that adding `spring-boot-starter-web` to your `pom.xml` gave you a running web server. You did not write any code to start Tomcat. You did not configure a port. You did not wire up a JSON converter. It all just worked.

How?

The answer is auto-configuration. This is the core idea that makes Spring Boot different from plain Spring. Understanding it transforms Spring Boot from "magic" into "engineering." When something goes wrong, and something always goes wrong, you will know where to look and what to adjust.

5.1 The Restaurant Kitchen Analogy

Imagine you walk into a restaurant kitchen. Two types of kitchens exist:

Kitchen A (Plain Spring): You walk in and the room is empty. There are counters, but no equipment. You need to:

1. Buy an oven and install it
2. Buy a refrigerator and plug it in
3. Buy pots, pans, and knives
4. Hire a chef
5. Write a menu

Only then can you cook.

Kitchen B (Spring Boot): You walk in and the kitchen is fully equipped. There is an oven, a refrigerator, pots, pans, and a chef. A default menu is pinned to the wall. You can start cooking immediately.

But here is the clever part: the kitchen adapts. If you bring in a pizza oven, the kitchen notices and adds pizza-making tools automatically. If you bring in a sushi bar, it adds rice cookers and soy sauce. The kitchen configures itself based on what it detects.

That is auto-configuration.

```
+-----------------------------------------------------------+
|        The Restaurant Kitchen Analogy                     |
+-----------------------------------------------------------+
|                                                           |
|   Plain Spring (Kitchen A):                               |
|     Empty kitchen. You install everything manually.       |
|     Full control, but lots of setup work.                 |
|                                                           |
|   Spring Boot (Kitchen B):                                |
|     Fully equipped kitchen. Pre-configured for you.       |
|     Detects what you bring in and adapts.                 |
|     You can still swap any piece of equipment.            |
|                                                           |
|   Key insight:                                            |
|     Auto-config gives you sensible defaults.              |
|     You can always override them.                         |
|                                                           |
+-----------------------------------------------------------+
```

5.2 Breaking Down @SpringBootApplication

In Chapter 3, you saw this annotation on your main class:

```java
@SpringBootApplication
public class HelloSpringBootApplication {
    // ...
}
```

`@SpringBootApplication` looks like one annotation, but it is actually three annotations combined. It is a shortcut. Here is what it expands to:

```java
@SpringBootConfiguration       // 1
@EnableAutoConfiguration       // 2
@ComponentScan                 // 3
public class HelloSpringBootApplication {
    // ...
}
```

Let us examine each one.

@SpringBootConfiguration

This annotation tells Spring: "This class contains configuration." It is a specialized version of `@Configuration`, which is a core Spring annotation.

Think of it as a sign on a door that says "Manager's Office." It tells the Spring framework: "Look in here for instructions on how to set up the application."

In practice, you rarely think about this annotation. It is there so Spring knows your main class is a configuration source.

@EnableAutoConfiguration

This is the star of this chapter. This annotation tells Spring Boot: "Look at the libraries on the classpath. For each library you recognize, configure it automatically."

The classpath is the list of all libraries your application can use. When you add `spring-boot-starter-web` to `pom.xml`, Maven downloads Tomcat, Jackson, and Spring MVC and puts them on the classpath.

`@EnableAutoConfiguration` sees Tomcat on the classpath and thinks: "I see a web server library. Let me configure a web server." It sees Jackson and thinks: "I see a JSON library. Let me set up JSON conversion."

```
+-----------------------------------------------------------+
|     How @EnableAutoConfiguration Thinks                   |
+-----------------------------------------------------------+
```

```
|                                                            |
|   Step 1: Look at the classpath                            |
|           (What libraries are available?)                  |
|                                                            |
|   Step 2: For each library, check if there is an         |
|           auto-configuration class                         |
|                                                            |
|   Step 3: If conditions are met, apply the configuration  |
|           (e.g., "If Tomcat is present, start Tomcat")    |
|                                                            |
|   Step 4: Use sensible defaults                            |
|           (e.g., port 8080, UTF-8 encoding)              |
|                                                            |
|   Step 5: Let the developer override any default          |
|           (via application.properties)                     |
|                                                            |
+------------------------------------------------------------+
```

@ComponentScan

This annotation tells Spring: "Scan this package and all sub-packages for classes with Spring annotations." When Spring finds a class with `@RestController`, `@Service`, `@Repository`, or `@Component`, it creates an instance and manages it.

This is why your `HelloController` was found automatically. It was in the same package as the main class. `@ComponentScan` found it.

```
+------------------------------------------------------------+
|     @SpringBootApplication = 3 Annotations               |
+------------------------------------------------------------+
|                                                            |
|   @SpringBootConfiguration                                 |
|     "This class is a configuration source."                |
|                                                            |
|   @EnableAutoConfiguration                                 |
|     "Configure libraries automatically based on            |
|      what is on the classpath."                            |
|                                                            |
|   @ComponentScan                                           |
|     "Scan this package for Spring components               |
|      (@Controller, @Service, etc.)."                       |
|                                                            |
+------------------------------------------------------------+
```

5.3 How Auto-Configuration Works Internally

Spring Boot ships with over 150 auto-configuration classes. Each one configures a specific library or feature. Here are a few examples:

Auto-Configuration Class	What It Configures
`DataSourceAutoConfiguration`	Database connection pool
`JacksonAutoConfiguration`	JSON serialization (Jackson)
`ServletWebServerFactoryAutoConfiguration`	Embedded web server (Tomcat)
`HttpEncodingAutoConfiguration`	UTF-8 character encoding
`ErrorMvcAutoConfiguration`	Default error pages

Each class follows the same pattern:

1. Check conditions. Is a specific library on the classpath? Is a property set?
2. If conditions are met, create and configure beans (objects managed by Spring).
3. If conditions are not met, do nothing.

The key mechanism is conditional annotations.

5.4 Understanding @ConditionalOnClass

`@ConditionalOnClass` is the most important conditional annotation. It says: "Only apply this configuration if a specific class exists on the classpath."

Here is a simplified example of how Tomcat auto-configuration works:

```java
@Configuration                                          // 1
@ConditionalOnClass(Tomcat.class)                       // 2
public class TomcatAutoConfiguration {

    @Bean                                                // 3
    public TomcatServletWebServerFactory tomcatFactory() {
        TomcatServletWebServerFactory factory =
            new TomcatServletWebServerFactory();         // 4
        factory.setPort(8080);                           // 5
        return factory;
    }
}
```

Line-by-line explanation:

- Line 1: `@Configuration` -- This class provides Spring configuration.
- Line 2: `@ConditionalOnClass(Tomcat.class)` -- "Only use this class if `Tomcat.class` is on the classpath." If you did not add `spring-boot-starter-web` (which includes Tomcat), this entire class is skipped.
- Line 3: `@Bean` -- This method creates an object that Spring will manage. The object is called a "bean."
- Line 4: Creates a new Tomcat server factory.
- Line 5: Sets the default port to 8080.

The beauty of this design:

- If Tomcat is on the classpath: Spring Boot creates a Tomcat server on port 8080. You did not write any server code.
- If Tomcat is not on the classpath: Nothing happens. No error. No unnecessary server.

```
+-------------------------------------------------------------+
|     @ConditionalOnClass in Action                           |
+-------------------------------------------------------------+
|                                                             |
|   You add spring-boot-starter-web to pom.xml                |
|        |                                                    |
|        v                                                    |
|   Maven downloads Tomcat JAR                                |
|        |                                                    |
|        v                                                    |
|   Tomcat.class is now on the classpath                      |
|        |                                                    |
|        v                                                    |
|   @ConditionalOnClass(Tomcat.class) --> TRUE                |
|        |                                                    |
|        v                                                    |
|   TomcatAutoConfiguration activates                         |
|        |                                                    |
|        v                                                    |
|   Tomcat starts on port 8080                                |
|                                                             |
+-------------------------------------------------------------+
```

There are other conditional annotations too:

Annotation	Condition
`@ConditionalOnClass`	A specific class exists on the classpath
`@ConditionalOnMissingClass`	A specific class is NOT on the classpath
`@ConditionalOnBean`	A specific bean already exists in the context

`@ConditionalOnMissingBean`	A specific bean does NOT exist (lets you replace defaults)
`@ConditionalOnProperty`	A specific property is set in application.properties

`@ConditionalOnMissingBean` is especially important. It means: "Use this default unless the developer provides their own." This is how you override auto-configuration.

5.5 Using the --debug Flag

Want to see what Spring Boot auto-configured and what it skipped? Use the `--debug` flag.

```bash
./mvnw spring-boot:run -Dspring-boot.run.arguments=--debug
```

Or add this to `application.properties`:

```properties
debug=true
```

This produces a detailed report in the console with two sections:

Positive Matches (What Was Configured)

```
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------

   ServletWebServerFactoryAutoConfiguration matched:
      - @ConditionalOnClass found required class
        'jakarta.servlet.ServletRequest' (OnClassCondition)

   JacksonAutoConfiguration matched:
      - @ConditionalOnClass found required class
        'com.fasterxml.jackson.databind.ObjectMapper' (OnClassCondition)
```

This tells you: "I configured a web server because I found `ServletRequest` on the classpath. I configured Jackson because I found `ObjectMapper`."

Negative Matches (What Was Skipped)

```
Negative matches:
-----------------

   DataSourceAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class
           'javax.sql.DataSource' (OnClassCondition)

   MongoAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class
           'com.mongodb.client.MongoClient' (OnClassCondition)
```

This tells you: "I did not configure a database because you did not add a database library. I did not configure MongoDB because the MongoDB library is not present."

```
+-----------------------------------------------------------+
|     Reading the Auto-Configuration Report                 |
+-----------------------------------------------------------+
|                                                            |
|   Positive matches:                                        |
|     "I configured this because conditions were met."       |
|     Example: Tomcat configured because Tomcat              |
|     JAR is on the classpath.                               |
|                                                            |
|   Negative matches:                                        |
|     "I skipped this because conditions were NOT met."      |
|     Example: MongoDB skipped because MongoDB               |
|     library is not on the classpath.                       |
|                                                            |
|   Use this report to debug:                                |
|     "Why is X not working?"                                |
|     Check the negative matches for X.                      |
|                                                            |
+-----------------------------------------------------------+
```

5.6 Overriding Auto-Configuration with Properties

Auto-configuration provides sensible defaults. But defaults are not always what you want. You can override them in `application.properties`.

Example 1: Change the Server Port

Default: Tomcat starts on port 8080.

Override: Add this to `application.properties`:

```properties
server.port=9090
```

Result: Tomcat now starts on port 9090.

Example 2: Change the Context Path

Default: Your endpoints are at the root (e.g., `/hello`).

Override:

```properties
server.servlet.context-path=/api
```

Result: Your endpoints are now at `/api/hello`.

Example 3: Change the JSON Date Format

Default: Jackson formats dates as timestamps (numbers).

Override:

```properties
spring.jackson.date-format=yyyy-MM-dd HH:mm:ss
spring.jackson.time-zone=UTC
```

Result: Dates in JSON responses appear as "2024-01-15 10:30:00" instead of a number.

Example 4: Change the Logging Level

Default: Logging level is `INFO`.

Override:

```properties
logging.level.root=WARN
logging.level.com.example.hellospringboot=DEBUG
```

Result: The root logger only shows warnings and errors. Your package shows everything including debug messages.

How Overriding Works

The process follows a simple priority system:

```
+-----------------------------------------------------------+
|     Configuration Priority (highest to lowest)            |
+-----------------------------------------------------------+
|                                                            |
|   1. Command-line arguments                                |
|      (java -jar app.jar --server.port=9090)               |
|                                                            |
|   2. application.properties / application.yml              |
|      (your custom settings)                                |
|                                                            |
|   3. Auto-configuration defaults                           |
|      (Spring Boot's built-in defaults)                     |
|                                                            |
|   Higher priority wins.                                    |
|   Your properties always beat the defaults.                |
|                                                            |
+-----------------------------------------------------------+
```

Your settings always win over auto-configuration defaults. Command-line arguments win over everything.

5.7 Disabling Specific Auto-Configurations

Sometimes you want to turn off an auto-configuration entirely. You can do this with the `exclude` parameter:

```java
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class
})
public class HelloSpringBootApplication {
    // ...
}
```

This tells Spring Boot: "Do not auto-configure a database, even if database libraries are on the classpath."

You can also do it in `application.properties`:

```properties
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAuto...
```

When would you use this?

- You added a library for a different purpose, but Spring Boot tries to configure it as something else.
- You want full manual control over a specific feature.
- A library on the classpath triggers unwanted configuration.

5.8 A Complete Example: Auto-Configuration in Action

Let us trace what happens when you start your `hello-spring-boot` application:

```
+-------------------------------------------------------------+
|     Startup: What Auto-Configuration Does                   |
+-------------------------------------------------------------+
|                                                             |
|   1. Spring Boot starts                                     |
|      (SpringApplication.run is called)                      |
|                                                             |
|   2. @ComponentScan runs                                    |
|      Finds: HelloController                                 |
|                                                             |
|   3. @EnableAutoConfiguration runs                          |
|      Checks 150+ auto-configuration classes                 |
|                                                             |
|   4. Checks: Is Tomcat on the classpath?                    |
|      YES (from spring-boot-starter-web)                     |
|      --> Configures embedded Tomcat on port 8080            |
|                                                             |
|   5. Checks: Is Jackson on the classpath?                   |
|      YES (from spring-boot-starter-web)                     |
|      --> Configures JSON serialization                      |
|                                                             |
|   6. Checks: Is a DataSource on the classpath?              |
|      NO (we did not add a database starter)                 |
|      --> Skips database configuration                       |
|                                                             |
|   7. Checks: Is Spring Security on the classpath?           |
|      NO (we did not add the security starter)               |
|      --> Skips security configuration                       |
|                                                             |
|   8. Reads application.properties                           |
|      Applies any overrides                                  |
```

```
|                                                          |
|   9. Starts Tomcat                                       |
|      Application is ready on localhost:8080              |
|                                                          |
+----------------------------------------------------------+
```

Everything that happened was driven by what is on the classpath. Add a library, get automatic configuration. Remove it, and the configuration disappears. No XML files. No manual wiring. Just add a dependency and go.

Common Mistakes

1. Thinking auto-configuration is "magic." It is not magic. It is a set of `if` statements. "If this library is present, create this bean." Understanding the conditions helps you debug problems.
2. Overriding properties with typos. `server.port=9090` works. `server.Port=9090` does not. Properties are case-sensitive.
3. Not using the --debug flag when things go wrong. The auto-configuration report tells you exactly what was configured and what was skipped. Use it.
4. Adding libraries without understanding what they auto-configure. Every starter triggers auto-configuration. Know what you are adding.
5. Excluding auto-configurations unnecessarily. Only exclude when there is a real conflict. Most of the time, the defaults work well.

Best Practices

1. Trust the defaults first. Spring Boot's defaults are battle-tested. Override them only when you have a specific reason.
2. Use `application.properties` for overrides. Do not create custom configuration classes when a property will do the job.
3. Run with `--debug` when troubleshooting. The conditions evaluation report is your best debugging tool for auto-configuration issues.
4. Read the Spring Boot documentation. Each starter has a list of supported properties. The docs at `docs.spring.io` list them all.

5. Add starters one at a time. When something breaks, you know which starter caused the problem.

Quick Summary

Auto-configuration is how Spring Boot eliminates boilerplate setup code. It follows three steps:

1. Detect what libraries are on the classpath.
2. Configure beans automatically if conditions are met (using `@ConditionalOnClass`, `@ConditionalOnMissingBean`, etc.).
3. Allow overrides through `application.properties` or custom beans.

`@SpringBootApplication` combines three annotations: `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan`. Together, they start the auto-configuration engine and scan for your components.

Key Points

- `@SpringBootApplication` = `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`.
- Auto-configuration detects libraries on the classpath and configures them with sensible defaults.
- `@ConditionalOnClass` activates configuration only if a specific class is present.
- `@ConditionalOnMissingBean` lets you replace default beans with your own.
- The `--debug` flag (or `debug=true` in `application.properties`) produces an auto-configuration report.
- Override defaults in `application.properties` (e.g., `server.port=9090`).
- Use `exclude` to disable specific auto-configurations you do not want.
- Configuration priority: command-line arguments > application.properties > auto-configuration defaults.

Practice Questions

1. What three annotations does `@SpringBootApplication` combine? What does each one do?
2. Explain in your own words how `@ConditionalOnClass` works. Give an example.
3. You add `spring-boot-starter-data-jpa` to your `pom.xml` but do not configure a database. The application fails to start with a "DataSource" error. Why does this happen?
4. How can you see which auto-configurations were applied and which were skipped when your application starts?
5. You want Tomcat to run on port 3000 instead of 8080. Where do you make this change and what property do you set?

Exercises

Exercise 1: Read the Auto-Configuration Report

Add `debug=true` to your `application.properties`. Start the application. Find the "CONDITIONS EVALUATION REPORT" in the console output. Answer these questions:

- How many positive matches are there?
- Is `DataSourceAutoConfiguration` in the positive or negative matches? Why?
- Find `JacksonAutoConfiguration`. What class was it looking for on the classpath?

Exercise 2: Override a Default

Change the server port to 3000 using `application.properties`. Start the application and verify it works at `http://localhost:3000/hello`. Then change the port using a command-line argument instead:

```bash
./mvnw spring-boot:run -Dspring-boot.run.arguments=--server.port=4000
```

Verify the application now runs on port 4000. Which one takes priority: the command-line argument or the properties file?

Exercise 3: Exclude an Auto-Configuration

Add the H2 database dependency to your `pom.xml`:

```xml
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Start the application with `debug=true`. Check the report. Is `DataSourceAutoConfiguration` now a positive match?

Next, exclude it using `@SpringBootApplication(exclude = ...)`. Restart. Verify it moved to the exclusions section of the report.

What Is Next?

You now understand how Spring Boot configures itself. But auto-configuration creates beans -- objects that Spring manages. How does Spring know which bean to give to which class? How do your controllers get the services they need?

In the next chapter, we will explore Dependency Injection, the mechanism that wires your application together.

Chapter 6: Dependency Injection -- Wiring Your Application Together

What You Will Learn

- What a dependency is in programming
- Why manually creating objects causes problems (the old way)
- How Spring creates and manages objects for you (the Spring way)
- What the IoC container is (and a hiring agency analogy)
- How constructor injection works
- What `@Autowired` does and when you need it
- Why dependency injection matters for testing and maintainability
- A real example with `OrderService` and `EmailService`

Why This Chapter Matters

Every real application has multiple parts that work together. A controller calls a service. A service calls a repository. A repository talks to a database.

These parts depend on each other. How you connect them determines whether your code is easy to change, easy to test, and easy to understand -- or a tangled mess.

Dependency injection is the single most important concept in Spring. Every Spring application uses it. Every Spring annotation relates to it in some way. If you understand this chapter, the rest of the framework clicks into place.

6.1 What Is a Dependency?

In programming, a dependency is an object that another object needs to do its job.

Think of a chef in a restaurant. The chef depends on a knife to chop vegetables. The knife is a dependency of the chef. Without it, the chef cannot do their work.

In code, if class A uses class B, then B is a dependency of A.

```java
public class OrderService {
    private EmailService emailService;  // EmailService is a dependency
}
```

`OrderService` needs `EmailService` to send order confirmations. Without it, `OrderService` cannot complete its job.

```
+----------------------------------------------------+
|           What Is a Dependency?                    |
+----------------------------------------------------+
|                                                     |
|   OrderService depends on EmailService              |
|                                                     |
|   OrderService ----needs----> EmailService          |
|                                                     |
|   "I need you to send emails for me."               |
|                                                     |
|   The chef      ----needs----> the knife            |
|   "I need you to chop vegetables for me."           |
|                                                     |
+----------------------------------------------------+
```

6.2 The Old Way: Creating Objects Yourself

Before Spring, developers created dependencies manually. Here is what that looked like:

```java
public class OrderService {

    private EmailService emailService;

    public OrderService() {
        this.emailService = new EmailService();     // 1
    }

    public void placeOrder(String item) {
        System.out.println("Order placed: " + item);
```

```
        emailService.sendConfirmation(item);        // 2
    }
}
```

- Line 1: `OrderService` creates its own `EmailService` using `new`. This is the problem.
- Line 2: `OrderService` uses the `EmailService` to send a confirmation email.

This looks simple. What is wrong with it?

Problem 1: Tight Coupling

`OrderService` is tightly coupled to `EmailService`. The word "coupled" means "connected." Tight coupling means the connection is rigid and hard to change.

If you want to replace `EmailService` with `SmsService`, you must change the code inside `OrderService`. In a big application, `EmailService` might be used in 50 classes. You would need to change all 50.

Problem 2: Hard to Test

When you write tests for `OrderService`, you do not want to send real emails. You want a fake `EmailService` that pretends to send emails. But `OrderService` creates its own `EmailService` with `new`. You cannot swap it for a fake one.

Problem 3: Hidden Dependencies

Looking at `OrderService` from the outside, you cannot see what it needs. The dependency is hidden inside the constructor. If `OrderService` needs five other objects, they are all buried in the code.

```
+-------------------------------------------------------------+
|     Problems with "new" (Creating Objects Yourself)      |
+-------------------------------------------------------------+
|                                                              |
|   Problem 1: Tight Coupling                                  |
|     OrderService is glued to EmailService.                    |
|     You cannot swap it without changing the code.             |
|                                                              |
|   Problem 2: Hard to Test                                    |
|     You cannot replace EmailService with a fake               |
|     version during testing.                                   |
|                                                              |
|   Problem 3: Hidden Dependencies                             |
|     You cannot see what OrderService needs                    |
|     without reading its internal code.                        |
|                                                              |
```

```
+-------------------------------------------------------------+
```

6.3 The Spring Way: Let Someone Else Create Objects

Spring takes a different approach. Instead of each class creating its own dependencies, Spring creates all the objects and injects them where they are needed.

Here is the same example, the Spring way:

```java
@Service                                                    // 1
public class EmailService {

    public void sendConfirmation(String item) {
        System.out.println("Email sent for: " + item);
    }
}
```

```java
@Service                                                    // 2
public class OrderService {

    private final EmailService emailService;                // 3

    public OrderService(EmailService emailService) {        // 4
        this.emailService = emailService;
    }

    public void placeOrder(String item) {
        System.out.println("Order placed: " + item);
        emailService.sendConfirmation(item);
    }
}
```

Line-by-line explanation:

- Line 1: `@Service` on `EmailService` tells Spring: "Create an instance of this class and manage it." Spring creates one `EmailService` object and keeps it ready.
- Line 2: `@Service` on `OrderService` tells Spring the same thing.
- Line 3: `private final EmailService emailService;` -- The dependency is declared as a field. The `final` keyword means it must be set once and cannot change.

- Line 4: The constructor takes `EmailService` as a parameter. Spring sees this and says: "OrderService needs an EmailService. I already created one. Let me pass it in."

Notice what is different:

- No `new EmailService()` anywhere.
- `OrderService` does not create its own dependency.
- Spring creates both objects and connects them.

This is dependency injection. The dependency (`EmailService`) is injected into `OrderService` through the constructor.

6.4 The IoC Container: The Hiring Agency Analogy

Spring uses something called an IoC Container. IoC stands for Inversion of Control.

What does "Inversion of Control" mean?

Normally, your code controls everything. It creates objects, connects them, and manages their lifecycle. With IoC, this control is inverted (flipped). The framework (Spring) takes control. Your code just declares what it needs.

Think of it like hiring employees at a company:

Without IoC (You do everything yourself):

1. You write a job posting
2. You review resumes
3. You interview candidates
4. You hire the person
5. You train them
6. You assign them to a team

With IoC (Hiring agency does it for you):

1. You tell the agency: "I need a Java developer"
2. The agency finds one, hires them, and sends them to your office

The hiring agency is the IoC container. You declare what you need. The agency provides it.

```
+-----------------------------------------------------------+
|     The Hiring Agency Analogy                             |
+-----------------------------------------------------------+
```

```
|                                                            |
|   Spring IoC Container = Hiring Agency                     |
|                                                            |
|   You (OrderService):                                      |
|     "I need an EmailService employee."                     |
|                                                            |
|   Hiring Agency (Spring):                                  |
|     "I have one. Here you go."                             |
|     [passes EmailService to OrderService]                  |
|                                                            |
|   You (OrderController):                                   |
|     "I need an OrderService employee."                     |
|                                                            |
|   Hiring Agency (Spring):                                  |
|     "I have one (with EmailService already assigned).      |
|      Here you go."                                         |
|     [passes OrderService to OrderController]               |
|                                                            |
+------------------------------------------------------------+
```

The IoC container holds all the objects (called beans) and knows how to wire them together. When a bean needs another bean, the container provides it.

```
+------------------------------------------------------------+
|     Inside the IoC Container                               |
+------------------------------------------------------------+
|                                                            |
|   +------------------+                                     |
|   |  IoC Container   |                                     |
|   |                  |                                     |
|   |  Beans:          |                                     |
|   |   - EmailService |-----+                               |
|   |   - OrderService |--+  |                               |
|   |   - OrderController |  |                               |
|   |                  |  |  |                               |
|   +------------------+  |  |                               |
|                         |  |                               |
|   OrderService needs    |  |                               |
|   EmailService -------->+--+                               |
|                                                            |
|   OrderController needs                                    |
|   OrderService -------->+                                  |
|                                                            |
+------------------------------------------------------------+
```

6.5 Constructor Injection (The Recommended Way)

There are three ways to inject dependencies in Spring:

1. Constructor injection (recommended)
2. Field injection
3. Setter injection

We will focus on constructor injection because it is the best practice recommended by the Spring team.

How Constructor Injection Works

```java
@Service
public class OrderService {

    private final EmailService emailService;
    private final PaymentService paymentService;

    // Constructor injection: Spring passes both dependencies
    public OrderService(EmailService emailService,
                        PaymentService paymentService) {
        this.emailService = emailService;
        this.paymentService = paymentService;
    }

    public void placeOrder(String item, double amount) {
        paymentService.charge(amount);
        System.out.println("Order placed: " + item);
        emailService.sendConfirmation(item);
    }
}
```

Spring sees the constructor and says: "This class needs `EmailService` and `PaymentService`. I have both in my container. Let me pass them in."

Why Constructor Injection Is Best

Benefit	Explanation
Immutability	Fields are `final`. Once set, they cannot change.
Required dependencies	If a dependency is missing, the app fails at startup, not at runtime.
Easy to test	In tests, just call `new OrderService(mockEmail, mockPayment)`.
Clear dependencies	The constructor lists everything the class needs.

```
+-----------------------------------------------------------+
|     Why Constructor Injection Wins                        |
+-----------------------------------------------------------+
|                                                           |
|   Constructor injection:                                  |
|     - All dependencies are visible in the constructor.    |
|     - Fields are final (immutable).                       |
|     - Missing dependency? App won't start. Fail fast.     |
|     - Easy to test: just pass mocks to the constructor.   |
|                                                           |
|   Field injection (@Autowired on fields):                 |
|     - Dependencies are hidden.                            |
|     - Fields are not final (can be null).                 |
|     - Missing dependency? Null pointer at runtime.        |
|     - Hard to test without Spring's help.                 |
|                                                           |
+-----------------------------------------------------------+
```

6.6 Understanding @Autowired

You might see `@Autowired` in tutorials and older code:

```java
@Service
public class OrderService {

    @Autowired                              // Field injection
    private EmailService emailService;
}
```

`@Autowired` tells Spring: "Inject a bean here." It can be used on fields, constructors, and setter methods.

Important rule: If a class has only one constructor, Spring automatically injects dependencies through it. You do not need `@Autowired`.

```java
@Service
public class OrderService {

    private final EmailService emailService;

    // @Autowired is optional here (only one constructor)
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
```

```
    }
}
```

This is why most modern Spring Boot code does not use `@Autowired` at all. One constructor + Spring = automatic injection.

When do you need `@Autowired`?

Only when a class has multiple constructors and you want to tell Spring which one to use:

```java
@Service
public class OrderService {

    private final EmailService emailService;
    private final int maxRetries;

    @Autowired                              // Tell Spring: use this constructor
    public OrderService(EmailService emailService) {
        this(emailService, 3);
    }

    public OrderService(EmailService emailService, int maxRetries) {
        this.emailService = emailService;
        this.maxRetries = maxRetries;
    }
}
```

In practice, most classes have one constructor. So you rarely need `@Autowired`.

6.7 A Complete Example: OrderService + EmailService

Let us build a complete, working example. We will create an order system where `OrderController` calls `OrderService`, and `OrderService` calls `EmailService`.

The Wiring Diagram

```
+------------------------------------------------------------+
|     Dependency Wiring Diagram                              |
+------------------------------------------------------------+
|                                                             |
```

```
|   Browser                                                  |
|     |                                                      |
|     | GET /orders/place?item=Laptop                        |
|     v                                                      |
|   +------------------+                                     |
|   | OrderController  |                                     |
|   |                  |                                     |
|   |  needs:          |                                     |
|   |  - OrderService  |---------+                           |
|   +------------------+         |                           |
|                                |                           |
|                                v                           |
|                    +------------------+                     |
|                    | OrderService     |                     |
|                    |                  |                     |
|                    |  needs:          |                     |
|                    |  - EmailService  |------+              |
|                    +------------------+      |              |
|                                              |              |
|                                              v              |
|                                 +------------------+        |
|                                 | EmailService     |        |
|                                 |                  |        |
|                                 |  no dependencies |        |
|                                 +------------------+        |
|                                                            |
|   Spring creates all three objects and wires them.          |
|                                                            |
+------------------------------------------------------------+
```

Step 1: Create EmailService

Create `src/main/java/com/example/hellospringboot/service/EmailService.java`:

```java
package com.example.hellospringboot.service;             // 1

import org.springframework.stereotype.Service;            // 2

@Service                                                  // 3
public class EmailService {

    public void sendConfirmation(String item) {           // 4
        // In a real app, this would send an actual email
        System.out.println(                               // 5
            "[EMAIL] Confirmation sent for item: " + item
        );
    }
}
```

Line-by-line explanation:

- Line 1: The class is in the `service` sub-package. Spring Boot will find it because it is under the base package.
- Line 2: Import the `@Service` annotation.
- Line 3: `@Service` tells Spring: "This class is a service. Create a bean for it and manage it." Spring will create exactly one instance of `EmailService`.
- Line 4: A simple method that "sends" a confirmation.
- Line 5: We use `System.out.println` to simulate sending an email. In a real application, this would use an email library.

Step 2: Create OrderService

Create `src/main/java/com/example/hellospringboot/service/OrderService.java`:

```java
package com.example.hellospringboot.service;             // 1

import org.springframework.stereotype.Service;            // 2

@Service                                                  // 3
public class OrderService {

    private final EmailService emailService;              // 4

    public OrderService(EmailService emailService) {      // 5
        this.emailService = emailService;
    }

    public String placeOrder(String item) {               // 6
        // Process the order
        String message = "Order placed successfully: "
                         + item;
        System.out.println("[ORDER] " + message);         // 7

        // Send confirmation email
        emailService.sendConfirmation(item);              // 8

        return message;                                   // 9
    }
}
```

Line-by-line explanation:

- Line 3: `@Service` registers this class with Spring.
- Line 4: `private final EmailService emailService;` -- Declares the dependency. `final` ensures it is set once.
- Line 5: Constructor injection. Spring will pass an `EmailService` instance here.
- Line 6: The `placeOrder` method takes an item name and returns a confirmation message.
- Line 7: Logs the order to the console.

- Line 8: Uses the injected `EmailService` to send a confirmation.
- Line 9: Returns the confirmation message (which the controller will send to the browser).

Step 3: Create OrderController

Create `src/main/java/com/example/hellospringboot/controller/OrderController.java`:

```java
package com.example.hellospringboot.controller;          // 1

import com.example.hellospringboot.service.OrderService; // 2
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController                                          // 3
public class OrderController {

    private final OrderService orderService;             // 4

    public OrderController(OrderService orderService) {  // 5
        this.orderService = orderService;
    }

    @GetMapping("/orders/place")                         // 6
    public String placeOrder(
            @RequestParam String item) {                 // 7
        return orderService.placeOrder(item);            // 8
    }
}
```

Line-by-line explanation:

- Line 3: `@RestController` marks this class as a web controller.
- Line 4: Declares the dependency on `OrderService`.
- Line 5: Constructor injection. Spring passes `OrderService` (which already has `EmailService` injected).
- Line 6: `@GetMapping("/orders/place")` maps the URL `/orders/place` to this method.
- Line 7: `@RequestParam String item` tells Spring to read a query parameter named `item` from the URL. If the URL is `/orders/place?item=Laptop`, then `item` will be `"Laptop"`.
- Line 8: Calls the service and returns its result to the browser.

Step 4: Run and Test

Start the application:

```bash
./mvnw spring-boot:run
```

Open your browser and visit:

```
http://localhost:8080/orders/place?item=Laptop
```

Browser shows:

```
Order placed successfully: Laptop
```

Console shows:

```
[ORDER] Order placed successfully: Laptop
[EMAIL] Confirmation sent for item: Laptop
```

The request flowed through three layers:

```
Browser --> OrderController --> OrderService --> EmailService
```

Spring created all three objects and wired them together. You never called `new` on any of them.

6.8 Why Dependency Injection Matters

Reason 1: Easy to Test

With dependency injection, testing is straightforward:

```java
// In a test, create a mock EmailService
EmailService mockEmail = new EmailService() {
    @Override
    public void sendConfirmation(String item) {
        // Do nothing (fake email)
    }
};
```

```
// Pass the mock to OrderService
OrderService orderService = new OrderService(mockEmail);

// Test the order logic without sending real emails
String result = orderService.placeOrder("Laptop");
```

No Spring needed. No configuration. Just plain Java. You inject a fake dependency and test the logic.

Reason 2: Easy to Change

Want to switch from email to SMS notifications? Create a new service and swap it:

```java
@Service
public class SmsService {
    public void sendConfirmation(String item) {
        System.out.println("[SMS] Confirmation for: " + item);
    }
}
```

If you program to an interface (which we will cover in later chapters), you can swap `EmailService` for `SmsService` without changing `OrderService` at all.

Reason 3: Clear Dependencies

The constructor tells you exactly what a class needs:

```java
public OrderService(EmailService emailService,
                    PaymentService paymentService,
                    InventoryService inventoryService) {
```

One look at the constructor and you know: `OrderService` needs three things. No surprises. No hidden connections.

6.9 Annotations That Create Beans

You have seen `@Service` and `@RestController`. Here is the full list of annotations that tell Spring to create a bean:

Annotation	Purpose
`@Component`	General-purpose bean. The base annotation.
`@Service`	A service class (business logic).
`@Repository`	A data access class (database).
`@Controller`	A web controller (returns views).
`@RestController`	A REST controller (returns data).
`@Configuration`	A class that defines beans.

They all do the same basic thing: tell Spring to create and manage an instance. The different names (`@Service`, `@Repository`, etc.) are for clarity. They tell other developers the purpose of the class.

```
+-------------------------------------------------------------+
|     Bean Stereotype Annotations                             |
+-------------------------------------------------------------+
|                                                             |
|                   @Component                                |
|                   (base annotation)                         |
|                       |                                     |
|          +------------+------------+                        |
|          |            |            |                        |
|      @Service    @Repository  @Controller                   |
|    (business     (database    (web layer)                   |
|     logic)       access)          |                         |
|                               @RestController               |
|                              (REST API)                     |
|                                                             |
|   All of these tell Spring:                                 |
|   "Create a bean from this class."                          |
|                                                             |
+-------------------------------------------------------------+
```

Common Mistakes

1. Using `new` to create Spring beans. If you write `new OrderService()`, Spring does not know about that instance. Dependencies will not be injected. Always let Spring create the objects.

2. Forgetting `@Service` or `@Component`. If a class does not have a stereotype annotation, Spring will not create a bean for it. You will get an error: "No qualifying bean of type 'X' found."
3. Using field injection in production code. `@Autowired` on fields works, but hides dependencies and makes testing harder. Use constructor injection instead.
4. Circular dependencies. If A depends on B and B depends on A, Spring cannot create either one. Redesign your classes to break the circle.
5. Putting classes outside the component scan range. If your class is not in the base package or a sub-package, Spring will not find it. Always check your package structure.

Best Practices

1. Always use constructor injection. Make dependencies `final` and pass them through the constructor.
2. Omit `@Autowired` when there is only one constructor. Spring auto-detects it. Less code, same result.
3. Use the right stereotype annotation. Use `@Service` for services, `@Repository` for data access, `@RestController` for REST endpoints. This makes your code self-documenting.
4. Keep beans focused. Each bean should do one thing well. `EmailService` sends emails. `OrderService` manages orders. Do not mix responsibilities.
5. Program to interfaces. Define an interface (e.g., `NotificationService`) and implement it (e.g., `EmailService`). This makes swapping implementations easy. We will cover this in a later chapter.

Quick Summary

Dependency injection is how Spring connects your classes together:

1. You annotate classes with `@Service`, `@Repository`, or `@RestController`.
2. Spring creates instances (beans) of those classes.
3. Spring looks at constructors to see what each bean needs.
4. Spring passes the required beans through constructors (injection).

The IoC container manages the entire lifecycle. You declare what you need. Spring provides it. This makes your code loosely coupled, easy to test, and easy to change.

Key Points

- A dependency is an object that another object needs to do its job.
- The old way (`new EmailService()`) creates tight coupling. The code is hard to test and hard to change.
- Dependency injection means the framework creates objects and passes them where needed.
- The IoC container is Spring's object manager. It creates beans and wires them together.
- Constructor injection is the recommended way. Declare dependencies as `final` fields. Pass them through the constructor.
- `@Autowired` is optional when a class has only one constructor.
- `@Service`, `@Repository`, `@Controller`, and `@RestController` all tell Spring to create a bean.
- Dependency injection makes code testable (inject mocks), flexible (swap implementations), and clear (dependencies are visible in the constructor).

Practice Questions

1. What is a dependency? Give a real-life analogy and a code example.
2. Why is using `new` to create dependencies considered a bad practice in Spring applications?
3. Explain the Inversion of Control (IoC) concept using the hiring agency analogy.
4. What is the difference between constructor injection and field injection? Why is constructor injection preferred?
5. You see this error: `"No qualifying bean of type 'EmailService' found."` What is the most likely cause? How do you fix it?

Exercises

Exercise 1: Add a PaymentService

Create a `PaymentService` class with a method `processPayment(double amount)` that prints "Payment processed: $" followed by the amount. Inject it into `OrderService`. Update the `placeOrder` method to call `processPayment` before sending the confirmation email.

Test it by visiting:

```
http://localhost:8080/orders/place?item=Laptop
```

The console should show:

```
[PAYMENT] Payment processed: $999.99
[ORDER] Order placed successfully: Laptop
[EMAIL] Confirmation sent for item: Laptop
```

Exercise 2: Count Orders

Add an `OrderCountService` that keeps track of how many orders have been placed. It should have two methods:

- `increment()` -- adds 1 to the count
- `getCount()` -- returns the current count

Inject it into `OrderService`. Call `increment()` every time an order is placed. Add a new endpoint `/orders/count` in the controller that returns the current count.

Exercise 3: Trace the Wiring

Draw a diagram (on paper or in a text file) that shows:

1. All the beans in your application
2. Which bean depends on which other bean
3. The arrows showing the direction of dependency

Compare your diagram with the ASCII wiring diagram in this chapter.

What Is Next?

You now understand how Spring connects your classes together using dependency injection. In the next chapters, we will build on this foundation. You will learn how to work with

databases, validate user input, and handle errors -- all using the patterns you learned here.

Chapter 7: Beans and Annotations

What You Will Learn

In this chapter, you will learn:

- What a Spring Bean is and why it matters
- How Spring creates and manages objects for you
- The difference between `@Component`, `@Service`, `@Repository`, and `@Controller`
- How to choose the right annotation with a simple flowchart
- The difference between singleton and prototype beans
- How to create beans using `@Bean` in a `@Configuration` class
- The lifecycle of a Spring Bean from birth to death
- How component scanning finds your beans automatically

Why This Chapter Matters

Imagine you run a company. You have employees on your payroll. You do not go to the street every morning to find random people to do work. Instead, you hire employees once. They are on your payroll. You know their names. You assign them tasks. They show up when needed.

Spring Boot works the same way. The objects in your application are like employees. Spring keeps them on its "payroll." These managed objects are called beans. Spring creates them, tracks them, and hands them out when other parts of your code need them.

Without beans, you would create objects manually everywhere. You would lose track of them. You would create duplicates. Your code would become messy. Beans solve all of these problems.

This chapter is the foundation of everything in Spring. Every feature you will learn after this -- REST controllers, data access, security -- they all depend on beans. Master this chapter, and the rest of Spring will make sense.

7.1 What Is a Spring Bean?

A bean is simply a Java object that Spring creates and manages for you.

Think of it this way:

- A regular Java object is like a freelancer. You create it yourself. You manage it yourself. When you are done, you throw it away.
- A Spring Bean is like a full-time employee. Spring hires it. Spring manages it. Spring makes it available to anyone who needs it.

Here is a regular Java object:

```java
// You create it yourself
NotificationService service = new NotificationService();
service.sendEmail("hello@example.com", "Welcome!");
```

Here is a Spring Bean:

```java
// Spring creates it and gives it to you
@Autowired
private NotificationService service;

// You just use it
service.sendEmail("hello@example.com", "Welcome!");
```

The difference is who creates the object. With regular Java, you do it. With Spring, the framework does it.

The Spring Container

Spring keeps all its beans in a special place called the Application Context. Think of it as an employee directory. When any part of your code needs a bean, Spring looks it up in this directory and hands it over.

```
+-----------------------------------------------------+
|           Spring Application Context                |
|              (Employee Directory)                    |
|                                                      |
|  +----------------+  +--------------------+         |
|  | UserService    |  | EmailService       |         |
```

```
|  | (Bean)         |  | (Bean)            |        |
|  +----------------+  +-------------------+        |
|                                                     |
|  +----------------+  +-------------------+        |
|  | UserRepository |  | OrderController   |        |
|  | (Bean)         |  | (Bean)            |        |
|  +----------------+  +-------------------+        |
|                                                     |
+-----------------------------------------------------+
```

Application Context is the container that holds all your beans. It is like a big box where Spring puts every object it creates. When your code needs an object, Spring reaches into this box and pulls out the right one.

7.2 The @Component Annotation

The simplest way to tell Spring "please manage this object" is the `@Component` annotation.

Annotation means a special label you put on your Java class. It starts with the `@` symbol. It gives Spring extra information about your class.

```java
package com.example.demo.util;

import org.springframework.stereotype.Component;

@Component  // "Hey Spring, please manage this class!"
public class GreetingGenerator {

    public String generateGreeting(String name) {
        return "Hello, " + name + "! Welcome aboard!";
    }
}
```

Line-by-line explanation:

- `@Component` -- This annotation tells Spring: "Create an object from this class and keep it in your container." Without this annotation, Spring ignores the class completely.
- `public class GreetingGenerator` -- A regular Java class. Nothing special about it.
- `public String generateGreeting(String name)` -- A regular method. It takes a name and returns a greeting message.

When your application starts, Spring sees the `@Component` annotation. It creates one instance of `GreetingGenerator`. It stores that instance in the Application Context. Now any other bean

can use it.

Using a Component Bean

```java
package com.example.demo.service;

import com.example.demo.util.GreetingGenerator;
import org.springframework.stereotype.Service;

@Service
public class WelcomeService {

    private final GreetingGenerator greetingGenerator;

    // Spring automatically provides the GreetingGenerator bean here
    public WelcomeService(GreetingGenerator greetingGenerator) {
        this.greetingGenerator = greetingGenerator;
    }

    public String welcomeUser(String username) {
        return greetingGenerator.generateGreeting(username);
    }
}
```

Line-by-line explanation:

- `@Service` -- Another annotation similar to `@Component`. We will explain the difference shortly.
- `private final GreetingGenerator greetingGenerator;` -- A field to hold the `GreetingGenerator` bean. The `final` keyword means it cannot change after being set.
- `public WelcomeService(GreetingGenerator greetingGenerator)` -- This is constructor injection. Spring sees that this constructor needs a `GreetingGenerator`. It looks in the Application Context, finds one, and passes it in automatically.
- `greetingGenerator.generateGreeting(username)` -- We use the bean just like a normal object.

Constructor injection means Spring provides the required beans through the constructor of your class. This is the recommended way to get beans in Spring.

7.3 Stereotype Annotations: @Service, @Repository, @Controller

Spring gives you four annotations to mark your beans. They all do the same basic thing: they tell Spring to manage the class. But each one has a specific meaning.

Annotation	Purpose	Real-Life Analogy
`@Component`	Generic bean, no specific role	General employee
`@Service`	Business logic	Manager who makes decisions
`@Repository`	Database access	Filing clerk in the archive
`@Controller`	Handles web requests	Receptionist at the front desk

@Component -- The Generic Worker

Use `@Component` when your class does not fit into service, repository, or controller categories.

```java
@Component
public class PdfGenerator {
    public byte[] generateReport(String title, String content) {
        // Logic to create a PDF document
        return new byte[0]; // Simplified
    }
}
```

A `PdfGenerator` is not business logic. It is not database access. It is not a web controller. It is a utility. So `@Component` is the right choice.

@Service -- The Business Brain

Use `@Service` for classes that contain your business logic. Business logic means the rules and decisions your application makes.

```java
package com.example.demo.service;

import org.springframework.stereotype.Service;

@Service
public class PricingService {

    private static final double TAX_RATE = 0.08;

    public double calculateTotal(double price, int quantity) {
        double subtotal = price * quantity;
```

```
        double tax = subtotal * TAX_RATE;
        return subtotal + tax;
    }

    public double applyDiscount(double total, double discountPercent) {
        return total * (1 - discountPercent / 100);
    }
}
```

This class makes business decisions. "What is the total price?" and "How much discount to apply?" are business questions. So we use `@Service`.

@Repository -- The Data Keeper

Use `@Repository` for classes that talk to the database. Spring gives `@Repository` a special power: it automatically translates database errors into Spring exceptions.

```java
package com.example.demo.repository;

import com.example.demo.model.Product;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class ProductRepository {

    private final List<Product> products = new ArrayList<>();
    private long nextId = 1;

    public Product save(Product product) {
        product.setId(nextId++);
        products.add(product);
        return product;
    }

    public Optional<Product> findById(Long id) {
        return products.stream()
                .filter(p -> p.getId().equals(id))
                .findFirst();
    }

    public List<Product> findAll() {
        return new ArrayList<>(products);
    }
}
```

Optional is a Java class that may or may not contain a value. It helps you avoid `NullPointerException`. Instead of returning `null` when a product is not found, you return an empty `Optional`.

@Controller -- The Front Desk

Use `@Controller` (or `@RestController`) for classes that handle web requests. We will cover this in detail in Chapter 9.

```java
package com.example.demo.controller;

import com.example.demo.service.PricingService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class PriceController {

    private final PricingService pricingService;

    public PriceController(PricingService pricingService) {
        this.pricingService = pricingService;
    }

    @GetMapping("/price")
    public String getPrice(@RequestParam double price,
                           @RequestParam int quantity) {
        double total = pricingService.calculateTotal(price, quantity);
        return "Total: $" + String.format("%.2f", total);
    }
}
```

The Truth About These Annotations

Here is a secret. `@Service`, `@Repository`, and `@Controller` are all built on top of `@Component`. They are specialized versions of `@Component`.

```
         @Component
        /    |     \
       /     |      \
@Service  @Repository  @Controller
```

If you replaced `@Service` with `@Component`, your code would still work. But using the right annotation makes your code easier to read. When another developer sees `@Service`, they immediately know this class contains business logic. When they see `@Repository`, they know it

handles data.

It is like wearing a uniform. A chef and a doctor both work at a hospital cafeteria. But the chef wears a chef hat, and the doctor wears a white coat. You can tell who does what just by looking.

7.4 Decision Flowchart: Which Annotation to Use?

When you create a new class, follow this flowchart:

```
+---------------------------+
| Does this class handle     |
| HTTP requests?             |
+---------------------------+
       |            |
      YES           NO
       |            |
+-------------+     |
| @Controller |     |
| or          |     |
| @RestController   |
+-------------+     |
                    |
+---------------------------+
| Does this class talk to    |
| a database or external     |
| data source?               |
+---------------------------+
       |            |
      YES           NO
       |            |
+--------------+    |
| @Repository  |    |
+--------------+    |
                    |
+---------------------------+
| Does this class contain    |
| business logic or rules?   |
+---------------------------+
       |            |
      YES           NO
       |            |
+-----------+  +-----------+
| @Service  |  | @Component|
+-----------+  +-----------+
```

Quick rule: Controller > Repository > Service > Component. Check in that order.

7.5 Singleton vs Prototype Scope

By default, Spring creates one instance of each bean. This is called singleton scope. Every time someone asks for that bean, they get the same object.

Singleton Scope (Default)

Singleton means "only one." Spring creates the bean once and reuses it everywhere.

```java
@Service
public class CounterService {

    private int count = 0;

    public int increment() {
        return ++count;
    }

    public int getCount() {
        return count;
    }
}
```

If three different parts of your application use `CounterService`, they all share the same instance. If one part calls `increment()`, the other parts see the updated count.

```
+------------------+
| CounterService   |  <-- Only ONE instance exists
| count = 5       |
+------------------+
      ^   ^   ^
      |   |   |
 ControllerA  |  ControllerC
      ControllerB

All three controllers share the SAME CounterService.
If ControllerA increments, ControllerB sees the change.
```

This is like having one whiteboard in the office. Everyone reads and writes on the same whiteboard.

Prototype Scope

Sometimes you want a fresh object every time. Use the `@Scope` annotation with `prototype`.

Prototype means "make a new copy each time." Spring creates a brand new bean every time someone asks for it.

```java
package com.example.demo.model;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class ShoppingCart {

    private final List<String> items = new ArrayList<>();

    public void addItem(String item) {
        items.add(item);
    }

    public List<String> getItems() {
        return new ArrayList<>(items);
    }

    public int getItemCount() {
        return items.size();
    }
}
```

```
Request from User A --> new ShoppingCart (items: [Book])
Request from User B --> new ShoppingCart (items: [Pen, Notebook])
Request from User C --> new ShoppingCart (items: [])

Each user gets their OWN ShoppingCart. They do NOT share.
```

This is like giving each customer their own shopping basket at a store. They do not share baskets.

When to Use Each Scope

Scope	When to Use	Example
Singleton	Stateless services, shared logic	PricingService
Prototype	Stateful objects, per-user/per-request data	ShoppingCart

Stateless means the bean does not remember anything between method calls. It processes data and returns a result. It does not store data inside itself.

Stateful means the bean remembers data. A shopping cart remembers what items are in it.

> Rule of thumb: If your bean has fields that change during use, consider prototype scope. If your bean only has methods that compute and return results, use singleton (the default).

7.6 Creating Beans with @Bean and @Configuration

Sometimes you cannot add `@Component` to a class. Maybe the class comes from an external library. Maybe you need to configure the object before using it. In these cases, use `@Bean` inside a `@Configuration` class.

@Configuration marks a class as a source of bean definitions. Think of it as a recipe book. Each `@Bean` method is one recipe.

@Bean marks a method that creates and returns an object. Spring will call this method, take the returned object, and register it as a bean.

Example: Configuring an External Library

Imagine you use a library class called `ObjectMapper` from the Jackson JSON library. You cannot edit its source code to add `@Component`. So you create a configuration class:

```java
package com.example.demo.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.enable(SerializationFeature.INDENT_OUTPUT);
```

```
        return mapper;
    }
}
```

Line-by-line explanation:

- `@Configuration` -- Tells Spring: "This class contains bean definitions. Scan it for `@Bean` methods."
- `@Bean` -- Tells Spring: "Call this method at startup. Take the returned object and manage it as a bean."
- `public ObjectMapper objectMapper()` -- The method name (`objectMapper`) becomes the bean name by default.
- `mapper.enable(SerializationFeature.INDENT_OUTPUT)` -- We configure the object before Spring takes it. This is the advantage of `@Bean`. You can customize the object.
- `return mapper` -- Spring takes this returned object and puts it in the Application Context.

Example: Multiple Beans of the Same Type

You can create multiple beans of the same type. Give them different names.

```java
@Configuration
public class DataSourceConfig {

    @Bean
    public DataSource primaryDataSource() {
        // Configuration for the main database
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:h2:mem:maindb");
        ds.setUsername("sa");
        return ds;
    }

    @Bean
    public DataSource reportingDataSource() {
        // Configuration for the reporting database
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:h2:mem:reportdb");
        ds.setUsername("sa");
        return ds;
    }
}
```

When you have multiple beans of the same type, use `@Qualifier` to tell Spring which one you want:

```java
@Service
public class ReportService {

    private final DataSource dataSource;

    public ReportService(
            @Qualifier("reportingDataSource") DataSource dataSource) {
        this.dataSource = dataSource;
    }
}
```

@Qualifier tells Spring: "I do not want just any DataSource. I want the specific one named `reportingDataSource`."

@Component vs @Bean: When to Use Which

Feature	@Component	@Bean
Where to use	On the class itself	On a method in @Configuration
Best for	Your own classes	Third-party library classes
Customization	Limited	Full control over creation
Auto-detected	Yes, by component scanning	Yes, through @Configuration

---

7.7 Bean Lifecycle

Every bean goes through a lifecycle. It is born, it lives, and it dies. Spring gives you hooks to run code at each stage.

```
+-------------+     +-------------+     +-------------+
|  Bean       | --> |  Bean       | --> |  Bean       |
|  Created    |     |  Initialized|     |  Destroyed  |
|             |     |  (Ready)    |     |             |
+-------------+     +-------------+     +-------------+
   |                    |                    |
   |                    |                    |
Constructor        @PostConstruct       @PreDestroy
called             method called        method called
```

@PostConstruct -- Run Code After Bean Is Created

@PostConstruct marks a method that Spring calls right after creating the bean and injecting all its dependencies. Use it for initialization logic.

```java
package com.example.demo.service;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Service;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class CacheService {

    private static final Logger log =
            LoggerFactory.getLogger(CacheService.class);

    private final Map<String, String> cache = new HashMap<>();

    @PostConstruct
    public void init() {
        log.info("CacheService is starting up...");
        cache.put("app.name", "My Spring App");
        cache.put("app.version", "1.0.0");
        log.info("Cache initialized with {} entries", cache.size());
    }

    public String get(String key) {
        return cache.get(key);
    }

    public void put(String key, String value) {
        cache.put(key, value);
    }

    @PreDestroy
    public void cleanup() {
        log.info("CacheService is shutting down...");
        cache.clear();
        log.info("Cache cleared");
    }
}
```

Line-by-line explanation:

- `@PostConstruct` on `init()` -- Spring calls this method after creating the `CacheService` bean. We use it to pre-load some data into the cache.
- `@PreDestroy` on `cleanup()` -- Spring calls this method before destroying the bean (when the application shuts down). We use it to clean up resources.

Output when the application starts:

```
CacheService is starting up...
Cache initialized with 2 entries
```

Output when the application stops:

```
CacheService is shutting down...
Cache cleared
```

The Complete Bean Lifecycle

Here is the full lifecycle in order:

```
1. Spring finds the class (component scanning)
2. Spring calls the constructor
3. Spring injects dependencies
4. Spring calls @PostConstruct method
5. Bean is ready to use
6. ... application runs ...
7. Application shutdown begins
8. Spring calls @PreDestroy method
9. Bean is destroyed
```

Think of it like an employee's journey:

1. HR finds the candidate (component scanning)
2. Employee is hired (constructor)
3. Employee gets a desk, computer, and tools (dependency injection)
4. Employee completes orientation training (@PostConstruct)
5. Employee works daily (bean is in use)
6. Employee gets retirement notice (application shutdown)
7. Employee returns equipment and wraps up (@PreDestroy)
8. Employee leaves the company (bean destroyed)

7.8 Component Scanning

How does Spring find your beans? Through component scanning.

Component scanning means Spring automatically searches through your packages to find classes with annotations like `@Component`, `@Service`, `@Repository`, and `@Controller`.

How It Works

Your main application class has `@SpringBootApplication`. This annotation includes `@ComponentScan`, which tells Spring to scan the package of the main class and all sub-packages.

```java
package com.example.demo;  // <-- Spring scans from here

@SpringBootApplication  // Includes @ComponentScan
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Spring will scan:

```
com.example.demo           <-- Scanned (main package)
com.example.demo.service   <-- Scanned (sub-package)
com.example.demo.repository <-- Scanned (sub-package)
com.example.demo.controller <-- Scanned (sub-package)
com.example.demo.config    <-- Scanned (sub-package)

com.example.other          <-- NOT scanned (different root)
com.another.project        <-- NOT scanned (different root)
```

The Scanning Process

```
+-------------------------------------------------------+
|  Application Starts                                   |
+-------------------------------------------------------+
          |
          v
+-------------------------------------------------------+
|  @SpringBootApplication triggers component scanning   |
+-------------------------------------------------------+
          |
          v
+-------------------------------------------------------+
|  Spring scans com.example.demo and sub-packages       |
+-------------------------------------------------------+
          |
          v
```

```
+-------------------------------------------------------+
|  Found @Component on GreetingGenerator? --> Create    |
|  Found @Service on PricingService?      --> Create    |
|  Found @Repository on ProductRepository? --> Create   |
|  Found @Controller on PriceController?  --> Create    |
|  Found plain class Calculator?          --> Skip      |
+-------------------------------------------------------+
          |
          v
+-------------------------------------------------------+
|  All beans created and stored in Application Context  |
+-------------------------------------------------------+
```

Custom Component Scanning

You can tell Spring to scan additional packages:

```java
@SpringBootApplication
@ComponentScan(basePackages = {
    "com.example.demo",
    "com.example.shared"
})
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

But in most projects, the default scanning is enough. Just keep all your classes under the same root package as your main class.

7.9 Putting It All Together

Let us build a small example that uses everything we learned. We will create a simple notification system.

Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── config/
```

```
│   └── NotificationConfig.java
├── model/
│   └── Notification.java
├── repository/
│   └── NotificationRepository.java
├── service/
│   └── NotificationService.java
└── util/
    └── MessageFormatter.java
```

The Model

```java
package com.example.demo.model;

import java.time.LocalDateTime;

public class Notification {

    private Long id;
    private String recipient;
    private String message;
    private boolean sent;
    private LocalDateTime createdAt;

    public Notification() {
        this.createdAt = LocalDateTime.now();
        this.sent = false;
    }

    public Notification(String recipient, String message) {
        this();
        this.recipient = recipient;
        this.message = message;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getRecipient() { return recipient; }
    public void setRecipient(String recipient) {
        this.recipient = recipient;
    }
    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }
    public boolean isSent() { return sent; }
    public void setSent(boolean sent) { this.sent = sent; }
    public LocalDateTime getCreatedAt() { return createdAt; }

    @Override
    public String toString() {
```

```
        return "Notification{" +
                "id=" + id +
                ", recipient='" + recipient + '\'' +
                ", message='" + message + '\'' +
                ", sent=" + sent +
                '}';
    }
}
```

This is a plain Java class. No annotations. It is not a bean. It is just a data container.

The Utility (@Component)

```java
package com.example.demo.util;

import org.springframework.stereotype.Component;

@Component
public class MessageFormatter {

    private static final String PREFIX = "[NOTIFICATION] ";

    public String format(String recipient, String message) {
        return PREFIX + "To: " + recipient + " | " + message;
    }

    public String formatUrgent(String recipient, String message) {
        return "*** URGENT *** " +
                PREFIX + "To: " + recipient + " | " + message;
    }
}
```

We use `@Component` because this is a utility class. It does not contain business logic, access data, or handle web requests.

The Repository (@Repository)

```java
package com.example.demo.repository;

import com.example.demo.model.Notification;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
```

```
@Repository
public class NotificationRepository {

    private final List<Notification> notifications = new ArrayList<>();
    private long nextId = 1;

    public Notification save(Notification notification) {
        notification.setId(nextId++);
        notifications.add(notification);
        return notification;
    }

    public Optional<Notification> findById(Long id) {
        return notifications.stream()
                .filter(n -> n.getId().equals(id))
                .findFirst();
    }

    public List<Notification> findAll() {
        return new ArrayList<>(notifications);
    }

    public List<Notification> findUnsent() {
        return notifications.stream()
                .filter(n -> !n.isSent())
                .toList();
    }
}
```

We use `@Repository` because this class stores and retrieves data.

The Service (@Service)

```java
package com.example.demo.service;

import com.example.demo.model.Notification;
import com.example.demo.repository.NotificationRepository;
import com.example.demo.util.MessageFormatter;
import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class NotificationService {

    private static final Logger log =
            LoggerFactory.getLogger(NotificationService.class);
```

```
    private final NotificationRepository repository;
    private final MessageFormatter formatter;

    public NotificationService(NotificationRepository repository,
                               MessageFormatter formatter) {
        this.repository = repository;
        this.formatter = formatter;
    }

    @PostConstruct
    public void init() {
        log.info("NotificationService is ready");
    }

    public Notification createNotification(String recipient,
                                           String message) {
        String formattedMessage = formatter.format(recipient, message);
        Notification notification =
                new Notification(recipient, formattedMessage);
        return repository.save(notification);
    }

    public Notification sendNotification(Long id) {
        Notification notification = repository.findById(id)
                .orElseThrow(() -> new RuntimeException(
                        "Notification not found: " + id));

        // Simulate sending
        log.info("Sending: {}", notification.getMessage());
        notification.setSent(true);
        return notification;
    }

    public List<Notification> getUnsentNotifications() {
        return repository.findUnsent();
    }

    public List<Notification> getAllNotifications() {
        return repository.findAll();
    }
}
```

We use `@Service` because this class contains business logic. It decides how to create, format, and send notifications.

Notice the constructor takes two parameters: `NotificationRepository` and `MessageFormatter`. Spring injects both automatically.

The Configuration (@Configuration with @Bean)

```java
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.format.DateTimeFormatter;

@Configuration
public class NotificationConfig {

    @Bean
    public DateTimeFormatter notificationDateFormatter() {
        return DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    }
}
```

We use `@Bean` here because `DateTimeFormatter` is a Java standard library class. We cannot add `@Component` to it. So we create it in a `@Configuration` class.

How Spring Wires It All Together

```
+---------------------------------------------------+
|           Spring Application Context              |
|                                                    |
|  Scanning com.example.demo...                     |
|                                                    |
|  Found: @Component MessageFormatter    --> Create  |
|  Found: @Repository NotificationRepo  --> Create   |
|  Found: @Service NotificationService   --> Create  |
|  Found: @Configuration NotificationConfig          |
|         @Bean notificationDateFormatter --> Create  |
|                                                    |
|  Wiring dependencies:                              |
|  NotificationService needs:                        |
|    - NotificationRepository  ✓ Found              |
|    - MessageFormatter        ✓ Found              |
|                                                    |
|  All beans created and wired!                      |
+---------------------------------------------------+
```

Common Mistakes

Mistake 1: Forgetting the Annotation

```java
// WRONG: No annotation -- Spring ignores this class
public class EmailService {
    public void sendEmail(String to, String body) {
        // ...
    }
}
```

```java
// CORRECT: Add @Service so Spring manages it
@Service
public class EmailService {
    public void sendEmail(String to, String body) {
        // ...
    }
}
```

Mistake 2: Putting Classes in the Wrong Package

```
// WRONG: Class is outside the scanned package
com.other.project.MyService  <-- Spring will NOT find this

// CORRECT: Class is under the main package
com.example.demo.service.MyService  <-- Spring WILL find this
```

Mistake 3: Using new Instead of Injection

```java
// WRONG: Creating the object yourself
@Service
public class OrderService {
    public void processOrder() {
        PricingService pricing = new PricingService();  // BAD!
        pricing.calculateTotal(10.0, 2);
    }
}
```

```java
// CORRECT: Let Spring inject it
@Service
public class OrderService {
    private final PricingService pricingService;

    public OrderService(PricingService pricingService) {
        this.pricingService = pricingService;
    }
```

```
    public void processOrder() {
        pricingService.calculateTotal(10.0, 2);
    }
}
```

When you use `new`, you get a different object than the one Spring manages. That object will not have its own dependencies injected. Always let Spring provide beans through injection.

Mistake 4: Circular Dependencies

```java
// WRONG: A depends on B, and B depends on A
@Service
public class ServiceA {
    public ServiceA(ServiceB serviceB) { }  // Needs B
}

@Service
public class ServiceB {
    public ServiceB(ServiceA serviceA) { }  // Needs A
}
// Spring cannot create either one first!
```

This is like two employees who each say "I will not start until the other person starts first." Nobody starts. Redesign your code to break the cycle.

Best Practices

1. Use the most specific annotation. Use `@Service` for services, `@Repository` for data access, `@Controller` for web controllers. Only use `@Component` when none of the others fit.
2. Use constructor injection. It makes dependencies clear, supports immutability with `final` fields, and makes testing easier.
3. Keep beans stateless when possible. Singleton beans are shared. If they store mutable state, you will get bugs with concurrent access.
4. One responsibility per bean. A service should not also access the database directly. Delegate to a repository.
5. Keep your classes in sub-packages of the main package. This ensures component scanning finds them.

6. Use `@Bean` for third-party classes. You cannot annotate library code, so configure it in a `@Configuration` class.
7. Name your beans clearly. The class name becomes the bean name (with a lowercase first letter). `PricingService` becomes the bean `pricingService`.

Quick Summary

- A bean is an object managed by Spring. Spring creates it, stores it, and provides it to other beans.
- The Application Context is the container that holds all beans.
- `@Component` is the generic annotation. `@Service`, `@Repository`, and `@Controller` are specialized versions.
- Singleton scope (default) means one instance shared everywhere. Prototype scope means a new instance each time.
- `@Bean` in a `@Configuration` class lets you create beans from third-party classes or when you need custom setup.
- `@PostConstruct` runs code after a bean is created. `@PreDestroy` runs code before it is destroyed.
- Component scanning automatically finds annotated classes in the main package and sub-packages.

Key Points

- Every Spring feature depends on beans. They are the foundation.
- Always use constructor injection over field injection.
- Use the right stereotype annotation for readability.
- Beans are singleton by default. Use prototype only when needed.
- Do not use `new` to create objects that should be beans.
- Keep beans in sub-packages of your main application class.

Practice Questions

1. What is the difference between a regular Java object and a Spring Bean?
2. You have a class that sends SMS messages. It does not access a database and does not handle web requests. Which annotation should you use: `@Component`, `@Service`, `@Repository`, or `@Controller`? Why?
3. Explain the difference between singleton and prototype scope. Give an example of when you would use each.
4. Why would you use `@Bean` in a `@Configuration` class instead of putting `@Component` on the class directly?
5. What happens if you put a class in a package that is not under the main application package?

Exercises

Exercise 1: Build a Calculator Service

Create a Spring Boot application with the following beans:

- `MathUtils` (`@Component`) -- has methods `add(int a, int b)`, `subtract(int a, int b)`, `multiply(int a, int b)`
- `CalculatorService` (`@Service`) -- uses `MathUtils` to perform calculations and logs the results
- `AppConfig` (`@Configuration`) -- creates a `DecimalFormat` bean configured with pattern `"#,###.##"`

Run the application and verify all beans are created using `@PostConstruct` logging.

Exercise 2: Employee Management System

Create beans for a simple employee management system:

- `Employee` model class (not a bean) with fields: `id`, `name`, `department`, `salary`
- `EmployeeRepository` (`@Repository`) -- stores employees in a `List`, with `save()`, `findById()`, `findAll()`, and `findByDepartment()` methods
- `SalaryCalculator` (`@Component`) -- has a method `calculateAnnualSalary(double monthlySalary)` that multiplies by 12 and adds a 10% bonus
- `EmployeeService` (`@Service`) -- uses both `EmployeeRepository` and `SalaryCalculator`

Add `@PostConstruct` to `EmployeeService` to pre-load three employees.

Exercise 3: Prototype Scope Experiment

Create a `RequestTracker` bean with prototype scope that has an `id` field set to a random UUID in its constructor. Create a `@Service` that gets two `RequestTracker` instances and prints their IDs. Verify they are different objects.

Hint: You cannot inject a prototype bean into a singleton bean using simple constructor injection and get a new instance each time. Look up `ObjectProvider<T>` or `ApplicationContext.getBean()`.

What Is Next?

You now understand how Spring manages objects. But real applications need configuration. They need different settings for development, testing, and production. In the next chapter, you will learn about properties and profiles -- how to configure your Spring Boot application for different environments without changing code.

Chapter 8: Properties and Profiles

What You Will Learn

In this chapter, you will learn:

- The difference between `.properties` and `.yml` configuration files
- The most common Spring Boot properties
- How to read property values using `@Value`
- How to group related properties using `@ConfigurationProperties`
- What profiles are and why you need them
- How to create profile-specific configuration files
- How to activate profiles
- The order in which Spring resolves properties

Why This Chapter Matters

Imagine you are building a house. The blueprint is the same, but the materials change depending on where you build. In a cold climate, you use thick insulation. In a warm climate, you use lighter materials. The house design stays the same. Only the materials change.

Software works the same way. Your code stays the same. But the configuration changes depending on where the application runs. On your laptop, you connect to a local test database. On the production server, you connect to the real database. On a test server, you connect to a mock database.

Without proper configuration management, you would need to change your code every time you deploy to a different environment. That is risky and error-prone. Spring Boot profiles and properties solve this problem elegantly.

8.1 Properties Files: .properties vs .yml

Spring Boot reads configuration from files in your `src/main/resources` folder. You have two format choices.

The .properties Format

This is the traditional Java format. Each setting is a key-value pair on its own line.

```properties
# src/main/resources/application.properties

# Server settings
server.port=8081
server.servlet.context-path=/api

# Application info
app.name=BookStore
app.version=1.0.0

# Database settings
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.username=sa
spring.datasource.password=
```

Key-value pair means a setting name (key) and its value, separated by an equals sign. `server.port=8081` means "the server port is 8081."

The `#` symbol starts a comment. Spring ignores everything after it on that line.

The .yml (YAML) Format

YAML stands for "YAML Ain't Markup Language." It uses indentation instead of dots to show hierarchy.

```yaml
# src/main/resources/application.yml

# Server settings
server:
  port: 8081
  servlet:
    context-path: /api

# Application info
app:
  name: BookStore
  version: 1.0.0
```

```
# Database settings
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    username: sa
    password:
```

Side-by-Side Comparison

```
.properties                          .yml
____________________                 ____________________

server.port=8081                     server:
server.servlet.context-path=/api       port: 8081
                                       servlet:
                                         context-path: /api

app.name=BookStore                   app:
app.version=1.0.0                      name: BookStore
                                       version: 1.0.0
```

Which Format Should You Choose?

Feature	.properties	.yml
Readability	Good for flat settings	Better for nested settings
Hierarchy	Uses dots	Uses indentation
Learning curve	Easier for beginners	Slightly harder
Multiple docs	Not supported	Supports `---` separator
Whitespace matters	No	Yes (indentation)

Recommendation for beginners: Start with `.properties`. It is simpler and harder to mess up. Indentation errors in YAML can cause hard-to-find bugs.

Important: Do not use both `application.properties` and `application.yml` at the same time. Pick one format and stick with it. If both exist, `.properties` takes priority.

8.2 Common Spring Boot Properties

Here are the most frequently used properties. You do not need to memorize them. Bookmark this page and come back when needed.

Server Properties

```properties
# Change the port (default is 8080)
server.port=8081

# Add a base path to all URLs
server.servlet.context-path=/api

# Set connection timeout
server.tomcat.connection-timeout=5000
```

Port is a number that identifies your application on the network. Think of it like an apartment number. The computer is the building. The port is the apartment. Port 8080 is the default apartment where Spring Boot lives.

Database Properties

```properties
# H2 in-memory database
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Enable H2 web console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

JPA and Hibernate Properties

```properties
# Show SQL queries in the console
spring.jpa.show-sql=true

# Format SQL output for readability
spring.jpa.properties.hibernate.format_sql=true

# Auto-create database tables from entity classes
spring.jpa.hibernate.ddl-auto=update
```

JPA stands for Java Persistence API. It is the standard way Java applications talk to databases. Hibernate is the most popular implementation of JPA. Think of JPA as the job description, and Hibernate as the person doing the job.

Logging Properties

```properties
# Set the overall logging level
logging.level.root=INFO

# Set logging level for your package
logging.level.com.example.demo=DEBUG

# Log to a file
logging.file.name=app.log
```

Logging levels control how much detail you see in the output. From most to least detail: TRACE > DEBUG > INFO > WARN > ERROR. Setting the level to `INFO` means you see INFO, WARN, and ERROR messages, but not DEBUG or TRACE.

Application Properties

```properties
# Custom application name
spring.application.name=bookstore-api

# Jackson JSON settings
spring.jackson.date-format=yyyy-MM-dd HH:mm:ss
spring.jackson.time-zone=UTC
```

8.3 Reading Properties with @Value

The `@Value` annotation injects a property value into a field in your bean.

```java
package com.example.demo.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

@Service
public class AppInfoService {
```

```
    @Value("${app.name}")
    private String appName;

    @Value("${app.version}")
    private String appVersion;

    @Value("${app.max-users:100}")
    private int maxUsers;

    public String getAppInfo() {
        return appName + " v" + appVersion +
               " (max users: " + maxUsers + ")";
    }
}
```

Line-by-line explanation:

- `@Value("${app.name}")` -- Tells Spring: "Find the property called `app.name` and put its value into this field." The `${}` syntax means "look up this property."
- `@Value("${app.version}")` -- Same thing for the version property.
- `@Value("${app.max-users:100}")` -- The `:100` part is a default value. If `app.max-users` is not defined anywhere, Spring uses `100` instead. This prevents your application from crashing when a property is missing.

Properties file:

```properties
# src/main/resources/application.properties
app.name=BookStore
app.version=2.1.0
# app.max-users is not set, so the default value 100 will be used
```

Output:

```
BookStore v2.1.0 (max users: 100)
```

@Value with Different Types

Spring automatically converts property strings to the right Java type.

```java
@Value("${server.port}")
private int port;              // Converts "8081" to int 8081
```

```
@Value("${app.debug:false}")
private boolean debugMode;     // Converts "true"/"false" to boolean

@Value("${app.tags:}")
private String tags;           // Empty string as default

@Value("${app.rate:0.05}")
private double taxRate;        // Converts to double
```

@Value Limitations

`@Value` is fine for simple values. But if you have many related properties, it becomes messy:

```java
// This gets messy with many properties
@Value("${mail.host}")
private String mailHost;

@Value("${mail.port}")
private int mailPort;

@Value("${mail.username}")
private String mailUsername;

@Value("${mail.password}")
private String mailPassword;

@Value("${mail.from}")
private String mailFrom;
```

That is five annotations for five related properties. There is a better way.

8.4 Grouping Properties with @ConfigurationProperties

`@ConfigurationProperties` binds a group of related properties to a Java class. It is cleaner than using multiple `@Value` annotations.

Step 1: Define Your Properties

```properties
# src/main/resources/application.properties
mail.host=smtp.example.com
mail.port=587
mail.username=noreply@example.com
mail.password=secret123
mail.from=BookStore <noreply@example.com>
mail.enabled=true
```

Step 2: Create a Properties Class

```java
package com.example.demo.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Component
@ConfigurationProperties(prefix = "mail")
public class MailProperties {

    private String host;
    private int port;
    private String username;
    private String password;
    private String from;
    private boolean enabled;

    // Getters and setters (required for binding)
    public String getHost() { return host; }
    public void setHost(String host) { this.host = host; }

    public int getPort() { return port; }
    public void setPort(int port) { this.port = port; }

    public String getUsername() { return username; }
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() { return password; }
    public void setPassword(String password) {
        this.password = password;
    }

    public String getFrom() { return from; }
    public void setFrom(String from) { this.from = from; }

    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) {
        this.enabled = enabled;
    }
```

```
    @Override
    public String toString() {
        return "MailProperties{" +
                "host='" + host + '\'' +
                ", port=" + port +
                ", username='" + username + '\'' +
                ", from='" + from + '\'' +
                ", enabled=" + enabled +
                '}';
    }
}
```

Line-by-line explanation:

- `@ConfigurationProperties(prefix = "mail")` -- Tells Spring: "Take every property that starts with `mail.` and map it to fields in this class." The property `mail.host` maps to the `host` field. The property `mail.port` maps to the `port` field.
- `@Component` -- Makes this class a Spring Bean so it can be injected into other beans.
- Getter and setter methods are required. Spring uses the setters to put the property values into the fields.

Step 3: Use the Properties Class

```java
package com.example.demo.service;

import com.example.demo.config.MailProperties;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    private static final Logger log =
            LoggerFactory.getLogger(EmailService.class);

    private final MailProperties mailProperties;

    public EmailService(MailProperties mailProperties) {
        this.mailProperties = mailProperties;
    }

    public void sendEmail(String to, String subject, String body) {
        if (!mailProperties.isEnabled()) {
            log.info("Email sending is disabled. Skipping.");
            return;
        }
```

```
        log.info("Connecting to {} on port {}",
                mailProperties.getHost(),
                mailProperties.getPort());
        log.info("From: {}", mailProperties.getFrom());
        log.info("To: {}", to);
        log.info("Subject: {}", subject);
        log.info("Email sent successfully!");
    }
}
```

Much cleaner. One injection instead of five separate `@Value` fields.

Nested Properties

You can nest properties using inner classes:

```properties
# application.properties
app.name=BookStore
app.security.enabled=true
app.security.token-expiry=3600
app.security.secret-key=mySecret123
```

```java
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String name;
    private Security security = new Security();

    // Getters and setters for name...
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public Security getSecurity() { return security; }
    public void setSecurity(Security security) {
        this.security = security;
    }

    public static class Security {
        private boolean enabled;
        private int tokenExpiry;
        private String secretKey;

        // Getters and setters
        public boolean isEnabled() { return enabled; }
        public void setEnabled(boolean enabled) {
            this.enabled = enabled;
```

```
        }
        public int getTokenExpiry() { return tokenExpiry; }
        public void setTokenExpiry(int tokenExpiry) {
            this.tokenExpiry = tokenExpiry;
        }
        public String getSecretKey() { return secretKey; }
        public void setSecretKey(String secretKey) {
            this.secretKey = secretKey;
        }
    }
}
```

Notice that `token-expiry` in the properties file maps to `tokenExpiry` in Java. Spring Boot automatically converts kebab-case (words-with-dashes) to camelCase (wordsWithCapitals).

@Value vs @ConfigurationProperties

Feature	@Value	@ConfigurationProperties
Number of properties	One at a time	Group of related ones
Type safety	Basic	Full
Validation support	No	Yes (with @Validated)
Default values	Yes (:default)	Yes (field initializers)
Nested properties	No	Yes
Best for	1-3 simple values	4+ related properties

8.5 Understanding Profiles

A profile is a named set of configuration that applies only in a specific environment.

Think of profiles like outfits. You wear different clothes for different situations. Business suit for work. Casual clothes at home. Sports gear at the gym. The person is the same. The outfit changes.

```
+-------------------+    +-------------------+    +-------------------+
|   Development     |    |     Testing       |    |    Production     |
|   (dev profile)   |    |   (test profile)  |    |  (prod profile)   |
```

```
+-------------------+    +-------------------+    +-------------------+
| H2 in-memory DB   |    | H2 file-based DB  |    | PostgreSQL DB     |
| Debug logging      |    | Warn logging      |    | Error logging     |
| Port 8080          |    | Port 9090         |    | Port 80           |
| No email sending   |    | Mock email        |    | Real email        |
+-------------------+    +-------------------+    +-------------------+

Same application. Different configurations.
```

Why You Need Profiles

Without profiles, you would have to:

1. Change the database URL before every deployment
2. Change the logging level manually
3. Remember to turn off debug features in production
4. Risk deploying with wrong settings

Profiles automate all of this. You set the profile once, and Spring loads the right configuration.

8.6 Creating Profile-Specific Files

Spring Boot looks for configuration files with a specific naming pattern.

File Naming Convention

```
application.properties            <-- Default (always loaded)
application-dev.properties        <-- Loaded when "dev" profile is active
application-test.properties       <-- Loaded when "test" profile is active
application-prod.properties       <-- Loaded when "prod" profile is active
```

The pattern is: `application-{profile}.properties`

Example: Three Environments

Default configuration (always loaded):

```properties
# src/main/resources/application.properties
app.name=BookStore
app.version=1.0.0
spring.application.name=bookstore
```

Development profile:

```properties
# src/main/resources/application-dev.properties
server.port=8080

# H2 in-memory database for quick development
spring.datasource.url=jdbc:h2:mem:devdb
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true

# Show all SQL queries
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Detailed logging
logging.level.com.example.demo=DEBUG
logging.level.org.springframework=INFO

# Custom app settings
app.email.enabled=false
app.debug-mode=true
```

Test profile:

```properties
# src/main/resources/application-test.properties
server.port=9090

# H2 file-based database for persistent test data
spring.datasource.url=jdbc:h2:file:./testdb
spring.datasource.username=sa
spring.datasource.password=

# Moderate logging
logging.level.com.example.demo=INFO
logging.level.org.springframework=WARN

# Custom app settings
app.email.enabled=false
app.debug-mode=false
```

Production profile:

```properties
# src/main/resources/application-prod.properties
server.port=80

# Real database
spring.datasource.url=jdbc:postgresql://db-server:5432/bookstore
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

# Minimal logging
logging.level.root=WARN
logging.level.com.example.demo=INFO

# Custom app settings
app.email.enabled=true
app.debug-mode=false
```

Notice `${DB_USERNAME}` and `${DB_PASSWORD}` in the production file. These are environment variable placeholders. Spring replaces them with values from the operating system environment. This way, passwords never appear in your configuration files.

Environment variable means a value stored in your operating system, outside your application. It is like a sticky note on your computer's desktop. Your application can read it, but it is not part of your code.

File Structure

```
src/main/resources/
├── application.properties          <-- Common settings
├── application-dev.properties      <-- Dev overrides
├── application-test.properties     <-- Test overrides
└── application-prod.properties     <-- Prod overrides
```

8.7 Activating Profiles

You have created profile-specific files. Now how do you tell Spring which profile to use?

Method 1: In application.properties

```properties
# src/main/resources/application.properties
spring.profiles.active=dev
```

This is the simplest way. But it means you have to change this line before deploying.

Method 2: Command Line Argument (Recommended for Deployment)

```bash
java -jar bookstore.jar --spring.profiles.active=prod
```

This overrides whatever is in `application.properties`. It is the most common way to set profiles in production.

Method 3: Environment Variable

```bash
export SPRING_PROFILES_ACTIVE=prod
java -jar bookstore.jar
```

Spring Boot automatically reads the `SPRING_PROFILES_ACTIVE` environment variable.

Method 4: In Your IDE

In IntelliJ IDEA:

1. Open Run Configuration
2. Find "Active profiles" field
3. Type `dev`

In VS Code (launch.json):

```json
{
    "env": {
        "SPRING_PROFILES_ACTIVE": "dev"
    }
}
```

Method 5: In Maven (for Tests)

```xml
<!-- pom.xml -->
<properties>
    <spring.profiles.active>test</spring.profiles.active>
</properties>
```

Multiple Active Profiles

You can activate more than one profile:

```bash
java -jar bookstore.jar --spring.profiles.active=prod,email,metrics
```

Spring loads all three profile files:

- `application-prod.properties`
- `application-email.properties`
- `application-metrics.properties`

Verifying the Active Profile

When your application starts, Spring prints the active profiles in the console:

```
The following 1 profile is active: "dev"
```

You can also check it programmatically:

```java
package com.example.demo.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Service;

import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class ProfileInfoService {

    private static final Logger log =
            LoggerFactory.getLogger(ProfileInfoService.class);

    private final Environment environment;

    public ProfileInfoService(Environment environment) {
        this.environment = environment;
    }

    @PostConstruct
    public void logActiveProfiles() {
        String[] profiles = environment.getActiveProfiles();
```

```
        if (profiles.length == 0) {
            log.info("No active profiles. Using default configuration.");
        } else {
            log.info("Active profiles: {}",
                    String.join(", ", profiles));
        }
    }
}
```

Output with dev profile:

```
Active profiles: dev
```

8.8 Property Resolution Order

What happens when the same property is defined in multiple places? Spring has a clear order of priority. Higher priority wins.

```
Priority (highest to lowest):
──────────────────────────────────────────────────────────
1. Command line arguments
   --server.port=9999

2. Environment variables
   SERVER_PORT=9999

3. Profile-specific properties file
   application-dev.properties

4. Default properties file
   application.properties

5. @PropertySource annotations

6. Default values in @Value
   @Value("${server.port:8080}")
──────────────────────────────────────────────────────────
```

Example: Resolution in Action

```properties
# application.properties
server.port=8080

# application-dev.properties
server.port=8081
```

```bash
# Running with dev profile and command line override
java -jar app.jar --spring.profiles.active=dev --server.port=9999
```

What port does the application use?

```
Step 1: application.properties says 8080
Step 2: application-dev.properties overrides to 8081
Step 3: Command line overrides to 9999

Result: Port 9999 (command line wins)
```

Environment Variable Naming

Spring Boot maps properties to environment variables using this pattern:

```
Property name:       spring.datasource.url
Environment variable: SPRING_DATASOURCE_URL

Rules:
- Replace dots (.) with underscores (_)
- Replace dashes (-) with underscores (_)
- Convert to UPPERCASE
```

More examples:

```
server.port                  --> SERVER_PORT
app.max-users                --> APP_MAX_USERS
spring.jpa.show-sql          --> SPRING_JPA_SHOW_SQL
mail.smtp.auth               --> MAIL_SMTP_AUTH
```

8.9 Profile-Specific Beans

You can create beans that only exist in certain profiles using `@Profile`.

```java
package com.example.demo.service;

public interface EmailSender {
    void send(String to, String subject, String body);
}
```

```java
package com.example.demo.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Service;

@Service
@Profile("dev")
public class MockEmailSender implements EmailSender {

    private static final Logger log =
            LoggerFactory.getLogger(MockEmailSender.class);

    @Override
    public void send(String to, String subject, String body) {
        log.info("[MOCK] Email to: {} | Subject: {} | Body: {}",
                to, subject, body);
        // Does not actually send an email
    }
}
```

```java
package com.example.demo.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Service;

@Service
@Profile("prod")
public class SmtpEmailSender implements EmailSender {

    private static final Logger log =
            LoggerFactory.getLogger(SmtpEmailSender.class);

    @Override
    public void send(String to, String subject, String body) {
        log.info("Sending real email to: {}", to);
        // Real SMTP logic here
    }
}
```

```
Profile = dev  --> MockEmailSender is created (logs but does not send)
Profile = prod --> SmtpEmailSender is created (sends real email)
```

This is powerful. In development, you never accidentally send real emails. In production, you use the real email service. The code that uses `EmailSender` does not know or care which implementation is active.

8.10 Putting It All Together

Let us create a complete example with properties, profiles, and configuration properties.

Properties Files

```properties
# src/main/resources/application.properties
spring.application.name=bookstore
app.name=BookStore
app.version=2.0.0
app.support-email=support@bookstore.com
```

```properties
# src/main/resources/application-dev.properties
server.port=8080
spring.datasource.url=jdbc:h2:mem:devdb
spring.h2.console.enabled=true
spring.jpa.show-sql=true
logging.level.com.example.demo=DEBUG
app.environment=Development
app.email.enabled=false
```

```properties
# src/main/resources/application-prod.properties
server.port=80
spring.datasource.url=jdbc:postgresql://localhost:5432/bookstore
spring.jpa.show-sql=false
logging.level.com.example.demo=INFO
app.environment=Production
app.email.enabled=true
```

Configuration Properties Class

```java
package com.example.demo.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String name;
    private String version;
    private String supportEmail;
    private String environment;
    private Email email = new Email();

    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getVersion() { return version; }
    public void setVersion(String version) { this.version = version; }

    public String getSupportEmail() { return supportEmail; }
    public void setSupportEmail(String supportEmail) {
        this.supportEmail = supportEmail;
    }

    public String getEnvironment() { return environment; }
    public void setEnvironment(String environment) {
        this.environment = environment;
    }

    public Email getEmail() { return email; }
    public void setEmail(Email email) { this.email = email; }

    public static class Email {
        private boolean enabled;

        public boolean isEnabled() { return enabled; }
        public void setEnabled(boolean enabled) {
            this.enabled = enabled;
        }
    }
}
```

Info Endpoint

```java
package com.example.demo.controller;

import com.example.demo.config.AppProperties;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
public class AppInfoController {

    private final AppProperties appProperties;

    public AppInfoController(AppProperties appProperties) {
        this.appProperties = appProperties;
    }

    @GetMapping("/info")
    public Map<String, Object> getInfo() {
        return Map.of(
            "name", appProperties.getName(),
            "version", appProperties.getVersion(),
            "environment", appProperties.getEnvironment(),
            "support", appProperties.getSupportEmail(),
            "emailEnabled", appProperties.getEmail().isEnabled()
        );
    }
}
```

Output when running with dev profile:

```json
{
    "name": "BookStore",
    "version": "2.0.0",
    "environment": "Development",
    "support": "support@bookstore.com",
    "emailEnabled": false
}
```

Output when running with prod profile:

```json
{
    "name": "BookStore",
    "version": "2.0.0",
    "environment": "Production",
    "support": "support@bookstore.com",
    "emailEnabled": true
}
```

Same code. Different output. That is the power of profiles.

Common Mistakes

Mistake 1: YAML Indentation Errors

```yaml
# WRONG: Inconsistent indentation
server:
  port: 8080
    servlet:    # <-- Wrong! Extra indent
      context-path: /api
```

```yaml
# CORRECT: Consistent two-space indentation
server:
  port: 8080
  servlet:
    context-path: /api
```

YAML is very strict about indentation. Always use spaces, never tabs. Use consistent indentation (two spaces is standard).

Mistake 2: Missing Setter Methods with @ConfigurationProperties

```java
// WRONG: No setters -- Spring cannot set the values
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    // Missing setName() method!
}
```

```java
// CORRECT: Include setters
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Mistake 3: Wrong Property Name Format

```properties
# WRONG: Using camelCase in properties
app.maxUsers=100
app.supportEmail=help@example.com

# CORRECT: Using kebab-case (recommended by Spring Boot)
app.max-users=100
app.support-email=help@example.com
```

Spring Boot recommends kebab-case for property names. It automatically maps `app.support-email` to `supportEmail` in Java.

Mistake 4: Forgetting to Set an Active Profile

If you do not set a profile, Spring uses only `application.properties`. Your profile-specific settings will not apply.

Always set a profile:

- In `application.properties`: `spring.profiles.active=dev`
- On the command line: `--spring.profiles.active=dev`
- As an environment variable: `SPRING_PROFILES_ACTIVE=dev`

Mistake 5: Putting Secrets in Properties Files

```properties
# WRONG: Passwords in source code
spring.datasource.password=MySecretPassword123!
api.key=sk-abc123def456
```

```properties
# CORRECT: Use environment variables
spring.datasource.password=${DB_PASSWORD}
api.key=${API_KEY}
```

Never commit passwords, API keys, or secrets to your code repository. Use environment variables instead.

Best Practices

1. Use kebab-case for property names: `app.max-retries` instead of `app.maxRetries`.

2. Use `@ConfigurationProperties` when you have four or more related properties. It is cleaner and supports validation.
3. Always set a default profile in `application.properties` with `spring.profiles.active=dev` for local development.
4. Keep secrets out of property files. Use environment variables for passwords, API keys, and tokens.
5. Use the same format everywhere. Pick either `.properties` or `.yml` and use it consistently across all environments.
6. Put common settings in `application.properties` and only overrides in profile-specific files. Do not duplicate settings.
7. Name your profiles clearly. Use `dev`, `test`, `prod`. Avoid vague names like `profile1` or `local2`.
8. Document custom properties. Add comments explaining what each custom property does and what values are acceptable.

Quick Summary

- Spring Boot reads configuration from `application.properties` or `application.yml`.
- `.properties` uses key=value format. `.yml` uses indentation-based hierarchy.
- `@Value("${property.name}")` injects a single property value. Use `:default` for fallback values.
- `@ConfigurationProperties(prefix = "group")` maps a group of properties to a Java class.
- Profiles let you have different settings for different environments (dev, test, prod).
- Profile-specific files follow the pattern `application-{profile}.properties`.
- Activate profiles via command line, environment variable, or properties file.
- Command line arguments have the highest priority. Default `@Value` values have the lowest.

Key Points

- Configuration should change between environments. Code should not.
- Use `@ConfigurationProperties` over `@Value` for groups of related settings.
- Never put secrets in property files. Use environment variables.

- Always know which profile is active. Check the startup log.
- Properties follow a strict resolution order. Command line always wins.
- YAML requires careful indentation. Properties files are more forgiving.

Practice Questions

1. What is the difference between `application.properties` and `application-dev.properties`? When does each file get loaded?
2. You have a property `app.retry-count=3` in `application.properties` and `app.retry-count=5` in `application-prod.properties`. You run the app with `--spring.profiles.active=prod --app.retry-count=10`. What value does `app.retry-count` have? Why?
3. Explain the difference between `@Value` and `@ConfigurationProperties`. When would you use each?
4. How does Spring Boot convert a property name like `app.max-retry-count` to a Java field name?
5. You want to use a real email service in production but a mock in development. How would you achieve this using profiles?

Exercises

Exercise 1: Multi-Profile Application

Create a Spring Boot application with:

- Three profiles: `dev`, `staging`, `prod`
- A `@ConfigurationProperties` class for settings: `app.name`, `app.environment`, `app.max-connections`, `app.cache.enabled`, `app.cache.ttl-seconds`
- An endpoint `GET /config` that returns all the current configuration values
- Different values for each profile

Test by switching profiles and hitting the endpoint.

Exercise 2: External Configuration

Create an application that reads database connection properties using `@ConfigurationProperties`. Configure it to work with:

- H2 in-memory database for `dev` profile
- H2 file-based database for `test` profile
- Environment variable placeholders for `prod` profile

Log the active profile and database URL at startup using `@PostConstruct`.

Exercise 3: Feature Flags

Create a feature flag system using profiles and properties:

- Define properties: `feature.dark-mode`, `feature.beta-search`, `feature.new-checkout`
- Create a `FeatureFlags` class with `@ConfigurationProperties`
- Create a `GET /features` endpoint that returns which features are enabled
- Enable different features in different profiles

What Is Next?

You now know how to configure your Spring Boot application for different environments. But an application without a web interface is not very useful. In the next chapter, you will build your first REST controller. You will create endpoints that accept HTTP requests and return JSON responses. This is where your application starts talking to the outside world.

Chapter 9: Your First REST Controller

What You Will Learn

In this chapter, you will learn:

- What REST is and how it works (explained with a restaurant analogy)
- The five main HTTP methods and when to use each
- How to create a REST controller with `@RestController`
- How to map URLs to methods with `@GetMapping` and `@PostMapping`
- How Spring Boot automatically converts Java objects to JSON
- How to test your endpoints with `curl`
- How to build a complete Book API with GET and POST endpoints

Why This Chapter Matters

Your Spring Boot application is like a kitchen. It can cook amazing meals. But without a waiter to take orders and deliver food, nobody can eat. The kitchen is useless without the front-of-house staff.

A REST controller is that waiter. It stands between the outside world (web browsers, mobile apps, other services) and your application logic. It takes requests, passes them to the right service, and returns responses.

Almost every modern application needs a REST API. Mobile apps talk to REST APIs. Single-page web applications talk to REST APIs. Microservices talk to each other through REST APIs. Learning to build REST controllers is one of the most practical skills in software development.

9.1 What Is REST?

REST stands for REpresentational State Transfer. That name is confusing. Let us use a simpler explanation.

REST is a set of rules for how computers talk to each other over the internet. It uses the same HTTP protocol that your web browser uses to load web pages. But instead of returning HTML pages, REST returns data (usually in JSON format).

The Restaurant Analogy

Think of a REST API like a restaurant:

```
+------------------+     +------------------+     +------------------+
|    Customer      |     |     Waiter       |     |     Kitchen      |
|  (Client App)    |     |  (REST API)      |     |  (Your Code)     |
+------------------+     +------------------+     +------------------+
        |                        |                        |
        |  "I want the menu"     |                        |
        |  GET /menu             |                        |
        |----------------------->|                        |
        |                        |  "Get all dishes"      |
        |                        |----------------------->|
        |                        |                        |
        |                        |  [list of dishes]      |
        |                        |<-----------------------|
        |  [menu as JSON]        |                        |
        |<-----------------------|                        |
        |                        |                        |
        |  "I want to order"     |                        |
        |  POST /orders          |                        |
        |  {dish: "pasta"}       |                        |
        |----------------------->|                        |
        |                        |  "Create new order"    |
        |                        |----------------------->|
        |                        |                        |
        |                        |  order #42 created     |
        |                        |<-----------------------|
        |  {orderId: 42}         |                        |
        |<-----------------------|                        |
```

- The customer (client) does not go into the kitchen. They talk to the waiter.
- The waiter (REST API) takes the order, delivers it to the kitchen, and brings back the result.
- The kitchen (your services and repositories) does the actual work.

The customer does not need to know how the kitchen works. They just need to know what is on the menu (the API endpoints) and how to order (HTTP methods).

What Is JSON?

JSON stands for JavaScript Object Notation. It is a simple text format for representing data. It looks like this:

```json
{
    "id": 1,
    "title": "Spring Boot in Action",
    "author": "Craig Walls",
    "price": 39.99,
    "available": true
}
```

JSON uses curly braces `{}` for objects, square brackets `[]` for lists, and key-value pairs separated by colons. It is the most common format for REST APIs because it is easy for both humans and computers to read.

9.2 HTTP Methods

HTTP defines several methods (also called verbs) that describe what action to take. Think of them as types of orders at a restaurant.

HTTP Method	Purpose	Restaurant Analogy	Example
`GET`	Read data	"Show me the menu"	`GET /books`
`POST`	Create new data	"I want to place an order"	`POST /books`
`PUT`	Update existing data	"Change my order entirely"	`PUT /books/1`
`PATCH`	Partially update data	"Add extra cheese"	`PATCH /books/1`
`DELETE`	Remove data	"Cancel my order"	`DELETE /books/1`

The Two Most Important Methods

For this chapter, we focus on `GET` and `POST`:

GET retrieves data. It is like asking a question. "What books do you have?" A GET request should never change data on the server. You can call it 100 times and nothing changes. This

property is called idempotent.

Idempotent means doing something multiple times has the same effect as doing it once. Reading a book does not change the book. Reading it 10 times still gives you the same book. That is idempotent.

POST creates new data. It is like placing an order. "Here is a new book to add." A POST request changes data on the server. Calling it twice creates two records.

9.3 Creating a REST Controller with @RestController

Let us build our first REST controller step by step.

Step 1: A Simple Hello Endpoint

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```

Line-by-line explanation:

- `@RestController` -- This annotation combines two things: `@Controller` (handles web requests) and `@ResponseBody` (return values are sent directly as the response body, not as view names). It tells Spring: "This class handles HTTP requests and returns data."
- `@GetMapping("/hello")` -- Maps HTTP GET requests to the URL path `/hello` to this method. When someone visits `http://localhost:8080/hello`, Spring calls this method.
- `public String sayHello()` -- A regular method that returns a String. Spring sends this String directly to the client.

Testing with your browser:

Start your application and open `http://localhost:8080/hello` in your browser.

Output:

```
Hello, World!
```

That is it. You just built your first REST endpoint.

How It Works Under the Hood

```
Browser types: http://localhost:8080/hello
         |
         v
+-----------------------------+
| Spring Boot Embedded Server |
| (Tomcat on port 8080)       |
+-----------------------------+
         |
         | "Someone wants GET /hello"
         v
+-----------------------------+
| DispatcherServlet           |
| (Traffic Controller)        |
+-----------------------------+
         |
         | "Which method handles GET /hello?"
         | "Found: HelloController.sayHello()"
         v
+-----------------------------+
| HelloController             |
| sayHello() returns "Hello"  |
+-----------------------------+
         |
         | "Hello, World!"
         v
+-----------------------------+
| Response sent to browser    |
| Status: 200 OK              |
| Body: Hello, World!         |
+-----------------------------+
```

DispatcherServlet is Spring's traffic controller. It receives all incoming requests and routes them to the correct controller method. You never interact with it directly. It works behind the scenes.

9.4 Returning JSON

Returning plain text is fine for simple cases. But real APIs return JSON. The good news: Spring Boot converts Java objects to JSON automatically.

Step 1: Create a Model Class

```java
package com.example.demo.model;

public class Book {

    private Long id;
    private String title;
    private String author;
    private double price;

    public Book() {
    }

    public Book(Long id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

Important: The class needs getter methods. Spring uses them to convert the object to JSON. Without getters, the JSON will be empty `{}`.

The no-argument constructor `public Book() {}` is also needed. Spring uses it when converting JSON back to a Java object (deserialization).

Step 2: Return the Object from a Controller

```java
package com.example.demo.controller;

import com.example.demo.model.Book;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class BookController {

    @GetMapping("/books")
    public List<Book> getAllBooks() {
        return List.of(
            new Book(1L, "Spring Boot in Action", "Craig Walls", 39.99),
            new Book(2L, "Clean Code", "Robert C. Martin", 34.99),
            new Book(3L, "Effective Java", "Joshua Bloch", 44.99)
        );
    }

    @GetMapping("/books/featured")
    public Book getFeaturedBook() {
        return new Book(1L, "Spring Boot in Action",
                        "Craig Walls", 39.99);
    }
}
```

Output for `GET /books`:

```json
[
    {
        "id": 1,
        "title": "Spring Boot in Action",
        "author": "Craig Walls",
        "price": 39.99
    },
    {
        "id": 2,
        "title": "Clean Code",
        "author": "Robert C. Martin",
        "price": 34.99
    },
    {
        "id": 3,
        "title": "Effective Java",
        "author": "Joshua Bloch",
        "price": 44.99
    }
]
```

Output for `GET /books/featured`:

```json
{
    "id": 1,
    "title": "Spring Boot in Action",
    "author": "Craig Walls",
    "price": 39.99
}
```

Spring Boot uses a library called Jackson to convert Java objects to JSON. Jackson looks at the getter methods of your class and creates JSON properties from them. `getTitle()` becomes `"title"` in JSON. `getPrice()` becomes `"price"`.

```
Java Object                          JSON Output
──────────────                       ───────────
book.getId()      = 1         -->    "id": 1
book.getTitle()   = "Clean.." -->    "title": "Clean Code"
book.getAuthor()  = "Robert." -->    "author": "Robert C. Martin"
book.getPrice()   = 34.99     -->    "price": 34.99
```

9.5 The @PostMapping Annotation

`@GetMapping` handles GET requests (reading data). `@PostMapping` handles POST requests (creating data).

```java
package com.example.demo.controller;

import com.example.demo.model.Book;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
public class BookController {

    private final List<Book> books = new ArrayList<>();
    private long nextId = 1;

    @GetMapping("/books")
    public List<Book> getAllBooks() {
        return books;
    }

    @PostMapping("/books")
```

```
    public Book createBook(@RequestBody Book book) {
        book.setId(nextId++);
        books.add(book);
        return book;
    }
}
```

Line-by-line explanation:

- `private final List<Book> books = new ArrayList<>()` -- A simple in-memory list to store books. In a real application, you would use a database.
- `private long nextId = 1` -- A counter to generate unique IDs.
- `@PostMapping("/books")` -- Maps HTTP POST requests to `/books` to this method.
- `@RequestBody Book book` -- Tells Spring: "Take the JSON from the request body and convert it into a `Book` object." This is the opposite of what happens in GET -- instead of Java to JSON, it goes JSON to Java.
- `book.setId(nextId++)` -- Assigns an auto-incrementing ID.
- `books.add(book)` -- Stores the book in the list.
- `return book` -- Returns the created book (with its new ID) as JSON.

How @RequestBody Works

```
Client sends POST request:
──────────────────────────

POST /books
Content-Type: application/json

{
    "title": "Head First Java",
    "author": "Kathy Sierra",
    "price": 29.99
}

Spring's Jackson library converts JSON to Java:
───────────────────────────────────────────────

Book book = new Book();
book.setTitle("Head First Java");
book.setAuthor("Kathy Sierra");
book.setPrice(29.99);

Your method receives the ready-to-use Java object.
```

@RequestBody tells Spring to read the HTTP request body (the JSON data the client sends) and convert it into a Java object. Without this annotation, Spring does not know where to find the data.

9.6 Testing with curl

curl is a command-line tool for making HTTP requests. It comes pre-installed on Mac and Linux. On Windows, use Git Bash or PowerShell.

Testing GET Requests

```bash
# Simple GET request
curl http://localhost:8080/books
```

Output:

```json
[]
```

The list is empty because we have not added any books yet.

```bash
# GET with pretty-printed JSON
curl -s http://localhost:8080/books | python3 -m json.tool
```

The `-s` flag means "silent" (hides progress information). We pipe the output to Python's JSON tool for formatting.

Testing POST Requests

```bash
# Create a new book
curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Spring Boot in Action", "author": "Craig Walls", "price": 39.99}'
```

Breaking down the curl command:

- `curl` -- The command itself.
- `-X POST` -- Use the POST method instead of the default GET.
- `http://localhost:8080/books` -- The URL to send the request to.
- `-H "Content-Type: application/json"` -- A header that tells the server we are sending JSON data.

- `-d '{"title": ...}'` -- The data (request body) to send. This is the JSON that becomes a Book object.

Output:

```json
{
    "id": 1,
    "title": "Spring Boot in Action",
    "author": "Craig Walls",
    "price": 39.99
}
```

Notice the `id` is now `1`. Our server assigned it.

Testing the Full Flow

```bash
# Step 1: Add first book
curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Spring Boot in Action", "author": "Craig Walls", "price": 39.99}'

# Output: {"id":1,"title":"Spring Boot in Action","author":"Craig Walls","price":39.99}

# Step 2: Add second book
curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Clean Code", "author": "Robert C. Martin", "price": 34.99}'

# Output: {"id":2,"title":"Clean Code","author":"Robert C. Martin","price":34.99}

# Step 3: Get all books
curl http://localhost:8080/books

# Output: [{"id":1,"title":"Spring Boot in Action",...},{"id":2,"title":"Clean Code",...}]
```

9.7 Building a Complete Book API

Let us put everything together into a proper Book API with a service layer.

Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── model/
│   └── Book.java
├── repository/
│   └── BookRepository.java
├── service/
│   └── BookService.java
└── controller/
    └── BookController.java
```

The Model

```java
package com.example.demo.model;

public class Book {

    private Long id;
    private String title;
    private String author;
    private double price;
    private String isbn;

    public Book() {
    }

    public Book(String title, String author, double price, String isbn) {
        this.title = title;
        this.author = author;
        this.price = price;
        this.isbn = isbn;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    @Override
```

```
    public String toString() {
        return "Book{id=" + id + ", title='" + title + "', " +
               "author='" + author + "'}";
    }
}
```

The Repository

```java
package com.example.demo.repository;

import com.example.demo.model.Book;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class BookRepository {

    private final List<Book> books = new ArrayList<>();
    private long nextId = 1;

    public Book save(Book book) {
        if (book.getId() == null) {
            book.setId(nextId++);
            books.add(book);
        } else {
            // Update existing book
            deleteById(book.getId());
            books.add(book);
        }
        return book;
    }

    public Optional<Book> findById(Long id) {
        return books.stream()
                .filter(book -> book.getId().equals(id))
                .findFirst();
    }

    public List<Book> findAll() {
        return new ArrayList<>(books);
    }

    public boolean deleteById(Long id) {
        return books.removeIf(book -> book.getId().equals(id));
    }

    public boolean existsById(Long id) {
```

```
        return books.stream()
                .anyMatch(book -> book.getId().equals(id));
    }
}
```

The Service

```java
package com.example.demo.service;

import com.example.demo.model.Book;
import com.example.demo.repository.BookRepository;
import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class BookService {

    private static final Logger log =
            LoggerFactory.getLogger(BookService.class);

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @PostConstruct
    public void init() {
        // Pre-load some sample books
        createBook(new Book("Spring Boot in Action",
                            "Craig Walls", 39.99, "978-1617292545"));
        createBook(new Book("Clean Code",
                            "Robert C. Martin", 34.99, "978-0132350884"));
        createBook(new Book("Effective Java",
                            "Joshua Bloch", 44.99, "978-0134685991"));
        log.info("Sample books loaded: {}",
                bookRepository.findAll().size());
    }

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    public Optional<Book> getBookById(Long id) {
        return bookRepository.findById(id);
```

```
    }

    public Book createBook(Book book) {
        Book saved = bookRepository.save(book);
        log.info("Created book: {}", saved);
        return saved;
    }
}
```

The Controller

```java
package com.example.demo.controller;

import com.example.demo.model.Book;
import com.example.demo.service.BookService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping
    public List<Book> getAllBooks() {
        return bookService.getAllBooks();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return bookService.getBookById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        Book created = bookService.createBook(book);
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(created);
    }
}
```

Line-by-line explanation:

- `@RequestMapping("/api/books")` -- Sets a base URL for all endpoints in this controller. All mappings below are relative to `/api/books`.
- `@GetMapping` -- Without a path, this maps to the base URL `/api/books`.
- `@GetMapping("/{id}")` -- The `{id}` is a path variable. It matches any value. `/api/books/1`, `/api/books/42`, etc.
- `@PathVariable Long id` -- Extracts the value from the URL and converts it to a Long. We will explain this in detail in the next chapter.
- `ResponseEntity<Book>` -- A wrapper that lets you control the HTTP status code and headers. We will cover this in detail in the next chapter too.
- `ResponseEntity.status(HttpStatus.CREATED)` -- Returns HTTP status 201 (Created) instead of the default 200 (OK). This is the correct status for creating a new resource.

Testing the Complete API

```bash
# Get all books (pre-loaded sample data)
curl http://localhost:8080/api/books
```

Output:

```json
[
    {
        "id": 1,
        "title": "Spring Boot in Action",
        "author": "Craig Walls",
        "price": 39.99,
        "isbn": "978-1617292545"
    },
    {
        "id": 2,
        "title": "Clean Code",
        "author": "Robert C. Martin",
        "price": 34.99,
        "isbn": "978-0132350884"
    },
    {
        "id": 3,
        "title": "Effective Java",
        "author": "Joshua Bloch",
        "price": 44.99,
        "isbn": "978-0134685991"
    }
]
```

```bash
# Get a specific book
curl http://localhost:8080/api/books/1
```

Output:

```json
{
    "id": 1,
    "title": "Spring Boot in Action",
    "author": "Craig Walls",
    "price": 39.99,
    "isbn": "978-1617292545"
}
```

```bash
# Get a book that does not exist
curl -v http://localhost:8080/api/books/99
```

Output (note the 404 status):

```
< HTTP/1.1 404 Not Found
```

```bash
# Create a new book
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Head First Java", "author": "Kathy Sierra", "price": 29.99, "isbn": "9...
```

Output (note the 201 status):

```json
{
    "id": 4,
    "title": "Head First Java",
    "author": "Kathy Sierra",
    "price": 29.99,
    "isbn": "978-0596009205"
}
```

Request-Response Flow

```
Client: POST /api/books
        Body: {"title": "Head First Java", ...}
         |
         v
```

```
+-----------------------------------+
| BookController.createBook()       |
| @RequestBody converts JSON        |
| to Book object                    |
+-----------------------------------+
         |
         v
+-----------------------------------+
| BookService.createBook()          |
| Logs the creation                 |
+-----------------------------------+
         |
         v
+-----------------------------------+
| BookRepository.save()             |
| Assigns ID, stores in list        |
+-----------------------------------+
         |
         v
+-----------------------------------+
| Response flows back:              |
| Book (id=4) -> JSON -> Client     |
| Status: 201 Created               |
+-----------------------------------+
```

Common Mistakes

Mistake 1: Forgetting @RestController

```java
// WRONG: Missing @RestController
public class BookController {
    @GetMapping("/books")
    public List<Book> getBooks() {
        return List.of();
    }
}
// Result: 404 Not Found (Spring does not know about this class)
```

```java
// CORRECT: Add @RestController
@RestController
public class BookController {
    @GetMapping("/books")
    public List<Book> getBooks() {
        return List.of();
```

```
    }
}
```

Mistake 2: Missing @RequestBody on POST

```java
// WRONG: Missing @RequestBody
@PostMapping("/books")
public Book createBook(Book book) {
    // book will have all null fields!
    return bookService.createBook(book);
}
```

```java
// CORRECT: Add @RequestBody
@PostMapping("/books")
public Book createBook(@RequestBody Book book) {
    return bookService.createBook(book);
}
```

Without `@RequestBody`, Spring does not know to read the JSON from the request body. The Book object will be empty.

Mistake 3: Missing No-Argument Constructor

```java
// WRONG: No default constructor
public class Book {
    private String title;

    public Book(String title) {
        this.title = title;
    }
}
// Jackson cannot deserialize JSON to this class
```

```java
// CORRECT: Add a no-argument constructor
public class Book {
    private String title;

    public Book() {}  // Required for JSON deserialization

    public Book(String title) {
        this.title = title;
    }
}
```

Jackson needs a no-argument constructor to create the object first, then uses setters to fill in the values.

Mistake 4: Missing Getters

```java
// WRONG: No getters
public class Book {
    private String title;

    public void setTitle(String title) {
        this.title = title;
    }
    // Missing getTitle()!
}
// JSON output will be: {}
```

```java
// CORRECT: Include getters
public class Book {
    private String title;

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
}
// JSON output will be: {"title": "..."}
```

Mistake 5: Wrong Content-Type Header in curl

```bash
# WRONG: No Content-Type header
curl -X POST http://localhost:8080/books \
  -d '{"title": "Test"}'
# Result: 415 Unsupported Media Type
```

```bash
# CORRECT: Include Content-Type header
curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Test"}'
```

The server needs to know the data format. The `Content-Type: application/json` header tells it "I am sending JSON."

Best Practices

1. Use a base path with `@RequestMapping`. Put `/api/` in front of your endpoints: `/api/books`, `/api/users`. This separates API endpoints from web pages.
2. Follow RESTful naming conventions. Use nouns, not verbs. Use `/books` not `/getBooks` or `/createBook`. The HTTP method (GET, POST) already describes the action.
3. Use the right HTTP status codes. Return 201 for creation, 404 for not found. Do not always return 200. We will cover status codes in detail in the next chapter.
4. Keep controllers thin. Controllers should only handle request/response logic. Put business logic in services.
5. Use plural nouns for collections. Use `/books` not `/book`. The URL represents a collection of books.
6. Return the created resource. After a POST, return the created object with its assigned ID. This saves the client from making another request.
7. Pre-load sample data for development. Use `@PostConstruct` in a service to load test data when the app starts.

Quick Summary

- REST is a way for programs to communicate over HTTP. It uses standard HTTP methods and returns JSON data.
- GET reads data. POST creates data. They are the two most common HTTP methods.
- `@RestController` marks a class as a REST endpoint handler. It combines `@Controller` and `@ResponseBody`.
- `@GetMapping("/path")` maps GET requests to a method. `@PostMapping("/path")` maps POST requests.
- Spring Boot uses Jackson to automatically convert Java objects to JSON and back.
- `@RequestBody` tells Spring to convert the incoming JSON into a Java object.
- `@RequestMapping` on a class sets a base URL for all endpoints in that controller.
- curl is a command-line tool for testing REST endpoints.

Key Points

- Controllers are the front door of your application. Keep them thin.
- Jackson needs getters for serialization and a no-arg constructor for deserialization.
- Always include `Content-Type: application/json` when sending JSON data.
- Use `@RequestMapping` at the class level for a common base path.
- REST uses nouns for URLs and HTTP methods for actions.
- Return meaningful HTTP status codes, not just 200 for everything.

Practice Questions

1. What is the difference between `@Controller` and `@RestController`? Why do we use `@RestController` for APIs?
2. Explain what `@RequestBody` does. What happens if you forget it on a POST method?
3. A client sends a POST request with `{"name": "Alice", "age": 30}` to your endpoint. Your method parameter is a `Person` object. What must the `Person` class have for this to work?
4. Why should you use `/api/books` instead of `/api/getBooks` for a GET endpoint?
5. What curl command would you use to send a POST request with JSON data to `http://localhost:8080/api/orders`?

Exercises

Exercise 1: Movie API

Create a REST API for movies with:

- `Movie` model with fields: `id`, `title`, `director`, `year`, `genre`
- `GET /api/movies` -- returns all movies
- `GET /api/movies/{id}` -- returns a specific movie
- `POST /api/movies` -- creates a new movie
- Pre-load 3 sample movies using `@PostConstruct`

Test all endpoints with curl.

Exercise 2: Todo API

Create a simple Todo API with:

- `TodoItem` model with fields: `id`, `title`, `completed` (boolean), `createdAt` (LocalDateTime)
- `GET /api/todos` -- returns all todos
- `POST /api/todos` -- creates a new todo (set `completed` to false and `createdAt` to now)
- `GET /api/todos/pending` -- returns only todos where `completed` is false

Exercise 3: Student Grade Tracker

Create an API that manages student grades:

- `Student` model with fields: `id`, `name`, `grade` (String like "A", "B", "C")
- `GET /api/students` -- returns all students
- `POST /api/students` -- creates a new student
- `GET /api/students/honor-roll` -- returns only students with grade "A"

Add a service layer that validates the grade is one of A, B, C, D, or F before saving.

What Is Next?

You have built your first REST controller. You can handle GET and POST requests. But real APIs need more. They need to handle path variables, query parameters, different HTTP status codes, and data transfer objects. In the next chapter, you will learn request and response handling in depth -- everything you need to build a complete CRUD API.

Chapter 10: Request and Response Handling

What You Will Learn

In this chapter, you will learn:

- How to extract values from URLs with `@PathVariable`
- How to read query parameters with `@RequestParam`
- How to accept JSON request bodies with `@RequestBody`
- How to control HTTP responses with `ResponseEntity`
- The meaning of common HTTP status codes
- What DTOs are and why you should use them
- How to build a complete CRUD controller

Why This Chapter Matters

In the previous chapter, you built simple GET and POST endpoints. But real-world APIs are more complex. Users need to search for specific items, filter results, send data in different formats, and receive meaningful responses.

Think of it this way. In the previous chapter, you learned to open the front door of a restaurant and seat guests. In this chapter, you learn to take complex orders, handle special requests, and deliver meals with the right presentation.

This chapter gives you the full toolkit for handling any HTTP request and crafting proper responses. After this chapter, you will be able to build a production-quality REST API.

10.1 Extracting Values from URLs with @PathVariable

A path variable is a value embedded in the URL path itself. It identifies a specific resource.

```
GET /api/books/42
               ^^
               This is the path variable (book ID = 42)
```

Basic Usage

```java
@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping("/{id}")
    public Book getBookById(@PathVariable Long id) {
        return bookService.getBookById(id);
    }
}
```

Line-by-line explanation:

- `@GetMapping("/{id}")` -- The curly braces define a placeholder. Any value in that position will be captured. `/api/books/1`, `/api/books/42`, `/api/books/999` all match.
- `@PathVariable Long id` -- Spring takes the value from the URL and converts it to a `Long`. The parameter name (`id`) must match the placeholder name (`{id}`).

How it works:

```
Request: GET /api/books/42

URL pattern: /api/books/{id}
                         ↓
@PathVariable Long id = 42
```

Multiple Path Variables

You can have more than one path variable in a URL.

```java
@GetMapping("/{bookId}/reviews/{reviewId}")
public Review getReview(@PathVariable Long bookId,
                        @PathVariable Long reviewId) {
    return reviewService.getReview(bookId, reviewId);
}
```

```
Request: GET /api/books/5/reviews/12

URL pattern: /api/books/{bookId}/reviews/{reviewId}
                         ↓                    ↓
@PathVariable Long bookId = 5   Long reviewId = 12
```

Custom Parameter Name

If the method parameter name does not match the placeholder, use the `value` attribute:

```java
@GetMapping("/{book-id}")
public Book getBook(
        @PathVariable("book-id") Long bookId) {
    return bookService.getBookById(bookId);
}
```

Here the URL uses `{book-id}` (with a dash), but the Java variable is `bookId` (camelCase). The `@PathVariable("book-id")` tells Spring which placeholder to use.

10.2 Reading Query Parameters with @RequestParam

A query parameter is a value added to the end of a URL after a question mark `?`. It is used for filtering, searching, sorting, and pagination.

```
GET /api/books?author=Craig&minPrice=20
              ^^^^^^^^^^^^^^^^^^^^^^^
              These are query parameters
```

Basic Usage

```java
@GetMapping("/search")
public List<Book> searchBooks(
        @RequestParam String author) {
    return bookService.findByAuthor(author);
}
```

```
Request: GET /api/books/search?author=Craig

@RequestParam String author = "Craig"
```

Optional Parameters with Default Values

Not all query parameters are required. Use `required = false` or `defaultValue`.

```java
@GetMapping
public List<Book> getAllBooks(
        @RequestParam(required = false) String author,
        @RequestParam(defaultValue = "0") double minPrice,
        @RequestParam(defaultValue = "1000") double maxPrice,
        @RequestParam(defaultValue = "title") String sortBy) {

    List<Book> result = bookService.getAllBooks();

    if (author != null) {
        result = result.stream()
                .filter(b -> b.getAuthor()
                        .toLowerCase()
                        .contains(author.toLowerCase()))
                .toList();
    }

    result = result.stream()
            .filter(b -> b.getPrice() >= minPrice
                      && b.getPrice() <= maxPrice)
            .toList();

    return result;
}
```

Line-by-line explanation:

- `@RequestParam(required = false) String author` -- This parameter is optional. If the client does not provide it, the value is `null`.

- `@RequestParam(defaultValue = "0") double minPrice` -- If the client does not provide `minPrice`, it defaults to `0`.
- `@RequestParam(defaultValue = "title") String sortBy` -- Defaults to `"title"` if not provided.

Example requests:

```bash
# All books (no filters)
curl http://localhost:8080/api/books

# Filter by author
curl "http://localhost:8080/api/books?author=Craig"

# Filter by price range
curl "http://localhost:8080/api/books?minPrice=30&maxPrice=50"

# Multiple filters
curl "http://localhost:8080/api/books?author=Craig&minPrice=20&sortBy=price"
```

> Note: When using curl with query parameters, wrap the URL in quotes. The `&` symbol has special meaning in the shell.

@PathVariable vs @RequestParam

Feature	@PathVariable	@RequestParam
Location in URL	Part of the path	After the `?`
Purpose	Identify a specific item	Filter, search, sort
Required?	Yes (by default)	Configurable
Example	`/books/42`	`/books?author=Craig`

Rule of thumb: Use `@PathVariable` to identify WHAT you want. Use `@RequestParam` to specify HOW you want it.

```
GET /api/books/42         --> "Give me book number 42"     (@PathVariable)
GET /api/books?author=Bob --> "Give me books by Bob"       (@RequestParam)
```

10.3 Accepting JSON with @RequestBody

You learned `@RequestBody` in the previous chapter. Let us go deeper.

@RequestBody tells Spring to read the entire HTTP request body and convert it from JSON into a Java object.

```java
@PostMapping
public Book createBook(@RequestBody Book book) {
    return bookService.createBook(book);
}
```

How the Conversion Works

```
HTTP Request:
────────────────────────────────────────
POST /api/books
Content-Type: application/json

{
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "price": 34.99,
    "isbn": "978-0132350884"
}

Jackson Deserialization:
────────────────────────────────────────
1. Create: Book book = new Book()
2. Set:    book.setTitle("Clean Code")
3. Set:    book.setAuthor("Robert C. Martin")
4. Set:    book.setPrice(34.99)
5. Set:    book.setIsbn("978-0132350884")

Result: A fully populated Book object
```

Deserialization means converting data from a format (like JSON) into a Java object. The opposite -- converting a Java object to JSON -- is called serialization.

Important Rules for @RequestBody

1. The Java class needs a no-argument constructor
2. The class needs setter methods (or public fields) for each JSON property
3. The request must have the `Content-Type: application/json` header
4. JSON property names must match Java field names (or getter/setter names)

Ignoring Unknown Properties

By default, Spring Boot ignores JSON properties that do not match any field in the Java class. This is a safe behavior.

```
// Client sends:
{
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "extraField": "this is ignored"
}
```

The `extraField` is silently ignored. No error occurs.

10.4 Controlling Responses with ResponseEntity

`ResponseEntity` is a wrapper that gives you full control over the HTTP response: the status code, headers, and body.

Why Use ResponseEntity?

Without `ResponseEntity`, Spring always returns status 200 (OK):

```
// Always returns 200 OK, even if something went wrong
@GetMapping("/{id}")
public Book getBook(@PathVariable Long id) {
    return bookService.getBookById(id);  // What if the book is null?
}
```

With `ResponseEntity`, you control the status code:

```
// Returns 200 if found, 404 if not found
@GetMapping("/{id}")
public ResponseEntity<Book> getBook(@PathVariable Long id) {
    return bookService.getBookById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

Common ResponseEntity Patterns

Return 200 OK with a body:

```java
return ResponseEntity.ok(book);
// Same as:
return ResponseEntity.status(HttpStatus.OK).body(book);
```

Return 201 Created:

```java
return ResponseEntity.status(HttpStatus.CREATED).body(createdBook);
```

Return 204 No Content (for deletes):

```java
return ResponseEntity.noContent().build();
```

Return 404 Not Found:

```java
return ResponseEntity.notFound().build();
```

Return 400 Bad Request:

```java
return ResponseEntity.badRequest().body(errorMessage);
```

Adding Custom Headers

```java
@PostMapping
public ResponseEntity<Book> createBook(@RequestBody Book book) {
    Book created = bookService.createBook(book);
    return ResponseEntity
            .status(HttpStatus.CREATED)
            .header("X-Book-Id", String.valueOf(created.getId()))
            .body(created);
}
```

Headers are metadata sent along with the response. They provide extra information like content type, caching rules, and custom data.

10.5 HTTP Status Codes

HTTP status codes are three-digit numbers that tell the client what happened. They are grouped by the first digit.

Status Code Families

```
+-------+------------------+-------------------------------------+
| Range | Category         | Meaning                             |
+-------+------------------+-------------------------------------+
| 1xx   | Informational    | "Hold on..."                        |
| 2xx   | Success          | "Here you go!"                      |
| 3xx   | Redirection      | "Go look over there"                |
| 4xx   | Client Error     | "You made a mistake"                |
| 5xx   | Server Error     | "I made a mistake"                  |
+-------+------------------+-------------------------------------+
```

Most Common Status Codes

Code	Name	When to Use	Spring Constant
200	OK	Successful GET, PUT, PATCH	`HttpStatus.OK`
201	Created	Successful POST (new resource)	`HttpStatus.CREATED`
204	No Content	Successful DELETE (nothing to return)	`HttpStatus.NO_CONTENT`
400	Bad Request	Invalid input from client	`HttpStatus.BAD_REQUEST`
401	Unauthorized	Not logged in	`HttpStatus.UNAUTHORIZED`
403	Forbidden	Logged in but not allowed	`HttpStatus.FORBIDDEN`
404	Not Found	Resource does not exist	`HttpStatus.NOT_FOUND`
409	Conflict	Duplicate data or state conflict	`HttpStatus.CONFLICT`
500	Internal Server Error	Bug or crash on the server	`HttpStatus.INTERNAL_SERVER_ERROR`

A Helpful Way to Remember

Think of status codes like restaurant responses:

- 200: "Here is your food!" (Success)
- 201: "Your order has been placed!" (Created)
- 204: "Your plate has been cleared." (Deleted, nothing to show)
- 400: "Sorry, we do not have that item. Please order something else." (Bad request)
- 404: "We do not have a table number 99." (Not found)
- 500: "The kitchen is on fire. Sorry!" (Server error)

10.6 Data Transfer Objects (DTOs)

A DTO (Data Transfer Object) is a simple object used to transfer data between layers of your application. It separates what the client sees from what your internal model looks like.

Why Use DTOs?

Imagine your `Book` entity has a field called `purchaseCost` -- the wholesale price you paid. You do not want customers to see this. If you return the entity directly, all fields are exposed.

```
Without DTOs:
─────────────
Client sees EVERYTHING:
{
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "price": 34.99,
    "purchaseCost": 12.50,    <-- Client should NOT see this
    "supplierCode": "SUP-123"  <-- Client should NOT see this
}

With DTOs:
──────────
Client sees ONLY what you choose:
{
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "price": 34.99
}
```

Request DTO vs Response DTO

Use separate DTOs for requests (input) and responses (output).

Request DTO (what the client sends):

```java
package com.example.demo.dto;

public class BookCreateRequest {

    private String title;
    private String author;
    private double price;
    private String isbn;

    // Getters and setters
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }
}
```

Notice there is no `id` field. The client does not set the ID -- the server generates it.

Response DTO (what the client receives):

```java
package com.example.demo.dto;

import java.time.LocalDateTime;

public class BookResponse {

    private Long id;
    private String title;
    private String author;
    private double price;
    private String isbn;
    private LocalDateTime createdAt;

    // Constructor from entity
    public BookResponse(Long id, String title, String author,
                        double price, String isbn,
                        LocalDateTime createdAt) {
```

```
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
        this.isbn = isbn;
        this.createdAt = createdAt;
    }

    // Getters (no setters needed -- response is read-only)
    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }
    public String getIsbn() { return isbn; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}
```

Update DTO (for PUT/PATCH requests):

```java
package com.example.demo.dto;

public class BookUpdateRequest {

    private String title;
    private String author;
    private double price;

    // Getters and setters
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

The update DTO does not have `id` (it comes from the URL) or `isbn` (cannot be changed).

Mapping Between Entities and DTOs

Create a helper method or use the service layer to convert between entities and DTOs:

```java
@Service
public class BookService {

    private final BookRepository bookRepository;
```

```
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public BookResponse createBook(BookCreateRequest request) {
        Book book = new Book();
        book.setTitle(request.getTitle());
        book.setAuthor(request.getAuthor());
        book.setPrice(request.getPrice());
        book.setIsbn(request.getIsbn());

        Book saved = bookRepository.save(book);
        return toResponse(saved);
    }

    public BookResponse toResponse(Book book) {
        return new BookResponse(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getPrice(),
                book.getIsbn(),
                book.getCreatedAt()
        );
    }
}
```

DTO Naming Convention

```
+--------------------+-----------------------------------+
| Name Pattern       | Purpose                           |
+--------------------+-----------------------------------+
| BookCreateRequest  | Client sends to create a book     |
| BookUpdateRequest  | Client sends to update a book     |
| BookResponse       | Server sends back to client       |
| BookSummary        | Shortened version for lists       |
+--------------------+-----------------------------------+
```

10.7 Building a Complete CRUD Controller

CRUD stands for Create, Read, Update, Delete. These are the four basic operations for managing data.

Let us build a complete CRUD controller for books.

Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── model/
│   └── Book.java
├── dto/
│   ├── BookCreateRequest.java
│   ├── BookUpdateRequest.java
│   └── BookResponse.java
├── repository/
│   └── BookRepository.java
├── service/
│   └── BookService.java
└── controller/
    └── BookController.java
```

The Model

```java
package com.example.demo.model;

import java.time.LocalDateTime;

public class Book {

    private Long id;
    private String title;
    private String author;
    private double price;
    private String isbn;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    public Book() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
```

```
    public void setPrice(double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }
}
```

The DTOs

```java
package com.example.demo.dto;

public class BookCreateRequest {

    private String title;
    private String author;
    private double price;
    private String isbn;

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }
}
```

```java
package com.example.demo.dto;

public class BookUpdateRequest {

    private String title;
    private String author;
    private double price;
```

```
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

```java
package com.example.demo.dto;

import java.time.LocalDateTime;

public class BookResponse {

    private Long id;
    private String title;
    private String author;
    private double price;
    private String isbn;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    public BookResponse() {}

    public BookResponse(Long id, String title, String author,
                        double price, String isbn,
                        LocalDateTime createdAt,
                        LocalDateTime updatedAt) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
        this.isbn = isbn;
        this.createdAt = createdAt;
        this.updatedAt = updatedAt;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }
    public String getIsbn() { return isbn; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

The Repository

```java
package com.example.demo.repository;

import com.example.demo.model.Book;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class BookRepository {

    private final List<Book> books = new ArrayList<>();
    private long nextId = 1;

    public Book save(Book book) {
        if (book.getId() == null) {
            book.setId(nextId++);
        } else {
            books.removeIf(b -> b.getId().equals(book.getId()));
        }
        books.add(book);
        return book;
    }

    public Optional<Book> findById(Long id) {
        return books.stream()
                .filter(b -> b.getId().equals(id))
                .findFirst();
    }

    public List<Book> findAll() {
        return new ArrayList<>(books);
    }

    public boolean deleteById(Long id) {
        return books.removeIf(b -> b.getId().equals(id));
    }

    public boolean existsById(Long id) {
        return books.stream().anyMatch(b -> b.getId().equals(id));
    }
}
```

The Service

```java
package com.example.demo.service;

import com.example.demo.dto.BookCreateRequest;
import com.example.demo.dto.BookResponse;
```

```
import com.example.demo.dto.BookUpdateRequest;
import com.example.demo.model.Book;
import com.example.demo.repository.BookRepository;
import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Service
public class BookService {

    private static final Logger log =
            LoggerFactory.getLogger(BookService.class);

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @PostConstruct
    public void init() {
        BookCreateRequest book1 = new BookCreateRequest();
        book1.setTitle("Spring Boot in Action");
        book1.setAuthor("Craig Walls");
        book1.setPrice(39.99);
        book1.setIsbn("978-1617292545");
        createBook(book1);

        BookCreateRequest book2 = new BookCreateRequest();
        book2.setTitle("Clean Code");
        book2.setAuthor("Robert C. Martin");
        book2.setPrice(34.99);
        book2.setIsbn("978-0132350884");
        createBook(book2);

        log.info("Loaded {} sample books", bookRepository.findAll().size());
    }

    public List<BookResponse> getAllBooks() {
        return bookRepository.findAll().stream()
                .map(this::toResponse)
                .toList();
    }

    public Optional<BookResponse> getBookById(Long id) {
        return bookRepository.findById(id)
                .map(this::toResponse);
    }

    public BookResponse createBook(BookCreateRequest request) {
```

```
        Book book = new Book();
        book.setTitle(request.getTitle());
        book.setAuthor(request.getAuthor());
        book.setPrice(request.getPrice());
        book.setIsbn(request.getIsbn());

        Book saved = bookRepository.save(book);
        log.info("Created book: {} (ID: {})",
                saved.getTitle(), saved.getId());
        return toResponse(saved);
    }

    public Optional<BookResponse> updateBook(Long id,
                                BookUpdateRequest request) {
        return bookRepository.findById(id)
                .map(existing -> {
                    existing.setTitle(request.getTitle());
                    existing.setAuthor(request.getAuthor());
                    existing.setPrice(request.getPrice());
                    existing.setUpdatedAt(LocalDateTime.now());
                    Book updated = bookRepository.save(existing);
                    log.info("Updated book ID: {}", id);
                    return toResponse(updated);
                });
    }

    public boolean deleteBook(Long id) {
        boolean deleted = bookRepository.deleteById(id);
        if (deleted) {
            log.info("Deleted book ID: {}", id);
        }
        return deleted;
    }

    private BookResponse toResponse(Book book) {
        return new BookResponse(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getPrice(),
                book.getIsbn(),
                book.getCreatedAt(),
                book.getUpdatedAt()
        );
    }
}
```

The Complete CRUD Controller

```java
package com.example.demo.controller;

import com.example.demo.dto.BookCreateRequest;
import com.example.demo.dto.BookResponse;
import com.example.demo.dto.BookUpdateRequest;
import com.example.demo.service.BookService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    // CREATE - POST /api/books
    @PostMapping
    public ResponseEntity<BookResponse> createBook(
            @RequestBody BookCreateRequest request) {
        BookResponse created = bookService.createBook(request);
        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(created);
    }

    // READ ALL - GET /api/books
    @GetMapping
    public ResponseEntity<List<BookResponse>> getAllBooks() {
        List<BookResponse> books = bookService.getAllBooks();
        return ResponseEntity.ok(books);
    }

    // READ ONE - GET /api/books/{id}
    @GetMapping("/{id}")
    public ResponseEntity<BookResponse> getBookById(
            @PathVariable Long id) {
        return bookService.getBookById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // UPDATE - PUT /api/books/{id}
    @PutMapping("/{id}")
    public ResponseEntity<BookResponse> updateBook(
            @PathVariable Long id,
            @RequestBody BookUpdateRequest request) {
        return bookService.updateBook(id, request)
```

```
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // DELETE - DELETE /api/books/{id}
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        if (bookService.deleteBook(id)) {
            return ResponseEntity.noContent().build();
        }
        return ResponseEntity.notFound().build();
    }
}
```

Line-by-line explanation of the CRUD operations:

- CREATE (`@PostMapping`): Takes a `BookCreateRequest` from the request body. Returns 201 Created with the new book.
- READ ALL (`@GetMapping`): Returns all books as a list. Returns 200 OK.
- READ ONE (`@GetMapping("/{id}")`): Returns a single book by ID. Returns 200 OK if found, 404 Not Found if not.
- UPDATE (`@PutMapping("/{id}")`): Takes the ID from the URL and update data from the body. Returns 200 OK if updated, 404 if the book does not exist.
- DELETE (`@DeleteMapping("/{id}")`): Takes the ID from the URL. Returns 204 No Content if deleted, 404 if not found.

CRUD Endpoint Summary

```
+--------+------------------+---------------------+---------+
| Method | URL              | Action              | Status  |
+--------+------------------+---------------------+---------+
| POST   | /api/books       | Create a new book   | 201     |
| GET    | /api/books       | Get all books       | 200     |
| GET    | /api/books/{id}  | Get one book        | 200/404 |
| PUT    | /api/books/{id}  | Update a book       | 200/404 |
| DELETE | /api/books/{id}  | Delete a book       | 204/404 |
+--------+------------------+---------------------+---------+
```

Testing the Complete CRUD API

```bash
# CREATE: Add a new book
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
```

```
  -d '{"title": "Head First Java", "author": "Kathy Sierra", "price": 29.99, "isbn": "9...

# Response: 201 Created
# {"id":3,"title":"Head First Java","author":"Kathy Sierra",
#  "price":29.99,"isbn":"978-0596009205",
#  "createdAt":"2024-01-15T10:30:00","updatedAt":"2024-01-15T10:30:00"}
```

```bash
# READ ALL: Get all books
curl http://localhost:8080/api/books

# Response: 200 OK
# [{"id":1,"title":"Spring Boot in Action",...},
#  {"id":2,"title":"Clean Code",...},
#  {"id":3,"title":"Head First Java",...}]
```

```bash
# READ ONE: Get book by ID
curl http://localhost:8080/api/books/1

# Response: 200 OK
# {"id":1,"title":"Spring Boot in Action","author":"Craig Walls",...}
```

```bash
# UPDATE: Update book price and title
curl -X PUT http://localhost:8080/api/books/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Spring Boot in Action (2nd Edition)", "author": "Craig Walls", "price"...

# Response: 200 OK
# {"id":1,"title":"Spring Boot in Action (2nd Edition)",...,"price":44.99,...}
```

```bash
# DELETE: Remove a book
curl -X DELETE http://localhost:8080/api/books/3

# Response: 204 No Content (empty body)
```

```bash
# Verify deletion
curl http://localhost:8080/api/books/3

# Response: 404 Not Found
```

Common Mistakes

Mistake 1: Wrong HTTP Method

```bash
# WRONG: Using GET to create a resource
curl "http://localhost:8080/api/books?title=Test&author=Bob"
# This reads, it does not create

# CORRECT: Use POST to create
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Test", "author": "Bob"}'
```

Mistake 2: Returning Entity Instead of DTO

```java
// WRONG: Exposes internal fields
@GetMapping("/{id}")
public Book getBook(@PathVariable Long id) {
    return bookRepository.findById(id).orElseThrow();
    // Client sees: purchaseCost, supplierCode, etc.
}
```

```java
// CORRECT: Return a DTO
@GetMapping("/{id}")
public ResponseEntity<BookResponse> getBook(@PathVariable Long id) {
    return bookService.getBookById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

Mistake 3: Always Returning 200

```java
// WRONG: Returns 200 even for creation
@PostMapping
public Book createBook(@RequestBody BookCreateRequest request) {
    return bookService.createBook(request);
    // Status is 200, but it should be 201
}
```

```java
// CORRECT: Return appropriate status codes
@PostMapping
public ResponseEntity<BookResponse> createBook(
        @RequestBody BookCreateRequest request) {
```

```
    BookResponse created = bookService.createBook(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}
```

Mistake 4: Path Variable Name Mismatch

```java
// WRONG: Name mismatch
@GetMapping("/{bookId}")
public Book getBook(@PathVariable Long id) {
    // "bookId" in URL but "id" in parameter -- Spring throws an error
}
```

```java
// CORRECT: Names must match
@GetMapping("/{bookId}")
public Book getBook(@PathVariable Long bookId) {
    return bookService.getBookById(bookId);
}
// OR use explicit naming:
@GetMapping("/{bookId}")
public Book getBook(@PathVariable("bookId") Long id) {
    return bookService.getBookById(id);
}
```

Best Practices

1. Use DTOs. Never expose your internal entities directly. DTOs protect your API from internal changes.
2. Return proper HTTP status codes. 201 for creation, 204 for deletion, 404 for not found. Clients depend on these codes.
3. Use `ResponseEntity` for all endpoints. It gives you consistent control over status codes and headers.
4. Validate path variables. Check that the resource exists before operating on it. Return 404 if it does not.
5. Use `@RequestMapping` at the class level for a common base path. This keeps your method-level mappings short.
6. Make request DTOs separate from response DTOs. They serve different purposes and often have different fields.

7. Keep controllers thin. The controller should only handle HTTP concerns: reading parameters, calling services, and building responses. Business logic belongs in the service layer.
8. Use meaningful URL paths. `/api/books/{id}` is better than `/api/getBook?id=1`.

Quick Summary

- `@PathVariable` extracts values from the URL path (`/books/{id}`). Use it to identify specific resources.
- `@RequestParam` extracts query parameters (`/books?author=Craig`). Use it for filtering and searching.
- `@RequestBody` reads JSON from the request body and converts it to a Java object.
- `ResponseEntity` gives you full control over the HTTP response: status code, headers, and body.
- HTTP status codes tell the client what happened: 200 OK, 201 Created, 204 No Content, 404 Not Found.
- DTOs separate your internal data model from what the API exposes. Use separate DTOs for requests and responses.
- CRUD operations map to HTTP methods: POST (Create), GET (Read), PUT (Update), DELETE (Delete).

Key Points

- Path variables identify resources. Query parameters filter or modify the result.
- Always use `ResponseEntity` to return appropriate HTTP status codes.
- DTOs protect your internal model from being exposed to clients.
- A complete CRUD controller needs five endpoints: Create, Read All, Read One, Update, Delete.
- Keep controllers thin -- delegate business logic to services.
- Match path variable names in the URL template to the method parameter.

Practice Questions

1. What is the difference between `@PathVariable` and `@RequestParam`? Give an example URL for each.
2. Why should you use DTOs instead of returning your entity objects directly?
3. What HTTP status code should you return when: (a) a resource is created, (b) a resource is deleted, (c) a resource is not found?
4. Explain how `ResponseEntity` differs from returning a plain object from a controller method.
5. You need an endpoint where clients can search for books by title and optionally filter by minimum price. What would the URL look like, and which annotations would you use?

Exercises

Exercise 1: Employee CRUD API

Build a complete CRUD API for employees with:

- `Employee` model: `id`, `name`, `email`, `department`, `salary`, `hireDate`
- `EmployeeCreateRequest` DTO (no id, no hireDate)
- `EmployeeResponse` DTO (all fields)
- `EmployeeUpdateRequest` DTO (name, department, salary only)
- All five CRUD endpoints with proper status codes
- A `GET /api/employees/department/{dept}` endpoint that returns employees in a specific department

Exercise 2: Product API with Search

Build a product API with:

- `Product` model: `id`, `name`, `category`, `price`, `inStock` (boolean)
- `GET /api/products` with optional query parameters: `category`, `minPrice`, `maxPrice`, `inStock`
- `POST /api/products` to create
- `PUT /api/products/{id}` to update
- `DELETE /api/products/{id}` to delete
- Use DTOs for all request and response objects

Exercise 3: Convert Chapter 9 Exercises

Take the Movie API or Todo API from Chapter 9 exercises and:

- Add DTOs for request and response
- Use `ResponseEntity` with proper status codes
- Add query parameter search (e.g., search movies by genre or year)
- Add PUT and DELETE endpoints

What Is Next?

You can now handle all kinds of requests and build complete CRUD APIs. But what happens when a client sends invalid data? An empty title? A negative price? An email without an `@` symbol? Your application should reject bad data gracefully. In the next chapter, you will learn input validation -- how to ensure the data your API receives is correct before processing it.

Chapter 11: Input Validation

What You Will Learn

In this chapter, you will learn:

- Why input validation matters and what happens without it
- How to add the `spring-boot-starter-validation` dependency
- How to use `@Valid` to trigger validation
- The most common validation annotations: `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Pattern`, `@Min`, `@Max`
- How to create your own custom validator
- How to validate nested objects
- How to build a complete user registration example with validation

Why This Chapter Matters

Imagine you work at a bank. A customer walks in and hands you a deposit slip. The slip says the amount is "negative five million dollars." Do you process it? Of course not. You check the slip first. The amount must be a positive number. The account number must exist. The signature must match.

Your API is like that bank teller. Clients send data to your endpoints. Some of that data will be wrong -- accidentally or intentionally. Without validation, bad data enters your system and causes bugs, crashes, or security problems.

Validation is your first line of defense. It catches bad data at the door, before it reaches your business logic or database. This chapter teaches you how to build that defense.

11.1 Adding the Validation Dependency

Spring Boot validation is not included by default. You need to add a dependency to your `pom.xml`.

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Dependency means a library that your project needs. Maven downloads it automatically when you add it to `pom.xml`.

This starter brings in Jakarta Bean Validation (formerly Java Bean Validation) and Hibernate Validator (the implementation). Do not confuse Hibernate Validator with Hibernate ORM. They share a name but serve different purposes.

- Hibernate ORM talks to databases.
- Hibernate Validator validates Java objects.

After adding the dependency, rebuild your project:

```bash
mvn clean compile
```

11.2 Your First Validation with @Valid

Let us start with a simple example. We have a `BookCreateRequest` DTO that we want to validate.

Step 1: Add Validation Annotations to the DTO

```java
package com.example.demo.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;
```

```
public class BookCreateRequest {

    @NotBlank(message = "Title is required")
    @Size(min = 1, max = 200, message = "Title must be between 1 and 200 characters")
    private String title;

    @NotBlank(message = "Author is required")
    private String author;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than zero")
    private Double price;

    @NotBlank(message = "ISBN is required")
    private String isbn;

    // Getters and setters
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }
}
```

Line-by-line explanation:

- `@NotBlank(message = "Title is required")` -- The title must not be null, must not be empty, and must not be just whitespace. The `message` is what the user sees when validation fails.
- `@Size(min = 1, max = 200, ...)` -- The title must be between 1 and 200 characters long.
- `@NotNull(message = "Price is required")` -- The price must not be null. We use `Double` (the wrapper class) instead of `double` (the primitive) so it can be null. A primitive `double` is always 0.0, never null.
- `@Positive(message = "Price must be greater than zero")` -- The price must be a positive number (greater than 0).

Step 2: Add @Valid to the Controller

```java
package com.example.demo.controller;

import com.example.demo.dto.BookCreateRequest;
import com.example.demo.dto.BookResponse;
import com.example.demo.service.BookService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    public ResponseEntity<BookResponse> createBook(
            @Valid @RequestBody BookCreateRequest request) {
        BookResponse created = bookService.createBook(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

The key change is `@Valid` before `@RequestBody`. This one annotation activates all the validation rules on `BookCreateRequest`.

@Valid tells Spring: "Before calling this method, check all the validation annotations on this object. If any validation fails, do not call the method. Throw an exception instead."

What Happens When Validation Fails?

```bash
# Send a request with missing fields
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "", "price": -5}'
```

Default Spring Boot error response:

```json
{
    "timestamp": "2024-01-15T10:30:00.000+00:00",
    "status": 400,
    "error": "Bad Request",
```

```
    "path": "/api/books"
}
```

Spring returns a 400 Bad Request status. The default error message is not very helpful for the client. We will fix this in Chapter 12 (Exception Handling).

The Validation Flow

```
Client sends POST request with JSON body
         |
         v
+---------------------------+
| Spring reads JSON and     |
| creates BookCreateRequest |
+---------------------------+
         |
         v
+---------------------------+
| @Valid triggers validation |
| Check @NotBlank on title  | --> FAIL: title is empty
| Check @NotNull on price   | --> PASS
| Check @Positive on price  | --> FAIL: price is -5
| Check @NotBlank on author | --> FAIL: author is null
| Check @NotBlank on isbn   | --> FAIL: isbn is null
+---------------------------+
         |
         | 4 violations found!
         v
+---------------------------+
| Spring throws              |
| MethodArgumentNot          |
| ValidException             |
| Returns 400 Bad Request    |
+---------------------------+
         |
         | Controller method is NEVER called
         v
```

11.3 Common Validation Annotations

Here is a complete reference of the most useful validation annotations.

String Validations

Annotation	What It Checks	Example
`@NotNull`	Not null (but can be empty "")	`@NotNull String name`
`@NotEmpty`	Not null AND not empty ""	`@NotEmpty String name`
`@NotBlank`	Not null, not empty, not just spaces	`@NotBlank String name`
`@Size`	Length within range	`@Size(min=2, max=50)`
`@Email`	Valid email format	`@Email String email`
`@Pattern`	Matches a regular expression	`@Pattern(regexp="[A-Z]+")`

The Difference Between @NotNull, @NotEmpty, and @NotBlank

```
Value          | @NotNull | @NotEmpty | @NotBlank
---------------|----------|-----------|----------
null           |  FAIL    |   FAIL    |   FAIL
""             |  PASS    |   FAIL    |   FAIL
"   "          |  PASS    |   PASS    |   FAIL
"hello"        |  PASS    |   PASS    |   PASS
```

Use `@NotBlank` for most string fields. It is the strictest and catches all forms of "empty" input.

Number Validations

Annotation	What It Checks	Example
`@Min`	Minimum value (inclusive)	`@Min(0) int age`
`@Max`	Maximum value (inclusive)	`@Max(150) int age`
`@Positive`	Greater than 0	`@Positive double price`
`@PositiveOrZero`	Greater than or equal to 0	`@PositiveOrZero int stock`
`@Negative`	Less than 0	`@Negative double discount`
`@Digits`	Specific integer/fraction digits	`@Digits(integer=5, fraction=2)`
`@DecimalMin`	Minimum decimal value	`@DecimalMin("0.01")`

Date Validations

Annotation	What It Checks	Example
`@Past`	Date is in the past	`@Past LocalDate birthDate`
`@PastOrPresent`	Date is today or earlier	`@PastOrPresent LocalDate`
`@Future`	Date is in the future	`@Future LocalDate deadline`
`@FutureOrPresent`	Date is today or later	`@FutureOrPresent LocalDate`

Boolean and Other

Annotation	What It Checks	Example
`@AssertTrue`	Must be true	`@AssertTrue boolean agreed`
`@AssertFalse`	Must be false	`@AssertFalse boolean expired`

11.4 Practical Example: @Email and @Pattern

Validating Email Addresses

```java
@NotBlank(message = "Email is required")
@Email(message = "Email must be a valid email address")
private String email;
```

Valid: `user@example.com`, `alice.bob@company.co.uk` Invalid: `not-an-email`, `@missing.com`, `user@`

Validating with Regular Expressions

`@Pattern` lets you define custom validation rules using regular expressions (regex).

Regular expression is a pattern that describes what a string should look like. For example, `[0-9]{3}-[0-9]{4}` means "three digits, a dash, then four digits" (like a phone number: 555-1234).

```java
@NotBlank(message = "Phone number is required")
@Pattern(
    regexp = "^\\d{3}-\\d{3}-\\d{4}$",
    message = "Phone must be in format XXX-XXX-XXXX"
)
private String phone;
```

Common patterns:

```java
// Only letters and spaces
@Pattern(regexp = "^[a-zA-Z ]+$",
         message = "Name must contain only letters and spaces")
private String name;

// Alphanumeric username, 3-20 characters
@Pattern(regexp = "^[a-zA-Z0-9]{3,20}$",
         message = "Username must be 3-20 alphanumeric characters")
private String username;

// Strong password: at least 8 chars, one upper, one lower, one digit
@Pattern(
    regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).{8,}$",
    message = "Password must be 8+ chars with uppercase, lowercase, and digit"
)
private String password;

// ISBN-13 format
@Pattern(regexp = "^978-\\d{10}$",
         message = "ISBN must be in format 978-XXXXXXXXXX")
private String isbn;
```

11.5 A Complete Validation Example

Let us build a fully validated DTO that shows many annotations working together.

```java
package com.example.demo.dto;

import jakarta.validation.constraints.*;
```

```
public class BookCreateRequest {

    @NotBlank(message = "Title is required")
    @Size(min = 1, max = 200,
          message = "Title must be between 1 and 200 characters")
    private String title;

    @NotBlank(message = "Author is required")
    @Size(min = 2, max = 100,
          message = "Author name must be between 2 and 100 characters")
    private String author;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01",
                message = "Price must be at least $0.01")
    @DecimalMax(value = "9999.99",
                message = "Price cannot exceed $9,999.99")
    private Double price;

    @NotBlank(message = "ISBN is required")
    @Pattern(regexp = "^978-\\d{10}$",
             message = "ISBN must be in format 978-XXXXXXXXXX")
    private String isbn;

    @Size(max = 2000,
          message = "Description cannot exceed 2000 characters")
    private String description;

    @Min(value = 1, message = "Page count must be at least 1")
    @Max(value = 10000, message = "Page count cannot exceed 10,000")
    private Integer pageCount;

    // Getters and setters
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public String getDescription() { return description; }
    public void setDescription(String description) {
        this.description = description;
    }

    public Integer getPageCount() { return pageCount; }
    public void setPageCount(Integer pageCount) {
        this.pageCount = pageCount;
    }
}
```

Testing Validation

```bash
# Valid request
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Spring Boot in Action",
    "author": "Craig Walls",
    "price": 39.99,
    "isbn": "978-1617292545",
    "description": "A great book about Spring Boot",
    "pageCount": 350
  }'

# Response: 201 Created
```

```bash
# Invalid request (multiple errors)
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{
    "title": "",
    "author": "A",
    "price": -5.00,
    "isbn": "invalid",
    "pageCount": -1
  }'

# Response: 400 Bad Request
```

11.6 Custom Validators

Sometimes the built-in annotations are not enough. You need custom validation logic. For example, "the end date must be after the start date" or "this field must be unique."

Step 1: Create the Annotation

```java
package com.example.demo.validation;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
```

```
import java.lang.annotation.*;

@Documented
@Constraint(validatedBy = NoOffensiveWordsValidator.class)
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface NoOffensiveWords {

    String message() default "Field contains offensive language";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}
```

Line-by-line explanation:

- `@Constraint(validatedBy = NoOffensiveWordsValidator.class)` -- Links this annotation to the class that contains the validation logic.
- `@Target({ElementType.FIELD})` -- This annotation can only be placed on fields.
- `@Retention(RetentionPolicy.RUNTIME)` -- The annotation is available at runtime (so Spring can read it).
- `String message() default "..."` -- The default error message. Users can override it.
- `groups()` and `payload()` -- Required by the Bean Validation spec. You can leave them with default values.

Step 2: Create the Validator Class

```java
package com.example.demo.validation;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

import java.util.List;

public class NoOffensiveWordsValidator
        implements ConstraintValidator<NoOffensiveWords, String> {

    private static final List<String> BLOCKED_WORDS =
            List.of("spam", "scam", "fake");

    @Override
    public boolean isValid(String value,
                           ConstraintValidatorContext context) {
        if (value == null) {
            return true;  // Let @NotNull handle null checks
```

```
        }

        String lowerValue = value.toLowerCase();
        for (String word : BLOCKED_WORDS) {
            if (lowerValue.contains(word)) {
                return false;
            }
        }
        return true;
    }
}
```

Line-by-line explanation:

- `implements ConstraintValidator<NoOffensiveWords, String>` -- This validator is linked to the `@NoOffensiveWords` annotation and validates `String` values.
- `isValid(...)` -- The method that performs the validation. Return `true` if valid, `false` if invalid.
- `if (value == null) return true` -- Important convention: a custom validator should not check for null. Let `@NotNull` or `@NotBlank` handle null. This allows optional fields to skip this check when they are null.

Step 3: Use the Custom Annotation

```java
public class BookCreateRequest {

    @NotBlank(message = "Title is required")
    @NoOffensiveWords(message = "Title contains inappropriate language")
    private String title;

    @NotBlank(message = "Author is required")
    @NoOffensiveWords
    private String author;

    // ... other fields
}
```

```bash
# Test with offensive word
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "This is spam content", "author": "Bob", "price": 9.99, "isbn": "978-12...

# Response: 400 Bad Request
# Error: "Title contains inappropriate language"
```

11.7 Validating Nested Objects

When your DTO contains another object, you need to validate the nested object too. Use `@Valid` on the nested field.

Example: Order with Shipping Address

```java
package com.example.demo.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.*;

public class OrderCreateRequest {

    @NotBlank(message = "Customer name is required")
    private String customerName;

    @NotBlank(message = "Customer email is required")
    @Email(message = "Email must be valid")
    private String customerEmail;

    @NotNull(message = "Shipping address is required")
    @Valid  // This tells Spring to validate the Address object too
    private Address shippingAddress;

    // Getters and setters
    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public String getCustomerEmail() { return customerEmail; }
    public void setCustomerEmail(String customerEmail) {
        this.customerEmail = customerEmail;
    }

    public Address getShippingAddress() { return shippingAddress; }
    public void setShippingAddress(Address shippingAddress) {
        this.shippingAddress = shippingAddress;
    }
}
```

```java
package com.example.demo.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
```

```
public class Address {

    @NotBlank(message = "Street is required")
    @Size(max = 200, message = "Street cannot exceed 200 characters")
    private String street;

    @NotBlank(message = "City is required")
    private String city;

    @NotBlank(message = "State is required")
    @Size(min = 2, max = 2, message = "State must be 2 characters")
    private String state;

    @NotBlank(message = "ZIP code is required")
    @Pattern(regexp = "^\\d{5}(-\\d{4})?$",
             message = "ZIP code must be in format XXXXX or XXXXX-XXXX")
    private String zipCode;

    // Getters and setters
    public String getStreet() { return street; }
    public void setStreet(String street) { this.street = street; }

    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }

    public String getState() { return state; }
    public void setState(String state) { this.state = state; }

    public String getZipCode() { return zipCode; }
    public void setZipCode(String zipCode) {
        this.zipCode = zipCode;
    }
}
```

The key annotation is `@Valid` on the `shippingAddress` field. Without it, Spring creates the `Address` object but does not validate its fields. The address could have empty streets, invalid ZIP codes, and Spring would not catch it.

```
Without @Valid on shippingAddress:
──────────────────────────────────
customerName  --> validated (must not be blank)
customerEmail --> validated (must be valid email)
shippingAddress --> only checked for null (@NotNull)
    street     --> NOT validated!
    city       --> NOT validated!
    zipCode    --> NOT validated!

With @Valid on shippingAddress:
───────────────────────────────
customerName  --> validated
customerEmail --> validated
shippingAddress --> checked for null AND all fields validated
```

```
    street     --> validated (must not be blank)
    city       --> validated (must not be blank)
    zipCode    --> validated (must match pattern)
```

Testing Nested Validation

```bash
# Invalid nested object
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -d '{
    "customerName": "Alice Johnson",
    "customerEmail": "alice@example.com",
    "shippingAddress": {
      "street": "",
      "city": "",
      "state": "California",
      "zipCode": "abc"
    }
  }'

# Response: 400 Bad Request
# Errors:
# - "Street is required"
# - "City is required"
# - "State must be 2 characters"
# - "ZIP code must be in format XXXXX or XXXXX-XXXX"
```

11.8 Complete Example: User Registration

Let us build a complete user registration system with thorough validation.

The Registration Request DTO

```java
package com.example.demo.dto;

import jakarta.validation.constraints.*;

public class UserRegistrationRequest {

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 20,
```

```
      message = "Username must be between 3 and 20 characters")
@Pattern(regexp = "^[a-zA-Z0-9_]+$",
         message = "Username can only contain letters, numbers, and underscores")
private String username;

@NotBlank(message = "Email is required")
@Email(message = "Please provide a valid email address")
private String email;

@NotBlank(message = "Password is required")
@Size(min = 8, max = 100,
      message = "Password must be between 8 and 100 characters")
@Pattern(
    regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).+$",
    message = "Password must contain at least one uppercase letter, "
            + "one lowercase letter, and one digit"
)
private String password;

@NotBlank(message = "First name is required")
@Size(min = 1, max = 50,
      message = "First name cannot exceed 50 characters")
private String firstName;

@NotBlank(message = "Last name is required")
@Size(min = 1, max = 50,
      message = "Last name cannot exceed 50 characters")
private String lastName;

@Min(value = 13,
     message = "You must be at least 13 years old to register")
@Max(value = 150,
     message = "Please enter a valid age")
private Integer age;

@Pattern(regexp = "^\\d{3}-\\d{3}-\\d{4}$",
         message = "Phone must be in format XXX-XXX-XXXX")
private String phone;  // Optional -- no @NotBlank

// Getters and setters
public String getUsername() { return username; }
public void setUsername(String username) {
    this.username = username;
}

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPassword() { return password; }
public void setPassword(String password) {
    this.password = password;
}

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) {
```

```
        this.firstName = firstName;
    }

    public String getLastName() { return lastName; }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public Integer getAge() { return age; }
    public void setAge(Integer age) { this.age = age; }

    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }
}
```

The User Response DTO

```java
package com.example.demo.dto;

import java.time.LocalDateTime;

public class UserResponse {

    private Long id;
    private String username;
    private String email;
    private String firstName;
    private String lastName;
    private Integer age;
    private String phone;
    private LocalDateTime registeredAt;

    public UserResponse() {}

    public UserResponse(Long id, String username, String email,
                        String firstName, String lastName,
                        Integer age, String phone,
                        LocalDateTime registeredAt) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.phone = phone;
        this.registeredAt = registeredAt;
    }

    // Getters only (no password!)
    public Long getId() { return id; }
```

```
    public String getUsername() { return username; }
    public String getEmail() { return email; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public Integer getAge() { return age; }
    public String getPhone() { return phone; }
    public LocalDateTime getRegisteredAt() { return registeredAt; }
}
```

Notice the response DTO does not include the password. Never return passwords in API responses.

The Controller

```java
package com.example.demo.controller;

import com.example.demo.dto.UserRegistrationRequest;
import com.example.demo.dto.UserResponse;
import com.example.demo.service.UserService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping("/register")
    public ResponseEntity<UserResponse> register(
            @Valid @RequestBody UserRegistrationRequest request) {
        UserResponse user = userService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }
}
```

The Service

```java
package com.example.demo.service;
```

```
import com.example.demo.dto.UserRegistrationRequest;
import com.example.demo.dto.UserResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class UserService {

    private static final Logger log =
            LoggerFactory.getLogger(UserService.class);

    private final AtomicLong nextId = new AtomicLong(1);

    public UserResponse register(UserRegistrationRequest request) {
        // At this point, all validation has already passed
        Long id = nextId.getAndIncrement();

        log.info("Registering user: {} ({})",
                request.getUsername(), request.getEmail());

        return new UserResponse(
                id,
                request.getUsername(),
                request.getEmail(),
                request.getFirstName(),
                request.getLastName(),
                request.getAge(),
                request.getPhone(),
                LocalDateTime.now()
        );
    }
}
```

Testing the Registration

```bash
# Valid registration
curl -X POST http://localhost:8080/api/users/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "alice_dev",
    "email": "alice@example.com",
    "password": "SecurePass1",
    "firstName": "Alice",
    "lastName": "Johnson",
    "age": 28,
    "phone": "555-123-4567"
  }'
```

Output:

```json
{
    "id": 1,
    "username": "alice_dev",
    "email": "alice@example.com",
    "firstName": "Alice",
    "lastName": "Johnson",
    "age": 28,
    "phone": "555-123-4567",
    "registeredAt": "2024-01-15T10:30:00"
}
```

```bash
# Invalid registration (multiple errors)
curl -X POST http://localhost:8080/api/users/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "a",
    "email": "not-an-email",
    "password": "weak",
    "firstName": "",
    "lastName": "",
    "age": 10,
    "phone": "12345"
  }'
```

This request fails with a 400 status and multiple validation errors.

11.9 Validation on Update Endpoints

Do not forget to validate update requests too.

```java
@PutMapping("/{id}")
public ResponseEntity<UserResponse> updateUser(
        @PathVariable Long id,
        @Valid @RequestBody UserUpdateRequest request) {
    return userService.updateUser(id, request)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

The update DTO may have different validation rules than the create DTO. For example, you might not require the password on update:

```java
public class UserUpdateRequest {

    @NotBlank(message = "First name is required")
    private String firstName;

    @NotBlank(message = "Last name is required")
    private String lastName;

    @Email(message = "Please provide a valid email address")
    private String email;

    // Password is optional on update
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;

    // Getters and setters...
}
```

Common Mistakes

Mistake 1: Forgetting @Valid

```java
// WRONG: No @Valid -- validation annotations are IGNORED
@PostMapping
public ResponseEntity<UserResponse> register(
        @RequestBody UserRegistrationRequest request) {
    // No validation happens! All bad data gets through!
}
```

```java
// CORRECT: Add @Valid to activate validation
@PostMapping
public ResponseEntity<UserResponse> register(
        @Valid @RequestBody UserRegistrationRequest request) {
    // Validation runs before this method is called
}
```

This is the most common mistake. Without `@Valid`, the annotations on your DTO are decorations. They do nothing.

Mistake 2: Using Primitive Types for Optional Fields

```java
// WRONG: Primitive double is never null, defaults to 0.0
@NotNull
private double price;  // @NotNull never fails on primitives

// CORRECT: Use wrapper type Double
@NotNull(message = "Price is required")
@Positive(message = "Price must be positive")
private Double price;  // Can be null, so @NotNull works
```

Mistake 3: Not Validating Nested Objects

```java
// WRONG: Missing @Valid on nested object
@NotNull
private Address address;  // Address fields are NOT validated

// CORRECT: Add @Valid
@NotNull
@Valid
private Address address;  // Address fields ARE validated
```

Mistake 4: Missing the Validation Dependency

If you see this error:

```
jakarta.validation.constraints.NotBlank cannot be resolved
```

You forgot to add the dependency:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Mistake 5: Putting Business Validation in Annotations

```java
// WRONG: This is business logic, not data validation
@UniqueEmail  // Custom annotation that checks the database
private String email;
```

Annotations should validate the format of data (not blank, valid email, correct length). Business rules like "email must be unique" belong in the service layer, not in annotations.

Best Practices

1. Always use `@Valid` on controller method parameters. It is easy to forget and hard to catch.
2. Use `@NotBlank` for strings, not `@NotNull`. `@NotBlank` catches null, empty, and whitespace-only strings.
3. Use wrapper types (`Double`, `Integer`) instead of primitives (`double`, `int`) when the field can be null. Primitives cannot be null, so `@NotNull` never fails on them.
4. Write clear, user-friendly error messages. "Price is required" is better than "must not be null."
5. Keep validation annotations on DTOs, not on entities. DTOs are the API boundary. Entities are the database boundary. They may have different rules.
6. Validate at the API boundary. By the time data reaches your service, it should already be validated. Services should not need to check for null or empty strings.
7. Use separate DTOs for create and update. Create may require fields that update does not (like password).
8. Let `@NotNull` handle null checks in custom validators. Your custom validator's `isValid()` method should return `true` for null values.

Quick Summary

- Add `spring-boot-starter-validation` to your `pom.xml` to enable validation.
- `@Valid` on a method parameter activates validation. Without it, annotations do nothing.
- `@NotBlank` validates strings (not null, not empty, not whitespace). `@NotNull` only checks for null.
- `@Size` validates length. `@Min`/`@Max` validate numeric ranges. `@Email` validates email format.
- `@Pattern` validates against a regular expression for custom formats.
- Custom validators need two parts: an annotation and a validator class.
- Use `@Valid` on nested object fields to validate their contents.
- Use wrapper types (`Double`, `Integer`) instead of primitives when fields can be null.

Key Points

- Validation is your first line of defense against bad data.
- Always add `@Valid` to your controller parameters -- it is the trigger.
- Use `@NotBlank` for strings, wrapper types for numbers.
- Keep error messages clear and user-friendly.
- Validate at the API boundary (DTOs), not deep in your business logic.
- Nested objects need `@Valid` on the field to be validated.

Practice Questions

1. What is the difference between `@NotNull`, `@NotEmpty`, and `@NotBlank`? Which should you use for a user's name field?
2. Why should you use `Double` instead of `double` for a price field that has `@NotNull`?
3. You have an `Order` DTO with a `List<OrderItem> items` field. Each `OrderItem` has validation annotations. What do you need to add to the `items` field to make the validation work?
4. Explain the two parts needed to create a custom validator. What does each part do?
5. What happens if you put validation annotations on a DTO but forget `@Valid` on the controller parameter?

Exercises

Exercise 1: Contact Form Validation

Create a contact form API endpoint `POST /api/contact` with a DTO that validates:

- `name` -- required, 2-100 characters
- `email` -- required, valid email
- `subject` -- required, 5-200 characters
- `message` -- required, 10-5000 characters
- `phone` -- optional, but if provided must match format XXX-XXX-XXXX
- `urgency` -- optional, must be one of: "low", "medium", "high" (use @Pattern)

Exercise 2: Product with Nested Category

Create a `POST /api/products` endpoint with:

- `ProductCreateRequest` with fields: `name` (required, 2-100 chars), `price` (required, positive), `description` (optional, max 1000 chars)
- Nested `Category` object with fields: `name` (required), `code` (required, exactly 3 uppercase letters)
- Make sure the nested Category is validated

Exercise 3: Custom Date Range Validator

Create a custom `@ValidDateRange` annotation that validates an object has `startDate` before `endDate`. Apply it to a class-level annotation on an `EventCreateRequest` DTO with `name`, `startDate`, and `endDate` fields.

Hint: Use `@Target(ElementType.TYPE)` for class-level validation and implement `ConstraintValidator<ValidDateRange, EventCreateRequest>`.

What Is Next?

You now know how to validate input data. But what happens when validation fails? Or when a requested resource does not exist? Or when an unexpected error occurs? The default Spring Boot error responses are ugly and inconsistent. In the next chapter, you will learn exception handling -- how to return clean, consistent, and helpful error responses from your API.

Chapter 12: Exception Handling

What You Will Learn

In this chapter, you will learn:

- Why default Spring Boot error responses are not good enough
- How to create custom exception classes
- How to use `@ControllerAdvice` to handle exceptions globally
- How to use `@ExceptionHandler` to catch specific exceptions
- How to design a clean error response DTO
- How to handle validation errors with proper field-level messages
- How to build a complete, production-quality error handling system

Why This Chapter Matters

Imagine calling a customer service hotline. You explain your problem. The representative says "Error. Goodbye." and hangs up. That is how most APIs respond to errors without proper exception handling.

Good error responses are like a helpful customer service agent. They tell you:

- What went wrong ("The book you requested was not found")
- Why it went wrong ("No book exists with ID 999")
- What you can do about it ("Check the ID and try again")

Without proper exception handling, your API returns ugly stack traces, inconsistent error formats, and generic messages. Clients cannot figure out what they did wrong. Developers cannot debug problems. Your API feels broken even when it is working correctly.

This chapter teaches you to handle every error gracefully and return clean, consistent, helpful responses.

12.1 The Problem with Default Error Responses

Let us look at what Spring Boot returns by default when things go wrong.

Default 404 Response

When a resource is not found, Spring returns:

```json
{
    "timestamp": "2024-01-15T10:30:00.000+00:00",
    "status": 404,
    "error": "Not Found",
    "path": "/api/books/999"
}
```

This is okay but generic. It does not tell the client what was not found.

Default Validation Error Response

When validation fails (from Chapter 11), Spring returns:

```json
{
    "timestamp": "2024-01-15T10:30:00.000+00:00",
    "status": 400,
    "error": "Bad Request",
    "path": "/api/books"
}
```

This is worse. The client knows the request was bad but has no idea which field was wrong or why.

Default 500 Response

When an unexpected exception occurs:

```json
{
    "timestamp": "2024-01-15T10:30:00.000+00:00",
    "status": 500,
    "error": "Internal Server Error",
    "path": "/api/books"
}
```

This is useless to both the client and the developer.

What We Want Instead

We want consistent, informative responses like:

```json
{
    "status": 404,
    "error": "Not Found",
    "message": "Book not found with ID: 999",
    "timestamp": "2024-01-15T10:30:00"
}
```

And for validation errors:

```json
{
    "status": 400,
    "error": "Validation Failed",
    "message": "One or more fields have errors",
    "timestamp": "2024-01-15T10:30:00",
    "fieldErrors": [
        {
            "field": "title",
            "message": "Title is required"
        },
        {
            "field": "price",
            "message": "Price must be greater than zero"
        }
    ]
}
```

Now the client knows exactly what went wrong and how to fix it.

12.2 Creating Custom Exceptions

The first step is creating your own exception classes. These represent specific error conditions in your application.

ResourceNotFoundException

This exception is thrown when a requested resource does not exist.

```java
package com.example.demo.exception;

public class ResourceNotFoundException extends RuntimeException {

    private final String resourceName;
    private final String fieldName;
    private final Object fieldValue;

    public ResourceNotFoundException(String resourceName,
                                     String fieldName,
                                     Object fieldValue) {
        super(String.format("%s not found with %s: '%s'",
                resourceName, fieldName, fieldValue));
        this.resourceName = resourceName;
        this.fieldName = fieldName;
        this.fieldValue = fieldValue;
    }

    public String getResourceName() { return resourceName; }
    public String getFieldName() { return fieldName; }
    public Object getFieldValue() { return fieldValue; }
}
```

Line-by-line explanation:

- `extends RuntimeException` -- Our exception extends `RuntimeException`, which means it is an unchecked exception. You do not need to declare it in method signatures with `throws`.
- `String resourceName` -- The type of resource ("Book", "User", "Order").
- `String fieldName` -- The field used to look up the resource ("id", "isbn", "email").
- `Object fieldValue` -- The value that was not found (42, "978-123", "alice@example.com").
- `super(String.format(...))` -- Creates a human-readable message like "Book not found with id: '999'".

RuntimeException is a Java exception that does not require explicit handling with try-catch. Spring can catch it automatically using `@ExceptionHandler`.

DuplicateResourceException

This exception is thrown when trying to create a resource that already exists.

```java
package com.example.demo.exception;

public class DuplicateResourceException extends RuntimeException {

    private final String resourceName;
```

```
    private final String fieldName;
    private final Object fieldValue;

    public DuplicateResourceException(String resourceName,
                                      String fieldName,
                                      Object fieldValue) {
        super(String.format("%s already exists with %s: '%s'",
                resourceName, fieldName, fieldValue));
        this.resourceName = resourceName;
        this.fieldName = fieldName;
        this.fieldValue = fieldValue;
    }

    public String getResourceName() { return resourceName; }
    public String getFieldName() { return fieldName; }
    public Object getFieldValue() { return fieldValue; }
}
```

Using Custom Exceptions in the Service Layer

```java
package com.example.demo.service;

import com.example.demo.dto.BookCreateRequest;
import com.example.demo.dto.BookResponse;
import com.example.demo.exception.DuplicateResourceException;
import com.example.demo.exception.ResourceNotFoundException;
import com.example.demo.model.Book;
import com.example.demo.repository.BookRepository;
import org.springframework.stereotype.Service;

@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public BookResponse getBookById(Long id) {
        Book book = bookRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException(
                        "Book", "id", id));
        return toResponse(book);
    }

    public BookResponse createBook(BookCreateRequest request) {
        // Check for duplicate ISBN
        if (bookRepository.existsByIsbn(request.getIsbn())) {
            throw new DuplicateResourceException(
```

```
                    "Book", "isbn", request.getIsbn());
        }

        Book book = new Book();
        book.setTitle(request.getTitle());
        book.setAuthor(request.getAuthor());
        book.setPrice(request.getPrice());
        book.setIsbn(request.getIsbn());

        Book saved = bookRepository.save(book);
        return toResponse(saved);
    }

    public BookResponse updateBook(Long id,
                                   BookUpdateRequest request) {
        Book book = bookRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException(
                        "Book", "id", id));

        book.setTitle(request.getTitle());
        book.setAuthor(request.getAuthor());
        book.setPrice(request.getPrice());

        Book updated = bookRepository.save(book);
        return toResponse(updated);
    }

    public void deleteBook(Long id) {
        if (!bookRepository.existsById(id)) {
            throw new ResourceNotFoundException("Book", "id", id);
        }
        bookRepository.deleteById(id);
    }

    private BookResponse toResponse(Book book) {
        return new BookResponse(
                book.getId(), book.getTitle(),
                book.getAuthor(), book.getPrice(),
                book.getIsbn(), book.getCreatedAt(),
                book.getUpdatedAt());
    }
}
```

Notice how the service now throws specific exceptions instead of returning `Optional` or `null`. The controller does not need to check for missing resources. The exception handler takes care of it.

12.3 The Error Response DTO

Before we handle exceptions, let us define what our error responses should look like.

```java
package com.example.demo.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.LocalDateTime;
import java.util.List;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private int status;
    private String error;
    private String message;
    private LocalDateTime timestamp;
    private String path;
    private List<FieldError> fieldErrors;

    public ErrorResponse() {
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponse(int status, String error, String message) {
        this();
        this.status = status;
        this.error = error;
        this.message = message;
    }

    // Getters and setters
    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }

    public String getError() { return error; }
    public void setError(String error) { this.error = error; }

    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }

    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public String getPath() { return path; }
    public void setPath(String path) { this.path = path; }

    public List<FieldError> getFieldErrors() { return fieldErrors; }
    public void setFieldErrors(List<FieldError> fieldErrors) {
        this.fieldErrors = fieldErrors;
    }
```

```
    // Inner class for field-level validation errors
    public static class FieldError {

        private String field;
        private String message;
        private Object rejectedValue;

        public FieldError() {}

        public FieldError(String field, String message,
                          Object rejectedValue) {
            this.field = field;
            this.message = message;
            this.rejectedValue = rejectedValue;
        }

        public String getField() { return field; }
        public void setField(String field) { this.field = field; }

        public String getMessage() { return message; }
        public void setMessage(String message) {
            this.message = message;
        }

        public Object getRejectedValue() { return rejectedValue; }
        public void setRejectedValue(Object rejectedValue) {
            this.rejectedValue = rejectedValue;
        }
    }
}
```

Line-by-line explanation:

- `@JsonInclude(JsonInclude.Include.NON_NULL)` -- This Jackson annotation means "only include fields in the JSON output if they are not null." So if there are no `fieldErrors`, that field is omitted from the response entirely. This keeps responses clean.
- `ErrorResponse` contains the standard error information: status code, error name, message, timestamp, and request path.
- `FieldError` is a nested class for validation errors. It includes the field name, the error message, and the rejected value.
- `rejectedValue` shows the client what value they sent. This is helpful for debugging. "The field 'price' has value '-5' which is invalid."

What the Error Responses Look Like

Simple error (resource not found):

```json
{
    "status": 404,
    "error": "Not Found",
    "message": "Book not found with id: '999'",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books/999"
}
```

Validation error (with field details):

```json
{
    "status": 400,
    "error": "Validation Failed",
    "message": "One or more fields have validation errors",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books",
    "fieldErrors": [
        {
            "field": "title",
            "message": "Title is required",
            "rejectedValue": ""
        },
        {
            "field": "price",
            "message": "Price must be greater than zero",
            "rejectedValue": -5.0
        }
    ]
}
```

12.4 Global Exception Handling with @ControllerAdvice

`@ControllerAdvice` is a special annotation that lets you handle exceptions across all controllers in one place. Without it, you would need to handle exceptions in every controller separately.

@ControllerAdvice tells Spring: "This class contains exception handling logic that applies to all controllers." Think of it as a safety net under all your controllers.

@ExceptionHandler marks a method that handles a specific type of exception. When that exception is thrown anywhere in your controllers, Spring calls this method instead of returning the default error.

The Complete Global Exception Handler

```java
package com.example.demo.exception;

import com.example.demo.dto.ErrorResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;

import java.util.List;

@ControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log =
            LoggerFactory.getLogger(GlobalExceptionHandler.class);

    // Handle resource not found
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFound(
            ResourceNotFoundException ex, WebRequest request) {

        log.warn("Resource not found: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                "Not Found",
                ex.getMessage()
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    // Handle duplicate resource
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse> handleDuplicateResource(
            DuplicateResourceException ex, WebRequest request) {

        log.warn("Duplicate resource: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.CONFLICT.value(),
                "Conflict",
                ex.getMessage()
        );
        error.setPath(getPath(request));
```

```
        return new ResponseEntity<>(error, HttpStatus.CONFLICT);
    }

    // Handle validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationErrors(
            MethodArgumentNotValidException ex,
            WebRequest request) {

        log.warn("Validation failed: {} errors",
                ex.getBindingResult().getErrorCount());

        List<ErrorResponse.FieldError> fieldErrors =
                ex.getBindingResult().getFieldErrors().stream()
                    .map(fieldError -> new ErrorResponse.FieldError(
                            fieldError.getField(),
                            fieldError.getDefaultMessage(),
                            fieldError.getRejectedValue()
                    ))
                    .toList();

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Validation Failed",
                "One or more fields have validation errors"
        );
        error.setPath(getPath(request));
        error.setFieldErrors(fieldErrors);

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    // Handle all other unexpected exceptions
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAllOtherExceptions(
            Exception ex, WebRequest request) {

        log.error("Unexpected error occurred", ex);

        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Internal Server Error",
                "An unexpected error occurred. Please try again later."
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error,
                HttpStatus.INTERNAL_SERVER_ERROR);
    }

    private String getPath(WebRequest request) {
        return request.getDescription(false).replace("uri=", "");
    }
}
```

Line-by-line explanation of each handler:

handleResourceNotFound

- `@ExceptionHandler(ResourceNotFoundException.class)` -- This method catches all `ResourceNotFoundException` instances thrown by any controller.
- `ResourceNotFoundException ex` -- The actual exception object, which contains the error message.
- `WebRequest request` -- Contains information about the HTTP request, including the URL path.
- `log.warn(...)` -- Logs the error at WARN level. Not found errors are not critical but should be logged for monitoring.
- Returns a 404 status with our clean error response.

handleDuplicateResource

- Similar to the above, but returns HTTP 409 (Conflict).
- A 409 Conflict means the request conflicts with the current state of the server. Creating a book with an ISBN that already exists is a conflict.

handleValidationErrors

- `@ExceptionHandler(MethodArgumentNotValidException.class)` -- This exception is thrown by Spring when `@Valid` validation fails.
- `ex.getBindingResult().getFieldErrors()` -- Gets the list of field-level errors from the validation result.
- We convert each field error into our `ErrorResponse.FieldError` format with the field name, message, and rejected value.
- Returns a 400 status with field-level error details.

MethodArgumentNotValidException is the exception Spring throws when `@Valid` detects validation errors. It contains a `BindingResult` which holds all the field errors.

handleAllOtherExceptions

- `@ExceptionHandler(Exception.class)` -- This is the catch-all handler. It catches any exception not handled by the other methods.
- `log.error("Unexpected error occurred", ex)` -- Logs the full stack trace at ERROR level. This is critical for debugging.
- The response message is generic on purpose. You do not want to expose internal error details to the client. Stack traces, class names, and database errors should stay in the logs.

How Exception Handling Flows

```
Controller throws ResourceNotFoundException
         |
         v
+-----------------------------------+
| Spring looks for @ExceptionHandler |
| that matches the exception type    |
+-----------------------------------+
         |
         | Found: handleResourceNotFound()
         v
+-----------------------------------+
| Handler creates ErrorResponse      |
| Sets status, message, path         |
+-----------------------------------+
         |
         v
+-----------------------------------+
| Spring sends the response:         |
| Status: 404                        |
| Body: ErrorResponse as JSON        |
+-----------------------------------+
```

```
No matching specific handler?
         |
         v
+-----------------------------------+
| Falls through to catch-all:        |
| handleAllOtherExceptions()         |
+-----------------------------------+
         |
         v
+-----------------------------------+
| Status: 500                        |
| Body: Generic error message        |
| Stack trace logged to server       |
+-----------------------------------+
```

12.5 Simplified Controllers with Exception Handling

Now that exceptions are handled globally, your controllers become much simpler.

Before (Without Global Exception Handling)

```java
@GetMapping("/{id}")
public ResponseEntity<BookResponse> getBookById(
        @PathVariable Long id) {
    return bookService.getBookById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    // Ugly: 404 with no body, no message
}

@PostMapping
public ResponseEntity<?> createBook(
        @Valid @RequestBody BookCreateRequest request) {
    try {
        BookResponse created = bookService.createBook(request);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(created);
    } catch (DuplicateResourceException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(Map.of("error", e.getMessage()));
        // Inconsistent error format
    }
}
```

After (With Global Exception Handling)

```java
@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    public ResponseEntity<BookResponse> createBook(
            @Valid @RequestBody BookCreateRequest request) {
        BookResponse created = bookService.createBook(request);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(created);
    }

    @GetMapping
    public ResponseEntity<List<BookResponse>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @GetMapping("/{id}")
    public ResponseEntity<BookResponse> getBookById(
```

```
            @PathVariable Long id) {
        BookResponse book = bookService.getBookById(id);
        return ResponseEntity.ok(book);
    }

    @PutMapping("/{id}")
    public ResponseEntity<BookResponse> updateBook(
            @PathVariable Long id,
            @Valid @RequestBody BookUpdateRequest request) {
        BookResponse updated = bookService.updateBook(id, request);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.noContent().build();
    }
}
```

Look how clean this is. No try-catch blocks. No null checks. No Optional handling. The controller focuses on the happy path. The `GlobalExceptionHandler` takes care of all errors.

```
+---------------------------------------------------+
|   Controller (Happy Path Only)                    |
|   - Takes request                                 |
|   - Calls service                                 |
|   - Returns response                              |
+---------------------------------------------------+
         |
         | If exception thrown...
         v
+---------------------------------------------------+
|   GlobalExceptionHandler (All Error Handling)     |
|   - Catches exception                             |
|   - Creates clean ErrorResponse                   |
|   - Returns proper HTTP status                    |
+---------------------------------------------------+
```

12.6 Handling Additional Exception Types

Your application may encounter other types of exceptions. Here are common ones and how to handle them.

Handling Malformed JSON

When a client sends invalid JSON, Spring throws `HttpMessageNotReadableException`:

```java
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponse> handleMalformedJson(
        HttpMessageNotReadableException ex,
        WebRequest request) {

    log.warn("Malformed JSON request: {}", ex.getMessage());

    ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            "Malformed JSON",
            "The request body contains invalid JSON. "
            + "Please check the syntax."
    );
    error.setPath(getPath(request));

    return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
}
```

Test:

```bash
curl -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{invalid json here}'
```

Response:

```json
{
    "status": 400,
    "error": "Malformed JSON",
    "message": "The request body contains invalid JSON. Please check the syntax.",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books"
}
```

Handling Wrong HTTP Method

When a client uses GET on a POST-only endpoint, Spring throws `HttpRequestMethodNotSupportedException`:

```java
@ExceptionHandler(HttpRequestMethodNotSupportedException.class)
public ResponseEntity<ErrorResponse> handleMethodNotSupported(
        HttpRequestMethodNotSupportedException ex,
```

```
        WebRequest request) {

    String message = String.format(
            "HTTP method '%s' is not supported for this endpoint. "
            + "Supported methods: %s",
            ex.getMethod(),
            ex.getSupportedHttpMethods());

    ErrorResponse error = new ErrorResponse(
            HttpStatus.METHOD_NOT_ALLOWED.value(),
            "Method Not Allowed",
            message
    );
    error.setPath(getPath(request));

    return new ResponseEntity<>(error,
            HttpStatus.METHOD_NOT_ALLOWED);
}
```

Handling Missing Path Variables or Parameters

When a required request parameter is missing:

```java
@ExceptionHandler(MissingServletRequestParameterException.class)
public ResponseEntity<ErrorResponse> handleMissingParameter(
        MissingServletRequestParameterException ex,
        WebRequest request) {

    String message = String.format(
            "Required parameter '%s' of type '%s' is missing",
            ex.getParameterName(), ex.getParameterType());

    ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            "Missing Parameter",
            message
    );
    error.setPath(getPath(request));

    return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
}
```

12.7 The Complete Exception Handling System

Here is the full picture of all the pieces working together.

Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── controller/
│   └── BookController.java
├── dto/
│   ├── BookCreateRequest.java
│   ├── BookUpdateRequest.java
│   ├── BookResponse.java
│   └── ErrorResponse.java
├── exception/
│   ├── GlobalExceptionHandler.java
│   ├── ResourceNotFoundException.java
│   └── DuplicateResourceException.java
├── model/
│   └── Book.java
├── repository/
│   └── BookRepository.java
└── service/
    └── BookService.java
```

Complete GlobalExceptionHandler

```java
package com.example.demo.exception;

import com.example.demo.dto.ErrorResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.HttpRequestMethodNotSupportedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.util.List;

@ControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log =
            LoggerFactory.getLogger(GlobalExceptionHandler.class);

    /**
```

```
     * Handle resource not found (404)
     */
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFound(
            ResourceNotFoundException ex, WebRequest request) {

        log.warn("Resource not found: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                "Not Found",
                ex.getMessage()
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    /**
     * Handle duplicate resource (409)
     */
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse> handleDuplicateResource(
            DuplicateResourceException ex, WebRequest request) {

        log.warn("Duplicate resource: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.CONFLICT.value(),
                "Conflict",
                ex.getMessage()
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.CONFLICT);
    }

    /**
     * Handle validation errors (400)
     */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationErrors(
            MethodArgumentNotValidException ex,
            WebRequest request) {

        log.warn("Validation failed: {} error(s)",
                ex.getBindingResult().getErrorCount());

        List<ErrorResponse.FieldError> fieldErrors =
                ex.getBindingResult().getFieldErrors().stream()
                    .map(fe -> new ErrorResponse.FieldError(
                            fe.getField(),
                            fe.getDefaultMessage(),
                            fe.getRejectedValue()
                    ))
```

```
                    .toList();

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Validation Failed",
                "One or more fields have validation errors"
        );
        error.setPath(getPath(request));
        error.setFieldErrors(fieldErrors);

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    /**
     * Handle malformed JSON (400)
     */
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ErrorResponse> handleMalformedJson(
            HttpMessageNotReadableException ex,
            WebRequest request) {

        log.warn("Malformed JSON: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Malformed Request",
                "The request body is missing or contains "
                + "invalid JSON"
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    /**
     * Handle wrong HTTP method (405)
     */
    @ExceptionHandler(HttpRequestMethodNotSupportedException.class)
    public ResponseEntity<ErrorResponse> handleMethodNotSupported(
            HttpRequestMethodNotSupportedException ex,
            WebRequest request) {

        log.warn("Method not supported: {}", ex.getMessage());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.METHOD_NOT_ALLOWED.value(),
                "Method Not Allowed",
                "HTTP method '" + ex.getMethod()
                + "' is not supported for this endpoint"
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error,
                HttpStatus.METHOD_NOT_ALLOWED);
    }
```

```
    /**
     * Handle missing request parameters (400)
     */
    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ErrorResponse> handleMissingParameter(
            MissingServletRequestParameterException ex,
            WebRequest request) {

        log.warn("Missing parameter: {}", ex.getParameterName());

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Missing Parameter",
                "Required parameter '"
                + ex.getParameterName() + "' is missing"
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    /**
     * Handle type mismatch (e.g., string where number expected) (400)
     */
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ErrorResponse> handleTypeMismatch(
            MethodArgumentTypeMismatchException ex,
            WebRequest request) {

        log.warn("Type mismatch: {}", ex.getMessage());

        String message = String.format(
                "Parameter '%s' should be of type '%s'",
                ex.getName(),
                ex.getRequiredType() != null
                    ? ex.getRequiredType().getSimpleName()
                    : "unknown");

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Type Mismatch",
                message
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    /**
     * Handle all other exceptions (500)
     * This is the catch-all handler.
     */
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAllOtherExceptions(
```

```
            Exception ex, WebRequest request) {

        log.error("Unexpected error", ex);

        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Internal Server Error",
                "An unexpected error occurred. "
                + "Please try again later."
        );
        error.setPath(getPath(request));

        return new ResponseEntity<>(error,
                HttpStatus.INTERNAL_SERVER_ERROR);
    }

    private String getPath(WebRequest request) {
        return request.getDescription(false)
                .replace("uri=", "");
    }
}
```

Testing All Error Scenarios

```bash
# 1. Resource not found (404)
curl -s http://localhost:8080/api/books/999 | python3 -m json.tool
```

```json
{
    "status": 404,
    "error": "Not Found",
    "message": "Book not found with id: '999'",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books/999"
}
```

```bash
# 2. Validation error (400)
curl -s -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "", "price": -5}' | python3 -m json.tool
```

```json
{
    "status": 400,
    "error": "Validation Failed",
    "message": "One or more fields have validation errors",
    "timestamp": "2024-01-15T10:30:00",
```

```
    "path": "/api/books",
    "fieldErrors": [
        {
            "field": "title",
            "message": "Title is required",
            "rejectedValue": ""
        },
        {
            "field": "author",
            "message": "Author is required",
            "rejectedValue": null
        },
        {
            "field": "price",
            "message": "Price must be greater than zero",
            "rejectedValue": -5.0
        },
        {
            "field": "isbn",
            "message": "ISBN is required",
            "rejectedValue": null
        }
    ]
}
```

```bash
# 3. Duplicate resource (409)
curl -s -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Test", "author": "Bob", "price": 9.99, "isbn": "978-1617292545"}' \
  | python3 -m json.tool
```

```json
{
    "status": 409,
    "error": "Conflict",
    "message": "Book already exists with isbn: '978-1617292545'",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books"
}
```

```bash
# 4. Malformed JSON (400)
curl -s -X POST http://localhost:8080/api/books \
  -H "Content-Type: application/json" \
  -d '{broken json' | python3 -m json.tool
```

```json
{
    "status": 400,
    "error": "Malformed Request",
```

```
    "message": "The request body is missing or contains invalid JSON",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books"
}
```

```bash
# 5. Type mismatch (400)
curl -s http://localhost:8080/api/books/abc | python3 -m json.tool
```

```json
{
    "status": 400,
    "error": "Type Mismatch",
    "message": "Parameter 'id' should be of type 'Long'",
    "timestamp": "2024-01-15T10:30:00",
    "path": "/api/books/abc"
}
```

Common Mistakes

Mistake 1: Catching Exceptions in the Controller

```java
// WRONG: Handling exceptions in the controller
@GetMapping("/{id}")
public ResponseEntity<?> getBook(@PathVariable Long id) {
    try {
        BookResponse book = bookService.getBookById(id);
        return ResponseEntity.ok(book);
    } catch (ResourceNotFoundException e) {
        return ResponseEntity.status(404)
                .body(Map.of("error", e.getMessage()));
    }
}
```

```java
// CORRECT: Let the global handler catch it
@GetMapping("/{id}")
public ResponseEntity<BookResponse> getBook(
        @PathVariable Long id) {
    return ResponseEntity.ok(bookService.getBookById(id));
}
```

The whole point of `@ControllerAdvice` is to centralize error handling. Do not duplicate it in controllers.

Mistake 2: Exposing Stack Traces to Clients

```java
// WRONG: Sending the exception message directly
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
    ErrorResponse error = new ErrorResponse(
            500, "Error", ex.getMessage());
    // ex.getMessage() might contain: "Connection refused:
    // jdbc:postgresql://internal-db:5432/mydb"
    // This exposes internal infrastructure details!
    return new ResponseEntity<>(error,
            HttpStatus.INTERNAL_SERVER_ERROR);
}
```

```java
// CORRECT: Use a generic message for unexpected errors
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
    log.error("Unexpected error", ex);  // Log full details
    ErrorResponse error = new ErrorResponse(
            500, "Internal Server Error",
            "An unexpected error occurred. Please try again later.");
    return new ResponseEntity<>(error,
            HttpStatus.INTERNAL_SERVER_ERROR);
}
```

Log the full details for debugging. Send a generic message to the client. Never expose database URLs, file paths, class names, or stack traces.

Mistake 3: Forgetting the Catch-All Handler

Without a catch-all `@ExceptionHandler(Exception.class)`, unexpected exceptions fall through to Spring's default handler. That default handler returns a different format than your custom handlers, creating inconsistency.

Always include a catch-all handler as the last `@ExceptionHandler` method.

Mistake 4: Inconsistent Error Response Format

```java
// WRONG: Different formats in different handlers
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<Map<String, String>> handle404(...) {
    return ResponseEntity.status(404)
            .body(Map.of("error", ex.getMessage()));
}
```

```
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handle500(...) {
    return ResponseEntity.status(500).body("Something went wrong");
}
```

Always use the same `ErrorResponse` class for all error responses. Clients should be able to parse every error response with the same code.

Best Practices

1. Use one `@ControllerAdvice` class for all exception handling. Having multiple can cause confusion about which handler runs.
2. Create specific exception classes for specific error conditions. `ResourceNotFoundException`, `DuplicateResourceException`, and `InvalidOperationException` are better than throwing generic `RuntimeException`.
3. Always include a catch-all handler. Unexpected exceptions should return a clean 500 response, not a stack trace.
4. Never expose internal details in error messages. Database errors, file paths, and class names are for logs, not for clients.
5. Log errors at the right level. Use `log.warn()` for client errors (4xx) and `log.error()` for server errors (5xx).
6. Include the request path in error responses. It helps clients and developers identify which endpoint failed.
7. Use consistent error response format. Every error from your API should use the same `ErrorResponse` structure.
8. Keep error messages actionable. "Book not found with id: '999'" is better than "Error occurred." The client can fix the first message; the second is useless.

Quick Summary

- Custom exceptions like `ResourceNotFoundException` represent specific error conditions in your application.
- @ControllerAdvice marks a class that handles exceptions across all controllers.

- @ExceptionHandler marks a method that catches a specific exception type and returns a custom response.
- ErrorResponse is a DTO that gives errors a consistent, clean format with status, error type, message, timestamp, and path.
- Validation errors from `@Valid` throw `MethodArgumentNotValidException`, which you catch and convert into field-level error details.
- Always include a catch-all handler for `Exception.class` to handle unexpected errors.
- Never expose stack traces, database details, or internal class names to the client.

Key Points

- Centralize all exception handling in one `@ControllerAdvice` class.
- Keep controllers focused on the happy path. Let the exception handler deal with errors.
- Use specific exception classes for specific error conditions.
- Log full error details. Return generic messages to clients.
- Use consistent error response format across all error types.
- Handle common Spring exceptions: validation errors, malformed JSON, type mismatches, missing parameters.
- The catch-all handler is your safety net. Never skip it.

Practice Questions

1. What is the difference between `@ControllerAdvice` and putting try-catch blocks in every controller method? Why is `@ControllerAdvice` better?
2. Why should you never return `ex.getMessage()` directly in a 500 error response? What should you do instead?
3. What exception does Spring throw when `@Valid` validation fails? What information can you extract from it?
4. Explain the purpose of `@JsonInclude(JsonInclude.Include.NON_NULL)` on the `ErrorResponse` class.
5. You have a `UserService` that throws `IllegalArgumentException` when a username contains special characters. How would you handle this in a `@ControllerAdvice` class?

Exercises

Exercise 1: E-Commerce Error Handling

Build an error handling system for an e-commerce API with these custom exceptions:

- `ProductNotFoundException` -- when a product ID does not exist
- `InsufficientStockException` -- when ordering more items than available
- `InvalidOrderException` -- when an order has invalid data (e.g., negative quantity)

Create a `GlobalExceptionHandler` that handles each one with appropriate HTTP status codes (404, 409, and 400 respectively). Test with curl.

Exercise 2: User Management Errors

Build a user management API with:

- `UserAlreadyExistsException` for duplicate email addresses
- Validation on the user registration DTO (email, password, name)
- Global exception handler that returns clean error responses for both validation errors and duplicate user errors
- Handle the case where a client sends a non-numeric ID in the URL (`/api/users/abc`)

Exercise 3: Complete API with Full Error Handling

Combine everything from Chapters 9 through 12 into a complete Task Management API:

- Task model: `id`, `title`, `description`, `priority` (1-5), `status` (PENDING/IN_PROGRESS/COMPLETED), `dueDate`
- Full CRUD endpoints with DTOs
- Validation on all input fields
- Custom exceptions: `TaskNotFoundException`, `InvalidStatusTransitionException` (e.g., cannot go from COMPLETED to PENDING)
- Global exception handler covering all error scenarios
- Test every success and error scenario with curl

What Is Next?

Congratulations! You have now built a complete, production-quality REST API. You know how to:

- Create and manage beans (Chapter 7)
- Configure your application for different environments (Chapter 8)
- Build REST controllers that handle HTTP requests (Chapter 9)
- Handle path variables, query parameters, and request bodies (Chapter 10)
- Validate input data (Chapter 11)
- Handle errors gracefully (Chapter 12)

These six chapters cover the essential skills for building Spring Boot REST APIs. In the upcoming chapters, you will learn how to persist data to a database with Spring Data JPA, secure your API with Spring Security, and write tests to verify your code works correctly. The foundation you built here will make those topics much easier to learn.

Chapter 13: Interceptors and Filters

What You Will Learn

By the end of this chapter, you will be able to:

- Understand how HTTP requests travel through a Spring Boot application
- Create custom servlet filters using `OncePerRequestFilter`
- Build handler interceptors with `preHandle`, `postHandle`, and `afterCompletion`
- Know when to use a filter versus an interceptor
- Implement request logging that tracks every incoming request
- Measure request processing time for performance monitoring

Why This Chapter Matters

Imagine you work at a large office building. Before anyone reaches their meeting room, they must pass through several checkpoints. First, the security guard at the front door checks their ID. Then, the receptionist logs their visit in a book. Finally, a floor guide directs them to the correct room.

HTTP requests in a Spring Boot application work the same way. Before a request reaches your controller method, it can pass through filters and interceptors. These are checkpoints that can inspect, modify, log, or even reject requests before they arrive at their destination.

This matters because real applications need cross-cutting concerns. You need to log every request for debugging. You need to measure response times for performance. You need to check authentication tokens. You need to add security headers. Filters and interceptors let you do all of this without cluttering your controller code.

The Request Lifecycle

Before we write any code, let us understand the complete journey of an HTTP request through a Spring Boot application.

```
+-------------------------------------------------------------------+
|                        CLIENT (Browser/Postman)                   |
|                     Sends HTTP Request (GET /api/products)        |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                      EMBEDDED TOMCAT SERVER                       |
|                   (Receives the raw HTTP request)                 |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                     SERVLET FILTER CHAIN                          |
|  +-------------------------------------------------------------+  |
|  |  Filter 1 (e.g., CharacterEncodingFilter)                  |  |
|  |    |                                                        |  |
|  |    v                                                        |  |
|  |  Filter 2 (e.g., Your Custom LoggingFilter)                |  |
|  |    |                                                        |  |
|  |    v                                                        |  |
|  |  Filter 3 (e.g., SecurityFilter)                           |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                     DISPATCHER SERVLET                            |
|              (Spring's front controller - routes requests)        |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                   HANDLER INTERCEPTOR CHAIN                       |
|  +-------------------------------------------------------------+  |
|  |  preHandle()   -->  Runs BEFORE the controller              |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                        CONTROLLER METHOD                          |
|                   @GetMapping("/api/products")                    |
|                   public List<Product> getAll()                   |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
```

```
|                   HANDLER INTERCEPTOR CHAIN                        |
|  +-------------------------------------------------------------+  |
|  |  postHandle()  -->  Runs AFTER controller, BEFORE view      |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                     VIEW RESOLUTION (if any)                      |
|                  (For REST APIs, JSON serialization)              |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                   HANDLER INTERCEPTOR CHAIN                       |
|  +-------------------------------------------------------------+  |
|  |  afterCompletion() --> Runs AFTER everything is done        |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                  SERVLET FILTER CHAIN (Reverse)                   |
|               (Each filter's post-processing runs)                |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                        CLIENT (Browser/Postman)                   |
|                      Receives HTTP Response                       |
+-------------------------------------------------------------------+
```

Think of it like an airport:

- Filters are the airport security checkpoints. Everyone goes through them, no matter which gate (controller) they are heading to.
- DispatcherServlet is the flight information board that tells you which gate to go to.
- Interceptors are the gate agents who check your boarding pass right before you board the plane.

Part 1: Servlet Filters with OncePerRequestFilter

What Is a Servlet Filter?

A servlet filter sits between the client and the DispatcherServlet. It can inspect or modify the request and response at the lowest level. Every HTTP request passes through filters,

regardless of which controller handles it.

Spring provides `OncePerRequestFilter` as a convenient base class. As the name suggests, it guarantees your filter logic runs exactly once per request. This is important because in some setups, a request might pass through the filter chain multiple times due to internal forwards or includes.

Creating a Request Logging Filter

Let us create a filter that logs every incoming HTTP request with its method, URL, and processing time.

Step 1: Create the filter class

```java
package com.example.demo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component                                                      // Line 1
@Order(1)                                                       // Line 2
public class RequestLoggingFilter extends OncePerRequestFilter { // Line 3

    private static final Logger logger =
        LoggerFactory.getLogger(RequestLoggingFilter.class);  // Line 4

    @Override
    protected void doFilterInternal(                          // Line 5
            HttpServletRequest request,                       // Line 6
            HttpServletResponse response,                     // Line 7
            FilterChain filterChain                           // Line 8
    ) throws ServletException, IOException {

        // Code that runs BEFORE the request is processed
        long startTime = System.currentTimeMillis();          // Line 9
        String method = request.getMethod();                  // Line 10
        String uri = request.getRequestURI();                 // Line 11
        String queryString = request.getQueryString();        // Line 12

        logger.info("FILTER START => {} {} {}",
            method, uri,
```

```
            queryString != null ? "?" + queryString : "");    // Line 13

        // Pass the request to the next filter or controller
        filterChain.doFilter(request, response);              // Line 14

        // Code that runs AFTER the response is ready
        long duration = System.currentTimeMillis() - startTime; // Line 15
        int status = response.getStatus();                    // Line 16

        logger.info("FILTER END   <= {} {} | Status: {} | Time: {}ms",
            method, uri, status, duration);                   // Line 17
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Component`	Registers this filter as a Spring bean so Spring automatically adds it to the filter chain
2	`@Order(1)`	Sets the priority of this filter. Lower numbers run first. If you have multiple filters, this controls the order
3	`extends OncePerRequestFilter`	Inherits from Spring's base filter class that guarantees single execution per request
4	`LoggerFactory.getLogger(...)`	Creates a logger instance for this class. We use SLF4J for logging instead of `System.out.println`
5	`doFilterInternal(...)`	The main method you override. This is where your filter logic goes
6	`HttpServletRequest request`	The incoming HTTP request object. Contains method, URL, headers, body, and more
7	`HttpServletResponse response`	The outgoing HTTP response object. Contains status code, headers, and body
8	`FilterChain filterChain`	The chain of remaining filters. You must call `doFilter` to pass the request along

9	`System.currentTimeMillis()`	Records the start time so we can calculate how long the request took
10	`request.getMethod()`	Gets the HTTP method (GET, POST, PUT, DELETE)
11	`request.getRequestURI()`	Gets the request path (e.g., `/api/products`)
12	`request.getQueryString()`	Gets query parameters (e.g., `name=phone&page=1`)
13	`logger.info(...)`	Logs the request details before processing
14	`filterChain.doFilter(request, response)`	Critical line! Passes the request to the next filter or to the DispatcherServlet. Without this line, the request stops here
15	`System.currentTimeMillis() - startTime`	Calculates total processing time after the response is ready
16	`response.getStatus()`	Gets the HTTP status code (200, 404, 500, etc.)
17	`logger.info(...)`	Logs the response details after processing

How filterChain.doFilter() works:

```
Your Filter Code (before doFilter)
        |
        v
filterChain.doFilter(request, response)  -----> Next Filter / Controller
        |                                                |
        |     <-- control returns after processing --  |
        v
Your Filter Code (after doFilter)
```

Think of `filterChain.doFilter()` like passing a baton in a relay race. You do your work, pass the baton, wait for everyone else to finish, and then do any cleanup work.

Creating a Simple Controller to Test

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping
    public List<Map<String, Object>> getAllProducts() {
        return List.of(
            Map.of("id", 1, "name", "Laptop", "price", 999.99),
            Map.of("id", 2, "name", "Phone", "price", 699.99)
        );
    }

    @GetMapping("/{id}")
    public Map<String, Object> getProduct(@PathVariable int id) {
        return Map.of("id", id, "name", "Laptop", "price", 999.99);
    }
}
```

Testing the Filter

When you send a GET request to `http://localhost:8080/api/products`, you will see this in the console:

```
INFO  c.e.d.filter.RequestLoggingFilter : FILTER START => GET /api/products
INFO  c.e.d.filter.RequestLoggingFilter : FILTER END   <= GET /api/products | Status: 2...
```

When you send a GET request to `http://localhost:8080/api/products/1`:

```
INFO  c.e.d.filter.RequestLoggingFilter : FILTER START => GET /api/products/1
INFO  c.e.d.filter.RequestLoggingFilter : FILTER END   <= GET /api/products/1 | Status:...
```

Excluding Certain URLs from the Filter

Sometimes you do not want a filter to run for every request. For example, you might want to skip logging for health check endpoints or static resources.

```java
@Component
@Order(1)
public class RequestLoggingFilter extends OncePerRequestFilter {

    private static final Logger logger =
        LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        // Skip filtering for these paths
        return path.startsWith("/actuator")
            || path.startsWith("/health")
            || path.endsWith(".css")
            || path.endsWith(".js");
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {
        long startTime = System.currentTimeMillis();
        logger.info("=> {} {}", request.getMethod(), request.getRequestURI());

        filterChain.doFilter(request, response);

        long duration = System.currentTimeMillis() - startTime;
        logger.info("<= {} {} | {}ms",
            request.getMethod(), request.getRequestURI(), duration);
    }
}
```

The `shouldNotFilter()` method returns `true` for paths that should skip this filter entirely.

Adding a Custom Response Header Filter

Here is another practical filter that adds a custom header to every response:

```java
package com.example.demo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import java.io.IOException;
import java.util.UUID;

@Component
@Order(2)
public class CorrelationIdFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {

        // Check if client sent a correlation ID
        String correlationId = request.getHeader("X-Correlation-Id");

        // If not, generate one
        if (correlationId == null || correlationId.isEmpty()) {
            correlationId = UUID.randomUUID().toString();
        }

        // Add it to the response so the client can see it
        response.setHeader("X-Correlation-Id", correlationId);

        filterChain.doFilter(request, response);
    }
}
```

A correlation ID is a unique identifier attached to each request. It helps you trace a single request across multiple services and log files. Think of it like a tracking number for a package.

Output (Response Headers):

```
HTTP/1.1 200 OK
X-Correlation-Id: 3f7a8b2c-1234-5678-abcd-ef0123456789
Content-Type: application/json
```

Part 2: Handler Interceptors

What Is a Handler Interceptor?

A handler interceptor sits between the DispatcherServlet and your controller. Unlike filters, interceptors are part of the Spring MVC framework and have access to Spring-specific information like which controller method will handle the request.

```
+------------------+     +---------------------+     +------------------+
|  Servlet Filter  | --> | DispatcherServlet   | --> | HandlerInterceptor|
|  (Low level)     |     | (Spring's Router)   |     | (Spring MVC)     |
|                  |     |                     |     |                  |
| Has access to:   |     |                     |     | Has access to:   |
| - Raw request    |     |                     |     | - Raw request    |
| - Raw response   |     |                     |     | - Raw response   |
| - Filter chain   |     |                     |     | - Handler method |
|                  |     |                     |     | - ModelAndView   |
|                  |     |                     |     | - Exception info |
+------------------+     +---------------------+     +------------------+
```

The Three Methods of HandlerInterceptor

```
         Request arrives
              |
              v
    +-------------------+
    |   preHandle()     |    Runs BEFORE the controller
    |   return true?    |----- false ----> Request stops here
    +--------+----------+
             | true
             v
    +-------------------+
    |    Controller     |    Your @GetMapping / @PostMapping method
    |    Method runs    |
    +--------+----------+
             |
             v
    +-------------------+
    |   postHandle()    |    Runs AFTER controller, BEFORE response
    |                   |    Only if controller succeeded
    +--------+----------+
             |
             v
    +-------------------+
    |  afterCompletion()|    ALWAYS runs, even if an exception occurred
    |                   |    Good for cleanup (like a finally block)
    +-------------------+
             |
             v
      Response sent to client
```

Creating a Handler Interceptor

Step 1: Create the interceptor class

```java
package com.example.demo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

@Component                                                      // Line 1
public class PerformanceInterceptor implements HandlerInterceptor { // Line 2

    private static final Logger logger =
        LoggerFactory.getLogger(PerformanceInterceptor.class);

    @Override
    public boolean preHandle(                                   // Line 3
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler                                      // Line 4
    ) throws Exception {

        // Store the start time as a request attribute
        request.setAttribute("startTime",
            System.currentTimeMillis());                        // Line 5

        // Log which controller method will handle this request
        if (handler instanceof HandlerMethod handlerMethod) { // Line 6
            String className = handlerMethod
                .getBeanType().getSimpleName();                 // Line 7
            String methodName = handlerMethod
                .getMethod().getName();                         // Line 8
            logger.info("INTERCEPTOR preHandle => {}.{}()",
                className, methodName);                         // Line 9
        }

        return true;                                            // Line 10
    }

    @Override
    public void postHandle(                                     // Line 11
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            ModelAndView modelAndView                           // Line 12
    ) throws Exception {
        logger.info("INTERCEPTOR postHandle => Response status: {}",
            response.getStatus());
    }

    @Override
    public void afterCompletion(                                // Line 13
```

```
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            Exception ex                                    // Line 14
    ) throws Exception {

        long startTime = (Long) request.getAttribute("startTime"); // Line 15
        long duration = System.currentTimeMillis() - startTime;

        if (ex != null) {                                   // Line 16
            logger.error(
                "INTERCEPTOR afterCompletion => FAILED in {}ms. Error: {}",
                duration, ex.getMessage());
        } else {
            logger.info(
                "INTERCEPTOR afterCompletion => Completed in {}ms",
                duration);                                  // Line 17
        }
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Component`	Registers the interceptor as a Spring bean
2	`implements HandlerInterceptor`	Implements Spring's interceptor interface which defines the three callback methods
3	`preHandle(...)`	Called before the controller method. Returns `boolean` to control whether the request should continue
4	`Object handler`	The handler that will process the request. Usually a `HandlerMethod` that wraps your controller method
5	`request.setAttribute(...)`	Stores data in the request that can be retrieved later. Think of it as a sticky note you attach to the request
6	`handler instanceof HandlerMethod`	Checks if the handler is a controller method (it could also be a static resource handler). Uses Java 16+ pattern matching

7	`getBeanType().getSimpleName()`	Gets the controller class name (e.g., `ProductController`)
8	`getMethod().getName()`	Gets the method name (e.g., `getAllProducts`)
9	`logger.info(...)`	Logs which controller method will handle this request
10	`return true`	Returning `true` means "continue processing." Returning `false` would stop the request here
11	`postHandle(...)`	Called after the controller method runs successfully but before the response is committed
12	`ModelAndView modelAndView`	The model and view returned by the controller. For REST APIs, this is usually `null`
13	`afterCompletion(...)`	Always called after the complete request is finished, even if an exception occurred
14	`Exception ex`	If the controller threw an exception, it is passed here. Otherwise `null`
15	`request.getAttribute(...)`	Retrieves the start time we stored in `preHandle`
16	`if (ex != null)`	Checks whether an error occurred during processing
17	`logger.info(...)`	Logs the total time taken for the request

Step 2: Register the interceptor

Unlike filters, interceptors must be explicitly registered with Spring MVC through a configuration class.

```java
package com.example.demo.config;

import com.example.demo.interceptor.PerformanceInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration                                              // Line 1
```

```
public class WebConfig implements WebMvcConfigurer {          // Line 2

    private final PerformanceInterceptor performanceInterceptor;

    public WebConfig(PerformanceInterceptor performanceInterceptor) {
        this.performanceInterceptor = performanceInterceptor; // Line 3
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) { // Line 4
        registry.addInterceptor(performanceInterceptor)       // Line 5
                .addPathPatterns("/api/**")                    // Line 6
                .excludePathPatterns("/api/health");           // Line 7
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Configuration`	Marks this as a configuration class that Spring processes at startup
2	`implements WebMvcConfigurer`	Provides callback methods to customize Spring MVC configuration
3	Constructor injection	Spring automatically injects the `PerformanceInterceptor` bean
4	`addInterceptors(...)`	Override this method to register your interceptors
5	`registry.addInterceptor(...)`	Adds your interceptor to the chain
6	`.addPathPatterns("/api/**")`	This interceptor only runs for URLs starting with `/api/`. The `**` means any sub-path
7	`.excludePathPatterns(...)`	Skips the interceptor for the health endpoint

Testing the Interceptor

When you send a GET request to `http://localhost:8080/api/products`:

```
INFO  c.e.d.filter.RequestLoggingFilter       : FILTER START => GET /api/products
INFO  c.e.d.interceptor.PerformanceInterceptor : INTERCEPTOR preHandle => ProductContro...
INFO  c.e.d.interceptor.PerformanceInterceptor : INTERCEPTOR postHandle => Response sta...
INFO  c.e.d.interceptor.PerformanceInterceptor : INTERCEPTOR afterCompletion => Complet...
INFO  c.e.d.filter.RequestLoggingFilter       : FILTER END   <= GET /api/products | Sta...
```

Notice the order: Filter starts first, then the interceptor runs, then the interceptor finishes, and finally the filter finishes. This is because filters wrap around the entire Spring MVC processing.

```
Filter Start
    Interceptor preHandle
        Controller Method
    Interceptor postHandle
    Interceptor afterCompletion
Filter End
```

Using preHandle to Block Requests

One powerful use of `preHandle` is to block requests that do not meet certain criteria. For example, you could require a specific API key header:

```java
@Component
public class ApiKeyInterceptor implements HandlerInterceptor {

    private static final String API_KEY = "my-secret-key-123";

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) throws Exception {

        String apiKey = request.getHeader("X-API-Key");

        if (apiKey == null || !apiKey.equals(API_KEY)) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json");
            response.getWriter().write(
                "{\"error\": \"Invalid or missing API key\"}"
            );
            return false;   // Stop! Do not proceed to the controller
        }

        return true;        // API key is valid, continue
    }
}
```

When `preHandle` returns `false`:

- The controller method is never called
- `postHandle` is never called
- `afterCompletion` IS still called (for cleanup)

Part 3: Filter vs Interceptor Comparison

Now that you have seen both in action, here is a detailed comparison:

```
+-----------------------------+-------------------------------+-------------------------...
|        Feature              |         Servlet Filter        |     Handler Interceptor...
+-----------------------------+-------------------------------+-------------------------...
| Part of                     | Servlet API (Jakarta EE)      | Spring MVC Framework   ...
+-----------------------------+-------------------------------+-------------------------...
| Scope                       | All requests (including       | Only requests handled b...
|                             | static resources)             | Spring MVC controllers ...
+-----------------------------+-------------------------------+-------------------------...
| When it runs                | Before DispatcherServlet      | After DispatcherServlet...
|                             |                               | before/after Controller...
+-----------------------------+-------------------------------+-------------------------...
| Access to handler info      | No                            | Yes (knows which contro...
|                             |                               | method will run)       ...
+-----------------------------+-------------------------------+-------------------------...
| Access to ModelAndView      | No                            | Yes (in postHandle)    ...
+-----------------------------+-------------------------------+-------------------------...
| Access to exceptions        | No (must use try-catch)       | Yes (in afterCompletion...
+-----------------------------+-------------------------------+-------------------------...
| Can modify request/response | Yes (can wrap them)           | Yes                    ...
+-----------------------------+-------------------------------+-------------------------...
| Registration                | @Component or @Bean           | WebMvcConfigurer +     ...
|                             |                               | addInterceptors()      ...
+-----------------------------+-------------------------------+-------------------------...
| URL pattern control         | shouldNotFilter() or          | addPathPatterns() /    ...
|                             | FilterRegistrationBean        | excludePathPatterns()  ...
+-----------------------------+-------------------------------+-------------------------...
| Best for                    | Security, CORS, encoding,     | Logging with controller...
|                             | compression, low-level tasks  | auth checks, performanc...
+-----------------------------+-------------------------------+-------------------------...
```

When to Use What

Use a Filter when:

- You need to process every request, including static resources

- You need to modify the raw request or response (e.g., wrapping them)
- You are working with non-Spring concerns (e.g., servlet-level security)
- You need to set response headers for all requests
- You are implementing CORS, encoding, or compression

Use an Interceptor when:

- You need to know which controller method will handle the request
- You need access to Spring MVC concepts like `ModelAndView`
- You want fine-grained URL pattern matching
- You need separate logic for before-controller, after-controller, and after-completion
- You want to access the exception that a controller threw

Part 4: Complete Request Logging Example

Let us put everything together into a complete, production-ready request logging system that uses both a filter and an interceptor.

Project structure:

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── config/
│   └── WebConfig.java
├── controller/
│   └── ProductController.java
├── filter/
│   └── RequestLoggingFilter.java
└── interceptor/
    └── RequestTimingInterceptor.java
```

application.properties:

```properties
# Application name
spring.application.name=interceptor-filter-demo

# Server port
server.port=8080

# Logging level for our packages
logging.level.com.example.demo=DEBUG
```

DemoApplication.java:

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

RequestLoggingFilter.java:

```java
package com.example.demo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class RequestLoggingFilter extends OncePerRequestFilter {

    private static final Logger logger =
        LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return path.contains("favicon")
            || path.startsWith("/actuator");
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {

        String method = request.getMethod();
        String uri = request.getRequestURI();
```

```
        String clientIp = request.getRemoteAddr();

        logger.info("========== REQUEST START ==========");
        logger.info("Method: {} | URI: {} | Client IP: {}",
            method, uri, clientIp);
        logger.info("Content-Type: {}",
            request.getContentType());
        logger.info("User-Agent: {}",
            request.getHeader("User-Agent"));

        long startTime = System.currentTimeMillis();

        try {
            filterChain.doFilter(request, response);
        } finally {
            long duration = System.currentTimeMillis() - startTime;
            logger.info("Status: {} | Duration: {}ms",
                response.getStatus(), duration);
            logger.info("========== REQUEST END ============");
        }
    }
}
```

RequestTimingInterceptor.java:

```java
package com.example.demo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

@Component
public class RequestTimingInterceptor implements HandlerInterceptor {

    private static final Logger logger =
        LoggerFactory.getLogger(RequestTimingInterceptor.class);

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) throws Exception {
        request.setAttribute("controllerStartTime",
            System.currentTimeMillis());

        if (handler instanceof HandlerMethod hm) {
```

```
            logger.debug("  -> Handler: {}.{}()",
                hm.getBeanType().getSimpleName(),
                hm.getMethod().getName());
        }

        return true;
    }

    @Override
    public void postHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            ModelAndView modelAndView
    ) throws Exception {
        if (handler instanceof HandlerMethod hm) {
            long startTime = (Long)
                request.getAttribute("controllerStartTime");
            long controllerTime =
                System.currentTimeMillis() - startTime;
            logger.debug("  -> Controller execution time: {}ms",
                controllerTime);
        }
    }

    @Override
    public void afterCompletion(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            Exception ex
    ) throws Exception {
        if (ex != null) {
            logger.error("  -> Exception occurred: {}",
                ex.getMessage());
        }
    }
}
```

WebConfig.java:

```java
package com.example.demo.config;

import com.example.demo.interceptor.RequestTimingInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final RequestTimingInterceptor requestTimingInterceptor;
```

```
    public WebConfig(RequestTimingInterceptor requestTimingInterceptor) {
        this.requestTimingInterceptor = requestTimingInterceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(requestTimingInterceptor)
                .addPathPatterns("/api/**");
    }
}
```

ProductController.java:

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping
    public List<Map<String, Object>> getAllProducts() {
        return List.of(
            Map.of("id", 1, "name", "Laptop", "price", 999.99),
            Map.of("id", 2, "name", "Phone", "price", 699.99),
            Map.of("id", 3, "name", "Tablet", "price", 449.99)
        );
    }

    @GetMapping("/{id}")
    public Map<String, Object> getProductById(@PathVariable int id) {
        return Map.of("id", id, "name", "Laptop", "price", 999.99);
    }

    @PostMapping
    public Map<String, Object> createProduct(
            @RequestBody Map<String, Object> product) {
        product.put("id", 100);
        return product;
    }
}
```

Complete Output

GET http://localhost:8080/api/products

```
INFO  c.e.d.filter.RequestLoggingFilter           : ========== REQUEST START ==========
INFO  c.e.d.filter.RequestLoggingFilter           : Method: GET | URI: /api/products | ...
INFO  c.e.d.filter.RequestLoggingFilter           : Content-Type: null
INFO  c.e.d.filter.RequestLoggingFilter           : User-Agent: PostmanRuntime/7.36.0
DEBUG c.e.d.interceptor.RequestTimingInterceptor   :   -> Handler: ProductController.ge...
DEBUG c.e.d.interceptor.RequestTimingInterceptor   :   -> Controller execution time: 15ms
INFO  c.e.d.filter.RequestLoggingFilter           : Status: 200 | Duration: 42ms
INFO  c.e.d.filter.RequestLoggingFilter           : ========== REQUEST END ============
```

Response body:

```json
[
    {"id": 1, "name": "Laptop", "price": 999.99},
    {"id": 2, "name": "Phone", "price": 699.99},
    {"id": 3, "name": "Tablet", "price": 449.99}
]
```

Common Mistakes

Mistake 1: Forgetting to Call filterChain.doFilter()

```java
// WRONG - Request gets stuck in the filter and never reaches the controller
@Override
protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
) throws ServletException, IOException {
    logger.info("Processing request...");
    // Oops! Forgot to call filterChain.doFilter()
    // The client will get an empty response
}

// CORRECT - Always call filterChain.doFilter() unless you intentionally
// want to block the request
@Override
protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
) throws ServletException, IOException {
    logger.info("Processing request...");
    filterChain.doFilter(request, response);  // Pass the request along!
}
```

Mistake 2: Not Registering the Interceptor

```java
// WRONG - Creating the interceptor class but not registering it
// The interceptor exists but Spring MVC does not know about it
@Component
public class MyInterceptor implements HandlerInterceptor {
    // This alone is not enough!
}

// CORRECT - You must also register it in a WebMvcConfigurer
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(myInterceptor)
                .addPathPatterns("/**");  // Now it is registered!
    }
}
```

Mistake 3: Modifying the Response After filterChain.doFilter()

```java
// WRONG - Setting headers after the response has been committed
@Override
protected void doFilterInternal(...) {
    filterChain.doFilter(request, response);
    response.setHeader("X-Custom", "value"); // May not work!
    // Response might already be sent to the client
}

// CORRECT - Set response headers BEFORE calling doFilter
@Override
protected void doFilterInternal(...) {
    response.setHeader("X-Custom", "value"); // Set before processing
    filterChain.doFilter(request, response);
}
```

Mistake 4: Using Filter When You Need Controller Info

```java
// WRONG - Trying to get controller info in a filter
// Filters run BEFORE Spring MVC, so they do not know which
// controller will handle the request
@Override
protected void doFilterInternal(...) {
```

```
    // There is no way to know which controller method
    // will handle this request from a filter
}

// CORRECT - Use an interceptor when you need controller info
@Override
public boolean preHandle(HttpServletRequest request,
                         HttpServletResponse response,
                         Object handler) {
    if (handler instanceof HandlerMethod hm) {
        String methodName = hm.getMethod().getName(); // Works!
    }
    return true;
}
```

Best Practices

1. Use `OncePerRequestFilter` instead of `Filter`: It prevents double execution in forwarded requests. Always extend `OncePerRequestFilter` unless you have a specific reason not to.
2. Wrap `filterChain.doFilter()` in try-finally: This ensures your cleanup code runs even if an exception occurs downstream.
3. Keep filters and interceptors focused: Each filter or interceptor should do one thing well. Do not create a single filter that does logging, authentication, and header modification all at once.
4. Use `@Order` to control filter execution order: When you have multiple filters, use `@Order` annotations to define a clear execution sequence. Lower numbers run first.
5. Use interceptors for Spring-specific concerns: If you need access to the handler method, `ModelAndView`, or Spring's exception handling, use an interceptor.
6. Use `shouldNotFilter()` to exclude paths: Do not add `if` statements inside `doFilterInternal`. Override `shouldNotFilter()` for cleaner code.
7. Log at appropriate levels: Use `DEBUG` for detailed information and `INFO` for important events. Do not log everything at `INFO` level in production.
8. Be careful with request body reading: Once you read the request body in a filter, the controller cannot read it again. If you need to read the body, wrap the request in a `ContentCachingRequestWrapper`.

Quick Summary

```
+-----------------------------------+
|  Filters and Interceptors         |
|  Quick Reference                  |
+-----------------------------------+
|                                   |
|  FILTER (OncePerRequestFilter)    |
|  - Sits before DispatcherServlet  |
|  - Processes ALL requests         |
|  - doFilterInternal() method      |
|  - Must call filterChain.doFilter|
|  - @Component + @Order            |
|  - Best for: logging, headers,    |
|    security, encoding             |
|                                   |
|  INTERCEPTOR (HandlerInterceptor)|
|  - Sits after DispatcherServlet   |
|  - Only Spring MVC requests       |
|  - preHandle() -> boolean         |
|  - postHandle() -> after ctrl     |
|  - afterCompletion() -> cleanup   |
|  - Register in WebMvcConfigurer   |
|  - Best for: controller-aware     |
|    logging, auth, timing          |
|                                   |
+-----------------------------------+
```

Key Points

1. Filters are part of the Servlet API and run before the DispatcherServlet. Interceptors are part of Spring MVC and run after the DispatcherServlet.
2. `OncePerRequestFilter` guarantees your filter logic runs exactly once per request, even with internal forwards.
3. You must call `filterChain.doFilter(request, response)` in a filter, or the request will never reach the controller.
4. Handler interceptors have three methods: `preHandle` (before controller), `postHandle` (after controller), and `afterCompletion` (always runs, like a finally block).
5. Returning `false` from `preHandle` stops the request from reaching the controller.
6. Interceptors must be registered in a `WebMvcConfigurer` implementation using `addInterceptors()`.
7. Filters are registered automatically with `@Component`, or manually with `FilterRegistrationBean` for more control.

Practice Questions

1. What is the difference between a servlet filter and a handler interceptor? When would you choose one over the other?
2. What happens if you forget to call `filterChain.doFilter(request, response)` inside a filter?
3. Explain the three methods of `HandlerInterceptor`. When does each one run? Which one always runs regardless of exceptions?
4. How do you control the order in which multiple filters execute? How do you control interceptor order?
5. Why does Spring provide `OncePerRequestFilter` instead of just using the basic `Filter` interface?

Exercises

Exercise 1: Rate Limiting Interceptor

Create an interceptor that limits each client to 10 requests per minute. Store the request count in a `ConcurrentHashMap` using the client's IP address as the key. If the limit is exceeded, return a 429 (Too Many Requests) status code.

Hints:

- Use `request.getRemoteAddr()` to get the client IP
- Use `System.currentTimeMillis()` to track time windows
- Return `false` from `preHandle` to block the request

Exercise 2: Request/Response Body Logging Filter

Create a filter that logs the request body for POST and PUT requests, and the response body for all requests. You will need to use `ContentCachingRequestWrapper` and `ContentCachingResponseWrapper` from Spring.

Hints:

- Wrap the request: `new ContentCachingRequestWrapper(request)`
- Wrap the response: `new ContentCachingResponseWrapper(response)`
- Read cached content after `filterChain.doFilter()` completes
- Call `response.copyBodyToResponse()` to ensure the client receives the response

Exercise 3: Multi-Interceptor Chain

Create three interceptors: `AuthInterceptor`, `LoggingInterceptor`, and `TimingInterceptor`. Register them in a specific order and observe how they execute. Add logging to each method to verify the execution order matches your expectations.

What Is Next?

Now that you understand how to intercept and process requests at different levels, it is time to work with data. In the next chapter, you will learn about Spring Data JPA and H2 -- how to set up a database, create entity classes, and let Spring automatically create your database tables. This is where your applications start to store and retrieve real data.

Chapter 14: Spring Data JPA and H2

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what JPA is and why it exists
- Add Spring Data JPA and H2 dependencies to your project
- Create entity classes that map to database tables
- Use annotations like `@Entity`, `@Id`, `@GeneratedValue`, and `@Column`
- Configure H2 as an in-memory database for development
- Access the H2 console to inspect your database
- Pre-load data using `data.sql`
- Configure `show-sql` and `ddl-auto` properties

Why This Chapter Matters

So far, your Spring Boot applications have been stateless. Every time you restart the server, all data disappears. That is like a restaurant where the waiters have amnesia -- they forget every order the moment they turn around.

Real applications need to store data. Customer information, product catalogs, order histories -- all of this needs to persist somewhere. That somewhere is a database.

But here is the problem: Java speaks objects (classes, fields, methods), and databases speak tables (rows, columns, SQL). These are two very different languages. JPA (Java Persistence API) acts as a translator between them. It lets you work with Java objects while JPA handles all the SQL behind the scenes.

And H2 is a lightweight database that runs inside your application's memory. It is perfect for learning and development because you do not need to install anything. No MySQL server, no PostgreSQL setup -- just add a dependency and you have a database.

What Is JPA?

JPA stands for Java Persistence API. It is a specification (a set of rules) that defines how Java objects should be mapped to database tables. Think of JPA as a contract that says "if you follow these rules, I will handle your database operations."

```
+-----------------------------------------+
|          YOUR JAVA APPLICATION          |
|                                         |
|   Product product = new Product();      |
|   product.setName("Laptop");            |
|   product.setPrice(999.99);             |
|   repository.save(product);             |
|                                         |
+-----------------+-----------------------+
                  |
                  |  JPA translates your Java
                  |  objects into SQL statements
                  v
+-----------------------------------------+
|               JPA LAYER                 |
|   (Hibernate is the implementation)     |
|                                         |
|   INSERT INTO products (name, price)    |
|   VALUES ('Laptop', 999.99);            |
|                                         |
+-----------------+-----------------------+
                  |
                  v
+-----------------------------------------+
|              DATABASE                   |
|   +----+--------+--------+             |
|   | id | name   | price  |             |
|   +----+--------+--------+             |
|   | 1  | Laptop | 999.99 |             |
|   +----+--------+--------+             |
+-----------------------------------------+
```

Hibernate is the most popular implementation of JPA. When you use Spring Data JPA, Hibernate is included automatically. You write Java annotations, and Hibernate generates the SQL for you.

Real-Life Analogy

Imagine you are staying at a hotel in a foreign country. You do not speak the local language, but the hotel has a concierge who speaks both your language and the local language.

- You = Your Java application
- Concierge = JPA (Hibernate)
- Local people = The database
- Your language = Java objects
- Local language = SQL

You tell the concierge "I need a taxi to the airport." The concierge translates that into the local language and communicates with the taxi driver. You never need to learn the local language.

Setting Up the Project

Step 1: Add Dependencies

If you are using Spring Initializr (start.spring.io), select these dependencies:

- Spring Web
- Spring Data JPA
- H2 Database

If you already have a project, add these to your `pom.xml`:

```xml
<dependencies>
    <!-- Spring Web for REST APIs -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Data JPA for database operations -->
    <dependency>                                            <!-- Line 1 -->
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId> <!-- Line 2 -->
    </dependency>

    <!-- H2 in-memory database -->
    <dependency>                                            <!-- Line 3 -->
        <groupId>com.h2database</groupId>
```

```
        <artifactId>h2</artifactId>                          <!-- Line 4 -->
        <scope>runtime</scope>                               <!-- Line 5 -->
    </dependency>

    <!-- Spring Boot Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Line-by-line explanation:

Line	Code	What It Does
1-2	`spring-boot-starter-data-jpa`	Pulls in Spring Data JPA, Hibernate, and all related libraries. This is the "concierge" that translates between Java and SQL
3-4	`h2`	The H2 database engine. It is a lightweight, fast, in-memory database written in Java
5	`<scope>runtime</scope>`	Tells Maven that H2 is only needed when the application runs, not when compiling your code. Your Java code never directly references H2 classes

What Gets Pulled In

When you add `spring-boot-starter-data-jpa`, Maven downloads these key libraries:

```
spring-boot-starter-data-jpa
├── spring-data-jpa          (Repository abstractions)
├── hibernate-core           (JPA implementation)
├── jakarta.persistence-api  (JPA annotations like @Entity, @Id)
├── spring-jdbc              (Low-level database access)
├── HikariCP                 (Connection pool - manages DB connections)
└── spring-tx                (Transaction management)
```

Configuring the Application

Step 2: Configure application.properties

```properties
# ===== Application =====
spring.application.name=jpa-demo

# ===== H2 Database Configuration =====
# Use in-memory database named "testdb"
spring.datasource.url=jdbc:h2:mem:testdb                    # Line 1
spring.datasource.driver-class-name=org.h2.Driver           # Line 2
spring.datasource.username=sa                                # Line 3
spring.datasource.password=                                  # Line 4

# ===== H2 Console =====
# Enable the web-based H2 console
spring.h2.console.enabled=true                               # Line 5
spring.h2.console.path=/h2-console                           # Line 6

# ===== JPA / Hibernate Configuration =====
# Automatically create/update tables based on entity classes
spring.jpa.hibernate.ddl-auto=create-drop                    # Line 7
# Show SQL statements in the console
spring.jpa.show-sql=true                                     # Line 8
# Format the SQL for readability
spring.jpa.properties.hibernate.format_sql=true              # Line 9
# Use H2 dialect for SQL generation
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect  # Line 10

# ===== Data Initialization =====
# Run data.sql after tables are created
spring.jpa.defer-datasource-initialization=true              # Line 11
```

Line-by-line explanation:

Line	Property	What It Does
1	`spring.datasource.url`	The JDBC URL for the database. `jdbc:h2:mem:testdb` means "use H2, store in memory, name it testdb." Data is lost when the app stops
2	`spring.datasource.driver-class-name`	The JDBC driver class for H2. Spring Boot can auto-detect this, but being explicit is clearer
3	`spring.datasource.username`	Database username. H2 defaults to `sa` (system administrator)

4	`spring.datasource.password`	Database password. Empty by default for H2
5	`spring.h2.console.enabled`	Enables the H2 web console, a browser-based database viewer
6	`spring.h2.console.path`	The URL path for the H2 console. Access it at `http://localhost:8080/h2-console`
7	`spring.jpa.hibernate.ddl-auto`	Controls how Hibernate manages the database schema. `create-drop` creates tables on startup and drops them on shutdown
8	`spring.jpa.show-sql`	Prints every SQL statement Hibernate executes to the console. Very useful for learning
9	`spring.jpa.properties.hibernate.format_sql`	Formats the SQL output with indentation instead of a single long line
10	`spring.jpa.database-platform`	Tells Hibernate which SQL dialect to use. H2Dialect generates H2-compatible SQL
11	`spring.jpa.defer-datasource-initialization`	Ensures `data.sql` runs after Hibernate creates the tables, not before

Understanding ddl-auto Options

The `spring.jpa.hibernate.ddl-auto` property is one of the most important settings. Here is what each option does:

```
+---------------+-------------------------------------------------+
| Option        | What It Does                                    |
+---------------+-------------------------------------------------+
| none          | Do nothing. You manage the schema yourself      |
+---------------+-------------------------------------------------+
| validate      | Check that entities match existing tables.       |
|               | Throw error if they don't match. No changes made |
+---------------+-------------------------------------------------+
| update        | Add new columns/tables but never remove existing |
|               | ones. Safe for development, risky for production |
+---------------+-------------------------------------------------+
| create        | Drop all tables and recreate them on startup.   |
|               | All data is lost every restart                  |
```

```
+---------------+-------------------------------------------------+
| create-drop   | Same as create, but also drops tables when the   |
|               | application shuts down. Best for testing         |
+---------------+-------------------------------------------------+
```

For development with H2: Use `create-drop` or `create`. Since H2 is in-memory, data is lost anyway when the app stops.

For production with a real database: Use `validate` or `none`. Never use `create` or `create-drop` in production -- you would lose all your data!

Creating Entity Classes

Step 3: Create a Product Entity

An entity class is a Java class that maps to a database table. Each instance of the class represents a row in the table.

```java
package com.example.demo.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity                                                         // Line 1
@Table(name = "products")                                       // Line 2
public class Product {

    @Id                                                         // Line 3
    @GeneratedValue(strategy = GenerationType.IDENTITY)         // Line 4
    private Long id;                                            // Line 5

    @Column(name = "product_name", nullable = false,
            length = 100)                                       // Line 6
    private String name;

    @Column(nullable = false)                                   // Line 7
    private Double price;

    @Column(length = 500)                                       // Line 8
    private String description;
```

```
    @Column(name = "in_stock")                                  // Line 9
    private Boolean inStock = true;                             // Line 10

    // Default constructor (REQUIRED by JPA)
    public Product() {                                          // Line 11
    }

    // Parameterized constructor (for convenience)
    public Product(String name, Double price,
                   String description) {                        // Line 12
        this.name = name;
        this.price = price;
        this.description = description;
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public Boolean getInStock() {
        return inStock;
    }

    public void setInStock(Boolean inStock) {
        this.inStock = inStock;
```

```
    }

    @Override
    public String toString() {
        return "Product{id=" + id + ", name='" + name + "'"
            + ", price=" + price + ", inStock=" + inStock + "}";
    }
}
```

Line-by-line explanation:

Line	Annotation/Code	What It Does
1	`@Entity`	Marks this class as a JPA entity. Hibernate will create a database table for it
2	`@Table(name = "products")`	Specifies the table name. Without this, the table would be named `product` (the class name in lowercase)
3	`@Id`	Marks this field as the primary key of the table
4	`@GeneratedValue(strategy = GenerationType.IDENTITY)`	The database auto-generates the ID value. `IDENTITY` uses auto-increment (1, 2, 3, ...)
5	`private Long id`	Use `Long` (not `long`) for IDs because it can be `null` for new entities that have not been saved yet
6	`@Column(name = "product_name", nullable = false, length = 100)`	Maps to column `product_name`, cannot be NULL, max 100 characters
7	`@Column(nullable = false)`	Price cannot be NULL. Column name defaults to `price` (field name)
8	`@Column(length = 500)`	Description can be up to 500 characters. Can be NULL (default)
9	`@Column(name = "in_stock")`	Maps the `inStock` Java field to the `in_stock` database column
10	`= true`	Default value for new products. They are in stock unless specified otherwise

11	`public Product()`	Required! JPA needs a no-argument constructor to create entity instances via reflection
12	`public Product(String name, ...)`	Convenience constructor for creating products in your code

How the Entity Maps to a Table

```
Java Entity Class                    Database Table: products
+---------------------------+       +-------------+--------------+------+
| @Entity                   |       | Column      | Type         | Null |
| class Product {           |       +-------------+--------------+------+
|                           |       |             |              |      |
|   @Id                     | ----> | id          | BIGINT (PK)  | NO   |
|   Long id;                |       |             | AUTO_INCR    |      |
|                           |       |             |              |      |
|   @Column("product_name") | ----> | product_name| VARCHAR(100) | NO   |
|   String name;            |       |             |              |      |
|                           |       |             |              |      |
|   @Column                 | ----> | price       | DOUBLE       | NO   |
|   Double price;           |       |             |              |      |
|                           |       |             |              |      |
|   @Column(length=500)     | ----> | description | VARCHAR(500) | YES  |
|   String description;     |       |             |              |      |
|                           |       |             |              |      |
|   @Column("in_stock")     | ----> | in_stock    | BOOLEAN      | YES  |
|   Boolean inStock;        |       |             |              |      |
| }                         |       +-------------+--------------+------+
+---------------------------+
```

Understanding @GeneratedValue Strategies

```
+-------------------+-------------------------------------------+
| Strategy          | How It Works                              |
+-------------------+-------------------------------------------+
| IDENTITY          | Database auto-increments the ID.          |
|                   | Most common for H2, MySQL, PostgreSQL     |
|                   | Example: 1, 2, 3, 4, ...                  |
+-------------------+-------------------------------------------+
| SEQUENCE          | Uses a database sequence object.           |
|                   | Preferred for Oracle and PostgreSQL       |
|                   | Can pre-allocate IDs for better           |
|                   | performance                               |
+-------------------+-------------------------------------------+
| TABLE             | Uses a separate table to track IDs.       |
|                   | Works everywhere but is slower.           |
|                   | Rarely used in practice                   |
```

```
+-------------------+-------------------------------------------+
| AUTO              | Hibernate picks the best strategy for     |
|                   | your database. Convenient but less        |
|                   | predictable                               |
+-------------------+-------------------------------------------+
| UUID              | Generates a UUID (universally unique ID). |
| (Hibernate 6+)   | Example: 550e8400-e29b-41d4-a716-...      |
+-------------------+-------------------------------------------+
```

Creating More Entity Examples

A User Entity

```java
package com.example.demo.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String firstName;

    @Column(nullable = false, length = 50)
    private String lastName;

    @Column(nullable = false, unique = true, length = 100)   // Line 1
    private String email;

    @Column(nullable = false)
    private Boolean active = true;

    @Column(name = "created_at", updatable = false)          // Line 2
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist                                               // Line 3
```

```
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate                                                  // Line 4
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    // Default constructor
    public User() {
    }

    public User(String firstName, String lastName, String email) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
    }

    // Getters and setters omitted for brevity
    // (You must include them in your actual code)

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public Boolean getActive() { return active; }
    public void setActive(Boolean active) { this.active = active; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

Key annotations explained:

Line	Code	What It Does
1	`unique = true`	Creates a unique constraint on the `email` column. No two users can have the same email
2	`updatable = false`	Once `created_at` is set, it cannot be changed through JPA updates
3	`@PrePersist`	This method runs automatically before a new entity is saved to the database for the first time

4	`@PreUpdate`	This method runs automatically before an existing entity is updated in the database

---

Pre-Loading Data with data.sql

Step 4: Create data.sql

Spring Boot can automatically run a SQL script at startup to populate your database with initial data. Create a file called `data.sql` in the `src/main/resources` folder:

```sql
-- src/main/resources/data.sql

-- Insert sample products
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Laptop', 999.99, 'High-performance laptop with 16GB RAM', true);

INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Smartphone', 699.99, 'Latest model with 128GB storage', true);

INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Headphones', 149.99, 'Noise-cancelling wireless headphones', true);

INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Tablet', 449.99, '10-inch display tablet', false);

INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Smartwatch', 299.99, 'Fitness tracking smartwatch', true);

-- Insert sample users
INSERT INTO users (first_name, last_name, email, active, created_at, updated_at)
VALUES ('Alice', 'Johnson', 'alice@example.com', true, CURRENT_TIMESTAMP, CURRENT_TIMES...

INSERT INTO users (first_name, last_name, email, active, created_at, updated_at)
VALUES ('Bob', 'Smith', 'bob@example.com', true, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

INSERT INTO users (first_name, last_name, email, active, created_at, updated_at)
VALUES ('Charlie', 'Brown', 'charlie@example.com', false, CURRENT_TIMESTAMP, CURRENT_TI...
```

Important: Notice that we use `product_name` (the column name from `@Column(name = "product_name")`), not `name` (the Java field name). SQL works with column names, not Java field names.

Also note that `spring.jpa.defer-datasource-initialization=true` must be set in `application.properties` for `data.sql` to run after Hibernate creates the tables.

Using the H2 Console

Step 5: Access the H2 Console

The H2 console is a web-based database viewer that lets you inspect your tables and run SQL queries directly.

1. Start your Spring Boot application
2. Open your browser and go to `http://localhost:8080/h2-console`
3. Fill in the connection details:

```
+-----------------------------------------+
|          H2 Console Login               |
+-----------------------------------------+
| Driver Class: org.h2.Driver             |
| JDBC URL:     jdbc:h2:mem:testdb        |
| User Name:    sa                        |
| Password:     (leave empty)             |
+-----------------------------------------+
| [ Connect ]                             |
+-----------------------------------------+
```

4. Click Connect

You will see your tables listed on the left side:

```
+-----------------------------------------+
|  H2 Console                             |
+-----------------------------------------+
|  Tables:                                |
|  ├── PRODUCTS                           |
|  │   ├── ID (BIGINT, PK)               |
|  │   ├── PRODUCT_NAME (VARCHAR)         |
|  │   ├── PRICE (DOUBLE)                |
|  │   ├── DESCRIPTION (VARCHAR)         |
|  │   └── IN_STOCK (BOOLEAN)            |
|  └── USERS                             |
|      ├── ID (BIGINT, PK)               |
|      ├── FIRST_NAME (VARCHAR)           |
|      ├── LAST_NAME (VARCHAR)            |
|      ├── EMAIL (VARCHAR, UNIQUE)        |
|      ├── ACTIVE (BOOLEAN)              |
|      ├── CREATED_AT (TIMESTAMP)         |
|      └── UPDATED_AT (TIMESTAMP)         |
+-----------------------------------------+
```

You can run queries directly:

```sql
SELECT * FROM products;
SELECT * FROM users WHERE active = true;
SELECT product_name, price FROM products WHERE price > 500;
```

Seeing SQL in the Console

With `spring.jpa.show-sql=true` and `spring.jpa.properties.hibernate.format_sql=true`, you will see the generated SQL in your application console when the app starts:

```
Hibernate:
    drop table if exists products cascade
Hibernate:
    drop table if exists users cascade
Hibernate:
    create table products (
        id bigint generated by default as identity,
        description varchar(500),
        in_stock boolean,
        product_name varchar(100) not null,
        price double not null,
        primary key (id)
    )
Hibernate:
    create table users (
        id bigint generated by default as identity,
        active boolean,
        created_at timestamp(6),
        email varchar(100) not null unique,
        first_name varchar(50) not null,
        last_name varchar(50) not null,
        updated_at timestamp(6),
        primary key (id)
    )
```

This is Hibernate creating tables based on your `@Entity` classes. Notice how the Java annotations translated into SQL column definitions:

- `@Column(nullable = false)` became `not null`
- `@Column(length = 100)` became `varchar(100)`
- `@Column(unique = true)` became `unique`

- `@GeneratedValue(strategy = GenerationType.IDENTITY)` became `generated by default as identity`

Complete Project Structure

```
src/
├── main/
│   ├── java/com/example/demo/
│   │   ├── DemoApplication.java
│   │   └── entity/
│   │       ├── Product.java
│   │       └── User.java
│   └── resources/
│       ├── application.properties
│       └── data.sql
└── test/
    └── java/com/example/demo/
        └── DemoApplicationTests.java
```

DemoApplication.java:

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Startup Output

When you run the application, you will see:

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
```

```
 :: Spring Boot ::                (v3.4.1)

INFO  Starting DemoApplication...
INFO  HikariPool-1 - Starting...
INFO  HikariPool-1 - Start completed.
INFO  H2 console available at '/h2-console'. Database available at 'jdbc:h2:mem:testdb'

Hibernate:
    create table products (
        id bigint generated by default as identity,
        ...
    )

Hibernate:
    create table users (
        id bigint generated by default as identity,
        ...
    )

INFO  Started DemoApplication in 2.345 seconds
```

Java Type to SQL Type Mapping

Here is a quick reference for how Java types map to database column types:

```
+---------------------+-------------------------+
| Java Type           | SQL Type (H2)           |
+---------------------+-------------------------+
| String              | VARCHAR(255) default    |
| Integer / int       | INTEGER                 |
| Long / long         | BIGINT                  |
| Double / double     | DOUBLE                  |
| Float / float       | REAL                    |
| Boolean / boolean   | BOOLEAN                 |
| BigDecimal          | DECIMAL                 |
| LocalDate           | DATE                    |
| LocalDateTime       | TIMESTAMP               |
| LocalTime           | TIME                    |
| byte[]              | BLOB                    |
| @Lob String         | CLOB (large text)       |
| Enum (with @Enum)   | VARCHAR or INTEGER      |
+---------------------+-------------------------+
```

Common Mistakes

Mistake 1: Missing No-Argument Constructor

```java
// WRONG - No default constructor
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    // Only parameterized constructor
    public Product(String name) {
        this.name = name;
    }
    // JPA will throw: "No default constructor for entity"
}

// CORRECT - Always include a no-arg constructor
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    public Product() {}  // JPA needs this!

    public Product(String name) {
        this.name = name;
    }
}
```

Mistake 2: Using Wrong Column Names in data.sql

```sql
-- WRONG - Using Java field names instead of column names
INSERT INTO products (name, inStock) VALUES ('Laptop', true);
-- Error: Column "NAME" not found

-- CORRECT - Use the column names from @Column annotations
INSERT INTO products (product_name, in_stock) VALUES ('Laptop', true);
```

Mistake 3: Forgetting defer-datasource-initialization

```properties
# WRONG - data.sql runs BEFORE Hibernate creates tables
# This causes "Table not found" errors
spring.jpa.hibernate.ddl-auto=create-drop
# Missing: spring.jpa.defer-datasource-initialization=true

# CORRECT - data.sql runs AFTER Hibernate creates tables
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.defer-datasource-initialization=true
```

Mistake 4: Using create-drop in Production

```properties
# WRONG - ALL DATA WILL BE LOST when the app restarts!
# Never use this in production
spring.jpa.hibernate.ddl-auto=create-drop

# CORRECT for production
spring.jpa.hibernate.ddl-auto=validate
# Or
spring.jpa.hibernate.ddl-auto=none
```

Mistake 5: Wrong H2 Console JDBC URL

```
# WRONG - Using the default URL shown in H2 console
JDBC URL: jdbc:h2:~/test

# CORRECT - Must match your application.properties
JDBC URL: jdbc:h2:mem:testdb
```

Best Practices

1. Use `Long` for entity IDs: Use the wrapper class `Long` instead of primitive `long`. A new entity has a `null` ID before it is saved, which cannot be represented by `long`.
2. Always provide a no-arg constructor: JPA requires it. If you create a parameterized constructor, you must also explicitly add a no-arg constructor.
3. Use `@Column` for clarity: Even when the defaults are fine, explicitly specifying column names, nullability, and length makes your code self-documenting.
4. Use `@PrePersist` and `@PreUpdate` for timestamps: Instead of manually setting `createdAt` and `updatedAt`, use lifecycle callbacks to do it automatically.

5. Use `BigDecimal` for money: `Double` has floating-point precision issues. For financial data, always use `BigDecimal`.
6. Keep show-sql for development only: In production, SQL logging should be controlled through your logging framework, not `show-sql`.
7. Use profile-specific properties: Create `application-dev.properties` with H2 settings and `application-prod.properties` with real database settings.
8. Name your tables explicitly: Use `@Table(name = "products")` to avoid surprises when your class name does not match the table name you want.

Quick Summary

```
+-----------------------------------------+
|  Spring Data JPA + H2 Quick Reference    |
+-----------------------------------------+
|                                          |
|  DEPENDENCIES:                           |
|  - spring-boot-starter-data-jpa         |
|  - h2 (scope: runtime)                  |
|                                          |
|  KEY ANNOTATIONS:                        |
|  @Entity     -> marks a JPA entity      |
|  @Table      -> sets table name         |
|  @Id         -> primary key             |
|  @GeneratedValue -> auto-generate ID    |
|  @Column     -> column settings         |
|  @PrePersist -> before first save       |
|  @PreUpdate  -> before update           |
|                                          |
|  KEY PROPERTIES:                         |
|  ddl-auto=create-drop (dev)             |
|  ddl-auto=validate (prod)               |
|  show-sql=true (dev only)               |
|  h2.console.enabled=true (dev)          |
|  defer-datasource-initialization=true   |
|                                          |
|  H2 CONSOLE:                            |
|  URL: http://localhost:8080/h2-console  |
|  JDBC URL: jdbc:h2:mem:testdb           |
|  Username: sa | Password: (empty)       |
|                                          |
+-----------------------------------------+
```

Key Points

1. JPA is a specification for mapping Java objects to database tables. Hibernate is the implementation that Spring Boot uses by default.
2. H2 is an in-memory database perfect for development and testing. No installation needed -- just add the dependency.
3. Every entity needs `@Entity`, `@Id`, and a no-argument constructor.
4. `@GeneratedValue(strategy = GenerationType.IDENTITY)` lets the database auto-increment your primary key.
5. `@Column` lets you customize column names, nullability, length, and uniqueness.
6. `spring.jpa.hibernate.ddl-auto` controls how Hibernate manages your database schema. Use `create-drop` for development, `validate` for production.
7. `data.sql` in `src/main/resources` pre-loads data at startup. Remember to set `defer-datasource-initialization=true`.
8. The H2 console at `/h2-console` lets you visually inspect your database and run SQL queries.

Practice Questions

1. What is the difference between JPA and Hibernate? Why does Spring Boot include both?
2. Explain what `spring.jpa.hibernate.ddl-auto=create-drop` does. Why should you never use it in production?
3. Why does JPA require a no-argument constructor in entity classes?
4. What is the purpose of `spring.jpa.defer-datasource-initialization=true`? What happens without it?
5. How does `@Column(name = "product_name", nullable = false, length = 100)` translate to a database column?

Exercises

Exercise 1: Create a Book Entity

Create a `Book` entity with the following fields:

- `id` (auto-generated Long)
- `title` (required, max 200 characters)
- `author` (required, max 100 characters)
- `isbn` (unique, max 13 characters)
- `publishedYear` (integer)
- `available` (boolean, default true)

Create a `data.sql` file to insert 5 sample books. Start the application and verify the data in the H2 console.

Exercise 2: Create an Order Entity with Enum

Create an `Order` entity with:

- `id` (auto-generated)
- `customerName` (required)
- `totalAmount` (BigDecimal, required)
- `status` (enum: PENDING, PROCESSING, SHIPPED, DELIVERED)
- `createdAt` and `updatedAt` (auto-populated timestamps)

Hint: Use `@Enumerated(EnumType.STRING)` to store enum values as strings in the database.

Exercise 3: Explore Different ddl-auto Settings

Run your application four times, each time with a different `ddl-auto` setting: `create`, `create-drop`, `update`, and `validate`. Observe the console output for each setting. Document what happens and when each setting is appropriate.

What Is Next?

Now that you have entity classes and a database, you need a way to interact with the data. In the next chapter, you will learn about Repositories and CRUD Operations -- how to save, find, update, and delete data using Spring Data JPA's powerful `JpaRepository` interface without writing a single SQL query.

Chapter 15: Repositories and CRUD Operations

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what Spring Data JPA repositories are and why they are powerful
- Create a `JpaRepository` interface for any entity
- Perform CRUD operations: `save`, `findById`, `findAll`, `deleteById`
- Write derived query methods like `findByName` and `findByEmailAndActive`
- Use `@Query` for custom JPQL and native SQL queries
- Use `@Modifying` for update and delete queries
- Build a complete CRUD service and controller

Why This Chapter Matters

In the last chapter, you created entity classes that map to database tables. But having tables without the ability to read and write data is like having a filing cabinet you cannot open.

Think about a library. The books (entities) sit on shelves (tables), but you need a librarian to find books, add new ones, remove old ones, and organize them. In Spring Data JPA, that librarian is called a Repository.

The remarkable thing about Spring Data JPA repositories is how little code you need to write. You declare an interface with method names that describe what you want, and Spring generates the implementation automatically. You ask for `findByAuthor(String author)`, and Spring writes the SQL query for you. No boilerplate, no repetition, no SQL.

What Is a Repository?

A repository is an interface that provides methods to interact with the database for a specific entity. Spring Data JPA provides several levels of repository interfaces:

```
+-----------------------------------------------------+
|              Repository<T, ID>                      |
|              (Marker interface, no methods)          |
+-------------------------+---------------------------+
                          |
                          v
+-----------------------------------------------------+
|           CrudRepository<T, ID>                     |
|  save(), findById(), findAll(), delete(),           |
|  count(), existsById()                              |
+-------------------------+---------------------------+
                          |
                          v
+-----------------------------------------------------+
|        ListCrudRepository<T, ID>                    |
|  Same as CrudRepository but returns List            |
|  instead of Iterable                                |
+-------------------------+---------------------------+
                          |
                          v
+-----------------------------------------------------+
|         PagingAndSortingRepository<T, ID>           |
|  findAll(Sort), findAll(Pageable)                   |
+-------------------------+---------------------------+
                          |
                          v
+-----------------------------------------------------+
|           JpaRepository<T, ID>                      |
|  flush(), saveAndFlush(), deleteInBatch(),          |
|  findAll(Example), getReferenceById()               |
|  + ALL methods from above interfaces                |
+-----------------------------------------------------+
```

JpaRepository is the most commonly used because it includes everything. When in doubt, extend `JpaRepository`.

Creating Your First Repository

The Entity (from Chapter 14)

Let us use the `Product` entity from the previous chapter:

```java
package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;

    @Column(nullable = false)
    private Double price;

    @Column(length = 500)
    private String description;

    @Column(name = "in_stock")
    private Boolean inStock = true;

    public Product() {}

    public Product(String name, Double price, String description) {
        this.name = name;
        this.price = price;
        this.description = description;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public Boolean getInStock() { return inStock; }
    public void setInStock(Boolean inStock) { this.inStock = inStock; }

    @Override
    public String toString() {
        return "Product{id=" + id + ", name='" + name + "', price=" + price + "}";
    }
}
```

The Repository Interface

```java
package com.example.demo.repository;

import com.example.demo.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository                                                    // Line 1
public interface ProductRepository                             // Line 2
        extends JpaRepository<Product, Long> {                 // Line 3
    // That's it! No method implementations needed.
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Repository`	Marks this as a repository bean. Optional for interfaces extending `JpaRepository` (Spring detects them automatically), but it is good practice for clarity
2	`interface ProductRepository`	This is an interface, not a class. You do not write any implementation. Spring generates one for you at runtime
3	`JpaRepository<Product, Long>`	The first type parameter (`Product`) is the entity class. The second (`Long`) is the type of the entity's `@Id` field

With just these three lines, you get all of these methods for free:

```
+-----------------------------------+----------------------------------------+
| Method                            | What It Does                           |
+-----------------------------------+----------------------------------------+
| save(Product p)                   | Insert new or update existing          |
| saveAll(List<Product> list)       | Save multiple products at once         |
| findById(Long id)                 | Find one product by ID                 |
| findAll()                         | Get all products                       |
| findAllById(List<Long> ids)       | Find multiple products by IDs          |
| count()                           | Count total products                   |
| existsById(Long id)               | Check if a product exists              |
| deleteById(Long id)               | Delete a product by ID                 |
| delete(Product p)                 | Delete a product by entity             |
| deleteAll()                       | Delete all products                    |
| flush()                           | Force pending changes to database      |
| saveAndFlush(Product p)           | Save and immediately flush             |
```

```
+----------------------------------+--------------------------------------+
```

CRUD Operations in Detail

Create (Save)

```java
// Saving a NEW product (id is null, so JPA does an INSERT)
Product laptop = new Product("Laptop", 999.99, "High-performance laptop");
Product saved = productRepository.save(laptop);
System.out.println("Saved: " + saved.getId());  // Saved: 1

// Saving MULTIPLE products
List<Product> products = List.of(
    new Product("Phone", 699.99, "Smartphone"),
    new Product("Tablet", 449.99, "10-inch tablet")
);
List<Product> savedProducts = productRepository.saveAll(products);
```

Generated SQL:

```sql
Hibernate:
    insert into products (description, in_stock, product_name, price)
    values (?, ?, ?, ?)
```

Read (Find)

```java
// Find by ID (returns Optional)
Optional<Product> optionalProduct = productRepository.findById(1L);  // Line 1

if (optionalProduct.isPresent()) {                                   // Line 2
    Product product = optionalProduct.get();                         // Line 3
    System.out.println("Found: " + product.getName());
} else {
    System.out.println("Product not found");
}

// Shorter version using orElseThrow
Product product = productRepository.findById(1L)                     // Line 4
    .orElseThrow(() -> new RuntimeException("Product not found"));
```

```
// Find all products
List<Product> allProducts = productRepository.findAll();             // Line 5
System.out.println("Total products: " + allProducts.size());

// Check if exists
boolean exists = productRepository.existsById(1L);                  // Line 6
System.out.println("Exists: " + exists);  // Exists: true

// Count all
long count = productRepository.count();                              // Line 7
System.out.println("Total: " + count);
```

Line-by-line explanation:

Line	Code	What It Does
1	`findById(1L)`	Looks for a product with ID 1. Returns `Optional<Product>`, not `Product` directly
2	`isPresent()`	Checks if a product was found. Avoids `NullPointerException`
3	`optionalProduct.get()`	Extracts the actual `Product` from the `Optional`
4	`orElseThrow(...)`	Gets the product or throws an exception if not found. Cleaner than if-else
5	`findAll()`	Returns a `List<Product>` containing all products in the table
6	`existsById(1L)`	Returns `true` or `false`. More efficient than `findById` when you only need to check existence
7	`count()`	Returns the total number of rows in the table

Generated SQL for findById:

```sql
Hibernate:
    select p1_0.id, p1_0.description, p1_0.in_stock,
           p1_0.product_name, p1_0.price
    from products p1_0
    where p1_0.id=?
```

Update (Save with Existing ID)

```java
// Step 1: Find the existing product
Product product = productRepository.findById(1L)
    .orElseThrow(() -> new RuntimeException("Not found"));

// Step 2: Modify the fields
product.setPrice(899.99);
product.setDescription("Updated description");

// Step 3: Save again (id is NOT null, so JPA does an UPDATE)
Product updated = productRepository.save(product);
System.out.println("Updated price: " + updated.getPrice());  // 899.99
```

Generated SQL:

```sql
Hibernate:
    update products
    set description=?, in_stock=?, product_name=?, price=?
    where id=?
```

How does `save()` know to INSERT or UPDATE?

```
+---------------------------+
|     save(entity)          |
|         |                 |
|    Is entity.id null?     |
|      /          \         |
|    YES            NO      |
|     |              |      |
|   INSERT        UPDATE    |
|  (new row)    (existing)  |
+---------------------------+
```

Delete

```java
// Delete by ID
productRepository.deleteById(1L);

// Delete by entity
Product product = productRepository.findById(2L)
    .orElseThrow(() -> new RuntimeException("Not found"));
productRepository.delete(product);

// Delete all
productRepository.deleteAll();
```

Generated SQL:

```sql
Hibernate:
    delete from products where id=?
```

Derived Query Methods

One of the most powerful features of Spring Data JPA is derived query methods. You write a method name that describes what you want, and Spring generates the SQL automatically.

How It Works

```
Method name:  findByNameAndPriceGreaterThan(String name, Double price)
                |    |         |
                |    |         +-- Condition: price > ?
                |    +------------ Field: name = ?
                +----------------- Action: SELECT ... WHERE
```

Spring parses the method name and generates the query. Think of it as writing a query in plain English.

Adding Derived Methods to ProductRepository

```java
package com.example.demo.repository;

import com.example.demo.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Find products by exact name
    List<Product> findByName(String name);                    // Line 1

    // Find products by stock status
```

```
    List<Product> findByInStock(Boolean inStock);             // Line 2

    // Find products cheaper than a given price
    List<Product> findByPriceLessThan(Double price);          // Line 3

    // Find products within a price range
    List<Product> findByPriceBetween(
        Double minPrice, Double maxPrice);                    // Line 4

    // Find products by name containing a keyword (LIKE %keyword%)
    List<Product> findByNameContainingIgnoreCase(
        String keyword);                                      // Line 5

    // Find products by name AND in stock
    List<Product> findByNameAndInStock(
        String name, Boolean inStock);                        // Line 6

    // Find products by name OR description containing keyword
    List<Product> findByNameContainingOrDescriptionContaining(
        String nameKeyword, String descKeyword);              // Line 7

    // Find products ordered by price (ascending)
    List<Product> findByInStockOrderByPriceAsc(
        Boolean inStock);                                     // Line 8

    // Find products ordered by price (descending)
    List<Product> findByInStockOrderByPriceDesc(
        Boolean inStock);                                     // Line 9

    // Count products by stock status
    long countByInStock(Boolean inStock);                     // Line 10

    // Check if a product exists by name
    boolean existsByName(String name);                        // Line 11

    // Find first product by name
    Optional<Product> findFirstByName(String name);           // Line 12

    // Find top 3 cheapest products
    List<Product> findTop3ByOrderByPriceAsc();                // Line 13

    // Delete products by stock status
    void deleteByInStock(Boolean inStock);                    // Line 14
}
```

Line-by-line explanation:

Line	Method	Generated SQL (simplified)				

1	`findByName(name)`	`SELECT * FROM products WHERE product_name = ?`				
2	`findByInStock(inStock)`	`SELECT * FROM products WHERE in_stock = ?`				
3	`findByPriceLessThan(price)`	`SELECT * FROM products WHERE price < ?`				
4	`findByPriceBetween(min, max)`	`SELECT * FROM products WHERE price BETWEEN ? AND ?`				
5	`findByNameContainingIgnoreCase(kw)`	`SELECT * FROM products WHERE LOWER(product_name) LIKE LOWER('%'		?		'%')`
6	`findByNameAndInStock(name, stock)`	`SELECT * FROM products WHERE product_name = ? AND in_stock = ?`				

7	`findByNameContaining...`	`SELECT * FROM products WHERE product_name LIKE ? OR description LIKE ?`				
8	`findByInStockOrderByPriceAsc(stock)`	`SELECT * FROM products WHERE in_stock = ? ORDER BY price ASC`				
9	`findByInStockOrderByPriceDesc(stock)`	`SELECT * FROM products WHERE in_stock = ? ORDER BY price DESC`				
10	`countByInStock(stock)`	`SELECT COUNT(*) FROM products WHERE in_stock = ?`				
11	`existsByName(name)`	`SELECT COUNT(*) > 0 FROM products WHERE product_name = ?`				
12	`findFirstByName(name)`	`SELECT * FROM products WHERE product_name = ? LIMIT 1`				

13	`findTop3ByOrderByPriceAsc()`	`SELECT * FROM products ORDER BY price ASC LIMIT 3`				
14	`deleteByInStock(stock)`	`DELETE FROM products WHERE in_stock = ?`				

Derived Query Keywords Reference

```
+-----------------------+-----------------------------+
| Keyword               | SQL Equivalent              |
+-----------------------+-----------------------------+
| findBy                | SELECT ... WHERE            |
| countBy               | SELECT COUNT(*) WHERE       |
| existsBy              | SELECT COUNT(*) > 0 WHERE   |
| deleteBy              | DELETE ... WHERE            |
+-----------------------+-----------------------------+
| And                   | AND                         |
| Or                    | OR                          |
+-----------------------+-----------------------------+
| Is, Equals            | = (default, can be omitted) |
| Not                   | <>                          |
| IsNull                | IS NULL                     |
| IsNotNull             | IS NOT NULL                 |
| LessThan              | <                           |
| LessThanEqual         | <=                          |
| GreaterThan           | >                           |
| GreaterThanEqual      | >=                          |
| Between               | BETWEEN ? AND ?             |
| Like                  | LIKE                        |
| Containing            | LIKE '%?%'                  |
| StartingWith          | LIKE '?%'                   |
| EndingWith            | LIKE '%?'                   |
| IgnoreCase            | LOWER(...) = LOWER(?)       |
| In                    | IN (?, ?, ?)                |
| NotIn                 | NOT IN (?, ?, ?)            |
| True                  | = TRUE                      |
| False                 | = FALSE                     |
| OrderBy...Asc         | ORDER BY ... ASC            |
| OrderBy...Desc        | ORDER BY ... DESC           |
| First, Top            | LIMIT 1 / LIMIT N           |
+-----------------------+-----------------------------+
```

Custom Queries with @Query

Sometimes derived query methods get too long or you need a query that cannot be expressed through method naming. In these cases, use `@Query`.

JPQL Queries (Java Persistence Query Language)

JPQL is like SQL but uses entity class names and field names instead of table and column names.

```java
package com.example.demo.repository;

import com.example.demo.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // JPQL query using positional parameter
    @Query("SELECT p FROM Product p WHERE p.price > ?1")     // Line 1
    List<Product> findExpensiveProducts(Double minPrice);

    // JPQL query using named parameter
    @Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword%") // Line 2
    List<Product> searchByName(
        @Param("keyword") String keyword);                    // Line 3

    // JPQL query with multiple conditions
    @Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max "
         + "AND p.inStock = true ORDER BY p.price ASC")       // Line 4
    List<Product> findAvailableInPriceRange(
        @Param("min") Double minPrice,
        @Param("max") Double maxPrice);

    // JPQL query returning specific fields
    @Query("SELECT p.name, p.price FROM Product p "
         + "WHERE p.inStock = true")                          // Line 5
    List<Object[]> findProductNameAndPrice();

    // Native SQL query
```

```
    @Query(value = "SELECT * FROM products WHERE price > :price",
           nativeQuery = true)                                  // Line 6
    List<Product> findExpensiveProductsNative(
        @Param("price") Double price);

    // Count query
    @Query("SELECT COUNT(p) FROM Product p WHERE p.inStock = :inStock")
    long countByStockStatus(@Param("inStock") Boolean inStock); // Line 7
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`?1`	Positional parameter. `?1` refers to the first method parameter, `?2` to the second, etc.
2	`%:keyword%`	Named parameter with LIKE wildcards. Searches for products containing the keyword
3	`@Param("keyword")`	Maps the method parameter to the named parameter `:keyword` in the query
4	`BETWEEN :min AND :max`	Range query with two named parameters
5	`SELECT p.name, p.price`	Returns only specific fields instead of the full entity. Result is `List<Object[]>` where each array has two elements
6	`nativeQuery = true`	Uses raw SQL instead of JPQL. Notice it uses table name `products` and column name `price` instead of entity name `Product` and field name
7	`SELECT COUNT(p)`	JPQL aggregate function to count matching records

JPQL vs Native SQL

```
+------------------------+-------------------------------------------+
| Feature                | JPQL                  | Native SQL        |
+------------------------+-------------------------------------------+
```

```
| Uses                    | Entity/field names   | Table/column names  |
| Database portable       | Yes                  | No (DB-specific)    |
| Example                 | SELECT p FROM        | SELECT * FROM       |
|                         | Product p            | products            |
| Annotation              | @Query("...")         | @Query(value="...", |
|                         |                      |  nativeQuery=true)  |
| Best for                | Most queries         | Complex SQL,        |
|                         |                      | DB-specific features|
+-------------------------+-----------------------------------------+
```

@Modifying for UPDATE and DELETE Queries

By default, `@Query` methods are read-only (SELECT). To run UPDATE or DELETE queries, you need `@Modifying` and `@Transactional`.

```java
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Update query
    @Modifying                                                  // Line 1
    @Transactional                                              // Line 2
    @Query("UPDATE Product p SET p.price = :newPrice "
         + "WHERE p.name = :name")                              // Line 3
    int updatePriceByName(
        @Param("name") String name,
        @Param("newPrice") Double newPrice);                    // Line 4

    // Delete query
    @Modifying
    @Transactional
    @Query("DELETE FROM Product p WHERE p.inStock = false")     // Line 5
    int deleteOutOfStockProducts();                             // Line 6

    // Bulk update
    @Modifying
    @Transactional
    @Query("UPDATE Product p SET p.inStock = :status "
         + "WHERE p.price > :price")
    int updateStockStatusByPrice(
        @Param("status") Boolean status,
        @Param("price") Double price);
}
```

Line-by-line explanation:

Line	Code	What It Does

1	`@Modifying`	Tells Spring this query modifies data (not a SELECT). Required for UPDATE and DELETE queries
2	`@Transactional`	Wraps the query in a database transaction. Required for modifying queries
3	`UPDATE Product p SET...`	JPQL update statement. Uses entity name and field names
4	`int` return type	Returns the number of rows affected by the update
5	`DELETE FROM Product p`	JPQL delete statement
6	`int` return type	Returns the number of rows deleted

Building a Complete CRUD Service

Now let us put everything together into a proper service layer.

Why a Service Layer?

```
+------------------+     +------------------+     +------------------+
|    Controller    | --> |     Service      | --> |    Repository    |
|                  |     |                  |     |                  |
| Handles HTTP     |     | Business logic   |     | Database access  |
| requests/        |     | Validation       |     | CRUD operations  |
| responses        |     | Transformations  |     | Query execution  |
+------------------+     +------------------+     +------------------+
```

The service layer sits between the controller and the repository. It contains business logic and keeps your controllers thin.

ProductService

```java
package com.example.demo.service;

import com.example.demo.entity.Product;
```

```
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service                                                        // Line 1
public class ProductService {

    private final ProductRepository productRepository;        // Line 2

    public ProductService(ProductRepository productRepository) { // Line 3
        this.productRepository = productRepository;
    }

    // CREATE
    public Product createProduct(Product product) {           // Line 4
        return productRepository.save(product);
    }

    // READ - Get all
    public List<Product> getAllProducts() {                    // Line 5
        return productRepository.findAll();
    }

    // READ - Get by ID
    public Product getProductById(Long id) {                  // Line 6
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found with id: " + id));
    }

    // UPDATE
    public Product updateProduct(Long id, Product productDetails) { // Line 7
        Product existingProduct = getProductById(id);         // Line 8

        existingProduct.setName(productDetails.getName());    // Line 9
        existingProduct.setPrice(productDetails.getPrice());
        existingProduct.setDescription(
            productDetails.getDescription());
        existingProduct.setInStock(productDetails.getInStock());

        return productRepository.save(existingProduct);       // Line 10
    }

    // DELETE
    public void deleteProduct(Long id) {                      // Line 11
        Product product = getProductById(id);                 // Line 12
        productRepository.delete(product);
    }

    // SEARCH
    public List<Product> searchProducts(String keyword) {     // Line 13
        return productRepository
            .findByNameContainingIgnoreCase(keyword);
```

```
    }

    // FILTER by price range
    public List<Product> getProductsByPriceRange(
            Double minPrice, Double maxPrice) {
        return productRepository
            .findByPriceBetween(minPrice, maxPrice);
    }

    // FILTER by stock status
    public List<Product> getInStockProducts() {
        return productRepository.findByInStock(true);
    }

    // COUNT
    public long getProductCount() {
        return productRepository.count();
    }

    // BULK UPDATE
    @Transactional                                          // Line 14
    public int markExpensiveProductsOutOfStock(Double priceThreshold) {
        return productRepository
            .updateStockStatusByPrice(false, priceThreshold);
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Service`	Marks this class as a service component. Spring creates a single instance and manages it
2	`private final ProductRepository`	Declares the repository dependency as `final` for immutability
3	Constructor injection	Spring automatically injects the `ProductRepository` bean
4	`createProduct(...)`	Saves a new product. The returned product has a generated ID
5	`getAllProducts()`	Returns all products from the database
6	`getProductById(...)`	Finds a product by ID or throws an exception if not found

7	`updateProduct(...)`	Accepts the ID and new data to update
8	`getProductById(id)`	First, find the existing product (throws if not found)
9	`existingProduct.setName(...)`	Update each field with new values
10	`productRepository.save(...)`	Save the modified entity. Since it has an ID, JPA does an UPDATE
11	`deleteProduct(...)`	Deletes a product by ID
12	`getProductById(id)`	Verify the product exists before deleting. Gives a clear error message if not found
13	`searchProducts(...)`	Uses the derived query method for searching by name
14	`@Transactional`	Ensures the bulk update runs in a single database transaction

ProductController

```java
package com.example.demo.controller;

import com.example.demo.entity.Product;
import com.example.demo.service.ProductService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // GET /api/products
    @GetMapping
```

```
    public ResponseEntity<List<Product>> getAllProducts() {
        List<Product> products = productService.getAllProducts();
        return ResponseEntity.ok(products);
    }

    // GET /api/products/5
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(
            @PathVariable Long id) {
        Product product = productService.getProductById(id);
        return ResponseEntity.ok(product);
    }

    // POST /api/products
    @PostMapping
    public ResponseEntity<Product> createProduct(
            @RequestBody Product product) {
        Product created = productService.createProduct(product);
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(created);
    }

    // PUT /api/products/5
    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable Long id,
            @RequestBody Product product) {
        Product updated = productService.updateProduct(id, product);
        return ResponseEntity.ok(updated);
    }

    // DELETE /api/products/5
    @DeleteMapping("/{id}")
    public ResponseEntity<Map<String, String>> deleteProduct(
            @PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.ok(
            Map.of("message", "Product deleted successfully"));
    }

    // GET /api/products/search?keyword=laptop
    @GetMapping("/search")
    public ResponseEntity<List<Product>> searchProducts(
            @RequestParam String keyword) {
        List<Product> products = productService.searchProducts(keyword);
        return ResponseEntity.ok(products);
    }

    // GET /api/products/price-range?min=100&max=500
    @GetMapping("/price-range")
    public ResponseEntity<List<Product>> getProductsByPriceRange(
            @RequestParam Double min,
            @RequestParam Double max) {
        List<Product> products =
```

```
            productService.getProductsByPriceRange(min, max);
        return ResponseEntity.ok(products);
    }

    // GET /api/products/in-stock
    @GetMapping("/in-stock")
    public ResponseEntity<List<Product>> getInStockProducts() {
        List<Product> products = productService.getInStockProducts();
        return ResponseEntity.ok(products);
    }

    // GET /api/products/count
    @GetMapping("/count")
    public ResponseEntity<Map<String, Long>> getProductCount() {
        long count = productService.getProductCount();
        return ResponseEntity.ok(Map.of("count", count));
    }
}
```

Testing the Complete CRUD API

Create a product:

```
POST http://localhost:8080/api/products
Content-Type: application/json

{
    "name": "Mechanical Keyboard",
    "price": 129.99,
    "description": "RGB mechanical keyboard",
    "inStock": true
}
```

Response (201 Created):

```json
{
    "id": 6,
    "name": "Mechanical Keyboard",
    "price": 129.99,
    "description": "RGB mechanical keyboard",
    "inStock": true
}
```

Get all products:

```
GET http://localhost:8080/api/products
```

Response (200 OK):

```
[
    {"id": 1, "name": "Laptop", "price": 999.99, "description": "High-performance lapto...
    {"id": 2, "name": "Smartphone", "price": 699.99, "description": "Latest model", "in...
    {"id": 3, "name": "Headphones", "price": 149.99, "description": "Noise-cancelling",...
    {"id": 4, "name": "Tablet", "price": 449.99, "description": "10-inch display", "inS...
    {"id": 5, "name": "Smartwatch", "price": 299.99, "description": "Fitness tracking",...
]
```

Update a product:

```
PUT http://localhost:8080/api/products/1
Content-Type: application/json

{
    "name": "Laptop Pro",
    "price": 1299.99,
    "description": "Pro laptop with M3 chip",
    "inStock": true
}
```

Response (200 OK):

```
{
    "id": 1,
    "name": "Laptop Pro",
    "price": 1299.99,
    "description": "Pro laptop with M3 chip",
    "inStock": true
}
```

Delete a product:

```
DELETE http://localhost:8080/api/products/4
```

Response (200 OK):

```
{
    "message": "Product deleted successfully"
}
```

Search products:

```
GET http://localhost:8080/api/products/search?keyword=phone
```

Response (200 OK):

```json
[
    {"id": 2, "name": "Smartphone", "price": 699.99, "description": "Latest model", "in...
    {"id": 3, "name": "Headphones", "price": 149.99, "description": "Noise-cancelling",...
]
```

Complete Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── controller/
│   └── ProductController.java
├── entity/
│   └── Product.java
├── repository/
│   └── ProductRepository.java
└── service/
    └── ProductService.java

src/main/resources/
├── application.properties
└── data.sql
```

Common Mistakes

Mistake 1: Writing a Class Instead of an Interface

```java
// WRONG - Repository must be an interface
public class ProductRepository extends JpaRepository<Product, Long> {
    // This will not compile!
}

// CORRECT - Repository must be an interface
```

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Spring generates the implementation
}
```

Mistake 2: Wrong Field Names in Derived Queries

```java
// WRONG - Using column name instead of Java field name
List<Product> findByProduct_name(String name);
// Field is called "name" in Java, not "product_name"

// CORRECT - Use the Java field name
List<Product> findByName(String name);
```

Mistake 3: Missing @Transactional for Modifying Queries

```java
// WRONG - Will throw TransactionRequiredException
@Modifying
@Query("UPDATE Product p SET p.price = :price WHERE p.id = :id")
int updatePrice(@Param("id") Long id, @Param("price") Double price);

// CORRECT - Add @Transactional
@Modifying
@Transactional
@Query("UPDATE Product p SET p.price = :price WHERE p.id = :id")
int updatePrice(@Param("id") Long id, @Param("price") Double price);
```

Mistake 4: Not Handling Optional Properly

```java
// WRONG - Can throw NoSuchElementException
Product product = productRepository.findById(999L).get();

// CORRECT - Handle the empty case
Product product = productRepository.findById(999L)
    .orElseThrow(() -> new RuntimeException("Product not found"));

// Also CORRECT - Check first
Optional<Product> optional = productRepository.findById(999L);
if (optional.isPresent()) {
    Product product = optional.get();
}
```

Mistake 5: Using Entity Directly as Request/Response

While we use entities directly in this chapter for simplicity, in production you should use DTOs (Data Transfer Objects) to decouple your API from your database schema. We will address this in later chapters.

Best Practices

1. Always use the service layer: Do not call the repository directly from the controller. The service layer handles business logic and validation.
2. Use `Optional` properly: `findById` returns `Optional`. Always handle the empty case with `orElseThrow`, `orElse`, or `isPresent`.
3. Prefer derived query methods for simple queries: They are self-documenting and type-safe. Use `@Query` only when the method name would be too long.
4. Use `@Param` with named parameters: Named parameters are more readable than positional parameters (`?1`, `?2`).
5. Return `ResponseEntity`: Use `ResponseEntity` in controllers to control HTTP status codes and headers.
6. Verify existence before delete: Check that the entity exists before deleting. This gives users a clear error message instead of a silent no-op.
7. Use constructor injection: Always inject repositories through the constructor, not with `@Autowired` on fields.
8. Keep repositories focused: Each repository should manage exactly one entity. Do not add methods for multiple entities in a single repository.

Quick Summary

```
+-----------------------------------------+
|  Repositories & CRUD Quick Reference     |
+-----------------------------------------+
|                                         |
|  INTERFACE:                             |
|  JpaRepository<Entity, IdType>          |
|                                         |
|  BUILT-IN METHODS:                      |
|  save()       -> INSERT or UPDATE       |
|  findById()   -> SELECT by ID (Optional) |
|  findAll()    -> SELECT all             |
```

```
|  deleteById() -> DELETE by ID            |
|  count()      -> COUNT all              |
|  existsById() -> Boolean check           |
|                                          |
|  DERIVED QUERIES:                        |
|  findByField(value)                      |
|  findByFieldAndField(v1, v2)             |
|  findByFieldContaining(keyword)          |
|  findByFieldGreaterThan(value)           |
|  findByFieldOrderByFieldAsc()            |
|                                          |
|  CUSTOM QUERIES:                         |
|  @Query("SELECT p FROM Product p ...")   |
|  @Query(value="...", nativeQuery=true)   |
|                                          |
|  MODIFYING:                              |
|  @Modifying + @Transactional + @Query    |
|                                          |
+------------------------------------------+
```

Key Points

1. `JpaRepository<Product, Long>` gives you a full set of CRUD methods without writing any implementation code.
2. `save()` performs an INSERT when the entity ID is `null` and an UPDATE when the ID exists.
3. `findById()` returns `Optional<T>` -- always handle the empty case.
4. Derived query methods like `findByNameAndPriceLessThan` are parsed by Spring and converted to SQL automatically.
5. Use `@Query` for complex queries that cannot be expressed through method naming. JPQL uses entity names; native SQL uses table names.
6. `@Modifying` and `@Transactional` are required for UPDATE and DELETE queries.
7. The service layer contains business logic and sits between the controller and repository.

Practice Questions

1. What is the difference between `CrudRepository` and `JpaRepository`? Why would you choose one over the other?

2. How does `save()` decide whether to INSERT or UPDATE? What determines this behavior?
3. Write a derived query method to find all users whose email ends with `@gmail.com` and who are active, ordered by last name.
4. What is the difference between JPQL and native SQL in a `@Query` annotation? When would you use native SQL?
5. Why do you need both `@Modifying` and `@Transactional` for UPDATE queries?

Exercises

Exercise 1: Build a Complete User CRUD

Using the `User` entity from Chapter 14, create:

- `UserRepository` with derived methods: `findByEmail`, `findByActiveTrue`, `findByLastNameContaining`
- `UserService` with full CRUD operations and a method to deactivate a user
- `UserController` with endpoints for all CRUD operations and search

Test all endpoints with Postman or curl.

Exercise 2: Library Management System

Create a `Book` entity with fields: `id`, `title`, `author`, `isbn`, `genre`, `publishedYear`, `available`. Build a complete repository with these query methods:

- Find books by author
- Find books by genre
- Find available books published after a given year
- Search books by title keyword
- Count books by genre
- A `@Query` method to find the most expensive book in each genre

Exercise 3: Custom Query Challenge

Write these queries using `@Query`:

- Find the average price of all products
- Find all products whose name starts with a given letter

- Update all products to be in stock where the price is less than a threshold
- Find the top 5 most expensive in-stock products

What Is Next?

Now that you can perform CRUD operations, what happens when your database has thousands or millions of records? Returning all of them at once would be slow and wasteful. In the next chapter, you will learn about Pagination and Sorting -- how to return data in manageable chunks and in the order the user wants.

Chapter 16: Pagination and Sorting

What You Will Learn

By the end of this chapter, you will be able to:

- Understand why pagination and sorting are essential for any API
- Use `Pageable` and `PageRequest` to paginate query results
- Understand the difference between `Page` and `Slice`
- Sort data using the `Sort` class
- Auto-resolve `Pageable` parameters from HTTP request parameters
- Use `Page.map()` to transform paginated results
- Build complete paginated REST endpoints

Why This Chapter Matters

Imagine you walk into a library and ask the librarian: "Give me every book you have." The librarian stares at you. The library has 50,000 books. Should they stack all 50,000 on the counter?

Of course not. Instead, the librarian would say: "Here are the first 20 books. When you are done, come back for the next 20."

This is exactly what pagination does for your API. Without pagination, a `GET /api/products` call on a database with 100,000 products would:

- Load all 100,000 records into memory
- Serialize them all into JSON
- Send a massive response over the network
- Make the browser or client process all of them at once

With pagination, you return 20 or 50 records at a time, along with metadata like "page 3 of 500." The client asks for exactly the data it needs.

Sorting is equally important. Users want to see products from cheapest to most expensive, or newest to oldest. Without sorting, data comes back in whatever order the database feels like returning it.

How Pagination Works

```
Database has 50 products
Page size = 10

+--------+-------------------------------------------+
| Page 0 | Products 1-10                             |   <-- First page
+--------+-------------------------------------------+
| Page 1 | Products 11-20                            |
+--------+-------------------------------------------+
| Page 2 | Products 21-30                            |
+--------+-------------------------------------------+
| Page 3 | Products 31-40                            |
+--------+-------------------------------------------+
| Page 4 | Products 41-50                            |   <-- Last page
+--------+-------------------------------------------+

GET /api/products?page=0&size=10   ->  Returns products 1-10
GET /api/products?page=2&size=10   ->  Returns products 21-30
GET /api/products?page=4&size=10   ->  Returns products 41-50
```

Important: Pages are zero-indexed. The first page is page 0, not page 1.

The Pageable Interface

`Pageable` is Spring Data's way of capturing pagination and sorting information. It holds three pieces of data:

```
+-------------------------------------+
|           Pageable                  |
+-------------------------------------+
| page number  (which page?)          |
| page size    (how many per page?)   |
| sort         (what order?)          |
+-------------------------------------+
```

Creating Pageable with PageRequest

`PageRequest` is the most common way to create a `Pageable` object:

```java
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;

// Page 0 (first page), 10 items per page
Pageable pageable1 = PageRequest.of(0, 10);                  // Line 1

// Page 2 (third page), 20 items per page
Pageable pageable2 = PageRequest.of(2, 20);                  // Line 2

// Page 0, 10 items, sorted by price ascending
Pageable pageable3 = PageRequest.of(0, 10,
    Sort.by("price").ascending());                            // Line 3

// Page 0, 10 items, sorted by price descending
Pageable pageable4 = PageRequest.of(0, 10,
    Sort.by("price").descending());                           // Line 4

// Page 0, 10 items, sorted by multiple fields
Pageable pageable5 = PageRequest.of(0, 10,
    Sort.by("inStock").descending()
         .and(Sort.by("price").ascending()));                 // Line 5
```

Line-by-line explanation:

Line	Code	What It Does
1	`PageRequest.of(0, 10)`	First page, 10 items, no sorting
2	`PageRequest.of(2, 20)`	Third page (zero-indexed), 20 items per page
3	`Sort.by("price").ascending()`	Sort by the `price` field in ascending order (cheapest first)
4	`Sort.by("price").descending()`	Sort by price descending (most expensive first)
5	`.and(Sort.by(...))`	Sort by `inStock` first (in-stock items first), then by price within each group

Page vs Slice

Spring Data provides two return types for paginated queries: `Page` and `Slice`.

Page

`Page` extends `Slice` and adds a total count. It tells you exactly how many pages and total elements exist.

```
+-------------------------------------------+
|                Page<Product>              |
+-------------------------------------------+
| content      : List<Product>  (the data)  |
| number       : 2              (page num)  |
| size         : 10             (page size) |
| totalElements: 50             (total rows)|
| totalPages   : 5              (total pgs) |
| first        : false          (is first?) |
| last         : false          (is last?)  |
| hasNext      : true           (more pages)|
| hasPrevious  : true           (prev page) |
+-------------------------------------------+
```

SQL generated for Page:

```sql
-- Query 1: Get the data
SELECT * FROM products ORDER BY id LIMIT 10 OFFSET 20;

-- Query 2: Count total rows (extra query!)
SELECT COUNT(*) FROM products;
```

Slice

`Slice` does not count the total. It only knows if there is a next page, not how many total pages exist.

```
+-------------------------------------------+
|               Slice<Product>             |
+-------------------------------------------+
| content      : List<Product>  (the data)  |
| number       : 2              (page num)  |
| size         : 10             (page size) |
| first        : false          (is first?) |
| last         : false          (is last?)  |
| hasNext      : true           (more pages)|
```

```
| hasPrevious  : true           (prev page) |
|                                           |
| NO totalElements                          |
| NO totalPages                             |
+-------------------------------------------+
```

SQL generated for Slice:

```sql
-- Only one query! Requests size+1 items to check if next page exists
SELECT * FROM products ORDER BY id LIMIT 11 OFFSET 20;
```

When to Use Which

```
+-------------------+-----------------------------------------------+
| Use Page when     | Use Slice when                                |
+-------------------+-----------------------------------------------+
| You need to show  | You have infinite scroll (like social media   |
| "Page 3 of 50"    | feeds) and only need "Load More" buttons      |
+-------------------+-----------------------------------------------+
| You need total    | The total count query is too expensive        |
| count for UI      | (millions of rows)                            |
+-------------------+-----------------------------------------------+
| Dataset is        | Dataset is very large and counting all rows   |
| moderate in size  | would slow down the response                  |
+-------------------+-----------------------------------------------+
```

Using Pagination in Repositories

Adding Pageable to Repository Methods

Your `JpaRepository` already inherits a paginated `findAll` method. You can also add `Pageable` to your custom methods:

```java
package com.example.demo.repository;

import com.example.demo.entity.Product;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Built-in from JpaRepository:
    // Page<Product> findAll(Pageable pageable);              // Line 1

    // Paginated derived query
    Page<Product> findByInStock(
        Boolean inStock, Pageable pageable);                  // Line 2

    // Paginated search
    Page<Product> findByNameContainingIgnoreCase(
        String keyword, Pageable pageable);                   // Line 3

    // Paginated with price filter
    Page<Product> findByPriceGreaterThan(
        Double price, Pageable pageable);                     // Line 4

    // Slice for infinite scroll
    Slice<Product> findByPriceLessThan(
        Double price, Pageable pageable);                     // Line 5

    // Paginated custom JPQL query
    @Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max")
    Page<Product> findByPriceRange(
        @Param("min") Double min,
        @Param("max") Double max,
        Pageable pageable);                                   // Line 6
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`findAll(Pageable)`	Already inherited from `JpaRepository`. Returns a `Page` of all products
2	`findByInStock(..., Pageable)`	Adds pagination to a derived query. Just add `Pageable` as the last parameter
3	`findByNameContainingIgnoreCase(..., Pageable)`	Paginated search by name. The `Pageable` also handles sorting
4	`findByPriceGreaterThan(..., Pageable)`	Filter + paginate. Returns a `Page`

5	`findByPriceLessThan(..., Pageable)`	Returns a `Slice` instead of `Page` (no total count query)
6	Custom `@Query` with `Pageable`	Add `Pageable` as the last parameter to any `@Query` method for pagination

---

The Sort Class

The `Sort` class lets you specify how results should be ordered.

```java
import org.springframework.data.domain.Sort;

// Sort by a single field ascending
Sort sort1 = Sort.by("price");                              // Line 1
// Same as above (ascending is default)
Sort sort2 = Sort.by(Sort.Direction.ASC, "price");          // Line 2

// Sort by a single field descending
Sort sort3 = Sort.by(Sort.Direction.DESC, "price");         // Line 3

// Sort by multiple fields
Sort sort4 = Sort.by("name").ascending()
    .and(Sort.by("price").descending());                    // Line 4

// Sort by multiple fields (shorter syntax)
Sort sort5 = Sort.by(
    Sort.Order.asc("name"),
    Sort.Order.desc("price")
);                                                           // Line 5

// Unsorted (no ordering)
Sort sort6 = Sort.unsorted();                               // Line 6
```

Line-by-line explanation:

Line	Code	What It Does
1	`Sort.by("price")`	Sort by price ascending (default direction)
2	`Sort.by(Sort.Direction.ASC, "price")`	Explicit ascending sort by price

3	`Sort.by(Sort.Direction.DESC, "price")`	Sort by price descending (most expensive first)
4	`.and(Sort.by(...))`	Chain multiple sort criteria. Sort by name first, then by price within same names
5	`Sort.Order.asc/desc`	Alternative syntax using `Sort.Order` objects
6	`Sort.unsorted()`	No sorting applied

Using Sort in Repository

```java
// In the service layer
List<Product> products = productRepository.findAll(
    Sort.by("price").ascending()
);

// Combine sort with pagination
Pageable pageable = PageRequest.of(0, 10,
    Sort.by("price").descending());
Page<Product> page = productRepository.findAll(pageable);
```

Auto-Resolving Pageable from Request Parameters

Here is where Spring's magic shines. When you add a `Pageable` parameter to a controller method, Spring automatically creates it from the request's query parameters.

How Auto-Resolution Works

```
HTTP Request:
GET /api/products?page=2&size=10&sort=price,desc

                      |
                      v  Spring auto-creates:

Pageable pageable = PageRequest.of(2, 10,
    Sort.by("price").descending());
```

You do not need to manually parse `page`, `size`, or `sort` parameters. Spring does it all.

Supported Request Parameters

```
+-------------------+--------------------------------------+
| Parameter         | Example                              |
+-------------------+--------------------------------------+
| page              | ?page=0        (default: 0)          |
| size              | ?size=20       (default: 20)          |
| sort              | ?sort=price    (ascending by default)  |
| sort (desc)       | ?sort=price,desc                     |
| sort (multiple)   | ?sort=name,asc&sort=price,desc       |
+-------------------+--------------------------------------+
```

Controller with Auto-Resolved Pageable

```java
package com.example.demo.controller;

import com.example.demo.entity.Product;
import com.example.demo.service.ProductService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // GET /api/products?page=0&size=10&sort=price,desc
    @GetMapping
    public ResponseEntity<Page<Product>> getAllProducts(
            Pageable pageable) {                            // Line 1
        Page<Product> products =
            productService.getAllProducts(pageable);
        return ResponseEntity.ok(products);
    }

    // With custom defaults: page=0, size=5, sort by name asc
    @GetMapping("/custom")
    public ResponseEntity<Page<Product>> getProductsWithDefaults(
```

```
            @PageableDefault(size = 5, sort = "name",
                direction = Sort.Direction.ASC)
            Pageable pageable) {                            // Line 2
        Page<Product> products =
            productService.getAllProducts(pageable);
        return ResponseEntity.ok(products);
    }

    // Paginated search
    // GET /api/products/search?keyword=phone&page=0&size=10
    @GetMapping("/search")
    public ResponseEntity<Page<Product>> searchProducts(
            @RequestParam String keyword,
            Pageable pageable) {                            // Line 3
        Page<Product> products =
            productService.searchProducts(keyword, pageable);
        return ResponseEntity.ok(products);
    }

    // Paginated filter by stock status
    // GET /api/products/in-stock?page=0&size=10&sort=price,asc
    @GetMapping("/in-stock")
    public ResponseEntity<Page<Product>> getInStockProducts(
            Pageable pageable) {
        Page<Product> products =
            productService.getInStockProducts(pageable);
        return ResponseEntity.ok(products);
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`Pageable pageable`	Spring automatically creates this from `?page=`, `?size=`, and `?sort=` query parameters
2	`@PageableDefault(size = 5, ...)`	Sets default values when the client does not provide pagination parameters
3	`@RequestParam String keyword, Pageable pageable`	Combines regular request parameters with auto-resolved pagination

Configuring Default Pageable Values

You can set global defaults in `application.properties`:

```properties
# Default page size (default is 20)
spring.data.web.pageable.default-page-size=10

# Maximum page size (prevents clients from requesting too much data)
spring.data.web.pageable.max-page-size=100

# Use 1-based page numbering (default is 0-based)
# spring.data.web.pageable.one-indexed-parameters=true
```

The Service Layer with Pagination

```java
package com.example.demo.service;

import com.example.demo.entity.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Page<Product> getAllProducts(Pageable pageable) {
        return productRepository.findAll(pageable);
    }

    public Page<Product> searchProducts(
            String keyword, Pageable pageable) {
        return productRepository
            .findByNameContainingIgnoreCase(keyword, pageable);
    }

    public Page<Product> getInStockProducts(Pageable pageable) {
        return productRepository
            .findByInStock(true, pageable);
    }

    public Page<Product> getProductsByPriceRange(
            Double min, Double max, Pageable pageable) {
        return productRepository
```

```
            .findByPriceRange(min, max, pageable);
    }
}
```

Understanding the Page Response

When you return a `Page<Product>` from a controller, Spring serializes it into this JSON structure:

```
GET /api/products?page=1&size=3&sort=price,asc
```

Response:

```json
{
    "content": [
        {
            "id": 5,
            "name": "Smartwatch",
            "price": 299.99,
            "description": "Fitness tracking smartwatch",
            "inStock": true
        },
        {
            "id": 4,
            "name": "Tablet",
            "price": 449.99,
            "description": "10-inch display tablet",
            "inStock": false
        },
        {
            "id": 2,
            "name": "Smartphone",
            "price": 699.99,
            "description": "Latest model with 128GB storage",
            "inStock": true
        }
    ],
    "pageable": {
        "pageNumber": 1,
        "pageSize": 3,
        "sort": {
            "sorted": true,
            "empty": false,
            "unsorted": false
```

```
        },
        "offset": 3,
        "paged": true,
        "unpaged": false
    },
    "totalElements": 5,
    "totalPages": 2,
    "last": true,
    "size": 3,
    "number": 1,
    "sort": {
        "sorted": true,
        "empty": false,
        "unsorted": false
    },
    "numberOfElements": 2,
    "first": false,
    "empty": false
}
```

Let us break down each field:

```
+---------------------+-------+-------------------------------------------+
| Field               | Value | Meaning                                   |
+---------------------+-------+-------------------------------------------+
| content             | [...]  | The actual data for this page             |
| pageable.pageNumber | 1     | Current page number (0-indexed)           |
| pageable.pageSize   | 3     | Items per page                            |
| pageable.offset     | 3     | How many items were skipped (page * size) |
| totalElements       | 5     | Total items across all pages              |
| totalPages          | 2     | Total number of pages                     |
| last                | true  | Is this the last page?                    |
| first               | false | Is this the first page?                   |
| number              | 1     | Current page number (same as pageNumber)  |
| numberOfElements    | 2     | Items on THIS page (may be less than size |
|                     |       | on the last page)                         |
| empty               | false | Is the content empty?                     |
| sort.sorted         | true  | Is the result sorted?                     |
+---------------------+-------+-------------------------------------------+
```

Using Page.map() to Transform Results

Often you want to transform entities before returning them. For example, converting entities to DTOs. `Page.map()` transforms each element while preserving all pagination metadata.

```java
package com.example.demo.dto;

public class ProductSummaryDTO {
    private Long id;
    private String name;
    private Double price;
    private String availability;

    public ProductSummaryDTO(Long id, String name, Double price,
                             String availability) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.availability = availability;
    }

    // Getters
    public Long getId() { return id; }
    public String getName() { return name; }
    public Double getPrice() { return price; }
    public String getAvailability() { return availability; }
}
```

Using map() in the Service

```java
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Page<ProductSummaryDTO> getProductSummaries(
            Pageable pageable) {

        Page<Product> productPage =
            productRepository.findAll(pageable);              // Line 1

        return productPage.map(product ->                     // Line 2
            new ProductSummaryDTO(
                product.getId(),
                product.getName(),
                product.getPrice(),
                product.getInStock()
                    ? "In Stock" : "Out of Stock"             // Line 3
            )
        );
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`findAll(pageable)`	Gets a `Page<Product>` from the database
2	`productPage.map(...)`	Transforms each `Product` into a `ProductSummaryDTO`. Returns `Page<ProductSummaryDTO>` with the same pagination metadata
3	Ternary operator	Converts the boolean `inStock` to a human-readable string

Controller Using the DTO

```java
@GetMapping("/summaries")
public ResponseEntity<Page<ProductSummaryDTO>> getProductSummaries(
        Pageable pageable) {
    Page<ProductSummaryDTO> summaries =
        productService.getProductSummaries(pageable);
    return ResponseEntity.ok(summaries);
}
```

Response:

```json
{
    "content": [
        {"id": 1, "name": "Laptop", "price": 999.99, "availability": "In Stock"},
        {"id": 2, "name": "Smartphone", "price": 699.99, "availability": "In Stock"},
        {"id": 3, "name": "Headphones", "price": 149.99, "availability": "In Stock"}
    ],
    "totalElements": 5,
    "totalPages": 2,
    "number": 0,
    "size": 3,
    "first": true,
    "last": false
}
```

Complete Paginated REST API Example

Here is a complete working example with all the pieces together.

application.properties:

```properties
spring.application.name=pagination-demo
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.defer-datasource-initialization=true

# Pagination defaults
spring.data.web.pageable.default-page-size=10
spring.data.web.pageable.max-page-size=50
```

data.sql (sample data):

```sql
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Laptop', 999.99, 'High-performance laptop', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Smartphone', 699.99, 'Latest smartphone', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Headphones', 149.99, 'Wireless headphones', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Tablet', 449.99, '10-inch tablet', false);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Smartwatch', 299.99, 'Fitness smartwatch', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Monitor', 349.99, '27-inch 4K monitor', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Keyboard', 79.99, 'Mechanical keyboard', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Mouse', 49.99, 'Ergonomic mouse', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Webcam', 89.99, '1080p webcam', false);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Speaker', 129.99, 'Bluetooth speaker', true);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('Printer', 199.99, 'Laser printer', false);
INSERT INTO products (product_name, price, description, in_stock)
VALUES ('SSD', 89.99, '1TB solid state drive', true);
```

ProductController.java (complete):

```java
package com.example.demo.controller;

import com.example.demo.entity.Product;
import com.example.demo.service.ProductService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // Basic pagination
    // GET /api/products?page=0&size=5
    // GET /api/products?page=0&size=5&sort=price,desc
    // GET /api/products?page=0&size=5&sort=name,asc&sort=price,desc
    @GetMapping
    public ResponseEntity<Page<Product>> getAllProducts(
            @PageableDefault(size = 10, sort = "id")
            Pageable pageable) {
        return ResponseEntity.ok(
            productService.getAllProducts(pageable));
    }

    // Search with pagination
    // GET /api/products/search?keyword=smart&page=0&size=5
    @GetMapping("/search")
    public ResponseEntity<Page<Product>> searchProducts(
            @RequestParam String keyword,
            Pageable pageable) {
        return ResponseEntity.ok(
            productService.searchProducts(keyword, pageable));
    }

    // Filter by stock status with pagination
    // GET /api/products/available?page=0&size=5&sort=price,asc
    @GetMapping("/available")
    public ResponseEntity<Page<Product>> getAvailableProducts(
            Pageable pageable) {
        return ResponseEntity.ok(
            productService.getInStockProducts(pageable));
    }

    // Price range with pagination
    // GET /api/products/price-range?min=100&max=500&page=0&size=5
```

```
    @GetMapping("/price-range")
    public ResponseEntity<Page<Product>> getProductsByPriceRange(
            @RequestParam Double min,
            @RequestParam Double max,
            Pageable pageable) {
        return ResponseEntity.ok(
            productService.getProductsByPriceRange(
                min, max, pageable));
    }
}
```

Testing the Paginated API

Request 1: First page, 3 items, sorted by price ascending

```
GET http://localhost:8080/api/products?page=0&size=3&sort=price,asc
```

Response:

```json
{
    "content": [
        {"id": 8, "name": "Mouse", "price": 49.99, ...},
        {"id": 7, "name": "Keyboard", "price": 79.99, ...},
        {"id": 9, "name": "Webcam", "price": 89.99, ...}
    ],
    "totalElements": 12,
    "totalPages": 4,
    "number": 0,
    "size": 3,
    "first": true,
    "last": false
}
```

Request 2: Second page, same settings

```
GET http://localhost:8080/api/products?page=1&size=3&sort=price,asc
```

Response:

```json
{
    "content": [
        {"id": 12, "name": "SSD", "price": 89.99, ...},
        {"id": 10, "name": "Speaker", "price": 129.99, ...},
```

```
        {"id": 3, "name": "Headphones", "price": 149.99, ...}
    ],
    "totalElements": 12,
    "totalPages": 4,
    "number": 1,
    "size": 3,
    "first": false,
    "last": false
}
```

Request 3: Search with pagination

```
GET http://localhost:8080/api/products/search?keyword=smart&page=0&size=10
```

Response:

```json
{
    "content": [
        {"id": 2, "name": "Smartphone", "price": 699.99, ...},
        {"id": 5, "name": "Smartwatch", "price": 299.99, ...}
    ],
    "totalElements": 2,
    "totalPages": 1,
    "number": 0,
    "size": 10,
    "first": true,
    "last": true
}
```

Pagination Flow Diagram

```
Client Request                    Spring Boot Application
     |                                     |
     |  GET /api/products?                |
     |  page=1&size=3&sort=price,desc     |
     |------------------------------------>|
     |                                     |
     |                           +--------------------+
     |                           | Spring resolves    |
     |                           | Pageable from      |
     |                           | query parameters   |
     |                           +---------+----------+
     |                                     |
```

```
      |                           +--------v----------+
      |                           | Controller calls  |
      |                           | service method    |
      |                           +--------+----------+
      |                                    |
      |                           +--------v----------+
      |                           | Service calls     |
      |                           | repository with   |
      |                           | Pageable          |
      |                           +--------+----------+
      |                                    |
      |                           +--------v----------+
      |                           | Repository runs:  |
      |                           | SELECT * FROM     |
      |                           |   products        |
      |                           | ORDER BY price    |
      |                           |   DESC            |
      |                           | LIMIT 3 OFFSET 3  |
      |                           |                   |
      |                           | SELECT COUNT(*)   |
      |                           |   FROM products   |
      |                           +--------+----------+
      |                                    |
      |                           +--------v----------+
      |                           | Page<Product>     |
      |                           | serialized to JSON|
      |                           +--------+----------+
      |                                    |
      |  200 OK                            |
      |  { "content": [...],             |
      |    "totalElements": 12,            |
      |    "totalPages": 4, ... }          |
      |<-----------------------------------|
```

Common Mistakes

Mistake 1: Not Adding Pageable as the Last Parameter

```java
// WRONG - Pageable must be the last parameter in derived queries
Page<Product> findByPageableAndName(Pageable pageable, String name);

// CORRECT - Pageable is always the last parameter
Page<Product> findByName(String name, Pageable pageable);
```

Mistake 2: Forgetting That Pages Are Zero-Indexed

```
// Client expects page 1 to be the first page
GET /api/products?page=1&size=10
// But this actually returns the SECOND page!

// The first page is page 0
GET /api/products?page=0&size=10
// This returns the first page
```

If your frontend uses 1-based pagination, you can configure Spring to accept it:

```properties
spring.data.web.pageable.one-indexed-parameters=true
```

Mistake 3: Returning List Instead of Page

```java
// WRONG - Loses pagination metadata
@GetMapping
public List<Product> getAllProducts(Pageable pageable) {
    return productService.getAllProducts(pageable).getContent();
    // Client loses totalElements, totalPages, etc.
}

// CORRECT - Return the full Page
@GetMapping
public Page<Product> getAllProducts(Pageable pageable) {
    return productService.getAllProducts(pageable);
    // Client gets data + all pagination metadata
}
```

Mistake 4: Using Wrong Field Names in sort Parameter

```
// WRONG - Using column name instead of Java field name
GET /api/products?sort=product_name,asc
// Error: No property 'product_name' found

// CORRECT - Use the Java field name
GET /api/products?sort=name,asc
```

Mistake 5: Not Limiting Maximum Page Size

```properties
# Without this, a client could request ALL records:
# GET /api/products?size=999999
# This would load everything into memory!

# Add a maximum page size
spring.data.web.pageable.max-page-size=100
```

Best Practices

1. Always set a maximum page size: Use `spring.data.web.pageable.max-page-size` to prevent clients from requesting too much data at once.
2. Use `@PageableDefault` for sensible defaults: When no pagination parameters are provided, return a reasonable number of items (10-20).
3. Use `Page` for traditional pagination: When you need "Page X of Y" in the UI.
4. Use `Slice` for infinite scroll: When you only need "Load More" and total count is expensive to compute.
5. Sort by a unique field: Always include a unique field (like `id`) in your sort to ensure consistent ordering across pages.
6. Use `Page.map()` for DTOs: Transform entities to DTOs while preserving pagination metadata.
7. Document your API defaults: Make sure API consumers know the default page size and sort order.
8. Return `ResponseEntity<Page<T>>`: This gives you control over HTTP status codes while returning full pagination metadata.

Quick Summary

```
+---------------------------------------------+
|  Pagination & Sorting Quick Reference       |
+---------------------------------------------+
|                                             |
|  REQUEST PARAMETERS:                        |
|  ?page=0         (zero-indexed)             |
|  ?size=10        (items per page)           |
|  ?sort=field,asc (sort ascending)           |
|  ?sort=field,desc (sort descending)         |
```

```
|                                            |
|  JAVA CLASSES:                             |
|  Pageable        -> auto-resolved from     |
|                     request params         |
|  PageRequest.of(page, size, sort)          |
|  Sort.by("field").ascending()              |
|  Page<T>         -> data + total count     |
|  Slice<T>        -> data, no total count   |
|                                            |
|  REPOSITORY:                               |
|  Page<T> findAll(Pageable p)               |
|  Page<T> findByField(val, Pageable p)      |
|                                            |
|  CONTROLLER:                               |
|  method(Pageable pageable)                 |
|  method(@PageableDefault(...) Pageable p)  |
|                                            |
|  TRANSFORMATION:                           |
|  page.map(entity -> dto)                   |
|                                            |
+--------------------------------------------+
```

Key Points

1. Pagination prevents loading entire datasets into memory. Always paginate API endpoints that return collections.
2. Pages are zero-indexed by default. Page 0 is the first page.
3. `Pageable` is auto-resolved from request parameters (`page`, `size`, `sort`) when used as a controller method parameter.
4. `Page<T>` includes total count (runs an extra COUNT query). `Slice<T>` does not include total count (faster for large datasets).
5. Add `Pageable` as the last parameter to any repository method to make it paginated.
6. `Page.map()` transforms each element while preserving all pagination metadata.
7. Use `@PageableDefault` to set defaults and `spring.data.web.pageable.max-page-size` to limit the maximum page size.

Practice Questions

1. What is the difference between `Page` and `Slice`? When would you use each one?

2. What query parameters does Spring auto-resolve for a `Pageable` controller parameter? What are the defaults?
3. If you have 25 products and request `?page=2&size=10`, how many products will be returned? What will `totalPages` be?
4. How does `Page.map()` work? Why is it useful for transforming entities to DTOs?
5. Why should you always set `spring.data.web.pageable.max-page-size`? What could happen without it?

Exercises

Exercise 1: Paginated User API

Create a paginated API for the `User` entity with these endpoints:

- `GET /api/users` - List all users with pagination and sorting
- `GET /api/users/active` - List only active users with pagination
- `GET /api/users/search?name=alice` - Search by name with pagination

Include at least 15 users in `data.sql` and test with different page sizes and sort orders.

Exercise 2: Custom Page Response

The default `Page` JSON response includes a lot of metadata. Create a custom response wrapper class that simplifies the response to:

```json
{
    "data": [...],
    "currentPage": 0,
    "totalPages": 5,
    "totalItems": 50,
    "hasNext": true,
    "hasPrevious": false
}
```

Use `Page.map()` or create a utility method that converts any `Page<T>` to your custom response.

Exercise 3: Sorting Challenge

Create an endpoint that accepts a `category` parameter and returns products sorted by:

- In-stock products first, then out-of-stock

- Within each stock status group, sorted by price ascending

Test with at least 20 products across 3 categories.

What Is Next?

Your API can now store data, query it, and return it in paginated, sorted chunks. But there is one critical piece missing: security. Right now, anyone can access your API endpoints, create products, delete products, or view user data. In the next chapter, you will learn about Spring Security -- how to protect your API with authentication and authorization.

Chapter 17: Introduction to Spring Security

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the difference between authentication and authorization
- Add Spring Security to a Spring Boot application
- Understand the default security behavior
- Configure a `SecurityFilterChain` bean
- Define permitted (public) and protected routes
- Create in-memory users with roles
- Configure HTTP Basic and form-based login
- Understand CSRF protection and when to disable it
- Read the security filter chain diagram

Why This Chapter Matters

Imagine you built a beautiful house with no locks on the doors. Anyone can walk in, eat your food, rearrange your furniture, or take your valuables. That is what your API looks like right now. Every endpoint is wide open. Anyone can create, read, update, or delete data.

Security is not optional. Every real application needs it. You need to verify who is making requests (authentication) and what they are allowed to do (authorization).

Spring Security is the standard framework for securing Spring Boot applications. It is powerful but can be intimidating for beginners because it does a lot automatically. This chapter breaks it down step by step so you understand exactly what happens when you add security to your application.

Authentication vs Authorization

These two concepts are the foundation of security. They sound similar but are very different.

```
+-------------------------------------------------------------------+
|                                                                   |
|   AUTHENTICATION                    AUTHORIZATION                 |
|   "Who are you?"                    "What can you do?"            |
|                                                                   |
|   +------------------+             +------------------+           |
|   | Show me your ID  |             | What's your role?|           |
|   |                  |             |                  |           |
|   | Username: alice  |             | Role: ADMIN      |           |
|   | Password: ****   |             |                  |           |
|   +------------------+             +------------------+           |
|                                                                   |
|   Verifies identity               Checks permissions              |
|   Happens FIRST                   Happens AFTER authentication    |
|   "Are you really Alice?"         "Can Alice delete products?"    |
|                                                                   |
+-------------------------------------------------------------------+
```

Real-Life Analogy

Think of an office building:

1. Authentication = The security guard at the entrance checks your employee badge. They verify you are who you claim to be.
2. Authorization = Your badge has a color code. Green badges can access floors 1-3. Red badges can access all floors. The badge color determines what rooms you can enter.

Just because you are authenticated (the guard let you in) does not mean you are authorized for everything (you might not have access to the server room).

```
+-----------------------------------------------------------+
|  Step 1: AUTHENTICATION                                   |
|  Guard: "Badge please." (checking identity)               |
|  You: Show badge                                          |
|  Guard: "OK, you are Alice. Come in."                     |
+-----------------------------------------------------------+
                     |
                     v
+-----------------------------------------------------------+
|  Step 2: AUTHORIZATION                                    |
```

```
|  You try to open the server room door                    |
|  System: "Alice has role USER, not ADMIN"                |
|  System: "Access DENIED"                                 |
+----------------------------------------------------------+
```

Adding Spring Security

Step 1: Add the Dependency

Add `spring-boot-starter-security` to your `pom.xml`:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

That is it. Just one dependency. No version needed -- Spring Boot manages it.

What Happens Immediately

The moment you add this dependency and restart your application, everything changes:

1. All endpoints are protected -- Every URL requires authentication
2. A login page appears -- Going to any URL redirects you to `/login`
3. A default user is created -- Username is `user`, password is printed in the console
4. CSRF protection is enabled -- POST, PUT, DELETE requests need a CSRF token
5. Session management is enabled -- After login, a session cookie tracks you

Finding the Default Password

Look in the console for a line like this:

```
Using generated security password: 8f32a1b4-5678-9abc-def0-123456789abc

This generated password is for development use only. Your security
configuration must be updated before running your application in production.
```

You can now log in with:

- Username: `user`
- Password: (the generated password from the console)

Testing Default Security

Try accessing your API:

```
GET http://localhost:8080/api/products
```

Without authentication:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="Realm"
```

With HTTP Basic authentication:

```
GET http://localhost:8080/api/products
Authorization: Basic dXNlcjo4ZjMyYTFiNC01Njc4LTlhYmMtZGVmMC0xMjM0NTY3ODlhYmM=
```

Response: `200 OK` with your data.

The Security Filter Chain

Spring Security works through a chain of filters that process every request. This is one of the most important concepts to understand.

```
+-------------------------------------------------------------------+
|                     INCOMING HTTP REQUEST                         |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
|                   SPRING SECURITY FILTER CHAIN                    |
|                                                                   |
|  +-------------------------------------------------------------+ |
|  | 1. SecurityContextPersistenceFilter                         | |
|  |    Load security context from session                       | |
|  +-------------------------------------------------------------+ |
|  | 2. HeaderWriterFilter                                       | |
|  |    Add security headers (X-Frame-Options, etc.)             | |
```

```
|  +-------------------------------------------------------------+ |
|  | 3. CsrfFilter                                                | |
|  |    Validate CSRF token for POST/PUT/DELETE                   | |
|  +-------------------------------------------------------------+ |
|  | 4. LogoutFilter                                              | |
|  |    Handle /logout requests                                   | |
|  +-------------------------------------------------------------+ |
|  | 5. UsernamePasswordAuthenticationFilter                      | |
|  |    Handle form login (/login POST)                           | |
|  +-------------------------------------------------------------+ |
|  | 6. BasicAuthenticationFilter                                 | |
|  |    Handle HTTP Basic auth (Authorization header)             | |
|  +-------------------------------------------------------------+ |
|  | 7. RequestCacheAwareFilter                                   | |
|  |    Cache requests during authentication redirects            | |
|  +-------------------------------------------------------------+ |
|  | 8. SecurityContextHolderAwareRequestFilter                   | |
|  |    Wrap request with security methods                        | |
|  +-------------------------------------------------------------+ |
|  | 9. AnonymousAuthenticationFilter                             | |
|  |    Assign anonymous identity if not authenticated            | |
|  +-------------------------------------------------------------+ |
|  | 10. SessionManagementFilter                                  | |
|  |     Handle session fixation, concurrent sessions             | |
|  +-------------------------------------------------------------+ |
|  | 11. ExceptionTranslationFilter                               | |
|  |     Convert security exceptions to HTTP responses            | |
|  +-------------------------------------------------------------+ |
|  | 12. FilterSecurityInterceptor (AuthorizationFilter)          | |
|  |     Check if user has permission for this URL                | |
|  +-------------------------------------------------------------+ |
|                                                                   |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-----------------------------------------------------------------+
|                      YOUR CONTROLLER                            |
|                  (Only reached if authorized)                    |
+-----------------------------------------------------------------+
```

Every request passes through all these filters. Think of it as an airport security line with multiple checkpoints. If you fail any checkpoint, you are turned away.

Configuring SecurityFilterChain

The default behavior protects everything. But most applications need some endpoints to be public (like a login page or product listing) and others to be protected (like admin pages or user profiles).

Step 2: Create a Security Configuration

```java
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration                                                  // Line 1
@EnableWebSecurity                                              // Line 2
public class SecurityConfig {

    @Bean                                                       // Line 3
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http) throws Exception {               // Line 4

        http
            .authorizeHttpRequests(auth -> auth                 // Line 5
                .requestMatchers("/api/public/**").permitAll() // Line 6
                .requestMatchers("/h2-console/**").permitAll() // Line 7
                .requestMatchers("/api/admin/**")
                    .hasRole("ADMIN")                           // Line 8
                .requestMatchers("/api/products/**")
                    .hasAnyRole("USER", "ADMIN")                // Line 9
                .anyRequest().authenticated()                   // Line 10
            )
            .httpBasic(basic -> {})                             // Line 11
            .formLogin(form -> {})                              // Line 12
            .csrf(csrf -> csrf.disable());                      // Line 13

        // Allow H2 console frames
        http.headers(headers ->
            headers.frameOptions(frame ->
                frame.sameOrigin()));                           // Line 14

        return http.build();                                    // Line 15
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`@Configuration`	Marks this as a configuration class processed at startup
2	`@EnableWebSecurity`	Enables Spring Security's web security support

3	`@Bean`	Registers the `SecurityFilterChain` as a Spring bean
4	`HttpSecurity http`	The builder object for configuring security rules. Spring injects it
5	`authorizeHttpRequests(auth -> ...)`	Starts defining URL-based authorization rules
6	`.requestMatchers("/api/public/**").permitAll()`	URLs starting with `/api/public/` are accessible by everyone, no login needed
7	`.requestMatchers("/h2-console/**").permitAll()`	Allows access to the H2 console without authentication
8	`.hasRole("ADMIN")`	Only users with the ADMIN role can access `/api/admin/**`
9	`.hasAnyRole("USER", "ADMIN")`	Users with either USER or ADMIN role can access `/api/products/**`
10	`.anyRequest().authenticated()`	Everything else requires the user to be authenticated (any role)
11	`.httpBasic(basic -> {})`	Enables HTTP Basic authentication (credentials in the Authorization header)
12	`.formLogin(form -> {})`	Enables form-based login (Spring provides a default login page)
13	`.csrf(csrf -> csrf.disable())`	Disables CSRF protection. We will discuss why below
14	`.frameOptions(frame -> frame.sameOrigin())`	Allows H2 console to work (it uses HTML frames)
15	`http.build()`	Builds and returns the configured `SecurityFilterChain`

Understanding Authorization Rules

```
+-----------------------------------------------+
|  Rule Matching Order (TOP to BOTTOM)          |
+-----------------------------------------------+
|                                               |
|  /api/public/**     -->  Everyone (no login)  |
```

```
|  /h2-console/**     -->  Everyone (no login)  |
|  /api/admin/**       -->  ADMIN role only     |
|  /api/products/**    -->  USER or ADMIN role  |
|  Everything else     -->  Any logged-in user  |
|                                               |
+-----------------------------------------------+
|  IMPORTANT: Rules are checked in ORDER.       |
|  First matching rule wins. Put specific       |
|  rules BEFORE general ones.                   |
+-----------------------------------------------+
```

Common Authorization Methods

```
+--------------------------------+---------------------------------------------+
| Method                         | Who Can Access                              |
+--------------------------------+---------------------------------------------+
| .permitAll()                   | Everyone (no authentication needed)         |
| .authenticated()               | Any authenticated user (any role)           |
| .hasRole("ADMIN")              | Only users with ADMIN role                  |
| .hasAnyRole("USER", "ADMIN")   | Users with USER OR ADMIN role               |
| .hasAuthority("WRITE")         | Users with WRITE authority                  |
| .denyAll()                     | Nobody (useful for disabling endpoints)     |
+--------------------------------+---------------------------------------------+
```

What is the difference between Role and Authority?

- A Role is a group of authorities. Roles are prefixed with `ROLE_` internally (e.g., `ROLE_ADMIN`).
- An Authority is a specific permission (e.g., `READ`, `WRITE`, `DELETE`).

When you say `hasRole("ADMIN")`, Spring internally checks for `ROLE_ADMIN`.

In-Memory Users

For development and testing, you can define users directly in your configuration. These users exist only in memory and are lost when the application restarts.

Step 3: Define In-Memory Users

```java
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/h2-console/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/products/**")
                    .hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(basic -> {})
            .csrf(csrf -> csrf.disable())
            .headers(headers ->
                headers.frameOptions(frame -> frame.sameOrigin()));

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {          // Line 1

        UserDetails user = User.builder()                     // Line 2
            .username("alice")                                // Line 3
            .password(passwordEncoder().encode("password123")) // Line 4
            .roles("USER")                                    // Line 5
            .build();

        UserDetails admin = User.builder()
            .username("bob")
            .password(passwordEncoder().encode("admin456"))
            .roles("ADMIN")                                   // Line 6
            .build();

        UserDetails superUser = User.builder()
```

```
            .username("charlie")
            .password(passwordEncoder().encode("super789"))
            .roles("USER", "ADMIN")                          // Line 7
            .build();

        return new InMemoryUserDetailsManager(
            user, admin, superUser);                         // Line 8
    }

    @Bean
    public PasswordEncoder passwordEncoder() {               // Line 9
        return new BCryptPasswordEncoder();                  // Line 10
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`UserDetailsService`	A Spring Security interface that loads user data. Returns a bean that Spring uses for authentication
2	`User.builder()`	Starts building a user using Spring Security's `User` class (not your custom `User` entity)
3	`.username("alice")`	Sets the username for authentication
4	`.password(passwordEncoder().encode(...))`	Encodes the password using BCrypt. Never store plain text passwords!
5	`.roles("USER")`	Assigns the USER role. Internally stored as `ROLE_USER`
6	`.roles("ADMIN")`	Assigns the ADMIN role
7	`.roles("USER", "ADMIN")`	A user can have multiple roles
8	`InMemoryUserDetailsManager(...)`	Stores users in memory. Pass all users as arguments
9	`PasswordEncoder` bean	Defines how passwords are encoded and verified
10	`BCryptPasswordEncoder()`	BCrypt is a strong one-way hashing algorithm. The industry standard for password hashing

Why BCrypt?

```
Plain text:     "password123"
                     |
                     v (BCrypt encode)
BCrypt hash:    "$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy"

Key properties:
1. One-way: Cannot reverse the hash to get the password
2. Salted: Same password produces different hashes each time
3. Slow: Intentionally slow to prevent brute-force attacks
```

HTTP Basic Authentication

HTTP Basic is the simplest authentication method. The client sends credentials in the `Authorization` header with every request.

```
+-------------------------------------------------------------------+
|  HTTP BASIC AUTHENTICATION FLOW                                   |
+-------------------------------------------------------------------+
|                                                                   |
|  1. Client sends request WITHOUT credentials                      |
|     GET /api/products                                             |
|                                                                   |
|  2. Server responds with 401 Unauthorized                         |
|     WWW-Authenticate: Basic realm="Realm"                         |
|                                                                   |
|  3. Client sends request WITH credentials                         |
|     GET /api/products                                             |
|     Authorization: Basic YWxpY2U6cGFzc3dvcmQxMjM=               |
|                    |      |                                      |
|                    |      +-- Base64("alice:password123")         |
|                    +--------- Authentication type                 |
|                                                                   |
|  4. Server validates credentials                                  |
|     - Decode Base64 to get "alice:password123"                    |
|     - Look up user "alice"                                        |
|     - Verify password matches                                     |
|     - Check if alice has permission for /api/products             |
|                                                                   |
|  5. Server responds with 200 OK and data                          |
|                                                                   |
+-------------------------------------------------------------------+
```

Testing with curl

```bash
# Without authentication - 401 Unauthorized
curl -v http://localhost:8080/api/products

# With authentication (alice has USER role)
curl -u alice:password123 http://localhost:8080/api/products
# 200 OK - alice can access /api/products (requires USER or ADMIN)

# Admin endpoint (alice has USER role, not ADMIN)
curl -u alice:password123 http://localhost:8080/api/admin/dashboard
# 403 Forbidden - alice does not have ADMIN role

# Admin endpoint (bob has ADMIN role)
curl -u bob:admin456 http://localhost:8080/api/admin/dashboard
# 200 OK - bob has ADMIN role

# Public endpoint (no authentication needed)
curl http://localhost:8080/api/public/health
# 200 OK - public endpoints are accessible by everyone
```

Form Login

Form login provides a browser-based login page. When enabled, Spring Security generates a default HTML login form.

```
+-------------------------------------------------------------------+
|  FORM LOGIN FLOW                                                  |
+-------------------------------------------------------------------+
|                                                                   |
|  1. User visits /api/products in browser                          |
|     (not authenticated)                                           |
|                                                                   |
|  2. Spring redirects to /login                                    |
|     Shows the default login form                                  |
|                                                                   |
|  3. User enters username and password                             |
|     Form submits POST /login                                      |
|                                                                   |
|  4. Spring validates credentials                                  |
|     If valid: Redirect to the original URL (/api/products)        |
|     If invalid: Redirect to /login?error                          |
|                                                                   |
|  5. Spring creates a session cookie (JSESSIONID)                  |
|     All subsequent requests use this cookie                       |
|     User does not need to log in again                            |
```

```
|                                                                  |
+------------------------------------------------------------------+
```

Customizing the Form Login

```java
http
    .formLogin(form -> form
        .loginPage("/custom-login")          // Custom login page URL
        .loginProcessingUrl("/perform-login") // URL to submit the form to
        .defaultSuccessUrl("/dashboard")     // Redirect after successful login
        .failureUrl("/custom-login?error")   // Redirect after failed login
    );
```

For REST APIs, you typically use HTTP Basic or JWT (next chapter) instead of form login.

CSRF Protection

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks your browser into making requests to your application using your existing session.

```
+------------------------------------------------------------------+
|  CSRF ATTACK SCENARIO                                            |
+------------------------------------------------------------------+
|                                                                  |
|  1. Alice logs into yourbank.com                                 |
|     Browser stores session cookie                                |
|                                                                  |
|  2. Alice visits evil-site.com (in another tab)                  |
|     evil-site.com has hidden form:                               |
|     <form action="yourbank.com/transfer" method="POST">          |
|       <input name="to" value="hacker" />                         |
|       <input name="amount" value="10000" />                      |
|     </form>                                                      |
|     <script>document.forms[0].submit()</script>                  |
|                                                                  |
|  3. Browser sends the POST to yourbank.com                       |
|     WITH Alice's session cookie (automatic!)                     |
|                                                                  |
|  4. yourbank.com thinks Alice is making the transfer             |
|     $10,000 goes to the hacker                                   |
|                                                                  |
+------------------------------------------------------------------+
```

How CSRF protection works:

Spring Security generates a random CSRF token for each session. Every form submission (POST, PUT, DELETE) must include this token. Since the evil site cannot access the token, it cannot forge valid requests.

When to Disable CSRF

For REST APIs that use token-based authentication (like JWT), you typically disable CSRF because:

1. REST APIs do not use browser sessions
2. JWT tokens are not automatically sent by the browser (unlike cookies)
3. The client must explicitly include the token in headers

```java
// For REST APIs with JWT - disable CSRF
.csrf(csrf -> csrf.disable())

// For traditional web applications with sessions - keep CSRF enabled
.csrf(csrf -> {})  // Enabled by default
```

Creating Test Controllers

Let us create controllers to test our security configuration:

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/public")
public class PublicController {

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of(
            "status", "UP",
            "message", "Application is running"
```

```
        );
    }

    @GetMapping("/info")
    public Map<String, String> info() {
        return Map.of(
            "app", "Spring Security Demo",
            "version", "1.0.0"
        );
    }
}
```

```java
package com.example.demo.controller;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/profile")
    public Map<String, Object> getProfile() {
        Authentication auth =
            SecurityContextHolder.getContext()
                .getAuthentication();                        // Line 1

        return Map.of(
            "username", auth.getName(),                      // Line 2
            "authorities", auth.getAuthorities().toString()  // Line 3
        );
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`SecurityContextHolder.getContext().getAuthentication()`	Gets the currently authenticated user's details from the security context
2	`auth.getName()`	Returns the username of the authenticated user

3	`auth.getAuthorities()`	Returns the user's roles/authorities (e.g., `[ROLE_USER]`)

You can also inject `Authentication` directly as a method parameter:

```java
@GetMapping("/profile")
public Map<String, Object> getProfile(Authentication auth) {
    return Map.of(
        "username", auth.getName(),
        "authorities", auth.getAuthorities().toString()
    );
}
```

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    @GetMapping("/dashboard")
    public Map<String, Object> dashboard() {
        return Map.of(
            "message", "Welcome to the admin dashboard",
            "stats", Map.of(
                "totalUsers", 150,
                "totalProducts", 45,
                "totalOrders", 320
            )
        );
    }
}
```

Testing Results

```
# Public - No auth needed
GET /api/public/health        -> 200 OK

# User profile - requires authentication
GET /api/user/profile
  as alice (USER)             -> 200 OK {"username":"alice","authorities":"[ROLE_USER]"}
  as bob (ADMIN)              -> 200 OK {"username":"bob","authorities":"[ROLE_ADMIN]"}
  no auth                     -> 401 Unauthorized
```

```
# Products - requires USER or ADMIN role
GET /api/products
  as alice (USER)             -> 200 OK
  as bob (ADMIN)              -> 200 OK
  no auth                     -> 401 Unauthorized

# Admin - requires ADMIN role
GET /api/admin/dashboard
  as alice (USER)             -> 403 Forbidden
  as bob (ADMIN)              -> 200 OK
  as charlie (USER+ADMIN)     -> 200 OK
  no auth                     -> 401 Unauthorized
```

Notice the difference between 401 and 403:

- 401 Unauthorized = You have not proven who you are (not authenticated)
- 403 Forbidden = We know who you are, but you do not have permission (not authorized)

Complete Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── config/
│   └── SecurityConfig.java
├── controller/
│   ├── AdminController.java
│   ├── ProductController.java
│   ├── PublicController.java
│   └── UserController.java
├── entity/
│   └── Product.java
├── repository/
│   └── ProductRepository.java
└── service/
    └── ProductService.java

src/main/resources/
├── application.properties
└── data.sql
```

application.properties:

```properties
spring.application.name=security-demo
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false
spring.jpa.defer-datasource-initialization=true

# Logging security events
logging.level.org.springframework.security=DEBUG
```

Setting `logging.level.org.springframework.security=DEBUG` will show you exactly what Spring Security does with each request. Very helpful for learning and debugging.

Security Flow Summary

```
+-----------------------------------------------------------------+
|  Complete Security Flow for a Request                           |
+-----------------------------------------------------------------+
|                                                                 |
|  Client -> GET /api/products                                    |
|  Authorization: Basic YWxpY2U6cGFzc3dvcmQxMjM=                 |
|                                                                 |
|  Step 1: Security Filter Chain                                  |
|  +-------------------------------------------------------+     |
|  | BasicAuthenticationFilter:                            |     |
|  |   - Extract "alice:password123" from header           |     |
|  |   - Look up user "alice" via UserDetailsService       |     |
|  |   - Verify password with BCryptPasswordEncoder        |     |
|  |   - Password matches! Create Authentication object    |     |
|  |   - Store in SecurityContextHolder                    |     |
|  +-------------------------------------------------------+     |
|                                                                 |
|  Step 2: Authorization                                          |
|  +-------------------------------------------------------+     |
|  | AuthorizationFilter:                                  |     |
|  |   - URL: /api/products/** -> requires USER or ADMIN   |     |
|  |   - Alice has role: ROLE_USER                         |     |
|  |   - ROLE_USER matches! Access GRANTED                 |     |
|  +-------------------------------------------------------+     |
|                                                                 |
|  Step 3: Controller                                             |
|  +-------------------------------------------------------+     |
|  | ProductController.getAllProducts() executes            |     |
```

```
|  | Returns list of products                                  |   |
|  +-----------------------------------------------------------+   |
|                                                                   |
|  Response: 200 OK + product data                                  |
|                                                                   |
+-------------------------------------------------------------------+
```

Common Mistakes

Mistake 1: Putting General Rules Before Specific Ones

```java
// WRONG - anyRequest().authenticated() matches everything first
// The admin and public rules never get checked
http.authorizeHttpRequests(auth -> auth
    .anyRequest().authenticated()              // Catches everything!
    .requestMatchers("/api/public/**").permitAll()  // Never reached
    .requestMatchers("/api/admin/**").hasRole("ADMIN") // Never reached
);

// CORRECT - Put specific rules first, general rule last
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/public/**").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()              // Catches the rest
);
```

Mistake 2: Not Encoding Passwords

```java
// WRONG - Storing plain text password
// Spring Security 5+ REQUIRES encoded passwords
UserDetails user = User.builder()
    .username("alice")
    .password("password123")  // Will throw error!
    .roles("USER")
    .build();

// CORRECT - Encode password with BCrypt
UserDetails user = User.builder()
    .username("alice")
    .password(passwordEncoder.encode("password123"))
    .roles("USER")
    .build();
```

Mistake 3: Forgetting H2 Console Configuration

```java
// WRONG - H2 console will not work
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/h2-console/**").permitAll()
    // Missing: frame options and CSRF configuration
);

// CORRECT - H2 console needs frame options and CSRF exceptions
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/h2-console/**").permitAll()
)
.csrf(csrf -> csrf.disable())  // or csrf.ignoringRequestMatchers("/h2-console/**")
.headers(headers ->
    headers.frameOptions(frame -> frame.sameOrigin()));
```

Mistake 4: Using hasRole with ROLE_ Prefix

```java
// WRONG - Do not include the ROLE_ prefix
.requestMatchers("/api/admin/**").hasRole("ROLE_ADMIN")
// This checks for "ROLE_ROLE_ADMIN" internally!

// CORRECT - Spring adds ROLE_ automatically
.requestMatchers("/api/admin/**").hasRole("ADMIN")
// This checks for "ROLE_ADMIN" internally
```

Best Practices

1. Always use BCrypt: Never store passwords in plain text. BCrypt is the recommended password encoder.
2. Order rules from specific to general: Put specific URL patterns before `anyRequest()`.
3. Disable CSRF for REST APIs: If you use stateless authentication (like JWT), disable CSRF. Keep it enabled for session-based web applications.
4. Use roles for coarse-grained access: Roles like USER, ADMIN, MODERATOR work well for most applications.
5. Enable security logging for development: Set `logging.level.org.springframework.security=DEBUG` to see exactly what Spring Security does.

6. Do not use in-memory users in production: In-memory users are for development only. In production, load users from a database.
7. Return proper HTTP status codes: 401 for unauthenticated, 403 for unauthorized. Spring Security handles this automatically.
8. Keep security configuration in one place: Have a single `SecurityConfig` class that contains all your security rules.

Quick Summary

```
+-----------------------------------------+
|  Spring Security Quick Reference        |
+-----------------------------------------+
|                                         |
|  DEPENDENCY:                            |
|  spring-boot-starter-security           |
|                                         |
|  DEFAULT BEHAVIOR:                      |
|  - All endpoints protected              |
|  - Default user: "user"                |
|  - Password: printed in console        |
|  - Login page at /login                |
|                                         |
|  KEY CLASSES:                           |
|  SecurityFilterChain -> security rules  |
|  UserDetailsService  -> user storage    |
|  PasswordEncoder     -> password hashing|
|                                         |
|  AUTHORIZATION:                         |
|  .permitAll()     -> no auth needed     |
|  .authenticated() -> any logged in user |
|  .hasRole("X")    -> specific role      |
|  .hasAnyRole(..") -> any of the roles   |
|                                         |
|  AUTH METHODS:                          |
|  .httpBasic()     -> Authorization hdr  |
|  .formLogin()     -> login page         |
|                                         |
|  STATUS CODES:                          |
|  401 = Not authenticated               |
|  403 = Not authorized                  |
|                                         |
+-----------------------------------------+
```

Key Points

1. Authentication verifies identity ("who are you?"). Authorization checks permissions ("what can you do?").
2. Adding `spring-boot-starter-security` immediately protects all endpoints and generates a default user.
3. `SecurityFilterChain` is configured through a `@Bean` method using `HttpSecurity`. Rules are matched top to bottom.
4. `permitAll()` makes endpoints public. `hasRole("ADMIN")` restricts to specific roles. `authenticated()` requires any logged-in user.
5. Passwords must be encoded with `BCryptPasswordEncoder`. Never store plain text passwords.
6. In-memory users are for development only. Production applications should load users from a database.
7. CSRF protection should be disabled for stateless REST APIs but kept enabled for session-based web applications.
8. 401 means "not authenticated" (no credentials). 403 means "not authorized" (credentials valid but insufficient permissions).

Practice Questions

1. What is the difference between authentication and authorization? Give a real-world analogy.
2. What happens by default when you add `spring-boot-starter-security` to a project? List at least three changes.
3. Why must authorization rules be ordered from specific to general? What happens if you put `anyRequest().authenticated()` first?
4. Why does Spring Security require passwords to be encoded? What would happen if you used plain text?
5. When should you disable CSRF protection? When should you keep it enabled?

Exercises

Exercise 1: Multi-Role API

Create a REST API with three roles: USER, MODERATOR, ADMIN. Configure these access rules:

- `/api/public/**` - accessible by everyone
- `/api/posts/**` - accessible by USER, MODERATOR, ADMIN
- `/api/moderate/**` - accessible by MODERATOR, ADMIN
- `/api/admin/**` - accessible by ADMIN only

Create in-memory users for each role and test all combinations.

Exercise 2: Custom Access Rules

Create a product API where:

- GET endpoints are public (anyone can view products)
- POST and PUT endpoints require the USER role
- DELETE endpoints require the ADMIN role

Hint: Use `.requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()`.

Exercise 3: Security Debugging

Enable `DEBUG` logging for Spring Security and send requests to different endpoints. Trace each request through the log output and document:

- Which filters process the request
- Where authentication happens
- Where authorization is checked
- What happens when authentication fails vs. authorization fails

---

What Is Next?

In-memory users and HTTP Basic authentication work for learning, but production applications need a more robust approach. In the next chapter, you will learn about JWT Authentication -- the industry standard for securing REST APIs. You will build a complete login and registration flow with JSON Web Tokens.

Chapter 18: JWT Authentication

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what a JWT is and how it works
- Know the three parts of a JWT: header, payload, and signature
- Add the jjwt dependency for JWT operations
- Create a `JwtUtil` class to generate and validate tokens
- Build a `JwtAuthenticationFilter` that intercepts every request
- Create login and registration endpoints
- Build a complete authentication flow from registration to protected API access

Why This Chapter Matters

In the previous chapter, you learned about HTTP Basic authentication. It works, but it has a major problem: the client must send the username and password with every single request. That is like showing your passport at every door in the airport, not just at the security checkpoint.

JWT (JSON Web Token) solves this elegantly. You show your credentials once at login, and in return you get a token -- a signed digital pass. For every subsequent request, you just show this token. No need to send your password again.

The Concert Wristband Analogy

Imagine you buy a ticket to a music concert:

1. At the gate (login), you show your ticket and ID. The staff verifies them.
2. They give you a wristband (JWT token). The wristband has your name, your ticket type (VIP or General), and a holographic stamp that proves it is real.

3. Inside the venue, you just flash your wristband to enter different areas. No one asks for your ticket or ID again.
4. The wristband expires at the end of the concert. After that, it is useless.

A JWT works exactly like this wristband:

```
+-----------------------------------------------------+
|                    JWT TOKEN                        |
|  (Your digital concert wristband)                   |
+-----------------------------------------------------+
|                                                     |
|  WHO you are:      alice                            |
|  WHAT you can do:  ROLE_USER                        |
|  WHEN it expires:  2 hours from now                 |
|  PROOF it's real:  Cryptographic signature          |
|                                                     |
+-----------------------------------------------------+
```

What Is a JWT?

A JWT (pronounced "jot") is a compact, self-contained token used for securely transmitting information between parties. It is a string of text that looks like this:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIsInJvbGUiOiJVU0VSIiwiaWF0IjoxNzA5MjM0NTY3LCJleH...
```

It looks like gibberish, but it has three distinct parts separated by dots.

The Three Parts of a JWT

```
HEADER.PAYLOAD.SIGNATURE
  |       |        |
  |       |        +-- Verifies the token has not been tampered with
  |       +----------- Contains the actual data (claims)
  +------------------- Describes the token type and algorithm
```

Let us break each part down:

1. Header (Algorithm and token type)

```json
{
    "alg": "HS256",
    "typ": "JWT"
}
```

This says: "This is a JWT, and it was signed using the HMAC-SHA256 algorithm."

2. Payload (Claims -- the actual data)

```json
{
    "sub": "alice",
    "role": "USER",
    "iat": 1709234567,
    "exp": 1709241767
}
```

This contains:

- `sub` (subject): who the token belongs to
- `role`: what the user can do
- `iat` (issued at): when the token was created
- `exp` (expiration): when the token expires

3. Signature (Proof of authenticity)

```
HMACSHA256(
    base64UrlEncode(header) + "." + base64UrlEncode(payload),
    your-secret-key
)
```

The signature is created by combining the header and payload with a secret key. If anyone changes the payload (for example, changing `role` from `USER` to `ADMIN`), the signature will not match and the token will be rejected.

```
+-----------------------------------------------------------+
|  WHY THE SIGNATURE MATTERS                                |
+-----------------------------------------------------------+
|                                                           |
|  Original token:                                          |
|  Header.Payload("role":"USER").Signature_A                |
|                                                           |
|  Hacker modifies payload:                                 |
|  Header.Payload("role":"ADMIN").Signature_A               |
|                                                           |
|  Server recalculates signature:                           |
|  Expected: Signature_B (based on modified payload)        |
```

```
|  Actual:   Signature_A (from original token)             |
|                                                          |
|  Signature_A != Signature_B                              |
|  TOKEN REJECTED! Tampering detected.                     |
|                                                          |
+----------------------------------------------------------+
```

JWT Authentication Flow

```
+-------------------------------------------------------------------+
|  COMPLETE JWT AUTHENTICATION FLOW                                 |
+-------------------------------------------------------------------+
|                                                                   |
|  STEP 1: REGISTER                                                 |
|  POST /api/auth/register                                          |
|  {"username":"alice","password":"pass123","email":"a@b.com"}      |
|  Server: Save user to database with BCrypt-hashed password        |
|  Response: 201 Created                                            |
|                                                                   |
|  STEP 2: LOGIN                                                    |
|  POST /api/auth/login                                             |
|  {"username":"alice","password":"pass123"}                        |
|  Server: Verify credentials, generate JWT token                   |
|  Response: {"token":"eyJhbG..."}                                  |
|                                                                   |
|  STEP 3: ACCESS PROTECTED RESOURCES                               |
|  GET /api/products                                                |
|  Authorization: Bearer eyJhbG...                                  |
|  Server: Validate token, extract user, check permissions          |
|  Response: 200 OK + data                                          |
|                                                                   |
|  STEP 4: TOKEN EXPIRES                                            |
|  GET /api/products                                                |
|  Authorization: Bearer eyJhbG... (expired token)                  |
|  Server: Token expired!                                           |
|  Response: 401 Unauthorized                                       |
|  Client must login again to get a new token                       |
|                                                                   |
+-------------------------------------------------------------------+
```

Setting Up the Project

Step 1: Add Dependencies

```xml
<dependencies>
    <!-- Spring Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- Spring Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- H2 Database -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- JJWT - Java JWT library -->
    <dependency>                                                <!-- Line 1 -->
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>                       <!-- Line 2 -->
        <version>0.12.6</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>                      <!-- Line 3 -->
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>                   <!-- Line 4 -->
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

Line-by-line explanation:

Line	Artifact	What It Does

1-2	`jjwt-api`	The JJWT API interfaces. Your code compiles against this
3	`jjwt-impl`	The JJWT implementation. Needed at runtime to create and parse tokens
4	`jjwt-jackson`	JSON serialization support for JJWT. Uses Jackson (which Spring Boot already includes)

Step 2: Configure application.properties

```properties
spring.application.name=jwt-demo
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# JWT Configuration
jwt.secret=mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456  # Line 1
jwt.expiration=7200000                                                 # Line 2
```

Line	Property	What It Does
1	`jwt.secret`	The secret key used to sign tokens. Must be at least 256 bits (32 characters) for HS256. In production, use environment variables!
2	`jwt.expiration`	Token validity duration in milliseconds. 7200000 = 2 hours

---

Building the JWT System

Project Structure

```
src/main/java/com/example/demo/
├── DemoApplication.java
├── config/
│   └── SecurityConfig.java
├── controller/
│   ├── AuthController.java
│   └── ProductController.java
├── dto/
│   ├── LoginRequest.java
│   ├── LoginResponse.java
│   └── RegisterRequest.java
├── entity/
│   ├── Product.java
│   └── AppUser.java
├── filter/
│   └── JwtAuthenticationFilter.java
├── repository/
│   ├── ProductRepository.java
│   └── AppUserRepository.java
├── security/
│   └── JwtUtil.java
└── service/
    ├── AuthService.java
    └── ProductService.java
```

Step 3: Create the User Entity

```java
package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "app_users")                                    // Line 1
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false)
    private String password;                                   // Line 2

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false, length = 20)
```

```
    private String role = "USER";                            // Line 3

    public AppUser() {}

    public AppUser(String username, String password,
                   String email) {
        this.username = username;
        this.password = password;
        this.email = email;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

Key points:

Line	Code	What It Does
1	`@Table(name = "app_users")`	Named `app_users` to avoid conflict with H2's reserved word `USER`
2	`private String password`	This will store the BCrypt-hashed password, never plain text
3	`private String role = "USER"`	Default role for new users

Step 4: Create the User Repository

```java
package com.example.demo.repository;

import com.example.demo.entity.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;
```

```
@Repository
public interface AppUserRepository extends JpaRepository<AppUser, Long> {

    Optional<AppUser> findByUsername(String username);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);
}
```

Step 5: Create DTOs (Data Transfer Objects)

DTOs are simple classes used to transfer data between the client and server. They keep your entity classes separate from your API contract.

```java
package com.example.demo.dto;

public class RegisterRequest {

    private String username;
    private String password;
    private String email;

    // Default constructor
    public RegisterRequest() {}

    public RegisterRequest(String username, String password,
                           String email) {
        this.username = username;
        this.password = password;
        this.email = email;
    }

    // Getters and setters
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

```java
package com.example.demo.dto;

public class LoginRequest {

    private String username;
```

```
    private String password;

    public LoginRequest() {}

    public LoginRequest(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

```java
package com.example.demo.dto;

public class LoginResponse {

    private String token;
    private String username;
    private String role;
    private long expiresIn;

    public LoginResponse() {}

    public LoginResponse(String token, String username,
                         String role, long expiresIn) {
        this.token = token;
        this.username = username;
        this.role = role;
        this.expiresIn = expiresIn;
    }

    // Getters and setters
    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
    public long getExpiresIn() { return expiresIn; }
    public void setExpiresIn(long expiresIn) { this.expiresIn = expiresIn; }
}
```

Step 6: Create the JwtUtil Class

This is the heart of the JWT system. It creates tokens, validates them, and extracts information from them.

```java
package com.example.demo.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtUtil {

    private final SecretKey key;                              // Line 1
    private final long expirationMs;                          // Line 2

    public JwtUtil(
            @Value("${jwt.secret}") String secret,            // Line 3
            @Value("${jwt.expiration}") long expirationMs     // Line 4
    ) {
        this.key = Keys.hmacShaKeyFor(
            secret.getBytes(StandardCharsets.UTF_8));          // Line 5
        this.expirationMs = expirationMs;
    }

    /**
     * Generate a JWT token for a user.
     */
    public String generateToken(String username, String role) { // Line 6
        Date now = new Date();                                // Line 7
        Date expiration = new Date(
            now.getTime() + expirationMs);                    // Line 8

        return Jwts.builder()                                 // Line 9
            .subject(username)                                // Line 10
            .claim("role", role)                              // Line 11
            .issuedAt(now)                                    // Line 12
            .expiration(expiration)                           // Line 13
            .signWith(key)                                    // Line 14
            .compact();                                       // Line 15
    }

    /**
     * Extract the username from a JWT token.
     */
    public String extractUsername(String token) {             // Line 16
        return extractAllClaims(token).getSubject();
```

```
    }

    /**
     * Extract the role from a JWT token.
     */
    public String extractRole(String token) {                // Line 17
        return extractAllClaims(token)
            .get("role", String.class);
    }

    /**
     * Check if a token is valid (not expired, not tampered).
     */
    public boolean isTokenValid(String token) {              // Line 18
        try {
            extractAllClaims(token);                         // Line 19
            return true;
        } catch (JwtException | IllegalArgumentException e) { // Line 20
            return false;
        }
    }

    /**
     * Parse the token and extract all claims.
     * This also validates the signature and expiration.
     */
    private Claims extractAllClaims(String token) {          // Line 21
        return Jwts.parser()                                 // Line 22
            .verifyWith(key)                                 // Line 23
            .build()
            .parseSignedClaims(token)                        // Line 24
            .getPayload();                                   // Line 25
    }

    /**
     * Get the expiration time in milliseconds.
     */
    public long getExpirationMs() {
        return expirationMs;
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`SecretKey key`	The cryptographic key used for signing and verifying tokens
2	`long expirationMs`	How long tokens are valid (in milliseconds)

3	`@Value("${jwt.secret}")`	Reads the secret from `application.properties`
4	`@Value("${jwt.expiration}")`	Reads the expiration time from properties
5	`Keys.hmacShaKeyFor(...)`	Creates a proper HMAC-SHA key from the secret string
6	`generateToken(...)`	Creates a new JWT token for a given user
7	`new Date()`	Current timestamp
8	`now.getTime() + expirationMs`	Calculates the expiration time (now + 2 hours)
9	`Jwts.builder()`	Starts building a JWT token
10	`.subject(username)`	Sets the `sub` claim (who the token is for)
11	`.claim("role", role)`	Adds a custom claim for the user's role
12	`.issuedAt(now)`	Sets the `iat` claim (when the token was created)
13	`.expiration(expiration)`	Sets the `exp` claim (when the token expires)
14	`.signWith(key)`	Signs the token with the secret key. This creates the signature
15	`.compact()`	Builds the final JWT string (header.payload.signature)
16	`extractUsername(...)`	Gets the username from a token by reading the `sub` claim
17	`extractRole(...)`	Gets the role from a token by reading the custom `role` claim
18	`isTokenValid(...)`	Checks if a token is valid. If parsing succeeds, the token is valid
19	`extractAllClaims(token)`	If this throws an exception, the token is invalid
20	`JwtException`	Catches expired tokens, tampered tokens, and malformed tokens

21	`extractAllClaims(...)`	The core parsing method that validates and extracts token data
22	`Jwts.parser()`	Creates a JWT parser
23	`.verifyWith(key)`	Tells the parser to verify the signature with our secret key
24	`.parseSignedClaims(token)`	Parses the token, verifies the signature, checks expiration
25	`.getPayload()`	Extracts the claims (payload data) from the verified token

---

Step 7: Create the JwtAuthenticationFilter

This filter intercepts every request, checks for a JWT token in the Authorization header, and sets up the security context if the token is valid.

```java
package com.example.demo.filter;

import com.example.demo.security.JwtUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication
    .UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority
    .SimpleGrantedAuthority;
import org.springframework.security.core.context
    .SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthenticationFilter
        extends OncePerRequestFilter {                      // Line 1

    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(JwtUtil jwtUtil) {       // Line 2
        this.jwtUtil = jwtUtil;
```

```
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {

        // Step 1: Get the Authorization header
        String authHeader =
            request.getHeader("Authorization");              // Line 3

        // Step 2: Check if it starts with "Bearer "
        if (authHeader == null
                || !authHeader.startsWith("Bearer ")) {      // Line 4
            filterChain.doFilter(request, response);         // Line 5
            return;
        }

        // Step 3: Extract the token (remove "Bearer " prefix)
        String token = authHeader.substring(7);              // Line 6

        // Step 4: Validate the token
        if (jwtUtil.isTokenValid(token)) {                   // Line 7
            // Step 5: Extract user information from token
            String username = jwtUtil.extractUsername(token); // Line 8
            String role = jwtUtil.extractRole(token);        // Line 9

            // Step 6: Create an authentication object
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(
                    username,                                // Line 10
                    null,                                    // Line 11
                    List.of(new SimpleGrantedAuthority(
                        "ROLE_" + role))                     // Line 12
                );

            // Step 7: Set the authentication in the
            // security context
            SecurityContextHolder.getContext()
                .setAuthentication(authToken);                // Line 13
        }

        // Step 8: Continue the filter chain
        filterChain.doFilter(request, response);             // Line 14
    }

    @Override
    protected boolean shouldNotFilter(
            HttpServletRequest request) {
        String path = request.getRequestURI();
        return path.startsWith("/api/auth/")
            || path.startsWith("/h2-console");               // Line 15
    }
```

```
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`extends OncePerRequestFilter`	Ensures this filter runs exactly once per request
2	Constructor injection	Injects the `JwtUtil` for token operations
3	`request.getHeader("Authorization")`	Gets the Authorization header from the HTTP request
4	`!authHeader.startsWith("Bearer ")`	JWT tokens are sent as `Bearer <token>`. If the header is missing or does not start with "Bearer ", skip this filter
5	`filterChain.doFilter(...)` and `return`	No token found -- let the request continue. Other security rules will handle it
6	`authHeader.substring(7)`	Removes the "Bearer " prefix (7 characters) to get the actual token
7	`jwtUtil.isTokenValid(token)`	Validates the token's signature and checks if it is expired
8	`jwtUtil.extractUsername(token)`	Gets the username from the token's payload
9	`jwtUtil.extractRole(token)`	Gets the role from the token's payload
10	`username`	The principal (who the user is)
11	`null`	Credentials (not needed since we already verified the token)
12	`"ROLE_" + role`	The user's authorities. Spring Security requires the `ROLE_` prefix
13	`SecurityContextHolder...setAuthentication(...)`	Tells Spring Security that this request is authenticated. Now the user's identity is available throughout the request
14	`filterChain.doFilter(...)`	Continues processing the request with authentication set

15	`shouldNotFilter(...)`	Skips the JWT filter for auth endpoints (login/register) and H2 console

How the Filter Fits in the Chain

```
+-------------------------------------------------------------------+
|  Request with Authorization: Bearer eyJhbG...                     |
+-------------------------------------------------------------------+
        |
        v
+-------------------------------------------------------------------+
|  JwtAuthenticationFilter (your custom filter)                     |
|  1. Extract token from "Bearer eyJhbG..."                         |
|  2. Validate token (signature + expiration)                       |
|  3. Extract username and role                                     |
|  4. Create Authentication and set in SecurityContext              |
+-------------------------------------------------------------------+
        |
        v
+-------------------------------------------------------------------+
|  AuthorizationFilter (Spring Security's built-in)                 |
|  1. Check: Does this URL require authentication?                  |
|  2. Check: Does the user have the required role?                  |
|  3. If yes -> proceed to controller                               |
|  4. If no  -> 403 Forbidden                                       |
+-------------------------------------------------------------------+
        |
        v
+-------------------------------------------------------------------+
|  Your Controller                                                  |
+-------------------------------------------------------------------+
```

Step 8: Create the Auth Service

```java
package com.example.demo.service;

import com.example.demo.dto.LoginRequest;
import com.example.demo.dto.LoginResponse;
import com.example.demo.dto.RegisterRequest;
import com.example.demo.entity.AppUser;
import com.example.demo.repository.AppUserRepository;
import com.example.demo.security.JwtUtil;
```

```
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private final AppUserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    public AuthService(AppUserRepository userRepository,
                       PasswordEncoder passwordEncoder,
                       JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtil = jwtUtil;
    }

    /**
     * Register a new user.
     */
    public AppUser register(RegisterRequest request) {        // Line 1

        // Check if username already exists
        if (userRepository.existsByUsername(
                request.getUsername())) {                      // Line 2
            throw new RuntimeException(
                "Username already exists");
        }

        // Check if email already exists
        if (userRepository.existsByEmail(
                request.getEmail())) {                         // Line 3
            throw new RuntimeException(
                "Email already exists");
        }

        // Create new user with hashed password
        AppUser user = new AppUser();
        user.setUsername(request.getUsername());
        user.setPassword(
            passwordEncoder.encode(
                request.getPassword()));                       // Line 4
        user.setEmail(request.getEmail());
        user.setRole("USER");                                  // Line 5

        return userRepository.save(user);                      // Line 6
    }

    /**
     * Authenticate a user and return a JWT token.
     */
    public LoginResponse login(LoginRequest request) {        // Line 7

        // Find user by username
```

```
        AppUser user = userRepository
            .findByUsername(request.getUsername())
            .orElseThrow(() -> new RuntimeException(
                "Invalid username or password"));             // Line 8

        // Verify password
        if (!passwordEncoder.matches(
                request.getPassword(),
                user.getPassword())) {                        // Line 9
            throw new RuntimeException(
                "Invalid username or password");              // Line 10
        }

        // Generate JWT token
        String token = jwtUtil.generateToken(
            user.getUsername(), user.getRole());               // Line 11

        // Return response with token
        return new LoginResponse(
            token,
            user.getUsername(),
            user.getRole(),
            jwtUtil.getExpirationMs()                         // Line 12
        );
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`register(...)`	Handles new user registration
2	`existsByUsername(...)`	Checks for duplicate usernames before registration
3	`existsByEmail(...)`	Checks for duplicate emails
4	`passwordEncoder.encode(...)`	Hashes the password with BCrypt before storing. The plain text password is never saved
5	`user.setRole("USER")`	New users get the USER role by default
6	`userRepository.save(user)`	Saves the user to the database
7	`login(...)`	Handles user authentication

8	`findByUsername(...).orElseThrow(...)`	Finds the user or throws an error. The error message is intentionally vague to prevent username enumeration
9	`passwordEncoder.matches(rawPassword, hashedPassword)`	Compares the provided password with the stored hash. BCrypt handles the comparison
10	Same error message	We use the same error for both wrong username and wrong password. This prevents attackers from knowing whether a username exists
11	`jwtUtil.generateToken(...)`	Creates a JWT token for the authenticated user
12	`jwtUtil.getExpirationMs()`	Tells the client when the token will expire

Step 9: Create the Auth Controller

```java
package com.example.demo.controller;

import com.example.demo.dto.LoginRequest;
import com.example.demo.dto.LoginResponse;
import com.example.demo.dto.RegisterRequest;
import com.example.demo.entity.AppUser;
import com.example.demo.service.AuthService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }
```

```
    @PostMapping("/register")
    public ResponseEntity<?> register(
            @RequestBody RegisterRequest request) {           // Line 1
        try {
            AppUser user = authService.register(request);
            return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(Map.of(
                    "message", "User registered successfully",
                    "username", user.getUsername()
                ));
        } catch (RuntimeException e) {
            return ResponseEntity
                .badRequest()
                .body(Map.of("error", e.getMessage()));       // Line 2
        }
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(
            @RequestBody LoginRequest request) {              // Line 3
        try {
            LoginResponse response = authService.login(request);
            return ResponseEntity.ok(response);               // Line 4
        } catch (RuntimeException e) {
            return ResponseEntity
                .status(HttpStatus.UNAUTHORIZED)
                .body(Map.of("error", e.getMessage()));       // Line 5
        }
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`ResponseEntity<?>`	The `?` wildcard allows returning different response types (success or error)
2	`badRequest()`	Returns 400 Bad Request for registration errors (duplicate username/email)
3	`@RequestBody LoginRequest`	Deserializes the JSON request body into a `LoginRequest` object
4	`ResponseEntity.ok(response)`	Returns 200 OK with the JWT token
5	`HttpStatus.UNAUTHORIZED`	Returns 401 for invalid credentials

Step 10: Configure Security

```java
package com.example.demo.config;

import com.example.demo.filter.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication
    .UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())                    // Line 1

            .sessionManagement(session -> session
                .sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))         // Line 2

            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()  // Line 3
                .requestMatchers("/h2-console/**").permitAll()
                .requestMatchers("/api/admin/**")
                    .hasRole("ADMIN")                        // Line 4
                .anyRequest().authenticated()                // Line 5
            )

            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class)   // Line 6

            .headers(headers ->
```

```
                headers.frameOptions(frame ->
                    frame.sameOrigin()));

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Line-by-line explanation:

Line	Code	What It Does
1	`csrf.disable()`	Disables CSRF because we use stateless JWT authentication, not sessions
2	`SessionCreationPolicy.STATELESS`	Tells Spring to never create HTTP sessions. Every request must include a JWT token. This is key for REST APIs
3	`"/api/auth/**".permitAll()`	Login and registration endpoints are public (no token needed)
4	`hasRole("ADMIN")`	Only admin users can access admin endpoints
5	`anyRequest().authenticated()`	Everything else requires a valid JWT token
6	`addFilterBefore(jwtFilter, ...)`	Inserts your JWT filter before Spring's default authentication filter. This ensures JWT tokens are processed before any other authentication

---

Step 11: Create a Protected Controller

```java
package com.example.demo.controller;

import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping
    public List<Map<String, Object>> getAllProducts(
            Authentication auth) {

        return List.of(
            Map.of("id", 1, "name", "Laptop",
                   "price", 999.99,
                   "requestedBy", auth.getName()),
            Map.of("id", 2, "name", "Phone",
                   "price", 699.99,
                   "requestedBy", auth.getName())
        );
    }
}
```

```java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    @GetMapping("/dashboard")
    public Map<String, Object> dashboard() {
        return Map.of(
            "message", "Admin Dashboard",
            "stats", Map.of(
                "users", 150,
                "products", 45
            )
        );
    }
}
```

Testing the Complete Flow

Step 1: Register a New User

```
POST http://localhost:8080/api/auth/register
Content-Type: application/json

{
    "username": "alice",
    "password": "password123",
    "email": "alice@example.com"
}
```

Response (201 Created):

```json
{
    "message": "User registered successfully",
    "username": "alice"
}
```

Console SQL output:

```sql
Hibernate:
    insert into app_users (email, password, role, username)
    values (?, ?, ?, ?)
```

Step 2: Log In

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
    "username": "alice",
    "password": "password123"
}
```

Response (200 OK):

```json
{
    "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIsInJvbGUiOiJVU0VSIiwiaWF0IjoxNzA5...
    "username": "alice",
    "role": "USER",
    "expiresIn": 7200000
}
```

Step 3: Access a Protected Endpoint

```
GET http://localhost:8080/api/products
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIsInJvbGUiOiJVU0VSIiwiaWF0...
```

Response (200 OK):

```json
[
    {"id": 1, "name": "Laptop", "price": 999.99, "requestedBy": "alice"},
    {"id": 2, "name": "Phone", "price": 699.99, "requestedBy": "alice"}
]
```

Step 4: Access Without Token

```
GET http://localhost:8080/api/products
```

Response (401 Unauthorized):

```json
{
    "error": "Unauthorized",
    "status": 401
}
```

Step 5: Access Admin Endpoint (alice has USER role, not ADMIN)

```
GET http://localhost:8080/api/admin/dashboard
Authorization: Bearer eyJhbG...(alice's token)
```

Response (403 Forbidden):

```json
{
    "error": "Forbidden",
    "status": 403
}
```

Complete Authentication Flow Diagram

```
+-----------------------------------------------------------------+
|                                                                 |
|  REGISTRATION FLOW                                              |
|  ================                                              |
|                                                                 |
|  Client                    Server                               |
|    |                         |                                  |
|    |  POST /api/auth/register|                                  |
|    |  {username, password,   |                                  |
|    |   email}                |                                  |
|    |------------------------>|                                  |
|    |                         |  1. Check username not taken     |
|    |                         |  2. Check email not taken        |
|    |                         |  3. Hash password (BCrypt)       |
|    |                         |  4. Save user to database       |
|    |   201 Created           |                                  |
|    |<------------------------|                                  |
|                                                                 |
|                                                                 |
|  LOGIN FLOW                                                     |
|  ==========                                                    |
|                                                                 |
|  Client                    Server                               |
|    |                         |                                  |
|    |  POST /api/auth/login   |                                  |
|    |  {username, password}   |                                  |
|    |------------------------>|                                  |
|    |                         |  1. Find user by username        |
|    |                         |  2. Verify password (BCrypt)     |
|    |                         |  3. Generate JWT token           |
|    |   200 OK                |                                  |
|    |   {token, username,     |                                  |
|    |    role, expiresIn}     |                                  |
|    |<------------------------|                                  |
|    |                         |                                  |
|    |  (Client stores token)  |                                  |
|                                                                 |
|                                                                 |
|  AUTHENTICATED REQUEST FLOW                                     |
|  ==========================                                    |
|                                                                 |
```

```
|  Client                    Server                              |
|    |                         |                                 |
|    |  GET /api/products      |                                 |
|    |  Authorization:         |                                 |
|    |  Bearer eyJhbG...       |                                 |
|    |------------------------>|                                 |
|    |                         |  1. JwtFilter extracts token    |
|    |                         |  2. Validate signature          |
|    |                         |  3. Check not expired           |
|    |                         |  4. Extract username + role     |
|    |                         |  5. Set SecurityContext         |
|    |                         |  6. AuthorizationFilter checks  |
|    |                         |     role permissions            |
|    |                         |  7. Controller handles request  |
|    |   200 OK + data         |                                 |
|    |<------------------------|                                 |
|                                                                |
+----------------------------------------------------------------+
```

Common Mistakes

Mistake 1: Storing JWT Secret in Code

```java
// WRONG - Secret key hardcoded in source code
private static final String SECRET = "mySecretKey123";
// Anyone who sees your code knows the secret!

// CORRECT - Use application properties or environment variables
@Value("${jwt.secret}")
private String secret;

// BEST - Use environment variables in production
// Set JWT_SECRET=... in your deployment environment
// application.properties:
// jwt.secret=${JWT_SECRET}
```

Mistake 2: Not Adding "ROLE_" Prefix in the Filter

```java
// WRONG - Missing ROLE_ prefix
new SimpleGrantedAuthority(role)  // role = "USER"
// Spring checks for "ROLE_USER" but finds "USER"
// Authorization will fail!
```

```
// CORRECT - Add the ROLE_ prefix
new SimpleGrantedAuthority("ROLE_" + role)  // becomes "ROLE_USER"
```

Mistake 3: Using Sessions with JWT

```java
// WRONG - Creating sessions defeats the purpose of JWT
// If you use JWT, the server should be stateless
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED));

// CORRECT - Stateless session management with JWT
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

Mistake 4: Not Handling Token Expiration on the Client

```javascript
// WRONG - Client ignores token expiration
// Keeps using an expired token and gets 401 errors

// CORRECT - Client checks expiration and refreshes
// Store expiresIn from the login response
// Before each request, check if the token is about to expire
// If so, redirect to login or use a refresh token
```

Mistake 5: Sending Token in URL

```
// WRONG - Token visible in URL, browser history, server logs
GET /api/products?token=eyJhbG...

// CORRECT - Token in Authorization header (not logged by default)
GET /api/products
Authorization: Bearer eyJhbG...
```

Best Practices

1. Use environment variables for secrets: Never hardcode JWT secrets. Use environment variables or a secrets manager in production.

2. Set reasonable expiration times: Short-lived tokens (15 minutes to 2 hours) are more secure. Use refresh tokens for longer sessions.
3. Use HTTPS in production: JWTs are not encrypted -- they are only signed. Without HTTPS, anyone can read the token contents (including the username and role).
4. Do not store sensitive data in the payload: Remember, the JWT payload is Base64-encoded, not encrypted. Anyone can decode it. Never put passwords, credit card numbers, or other secrets in the payload.
5. Use a strong secret key: The secret must be at least 256 bits (32 characters) for HS256. Use a randomly generated key, not something guessable.
6. Return generic error messages: Do not say "username not found" or "wrong password." Say "invalid credentials" to prevent attackers from enumerating valid usernames.
7. Use stateless sessions: When using JWT, set `SessionCreationPolicy.STATELESS`. This ensures the server does not create sessions, and each request is independently authenticated.
8. Always validate tokens server-side: Never trust a token just because it exists. Always verify the signature and check the expiration.

Quick Summary

```
+-----------------------------------------+
|  JWT Authentication Quick Reference      |
+-----------------------------------------+
|                                          |
|  JWT STRUCTURE:                          |
|  Header.Payload.Signature               |
|                                          |
|  FLOW:                                   |
|  1. Register -> POST /api/auth/register |
|  2. Login    -> POST /api/auth/login    |
|     Response: {token: "eyJhbG..."}      |
|  3. Access   -> GET /api/products       |
|     Header: Authorization: Bearer token |
|                                          |
|  KEY CLASSES:                            |
|  JwtUtil    -> generate/validate tokens |
|  JwtFilter  -> intercept requests       |
|  AuthService -> login/register logic    |
|                                          |
|  DEPENDENCIES:                           |
|  jjwt-api + jjwt-impl + jjwt-jackson   |
|                                          |
|  SECURITY CONFIG:                        |
```

```
|  - CSRF disabled                        |
|  - Sessions stateless                   |
|  - Auth endpoints public                |
|  - JWT filter added before default      |
|                                          |
+------------------------------------------+
```

Key Points

1. A JWT is a self-contained token with three parts: header (algorithm), payload (claims/data), and signature (proof of authenticity).
2. The flow is: register, login (get token), and use the token in the `Authorization: Bearer <token>` header for all subsequent requests.
3. `JwtUtil` handles token generation and validation using the jjwt library.
4. `JwtAuthenticationFilter` extends `OncePerRequestFilter` and intercepts every request to extract and validate JWT tokens.
5. The security configuration must be stateless (`SessionCreationPolicy.STATELESS`) when using JWT.
6. Passwords are always hashed with BCrypt before storage. The plain text password is never saved.
7. JWT tokens are signed, not encrypted. Do not put sensitive data in the payload.
8. Use environment variables for the JWT secret key in production. Never hardcode it.

Practice Questions

1. What are the three parts of a JWT? What does each part contain?
2. Why is the signature important? What happens if someone modifies the payload of a JWT?
3. Explain the difference between `SessionCreationPolicy.STATELESS` and `SessionCreationPolicy.IF_REQUIRED`. Why must you use `STATELESS` with JWT?
4. Why do we use the same error message for both "user not found" and "wrong password" during login?
5. What is the purpose of `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`? Why does the JWT filter need to run

before the default authentication filter?

Exercises

Exercise 1: Refresh Token System

Implement a refresh token system:

- Short-lived access tokens (15 minutes)
- Long-lived refresh tokens (7 days) stored in the database
- `POST /api/auth/refresh` endpoint that accepts a refresh token and returns a new access token
- Invalidate refresh tokens on logout

Exercise 2: Role-Based Registration

Modify the registration to support creating ADMIN users:

- Only existing ADMIN users can create new ADMIN accounts
- Regular registration creates USER accounts
- Add a `POST /api/admin/create-admin` endpoint that requires ADMIN role

Exercise 3: Token Blacklisting

Implement a logout system:

- `POST /api/auth/logout` endpoint that invalidates the current token
- Store blacklisted tokens in an in-memory set or database table
- Check the blacklist in `JwtAuthenticationFilter` before accepting a token
- Clean up expired tokens from the blacklist periodically

What Is Next?

Congratulations! You now have a complete, secured Spring Boot application with JWT authentication. You can register users, log them in, and protect your API endpoints with role-based access control. In the chapters ahead, you will learn about exception handling,

validation, testing, and deploying your application to production.

Chapter 19: Password Encoding and Roles

What You Will Learn

- Why storing plain-text passwords is dangerous.
- What password hashing is and how BCrypt works.
- How to encode passwords during user registration.
- What roles and authorities are in Spring Security.
- How to assign roles like USER and ADMIN to users.
- How to restrict endpoints based on roles using @PreAuthorize.
- How to configure role-based access in SecurityFilterChain.
- How to build a complete registration and admin system.

Why This Chapter Matters

Imagine a bank that writes your PIN number on a sticky note and tapes it to the front of the vault. Anyone who walks by can see it. That is exactly what happens when you store passwords as plain text in a database.

In 2012, LinkedIn suffered a data breach that exposed over 6 million passwords. The passwords were hashed, but with a weak algorithm and no salt. Attackers cracked most of them within days.

Password encoding and role management are not optional features. They are the foundation of any secure application. In this chapter, you will learn how to store passwords safely using BCrypt and how to control what different users can do using roles.

19.1 The Problem with Plain-Text Passwords

Let us start with a simple question. What happens if you store passwords like this in your database?

id	username	password
1	alice	secret123
2	bob	mypassword
3	carol	qwerty

If an attacker gains access to your database (through a SQL injection, a misconfigured server, or a stolen backup), they can read every single password. They can log in as any user. They can even try these passwords on other websites since many people reuse passwords.

> Plain text: Storing data exactly as it was entered, without any transformation or protection.

This is why you must never store passwords in plain text. Ever.

19.2 What Is Password Hashing?

Hashing is a one-way transformation. You take a password, run it through a hashing function, and get a scrambled string. The key word is "one-way." You cannot reverse the process to get the original password back.

```
Password: "secret123"
    |
    v
[Hashing Function]
    |
    v
Hash: "$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy"
```

Think of hashing like putting meat through a meat grinder. You can turn a steak into ground beef, but you cannot turn ground beef back into a steak.

How Login Works with Hashing

When a user logs in, you do not decrypt the stored hash. Instead, you hash the password they just typed and compare the two hashes:

```
+----------------------------------------------+
|            LOGIN PROCESS                     |
+----------------------------------------------+
|                                              |
|  User types: "secret123"                     |
|       |                                      |
|       v                                      |
|  [Hash "secret123"]                          |
|       |                                      |
|       v                                      |
|  Computed: "$2a$10$N9qo8u..."                |
|       |                                      |
|       v                                      |
|  [Compare with stored hash]                  |
|       |                                      |
|       +---> Match? --> Login successful       |
|       +---> No match? --> Login failed        |
|                                              |
+----------------------------------------------+
```

What Is a Salt?

A salt is a random string added to the password before hashing. This means that even if two users have the same password, their hashes will be different.

```
User Alice: "secret123" + "random_salt_1" --> Hash A
User Bob:   "secret123" + "random_salt_2" --> Hash B

Hash A != Hash B (even though the passwords are the same)
```

Without a salt, an attacker could pre-compute hashes for common passwords (called a "rainbow table") and look up matches. Salting defeats this attack.

19.3 BCrypt: The Industry Standard

BCrypt is a password hashing algorithm designed specifically for passwords. It has three important features:

1. It is slow on purpose. This makes brute-force attacks impractical. A regular hash function can compute billions of hashes per second. BCrypt is designed to be slow.

2. It includes a salt automatically. You do not need to generate and store salts separately.
3. It has a configurable work factor. You can make it slower as computers get faster.

Here is what a BCrypt hash looks like:

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
 |  |  |                                                    |
 |  |  +--- Salt (22 characters) ---+                       |
 |  +------ Work factor (10 = 2^10 = 1024 iterations)       |
 +--------- Algorithm version (2a = BCrypt)                  |
            +--- Hash value (31 characters) -------------------+
```

Spring Security provides `BCryptPasswordEncoder` which handles all of this for you.

19.4 Setting Up the Project

Let us build a complete user registration system with password encoding and roles. Here is the Maven dependency section:

```xml
<!-- pom.xml -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

And the application properties:

```properties
# src/main/resources/application.properties

spring.datasource.url=jdbc:h2:mem:securitydb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

19.5 Creating the User Entity

Our User entity will store the username, encoded password, and role:

```java
// src/main/java/com/example/security/entity/User.java
package com.example.security.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;

@Entity
@Table(name = "app_user")  // "user" is a reserved word in H2
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String role;  // e.g., "ROLE_USER" or "ROLE_ADMIN"

    public User() {
    }

    public User(String username, String password, String role) {
        this.username = username;
```

```
        this.password = password;
        this.role = role;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

Line-by-line explanation:

- `@Table(name = "app_user")` -- We use "app_user" because "user" is a reserved keyword in H2 and many other databases.
- `@Column(nullable = false, unique = true)` on username -- Each username must be unique and cannot be null.
- `private String role` -- We store the role as a string. Spring Security expects roles to start with "ROLE_" (like "ROLE_USER" or "ROLE_ADMIN").

19.6 The User Repository

```java
// src/main/java/com/example/security/repository/UserRepository.java
package com.example.security.repository;

import com.example.security.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    boolean existsByUsername(String username);
}
```

Line-by-line explanation:

- `findByUsername` -- Spring Data JPA generates the SQL query automatically. Returns an Optional because the user might not exist.
- `existsByUsername` -- Returns true if a user with that username already exists. Useful for checking duplicates during registration.

19.7 Configuring BCryptPasswordEncoder

You need to tell Spring Security to use BCrypt. You do this by creating a bean:

```java
// src/main/java/com/example/security/config/SecurityConfig.java
package com.example.security.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Line-by-line explanation:

- `@Bean` -- This method creates a PasswordEncoder object and registers it with Spring's container.
- `new BCryptPasswordEncoder()` -- Creates a BCrypt encoder with the default strength of 10 (meaning 2^10 = 1024 hashing iterations).
- The return type is `PasswordEncoder` (the interface), not `BCryptPasswordEncoder` (the implementation). This follows the programming-to-interfaces principle.

What Happens When You Encode a Password

```java
PasswordEncoder encoder = new BCryptPasswordEncoder();

String rawPassword = "secret123";
String encodedPassword = encoder.encode(rawPassword);
```

```
System.out.println(encodedPassword);
// Output: $2a$10$dXJ3SW6G7P50lGmMQoeM0uO8Kd3FdGqBm/LqoC7Lxxx0n6PfGmRi2

// Verify a password
boolean matches = encoder.matches("secret123", encodedPassword);
System.out.println(matches);  // true

boolean wrong = encoder.matches("wrongpassword", encodedPassword);
System.out.println(wrong);  // false
```

Notice that you never decrypt the hash. You use `matches()` to check if a raw password matches the stored hash.

19.8 The Registration DTO

We use a Data Transfer Object to receive registration data. This keeps our entity separate from the API input:

```java
// src/main/java/com/example/security/dto/RegistrationRequest.java
package com.example.security.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class RegistrationRequest {

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be 3-50 characters")
    private String username;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;

    public RegistrationRequest() {
    }

    public RegistrationRequest(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
```

```
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

19.9 The User Service with Password Encoding

This is where the magic happens. The service encodes the password before saving:

```java
// src/main/java/com/example/security/service/UserService.java
package com.example.security.service;

import com.example.security.dto.RegistrationRequest;
import com.example.security.entity.User;
import com.example.security.repository.UserRepository;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public UserService(UserRepository userRepository,
                       PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public User registerUser(RegistrationRequest request) {
        // Check if username already exists
        if (userRepository.existsByUsername(request.getUsername())) {
            throw new RuntimeException("Username already taken: "
                + request.getUsername());
        }

        // Create user with encoded password and default USER role
        User user = new User(
            request.getUsername(),
            passwordEncoder.encode(request.getPassword()),  // ENCODE here!
            "ROLE_USER"
        );

        return userRepository.save(user);
    }
```

```
    public User registerAdmin(RegistrationRequest request) {
        if (userRepository.existsByUsername(request.getUsername())) {
            throw new RuntimeException("Username already taken: "
                + request.getUsername());
        }

        User user = new User(
            request.getUsername(),
            passwordEncoder.encode(request.getPassword()),
            "ROLE_ADMIN"
        );

        return userRepository.save(user);
    }
}
```

Line-by-line explanation:

- `passwordEncoder.encode(request.getPassword())` -- This is the critical line. The raw password "secret123" gets transformed into a BCrypt hash like "$2a$10$dXJ3SW6G7P..." before being saved to the database.
- `"ROLE_USER"` -- The default role for regular users. The "ROLE_" prefix is a Spring Security convention.
- `registerAdmin` -- A separate method for creating admin users. In a real application, only existing admins should be able to call this.

Here is what the flow looks like:

```
+-----------------------------------------------------+
|           REGISTRATION FLOW                         |
+-----------------------------------------------------+
|                                                     |
|  User sends: { "username": "alice",                 |
|                "password": "secret123" }            |
|       |                                             |
|       v                                             |
|  [Check: username "alice" exists?]                  |
|       |                                             |
|       +--> Yes --> Throw "Username taken"           |
|       |                                             |
|       +--> No --> Continue                          |
|       |                                             |
|       v                                             |
|  [Encode "secret123" with BCrypt]                   |
|       |                                             |
|       v                                             |
|  Result: "$2a$10$N9qo8uLO..."                       |
|       |                                             |
|       v                                             |
```

```
|  [Save to database]                                |
|  id=1, username="alice",                           |
|  password="$2a$10$N9qo8uLO...",                    |
|  role="ROLE_USER"                                  |
|                                                    |
+----------------------------------------------------+
```

19.10 The Custom UserDetailsService

Spring Security needs a way to load users during login. You provide this by implementing `UserDetailsService`:

```java
// src/main/java/com/example/security/service/CustomUserDetailsService.java
package com.example.security.service;

import com.example.security.entity.User;
import com.example.security.repository.UserRepository;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found: " + username));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            Collections.singletonList(
```

```
                new SimpleGrantedAuthority(user.getRole())
            )
        );
    }
}
```

Line-by-line explanation:

- `implements UserDetailsService` -- This interface has one method: `loadUserByUsername`. Spring Security calls it automatically when someone tries to log in.
- `orElseThrow(...)` -- If the user is not found in the database, throw an exception. Spring Security catches this and returns a "bad credentials" response.
- `new org.springframework.security.core.userdetails.User(...)` -- We create a Spring Security `User` object (different from our entity). It holds the username, encoded password, and authorities.
- `new SimpleGrantedAuthority(user.getRole())` -- Converts our role string (like "ROLE_ADMIN") into a Spring Security authority object.

19.11 Understanding Roles and Authorities

In Spring Security, roles and authorities are closely related but slightly different:

Concept	Description	Example
Authority	A permission string	"ROLE_USER", "READ_PRIVILEGE"
Role	An authority with the "ROLE_" prefix	"ROLE_ADMIN"

When you use `hasRole("ADMIN")` in Spring Security, it automatically checks for the authority "ROLE_ADMIN". This is why we store roles with the "ROLE_" prefix.

```
+----------------------------------------+
|       ROLES vs AUTHORITIES              |
+----------------------------------------+
|                                        |
|  hasRole("ADMIN")                      |
|       |                                |
|       v                                |
|  Checks for authority "ROLE_ADMIN"     |
```

```
|                                        |
|  hasAuthority("ROLE_ADMIN")            |
|       |                                |
|       v                                |
|  Checks for authority "ROLE_ADMIN"     |
|                                        |
|  Both do the same thing!               |
+----------------------------------------+
```

Think of it this way. A role is like a job title (Manager, Employee). An authority is a specific permission (CAN_APPROVE_EXPENSES, CAN_VIEW_REPORTS). In simple applications, roles are enough. In complex applications, you might use fine-grained authorities.

19.12 Role-Based SecurityFilterChain

Now let us configure which endpoints each role can access:

```java
// src/main/java/com/example/security/config/SecurityConfig.java
package com.example.security.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.EnableMethod...
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity  // Enables @PreAuthorize
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
```

```
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // Public endpoints - anyone can access
                .requestMatchers("/api/auth/register").permitAll()
                .requestMatchers("/h2-console/**").permitAll()

                // Admin-only endpoints
                .requestMatchers("/api/admin/**").hasRole("ADMIN")

                // User and Admin can access
                .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")

                // Everything else requires authentication
                .anyRequest().authenticated()
            )
            .httpBasic(httpBasic -> {});  // Use HTTP Basic for simplicity

        // Allow H2 console frames
        http.headers(headers ->
            headers.frameOptions(frame -> frame.disable()));

        return http.build();
    }
}
```

Line-by-line explanation:

- `@EnableMethodSecurity` -- Enables method-level security annotations like `@PreAuthorize`. Without this, those annotations are ignored.
- `.csrf(csrf -> csrf.disable())` -- Disables CSRF protection. Necessary for stateless REST APIs. We covered this in Chapter 17.
- `.requestMatchers("/api/auth/register").permitAll()` -- The registration endpoint is public. Anyone can create an account.
- `.requestMatchers("/api/admin/**").hasRole("ADMIN")` -- Only users with ROLE_ADMIN can access URLs starting with "/api/admin/". The `**` matches any path under "/api/admin/".
- `.requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")` -- Both regular users and admins can access these endpoints.
- `.anyRequest().authenticated()` -- Any URL not listed above requires the user to be logged in, regardless of role.
- `.httpBasic(httpBasic -> {})` -- Enables HTTP Basic authentication. The browser sends username:password encoded in Base64 with each request.

Here is a visual summary of the access rules:

```
+----------------------------------------------+
|         ENDPOINT ACCESS MATRIX               |
+----------------------------------------------+
|                                              |
|  Endpoint              Anonymous  USER  ADMIN|
|  ------------------------------------------------
|  POST /api/auth/register    OK     OK    OK  |
|  GET  /h2-console           OK     OK    OK  |
|  GET  /api/user/profile     --     OK    OK  |
|  GET  /api/admin/users      --     --    OK  |
|  DELETE /api/admin/users/1  --     --    OK  |
|  GET  /api/anything-else    --     OK    OK  |
|                                              |
|  OK = Allowed, -- = Denied (403 Forbidden)   |
+----------------------------------------------+
```

19.13 Using @PreAuthorize for Method-Level Security

Sometimes you want to control access at the method level instead of the URL level. The `@PreAuthorize` annotation does this:

```java
// src/main/java/com/example/security/controller/AdminController.java
package com.example.security.controller;

import com.example.security.entity.User;
import com.example.security.repository.UserRepository;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final UserRepository userRepository;

    public AdminController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
```

```
    @GetMapping("/users")
    @PreAuthorize("hasRole('ADMIN')")
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    @DeleteMapping("/users/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteUser(@PathVariable Long id) {
        userRepository.deleteById(id);
        return "User deleted successfully";
    }

    @GetMapping("/dashboard")
    @PreAuthorize("hasRole('ADMIN') or hasRole('MANAGER')")
    public String adminDashboard() {
        return "Welcome to the Admin Dashboard";
    }
}
```

Line-by-line explanation:

- `@PreAuthorize("hasRole('ADMIN')")` -- Before this method runs, Spring checks if the logged-in user has the ADMIN role. If not, it returns 403 Forbidden.
- `hasRole('ADMIN') or hasRole('MANAGER')` -- You can combine conditions. This method is accessible by admins OR managers.
- The `@PreAuthorize` annotation uses Spring Expression Language (SpEL). Common expressions include:
  - `hasRole('ADMIN')` -- Has a specific role.
  - `hasAnyRole('ADMIN', 'MANAGER')` -- Has any of the listed roles.
  - `hasAuthority('ROLE_ADMIN')` -- Has a specific authority (same as hasRole but you include the ROLE_ prefix).
  - `isAuthenticated()` -- Is logged in.
  - `permitAll()` -- Everyone can access.

19.14 The Registration Controller

```java
// src/main/java/com/example/security/controller/AuthController.java
package com.example.security.controller;
```

```
import com.example.security.dto.RegistrationRequest;
import com.example.security.entity.User;
import com.example.security.service.UserService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final UserService userService;

    public AuthController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping("/register")
    public ResponseEntity<Map<String, String>> register(
            @Valid @RequestBody RegistrationRequest request) {

        User user = userService.registerUser(request);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(Map.of(
                "message", "User registered successfully",
                "username", user.getUsername(),
                "role", user.getRole()
            ));
    }
}
```

19.15 The User Controller

```java
// src/main/java/com/example/security/controller/UserController.java
package com.example.security.controller;

import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/profile")
    public Map<String, Object> getProfile(Authentication authentication) {
        return Map.of(
            "username", authentication.getName(),
            "authorities", authentication.getAuthorities().toString()
        );
    }

    @GetMapping("/dashboard")
    public String userDashboard() {
        return "Welcome to your dashboard!";
    }
}
```

Line-by-line explanation:

- `Authentication authentication` -- Spring Security injects the current user's authentication object automatically. You can get the username and authorities from it.
- `authentication.getName()` -- Returns the username of the logged-in user.
- `authentication.getAuthorities()` -- Returns the list of roles/authorities the user has.

19.16 Testing the Complete System

Start the application and test with curl commands.

Step 1: Register a Regular User

```bash
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username": "alice", "password": "password123"}'
```

Output:

```json
{
    "message": "User registered successfully",
    "username": "alice",
    "role": "ROLE_USER"
}
```

Step 2: Access User Profile (with credentials)

```bash
curl -u alice:password123 http://localhost:8080/api/user/profile
```

Output:

```json
{
    "username": "alice",
    "authorities": "[ROLE_USER]"
}
```

Step 3: Try to Access Admin Endpoint as Regular User

```bash
curl -u alice:password123 http://localhost:8080/api/admin/users
```

Output:

```json
{
    "status": 403,
    "error": "Forbidden"
}
```

Alice has the USER role, so she cannot access admin endpoints. This is exactly what we want.

Step 4: Check the Database

Open `http://localhost:8080/h2-console` and connect. Run:

```sql
SELECT * FROM APP_USER;
```

Output:

```
ID | USERNAME | PASSWORD                                                | ROLE
1  | alice    | $2a$10$dXJ3SW6G7P50lGmMQoeM0uO8Kd3FdGqBm/LqoC7... | ROLE_USER
```

Notice that the password is stored as a BCrypt hash, not as "password123". Even if someone reads the database, they cannot determine the original password.

19.17 Creating an Admin Setup with CommandLineRunner

In many applications, you need a default admin user when the application starts. You can use `CommandLineRunner`:

```java
// src/main/java/com/example/security/config/DataInitializer.java
package com.example.security.config;

import com.example.security.entity.User;
import com.example.security.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class DataInitializer implements CommandLineRunner {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public DataInitializer(UserRepository userRepository,
                           PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public void run(String... args) {
        // Create default admin if not exists
        if (!userRepository.existsByUsername("admin")) {
            User admin = new User(
                "admin",
                passwordEncoder.encode("admin123"),
                "ROLE_ADMIN"
            );
            userRepository.save(admin);
```

```
            System.out.println("Default admin user created");
        }
    }
}
```

Line-by-line explanation:

- `implements CommandLineRunner` -- Spring Boot calls the `run` method after the application starts.
- `if (!userRepository.existsByUsername("admin"))` -- Only creates the admin if one does not already exist. This prevents duplicates on restart.
- `passwordEncoder.encode("admin123")` -- Even the default admin password gets encoded. Never store it in plain text.

Now you can test admin access:

```bash
curl -u admin:admin123 http://localhost:8080/api/admin/users
```

Output:

```json
[
    {
        "id": 1,
        "username": "admin",
        "password": "$2a$10$...",
        "role": "ROLE_ADMIN"
    }
]
```

> Warning: The password hash is visible in the response. In a real application, you should create a response DTO that excludes the password field. Never send password hashes to the client.

19.18 The Complete Request Flow

Here is what happens when a user makes a request to a protected endpoint:

```
+-----------------------------------------------------------------+
|                     REQUEST FLOW                                |
```

```
+-----------------------------------------------------------------+
|                                                                  |
|  1. Client sends request with credentials                        |
|     GET /api/admin/users                                         |
|     Authorization: Basic YWRtaW46YWRtaW4xMjM=                   |
|         |                                                        |
|         v                                                        |
|  2. SecurityFilterChain intercepts the request                   |
|         |                                                        |
|         v                                                        |
|  3. Extract credentials from Authorization header                |
|     username: "admin", password: "admin123"                      |
|         |                                                        |
|         v                                                        |
|  4. Call CustomUserDetailsService.loadUserByUsername("admin")    |
|     --> Load user from database                                  |
|         |                                                        |
|         v                                                        |
|  5. BCryptPasswordEncoder.matches("admin123", storedHash)        |
|     --> Compare raw password with stored BCrypt hash             |
|         |                                                        |
|         v                                                        |
|  6. Check: Does user have required role?                         |
|     /api/admin/** requires ROLE_ADMIN                            |
|     User has ROLE_ADMIN --> GRANTED                              |
|         |                                                        |
|         v                                                        |
|  7. Execute controller method                                    |
|     --> Return list of users                                     |
|                                                                  |
+-----------------------------------------------------------------+
```

Common Mistakes

1. Storing passwords in plain text. Always use `passwordEncoder.encode()` before saving passwords. Never store raw passwords in the database.
2. Forgetting the "ROLE_" prefix. When storing roles, use "ROLE_USER" not "USER". When checking with `hasRole("USER")`, Spring automatically adds the "ROLE_" prefix. If you use `hasAuthority()`, you must include the prefix yourself.
3. Encoding the password twice. Do not call `encode()` when the user logs in. Only encode during registration. Spring Security calls `matches()` automatically during authentication.
4. Not adding @EnableMethodSecurity. If you use `@PreAuthorize` but forget `@EnableMethodSecurity` on your config class, the annotations are silently ignored. Your endpoints will be unprotected.

5. Returning password hashes in API responses. Always create a response DTO that excludes the password field. Leaking hashes, while not as bad as plain text, still gives attackers material to work with.
6. Using a weak work factor. The default BCrypt strength of 10 is fine for most applications. Going below 10 is too weak. Going above 14 makes login painfully slow.

Best Practices

1. Always use BCrypt or Argon2 for password hashing. Never use MD5 or SHA-256 for passwords. They are too fast and designed for data integrity, not password storage.
2. Set minimum password length to at least 8 characters. Use validation annotations to enforce this.
3. Use the PasswordEncoder interface, not the concrete class. This makes it easy to switch algorithms later.
4. Create an admin account programmatically, not through SQL scripts. Use CommandLineRunner to ensure the admin password is properly encoded.
5. Apply the principle of least privilege. Give users the minimum role they need. Most users should be USER, not ADMIN.
6. Use @PreAuthorize for fine-grained control. It is easier to read and maintain than complex URL patterns in SecurityFilterChain.

Quick Summary

Password encoding protects user credentials by transforming them into irreversible hashes before storing them in the database. BCryptPasswordEncoder is the industry standard for Spring Boot applications. It automatically handles salting and uses a configurable work factor to resist brute-force attacks. Roles like USER and ADMIN control what different users can do. You define access rules in SecurityFilterChain using methods like hasRole() and hasAnyRole(). For method-level control, @PreAuthorize lets you specify role requirements directly on controller methods. Always encode passwords during registration, never store plain text, and never return password hashes in API responses.

Key Points

- Never store passwords in plain text. Always hash them with BCrypt.
- BCryptPasswordEncoder handles salting automatically.
- Use `encode()` during registration and `matches()` during login.
- Roles must have the "ROLE_" prefix when stored (e.g., "ROLE_USER").
- `hasRole("USER")` automatically checks for "ROLE_USER".
- @EnableMethodSecurity is required for @PreAuthorize to work.
- SecurityFilterChain defines URL-level access rules.
- @PreAuthorize defines method-level access rules.
- Use CommandLineRunner to create default admin users on startup.

Practice Questions

1. Why is BCrypt preferred over SHA-256 for password hashing?
2. What is the purpose of a salt in password hashing, and does BCrypt handle it automatically?
3. What happens if you store a role as "USER" instead of "ROLE_USER" and then use `hasRole("USER")` in your security configuration?
4. What is the difference between `hasRole("ADMIN")` and `hasAuthority("ROLE_ADMIN")`?
5. Why should you never call `passwordEncoder.encode()` during the login process?

Exercises

Exercise 1: Add a MANAGER Role

Extend the application to support a third role called MANAGER. Create a new endpoint at `/api/manager/reports` that only MANAGER and ADMIN users can access. Regular USERs should get a 403 Forbidden response.

Exercise 2: Create a User Response DTO

The current admin endpoint returns user objects with password hashes. Create a `UserResponse` DTO that includes only the id, username, and role. Modify the admin controller

to return this DTO instead of the entity.

Exercise 3: Password Strength Validation

Add custom validation to the registration endpoint that requires passwords to contain at least one uppercase letter, one lowercase letter, one digit, and one special character. Return helpful error messages when validation fails.

What Is Next?

Your application is now secure with password encoding and role-based access control. But what happens when a frontend application running on a different domain tries to call your API? The browser will block it because of CORS (Cross-Origin Resource Sharing). In Chapter 20, you will learn what CORS is, why browsers enforce it, and how to configure it in Spring Boot so your API can serve requests from any trusted frontend.

Chapter 20: CORS Configuration

What You Will Learn

- What CORS is and why browsers enforce it.
- What the Same-Origin Policy is.
- What preflight requests are and when they happen.
- How to use @CrossOrigin for controller-level CORS.
- How to configure global CORS with WebMvcConfigurer.
- How to make CORS work with Spring Security.
- Common CORS errors and how to fix them.

Why This Chapter Matters

Imagine you are at a restaurant. You order food, and the waiter brings it to your table. Everything works perfectly. Now imagine you try to reach over to the next table and grab their food. The waiter would stop you. That is not your table. That is not your order.

Browsers work the same way. When your frontend application at `http://localhost:3000` tries to call your Spring Boot API at `http://localhost:8080`, the browser says "Wait. Those are different origins. I am not sure you are allowed to do that." This is CORS -- Cross-Origin Resource Sharing.

If you have ever seen the error "Access to XMLHttpRequest has been blocked by CORS policy," this chapter will show you exactly what causes it and how to fix it. Almost every modern web application involves a frontend and backend running on different origins, so understanding CORS is essential.

20.1 Understanding the Same-Origin Policy

Before we talk about CORS, we need to understand the Same-Origin Policy. This is a security feature built into every web browser.

An origin is defined by three parts:

```
https://www.example.com:443/products
  |          |              |
  |          |              +-- Port (443)
  |          +----------------- Host (www.example.com)
  +----------------------------- Protocol (https)
```

Two URLs have the same origin only if all three parts match:

URL A	URL B	Same Origin?	Why?
`http://example.com`	`http://example.com/about`	Yes	Same protocol, host, port
`http://example.com`	`https://example.com`	No	Different protocol
`http://example.com`	`http://api.example.com`	No	Different host
`http://example.com`	`http://example.com:8080`	No	Different port
`http://localhost:3000`	`http://localhost:8080`	No	Different port

The Same-Origin Policy says: A web page can only make requests to the same origin it came from.

This means a React app running at `http://localhost:3000` cannot call a Spring Boot API at `http://localhost:8080`. The browser blocks it.

Why Does This Policy Exist?

Think about this scenario without the Same-Origin Policy:

```
+-------------------------------------------------------------+
|  WITHOUT SAME-ORIGIN POLICY (DANGEROUS)                     |
+-------------------------------------------------------------+
```

```
|                                                              |
|  1. You log into your bank at bank.com                       |
|     Browser stores your session cookie                       |
|          |                                                   |
|          v                                                   |
|  2. You visit evil-site.com in another tab                   |
|          |                                                   |
|          v                                                   |
|  3. evil-site.com runs JavaScript:                           |
|     fetch("https://bank.com/api/transfer",                   |
|       { method: "POST",                                      |
|         body: "amount=10000&to=evil-account" })              |
|          |                                                   |
|          v                                                   |
|  4. Browser sends the request WITH your bank cookie          |
|     Bank thinks it is YOU making the transfer                |
|          |                                                   |
|          v                                                   |
|  5. Your money is gone!                                      |
|                                                              |
+-------------------------------------------------------------+
```

The Same-Origin Policy prevents step 3. The browser blocks evil-site.com from making requests to bank.com.

20.2 What Is CORS?

CORS stands for Cross-Origin Resource Sharing. It is a mechanism that allows a server to say "I trust requests from these specific origins."

Think of CORS like a bouncer at a club. The Same-Origin Policy is the default rule: "Nobody gets in unless they are on the list." CORS is the VIP list. The server (the club owner) decides who gets on the list.

```
+----------------------------------------------+
|              CORS FLOW                       |
+----------------------------------------------+
|                                              |
|  React App                  Spring Boot API  |
|  (localhost:3000)           (localhost:8080)  |
|       |                          |           |
|       |  GET /api/products       |           |
|       |  Origin: localhost:3000  |           |
|       |-------------------------->|           |
|       |                          |           |
```

```
|       |                   [Check CORS config] |
|       |                   Is localhost:3000    |
|       |                   allowed?             |
|       |                          |             |
|       |  200 OK                  |             |
|       |  Access-Control-Allow-   |             |
|       |  Origin: localhost:3000  |             |
|       |<-------------------------|             |
|       |                          |             |
|  [Browser checks header]        |             |
|  Origin matches? YES            |             |
|  --> Allow response              |             |
|                                               |
+-----------------------------------------------+
```

The key CORS headers are:

Header	Purpose	Example
`Access-Control-Allow-Origin`	Which origins are allowed	`http://localhost:3000`
`Access-Control-Allow-Methods`	Which HTTP methods are allowed	`GET, POST, PUT, DELETE`
`Access-Control-Allow-Headers`	Which request headers are allowed	`Content-Type, Authorization`
`Access-Control-Allow-Credentials`	Whether cookies are allowed	`true`
`Access-Control-Max-Age`	How long to cache the preflight result	`3600` (seconds)

20.3 Preflight Requests

For "simple" requests (GET with no custom headers), the browser sends the request directly and checks the CORS headers in the response.

But for "complex" requests (POST with JSON body, requests with custom headers like Authorization), the browser sends a preflight request first. This is an OPTIONS request that asks the server "Am I allowed to do this?"

```
+-------------------------------------------------------+
|               PREFLIGHT FLOW                          |
+-------------------------------------------------------+
|                                                       |
|  Browser                        Spring Boot API       |
|      |                               |                |
|      |  1. OPTIONS /api/products     |                |
|      |  Origin: localhost:3000       |                |
|      |  Access-Control-Request-      |                |
|      |    Method: POST               |                |
|      |  Access-Control-Request-      |                |
|      |    Headers: Content-Type      |                |
|      |------------------------------>|                |
|      |                               |                |
|      |  2. 200 OK                    |                |
|      |  Access-Control-Allow-Origin: |                |
|      |    http://localhost:3000      |                |
|      |  Access-Control-Allow-Methods: |               |
|      |    GET, POST, PUT, DELETE     |                |
|      |  Access-Control-Allow-Headers: |               |
|      |    Content-Type               |                |
|      |<------------------------------|                |
|      |                               |                |
|      |  [Preflight passed!]          |                |
|      |                               |                |
|      |  3. POST /api/products        |                |
|      |  Content-Type: application/json|               |
|      |  Origin: localhost:3000       |                |
|      |  {"name": "Widget"}           |                |
|      |------------------------------>|                |
|      |                               |                |
|      |  4. 201 Created               |                |
|      |  Access-Control-Allow-Origin: |                |
|      |    http://localhost:3000      |                |
|      |<------------------------------|                |
|                                                       |
+-------------------------------------------------------+
```

When Does a Preflight Happen?

A preflight request is triggered when:

1. The HTTP method is anything other than GET, HEAD, or POST.
2. The Content-Type header is anything other than `application/x-www-form-urlencoded`, `multipart/form-data`, or `text/plain`.
3. Custom headers are present (like `Authorization` or `X-Custom-Header`).

Since most REST APIs use `Content-Type: application/json` and `Authorization` headers, almost every API call triggers a preflight.

20.4 CORS with @CrossOrigin (Controller Level)

The simplest way to enable CORS is to add the `@CrossOrigin` annotation to your controller or method:

```java
// src/main/java/com/example/cors/controller/ProductController.java
package com.example.cors.controller;

import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
import java.util.ArrayList;
import java.util.Map;

@RestController
@RequestMapping("/api/products")
@CrossOrigin(origins = "http://localhost:3000")  // Allow this origin
public class ProductController {

    private final List<Map<String, String>> products = new ArrayList<>();

    @GetMapping
    public List<Map<String, String>> getAllProducts() {
        return products;
    }

    @PostMapping
    public Map<String, String> addProduct(
            @RequestBody Map<String, String> product) {
        products.add(product);
        return product;
    }
}
```

Line-by-line explanation:

- `@CrossOrigin(origins = "http://localhost:3000")` -- This tells Spring to add CORS headers to all responses from this controller, allowing requests from `http://localhost:3000`.

More @CrossOrigin Options

```java
@CrossOrigin(
    origins = {"http://localhost:3000", "https://myapp.com"},
    methods = {RequestMethod.GET, RequestMethod.POST},
    allowedHeaders = {"Content-Type", "Authorization"},
    exposedHeaders = {"X-Custom-Header"},
    allowCredentials = "true",
    maxAge = 3600
)
```

Attribute	Purpose	Default
`origins`	Allowed origin URLs	All origins (*)
`methods`	Allowed HTTP methods	All methods from the mapping
`allowedHeaders`	Headers the client can send	All headers (*)
`exposedHeaders`	Headers the client can read from the response	None
`allowCredentials`	Allow cookies and authorization headers	`false`
`maxAge`	How long to cache preflight results (seconds)	1800 (30 minutes)

Method-Level @CrossOrigin

You can also apply CORS to individual methods:

```java
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping
    @CrossOrigin(origins = "http://localhost:3000")
    public List<Map<String, String>> getAllProducts() {
        return List.of(Map.of("name", "Widget"));
    }

    @PostMapping
    // No @CrossOrigin here -- only same-origin requests allowed
    public Map<String, String> addProduct(
            @RequestBody Map<String, String> product) {
        return product;
    }
}
```

In this example, `GET /api/products` allows cross-origin requests, but `POST /api/products` does not.

20.5 Global CORS Configuration with WebMvcConfigurer

Using `@CrossOrigin` on every controller gets repetitive. For global configuration, implement `WebMvcConfigurer`:

```java
// src/main/java/com/example/cors/config/CorsConfig.java
package com.example.cors.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")           // Apply to all /api/ URLs
            .allowedOrigins(
                "http://localhost:3000",          // React dev server
                "https://myapp.com"              // Production frontend
            )
            .allowedMethods(
                "GET", "POST", "PUT", "DELETE", "OPTIONS"
            )
            .allowedHeaders("*")                 // Allow all headers
            .allowCredentials(true)              // Allow cookies
            .maxAge(3600);                       // Cache preflight for 1 hour
    }
}
```

Line-by-line explanation:

- `implements WebMvcConfigurer` -- This interface lets you customize Spring MVC behavior. We override `addCorsMappings` to set CORS rules.
- `registry.addMapping("/api/**")` -- Apply these CORS rules to all URLs that start with "/api/". The `**` matches any path underneath.
- `.allowedOrigins(...)` -- List the exact origins that are allowed. You can list multiple origins.

- `.allowedMethods(...)` -- Which HTTP methods are allowed. Include "OPTIONS" for preflight requests.
- `.allowedHeaders("*")` -- Accept any header from the client. You could list specific headers instead.
- `.allowCredentials(true)` -- Allow cookies and Authorization headers. Important: When this is true, you cannot use `"*"` for allowedOrigins. You must list specific origins.
- `.maxAge(3600)` -- The browser can cache the preflight response for 3600 seconds (1 hour). This reduces the number of OPTIONS requests.

Multiple Path Patterns

You can define different CORS rules for different paths:

```java
@Override
public void addCorsMappings(CorsRegistry registry) {
    // Public API - allow any origin
    registry.addMapping("/api/public/**")
        .allowedOrigins("*")
        .allowedMethods("GET");

    // Protected API - only specific origins
    registry.addMapping("/api/protected/**")
        .allowedOrigins("https://myapp.com")
        .allowedMethods("GET", "POST", "PUT", "DELETE")
        .allowCredentials(true);
}
```

20.6 CORS with Spring Security

Here is the tricky part. If you are using Spring Security (and you should be), the CORS configuration from `WebMvcConfigurer` alone is not enough. Spring Security has its own filter chain that runs before your controllers. If CORS is not configured in the security filter chain, preflight OPTIONS requests will be blocked with a 401 or 403.

```
+-------------------------------------------------------+
|  WITHOUT CORS IN SECURITY CONFIG                      |
+-------------------------------------------------------+
|                                                       |
|  Browser sends OPTIONS preflight                      |
|       |                                               |
|       v                                               |
```

```
|  [Spring Security Filter Chain]                        |
|  "This request has no credentials!"                    |
|  --> 401 Unauthorized                                  |
|                                                        |
|  The actual request never reaches your controller      |
|  and WebMvcConfigurer CORS config is never applied     |
|                                                        |
+-------------------------------------------------------+
```

The fix is to configure CORS in your SecurityFilterChain:

```java
// src/main/java/com/example/cors/config/SecurityConfig.java
package com.example.cors.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors
                .configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic(httpBasic -> {});

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        // Which origins are allowed
        configuration.setAllowedOrigins(List.of(
            "http://localhost:3000",
            "https://myapp.com"
```

```
        ));

        // Which HTTP methods are allowed
        configuration.setAllowedMethods(List.of(
            "GET", "POST", "PUT", "DELETE", "OPTIONS"
        ));

        // Which headers are allowed
        configuration.setAllowedHeaders(List.of("*"));

        // Allow cookies and authorization headers
        configuration.setAllowCredentials(true);

        // Cache preflight for 1 hour
        configuration.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source =
            new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", configuration);

        return source;
    }
}
```

Line-by-line explanation:

- `.cors(cors -> cors.configurationSource(corsConfigurationSource()))` -- This tells Spring Security to use our CORS configuration. It processes CORS before authentication, so preflight requests are handled correctly.
- `new CorsConfiguration()` -- Creates a CORS configuration object where we set our rules.
- `configuration.setAllowedOrigins(...)` -- Same as `allowedOrigins` in WebMvcConfigurer, but configured for Spring Security.
- `new UrlBasedCorsConfigurationSource()` -- Maps URL patterns to CORS configurations. You can have different CORS rules for different paths.
- `source.registerCorsConfiguration("/api/**", configuration)` -- Apply our CORS configuration to all URLs starting with "/api/".

```
+-------------------------------------------------------+
|  WITH CORS IN SECURITY CONFIG                         |
+-------------------------------------------------------+
|                                                       |
|  Browser sends OPTIONS preflight                      |
|       |                                               |
|       v                                               |
|  [Spring Security Filter Chain]                       |
|  [CorsFilter] <-- Handles CORS before auth            |
|  "Origin http://localhost:3000 is allowed"            |
|  --> 200 OK with CORS headers                         |
```

```
|       |                                                |
|       v                                                |
|  Browser sends actual request                          |
|       |                                                |
|       v                                                |
|  [Authentication Filter]                               |
|  [Authorization Filter]                                |
|       |                                                |
|       v                                                |
|  [Controller] --> Response with CORS headers           |
|                                                        |
+--------------------------------------------------------+
```

20.7 Using Properties for CORS Configuration

Hard-coding origins in your Java code is not ideal. You might want different origins for development and production. You can use application properties:

```properties
# src/main/resources/application.properties

cors.allowed-origins=http://localhost:3000,http://localhost:5173
cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
cors.max-age=3600
```

```properties
# src/main/resources/application-prod.properties

cors.allowed-origins=https://myapp.com,https://www.myapp.com
cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
cors.max-age=3600
```

Then read these properties in your configuration:

```java
// src/main/java/com/example/cors/config/CorsProperties.java
package com.example.cors.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Configuration;

import java.util.List;

@Configuration
@ConfigurationProperties(prefix = "cors")
```

```
public class CorsProperties {

    private List<String> allowedOrigins;
    private List<String> allowedMethods;
    private long maxAge;

    // Getters and setters
    public List<String> getAllowedOrigins() { return allowedOrigins; }
    public void setAllowedOrigins(List<String> allowedOrigins) {
        this.allowedOrigins = allowedOrigins;
    }

    public List<String> getAllowedMethods() { return allowedMethods; }
    public void setAllowedMethods(List<String> allowedMethods) {
        this.allowedMethods = allowedMethods;
    }

    public long getMaxAge() { return maxAge; }
    public void setMaxAge(long maxAge) { this.maxAge = maxAge; }
}
```

```java
// Updated SecurityConfig.java
@Bean
public CorsConfigurationSource corsConfigurationSource(
        CorsProperties corsProperties) {

    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(corsProperties.getAllowedOrigins());
    configuration.setAllowedMethods(corsProperties.getAllowedMethods());
    configuration.setAllowedHeaders(List.of("*"));
    configuration.setAllowCredentials(true);
    configuration.setMaxAge(corsProperties.getMaxAge());

    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", configuration);

    return source;
}
```

Now you can change CORS settings without modifying code, and use different settings per environment.

20.8 Testing CORS

Testing with curl

You can simulate a CORS preflight request with curl:

```bash
# Simulate a preflight request
curl -v -X OPTIONS http://localhost:8080/api/products \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Content-Type"
```

Expected Output (CORS configured correctly):

```
< HTTP/1.1 200
< Access-Control-Allow-Origin: http://localhost:3000
< Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
< Access-Control-Allow-Headers: *
< Access-Control-Allow-Credentials: true
< Access-Control-Max-Age: 3600
```

Expected Output (CORS NOT configured):

```
< HTTP/1.1 403
```

No CORS headers in the response means the browser will block the request.

Testing with a Simple HTML Page

Create a simple HTML file to test CORS from the browser:

```html
<!-- test-cors.html (open this in a browser) -->
<!DOCTYPE html>
<html>
<body>
    <h1>CORS Test</h1>
    <button onclick="testCors()">Test CORS</button>
    <pre id="result"></pre>

    <script>
        async function testCors() {
            try {
                const response = await fetch(
                    'http://localhost:8080/api/products',
                    {
                        method: 'GET',
                        headers: {
                            'Content-Type': 'application/json'
                        }
```

```
                    }
                );
                const data = await response.json();
                document.getElementById('result').textContent =
                    JSON.stringify(data, null, 2);
            } catch (error) {
                document.getElementById('result').textContent =
                    'CORS Error: ' + error.message;
            }
        }
    </script>
</body>
</html>
```

Open this file in your browser (it will have the origin `null` or `file://`). If CORS is not configured to allow that origin, you will see the error.

20.9 Common CORS Errors and Solutions

Error 1: "No 'Access-Control-Allow-Origin' header is present"

```
Access to XMLHttpRequest at 'http://localhost:8080/api/products'
from origin 'http://localhost:3000' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

Cause: CORS is not configured on the server, or the origin is not in the allowed list.

Fix: Add CORS configuration and include the origin.

Error 2: "The value of 'Access-Control-Allow-Credentials' header must be 'true'"

```
Access to XMLHttpRequest at 'http://localhost:8080/api/products'
from origin 'http://localhost:3000' has been blocked by CORS policy:
The value of the 'Access-Control-Allow-Credentials' header in the response
is '' which must be 'true' when the request's credentials mode is 'include'.
```

Cause: Your frontend sends `credentials: 'include'` but the server does not set `allowCredentials(true)`.

Fix: Add `.allowCredentials(true)` to your CORS configuration.

Error 3: "Wildcard '*' cannot be used with credentials"

```
Access to XMLHttpRequest has been blocked by CORS policy:
The value of the 'Access-Control-Allow-Origin' header must not be the
wildcard '*' when the request's credentials mode is 'include'.
```

Cause: You set `allowCredentials(true)` with `allowedOrigins("*")`. This is not allowed for security reasons.

Fix: List specific origins instead of using the wildcard:

```java
// Wrong
configuration.setAllowedOrigins(List.of("*"));
configuration.setAllowCredentials(true);

// Correct
configuration.setAllowedOrigins(List.of(
    "http://localhost:3000"
));
configuration.setAllowCredentials(true);

// Or use allowedOriginPatterns for wildcard with credentials
configuration.setAllowedOriginPatterns(List.of("*"));
configuration.setAllowCredentials(true);
```

Error 4: "Method PUT is not allowed"

```
Access to XMLHttpRequest has been blocked by CORS policy:
Method PUT is not allowed by Access-Control-Allow-Methods
in preflight response.
```

Cause: Your CORS configuration does not include PUT in the allowed methods.

Fix: Add PUT to the allowed methods list:

```java
configuration.setAllowedMethods(List.of(
    "GET", "POST", "PUT", "DELETE", "OPTIONS"
));
```

Error 5: CORS Works Locally but Not in Production

Cause: Your production configuration does not include the production frontend origin.

Fix: Make sure your production properties include the correct production URL:

```properties
# application-prod.properties
cors.allowed-origins=https://myapp.com,https://www.myapp.com
```

20.10 CORS Decision Flowchart

Use this flowchart to decide how to configure CORS:

```
+-----------------------------------------------------+
|     Do you need CORS?                               |
+-----------------------------------------------------+
|                                                     |
|  Is your frontend on a different                    |
|  origin than your backend?                          |
|       |                                             |
|       +--> No --> You do not need CORS              |
|       |                                             |
|       +--> Yes --> Continue                         |
|       |                                             |
|       v                                             |
|  Are you using Spring Security?                     |
|       |                                             |
|       +--> No --> Use WebMvcConfigurer               |
|       |          or @CrossOrigin                     |
|       |                                             |
|       +--> Yes --> Configure CORS in                 |
|                    SecurityFilterChain                |
|                    (CorsConfigurationSource)          |
|       |                                             |
|       v                                             |
|  Do you need credentials (cookies,                  |
|  Authorization headers)?                            |
|       |                                             |
|       +--> No --> allowedOrigins("*") is fine         |
|       |                                             |
|       +--> Yes --> List specific origins              |
|                    and set allowCredentials(true)     |
|                                                     |
+-----------------------------------------------------+
```

Common Mistakes

1. Configuring CORS only in WebMvcConfigurer when using Spring Security. Spring Security processes requests before your controllers. You must configure CORS in the SecurityFilterChain or your preflight requests will get 401/403 responses.
2. Using `allowedOrigins("*")` with `allowCredentials(true)`. This is not allowed by the CORS specification. Use `allowedOriginPatterns("*")` or list specific origins.
3. Forgetting to include OPTIONS in allowedMethods. Preflight requests use the OPTIONS method. If you do not allow it, preflights will fail.
4. Adding the trailing slash to origins. `http://localhost:3000/` (with slash) and `http://localhost:3000` (without slash) are different. Use the version without the trailing slash.
5. Not testing with the browser. Testing with curl or Postman does not test CORS because those tools do not enforce the Same-Origin Policy. Only browsers enforce CORS. Always test with an actual browser.
6. Setting allowedOrigins to "*" in production. This allows any website to call your API. Always list specific, trusted origins in production.

Best Practices

1. Always configure CORS in SecurityFilterChain when using Spring Security. This ensures preflight requests are handled before authentication.
2. Use externalized configuration for origins. Put allowed origins in application properties so you can change them per environment without code changes.
3. List specific origins instead of using wildcards. In production, only allow your actual frontend domains.
4. Set a reasonable maxAge. A value of 3600 (1 hour) reduces unnecessary preflight requests. Do not set it too high because you cannot revoke cached preflights.
5. Include only the HTTP methods you actually use. Do not allow DELETE if your API only uses GET and POST.
6. Test CORS from the browser, not from curl. Curl does not enforce CORS. Use a real browser or a simple HTML test page.

Quick Summary

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls which origins can access your API. Browsers enforce the Same-Origin Policy, which blocks requests between different origins (different protocol, host, or port). For complex requests, the browser sends a preflight OPTIONS request to check if the cross-origin request is allowed. In Spring Boot, you can configure CORS at the controller level with @CrossOrigin or globally with WebMvcConfigurer. When using Spring Security, you must configure CORS through CorsConfigurationSource in the SecurityFilterChain to ensure preflight requests are handled before authentication. Never use wildcard origins in production, and always externalize your CORS configuration for different environments.

Key Points

- Same-Origin Policy blocks requests between different origins (protocol + host + port).
- CORS lets the server specify which origins are allowed to make cross-origin requests.
- Preflight OPTIONS requests are sent for complex requests (POST with JSON, custom headers).
- @CrossOrigin provides controller-level or method-level CORS configuration.
- WebMvcConfigurer provides global CORS configuration.
- When using Spring Security, configure CORS in SecurityFilterChain with CorsConfigurationSource.
- `allowedOrigins("*")` cannot be used with `allowCredentials(true)`.
- Always test CORS from a browser, not from curl or Postman.
- Externalize allowed origins in application properties for different environments.

Practice Questions

1. What three parts make up an origin, and when do two URLs have the same origin?
2. What is a preflight request, and when does the browser send one?
3. Why is it not enough to configure CORS only in WebMvcConfigurer when using Spring Security?
4. Why can you not use `allowedOrigins("*")` together with `allowCredentials(true)`?
5. A developer tests their API with Postman and everything works. When they try the same request from their React app, they get a CORS error. Why?

Exercises

Exercise 1: Configure CORS for Multiple Frontends

Create a Spring Boot application with two sets of CORS rules:

- `/api/public/**` allows requests from any origin but only GET requests.
- `/api/private/**` allows requests only from `http://localhost:3000` and `http://localhost:5173`, all HTTP methods, and credentials.

Test both configurations using curl to simulate preflight requests.

Exercise 2: Environment-Specific CORS

Set up a Spring Boot project with two profiles: `dev` and `prod`. In the dev profile, allow `http://localhost:3000`. In the prod profile, allow `https://myapp.com`. Verify that switching profiles changes the CORS behavior.

Exercise 3: Debug a CORS Problem

Create a Spring Boot API with Spring Security enabled. Intentionally configure CORS only in WebMvcConfigurer (not in SecurityFilterChain). Observe the CORS error from a browser. Then fix it by adding CORS to the security configuration.

What Is Next?

Your API is now accessible from frontend applications running on different origins. But users often need to do more than send JSON data. They need to upload files like profile pictures, documents, and spreadsheets. In Chapter 21, you will learn how to handle file uploads and downloads in Spring Boot, including validating file types, generating unique filenames, and serving files back to users.

Chapter 21: File Upload and Download

What You Will Learn

- How file uploads work in HTTP (multipart/form-data).
- How to use MultipartFile to receive uploaded files.
- How to configure maximum file size limits.
- How to save uploaded files to disk with unique filenames.
- How to validate file types and reject dangerous uploads.
- How to serve files back to users for download.
- How to set proper download headers (Content-Disposition).
- How to handle multiple file uploads in a single request.

Why This Chapter Matters

Think about every web application you use daily. Profile pictures on social media. Resumes on job boards. Invoices in accounting software. Attachments in email. File upload and download is one of the most common features in web applications.

Getting file handling right is important for two reasons. First, files can be large. A poorly configured upload system can crash your server or fill your disk. Second, files can be dangerous. An attacker could upload a malicious script disguised as an image. You need to validate, limit, and handle files carefully.

In this chapter, you will build a complete file upload and download system that is safe, reliable, and production-ready.

21.1 How File Uploads Work in HTTP

When you submit a regular HTML form, the data is sent as `application/x-www-form-urlencoded`. That works for text fields. But files are binary data (images, PDFs, spreadsheets), and they need a different encoding.

The multipart/form-data encoding splits the request body into multiple parts. Each part can be a text field or a file:

```
+-----------------------------------------------------+
|  HTTP REQUEST (multipart/form-data)                 |
+-----------------------------------------------------+
|                                                     |
|  POST /api/files/upload HTTP/1.1                    |
|  Content-Type: multipart/form-data;                 |
|    boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW   |
|                                                     |
|  ------WebKitFormBoundary7MA4YWxkTrZu0gW             |
|  Content-Disposition: form-data; name="description" |
|                                                     |
|  My vacation photo                                  |
|  ------WebKitFormBoundary7MA4YWxkTrZu0gW             |
|  Content-Disposition: form-data; name="file";       |
|    filename="beach.jpg"                             |
|  Content-Type: image/jpeg                           |
|                                                     |
|  [Binary data of the image file]                    |
|  ------WebKitFormBoundary7MA4YWxkTrZu0gW--           |
|                                                     |
+-----------------------------------------------------+
```

The boundary string separates each part. Spring Boot handles all of this parsing for you. You just need to use `MultipartFile`.

21.2 Configuring File Upload Limits

Before writing any code, configure the maximum file size in your application properties:

```properties
# src/main/resources/application.properties

# Maximum size of a single uploaded file (default is 1MB)
spring.servlet.multipart.max-file-size=10MB
```

```
# Maximum size of the entire request (all files combined)
spring.servlet.multipart.max-request-size=50MB

# Directory for storing uploaded files
file.upload-dir=uploads
```

Line-by-line explanation:

- `max-file-size=10MB` -- No single file can be larger than 10 megabytes. If someone tries to upload a 20MB file, Spring returns a 413 (Payload Too Large) error.
- `max-request-size=50MB` -- The total size of the entire request (if uploading multiple files) cannot exceed 50MB.
- `file.upload-dir=uploads` -- A custom property that tells our code where to save files. This is not a Spring property; we define it ourselves.

Size Unit Reference

Value	Size
`1KB`	1 Kilobyte
`1MB`	1 Megabyte
`10MB`	10 Megabytes
`1GB`	1 Gigabyte

21.3 Setting Up the File Storage Service

Let us create a service that handles saving and loading files:

```java
// src/main/java/com/example/fileupload/service/FileStorageService.java
package com.example.fileupload.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import jakarta.annotation.PostConstruct;
```

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.UUID;

@Service
public class FileStorageService {

    @Value("${file.upload-dir}")
    private String uploadDir;

    private Path uploadPath;

    @PostConstruct
    public void init() {
        uploadPath = Paths.get(uploadDir).toAbsolutePath().normalize();
        try {
            Files.createDirectories(uploadPath);
        } catch (IOException e) {
            throw new RuntimeException(
                "Could not create upload directory: " + uploadPath, e);
        }
    }

    public String storeFile(MultipartFile file) {
        // Get original filename
        String originalFilename = file.getOriginalFilename();

        // Generate unique filename to prevent overwriting
        String uniqueFilename = UUID.randomUUID().toString()
            + "_" + originalFilename;

        try {
            // Check for invalid path characters
            if (uniqueFilename.contains("..")) {
                throw new RuntimeException(
                    "Filename contains invalid path sequence: "
                    + uniqueFilename);
            }

            // Copy file to upload directory
            Path targetLocation = uploadPath.resolve(uniqueFilename);
            Files.copy(file.getInputStream(), targetLocation,
                StandardCopyOption.REPLACE_EXISTING);

            return uniqueFilename;

        } catch (IOException e) {
            throw new RuntimeException(
                "Could not store file " + uniqueFilename, e);
        }
    }
```

```
    public Path loadFile(String filename) {
        return uploadPath.resolve(filename).normalize();
    }
}
```

Line-by-line explanation:

- `@Value("${file.upload-dir}")` -- Reads the upload directory from application.properties.
- `@PostConstruct` -- This method runs after the bean is created. It creates the upload directory if it does not exist.
- `Files.createDirectories(uploadPath)` -- Creates the directory and any parent directories. Does nothing if the directory already exists.
- `UUID.randomUUID().toString()` -- Generates a unique identifier like "a1b2c3d4-e5f6-7890-abcd-ef1234567890". This prevents filename collisions when two users upload files with the same name.
- `uniqueFilename.contains("..")` -- This is a security check. The ".." sequence could be used for path traversal attacks (escaping the upload directory).
- `Files.copy(...)` -- Copies the uploaded file's input stream to the target location on disk.
- `StandardCopyOption.REPLACE_EXISTING` -- If a file with the same name exists (unlikely with UUID), replace it.

Here is the directory structure after a few uploads:

```
project/
  +-- uploads/
  |     +-- a1b2c3d4-..._beach.jpg
  |     +-- e5f6a7b8-..._resume.pdf
  |     +-- c9d0e1f2-..._report.xlsx
  +-- src/
  +-- pom.xml
```

21.4 The File Upload Controller

```java
// src/main/java/com/example/fileupload/controller/FileController.java
package com.example.fileupload.controller;

import com.example.fileupload.service.FileStorageService;
import org.springframework.core.io.Resource;
import org.springframework.core.io.UrlResource;
```

```
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.net.MalformedURLException;
import java.nio.file.Path;
import java.util.Map;

@RestController
@RequestMapping("/api/files")
public class FileController {

    private final FileStorageService fileStorageService;

    public FileController(FileStorageService fileStorageService) {
        this.fileStorageService = fileStorageService;
    }

    @PostMapping("/upload")
    public ResponseEntity<Map<String, String>> uploadFile(
            @RequestParam("file") MultipartFile file) {

        // Check if file is empty
        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body(
                Map.of("error", "Please select a file to upload"));
        }

        String filename = fileStorageService.storeFile(file);

        return ResponseEntity.ok(Map.of(
            "message", "File uploaded successfully",
            "filename", filename,
            "size", file.getSize() + " bytes",
            "contentType", file.getContentType()
        ));
    }
}
```

Line-by-line explanation:

- `@RequestParam("file") MultipartFile file` -- Spring automatically parses the multipart request and gives you a `MultipartFile` object. The parameter name "file" must match the form field name.
- `file.isEmpty()` -- Returns true if no file was selected. Always check this.

- `file.getSize()` -- Returns the file size in bytes.
- `file.getContentType()` -- Returns the MIME type (like "image/jpeg" or "application/pdf").

What MultipartFile Offers

Method	Returns	Example
`getOriginalFilename()`	Original filename	`"beach.jpg"`
`getContentType()`	MIME type	`"image/jpeg"`
`getSize()`	Size in bytes	`1048576`
`getBytes()`	File content as byte array	`[binary data]`
`getInputStream()`	Stream for reading	`InputStream`
`isEmpty()`	True if no file	`false`
`transferTo(File dest)`	Save to a File object	void

21.5 Testing File Upload with curl

```bash
# Upload a single file
curl -X POST http://localhost:8080/api/files/upload \
  -F "file=@/path/to/photo.jpg"
```

Output:

```json
{
    "message": "File uploaded successfully",
    "filename": "a1b2c3d4-e5f6-7890-abcd-ef1234567890_photo.jpg",
    "size": "245678 bytes",
    "contentType": "image/jpeg"
}
```

The `-F` flag tells curl to use multipart/form-data. The `@` symbol means "read the content from this file."

21.6 File Type Validation

Never trust file uploads blindly. An attacker could upload a `.exe` file renamed to `.jpg`. You need to validate both the file extension and the content type:

```java
// src/main/java/com/example/fileupload/service/FileValidationService.java
package com.example.fileupload.service;

import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.util.Set;

@Service
public class FileValidationService {

    private static final Set<String> ALLOWED_CONTENT_TYPES = Set.of(
        "image/jpeg",
        "image/png",
        "image/gif",
        "application/pdf",
        "text/plain",
        "application/vnd.openxmlformats-officedocument" +
            ".spreadsheetml.sheet"  // .xlsx
    );

    private static final Set<String> ALLOWED_EXTENSIONS = Set.of(
        "jpg", "jpeg", "png", "gif", "pdf", "txt", "xlsx"
    );

    private static final long MAX_FILE_SIZE = 10 * 1024 * 1024; // 10MB

    public void validate(MultipartFile file) {
        // Check file size
        if (file.getSize() > MAX_FILE_SIZE) {
            throw new RuntimeException(
                "File size exceeds maximum limit of 10MB. " +
                "Actual size: " + file.getSize() + " bytes");
        }

        // Check content type
        String contentType = file.getContentType();
        if (contentType == null
                || !ALLOWED_CONTENT_TYPES.contains(contentType)) {
            throw new RuntimeException(
                "File type not allowed: " + contentType +
                ". Allowed types: " + ALLOWED_CONTENT_TYPES);
        }

        // Check file extension
        String filename = file.getOriginalFilename();
```

```
        if (filename != null) {
            String extension = getFileExtension(filename).toLowerCase();
            if (!ALLOWED_EXTENSIONS.contains(extension)) {
                throw new RuntimeException(
                    "File extension not allowed: ." + extension +
                    ". Allowed extensions: " + ALLOWED_EXTENSIONS);
            }
        }
    }

    private String getFileExtension(String filename) {
        int dotIndex = filename.lastIndexOf('.');
        if (dotIndex < 0) {
            return "";
        }
        return filename.substring(dotIndex + 1);
    }
}
```

Line-by-line explanation:

- `ALLOWED_CONTENT_TYPES` -- A whitelist of accepted MIME types. Anything not on this list is rejected.
- `ALLOWED_EXTENSIONS` -- A whitelist of accepted file extensions. We check both because content type alone can be spoofed.
- `getFileExtension()` -- Extracts the extension from the filename. "photo.jpg" returns "jpg".
- We validate three things: size, content type, and extension. This defense-in-depth approach makes it harder for attackers to bypass validation.

Now update the controller to use validation:

```java
@PostMapping("/upload")
public ResponseEntity<Map<String, String>> uploadFile(
        @RequestParam("file") MultipartFile file) {

    if (file.isEmpty()) {
        return ResponseEntity.badRequest().body(
            Map.of("error", "Please select a file to upload"));
    }

    // Validate the file before storing
    fileValidationService.validate(file);

    String filename = fileStorageService.storeFile(file);

    return ResponseEntity.ok(Map.of(
        "message", "File uploaded successfully",
        "filename", filename,
        "size", file.getSize() + " bytes",
```

```
        "contentType", file.getContentType()
    ));
}
```

```
+-----------------------------------------------------+
|           FILE UPLOAD VALIDATION FLOW               |
+-----------------------------------------------------+
|                                                     |
|  File received                                      |
|       |                                             |
|       v                                             |
|  [Is file empty?]                                   |
|       |                                             |
|       +--> Yes --> 400 Bad Request                  |
|       |                                             |
|       +--> No --> Continue                          |
|       |                                             |
|       v                                             |
|  [Size <= 10MB?]                                    |
|       |                                             |
|       +--> No --> "File too large"                  |
|       |                                             |
|       +--> Yes --> Continue                         |
|       |                                             |
|       v                                             |
|  [Content type allowed?]                            |
|       |                                             |
|       +--> No --> "File type not allowed"           |
|       |                                             |
|       +--> Yes --> Continue                         |
|       |                                             |
|       v                                             |
|  [Extension allowed?]                               |
|       |                                             |
|       +--> No --> "Extension not allowed"           |
|       |                                             |
|       +--> Yes --> Save file to disk                |
|                                                     |
+-----------------------------------------------------+
```

21.7 File Download

Now let us add a download endpoint. The key is setting the right HTTP headers so the browser knows how to handle the file:

```java
// Add to FileController.java

@GetMapping("/download/{filename:.+}")
public ResponseEntity<Resource> downloadFile(
        @PathVariable String filename) {

    try {
        Path filePath = fileStorageService.loadFile(filename);
        Resource resource = new UrlResource(filePath.toUri());

        if (!resource.exists()) {
            return ResponseEntity.notFound().build();
        }

        // Determine content type
        String contentType = Files.probeContentType(filePath);
        if (contentType == null) {
            contentType = "application/octet-stream";
        }

        return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(contentType))
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" +
                resource.getFilename() + "\"")
            .body(resource);

    } catch (MalformedURLException e) {
        return ResponseEntity.badRequest().build();
    } catch (IOException e) {
        return ResponseEntity.internalServerError().build();
    }
}
```

Line-by-line explanation:

- `@GetMapping("/download/{filename:.+}")` -- The `:.+` regex pattern tells Spring not to truncate the filename at the dot. Without it, "photo.jpg" would be interpreted as path variable "photo" with format "jpg".
- `new UrlResource(filePath.toUri())` -- Wraps the file path as a Spring Resource object that can be returned as a response body.
- `Files.probeContentType(filePath)` -- Determines the MIME type from the file extension. Returns "image/jpeg" for .jpg files, "application/pdf" for .pdf files, etc.
- `"application/octet-stream"` -- A fallback MIME type that means "unknown binary data." The browser will download it rather than trying to display it.
- `Content-Disposition: attachment` -- This header tells the browser to download the file instead of displaying it in the browser window.

Inline vs Attachment

```java
// Force download (save dialog appears)
.header(HttpHeaders.CONTENT_DISPOSITION,
    "attachment; filename=\"photo.jpg\"")

// Display in browser (for images and PDFs)
.header(HttpHeaders.CONTENT_DISPOSITION,
    "inline; filename=\"photo.jpg\"")
```

Use `attachment` when you want the user to save the file. Use `inline` when you want the browser to display it (for images, PDFs, and text files).

Testing Download

```bash
# Download a file
curl -O http://localhost:8080/api/files/download/a1b2c3d4_photo.jpg
```

The `-O` flag tells curl to save the file with its original name.

21.8 Multiple File Upload

Users often need to upload multiple files at once. Spring Boot supports this easily:

```java
// Add to FileController.java

@PostMapping("/upload-multiple")
public ResponseEntity<Map<String, Object>> uploadMultipleFiles(
        @RequestParam("files") MultipartFile[] files) {

    if (files.length == 0) {
        return ResponseEntity.badRequest().body(
            Map.of("error", "Please select files to upload"));
    }

    List<Map<String, String>> uploadedFiles = new ArrayList<>();

    for (MultipartFile file : files) {
        if (!file.isEmpty()) {
            fileValidationService.validate(file);
            String filename = fileStorageService.storeFile(file);

            uploadedFiles.add(Map.of(
```

```
                "originalName", file.getOriginalFilename(),
                "storedName", filename,
                "size", file.getSize() + " bytes",
                "contentType", file.getContentType()
            ));
        }
    }

    return ResponseEntity.ok(Map.of(
        "message", uploadedFiles.size() + " files uploaded successfully",
        "files", uploadedFiles
    ));
}
```

Line-by-line explanation:

- `MultipartFile[] files` -- Instead of a single `MultipartFile`, we accept an array. Spring parses all files from the request.
- `for (MultipartFile file : files)` -- Loop through each file and process it individually.
- Each file is validated and stored separately, and we collect the results.

Testing Multiple Upload

```bash
# Upload multiple files
curl -X POST http://localhost:8080/api/files/upload-multiple \
  -F "files=@photo1.jpg" \
  -F "files=@photo2.jpg" \
  -F "files=@document.pdf"
```

Output:

```json
{
    "message": "3 files uploaded successfully",
    "files": [
        {
            "originalName": "photo1.jpg",
            "storedName": "a1b2c3d4-..._photo1.jpg",
            "size": "245678 bytes",
            "contentType": "image/jpeg"
        },
        {
            "originalName": "photo2.jpg",
            "storedName": "e5f6a7b8-..._photo2.jpg",
            "size": "189432 bytes",
            "contentType": "image/jpeg"
        },
```

```
        {
            "originalName": "document.pdf",
            "storedName": "c9d0e1f2-..._document.pdf",
            "size": "1024567 bytes",
            "contentType": "application/pdf"
        }
    ]
}
```

21.9 Uploading a File with Additional Data

Sometimes you need to upload a file along with additional form data, like a description or category:

```java
@PostMapping("/upload-with-data")
public ResponseEntity<Map<String, String>> uploadWithData(
        @RequestParam("file") MultipartFile file,
        @RequestParam("description") String description,
        @RequestParam(value = "category",
            defaultValue = "general") String category) {

    if (file.isEmpty()) {
        return ResponseEntity.badRequest().body(
            Map.of("error", "Please select a file"));
    }

    fileValidationService.validate(file);
    String filename = fileStorageService.storeFile(file);

    // In a real app, save metadata to database
    return ResponseEntity.ok(Map.of(
        "message", "File uploaded successfully",
        "filename", filename,
        "description", description,
        "category", category
    ));
}
```

Testing with Additional Data

```bash
curl -X POST http://localhost:8080/api/files/upload-with-data \
  -F "file=@photo.jpg" \
```

```
  -F "description=My vacation photo" \
  -F "category=travel"
```

Output:

```json
{
    "message": "File uploaded successfully",
    "filename": "a1b2c3d4-..._photo.jpg",
    "description": "My vacation photo",
    "category": "travel"
}
```

21.10 Listing Uploaded Files

Let us add an endpoint to list all uploaded files:

```java
// Add to FileStorageService.java

public List<String> listAllFiles() {
    try {
        return Files.list(uploadPath)
            .map(path -> path.getFileName().toString())
            .toList();
    } catch (IOException e) {
        throw new RuntimeException(
            "Could not list files in " + uploadPath, e);
    }
}
```

```java
// Add to FileController.java

@GetMapping("/list")
public ResponseEntity<Map<String, Object>> listFiles() {
    List<String> files = fileStorageService.listAllFiles();

    return ResponseEntity.ok(Map.of(
        "totalFiles", files.size(),
        "files", files
    ));
}
```

```bash
curl http://localhost:8080/api/files/list
```

Output:

```json
{
    "totalFiles": 3,
    "files": [
        "a1b2c3d4-..._photo1.jpg",
        "e5f6a7b8-..._photo2.jpg",
        "c9d0e1f2-..._document.pdf"
    ]
}
```

21.11 Exception Handling for File Operations

Add a global exception handler for file-related errors:

```java
// src/main/java/com/example/fileupload/exception/FileExceptionHandler.java
package com.example.fileupload.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.multipart.MaxUploadSizeExceededException;

import java.util.Map;

@RestControllerAdvice
public class FileExceptionHandler {

    @ExceptionHandler(MaxUploadSizeExceededException.class)
    public ResponseEntity<Map<String, String>> handleMaxSizeException(
            MaxUploadSizeExceededException e) {

        return ResponseEntity
            .status(HttpStatus.PAYLOAD_TOO_LARGE)
            .body(Map.of(
                "error", "File too large",
                "message", "Maximum upload size exceeded. " +
                    "The limit is 10MB per file."
            ));
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, String>> handleRuntimeException(
            RuntimeException e) {
```

```
        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(Map.of(
                "error", "File operation failed",
                "message", e.getMessage()
            ));
    }
}
```

Line-by-line explanation:

- `MaxUploadSizeExceededException` -- Spring throws this when a file exceeds the configured `max-file-size` or `max-request-size`.
- `HttpStatus.PAYLOAD_TOO_LARGE` -- Returns HTTP status 413, which is the standard code for "payload too large."

Now when a user tries to upload a file that is too large:

```bash
curl -X POST http://localhost:8080/api/files/upload \
  -F "file=@large-video.mp4"
```

Output:

```json
{
    "error": "File too large",
    "message": "Maximum upload size exceeded. The limit is 10MB per file."
}
```

21.12 Complete File Controller

Here is the complete controller with all endpoints together:

```java
// src/main/java/com/example/fileupload/controller/FileController.java
package com.example.fileupload.controller;

import com.example.fileupload.service.FileStorageService;
import com.example.fileupload.service.FileValidationService;
import org.springframework.core.io.Resource;
import org.springframework.core.io.UrlResource;
import org.springframework.http.HttpHeaders;
```

```
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.net.MalformedURLException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/files")
public class FileController {

    private final FileStorageService fileStorageService;
    private final FileValidationService fileValidationService;

    public FileController(FileStorageService fileStorageService,
                          FileValidationService fileValidationService) {
        this.fileStorageService = fileStorageService;
        this.fileValidationService = fileValidationService;
    }

    @PostMapping("/upload")
    public ResponseEntity<Map<String, String>> uploadFile(
            @RequestParam("file") MultipartFile file) {

        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body(
                Map.of("error", "Please select a file to upload"));
        }

        fileValidationService.validate(file);
        String filename = fileStorageService.storeFile(file);

        return ResponseEntity.ok(Map.of(
            "message", "File uploaded successfully",
            "filename", filename,
            "size", file.getSize() + " bytes",
            "contentType", file.getContentType()
        ));
    }

    @PostMapping("/upload-multiple")
    public ResponseEntity<Map<String, Object>> uploadMultipleFiles(
            @RequestParam("files") MultipartFile[] files) {

        List<Map<String, String>> uploadedFiles = new ArrayList<>();

        for (MultipartFile file : files) {
            if (!file.isEmpty()) {
                fileValidationService.validate(file);
```

```
                String filename = fileStorageService.storeFile(file);
                uploadedFiles.add(Map.of(
                    "originalName", file.getOriginalFilename(),
                    "storedName", filename
                ));
            }
        }

        return ResponseEntity.ok(Map.of(
            "message", uploadedFiles.size()
                + " files uploaded successfully",
            "files", uploadedFiles
        ));
    }

    @GetMapping("/download/{filename:.+}")
    public ResponseEntity<Resource> downloadFile(
            @PathVariable String filename) {

        try {
            Path filePath = fileStorageService.loadFile(filename);
            Resource resource = new UrlResource(filePath.toUri());

            if (!resource.exists()) {
                return ResponseEntity.notFound().build();
            }

            String contentType = Files.probeContentType(filePath);
            if (contentType == null) {
                contentType = "application/octet-stream";
            }

            return ResponseEntity.ok()
                .contentType(MediaType.parseMediaType(contentType))
                .header(HttpHeaders.CONTENT_DISPOSITION,
                    "attachment; filename=\""
                    + resource.getFilename() + "\"")
                .body(resource);

        } catch (MalformedURLException e) {
            return ResponseEntity.badRequest().build();
        } catch (IOException e) {
            return ResponseEntity.internalServerError().build();
        }
    }

    @GetMapping("/list")
    public ResponseEntity<Map<String, Object>> listFiles() {
        List<String> files = fileStorageService.listAllFiles();
        return ResponseEntity.ok(Map.of(
            "totalFiles", files.size(),
            "files", files
        ));
    }
}
```

Common Mistakes

1. Not setting file size limits. The default limit is only 1MB. If you expect larger files, increase `max-file-size` and `max-request-size` in application.properties.
2. Using the original filename directly. Two users could upload files with the same name, overwriting each other. Always generate unique filenames using UUID or timestamps.
3. Not validating file types. Never trust the file extension alone. Check both the content type and the extension. Even better, read the file's magic bytes to verify the actual content.
4. Storing files inside the application JAR. Files should be stored outside the application directory. When you redeploy, the JAR is replaced and all files inside it are lost.
5. Not handling the `..` path traversal. An attacker could send a filename like `../../etc/passwd` to read or write files outside the upload directory. Always sanitize filenames and check for `..` sequences.
6. Forgetting to create the upload directory. If the directory does not exist, file writes will fail. Use `@PostConstruct` to create it when the application starts.

Best Practices

1. Always validate file type, size, and extension. Use a whitelist approach: only allow specific types rather than blocking specific types.
2. Generate unique filenames with UUID. This prevents filename collisions and makes filenames unpredictable (attackers cannot guess them).
3. Store files outside the application directory. Use a configurable path from application.properties. In production, consider using cloud storage (like AWS S3).
4. Set appropriate size limits. Think about what your users actually need. Profile pictures do not need to be 100MB.
5. Return file metadata after upload. Tell the user the stored filename, size, and content type so they can reference it later.
6. Handle errors gracefully. Return clear error messages for oversized files, invalid types, and storage failures.

Quick Summary

File upload in Spring Boot uses MultipartFile to receive files sent as multipart/form-data. You configure size limits with spring.servlet.multipart.max-file-size and max-request-size in application.properties. Files should be saved to disk with unique filenames (using UUID) to prevent collisions and path traversal attacks. Always validate files by checking the content type, file extension, and size against a whitelist of allowed values. For downloads, use Spring's Resource interface with Content-Disposition headers to control whether the browser downloads or displays the file. Multiple file upload works by accepting a MultipartFile array instead of a single MultipartFile.

Key Points

- MultipartFile is Spring's abstraction for uploaded files.
- Configure max-file-size and max-request-size in application.properties.
- Always generate unique filenames with UUID to prevent overwriting.
- Validate content type and file extension using a whitelist approach.
- Check for ".." in filenames to prevent path traversal attacks.
- Use Content-Disposition: attachment for downloads, inline for display.
- The `{filename:.+}` path variable pattern preserves the file extension.
- Use @PostConstruct to create the upload directory on startup.
- Handle MaxUploadSizeExceededException for files that exceed limits.

Practice Questions

1. What is the difference between `multipart/form-data` and `application/x-www-form-urlencoded`, and when do you use each?
2. Why should you generate unique filenames instead of using the original filename from the upload?
3. What is a path traversal attack, and how does checking for ".." in filenames prevent it?
4. What is the difference between `Content-Disposition: attachment` and `Content-Disposition: inline`?

5. Why should you validate both the content type and the file extension, rather than just one of them?

Exercises

Exercise 1: Image-Only Upload

Create a file upload endpoint that only accepts image files (JPEG, PNG, GIF). Return a 400 Bad Request with a clear error message if a non-image file is uploaded. Test with both valid images and invalid files like PDFs.

Exercise 2: File Metadata Database

Extend the file upload system to save file metadata (original name, stored name, upload date, file size, content type) to an H2 database using JPA. Create an endpoint to search uploaded files by content type or date range.

Exercise 3: Delete Endpoint

Add a DELETE endpoint at `/api/files/{filename}` that deletes both the file from disk and its metadata from the database. Handle the case where the file does not exist gracefully.

What Is Next?

You can now handle file uploads and downloads in your Spring Boot application. But how do you know what is happening inside your application when something goes wrong? How do you track down a bug in production? In Chapter 22, you will learn about logging. You will use SLF4J and Logback to record what your application does, configure log levels, write logs to files, and use MDC to trace requests across multiple components.

Chapter 22: Logging

What You Will Learn

- Why logging matters and when to use it.
- What SLF4J and Logback are and how they work together.
- How to create a logger with LoggerFactory.
- The five log levels: TRACE, DEBUG, INFO, WARN, ERROR.
- How to configure log levels per package.
- How to write logs to files and configure log rotation.
- What MDC (Mapped Diagnostic Context) is and how to use it.
- How to use @Slf4j from Lombok to simplify logging.

Why This Chapter Matters

Imagine you are a detective investigating a crime. You arrive at the scene, but there are no security cameras, no witnesses, and no fingerprints. How do you figure out what happened?

Now imagine the building had security cameras on every floor, recording everything. You can rewind, see exactly who entered, what they did, and when they left. Case solved.

Logging is the security camera for your application. When a user reports a bug, when a server crashes at 3 AM, when a payment fails silently, logs are your only way to understand what happened. Without logging, you are debugging blind. With good logging, you can trace every request, every decision, and every error your application makes.

22.1 What Is Logging?

Logging is the practice of recording events that happen in your application. Instead of using `System.out.println()` (which prints to the console and disappears), you use a logging framework that can:

- Write to files that persist after the application stops.
- Include timestamps, thread names, and class names automatically.
- Filter messages by severity (show errors but hide debug messages).
- Rotate log files so they do not fill up your disk.

System.out.println vs Logging

Feature	System.out.println	Logging Framework
Timestamp	No	Yes (automatic)
Log levels	No	Yes (TRACE to ERROR)
Output to file	No (console only)	Yes (file, console, or both)
Filtering	No	Yes (per package, per level)
Performance	Blocking	Configurable (async)
Thread info	No	Yes (automatic)
Production use	Never	Always

Here is a comparison:

```java
// Bad - Using System.out.println
System.out.println("User logged in: alice");
// Output: User logged in: alice

// Good - Using a logger
log.info("User logged in: {}", username);
// Output: 2024-01-15 10:30:45.123 INFO  [main]
//   c.e.security.AuthService : User logged in: alice
```

The logger gives you the date, time, log level, thread name, class name, and your message. All automatically.

22.2 SLF4J and Logback

Spring Boot uses two libraries for logging:

1. SLF4J (Simple Logging Facade for Java) -- An API that your code uses. Think of it as the steering wheel.
2. Logback -- The implementation that does the actual logging. Think of it as the engine.

```
+----------------------------------------+
|           LOGGING ARCHITECTURE          |
+----------------------------------------+
|                                         |
|  Your Code                              |
|     |                                   |
|     v                                   |
|  [SLF4J API]  <-- You program to this   |
|     |                                   |
|     v                                   |
|  [Logback]    <-- Does the actual work  |
|     |                                   |
|     +---> Console output                |
|     +---> Log file                      |
|     +---> Remote server (optional)      |
|                                         |
+----------------------------------------+
```

Why use a facade? Because you might want to switch from Logback to Log4j2 later. If your code uses SLF4J, you only change the underlying engine. Your code stays the same.

The good news: Spring Boot includes both SLF4J and Logback automatically. You do not need to add any dependencies.

22.3 Creating Your First Logger

Here is how to create a logger in a Spring Boot class:

```java
// src/main/java/com/example/logging/service/ProductService.java
package com.example.logging.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class ProductService {
```

```
    // Create a logger for this class
    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    public String getProduct(Long id) {
        log.info("Fetching product with id: {}", id);

        // Simulate business logic
        if (id <= 0) {
            log.warn("Invalid product id: {}", id);
            throw new IllegalArgumentException("Product id must be positive");
        }

        log.debug("Product {} found in database", id);
        return "Product " + id;
    }
}
```

Line-by-line explanation:

- `import org.slf4j.Logger` -- Import the SLF4J Logger interface (not Logback's logger).
- `import org.slf4j.LoggerFactory` -- The factory that creates loggers.
- `private static final Logger log = LoggerFactory.getLogger(ProductService.class)` -- Creates a logger named after this class. We make it `static final` because one logger per class is enough.
- `log.info("Fetching product with id: {}", id)` -- Logs an INFO message. The `{}` is a placeholder that gets replaced with the value of `id`.
- `log.warn("Invalid product id: {}", id)` -- Logs a WARNING message. Something unexpected happened, but the application can continue.
- `log.debug("Product {} found in database", id)` -- Logs a DEBUG message. This is detailed information useful during development but usually hidden in production.

Why Use {} Placeholders Instead of String Concatenation?

```java
// Bad - String is always concatenated, even if level is disabled
log.debug("Product: " + product.toString());

// Good - toString() only called if debug is enabled
log.debug("Product: {}", product);
```

With `{}` placeholders, the argument is only converted to a string if the log level is enabled. This is a performance optimization. If debug logging is turned off (which it usually is in production), the string concatenation never happens.

22.4 Understanding Log Levels

SLF4J has five log levels, from most detailed to most critical:

```
+-----------------------------------------------+
|          LOG LEVELS (least to most severe)    |
+-----------------------------------------------+
|                                               |
|  TRACE  <-- Most detailed, rarely used        |
|    |                                          |
|    v                                          |
|  DEBUG  <-- Development details               |
|    |                                          |
|    v                                          |
|  INFO   <-- Normal operations                 |
|    |                                          |
|    v                                          |
|  WARN   <-- Something unexpected              |
|    |                                          |
|    v                                          |
|  ERROR  <-- Something broke                   |
|                                               |
+-----------------------------------------------+
```

Level	When to Use	Example
TRACE	Very detailed diagnostic info	Method entry/exit, variable values
DEBUG	Development-time details	SQL queries, cache hits, algorithm steps
INFO	Normal business events	User login, order placed, server started
WARN	Unexpected but recoverable	Retry attempt, deprecated API used, slow query
ERROR	Something failed	Database connection lost, payment failed, exception

How Level Filtering Works

When you set a log level, you see that level and everything above it:

```
Set level to WARN:
  TRACE  --> Hidden
  DEBUG  --> Hidden
  INFO   --> Hidden
  WARN   --> Visible
  ERROR  --> Visible

Set level to DEBUG:
  TRACE  --> Hidden
  DEBUG  --> Visible
  INFO   --> Visible
  WARN   --> Visible
  ERROR  --> Visible

Set level to TRACE:
  TRACE  --> Visible  (everything visible)
  DEBUG  --> Visible
  INFO   --> Visible
  WARN   --> Visible
  ERROR  --> Visible
```

Using Each Level Correctly

```java
@Service
public class OrderService {

    private static final Logger log =
        LoggerFactory.getLogger(OrderService.class);

    public void placeOrder(Order order) {
        log.trace("Entering placeOrder method with order: {}", order);

        log.debug("Validating order items: {}", order.getItems());

        log.info("Order placed successfully. OrderId: {}, " +
            "Total: ${}", order.getId(), order.getTotal());

        if (order.getTotal() > 10000) {
            log.warn("Large order detected. OrderId: {}, " +
                "Total: ${}", order.getId(), order.getTotal());
        }

        try {
            paymentService.charge(order);
        } catch (PaymentException e) {
            log.error("Payment failed for OrderId: {}. Reason: {}",
                order.getId(), e.getMessage(), e);
            // Note: passing the exception as last argument
            // prints the full stack trace
        }
```

```
        log.trace("Exiting placeOrder method");
    }
}
```

Important: When you pass an exception to a log method, pass it as the last argument. SLF4J will print the full stack trace. You do not need a `{}` placeholder for the exception:

```java
// Correct - exception is the last argument (no placeholder needed)
log.error("Payment failed: {}", order.getId(), exception);

// Wrong - this will NOT print the stack trace
log.error("Payment failed: {} {}", order.getId(), exception);
```

22.5 Configuring Log Levels in application.properties

Spring Boot's default log level is INFO. You can change it:

```properties
# src/main/resources/application.properties

# Change the root log level (affects everything)
logging.level.root=WARN

# Set level for your application packages
logging.level.com.example=DEBUG

# Set level for a specific class
logging.level.com.example.logging.service.ProductService=TRACE

# Set level for Spring framework classes
logging.level.org.springframework.web=DEBUG

# Set level for Hibernate SQL queries
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

Line-by-line explanation:

- `logging.level.root=WARN` -- The root logger affects all packages. Setting it to WARN hides INFO and DEBUG messages from all libraries.

- `logging.level.com.example=DEBUG` -- Shows DEBUG (and above) messages from your application code.
- `logging.level.org.springframework.web=DEBUG` -- Shows DEBUG messages from Spring's web layer. Useful for seeing how requests are processed.
- `logging.level.org.hibernate.SQL=DEBUG` -- Shows the actual SQL queries that Hibernate generates. Very useful for debugging database issues.

Per-Package Logging Strategy

```
+--------------------------------------------+
|     RECOMMENDED LOGGING STRATEGY           |
+--------------------------------------------+
|                                            |
|  Development:                              |
|    root = INFO                             |
|    com.example = DEBUG                     |
|    org.hibernate.SQL = DEBUG               |
|                                            |
|  Production:                               |
|    root = WARN                             |
|    com.example = INFO                      |
|    org.hibernate.SQL = WARN                |
|                                            |
|  Troubleshooting (temporary):              |
|    root = WARN                             |
|    com.example.problem.area = DEBUG        |
|                                            |
+--------------------------------------------+
```

22.6 Customizing the Log Format

Spring Boot uses a default log format. You can customize it:

```properties
# Default format (this is what Spring Boot uses)
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss.SSS} %5level [%15.15thread] %-40.40logge...

# Simpler format
logging.pattern.console=%d{HH:mm:ss} %-5level [%thread] %logger{20} - %msg%n
```

Pattern	Meaning	Example
`%d{...}`	Date/time	`2024-01-15 10:30:45.123`

`%level`	Log level	`INFO`
`%5level`	Level, right-padded to 5 chars	`INFO`
`%thread`	Thread name	`http-nio-8080-exec-1`
`%logger`	Logger name (class)	`c.e.l.s.ProductService`
`%msg`	Your message	`Fetching product with id: 42`
`%n`	New line	(line break)
`%-40.40`	Left-align, min/max 40 chars	Pads or truncates

Example output with the default format:

```
2024-01-15 10:30:45.123  INFO [http-nio-8080-exec-1] c.e.l.s.ProductService  : Fetching...
2024-01-15 10:30:45.124 DEBUG [http-nio-8080-exec-1] c.e.l.s.ProductService  : Product ...
2024-01-15 10:30:45.200  WARN [http-nio-8080-exec-2] c.e.l.s.ProductService  : Invalid ...
2024-01-15 10:30:45.300 ERROR [http-nio-8080-exec-3] c.e.l.s.OrderService    : Payment ...
```

22.7 Writing Logs to Files

In production, console output disappears when the terminal closes. You need to write logs to files:

```properties
# src/main/resources/application.properties

# Write logs to a file
logging.file.name=logs/application.log

# Set the file log format (can be different from console)
logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} %5level [%thread] %logger{36} : %msg%n
```

Line-by-line explanation:

- `logging.file.name=logs/application.log` -- Writes logs to the file `logs/application.log` relative to the application directory. The directory is created automatically.
- `logging.pattern.file` -- You can use a different format for file output than console output.

After starting the application, you will see:

```
project/
  +-- logs/
  |     +-- application.log
  +-- src/
  +-- pom.xml
```

22.8 Log File Rotation

Log files can grow very large. Log rotation automatically archives old logs and starts new ones:

```properties
# src/main/resources/application.properties

# Write to file
logging.file.name=logs/application.log

# Maximum size of a single log file (default 10MB)
logging.logback.rollingpolicy.max-file-size=10MB

# Maximum number of archived log files to keep (default 7)
logging.logback.rollingpolicy.max-history=30

# Maximum total size of all log files (default unlimited)
logging.logback.rollingpolicy.total-size-cap=1GB

# Name pattern for archived files
logging.logback.rollingpolicy.file-name-pattern=logs/application.%d{yyyy-MM-dd}.%i.log.gz
```

Line-by-line explanation:

- `max-file-size=10MB` -- When the log file reaches 10MB, it is archived and a new one starts.
- `max-history=30` -- Keep the last 30 days of log files. Older files are deleted automatically.
- `total-size-cap=1GB` -- The total size of all log files cannot exceed 1GB. This prevents logs from filling your disk.
- `file-name-pattern` -- Archived files are named like `application.2024-01-15.0.log.gz`. The `.gz` extension means they are compressed to save space.

Here is what the logs directory looks like after a few days:

```
logs/
  +-- application.log               (current, active log)
```

```
  +-- application.2024-01-14.0.log.gz  (yesterday, compressed)
  +-- application.2024-01-13.0.log.gz  (two days ago)
  +-- application.2024-01-12.0.log.gz  (three days ago)
```

22.9 MDC (Mapped Diagnostic Context)

When your server handles hundreds of requests simultaneously, log lines from different requests get mixed together. How do you trace all log lines for a single request?

MDC (Mapped Diagnostic Context) lets you attach contextual data (like a request ID or user ID) to every log line in the current thread:

```java
// src/main/java/com/example/logging/filter/RequestIdFilter.java
package com.example.logging.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.UUID;

@Component
public class RequestIdFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
                         ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        try {
            // Generate a unique ID for this request
            String requestId = UUID.randomUUID().toString()
                .substring(0, 8);

            // Get username if available
            HttpServletRequest httpRequest = (HttpServletRequest) request;
            String user = httpRequest.getRemoteUser();
            if (user == null) {
```

```
                user = "anonymous";
            }

            // Put values into MDC
            MDC.put("requestId", requestId);
            MDC.put("user", user);

            // Continue processing the request
            chain.doFilter(request, response);

        } finally {
            // Always clear MDC after request is done
            MDC.clear();
        }
    }
}
```

Line-by-line explanation:

- `MDC.put("requestId", requestId)` -- Adds a key-value pair to the MDC for the current thread. Every log statement on this thread will have access to this value.
- `MDC.clear()` -- Removes all MDC values. This must be in a `finally` block to prevent memory leaks and data bleeding between requests.

Now update the log pattern to include MDC values:

```properties
logging.pattern.console=%d{HH:mm:ss.SSS} %-5level [%thread] [%X{requestId}] [%X{user}] ...
```

- `%X{requestId}` -- Prints the MDC value for "requestId".
- `%X{user}` -- Prints the MDC value for "user".

Now your logs look like this:

```
10:30:45.123 INFO  [exec-1] [a1b2c3d4] [alice] ProductService - Fetching product with i...
10:30:45.124 DEBUG [exec-1] [a1b2c3d4] [alice] ProductService - Product 42 found
10:30:45.130 INFO  [exec-2] [e5f6a7b8] [bob]   OrderService   - Placing order 100
10:30:45.131 INFO  [exec-1] [a1b2c3d4] [alice] OrderService   - Placing order 101
10:30:45.132 ERROR [exec-2] [e5f6a7b8] [bob]   OrderService   - Payment failed
```

Even though Alice's and Bob's requests are interleaved on different threads, you can filter by `requestId` to see all log lines for a single request:

```bash
# Find all log lines for request a1b2c3d4
grep "a1b2c3d4" logs/application.log
```

```
10:30:45.123 INFO  [exec-1] [a1b2c3d4] [alice] ProductService - Fetching product with i...
10:30:45.124 DEBUG [exec-1] [a1b2c3d4] [alice] ProductService - Product 42 found
10:30:45.131 INFO  [exec-1] [a1b2c3d4] [alice] OrderService   - Placing order 101
```

22.10 Logging in Controllers and Services

Here is a complete example showing logging across different layers:

```java
// src/main/java/com/example/logging/controller/ProductController.java
package com.example.logging.controller;

import com.example.logging.service.ProductService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private static final Logger log =
        LoggerFactory.getLogger(ProductController.class);

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Map<String, Object>> getProduct(
            @PathVariable Long id) {

        log.info("Received request to get product: {}", id);
        long startTime = System.currentTimeMillis();

        try {
            Map<String, Object> product = productService.findById(id);
            long duration = System.currentTimeMillis() - startTime;

            log.info("Product {} retrieved in {} ms", id, duration);
            return ResponseEntity.ok(product);
```

```
        } catch (Exception e) {
            long duration = System.currentTimeMillis() - startTime;
            log.error("Failed to get product {} after {} ms",
                id, duration, e);
            return ResponseEntity.notFound().build();
        }
    }
}
```

```java
// src/main/java/com/example/logging/service/ProductService.java
package com.example.logging.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.util.Map;

@Service
public class ProductService {

    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    public Map<String, Object> findById(Long id) {
        log.debug("Searching for product {} in database", id);

        if (id <= 0) {
            log.warn("Invalid product id received: {}", id);
            throw new IllegalArgumentException(
                "Product id must be positive");
        }

        // Simulate database lookup
        log.debug("Product {} found, preparing response", id);

        return Map.of(
            "id", id,
            "name", "Widget " + id,
            "price", 29.99
        );
    }
}
```

Output when calling GET /api/products/42:

```
10:30:45.100 INFO  [exec-1] [a1b2c3d4] ProductController - Received request to get prod...
10:30:45.101 DEBUG [exec-1] [a1b2c3d4] ProductService    - Searching for product 42 in ...
10:30:45.105 DEBUG [exec-1] [a1b2c3d4] ProductService    - Product 42 found, preparing ...
10:30:45.106 INFO  [exec-1] [a1b2c3d4] ProductController - Product 42 retrieved in 6 ms
```

22.11 Using @Slf4j from Lombok

If you use Project Lombok, you can avoid the boilerplate logger declaration. Add Lombok to your dependencies:

```xml
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

Then use the `@Slf4j` annotation:

```java
// Without Lombok
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class ProductService {
    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    // ... use log.info(), log.debug(), etc.
}

// With Lombok @Slf4j
import lombok.extern.slf4j.Slf4j;

@Slf4j    // Lombok generates the Logger field for you
@Service
public class ProductService {
    // log is already available! No need to declare it.

    public void doSomething() {
        log.info("This works without declaring the logger");
    }
}
```

Line-by-line explanation:

- `@Slf4j` -- Lombok generates `private static final Logger log = LoggerFactory.getLogger(ProductService.class);` at compile time. The variable is always named `log`.
- You save two import lines and one field declaration. In a large project with hundreds of classes, this adds up.

> Note: Make sure your IDE has the Lombok plugin installed. Without it, your IDE will show errors because it cannot see the generated `log` field.

22.12 What to Log and What Not to Log

What to Log

Category	Examples
Business events	User registered, order placed, payment processed
Security events	Login success, login failure, access denied
Performance data	Request duration, slow queries, cache misses
Errors and exceptions	Database errors, API failures, validation errors
Application lifecycle	Startup, shutdown, configuration loaded

What NOT to Log

Category	Why
Passwords	Even encoded passwords should not appear in logs
Credit card numbers	PCI compliance prohibits logging card data
Personal data	GDPR and privacy laws restrict logging personal info
Session tokens	Attackers could steal them from log files
Large objects	Logging an entire database result set wastes disk space

```java
// BAD - logging sensitive data
log.info("User login: username={}, password={}", username, password);

// GOOD - log the event without sensitive data
log.info("User login successful: username={}", username);
```

```
// BAD - logging too much data
log.info("Query result: {}", entireDatabaseResultSet);

// GOOD - log a summary
log.info("Query returned {} results", results.size());
```

Common Mistakes

1. Using System.out.println instead of a logger. Print statements have no timestamps, no levels, and no file output. They should never appear in production code.
2. Logging too much in production. Setting the root level to DEBUG in production generates enormous log files and can slow down your application. Use INFO or WARN for production.
3. Logging too little. If you only log errors, you have no context when an error occurs. Log important business events at INFO level so you can understand the flow.
4. Using string concatenation instead of {} placeholders. `log.debug("User: " + user)` evaluates the string even when DEBUG is disabled. Use `log.debug("User: {}", user)` instead.
5. Forgetting to clear MDC. If you put values into MDC but do not clear them in a `finally` block, values can leak to other requests using the same thread.
6. Logging sensitive information. Never log passwords, credit card numbers, social security numbers, or session tokens. This is a security violation.

Best Practices

1. Use SLF4J with {} placeholders. This is both more readable and more performant than string concatenation.
2. Log at the right level. Use INFO for business events, WARN for unexpected situations, ERROR for failures, and DEBUG for development details.
3. Include context in log messages. Instead of "Error occurred", write "Payment failed for orderId: 123, reason: insufficient funds". The more context, the faster you can diagnose problems.
4. Use MDC for request tracing. Assign a unique ID to each request so you can follow it through all service layers.

5. Configure log rotation. Set max-file-size, max-history, and total-size-cap to prevent logs from filling your disk.
6. Use different logging configurations per environment. DEBUG in development, INFO in staging, WARN in production. Use Spring profiles to manage this.

Quick Summary

Logging records events that happen in your application for debugging and monitoring. Spring Boot uses SLF4J as the logging API and Logback as the implementation. You create loggers with LoggerFactory.getLogger() or by adding the @Slf4j Lombok annotation to your class. There are five log levels: TRACE (most detailed), DEBUG, INFO, WARN, and ERROR (most critical). Setting a level shows that level and everything above it. You configure per-package log levels in application.properties using logging.level.package.name=LEVEL. Logs can be written to files with automatic rotation to prevent disk space issues. MDC (Mapped Diagnostic Context) lets you attach contextual data like request IDs to every log line on the current thread, making it easy to trace a single request through multiple service layers.

Key Points

- SLF4J is the logging API. Logback is the implementation. Spring Boot includes both.
- Create loggers with `LoggerFactory.getLogger(YourClass.class)`.
- Five levels: TRACE < DEBUG < INFO < WARN < ERROR.
- Setting a level shows that level and all higher levels.
- Use `{}` placeholders instead of string concatenation for performance.
- Pass exceptions as the last argument to log methods for stack traces.
- Configure per-package levels with `logging.level.package=LEVEL`.
- Write to files with `logging.file.name`.
- Use log rotation to prevent disk space issues.
- MDC attaches contextual data (request ID, user) to log lines.
- @Slf4j from Lombok eliminates boilerplate logger declarations.
- Never log passwords, credit card numbers, or session tokens.

Practice Questions

1. What is the difference between SLF4J and Logback, and why does Spring Boot use both?
2. If you set the log level to WARN, which log levels will be visible and which will be hidden?
3. Why should you use `log.debug("User: {}", user)` instead of `log.debug("User: " + user)`?
4. What is MDC, and how does it help when debugging a production issue?
5. You set `logging.level.com.example=DEBUG` but you are not seeing DEBUG messages from your `com.example.service.OrderService` class. What could be wrong?

Exercises

Exercise 1: Add Logging to a REST Controller

Create a REST controller with GET and POST endpoints. Add appropriate logging:

- Log incoming requests at INFO level (include the request details).
- Log successful responses at INFO level (include response time).
- Log validation errors at WARN level.
- Log exceptions at ERROR level (include the stack trace).

Exercise 2: Configure Per-Environment Logging

Create three profile-specific property files:

- `application-dev.properties`: Root level WARN, your package DEBUG, console output only.
- `application-prod.properties`: Root level WARN, your package INFO, file output with rotation (5MB files, 30 days, 500MB total).

Test both profiles and verify the output differences.

Exercise 3: Request Tracing with MDC

Create a filter that generates a unique request ID and adds it to MDC. Configure the log pattern to include the request ID. Send multiple concurrent requests and use the request ID to trace each request through the logs.

What Is Next?

You now know how to record what your application does. But what about making your application do things automatically? In Chapter 23, you will learn about scheduling. You will set up tasks that run at fixed intervals, after delays, or on specific schedules using cron expressions. Think of it as setting alarm clocks for your application.

Chapter 23: Scheduling

What You Will Learn

- What scheduling is and when to use it.
- How to enable scheduling with @EnableScheduling.
- How to run tasks at fixed intervals with @Scheduled(fixedRate).
- How to run tasks with delays between executions using @Scheduled(fixedDelay).
- How to use cron expressions for complex schedules.
- How cron expression fields work (second, minute, hour, day, month, weekday).
- How to set an initial delay before the first execution.
- How to handle errors in scheduled tasks.

Why This Chapter Matters

Think about your daily routine. Your alarm clock goes off at 7 AM every morning. Your coffee maker starts brewing at 6:50 AM. Your sprinkler system waters the lawn every Tuesday and Thursday at 6 PM. These are all scheduled tasks. You set them up once, and they run automatically.

Applications need the same thing. You might want to:

- Clean up expired sessions every hour.
- Send a daily email report at 8 AM.
- Check for new orders every 30 seconds.
- Back up the database every night at midnight.
- Remove temporary files every Sunday.

Without scheduling, someone would have to manually trigger these tasks. That is not practical. Spring Boot's scheduling support lets you automate tasks with just a few annotations.

23.1 Enabling Scheduling

Scheduling is disabled by default in Spring Boot. You enable it by adding `@EnableScheduling` to a configuration class or your main application class:

```java
// src/main/java/com/example/scheduling/SchedulingApplication.java
package com.example.scheduling;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling  // Enables scheduling support
public class SchedulingApplication {

    public static void main(String[] args) {
        SpringApplication.run(SchedulingApplication.class, args);
    }
}
```

Line-by-line explanation:

- `@EnableScheduling` -- This annotation tells Spring Boot to look for methods annotated with `@Scheduled` and run them according to their schedule. Without this, `@Scheduled` annotations are silently ignored.

That is it. One annotation enables the entire scheduling system.

23.2 Fixed Rate Scheduling

The simplest type of scheduling runs a task at a fixed interval. The task runs every N milliseconds, regardless of how long the previous execution took:

```java
// src/main/java/com/example/scheduling/task/FixedRateTask.java
package com.example.scheduling.task;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

@Component
public class FixedRateTask {

    private static final Logger log =
        LoggerFactory.getLogger(FixedRateTask.class);

    private static final DateTimeFormatter FORMATTER =
        DateTimeFormatter.ofPattern("HH:mm:ss");

    @Scheduled(fixedRate = 5000)  // Run every 5 seconds
    public void reportCurrentTime() {
        String time = LocalDateTime.now().format(FORMATTER);
        log.info("Fixed Rate Task - Current time: {}", time);
    }
}
```

Line-by-line explanation:

- `@Component` -- The class must be a Spring bean for `@Scheduled` to work.
- `@Scheduled(fixedRate = 5000)` -- Run this method every 5000 milliseconds (5 seconds). The value is in milliseconds.
- The method must have `void` return type and no parameters.

Output:

```
10:30:00.001 INFO  FixedRateTask - Fixed Rate Task - Current time: 10:30:00
10:30:05.001 INFO  FixedRateTask - Fixed Rate Task - Current time: 10:30:05
10:30:10.002 INFO  FixedRateTask - Fixed Rate Task - Current time: 10:30:10
10:30:15.001 INFO  FixedRateTask - Fixed Rate Task - Current time: 10:30:15
```

How fixedRate Works

```
+-----------------------------------------------------+
|  fixedRate = 5000 (5 seconds)                       |
+-----------------------------------------------------+
|                                                     |
|  Time: 0s    5s    10s   15s   20s                 |
|        |     |     |     |     |                   |
|        v     v     v     v     v                   |
|       [T1]  [T2]  [T3]  [T4]  [T5]                |
|                                                     |
|  Tasks start every 5 seconds regardless of          |
```

```
|  how long each task takes.                          |
|                                                     |
|  If a task takes longer than the interval:          |
|  Time: 0s    5s    10s   15s                        |
|        |     |     |     |                         |
|        v     v     v     v                         |
|       [T1-----7s]                                   |
|              [T2--]                                 |
|                    [T3--]                           |
|                                                     |
|  T2 starts as soon as T1 finishes (not at 5s        |
|  because T1 was still running at 5s).               |
+-----------------------------------------------------+
```

> Important: If a task takes longer than the fixedRate interval, the next execution waits until the current one finishes. Tasks do not overlap by default.

Using Readable Time Units

Instead of counting milliseconds, you can use time unit strings:

```java
// These are all equivalent
@Scheduled(fixedRate = 60000)             // 60,000 milliseconds
@Scheduled(fixedRateString = "60000")     // Same, but as a string
@Scheduled(fixedRateString = "PT1M")      // ISO-8601 duration: 1 minute
@Scheduled(fixedRateString = "PT30S")     // 30 seconds
@Scheduled(fixedRateString = "PT1H")      // 1 hour
```

You can also read the value from application.properties:

```properties
# application.properties
scheduling.fixed-rate=5000
```

```java
@Scheduled(fixedRateString = "${scheduling.fixed-rate}")
public void configurableTask() {
    log.info("Running with configurable rate");
}
```

23.3 Fixed Delay Scheduling

`fixedDelay` waits for N milliseconds after the previous execution finishes before starting the next one:

```java
@Component
public class FixedDelayTask {

    private static final Logger log =
        LoggerFactory.getLogger(FixedDelayTask.class);

    @Scheduled(fixedDelay = 5000)  // Wait 5s after completion
    public void processQueue() {
        log.info("Processing message queue...");

        // Simulate work that takes 3 seconds
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        log.info("Queue processing complete");
    }
}
```

How fixedDelay Works

```
+-----------------------------------------------------+
|  fixedDelay = 5000 (5 seconds)                      |
+-----------------------------------------------------+
|                                                     |
|  Time: 0s       3s   8s      11s  16s               |
|        |        |    |       |    |                |
|        v        v    v       v    v                |
|       [T1--3s--]    [T2--3s]    [T3--3s]           |
|                 |        |           |             |
|                 5s delay 5s delay    5s delay       |
|                                                     |
|  Each task starts 5 seconds after the               |
|  previous task FINISHES.                            |
|                                                     |
|  Total cycle: 3s (work) + 5s (delay) = 8s          |
+-----------------------------------------------------+
```

fixedRate vs fixedDelay

Feature	fixedRate	fixedDelay

Timer starts from	Start of previous execution	End of previous execution
Use when	You need consistent intervals	You need a gap between executions
Example	"Check every 30 seconds"	"Wait 10 seconds after finishing"
Risk	Tasks can pile up if slow	Interval varies with task duration

When to use which:

- Use `fixedRate` when you want the task to run at regular intervals (like checking for new emails every 30 seconds). The timing is predictable.
- Use `fixedDelay` when you want to ensure a gap between executions (like processing a queue where you need to finish one batch before starting the next).

23.4 Initial Delay

Sometimes you do not want a scheduled task to run immediately when the application starts. Use `initialDelay` to wait before the first execution:

```java
@Scheduled(fixedRate = 10000, initialDelay = 30000)
public void delayedTask() {
    log.info("This task waits 30 seconds before the first run, " +
        "then runs every 10 seconds");
}
```

```
+-----------------------------------------------------+
|  fixedRate = 10s, initialDelay = 30s                |
+-----------------------------------------------------+
|                                                     |
|  App starts                                         |
|  |                                                 |
|  |--- 30 seconds (initial delay) ---|              |
|                                     |              |
|                                    [T1]            |
|                                     |--10s--|      |
|                                             [T2]   |
|                                              |     |
+-----------------------------------------------------+
```

This is useful when:

- You want the application to fully start up before running tasks.

- You need to wait for external services to become available.
- You want to stagger multiple tasks so they do not all start at the same time.

23.5 Cron Expressions

For more complex schedules (like "every weekday at 9 AM" or "the first day of every month"), use cron expressions:

```java
@Scheduled(cron = "0 0 9 * * MON-FRI")
public void weekdayMorningReport() {
    log.info("Generating weekday morning report");
}
```

Cron Expression Format

A Spring cron expression has six fields:

```
┌──────────────── second (0-59)
│ ┌──────────────── minute (0-59)
│ │ ┌──────────────── hour (0-23)
│ │ │ ┌──────────────── day of month (1-31)
│ │ │ │ ┌──────────────── month (1-12 or JAN-DEC)
│ │ │ │ │ ┌──────────────── day of week (0-7 or MON-SUN, 0 and 7 = Sunday)
│ │ │ │ │ │
* * * * * *
```

> Note: Standard Unix cron has five fields (no seconds). Spring cron has six fields because it adds seconds. This is a common source of confusion.

Special Characters

Character	Meaning	Example
`*`	Every value	`*` in hour = every hour
`,`	List of values	`1,15` in day = 1st and 15th
`-`	Range	`MON-FRI` = Monday to Friday

`/`	Increment	`0/15` in minutes = every 15 minutes
`?`	No specific value	Used for day-of-month or day-of-week

Common Cron Examples

```java
// Every day at midnight
@Scheduled(cron = "0 0 0 * * *")

// Every day at 9:00 AM
@Scheduled(cron = "0 0 9 * * *")

// Every weekday (Mon-Fri) at 8:30 AM
@Scheduled(cron = "0 30 8 * * MON-FRI")

// Every 15 minutes
@Scheduled(cron = "0 0/15 * * * *")

// Every hour on the hour
@Scheduled(cron = "0 0 * * * *")

// First day of every month at midnight
@Scheduled(cron = "0 0 0 1 * *")

// Every Sunday at 2:00 AM
@Scheduled(cron = "0 0 2 * * SUN")

// Every 30 seconds
@Scheduled(cron = "0/30 * * * * *")

// At 10:15 AM on the 15th of every month
@Scheduled(cron = "0 15 10 15 * *")
```

Cron Expression Cheat Sheet

```
+-------------------------------------------------------+
|           CRON EXPRESSION CHEAT SHEET                 |
+-------------------------------------------------------+
|                                                       |
|  "0 0 9 * * *"                                        |
|   | | | | | |                                       |
|   | | | | | +-- Day of week: * (every day)            |
|   | | | | +---- Month: * (every month)                |
|   | | | +------ Day of month: * (every day)           |
|   | | +-------- Hour: 9 (9 AM)                        |
|   | +---------- Minute: 0 (on the hour)               |
```

```
|   +------------ Second: 0 (on the second)             |
|                                                       |
|  Translation: "At 9:00:00 AM every day"               |
|                                                       |
|  "0 30 8 * * MON-FRI"                                 |
|   | |  | | |    |                                   |
|   | |  | | |    +-- Mon through Fri only              |
|   | |  | | +------- Every month                      |
|   | |  | +--------- Every day of month               |
|   | |  +----------- Hour: 8                          |
|   | +-------------- Minute: 30                        |
|   +---------------- Second: 0                         |
|                                                       |
|  Translation: "At 8:30:00 AM, Mon through Fri"        |
|                                                       |
|  "0 0/15 * * * *"                                     |
|   | |    | | | |                                    |
|   | |    | | | +-- Every day of week                  |
|   | |    | | +---- Every month                        |
|   | |    | +------ Every day                          |
|   | |    +-------- Every hour                         |
|   | +------------- Every 15 min (0, 15, 30, 45)       |
|   +---------------- Second: 0                         |
|                                                       |
|  Translation: "Every 15 minutes"                      |
|                                                       |
+-------------------------------------------------------+
```

Using Cron from Properties

```properties
# application.properties
scheduling.cron.cleanup=0 0 2 * * *
scheduling.cron.report=0 0 9 * * MON-FRI
```

```java
@Scheduled(cron = "${scheduling.cron.cleanup}")
public void cleanup() {
    log.info("Running cleanup task");
}

@Scheduled(cron = "${scheduling.cron.report}")
public void generateReport() {
    log.info("Generating daily report");
}
```

This is very useful because you can change schedules without modifying code.

23.6 A Complete Scheduled Task Example

Let us build a practical example: a task that cleans up expired temporary files:

```java
// src/main/java/com/example/scheduling/task/CleanupTask.java
package com.example.scheduling.task;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.concurrent.atomic.AtomicInteger;

@Component
public class CleanupTask {

    private static final Logger log =
        LoggerFactory.getLogger(CleanupTask.class);

    @Value("${file.upload-dir:uploads}")
    private String uploadDir;

    @Value("${cleanup.max-age-hours:24}")
    private int maxAgeHours;

    // Run every day at 2:00 AM
    @Scheduled(cron = "0 0 2 * * *")
    public void cleanupOldFiles() {
        log.info("Starting file cleanup task. " +
            "Removing files older than {} hours", maxAgeHours);

        Path uploadPath = Paths.get(uploadDir);

        if (!Files.exists(uploadPath)) {
            log.warn("Upload directory does not exist: {}", uploadPath);
            return;
        }

        Instant cutoff = Instant.now()
            .minus(maxAgeHours, ChronoUnit.HOURS);

        AtomicInteger deletedCount = new AtomicInteger(0);
        AtomicInteger errorCount = new AtomicInteger(0);
```

```
        try {
            Files.list(uploadPath).forEach(file -> {
                try {
                    Instant lastModified = Files
                        .getLastModifiedTime(file).toInstant();

                    if (lastModified.isBefore(cutoff)) {
                        Files.delete(file);
                        deletedCount.incrementAndGet();
                        log.debug("Deleted old file: {}", file);
                    }
                } catch (IOException e) {
                    errorCount.incrementAndGet();
                    log.error("Failed to process file: {}", file, e);
                }
            });
        } catch (IOException e) {
            log.error("Failed to list files in {}", uploadPath, e);
        }

        log.info("Cleanup complete. Deleted: {}, Errors: {}",
            deletedCount.get(), errorCount.get());
    }
}
```

Line-by-line explanation:

- `@Value("${cleanup.max-age-hours:24}")` -- Reads the max age from properties, with a default of 24 hours.
- `Instant.now().minus(maxAgeHours, ChronoUnit.HOURS)` -- Calculates the cutoff time. Files older than this are deleted.
- `AtomicInteger` -- Thread-safe counter. We use it inside a lambda where regular variables would need to be effectively final.
- `Files.getLastModifiedTime(file).toInstant()` -- Gets the last modification time of each file.
- The method logs a summary at the end (how many files deleted, how many errors).

Output at 2:00 AM:

```
02:00:00.001 INFO  CleanupTask - Starting file cleanup task. Removing files older than ...
02:00:00.015 DEBUG CleanupTask - Deleted old file: uploads/a1b2c3d4_old.jpg
02:00:00.016 DEBUG CleanupTask - Deleted old file: uploads/e5f6a7b8_expired.pdf
02:00:00.020 INFO  CleanupTask - Cleanup complete. Deleted: 2, Errors: 0
```

23.7 Error Handling in Scheduled Tasks

By default, if a scheduled task throws an exception, the exception is logged but the task continues to run on schedule. However, you should handle errors explicitly:

```java
// src/main/java/com/example/scheduling/task/ResilientTask.java
package com.example.scheduling.task;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class ResilientTask {

    private static final Logger log =
        LoggerFactory.getLogger(ResilientTask.class);

    private int consecutiveFailures = 0;
    private static final int MAX_FAILURES = 5;

    @Scheduled(fixedRate = 60000)  // Every minute
    public void processWithErrorHandling() {
        try {
            log.info("Starting scheduled processing");

            // Your business logic here
            doWork();

            // Reset failure counter on success
            consecutiveFailures = 0;
            log.info("Scheduled processing completed successfully");

        } catch (Exception e) {
            consecutiveFailures++;
            log.error("Scheduled task failed. " +
                "Consecutive failures: {}/{}",
                consecutiveFailures, MAX_FAILURES, e);

            if (consecutiveFailures >= MAX_FAILURES) {
                log.error("Task has failed {} consecutive times. " +
                    "Manual intervention may be needed.",
                    consecutiveFailures);
                // Optionally: send alert, disable task, etc.
            }
        }
    }

    private void doWork() {
        // Simulate work that might fail
```

```
        if (Math.random() < 0.3) {
            throw new RuntimeException("Simulated failure");
        }
    }
}
```

Custom Error Handler for All Scheduled Tasks

You can configure a global error handler:

```java
// src/main/java/com/example/scheduling/config/SchedulingConfig.java
package com.example.scheduling.config;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.SchedulingConfigurer;
import org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler;
import org.springframework.scheduling.config.ScheduledTaskRegistrar;

@Configuration
@EnableScheduling
public class SchedulingConfig implements SchedulingConfigurer {

    private static final Logger log =
        LoggerFactory.getLogger(SchedulingConfig.class);

    @Override
    public void configureTasks(ScheduledTaskRegistrar taskRegistrar) {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5);  // Allow 5 concurrent tasks
        scheduler.setThreadNamePrefix("scheduled-");
        scheduler.setErrorHandler(throwable -> {
            log.error("Scheduled task error: {}",
                throwable.getMessage(), throwable);
        });
        scheduler.initialize();
        taskRegistrar.setTaskScheduler(scheduler);
    }
}
```

Line-by-line explanation:

- `ThreadPoolTaskScheduler` -- Replaces the default single-threaded scheduler with a thread pool.
- `setPoolSize(5)` -- Allows up to 5 scheduled tasks to run simultaneously. The default is 1, which means all tasks share a single thread.

- `setThreadNamePrefix("scheduled-")` -- Makes it easy to identify scheduled task threads in logs.
- `setErrorHandler(...)` -- A global error handler that catches any unhandled exception from any scheduled task.

> Why is the pool size important? With the default pool size of 1, if one scheduled task blocks (takes too long or hangs), all other scheduled tasks are delayed. Setting the pool size to a higher value allows multiple tasks to run in parallel.

```
+-----------------------------------------------------+
|  DEFAULT: Pool Size = 1                              |
+-----------------------------------------------------+
|                                                      |
|  Task A (runs every 5s): [AAAA]   [AAAA]   [AAAA]  |
|  Task B (runs every 5s):       [BB]   [BB]   [BB]  |
|                                                      |
|  Task B has to wait for Task A to finish!            |
|                                                      |
+-----------------------------------------------------+
|  CONFIGURED: Pool Size = 5                           |
+-----------------------------------------------------+
|                                                      |
|  Task A (runs every 5s): [AAAA]   [AAAA]   [AAAA]  |
|  Task B (runs every 5s): [BB]     [BB]     [BB]    |
|                                                      |
|  Both tasks run independently!                       |
+-----------------------------------------------------+
```

23.8 Conditional Scheduling

Sometimes you want a scheduled task to run only in certain environments (like production but not development):

```java
@Component
@Profile("prod")  // Only runs in the prod profile
public class ProductionOnlyTask {

    private static final Logger log =
        LoggerFactory.getLogger(ProductionOnlyTask.class);

    @Scheduled(cron = "0 0 0 * * *")
    public void nightlyBackup() {
        log.info("Running nightly backup (production only)");
```

```
    }
}
```

You can also use `@ConditionalOnProperty`:

```java
@Component
@ConditionalOnProperty(
    name = "scheduling.cleanup.enabled",
    havingValue = "true",
    matchIfMissing = false
)
public class ConditionalCleanupTask {

    @Scheduled(fixedRate = 60000)
    public void cleanup() {
        // Only runs if scheduling.cleanup.enabled=true in properties
    }
}
```

```properties
# Enable or disable the cleanup task
scheduling.cleanup.enabled=true
```

23.9 Scheduling Summary Table

Attribute	Behavior	Example
`fixedRate`	Run every N ms from start	`@Scheduled(fixedRate = 5000)`
`fixedDelay`	Wait N ms after completion	`@Scheduled(fixedDelay = 5000)`
`cron`	Run on a schedule	`@Scheduled(cron = "0 0 9 * * *")`
`initialDelay`	Wait before first run	`@Scheduled(fixedRate = 5000, initialDelay = 10000)`
`fixedRateString`	Rate from property	`@Scheduled(fixedRateString = "${rate}")`

`fixedDelayString`	Delay from property	`@Scheduled(fixedDelayString = "${delay}")`

Common Mistakes

1. Forgetting @EnableScheduling. Without this annotation, all @Scheduled methods are silently ignored. Nothing happens, and there is no error message.
2. Forgetting @Component on the task class. The class must be a Spring bean. If it is not in the Spring context, the scheduled method will never run.
3. Using five-field cron expressions. Spring cron expressions have six fields (including seconds). Unix cron has five fields. A five-field expression will cause an error.
4. Not increasing the thread pool size. The default scheduler uses a single thread. If you have multiple scheduled tasks, they will block each other. Configure a ThreadPoolTaskScheduler with an appropriate pool size.
5. Ignoring exceptions in scheduled tasks. If a scheduled method throws an uncaught exception, it will be logged but the task will continue running on schedule. Always wrap your logic in try-catch and log errors properly.
6. Making scheduled methods return a value. Scheduled methods must return void and take no parameters. If you add a return type or parameters, Spring will not recognize it as a scheduled method.

Best Practices

1. Externalize schedules in application.properties. Use `fixedRateString = "${property.name}"` or `cron = "${cron.expression}"` so you can change schedules without redeploying.
2. Log the start and end of every scheduled task. Include how long the task took and whether it succeeded or failed. This makes troubleshooting much easier.
3. Use a thread pool for multiple tasks. Configure a `ThreadPoolTaskScheduler` with a pool size greater than 1 to prevent tasks from blocking each other.
4. Handle errors gracefully. Wrap scheduled task logic in try-catch. Track consecutive failures and alert if a task keeps failing.
5. Use @Profile or @ConditionalOnProperty for environment-specific tasks. Do not run production cleanup tasks in your development environment.

6. Keep scheduled tasks lightweight. If a task does heavy processing, make sure the interval is long enough for it to complete. Or use `fixedDelay` instead of `fixedRate`.

Quick Summary

Spring Boot scheduling lets you run tasks automatically at fixed intervals or on specific schedules. Enable it with @EnableScheduling on your application class. Use @Scheduled(fixedRate) to run a task every N milliseconds from the start of the previous run. Use @Scheduled(fixedDelay) to wait N milliseconds after the previous run finishes. For complex schedules like "every weekday at 9 AM," use cron expressions with six fields: second, minute, hour, day-of-month, month, day-of-week. Configure a ThreadPoolTaskScheduler to allow multiple tasks to run in parallel. Always handle errors in scheduled tasks and externalize schedules in application properties so you can change them without redeploying.

Key Points

- @EnableScheduling activates the scheduling system.
- @Scheduled methods must return void and have no parameters.
- fixedRate: interval between the START of executions.
- fixedDelay: interval between the END of one execution and the START of the next.
- Spring cron expressions have 6 fields (including seconds). Unix cron has 5.
- Use initialDelay to postpone the first execution after application startup.
- The default scheduler has a single thread. Use ThreadPoolTaskScheduler for multiple concurrent tasks.
- Externalize cron expressions and intervals in application.properties.
- Always handle exceptions in scheduled tasks with try-catch.
- Use @Profile to restrict tasks to specific environments.

Practice Questions

1. What is the difference between `fixedRate = 5000` and `fixedDelay = 5000` when a task takes 3 seconds to complete?
2. How many fields does a Spring cron expression have, and what does each field represent?
3. What happens if you use @Scheduled on a method but forget to add @EnableScheduling to your application?
4. Why should you configure a ThreadPoolTaskScheduler with a pool size greater than 1?
5. Write a cron expression that runs at 3:30 PM every Monday and Wednesday.

Exercises

Exercise 1: Heartbeat Logger

Create a scheduled task that logs a "heartbeat" message every 10 seconds. Include the current timestamp and the application uptime (how long since the application started).

Exercise 2: Database Cleanup Task

Create a scheduled task that runs at 3:00 AM every day to delete records from a "session" table where the "expires_at" column is in the past. Use JPA. Log how many expired sessions were removed.

Exercise 3: Multiple Tasks with Thread Pool

Create three scheduled tasks with different intervals (5 seconds, 10 seconds, 30 seconds). Each task should simulate work by sleeping for 2 seconds. Configure a ThreadPoolTaskScheduler with a pool size of 3. Verify that all three tasks run concurrently by checking the thread names in the logs.

What Is Next?

Your application can now run tasks automatically on any schedule. But some tasks, like looking up product data or computing reports, are slow because they hit the database every single time. What if you could remember the result and return it instantly the next time someone asks? That is caching. In Chapter 24, you will learn how to use Spring Boot's

caching annotations to dramatically speed up your application by storing frequently accessed data in memory.

Chapter 24: Caching

What You Will Learn

- What caching is and why it makes applications faster.
- How to enable caching with @EnableCaching.
- How to cache method results with @Cacheable.
- How to remove cached data with @CacheEvict.
- How to update cached data with @CachePut.
- How cache keys work and how to customize them.
- How ConcurrentMapCacheManager works as the default cache provider.
- When to cache and when not to cache.
- How to build a complete product service with caching.

Why This Chapter Matters

Imagine you work at an information desk. Someone asks "What time does the museum open?" You look it up in a binder, flip through 50 pages, and find the answer: 9 AM. Thirty seconds later, another person asks the same question. Do you flip through the binder again? Of course not. You remember the answer because you just looked it up.

That is caching. Your application remembers answers to questions it has already answered so it does not have to do the work again.

Consider a product catalog page on an e-commerce site. Every time a user visits the page, the application queries the database, builds the response, and sends it back. If 1000 users visit the same page in a minute, that is 1000 identical database queries. With caching, you query the database once, store the result in memory, and serve the next 999 requests from the cache. The response time drops from 200 milliseconds to 2 milliseconds.

24.1 What Is Caching?

Caching is storing frequently accessed data in a fast storage layer (usually memory) so you can retrieve it quickly instead of recalculating or re-fetching it every time.

```
+-------------------------------------------------------+
|         WITHOUT CACHING                               |
+-------------------------------------------------------+
|                                                       |
|  Request 1: GET /api/products/42                      |
|       --> Query database (200ms)                      |
|       --> Return product                              |
|                                                       |
|  Request 2: GET /api/products/42                      |
|       --> Query database (200ms)  <-- Same query!     |
|       --> Return product                              |
|                                                       |
|  Request 3: GET /api/products/42                      |
|       --> Query database (200ms)  <-- Same again!     |
|       --> Return product                              |
|                                                       |
|  Total time: 600ms for 3 identical queries            |
+-------------------------------------------------------+

+-------------------------------------------------------+
|         WITH CACHING                                  |
+-------------------------------------------------------+
|                                                       |
|  Request 1: GET /api/products/42                      |
|       --> Query database (200ms)                      |
|       --> Store in cache                              |
|       --> Return product                              |
|                                                       |
|  Request 2: GET /api/products/42                      |
|       --> Found in cache! (2ms)  <-- No database!     |
|       --> Return product                              |
|                                                       |
|  Request 3: GET /api/products/42                      |
|       --> Found in cache! (2ms)  <-- No database!     |
|       --> Return product                              |
|                                                       |
|  Total time: 204ms (100x faster for requests 2 & 3) |
+-------------------------------------------------------+
```

The Cache Flow

```
+-----------------------------------------------------+
|              CACHE LOOKUP FLOW                      |
+-----------------------------------------------------+
```

```
|                                                     |
|  Method called with arguments                       |
|       |                                             |
|       v                                             |
|  [Check cache: is there a stored result             |
|   for these arguments?]                             |
|       |                                             |
|       +--> YES (cache hit)                          |
|       |    Return cached result immediately         |
|       |    Method body does NOT execute              |
|       |                                             |
|       +--> NO (cache miss)                          |
|            Execute method body                      |
|            Store result in cache                    |
|            Return result                            |
|                                                     |
+-----------------------------------------------------+
```

24.2 Enabling Caching in Spring Boot

Add the caching starter to your project (it is included with spring-boot-starter-web, but you need to enable it):

```java
// src/main/java/com/example/caching/CachingApplication.java
package com.example.caching;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cache.annotation.EnableCaching;

@SpringBootApplication
@EnableCaching  // Enables caching support
public class CachingApplication {

    public static void main(String[] args) {
        SpringApplication.run(CachingApplication.class, args);
    }
}
```

Line-by-line explanation:

- `@EnableCaching` -- This annotation activates Spring's caching infrastructure. Without it, all caching annotations (@Cacheable, @CacheEvict, @CachePut) are silently ignored.

By default, Spring Boot uses `ConcurrentMapCacheManager`, which stores cached data in a simple in-memory `ConcurrentHashMap`. This is perfect for development and small applications.

For production, you can switch to Redis, Caffeine, or EhCache without changing your code.

24.3 @Cacheable: Caching Method Results

The `@Cacheable` annotation stores the return value of a method in the cache. The next time the method is called with the same arguments, the cached value is returned without executing the method:

```java
// src/main/java/com/example/caching/service/ProductService.java
package com.example.caching.service;

import com.example.caching.entity.Product;
import com.example.caching.repository.ProductRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Cacheable("products")
    public Product getProductById(Long id) {
        log.info("Fetching product {} from database", id);
        simulateSlowQuery();  // Simulate slow database query

        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found: " + id));
    }

    private void simulateSlowQuery() {
        try {
            Thread.sleep(2000);  // Simulate 2 second delay
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Line-by-line explanation:

- `@Cacheable("products")` -- The string "products" is the cache name. Think of it as a named container. All products are stored in the "products" cache. You can have different caches for different types of data (like "products", "users", "orders").
- The first time `getProductById(42)` is called, the method executes, queries the database, and the result is stored in the cache with key `42`.
- The second time `getProductById(42)` is called, Spring finds the result in the cache and returns it immediately. The method body never executes. You will not see the log message.

Testing the Cache

```bash
# First call - takes ~2 seconds (database query)
curl http://localhost:8080/api/products/1
# Log: "Fetching product 1 from database"

# Second call - instant (from cache)
curl http://localhost:8080/api/products/1
# No log message! Method body did not execute.

# Different product - takes ~2 seconds (not in cache yet)
curl http://localhost:8080/api/products/2
# Log: "Fetching product 2 from database"
```

24.4 Understanding Cache Keys

By default, Spring uses the method arguments as the cache key. Here is how it works:

```
+-----------------------------------------------------+
|           CACHE KEY MAPPING                         |
+-----------------------------------------------------+
|                                                     |
|  Cache name: "products"                             |
|                                                     |
|  Key        | Value                                 |
|  ----------|-------------------------------------  |
|  1          | Product{id=1, name="Widget", ...}     |
|  2          | Product{id=2, name="Gadget", ...}     |
|  42         | Product{id=42, name="Doohickey", ...} |
|                                                     |
|  getProductById(1)  --> uses key 1                  |
```

```
|  getProductById(2)  --> uses key 2                 |
|  getProductById(42) --> uses key 42                |
|                                                    |
+----------------------------------------------------+
```

Multiple Arguments as Key

When a method has multiple arguments, all arguments form the key:

```java
@Cacheable("products")
public List<Product> searchProducts(String category, double maxPrice) {
    // Key is the combination of category AND maxPrice
    // searchProducts("electronics", 100.0) has a different key than
    // searchProducts("electronics", 200.0)
    return productRepository.findByCategoryAndPriceLessThan(
        category, maxPrice);
}
```

Custom Cache Keys

You can specify which arguments to use as the key:

```java
// Use only the first argument as the key
@Cacheable(value = "products", key = "#category")
public List<Product> getByCategory(String category, int page) {
    return productRepository.findByCategory(category);
}

// Use a combination of arguments
@Cacheable(value = "products", key = "#category + '-' + #maxPrice")
public List<Product> searchProducts(String category, double maxPrice) {
    return productRepository.findByCategoryAndPriceLessThan(
        category, maxPrice);
}

// Use the object's property as the key
@Cacheable(value = "products", key = "#product.id")
public Product updateProduct(Product product) {
    return productRepository.save(product);
}
```

The `key` attribute uses Spring Expression Language (SpEL). The `#` prefix refers to method parameters.

24.5 @CacheEvict: Removing Cached Data

When data changes (update, delete), you need to remove the stale cached data. Otherwise, users will get outdated information:

```java
@Service
public class ProductService {

    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Cacheable("products")
    public Product getProductById(Long id) {
        log.info("Fetching product {} from database", id);
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found: " + id));
    }

    @CacheEvict(value = "products", key = "#id")
    public void deleteProduct(Long id) {
        log.info("Deleting product {} and evicting from cache", id);
        productRepository.deleteById(id);
    }

    @CacheEvict(value = "products", allEntries = true)
    public void clearProductCache() {
        log.info("All product cache entries cleared");
    }
}
```

Line-by-line explanation:

- `@CacheEvict(value = "products", key = "#id")` -- When this method runs, it removes the entry with the given key from the "products" cache. After deleting product 42 from the database, the cached copy is also removed.
- `@CacheEvict(value = "products", allEntries = true)` -- Removes ALL entries from the "products" cache. Use this when you need to clear the entire cache (like after a bulk import).

```
+----------------------------------------------------+
|           CACHE EVICTION FLOW                      |
+----------------------------------------------------+
```

```
|                                                      |
|  Before deleteProduct(42):                           |
|  Cache: { 1: Widget, 2: Gadget, 42: Doohickey }   |
|                                                      |
|  deleteProduct(42) runs:                             |
|    1. Delete from database                           |
|    2. @CacheEvict removes key 42 from cache          |
|                                                      |
|  After deleteProduct(42):                            |
|  Cache: { 1: Widget, 2: Gadget }                     |
|                                                      |
|  Next call to getProductById(42):                    |
|    Cache miss --> Query database --> Not found      |
|                                                      |
+-----------------------------------------------------+
```

24.6 @CachePut: Updating the Cache

`@CachePut` always executes the method and updates the cache with the result. Unlike `@Cacheable` (which skips the method if a cached value exists), `@CachePut` always runs the method:

```java
@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {
    log.info("Updating product {} and refreshing cache",
        product.getId());
    return productRepository.save(product);
}
```

Line-by-line explanation:

- `@CachePut(value = "products", key = "#product.id")` -- Executes the method, saves the product to the database, and stores the returned product in the cache. The next time someone calls `getProductById()` with this ID, they get the updated product from the cache.

@Cacheable vs @CachePut vs @CacheEvict

Annotation	Executes Method?	Updates Cache?	Use Case
`@Cacheable`	Only on cache miss	Yes (stores result)	Read operations

`@CachePut`	Always	Yes (updates result)	Update operations
`@CacheEvict`	Always	Yes (removes entry)	Delete operations

```
+-----------------------------------------------------+
|     COMPLETE CRUD WITH CACHING                      |
+-----------------------------------------------------+
|                                                     |
|  CREATE: save(product)                              |
|    @CachePut --> Store in cache after saving        |
|                                                     |
|  READ: getProductById(42)                           |
|    @Cacheable --> Return from cache if exists       |
|                   Query database if not             |
|                                                     |
|  UPDATE: updateProduct(product)                     |
|    @CachePut --> Update cache after saving          |
|                                                     |
|  DELETE: deleteProduct(42)                          |
|    @CacheEvict --> Remove from cache after delete   |
|                                                     |
+-----------------------------------------------------+
```

24.7 Complete Product Service Example

Let us build a complete service with all caching annotations:

The Product Entity

```java
// src/main/java/com/example/caching/entity/Product.java
package com.example.caching.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```
    private String category;
    private double price;

    public Product() {}

    public Product(String name, String category, double price) {
        this.name = name;
        this.category = category;
        this.price = price;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    @Override
    public String toString() {
        return "Product{id=" + id + ", name='" + name +
            "', category='" + category +
            "', price=" + price + "}";
    }
}
```

The Product Repository

```java
// src/main/java/com/example/caching/repository/ProductRepository.java
package com.example.caching.repository;

import com.example.caching.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface ProductRepository extends JpaRepository<Product, Long> {

    List<Product> findByCategory(String category);
}
```

The Complete Cached Product Service

```java
// src/main/java/com/example/caching/service/ProductService.java
package com.example.caching.service;

import com.example.caching.entity.Product;
import com.example.caching.repository.ProductRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.CachePut;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ProductService {

    private static final Logger log =
        LoggerFactory.getLogger(ProductService.class);

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Cacheable("products")
    public Product getProductById(Long id) {
        log.info("Cache miss - Fetching product {} from database", id);
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Product not found: " + id));
    }

    @Cacheable("productsByCategory")
    public List<Product> getProductsByCategory(String category) {
        log.info("Cache miss - Fetching products in category: {}",
            category);
        return productRepository.findByCategory(category);
    }

    @CachePut(value = "products", key = "#result.id")
    public Product createProduct(Product product) {
        log.info("Creating product: {}", product.getName());
        return productRepository.save(product);
    }

    @CachePut(value = "products", key = "#product.id")
    @CacheEvict(value = "productsByCategory", allEntries = true)
    public Product updateProduct(Product product) {
        log.info("Updating product: {}", product.getId());

        if (!productRepository.existsById(product.getId())) {
            throw new RuntimeException(
```

```
                "Product not found: " + product.getId());
        }

        return productRepository.save(product);
    }

    @CacheEvict(value = {"products", "productsByCategory"},
                allEntries = true)
    public void deleteProduct(Long id) {
        log.info("Deleting product: {}", id);
        productRepository.deleteById(id);
    }
}
```

Line-by-line explanation:

- `@Cacheable("products")` on `getProductById` -- Caches individual products by their ID.
- `@Cacheable("productsByCategory")` on `getProductsByCategory` -- Uses a separate cache for category queries. Different data, different cache.
- `@CachePut(value = "products", key = "#result.id")` on `createProduct` -- The `#result` refers to the return value of the method. After saving, the newly created product (with its generated ID) is cached.
- `@CachePut` + `@CacheEvict` on `updateProduct` -- Updates the individual product cache AND clears the category cache (because the product's category might have changed).
- `@CacheEvict(value = {"products", "productsByCategory"}, allEntries = true)` on `deleteProduct` -- Clears both caches when a product is deleted.

The Product Controller

```java
// src/main/java/com/example/caching/controller/ProductController.java
package com.example.caching.controller;

import com.example.caching.entity.Product;
import com.example.caching.service.ProductService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;
```

```
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProduct(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @GetMapping("/category/{category}")
    public ResponseEntity<List<Product>> getByCategory(
            @PathVariable String category) {
        return ResponseEntity.ok(
            productService.getProductsByCategory(category));
    }

    @PostMapping
    public ResponseEntity<Product> createProduct(
            @RequestBody Product product) {
        Product created = productService.createProduct(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable Long id,
            @RequestBody Product product) {
        product.setId(id);
        return ResponseEntity.ok(productService.updateProduct(product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

24.8 Testing the Caching Behavior

Step 1: Create a Product

```bash
curl -X POST http://localhost:8080/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget", "category": "electronics", "price": 29.99}'
```

Output:

```json
{"id": 1, "name": "Widget", "category": "electronics", "price": 29.99}
```

Log:

```
Creating product: Widget
```

Step 2: Get the Product (First Call - Cache Miss)

```bash
curl http://localhost:8080/api/products/1
```

Log:

```
Cache miss - Fetching product 1 from database
```

Step 3: Get the Product Again (Cache Hit)

```bash
curl http://localhost:8080/api/products/1
```

Log: (Nothing! The method body did not execute because the result came from the cache.)

Step 4: Update the Product

```bash
curl -X PUT http://localhost:8080/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "Super Widget", "category": "electronics", "price": 39.99}'
```

Log:

```
Updating product: 1
```

Step 5: Get the Updated Product (From Cache)

```bash
curl http://localhost:8080/api/products/1
```

Output: Shows the updated product. No log message because `@CachePut` already updated the cache during the update.

24.9 ConcurrentMapCacheManager

By default, Spring Boot uses `ConcurrentMapCacheManager`. It stores cache entries in `ConcurrentHashMap` objects in memory.

You can also configure it explicitly:

```java
// src/main/java/com/example/caching/config/CacheConfig.java
package com.example.caching.config;

import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cache.concurrent.ConcurrentMapCacheManager;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager(
            "products",
            "productsByCategory"
        );
    }
}
```

Line-by-line explanation:

- `ConcurrentMapCacheManager("products", "productsByCategory")` -- Pre-creates the named caches. You can also let Spring create them on demand (which is the default behavior).

Characteristics of ConcurrentMapCacheManager

Feature	ConcurrentMapCacheManager
Storage	In-memory (heap)
Eviction	None (entries stay forever until explicitly removed)
Max size	No limit (can cause OutOfMemoryError)
TTL (Time-to-live)	Not supported
Distributed	No (single JVM only)
Use case	Development, small apps, testing

For production with high traffic, consider switching to Caffeine (in-memory with TTL and size limits) or Redis (distributed, persistent). The beauty of Spring's cache abstraction is that you only need to change the CacheManager bean. Your `@Cacheable`, `@CacheEvict`, and `@CachePut` annotations stay the same.

24.10 Conditional Caching

Sometimes you want to cache results only under certain conditions:

```java
// Only cache if the product price is above 10
@Cacheable(value = "products",
           condition = "#id > 0")
public Product getProductById(Long id) {
    return productRepository.findById(id).orElseThrow();
}

// Only cache if the result is not null
@Cacheable(value = "products",
           unless = "#result == null")
public Product findProduct(Long id) {
    return productRepository.findById(id).orElse(null);
}

// Combine conditions
@Cacheable(value = "products",
           condition = "#id > 0",
           unless = "#result.price < 1.0")
public Product getProduct(Long id) {
    return productRepository.findById(id).orElseThrow();
}
```

Line-by-line explanation:

- `condition` -- Evaluated BEFORE the method runs. If it returns false, the cache is not checked and the result is not cached. Use `#paramName` to reference method arguments.
- `unless` -- Evaluated AFTER the method runs. If it returns true, the result is NOT cached. Use `#result` to reference the return value.

Attribute	When Evaluated	If True
`condition`	Before method	Cache is checked and result is cached
`unless`	After method	Result is NOT cached

24.11 When to Cache and When Not to Cache

Good Candidates for Caching

Data Type	Why Cache It
Product catalogs	Rarely change, frequently accessed
User profiles	Read often, updated rarely
Configuration data	Almost never changes
API responses from external services	Slow to fetch, rate-limited
Computed results	Expensive calculations, same inputs give same outputs

Bad Candidates for Caching

Data Type	Why NOT Cache It
Real-time stock prices	Changes every second
User session data	Unique per user, short-lived
Random or unique data	Every call produces different results

Data that must always be fresh	Account balances, inventory counts
Very large datasets	Can exhaust memory

The Caching Decision Flowchart

```
+-----------------------------------------------------+
|     SHOULD YOU CACHE THIS DATA?                     |
+-----------------------------------------------------+
|                                                     |
|  Is the data read frequently?                       |
|       |                                             |
|       +--> No --> Do not cache                      |
|       |                                             |
|       +--> Yes --> Continue                         |
|       |                                             |
|       v                                             |
|  Does the data change rarely?                       |
|       |                                             |
|       +--> No --> Do not cache (or use short TTL)   |
|       |                                             |
|       +--> Yes --> Continue                         |
|       |                                             |
|       v                                             |
|  Is fetching the data slow or expensive?            |
|       |                                             |
|       +--> No --> Caching provides little benefit   |
|       |                                             |
|       +--> Yes --> CACHE IT!                        |
|       |                                             |
|       v                                             |
|  Is stale data acceptable for a short time?         |
|       |                                             |
|       +--> No --> Cache with immediate eviction     |
|       |          on updates                         |
|       |                                             |
|       +--> Yes --> Cache with TTL                   |
|                                                     |
+-----------------------------------------------------+
```

24.12 Initializing Sample Data

For testing, let us create sample data on startup:

```java
// src/main/java/com/example/caching/config/DataInitializer.java
package com.example.caching.config;

import com.example.caching.entity.Product;
import com.example.caching.repository.ProductRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataInitializer implements CommandLineRunner {

    private final ProductRepository productRepository;

    public DataInitializer(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Override
    public void run(String... args) {
        productRepository.save(
            new Product("Laptop", "electronics", 999.99));
        productRepository.save(
            new Product("Phone", "electronics", 699.99));
        productRepository.save(
            new Product("Desk", "furniture", 249.99));
        productRepository.save(
            new Product("Chair", "furniture", 199.99));
        productRepository.save(
            new Product("Coffee Mug", "kitchen", 12.99));

        System.out.println("Sample products loaded: "
            + productRepository.count());
    }
}
```

And the application properties:

```properties
# src/main/resources/application.properties
spring.datasource.url=jdbc:h2:mem:cachingdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true

# Show cache-related logs
logging.level.org.springframework.cache=TRACE
logging.level.com.example.caching=DEBUG
```

Setting `org.springframework.cache` to TRACE lets you see cache hits and misses in the logs.

Common Mistakes

1. Forgetting @EnableCaching. Without this annotation, all caching annotations are ignored. Your application will work, but nothing will be cached.
2. Calling cached methods from within the same class. Spring caching uses proxies. If method A in a class calls method B (which has @Cacheable) in the same class, the cache is bypassed because the proxy is not involved. Always call cached methods from another class.

```java
@Service
public class ProductService {

    @Cacheable("products")
    public Product getById(Long id) { ... }

    public Product getAndProcess(Long id) {
        // THIS BYPASSES THE CACHE because it is a
        // self-invocation (same class)
        Product p = getById(id);
        return process(p);
    }
}
```

3. Not evicting the cache when data changes. If you update or delete data without evicting the cache, users will see stale data.
4. Caching mutable objects. If you cache an object and then modify it, the cached version is also modified (because they are the same object in memory). Cache immutable objects or copies.
5. Using the default ConcurrentMapCacheManager in production with high traffic. It has no size limit and no TTL. Entries stay forever and can cause OutOfMemoryError.
6. Caching null values unintentionally. By default, null values are cached. Use `unless = "#result == null"` to prevent this.

Best Practices

1. Cache at the service layer, not the controller layer. The service layer is where business logic lives. Caching there means any caller benefits from the cache.
2. Use meaningful cache names. Use names like "products", "userProfiles", "orderSummaries" instead of generic names like "cache1" or "myCache".

3. Always evict or update the cache when data changes. Pair every write operation with a @CacheEvict or @CachePut.
4. Log cache misses. Add a log statement in cached methods so you can see when the method actually executes versus when the cache is used.
5. Consider cache size and TTL for production. Switch from ConcurrentMapCacheManager to Caffeine (for size limits and TTL) or Redis (for distributed caching).
6. Test with caching enabled and disabled. Your application should work correctly even without caching. Caching is an optimization, not a requirement.

Quick Summary

Caching stores frequently accessed data in memory so your application can return results instantly instead of querying the database every time. Spring Boot's caching is enabled with @EnableCaching and uses three main annotations. @Cacheable stores a method's return value in the cache and returns the cached value on subsequent calls with the same arguments. @CacheEvict removes entries from the cache when data is deleted or you need to refresh. @CachePut always executes the method and updates the cache with the result, used for update operations. Cache keys are based on method arguments by default but can be customized with SpEL expressions. The default ConcurrentMapCacheManager is good for development but should be replaced with Caffeine or Redis for production use.

Key Points

- @EnableCaching activates Spring's caching infrastructure.
- @Cacheable caches method results. The method is skipped on cache hits.
- @CacheEvict removes entries from the cache.
- @CachePut always runs the method and updates the cache.
- Cache keys default to method arguments. Customize with the `key` attribute.
- Use `condition` to control when caching applies (evaluated before the method).
- Use `unless` to prevent caching certain results (evaluated after the method).
- Self-invocation (calling a cached method from within the same class) bypasses the cache.
- ConcurrentMapCacheManager is the default. It has no size limits or TTL.

- Always evict or update the cache when the underlying data changes.
- Caching is best for data that is read often, changes rarely, and is expensive to fetch.

Practice Questions

1. What is the difference between @Cacheable and @CachePut? When would you use each?
2. Why does calling a @Cacheable method from within the same class bypass the cache?
3. What happens if you use @Cacheable but forget to add @EnableCaching to your application?
4. You have a method `getUser(Long id)` with @Cacheable. A user updates their profile. How do you make sure the next call to `getUser` returns the updated data?
5. Why is ConcurrentMapCacheManager not recommended for high-traffic production applications?

Exercises

Exercise 1: Cache a User Service

Create a UserService with @Cacheable on `getUserById()`. Create a UserController that calls it. Call the endpoint three times and verify (using log output) that the database is only queried once.

Exercise 2: Implement Cache Eviction

Add update and delete operations to the UserService from Exercise 1. Use @CachePut for updates and @CacheEvict for deletes. Test that after updating a user, the cached version reflects the new data.

Exercise 3: Category-Based Caching

Create a product service that caches products by category using a separate cache called "productsByCategory". When a product is added, updated, or deleted, clear the category cache. Test by fetching products by category, adding a new product, and fetching again (the second fetch should hit the database because the cache was cleared).

What Is Next?

Your application is now faster thanks to caching. You have learned how to store results in memory, evict stale data, and update cached values when the underlying data changes. In Chapter 25, you will learn how to send emails from your Spring Boot application. You will configure an email service, send plain text and HTML emails, and attach files to messages.

Chapter 25: Sending Emails

What You Will Learn

- How email works behind the scenes (SMTP protocol basics).
- How to add email support to a Spring Boot application.
- How to configure SMTP settings for Gmail.
- How to send simple text emails using JavaMailSender.
- How to send HTML emails using MimeMessage.
- How to create beautiful email templates with Thymeleaf.
- How to send emails asynchronously so users do not wait.
- How to build a complete email verification flow for user registration.

Why This Chapter Matters

Think about the last time you signed up for a website. What happened? You got an email. Maybe it said "Verify your email address" or "Welcome to our platform." Emails are everywhere in modern applications.

Your banking app sends transaction alerts. Your favorite online store sends order confirmations. Your project management tool sends daily digests. Email is the backbone of application-to-user communication.

If you are building a real application, you will need to send emails. Whether it is password resets, welcome messages, order confirmations, or notification alerts, email is not optional. It is expected.

This chapter teaches you how to send emails from your Spring Boot application. You will start with simple text emails, move to beautiful HTML emails, and finish with a complete email verification system that real applications use.

25.1 How Email Works: The Postal Service Analogy

Before we write code, let us understand how email works. It is a lot like the postal service.

When you send a physical letter:

1. You write the letter (compose the email).
2. You put it in an envelope with a "to" address and a "from" address (set email headers).
3. You drop it at the post office (send it to the SMTP server).
4. The post office routes it to the recipient's local post office (SMTP server routes to recipient's email server).
5. The recipient picks up the letter from their mailbox (they read the email).

```
+----------+       +-----------+       +------------+       +----------+
|   Your   | SMTP  |   Gmail   |       | Recipient  |       |  User's  |
|   App    |------>|   Server  |------>|   Email    |------>|  Inbox   |
| (Sender) |       | (Post     |       |   Server   |       | (Reader) |
|          |       |  Office)  |       | (Delivery) |       |          |
+----------+       +-----------+       +------------+       +----------+

   Step 1:           Step 2:            Step 3:            Step 4:
   Compose           Route              Deliver            Read
   & Send            Email              Email              Email
```

What Is SMTP?

SMTP stands for Simple Mail Transfer Protocol. It is the standard way computers send emails to each other.

Think of SMTP as the postal service's rule book. It defines:

- How to address the envelope (headers like To, From, Subject).
- How to hand the letter to the post office (connect to port 587).
- How to prove you are who you say you are (authentication).
- How to seal the envelope so nobody reads it in transit (TLS encryption).

SMTP Term	Postal Analogy	Description
SMTP Server	Post Office	The server that accepts and routes emails
Port 587	Mail Slot	The door where you drop off the email

TLS	Sealed Envelope	Encryption to protect the email content
Authentication	ID Check	Proving you are allowed to send from this address

---

25.2 Adding Email Support to Spring Boot

Spring Boot makes sending emails easy with the `spring-boot-starter-mail` dependency.

Step 1: Add the Dependency

Open your `pom.xml` and add:

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

This single dependency brings in everything you need:

- JavaMail API (the standard Java email library).
- Spring's email abstraction layer.
- Auto-configuration for mail settings.

Step 2: What You Get Automatically

When Spring Boot sees this dependency, it automatically:

1. Creates a `JavaMailSender` bean.
2. Reads your SMTP configuration from `application.properties`.
3. Sets up connection pooling for efficiency.

```
+----------------------+       +-------------------+       +-------------+
| spring-boot-starter |       | Auto-             |       | JavaMail    |
| -mail (pom.xml)     |------>| Configuration     |------>| Sender Bean |
|                     |       | reads properties  |       | (Ready!)    |
+----------------------+       +-------------------+       +-------------+
                                        |
                                 +------+------+
```

```
                                | application |
                                | .properties |
                                +-------------+
```

25.3 Configuring SMTP for Gmail

Let us configure our application to send emails through Gmail. Gmail is free and perfect for development and small applications.

Step 1: Create an App Password in Gmail

Gmail does not allow regular passwords for third-party apps. You need an App Password.

1. Go to your Google Account settings.
2. Navigate to Security.
3. Enable 2-Step Verification (if not already enabled).
4. Go to App passwords.
5. Select "Mail" and "Other (Custom name)".
6. Enter "Spring Boot App" as the name.
7. Click Generate.
8. Copy the 16-character password.

> Important: An App Password is a special password that lets your Spring Boot app access Gmail without needing your main Google password. Never share it or commit it to version control.

Step 2: Configure application.properties

```properties
# application.properties

# SMTP Server Settings
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your.email@gmail.com
spring.mail.password=your-app-password

# TLS Encryption Settings
spring.mail.properties.mail.smtp.auth=true
```

```
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.starttls.required=true

# Connection Timeout Settings (in milliseconds)
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000
```

Let us break down each setting:

Property	Value	What It Does
`spring.mail.host`	`smtp.gmail.com`	Gmail's SMTP server address
`spring.mail.port`	`587`	Port for TLS-encrypted email
`spring.mail.username`	Your Gmail address	The "from" email address
`spring.mail.password`	App Password	The 16-character app password
`mail.smtp.auth`	`true`	Enable authentication
`mail.smtp.starttls.enable`	`true`	Enable TLS encryption
`mail.smtp.starttls.required`	`true`	Require TLS (reject unencrypted)

> Security Warning: Never hardcode your password in `application.properties` for production. Use environment variables instead. We will cover this in Chapter 29 (Externalized Configuration).

Using Environment Variables (Recommended)

```properties
# application.properties (safer approach)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.starttls.required=true
```

Set the environment variables before running your application:

```bash
export MAIL_USERNAME=your.email@gmail.com
export MAIL_PASSWORD=your-app-password
```

25.4 Sending Simple Text Emails

Now let us send our first email. We will start with a simple plain-text email.

Creating an Email Service

```java
// src/main/java/com/example/emaildemo/service/EmailService.java

package com.example.emaildemo.service;

import org.springframework.mail.SimpleMailMessage;       // 1
import org.springframework.mail.javamail.JavaMailSender;  // 2
import org.springframework.stereotype.Service;            // 3

@Service                                                  // 4
public class EmailService {

    private final JavaMailSender mailSender;               // 5

    public EmailService(JavaMailSender mailSender) {       // 6
        this.mailSender = mailSender;
    }

    public void sendSimpleEmail(String to,                 // 7
                                String subject,
                                String body) {

        SimpleMailMessage message = new SimpleMailMessage(); // 8
        message.setFrom("your.email@gmail.com");             // 9
        message.setTo(to);                                   // 10
        message.setSubject(subject);                         // 11
        message.setText(body);                               // 12

        mailSender.send(message);                            // 13

        System.out.println("Email sent to: " + to);         // 14
    }
}
```

Line-by-line explanation:

- Line 1: Import `SimpleMailMessage`, a class for plain-text emails.

- Line 2: Import `JavaMailSender`, the main interface for sending emails.
- Line 3: Import the `@Service` annotation.
- Line 4: `@Service` tells Spring this is a service component. Spring creates and manages it.
- Line 5: Declare the `JavaMailSender` field. Spring Boot auto-configured this for us.
- Line 6: Constructor injection. Spring gives us the `JavaMailSender` automatically.
- Line 7: Our method takes three parameters: recipient, subject, and body.
- Line 8: Create a new `SimpleMailMessage` object. Think of this as creating a new letter.
- Line 9: Set the sender's email address (the "From" field).
- Line 10: Set the recipient's email address (the "To" field).
- Line 11: Set the email subject line.
- Line 12: Set the email body text.
- Line 13: Send the email. This connects to Gmail's SMTP server and delivers the message.
- Line 14: Print a confirmation message to the console.

Creating a Controller to Trigger Emails

```java
// src/main/java/com/example/emaildemo/controller/EmailController.java

package com.example.emaildemo.controller;

import com.example.emaildemo.service.EmailService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/emails")
public class EmailController {

    private final EmailService emailService;

    public EmailController(EmailService emailService) {
        this.emailService = emailService;
    }

    @PostMapping("/send")
    public ResponseEntity<Map<String, String>> sendEmail(
            @RequestParam String to,
            @RequestParam String subject,
            @RequestParam String body) {

        emailService.sendSimpleEmail(to, subject, body);
```

```
        return ResponseEntity.ok(
            Map.of("message", "Email sent successfully",
                   "to", to)
        );
    }
}
```

Testing with curl

```bash
curl -X POST "http://localhost:8080/api/emails/send?\
to=friend@example.com&\
subject=Hello%20from%20Spring%20Boot&\
body=This%20is%20my%20first%20email%20from%20Spring%20Boot!"
```

Output:

```json
{
  "message": "Email sent successfully",
  "to": "friend@example.com"
}
```

Console Output:

```
Email sent to: friend@example.com
```

Sending to Multiple Recipients

```java
public void sendToMultiple(String[] recipients,
                           String subject,
                           String body) {

    SimpleMailMessage message = new SimpleMailMessage();
    message.setFrom("your.email@gmail.com");
    message.setTo(recipients);           // Pass an array of addresses
    message.setSubject(subject);
    message.setText(body);

    mailSender.send(message);
}
```

Adding CC and BCC

```java
public void sendWithCcBcc(String to, String subject, String body) {

    SimpleMailMessage message = new SimpleMailMessage();
    message.setFrom("your.email@gmail.com");
    message.setTo(to);
    message.setCc("manager@example.com");         // CC: visible copy
    message.setBcc("archive@example.com");        // BCC: hidden copy
    message.setSubject(subject);
    message.setText(body);

    mailSender.send(message);
}
```

25.5 Sending HTML Emails with MimeMessage

Plain text emails are functional, but they look boring. Real applications send HTML emails with formatting, images, and buttons. Think about emails you get from Amazon or Netflix. They look like mini web pages. That is because they are HTML.

HTML Email vs Plain Text Email

```
Plain Text Email:                    HTML Email:
+-------------------------+         +----------------------------+
| Hello John,             |         | +----------------------+  |
|                         |         | |    WELCOME, JOHN!    |  |
| Welcome to our app.     |         | +----------------------+  |
| Click this link to      |         |                            |
| verify: http://...      |         | Welcome to our app.        |
|                         |         |                            |
| Thanks,                 |         | +----------------------+  |
| The Team                |         | | VERIFY YOUR EMAIL    |  |
+-------------------------+         | +----------------------+  |
                                    |                            |
                                    | Thanks, The Team           |
                                    +----------------------------+
```

Creating an HTML Email Service

```java
// src/main/java/com/example/emaildemo/service/HtmlEmailService.java
```

```
package com.example.emaildemo.service;

import jakarta.mail.MessagingException;                          // 1
import jakarta.mail.internet.MimeMessage;                        // 2
import org.springframework.mail.javamail.JavaMailSender;         // 3
import org.springframework.mail.javamail.MimeMessageHelper;      // 4
import org.springframework.stereotype.Service;

@Service
public class HtmlEmailService {

    private final JavaMailSender mailSender;

    public HtmlEmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    public void sendHtmlEmail(String to,                         // 5
                              String subject,
                              String htmlContent)
                              throws MessagingException {        // 6

        MimeMessage message = mailSender.createMimeMessage();    // 7

        MimeMessageHelper helper =
            new MimeMessageHelper(message, true, "UTF-8");       // 8

        helper.setFrom("your.email@gmail.com");                  // 9
        helper.setTo(to);                                        // 10
        helper.setSubject(subject);                              // 11
        helper.setText(htmlContent, true);                       // 12

        mailSender.send(message);                                // 13
    }
}
```

Line-by-line explanation:

- Line 1: Import `MessagingException` from Jakarta Mail (the checked exception for mail errors).
- Line 2: Import `MimeMessage`, which supports HTML content, attachments, and more.
- Line 3: Import `JavaMailSender` to send the message.
- Line 4: Import `MimeMessageHelper`, a Spring utility that makes working with `MimeMessage` easier.
- Line 5: Method accepts recipient, subject, and HTML content as strings.
- Line 6: Declare that this method might throw a `MessagingException`.
- Line 7: Create a new `MimeMessage` using the mail sender. Unlike `SimpleMailMessage`, this supports rich content.

- Line 8: Create a `MimeMessageHelper`. The `true` parameter enables multipart mode (needed for HTML and attachments). `"UTF-8"` sets the character encoding.
- Line 9-11: Set from, to, and subject just like before.
- Line 12: Set the email body. The `true` parameter tells Spring this is HTML, not plain text.
- Line 13: Send the email.

Sending an HTML Email

```java
// In your controller or wherever you want to trigger the email

String html = """
    <html>
    <body style="font-family: Arial, sans-serif;">
        <div style="max-width: 600px; margin: 0 auto;
                    padding: 20px; background-color: #f9f9f9;">

            <h1 style="color: #333;">Welcome to Our App!</h1>

            <p>Hello <strong>John</strong>,</p>

            <p>Thank you for signing up. We are excited to
               have you on board.</p>

            <a href="https://example.com/verify?token=abc123"
               style="display: inline-block; padding: 12px 24px;
                      background-color: #007bff; color: white;
                      text-decoration: none; border-radius: 4px;">
                Verify Your Email
            </a>

            <p style="color: #666; margin-top: 20px;">
                If you did not create this account, please
                ignore this email.
            </p>

        </div>
    </body>
    </html>
    """;

htmlEmailService.sendHtmlEmail(
    "john@example.com",
    "Welcome to Our App!",
    html
);
```

Sending Emails with Attachments

```java
public void sendEmailWithAttachment(String to,
                                    String subject,
                                    String htmlContent,
                                    String attachmentPath)
                                    throws MessagingException {

    MimeMessage message = mailSender.createMimeMessage();

    MimeMessageHelper helper =
        new MimeMessageHelper(message, true, "UTF-8");    // true = multipart

    helper.setFrom("your.email@gmail.com");
    helper.setTo(to);
    helper.setSubject(subject);
    helper.setText(htmlContent, true);

    // Add the attachment
    FileSystemResource file =
        new FileSystemResource(new File(attachmentPath));

    helper.addAttachment(
        file.getFilename(),   // Name shown in the email
        file                  // The actual file
    );

    mailSender.send(message);
}
```

Embedding Images in HTML Emails

Sometimes you want images to appear inside the email body, not as attachments.

```java
public void sendEmailWithInlineImage(String to,
                                     String subject)
                                     throws MessagingException {

    MimeMessage message = mailSender.createMimeMessage();

    MimeMessageHelper helper =
        new MimeMessageHelper(message, true, "UTF-8");

    helper.setFrom("your.email@gmail.com");
    helper.setTo(to);
    helper.setSubject(subject);

    // Reference the image with cid:logoImage
    String html = """
        <html>
        <body>
            <img src="cid:logoImage" width="200" />
```

```
            <h1>Welcome!</h1>
            <p>Thanks for joining us.</p>
        </body>
        </html>
        """;

    helper.setText(html, true);

    // Add the inline image
    ClassPathResource image =
        new ClassPathResource("static/images/logo.png");

    helper.addInline("logoImage", image);   // ID must match cid:

    mailSender.send(message);
}
```

```
How Inline Images Work:
+-----------------------+
|  HTML Email           |
|  <img src="cid:logo"> |---+
|                       |   |    +------------+
|  Welcome!             |   +--->| logo.png   |
|  Thanks for joining.  |       | (embedded)  |
+-----------------------+       +------------+

The image travels WITH the email, not loaded from a URL.
```

25.6 Email Templates with Thymeleaf

Hardcoding HTML strings in Java is messy and hard to maintain. Imagine changing a button color and having to dig through Java files. That is like keeping your recipes written on napkins instead of in a cookbook.

Thymeleaf is a template engine that lets you create HTML templates as separate files. You fill in the dynamic parts (like the user's name) at runtime.

Step 1: Add Thymeleaf Dependency

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Step 2: Create an Email Template

```html
<!-- src/main/resources/templates/email/welcome.html -->

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
</head>
<body style="font-family: Arial, sans-serif;
             background-color: #f4f4f4;
             margin: 0; padding: 20px;">

    <div style="max-width: 600px; margin: 0 auto;
                background-color: #ffffff;
                border-radius: 8px; padding: 30px;">

        <!-- Dynamic greeting -->
        <h1 style="color: #2c3e50;">
            Welcome, <span th:text="${userName}">User</span>!
        </h1>

        <p style="color: #555; line-height: 1.6;">
            Thank you for joining
            <strong th:text="${appName}">Our App</strong>.
            We are glad to have you!
        </p>

        <!-- Dynamic verification link -->
        <div style="text-align: center; margin: 30px 0;">
            <a th:href="${verificationUrl}"
               style="display: inline-block;
                      padding: 14px 28px;
                      background-color: #3498db;
                      color: #ffffff;
                      text-decoration: none;
                      border-radius: 6px;
                      font-size: 16px;">
                Verify Your Email
            </a>
        </div>

        <p style="color: #999; font-size: 12px;">
            This link expires in
            <span th:text="${expirationHours}">24</span> hours.
        </p>
```

```
        <hr style="border: 1px solid #eee;"/>

        <p style="color: #999; font-size: 12px;">
            If you did not create an account, ignore this email.
        </p>
    </div>

</body>
</html>
```

Step 3: Create a Template Email Service

```java
// src/main/java/com/example/emaildemo/service/TemplateEmailService.java

package com.example.emaildemo.service;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;                            // 1
import org.thymeleaf.context.Context;                           // 2

@Service
public class TemplateEmailService {

    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;                // 3

    public TemplateEmailService(JavaMailSender mailSender,
                                TemplateEngine templateEngine) {
        this.mailSender = mailSender;
        this.templateEngine = templateEngine;                   // 4
    }

    public void sendWelcomeEmail(String to,                     // 5
                                 String userName)
                                 throws MessagingException {

        // Create the Thymeleaf context with variables
        Context context = new Context();                        // 6
        context.setVariable("userName", userName);              // 7
        context.setVariable("appName", "BookStore");            // 8
        context.setVariable("verificationUrl",                  // 9
            "https://example.com/verify?email=" + to);
        context.setVariable("expirationHours", 24);             // 10

        // Process the template into an HTML string
        String htmlContent = templateEngine.process(            // 11
```

```
            "email/welcome",                                    // 12
            context                                             // 13
        );

        // Send the HTML email
        MimeMessage message = mailSender.createMimeMessage();
        MimeMessageHelper helper =
            new MimeMessageHelper(message, true, "UTF-8");

        helper.setFrom("your.email@gmail.com");
        helper.setTo(to);
        helper.setSubject("Welcome to BookStore!");
        helper.setText(htmlContent, true);                      // 14

        mailSender.send(message);
    }
}
```

Line-by-line explanation:

- Line 1: Import Thymeleaf's `TemplateEngine`. This processes HTML templates.
- Line 2: Import `Context`, which holds the variables to inject into the template.
- Line 3: Declare the `TemplateEngine` field.
- Line 4: Spring injects the `TemplateEngine` automatically (thanks to the Thymeleaf starter).
- Line 5: Method takes the recipient email and user name.
- Line 6: Create a new Thymeleaf context. Think of this as a bag of variables.
- Line 7: Put the `userName` variable in the bag. In the template, `${userName}` will be replaced.
- Line 8: Set the app name. `${appName}` in the template becomes "BookStore".
- Line 9: Set the verification URL.
- Line 10: Set the expiration hours.
- Line 11: Process the template. This takes the HTML file and replaces all `${...}` placeholders.
- Line 12: The template path is relative to `src/main/resources/templates/`.
- Line 13: Pass the context with all our variables.
- Line 14: Set the processed HTML as the email body.

```
How Thymeleaf Email Templates Work:

+------------------+      +------------------+      +------------------+
| Template File    |      | Thymeleaf        |      | Final HTML       |
| (welcome.html)   |      | Engine           |      | (sent as email)  |
|                  |      |                  |      |                  |
| Welcome,         |      | Replaces:        |      | Welcome,         |
| ${userName}!     |----->| ${userName}      |----->| John!            |
```

```
|                  |      | -> "John"        |      |                  |
| ${appName}       |      | ${appName}       |      | BookStore        |
|                  |      | -> "BookStore"   |      |                  |
+------------------+      +------------------+      +------------------+
```

Creating Multiple Templates

You can create different templates for different email types:

```
src/main/resources/templates/email/
    welcome.html              <- Welcome email
    password-reset.html       <- Password reset email
    order-confirmation.html   <- Order confirmation
    newsletter.html           <- Weekly newsletter
```

Here is a password reset template:

```html
<!-- src/main/resources/templates/email/password-reset.html -->

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
</head>
<body style="font-family: Arial, sans-serif;
             background-color: #f4f4f4;
             margin: 0; padding: 20px;">

    <div style="max-width: 600px; margin: 0 auto;
                background-color: #ffffff;
                border-radius: 8px; padding: 30px;">

        <h1 style="color: #e74c3c;">Password Reset</h1>

        <p>Hello <span th:text="${userName}">User</span>,</p>

        <p>We received a request to reset your password.
           Click the button below to create a new password.</p>

        <div style="text-align: center; margin: 30px 0;">
            <a th:href="${resetUrl}"
               style="display: inline-block;
                      padding: 14px 28px;
                      background-color: #e74c3c;
                      color: #ffffff;
                      text-decoration: none;
                      border-radius: 6px;">
                Reset Password
```

```
            </a>
        </div>

        <p style="color: #999;">
            This link expires in
            <span th:text="${expirationMinutes}">30</span>
            minutes.
        </p>

        <p style="color: #999;">
            If you did not request this, ignore this email.
            Your password will remain unchanged.
        </p>
    </div>

</body>
</html>
```

25.7 Sending Emails Asynchronously with @Async

Sending an email takes time. Your application connects to Gmail's server, authenticates, sends the data, and waits for confirmation. This can take 1 to 5 seconds.

If you send emails synchronously (the default), your user has to wait:

```
Synchronous (User Waits):

User clicks       Email is         Response sent
"Register"        being sent       to user
    |                |                  |
    v                v                  v
----[=====WAIT=====][===SENDING===]---->
         500ms           3000ms

Total: 3500ms  (User waits 3.5 seconds!)
```

With `@Async`, the email is sent in a background thread:

```
Asynchronous (User Does NOT Wait):

User clicks       Response sent
"Register"        to user (fast!)
    |                |
    v                v
----[=====WAIT=====]---->
```

```
         500ms
                   Email sent in background
                   [===SENDING===]
                        3000ms
                   (User does not notice)

Total for user: 500ms  (Much better!)
```

Step 1: Enable Async Support

```java
// src/main/java/com/example/emaildemo/EmailDemoApplication.java

package com.example.emaildemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;    // 1

@SpringBootApplication
@EnableAsync                                                     // 2
public class EmailDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmailDemoApplication.class, args);
    }
}
```

- Line 1: Import the `@EnableAsync` annotation.
- Line 2: `@EnableAsync` tells Spring to enable asynchronous method execution. Without this, `@Async` does nothing.

Step 2: Configure the Thread Pool (Optional but Recommended)

```java
// src/main/java/com/example/emaildemo/config/AsyncConfig.java

package com.example.emaildemo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@Configuration
@EnableAsync
```

```
public class AsyncConfig {

    @Bean(name = "emailExecutor")
    public Executor emailExecutor() {
        ThreadPoolTaskExecutor executor =
            new ThreadPoolTaskExecutor();

        executor.setCorePoolSize(2);         // Minimum 2 threads
        executor.setMaxPoolSize(5);          // Maximum 5 threads
        executor.setQueueCapacity(100);      // Queue up to 100 emails
        executor.setThreadNamePrefix("email-");
        executor.initialize();

        return executor;
    }
}
```

Think of the thread pool like a team of mail carriers:

- Core Pool Size (2): You always have 2 mail carriers working.
- Max Pool Size (5): If things get busy, you can hire up to 5.
- Queue Capacity (100): If all carriers are busy, up to 100 letters can wait in line.

Step 3: Make the Email Method Async

```java
// Updated EmailService with @Async

@Service
public class EmailService {

    private final JavaMailSender mailSender;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Async("emailExecutor")                                     // 1
    public void sendSimpleEmail(String to,
                                String subject,
                                String body) {

        try {
            SimpleMailMessage message = new SimpleMailMessage();
            message.setFrom("your.email@gmail.com");
            message.setTo(to);
            message.setSubject(subject);
            message.setText(body);

            mailSender.send(message);
```

```
            System.out.println(
                Thread.currentThread().getName()               // 2
                + " - Email sent to: " + to
            );

        } catch (Exception e) {                                // 3
            System.err.println(
                "Failed to send email to " + to
                + ": " + e.getMessage()
            );
        }
    }
}
```

Line-by-line explanation:

- Line 1: `@Async("emailExecutor")` tells Spring to run this method in a separate thread from the "emailExecutor" pool. The caller will not wait for it to finish.
- Line 2: Print the thread name to verify it runs on a background thread (e.g., "email-1").
- Line 3: Important! Since this runs in the background, exceptions will not propagate to the caller. You must catch and handle them here.

Console Output:

```
email-1 - Email sent to: john@example.com
email-2 - Email sent to: jane@example.com
```

Notice the thread names (`email-1`, `email-2`) instead of the main thread.

> Important Rule: `@Async` only works when the method is called from a different class. If you call an `@Async` method from within the same class, it runs synchronously. This is because Spring uses proxies to intercept the call, and self-calls bypass the proxy.

```
Works (different class calls async method):
+------------------+       +------------------+
| EmailController  |------>| EmailService     |
| (calls method)   |       | @Async method    |
+------------------+       | (runs in bg)     |
                           +------------------+

Does NOT work (same class calls async method):
+------------------+
| EmailService     |
| method A() ------+---> @Async method B()
| (same class!)    |     (runs synchronously!)
+------------------+
```

25.8 Building an Email Verification Flow

Now let us put everything together and build a real email verification system. This is the flow you see when you sign up for almost any website.

The Complete Flow

```
+--------+    +----------+    +----------+    +---------+    +--------+
|  User  |    |  Spring  |    |  Save    |    |  Send   |    |  User  |
|  Signs |    |  Boot    |    |  Token   |    |  Email  |    |  Clicks|
|  Up    |--->|  App     |--->|  to DB   |--->|  with   |--->|  Link  |
|        |    |          |    |          |    |  Link   |    |        |
+--------+    +----------+    +----------+    +---------+    +---+----+
                                                                 |
                                                                 v
+--------+    +----------+    +----------+    +---------+        |
|  User  |    |  Mark    |    |  Verify  |    |  User   |        |
|  Can   |<---|  User as |<---|  Token   |<---|  Hits   |<-------+
|  Login |    |  Verified|    |  Valid?  |    |  /verify|
+--------+    +----------+    +----------+    +---------+
```

Step 1: Create the User Entity

```java
// src/main/java/com/example/emaildemo/entity/User.java

package com.example.emaildemo.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;
```

```
    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private boolean emailVerified = false;           // 1

    @Column(unique = true)
    private String verificationToken;                // 2

    private LocalDateTime tokenExpiryDate;           // 3

    // Constructors
    public User() {}

    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) {
        this.password = password;
    }

    public boolean isEmailVerified() { return emailVerified; }
    public void setEmailVerified(boolean emailVerified) {
        this.emailVerified = emailVerified;
    }

    public String getVerificationToken() {
        return verificationToken;
    }
    public void setVerificationToken(String verificationToken) {
        this.verificationToken = verificationToken;
    }

    public LocalDateTime getTokenExpiryDate() {
        return tokenExpiryDate;
    }
    public void setTokenExpiryDate(LocalDateTime tokenExpiryDate) {
        this.tokenExpiryDate = tokenExpiryDate;
    }
}
```

- Line 1: Tracks whether the user has verified their email. Defaults to `false`.
- Line 2: A unique random token sent in the verification email.
- Line 3: When the token expires (e.g., 24 hours after registration).

Step 2: Create the Repository

```java
// src/main/java/com/example/emaildemo/repository/UserRepository.java

package com.example.emaildemo.repository;

import com.example.emaildemo.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    Optional<User> findByVerificationToken(String token);
}
```

Step 3: Create the Verification Email Service

```java
// src/main/java/com/example/emaildemo/service/VerificationEmailService.java

package com.example.emaildemo.service;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

@Service
public class VerificationEmailService {

    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;

    public VerificationEmailService(JavaMailSender mailSender,
                                    TemplateEngine templateEngine) {
        this.mailSender = mailSender;
```

```
        this.templateEngine = templateEngine;
    }

    @Async("emailExecutor")
    public void sendVerificationEmail(String to,
                                      String name,
                                      String token) {
        try {
            Context context = new Context();
            context.setVariable("userName", name);
            context.setVariable("verificationUrl",
                "http://localhost:8080/api/auth/verify?token="
                + token);
            context.setVariable("appName", "BookStore");
            context.setVariable("expirationHours", 24);

            String html = templateEngine.process(
                "email/welcome", context);

            MimeMessage message = mailSender.createMimeMessage();
            MimeMessageHelper helper =
                new MimeMessageHelper(message, true, "UTF-8");

            helper.setFrom("your.email@gmail.com");
            helper.setTo(to);
            helper.setSubject("Verify Your Email - BookStore");
            helper.setText(html, true);

            mailSender.send(message);

            System.out.println("Verification email sent to: " + to);

        } catch (MessagingException e) {
            System.err.println(
                "Failed to send verification email: "
                + e.getMessage()
            );
        }
    }
}
```

Step 4: Create the Registration Service

```java
// src/main/java/com/example/emaildemo/service/RegistrationService.java

package com.example.emaildemo.service;

import com.example.emaildemo.entity.User;
import com.example.emaildemo.repository.UserRepository;
import org.springframework.stereotype.Service;
```

```
import java.time.LocalDateTime;
import java.util.UUID;

@Service
public class RegistrationService {

    private final UserRepository userRepository;
    private final VerificationEmailService emailService;

    public RegistrationService(
            UserRepository userRepository,
            VerificationEmailService emailService) {
        this.userRepository = userRepository;
        this.emailService = emailService;
    }

    public User registerUser(String name,
                             String email,
                             String password) {

        // Check if email already exists
        if (userRepository.findByEmail(email).isPresent()) {      // 1
            throw new RuntimeException(
                "Email already registered: " + email);
        }

        // Create the user
        User user = new User(name, email, password);              // 2

        // Generate a verification token
        String token = UUID.randomUUID().toString();              // 3
        user.setVerificationToken(token);
        user.setTokenExpiryDate(
            LocalDateTime.now().plusHours(24));                    // 4

        // Save to database
        User savedUser = userRepository.save(user);               // 5

        // Send verification email (async - does not block)
        emailService.sendVerificationEmail(                       // 6
            email, name, token);

        return savedUser;
    }

    public String verifyEmail(String token) {

        // Find user by token
        User user = userRepository                                // 7
            .findByVerificationToken(token)
            .orElseThrow(() -> new RuntimeException(
                "Invalid verification token"));

        // Check if token has expired
```

```
        if (user.getTokenExpiryDate()
                .isBefore(LocalDateTime.now())) {                // 8
            throw new RuntimeException(
                "Verification token has expired");
        }

        // Mark user as verified
        user.setEmailVerified(true);                             // 9
        user.setVerificationToken(null);                         // 10
        user.setTokenExpiryDate(null);
        userRepository.save(user);

        return "Email verified successfully!";
    }
}
```

Key lines explained:

- Line 1: Check for duplicate emails before registering.
- Line 2: Create a new user (not yet verified).
- Line 3: Generate a random UUID token. Example: `"a3b2c1d4-e5f6-7890-abcd-ef1234567890"`.
- Line 4: Token expires in 24 hours.
- Line 5: Save the user to the database.
- Line 6: Send verification email asynchronously. The method returns immediately.
- Line 7: When the user clicks the link, find them by the token.
- Line 8: Check if the token has expired.
- Line 9: Mark the email as verified.
- Line 10: Clear the token so it cannot be reused.

Step 5: Create the Auth Controller

```java
// src/main/java/com/example/emaildemo/controller/AuthController.java

package com.example.emaildemo.controller;

import com.example.emaildemo.entity.User;
import com.example.emaildemo.service.RegistrationService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
```

```
public class AuthController {

    private final RegistrationService registrationService;

    public AuthController(RegistrationService registrationService) {
        this.registrationService = registrationService;
    }

    @PostMapping("/register")
    public ResponseEntity<Map<String, String>> register(
            @RequestParam String name,
            @RequestParam String email,
            @RequestParam String password) {

        User user = registrationService.registerUser(
            name, email, password);

        return ResponseEntity.ok(Map.of(
            "message", "Registration successful! "
                     + "Check your email for verification.",
            "email", user.getEmail()
        ));
    }

    @GetMapping("/verify")
    public ResponseEntity<Map<String, String>> verify(
            @RequestParam String token) {

        String result = registrationService.verifyEmail(token);

        return ResponseEntity.ok(Map.of("message", result));
    }
}
```

Testing the Complete Flow

Step 1: Register a new user:

```bash
curl -X POST "http://localhost:8080/api/auth/register?\
name=John&\
email=john@example.com&\
password=secret123"
```

Output:

```json
{
  "message": "Registration successful! Check your email for verification.",
  "email": "john@example.com"
}
```

Step 2: User receives an email with a verification link.

Step 3: User clicks the verification link:

```bash
curl "http://localhost:8080/api/auth/verify?\
token=a3b2c1d4-e5f6-7890-abcd-ef1234567890"
```

Output:

```json
{
  "message": "Email verified successfully!"
}
```

25.9 Error Handling for Emails

Emails can fail for many reasons. The SMTP server might be down. The recipient address might be invalid. Your credentials might be wrong. Good applications handle these failures gracefully.

Creating a Custom Email Exception

```java
// src/main/java/com/example/emaildemo/exception/EmailSendException.java

package com.example.emaildemo.exception;

public class EmailSendException extends RuntimeException {

    public EmailSendException(String message) {
        super(message);
    }

    public EmailSendException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Adding Retry Logic

```java
// Improved EmailService with retry logic

@Service
public class EmailService {

    private final JavaMailSender mailSender;
    private static final int MAX_RETRIES = 3;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Async("emailExecutor")
    public void sendEmailWithRetry(String to,
                                   String subject,
                                   String body) {

        int attempts = 0;

        while (attempts < MAX_RETRIES) {
            try {
                SimpleMailMessage message =
                    new SimpleMailMessage();
                message.setFrom("your.email@gmail.com");
                message.setTo(to);
                message.setSubject(subject);
                message.setText(body);

                mailSender.send(message);

                System.out.println("Email sent to: " + to
                    + " (attempt " + (attempts + 1) + ")");
                return;  // Success! Exit the method.

            } catch (Exception e) {
                attempts++;
                System.err.println(
                    "Attempt " + attempts + " failed for "
                    + to + ": " + e.getMessage()
                );

                if (attempts >= MAX_RETRIES) {
                    System.err.println(
                        "All " + MAX_RETRIES
                        + " attempts failed for " + to
                    );
                    // Log to monitoring system, save to
                    // database for manual retry, etc.
                }

                try {
                    // Wait before retrying
                    // (exponential backoff)
                    Thread.sleep(
```

```
                        1000L * attempts);  // 1s, 2s, 3s
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }
}
```

Common Mistakes

Mistake 1: Committing Email Credentials to Git

```properties
# WRONG: Never commit real credentials
spring.mail.password=myActualPassword123
```

```properties
# CORRECT: Use environment variables
spring.mail.password=${MAIL_PASSWORD}
```

Mistake 2: Sending Emails Synchronously in API Endpoints

```java
// WRONG: User waits for email to send
@PostMapping("/register")
public ResponseEntity<String> register(...) {
    userService.save(user);
    emailService.sendEmail(user.getEmail(), ...);  // Blocks!
    return ResponseEntity.ok("Done");
}
```

```java
// CORRECT: Use @Async so user does not wait
@Async("emailExecutor")
public void sendEmail(String to, ...) {
    // Runs in background thread
}
```

Mistake 3: Calling @Async from the Same Class

```java
// WRONG: @Async is ignored when called from same class
@Service
public class EmailService {

    public void sendWelcome(String email) {
        sendEmail(email, "Welcome", "Hello!");  // NOT async!
    }

    @Async
    public void sendEmail(String to, String subject, String body) {
        // This runs synchronously because it is called
        // from the same class!
    }
}
```

```java
// CORRECT: Call @Async method from a different class
@Service
public class RegistrationService {

    private final EmailService emailService;  // Different class

    public void register(String email) {
        emailService.sendEmail(email, "Welcome", "Hello!");
        // Now it runs asynchronously!
    }
}
```

Mistake 4: Not Handling Email Failures

```java
// WRONG: Exception crashes the background thread silently
@Async
public void sendEmail(String to, String subject, String body) {
    mailSender.send(message);  // If this fails, nobody knows
}
```

```java
// CORRECT: Catch and handle exceptions
@Async
public void sendEmail(String to, String subject, String body) {
    try {
        mailSender.send(message);
    } catch (Exception e) {
        log.error("Email failed for {}: {}", to, e.getMessage());
        // Save to a retry queue or alert monitoring
    }
}
```

Mistake 5: Using Port 25 Instead of 587

```properties
# WRONG: Port 25 is unencrypted and often blocked
spring.mail.port=25
```

```properties
# CORRECT: Port 587 uses TLS encryption
spring.mail.port=587
```

Best Practices

1. Always use environment variables for credentials. Never hardcode email passwords in your source code or configuration files.
2. Send emails asynchronously. Users should never wait for emails to be sent. Use `@Async` with a dedicated thread pool.
3. Use Thymeleaf templates for HTML emails. Keep your HTML separate from your Java code. It is easier to maintain and designers can edit templates without touching Java.
4. Implement retry logic. SMTP servers can have temporary failures. Retry 2 to 3 times with exponential backoff before giving up.
5. Set timeouts on SMTP connections. Without timeouts, a hung SMTP server can block your thread forever. Always configure `connectiontimeout`, `timeout`, and `writetimeout`.
6. Log email failures. Since async emails run in background threads, failures can go unnoticed. Always log errors and consider alerting your monitoring system.
7. Use a dedicated email service in production. Gmail has sending limits (500 emails per day). For production, use services like SendGrid, Amazon SES, or Mailgun.
8. Validate email addresses before sending. Use a simple regex or a library to check that the email format is valid before attempting to send.

Quick Summary

In this chapter, you learned how to send emails from a Spring Boot application. You started with the basics of SMTP and configured Gmail as your email provider. You sent plain text emails using `SimpleMailMessage` and rich HTML emails using `MimeMessage`. You used Thymeleaf templates to separate your email designs from your Java code. You made email

sending asynchronous with `@Async` so users do not have to wait. Finally, you built a complete email verification flow that real applications use.

Key Points

Concept	Description
`spring-boot-starter-mail`	Adds email support to your Spring Boot application
SMTP	The protocol used to send emails (port 587 with TLS)
`JavaMailSender`	Spring's main interface for sending emails
`SimpleMailMessage`	For plain text emails
`MimeMessage`	For HTML emails, attachments, and inline images
`MimeMessageHelper`	Utility class that simplifies working with MimeMessage
Thymeleaf Templates	HTML files with placeholders for dynamic content
`@Async`	Runs a method in a background thread
`@EnableAsync`	Enables async processing in the application
Verification Token	A random UUID sent in the email to verify ownership
App Password	A special Gmail password for third-party applications

---

Practice Questions

1. What is the difference between `SimpleMailMessage` and `MimeMessage`? When would you use each one?
2. Why should you send emails asynchronously instead of synchronously? What happens to the user experience if you send emails synchronously?
3. What does the `@Async` annotation do, and why does it not work when you call the annotated method from the same class?
4. Explain the purpose of each SMTP configuration property: `spring.mail.host`, `spring.mail.port`, `spring.mail.properties.mail.smtp.starttls.enable`.

5. In the email verification flow, why do we set an expiration time on the verification token? What security problem would occur without it?

Exercises

Exercise 1: Order Confirmation Email

Create a Thymeleaf email template for an order confirmation. The template should display:

- Customer name
- Order number
- A list of ordered items with quantities and prices
- The total price
- An estimated delivery date

Create a service that populates this template and sends it.

Exercise 2: Password Reset Flow

Build a complete password reset flow:

1. User submits their email to `/api/auth/forgot-password`.
2. The app generates a token, saves it, and sends a reset email.
3. User clicks the link and submits a new password to `/api/auth/reset-password`.
4. The app verifies the token, updates the password, and invalidates the token.

Make sure the token expires after 30 minutes.

Exercise 3: Email Sending Dashboard

Create a REST endpoint that tracks all sent emails. Store each sent email's recipient, subject, status (sent/failed), timestamp, and number of retry attempts in the H2 database. Create endpoints to:

- GET `/api/emails/history` - List all sent emails with pagination.
- GET `/api/emails/stats` - Return counts of sent and failed emails.

What Is Next?

You can now send emails from your Spring Boot application. But how do you know your code actually works? How do you make sure it keeps working as you add new features? In the next chapter, we will learn about Testing. You will discover how to write unit tests, integration tests, and API tests that give you confidence your application works correctly. Testing is not optional in professional development. It is what separates hobby projects from production-ready applications.

Chapter 26: Testing

What You Will Learn

- What the test pyramid is and why it matters.
- How to write unit tests with JUnit 5 (@Test, assertions).
- How to use Mockito to mock dependencies (@Mock, when().thenReturn()).
- How to test REST controllers with @WebMvcTest and MockMvc.
- How to write full integration tests with @SpringBootTest.
- How to test database operations with @DataJpaTest.
- How to replace beans in tests with @MockBean.
- How to configure test-specific properties.
- How to write complete CRUD API tests from start to finish.

Why This Chapter Matters

Imagine you are a bridge builder. You build a beautiful bridge and open it to traffic. Cars drive across it every day. Then one day, you need to add a lane. You modify the bridge, and it collapses. People get hurt.

Now imagine you had a testing process. Before opening the bridge, you tested it with heavy trucks. Before adding the lane, you tested the modified design. You would have caught the weakness before anyone got hurt.

Software testing works the same way. Every time you change your code, there is a risk of breaking something. Tests are your safety net. They catch problems before your users do.

Professional developers do not ship code without tests. Companies like Google, Netflix, and Amazon run millions of tests every day. If you want to write production-quality Spring Boot applications, you need to test them.

This chapter teaches you every kind of test you need for a Spring Boot application.

26.1 The Test Pyramid

The test pyramid is a strategy for organizing your tests. It tells you what kinds of tests to write and how many of each.

```
              /\
             /  \
            / E2E\           Few (slow, expensive)
           / Tests\
          /--------\
         /Integration\       Some (medium speed)
        /   Tests     \
       /--------------\
      /   Unit Tests    \    Many (fast, cheap)
     /                   \
    +---------------------+

    More tests at bottom, fewer at top.
    Faster tests at bottom, slower at top.
```

Three Types of Tests

Type	What It Tests	Speed	Example
Unit Tests	A single class or method in isolation	Very fast (milliseconds)	Test that `calculateTotal()` returns the correct sum
Integration Tests	Multiple components working together	Medium (seconds)	Test that a controller saves data to the database
End-to-End (E2E) Tests	The entire application flow	Slow (minutes)	Test the full user registration process

Think of it like testing a car:

- Unit test: Test that each part works (engine starts, brakes grip, lights turn on).
- Integration test: Test that parts work together (engine connects to transmission, brakes work with ABS).
- E2E test: Drive the car on a real road and test everything at once.

Why the Pyramid Shape?

You want many unit tests because they are fast and pinpoint exact problems. You want some integration tests because they verify components work together. You want few E2E tests because they are slow and hard to maintain.

```
Test Type          Count    Speed       Maintenance
--------------------------------------------------
Unit Tests         100+     < 1 sec     Low
Integration Tests   20+     5-30 sec    Medium
E2E Tests            5+     1-5 min     High
```

26.2 Setting Up Your Test Environment

Spring Boot includes testing support out of the box. When you create a project with Spring Initializr, the test dependency is already there.

The Test Dependency

```xml
<!-- pom.xml (already included by Spring Initializr) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

This single dependency includes:

Library	Purpose
JUnit 5	The testing framework (write and run tests)
Mockito	Mocking library (fake dependencies)
AssertJ	Fluent assertions (readable test checks)
Spring Test	Spring-specific test support
MockMvc	Test controllers without a real server
JSONPath	Assert values in JSON responses

Project Structure

```
src/
  main/
    java/
      com/example/bookstore/
        controller/
          BookController.java
        service/
          BookService.java
        repository/
          BookRepository.java
        entity/
          Book.java
  test/
    java/
      com/example/bookstore/
        controller/
          BookControllerTest.java       <- Controller tests
        service/
          BookServiceTest.java          <- Service unit tests
        repository/
          BookRepositoryTest.java       <- Repository tests
        BookstoreApplicationTests.java  <- Integration test
    resources/
      application-test.properties      <- Test configuration
```

Notice that the test directory mirrors the main directory. This is a convention that makes tests easy to find.

Test Configuration

```properties
# src/test/resources/application-test.properties

# Use H2 in-memory database for tests
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# Disable email sending in tests
spring.mail.host=localhost
spring.mail.port=25
```

26.3 JUnit 5 Basics

JUnit 5 is the testing framework for Java. It provides annotations to mark test methods and assertions to verify results.

Your First Test

```java
// src/test/java/com/example/bookstore/CalculatorTest.java

package com.example.bookstore;

import org.junit.jupiter.api.Test;                      // 1
import static org.junit.jupiter.api.Assertions.*;       // 2

class CalculatorTest {                                   // 3

    @Test                                                // 4
    void shouldAddTwoNumbers() {                         // 5
        // Arrange
        int a = 5;                                       // 6
        int b = 3;

        // Act
        int result = a + b;                              // 7

        // Assert
        assertEquals(8, result);                         // 8
    }

    @Test
    void shouldSubtractTwoNumbers() {
        int result = 10 - 4;
        assertEquals(6, result);
    }
}
```

Line-by-line explanation:

- Line 1: Import the `@Test` annotation from JUnit 5.
- Line 2: Import assertion methods like `assertEquals`, `assertTrue`, etc.
- Line 3: Test classes do not need to be `public` in JUnit 5.
- Line 4: `@Test` marks this method as a test. JUnit will run it automatically.
- Line 5: Test method names should describe what they test. Use `should...` naming.
- Line 6: Arrange - Set up the test data.
- Line 7: Act - Perform the action you are testing.
- Line 8: Assert - Check the result. `assertEquals(expected, actual)`.

Output when you run the test:

```
CalculatorTest > shouldAddTwoNumbers()       PASSED
CalculatorTest > shouldSubtractTwoNumbers()  PASSED

2 tests completed, 2 passed
```

The AAA Pattern: Arrange, Act, Assert

Every test follows this pattern:

```
+-----------+     +-----------+     +-----------+
|  ARRANGE  |---->|    ACT    |---->|  ASSERT   |
|           |     |           |     |           |
| Set up    |     | Call the  |     | Check the |
| test data |     | method    |     | result    |
+-----------+     +-----------+     +-----------+
```

Common JUnit 5 Assertions

```java
import static org.junit.jupiter.api.Assertions.*;

class AssertionExamplesTest {

    @Test
    void demonstrateAssertions() {
        // assertEquals: Check two values are equal
        assertEquals(4, 2 + 2);
        assertEquals("hello", "hello");

        // assertNotEquals: Check two values are NOT equal
        assertNotEquals(5, 2 + 2);

        // assertTrue / assertFalse: Check boolean conditions
        assertTrue(10 > 5);
        assertFalse(10 < 5);

        // assertNull / assertNotNull: Check for null
        String name = "John";
        assertNotNull(name);

        String empty = null;
        assertNull(empty);

        // assertThrows: Check that an exception is thrown
        assertThrows(ArithmeticException.class, () -> {
            int result = 10 / 0;
        });
```

```
        // assertEquals with message (shown on failure)
        assertEquals(4, 2 + 2,
            "Basic addition should work");
    }
}
```

JUnit 5 Lifecycle Annotations

```java
import org.junit.jupiter.api.*;

class LifecycleTest {

    @BeforeAll                                  // 1
    static void setUpOnce() {
        System.out.println("Runs ONCE before all tests");
    }

    @BeforeEach                                 // 2
    void setUp() {
        System.out.println("Runs before EACH test");
    }

    @Test
    void testOne() {
        System.out.println("Test 1");
    }

    @Test
    void testTwo() {
        System.out.println("Test 2");
    }

    @AfterEach                                  // 3
    void tearDown() {
        System.out.println("Runs after EACH test");
    }

    @AfterAll                                   // 4
    static void tearDownOnce() {
        System.out.println("Runs ONCE after all tests");
    }
}
```

Output:

```
Runs ONCE before all tests
Runs before EACH test
Test 1
```

```
Runs after EACH test
Runs before EACH test
Test 2
Runs after EACH test
Runs ONCE after all tests
```

- Line 1: `@BeforeAll` runs once before any test. Must be `static`.
- Line 2: `@BeforeEach` runs before each individual test. Great for resetting state.
- Line 3: `@AfterEach` runs after each test. Good for cleanup.
- Line 4: `@AfterAll` runs once after all tests. Must be `static`.

@DisplayName for Readable Test Names

```java
@DisplayName("Book Price Calculator")
class BookPriceCalculatorTest {

    @Test
    @DisplayName("should apply 10% discount for orders over $100")
    void shouldApplyDiscount() {
        double price = 150.0;
        double discounted = price * 0.9;
        assertEquals(135.0, discounted);
    }

    @Test
    @DisplayName("should not apply discount for orders under $100")
    void shouldNotApplyDiscount() {
        double price = 80.0;
        assertEquals(80.0, price);
    }
}
```

Output:

```
Book Price Calculator
  should apply 10% discount for orders over $100    PASSED
  should not apply discount for orders under $100   PASSED
```

26.4 Unit Testing with Mockito

In a real application, classes depend on other classes. Your `BookService` depends on `BookRepository`. Your `OrderService` depends on `BookService` and `EmailService`.

When you unit test `BookService`, you do not want to use a real database. You want to test `BookService` in isolation. This is where Mockito comes in.

Mockito creates fake (mock) objects that pretend to be your dependencies.

```
Without Mocking:                    With Mocking:

+-------------+   +----------+     +-------------+   +----------+
| BookService |-->| Real     |     | BookService |-->| Mock     |
|   (test)    |   | Database |     |   (test)    |   | (fake)   |
+-------------+   +----------+     +-------------+   +----------+
                  Slow!                               Fast!
                  Needs DB setup!                     No DB needed!
                  Can fail for                        Predictable!
                  DB reasons!
```

The Book Entity and Service

Let us create the classes we will test:

```java
// src/main/java/com/example/bookstore/entity/Book.java

package com.example.bookstore.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false)
    private double price;

    public Book() {}

    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
```

```
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

```java
// src/main/java/com/example/bookstore/repository/BookRepository.java

package com.example.bookstore.repository;

import com.example.bookstore.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface BookRepository extends JpaRepository<Book, Long> {
    List<Book> findByAuthor(String author);
}
```

```java
// src/main/java/com/example/bookstore/service/BookService.java

package com.example.bookstore.service;

import com.example.bookstore.entity.Book;
import com.example.bookstore.repository.BookRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public Book createBook(Book book) {
        if (book.getPrice() < 0) {
            throw new IllegalArgumentException(
                "Price cannot be negative");
        }
```

```
        return bookRepository.save(book);
    }

    public Book getBookById(Long id) {
        return bookRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Book not found with id: " + id));
    }

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    public Book updateBook(Long id, Book updatedBook) {
        Book existing = getBookById(id);
        existing.setTitle(updatedBook.getTitle());
        existing.setAuthor(updatedBook.getAuthor());
        existing.setPrice(updatedBook.getPrice());
        return bookRepository.save(existing);
    }

    public void deleteBook(Long id) {
        if (!bookRepository.existsById(id)) {
            throw new RuntimeException(
                "Book not found with id: " + id);
        }
        bookRepository.deleteById(id);
    }

    public List<Book> getBooksByAuthor(String author) {
        return bookRepository.findByAuthor(author);
    }
}
```

Writing Unit Tests for BookService

```java
// src/test/java/com/example/bookstore/service/BookServiceTest.java

package com.example.bookstore.service;

import com.example.bookstore.entity.Book;
import com.example.bookstore.repository.BookRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;                          // 1
import org.mockito.Mock;                                 // 2
import org.mockito.junit.jupiter.MockitoExtension;      // 3
```

```
import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;           // 4
import static org.mockito.Mockito.*;                      // 5

@ExtendWith(MockitoExtension.class)                       // 6
@DisplayName("BookService Unit Tests")
class BookServiceTest {

    @Mock                                                  // 7
    private BookRepository bookRepository;

    @InjectMocks                                           // 8
    private BookService bookService;

    private Book sampleBook;

    @BeforeEach                                            // 9
    void setUp() {
        sampleBook = new Book("Spring Boot in Action",
                              "Craig Walls", 49.99);
        sampleBook.setId(1L);
    }

    // ---- CREATE Tests ----

    @Test
    @DisplayName("should create a book successfully")
    void shouldCreateBook() {
        // Arrange
        when(bookRepository.save(any(Book.class)))         // 10
            .thenReturn(sampleBook);                       // 11

        // Act
        Book created = bookService.createBook(sampleBook); // 12

        // Assert
        assertNotNull(created);                            // 13
        assertEquals("Spring Boot in Action",
                     created.getTitle());
        assertEquals(49.99, created.getPrice());

        verify(bookRepository, times(1))                   // 14
            .save(any(Book.class));
    }

    @Test
    @DisplayName("should throw exception for negative price")
    void shouldThrowExceptionForNegativePrice() {
        // Arrange
        Book invalidBook =
            new Book("Bad Book", "Author", -10.0);
```

```
        // Act & Assert
        assertThrows(IllegalArgumentException.class, () -> {
            bookService.createBook(invalidBook);
        });

        verify(bookRepository, never()).save(any());       // 15
    }

    // ---- READ Tests ----

    @Test
    @DisplayName("should return book when found by ID")
    void shouldReturnBookById() {
        // Arrange
        when(bookRepository.findById(1L))
            .thenReturn(Optional.of(sampleBook));

        // Act
        Book found = bookService.getBookById(1L);

        // Assert
        assertNotNull(found);
        assertEquals("Spring Boot in Action", found.getTitle());
    }

    @Test
    @DisplayName("should throw exception when book not found")
    void shouldThrowWhenBookNotFound() {
        // Arrange
        when(bookRepository.findById(99L))
            .thenReturn(Optional.empty());

        // Act & Assert
        RuntimeException exception =
            assertThrows(RuntimeException.class, () -> {
                bookService.getBookById(99L);
            });

        assertEquals("Book not found with id: 99",
                     exception.getMessage());
    }

    @Test
    @DisplayName("should return all books")
    void shouldReturnAllBooks() {
        // Arrange
        Book book2 = new Book("Clean Code",
                              "Robert Martin", 39.99);
        List<Book> books = Arrays.asList(sampleBook, book2);

        when(bookRepository.findAll()).thenReturn(books);

        // Act
        List<Book> result = bookService.getAllBooks();
```

```
        // Assert
        assertEquals(2, result.size());
        verify(bookRepository, times(1)).findAll();
    }

    // ---- UPDATE Tests ----

    @Test
    @DisplayName("should update book successfully")
    void shouldUpdateBook() {
        // Arrange
        Book updatedData = new Book("Updated Title",
                                    "New Author", 59.99);

        when(bookRepository.findById(1L))
            .thenReturn(Optional.of(sampleBook));
        when(bookRepository.save(any(Book.class)))
            .thenReturn(sampleBook);

        // Act
        Book updated = bookService.updateBook(1L, updatedData);

        // Assert
        assertNotNull(updated);
        verify(bookRepository).findById(1L);
        verify(bookRepository).save(any(Book.class));
    }

    // ---- DELETE Tests ----

    @Test
    @DisplayName("should delete book successfully")
    void shouldDeleteBook() {
        // Arrange
        when(bookRepository.existsById(1L)).thenReturn(true);

        // Act
        bookService.deleteBook(1L);

        // Assert
        verify(bookRepository, times(1)).deleteById(1L);
    }

    @Test
    @DisplayName("should throw exception when deleting non-existent book")
    void shouldThrowWhenDeletingNonExistentBook() {
        // Arrange
        when(bookRepository.existsById(99L)).thenReturn(false);

        // Act & Assert
        assertThrows(RuntimeException.class, () -> {
            bookService.deleteBook(99L);
        });
```

```
        verify(bookRepository, never()).deleteById(any());
    }
}
```

Key Mockito concepts explained:

- Line 6: `@ExtendWith(MockitoExtension.class)` enables Mockito in JUnit 5 tests.
- Line 7: `@Mock` creates a fake `BookRepository`. It does not connect to any database.
- Line 8: `@InjectMocks` creates a real `BookService` and injects the mock repository into it.
- Line 9: `@BeforeEach` creates fresh test data before each test.
- Line 10: `when(bookRepository.save(any(Book.class)))` means "when someone calls `save` with any Book..."
- Line 11: `.thenReturn(sampleBook)` means "...then return this sample book."
- Line 12: Now when `bookService.createBook()` internally calls `bookRepository.save()`, it gets our mock response.
- Line 13: Assert the result is what we expect.
- Line 14: `verify(bookRepository, times(1)).save(any())` confirms that `save` was called exactly once.
- Line 15: `verify(bookRepository, never()).save(any())` confirms that `save` was never called (because validation failed).

```
How Mockito Works:

Step 1: Create mocks
+-------------+     +-----------+
| @Mock       |---->| Fake Repo |  (no real database)
| bookRepo    |     | (mockito) |
+-------------+     +-----------+

Step 2: Define behavior
when(mockRepo.save(any()))  -->  return sampleBook

Step 3: Test the service
+-------------+     +-----------+
| BookService |---->| Fake Repo |
| (real code) |     | returns   |
|             |     | sampleBook|
+-------------+     +-----------+

Step 4: Verify calls
verify(mockRepo).save(any())  -->  Was save() called? YES!
```

Output:

```
BookService Unit Tests
  should create a book successfully                        PASSED
  should throw exception for negative price                PASSED
  should return book when found by ID                      PASSED
  should throw exception when book not found               PASSED
  should return all books                                  PASSED
  should update book successfully                          PASSED
  should delete book successfully                          PASSED
  should throw exception when deleting non-existent book  PASSED

8 tests completed, 8 passed
```

26.5 Testing REST Controllers with @WebMvcTest

`@WebMvcTest` tests your controllers without starting the full Spring application. It only loads the web layer (controllers, filters, advice), making tests fast.

The Book Controller

```java
// src/main/java/com/example/bookstore/controller/BookController.java

package com.example.bookstore.controller;

import com.example.bookstore.entity.Book;
import com.example.bookstore.service.BookService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        Book created = bookService.createBook(book);
        return ResponseEntity.status(HttpStatus.CREATED)
```

```
                                .body(created);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBook(@PathVariable Long id) {
        Book book = bookService.getBookById(id);
        return ResponseEntity.ok(book);
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(
            @PathVariable Long id,
            @RequestBody Book book) {
        Book updated = bookService.updateBook(id, book);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.noContent().build();
    }
}
```

Writing Controller Tests

```java
// src/test/java/com/example/bookstore/controller/BookControllerTest.java

package com.example.bookstore.controller;

import com.example.bookstore.entity.Book;
import com.example.bookstore.service.BookService;
import com.fasterxml.jackson.databind.ObjectMapper;          // 1
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .WebMvcTest;                                             // 2
import org.springframework.boot.test.mock.mockito.MockBean;  // 3
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;          // 4

import java.util.Arrays;
import java.util.List;
```

```
import static org.hamcrest.Matchers.*;                        // 5
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.doNothing;
import static org.springframework.test.web.servlet
    .request.MockMvcRequestBuilders.*;                       // 6
import static org.springframework.test.web.servlet
    .result.MockMvcResultMatchers.*;                         // 7

@WebMvcTest(BookController.class)                             // 8
@DisplayName("BookController Tests")
class BookControllerTest {

    @Autowired
    private MockMvc mockMvc;                                  // 9

    @MockBean                                                 // 10
    private BookService bookService;

    @Autowired
    private ObjectMapper objectMapper;                        // 11

    // ---- POST /api/books ----

    @Test
    @DisplayName("POST /api/books - should create book")
    void shouldCreateBook() throws Exception {

        // Arrange
        Book book = new Book("Spring Boot Guide",
                             "John Doe", 39.99);
        book.setId(1L);

        when(bookService.createBook(any(Book.class)))
            .thenReturn(book);

        // Act & Assert
        mockMvc.perform(                                      // 12
            post("/api/books")                                // 13
                .contentType(MediaType.APPLICATION_JSON)      // 14
                .content(objectMapper                         // 15
                    .writeValueAsString(book))
        )
        .andExpect(status().isCreated())                      // 16
        .andExpect(jsonPath("$.id").value(1))                 // 17
        .andExpect(jsonPath("$.title")                        // 18
            .value("Spring Boot Guide"))
        .andExpect(jsonPath("$.author")
            .value("John Doe"))
        .andExpect(jsonPath("$.price").value(39.99));
    }

    // ---- GET /api/books/{id} ----
```

```
    @Test
    @DisplayName("GET /api/books/1 - should return book")
    void shouldReturnBookById() throws Exception {

        Book book = new Book("Clean Code",
                             "Robert Martin", 44.99);
        book.setId(1L);

        when(bookService.getBookById(1L)).thenReturn(book);

        mockMvc.perform(
            get("/api/books/1")                              // 19
        )
        .andExpect(status().isOk())                          // 20
        .andExpect(jsonPath("$.title").value("Clean Code"))
        .andExpect(jsonPath("$.author")
            .value("Robert Martin"));
    }

    // ---- GET /api/books ----

    @Test
    @DisplayName("GET /api/books - should return all books")
    void shouldReturnAllBooks() throws Exception {

        Book book1 = new Book("Book 1", "Author 1", 29.99);
        book1.setId(1L);
        Book book2 = new Book("Book 2", "Author 2", 39.99);
        book2.setId(2L);
        List<Book> books = Arrays.asList(book1, book2);

        when(bookService.getAllBooks()).thenReturn(books);

        mockMvc.perform(
            get("/api/books")
        )
        .andExpect(status().isOk())
        .andExpect(jsonPath("$", hasSize(2)))                // 21
        .andExpect(jsonPath("$[0].title")                    // 22
            .value("Book 1"))
        .andExpect(jsonPath("$[1].title")
            .value("Book 2"));
    }

    // ---- PUT /api/books/{id} ----

    @Test
    @DisplayName("PUT /api/books/1 - should update book")
    void shouldUpdateBook() throws Exception {

        Book updatedBook = new Book("Updated Title",
                                    "Updated Author", 59.99);
        updatedBook.setId(1L);
```

```
        when(bookService.updateBook(eq(1L), any(Book.class)))
            .thenReturn(updatedBook);

        mockMvc.perform(
            put("/api/books/1")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(updatedBook))
        )
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.title")
            .value("Updated Title"))
        .andExpect(jsonPath("$.price").value(59.99));
    }

    // ---- DELETE /api/books/{id} ----

    @Test
    @DisplayName("DELETE /api/books/1 - should delete book")
    void shouldDeleteBook() throws Exception {

        doNothing().when(bookService).deleteBook(1L);         // 23

        mockMvc.perform(
            delete("/api/books/1")                            // 24
        )
        .andExpect(status().isNoContent());                   // 25

        verify(bookService).deleteBook(1L);
    }
}
```

Key lines explained:

- Line 2/8: `@WebMvcTest(BookController.class)` starts only the web layer for `BookController`. No database, no services. Very fast.
- Line 3/10: `@MockBean` creates a mock `BookService` and adds it to the Spring context. The controller will use this mock instead of the real service.
- Line 4/9: `MockMvc` lets you send fake HTTP requests to your controller without starting a real server.
- Line 11: `ObjectMapper` converts Java objects to JSON strings.
- Line 12: `mockMvc.perform(...)` sends a fake HTTP request.
- Line 13: `post("/api/books")` creates a POST request to `/api/books`.
- Line 14: Set the content type to JSON.
- Line 15: Convert the book object to a JSON string for the request body.
- Line 16: `status().isCreated()` checks that the response status is 201 Created.
- Line 17: `jsonPath("$.id").value(1)` checks that the JSON response has `"id": 1`.
- Line 18: `jsonPath("$.title")` checks the title field in the JSON response.

- Line 19: `get("/api/books/1")` creates a GET request.
- Line 20: `status().isOk()` checks for 200 OK.
- Line 21: `hasSize(2)` verifies the JSON array has 2 elements.
- Line 22: `$[0].title` accesses the first element's title in the JSON array.
- Line 23: `doNothing()` is used for void methods. "When deleteBook is called, do nothing."
- Line 24: `delete("/api/books/1")` creates a DELETE request.
- Line 25: `status().isNoContent()` checks for 204 No Content.

```
How @WebMvcTest Works:

+----------+     +----------+     +----------+
|  Test    |     | MockMvc  |     | Book     |
|  Class   |---->| (fake    |---->| Controller|
|          |     | server)  |     | (real)   |
+----------+     +----------+     +----+-----+
                                       |
                                  +----+-----+
                                  | @MockBean |
                                  | BookService|
                                  | (fake)    |
                                  +-----------+

What is loaded:     Controllers, Filters, Exception Handlers
What is NOT loaded: Services, Repositories, Database
```

Output:

```
BookController Tests
  POST /api/books - should create book           PASSED
  GET /api/books/1 - should return book           PASSED
  GET /api/books - should return all books        PASSED
  PUT /api/books/1 - should update book           PASSED
  DELETE /api/books/1 - should delete book        PASSED

5 tests completed, 5 passed
```

26.6 Integration Testing with @SpringBootTest

`@SpringBootTest` loads the complete Spring application context. It starts everything: controllers, services, repositories, and the database. This tests that all the pieces work together.

```java
// src/test/java/com/example/bookstore/BookstoreIntegrationTest.java

package com.example.bookstore;

import com.example.bookstore.entity.Book;
import com.example.bookstore.repository.BookRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .AutoConfigureMockMvc;                                  // 1
import org.springframework.boot.test.context
    .SpringBootTest;                                        // 2
import org.springframework.http.MediaType;
import org.springframework.test.context
    .ActiveProfiles;                                        // 3
import org.springframework.test.web.servlet.MockMvc;

import static org.hamcrest.Matchers.*;
import static org.springframework.test.web.servlet
    .request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet
    .result.MockMvcResultMatchers.*;

@SpringBootTest                                             // 4
@AutoConfigureMockMvc                                       // 5
@ActiveProfiles("test")                                     // 6
@DisplayName("Bookstore Integration Tests")
class BookstoreIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private BookRepository bookRepository;                  // 7

    @Autowired
    private ObjectMapper objectMapper;

    @BeforeEach
    void setUp() {
        bookRepository.deleteAll();                         // 8
    }

    @Test
    @DisplayName("Full CRUD flow - create, read, update, delete")
    void shouldPerformFullCrudFlow() throws Exception {

        // Step 1: Create a book
        Book book = new Book("Test Book",
                             "Test Author", 29.99);

        String response = mockMvc.perform(
            post("/api/books")
```

```
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(book))
        )
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").exists())                // 9
        .andExpect(jsonPath("$.title").value("Test Book"))
        .andReturn()                                         // 10
        .getResponse()
        .getContentAsString();

        // Extract the ID from the response
        Long bookId = objectMapper.readTree(response)        // 11
            .get("id").asLong();

        // Step 2: Read the book
        mockMvc.perform(get("/api/books/" + bookId))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.title")
                .value("Test Book"));

        // Step 3: Update the book
        Book updated = new Book("Updated Book",
                                "New Author", 39.99);

        mockMvc.perform(
            put("/api/books/" + bookId)
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(updated))
        )
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.title")
            .value("Updated Book"))
        .andExpect(jsonPath("$.price").value(39.99));

        // Step 4: Verify the update persisted
        mockMvc.perform(get("/api/books/" + bookId))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.title")
                .value("Updated Book"));

        // Step 5: Delete the book
        mockMvc.perform(delete("/api/books/" + bookId))
            .andExpect(status().isNoContent());

        // Step 6: Verify deletion
        mockMvc.perform(get("/api/books"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(0)));
    }

    @Test
    @DisplayName("should return empty list when no books exist")
    void shouldReturnEmptyList() throws Exception {
```

```
        mockMvc.perform(get("/api/books"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(0)));
    }

    @Test
    @DisplayName("should create multiple books and list them")
    void shouldCreateAndListMultipleBooks() throws Exception {

        // Create two books
        Book book1 = new Book("Book One", "Author A", 19.99);
        Book book2 = new Book("Book Two", "Author B", 29.99);

        mockMvc.perform(
            post("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(book1))
        ).andExpect(status().isCreated());

        mockMvc.perform(
            post("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(book2))
        ).andExpect(status().isCreated());

        // List all books
        mockMvc.perform(get("/api/books"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(2)))
            .andExpect(jsonPath("$[0].title")
                .value("Book One"))
            .andExpect(jsonPath("$[1].title")
                .value("Book Two"));
    }
}
```

Key differences from @WebMvcTest:

- Line 2/4: `@SpringBootTest` loads the entire application (all beans, database, etc.).
- Line 1/5: `@AutoConfigureMockMvc` provides MockMvc for sending HTTP requests.
- Line 3/6: `@ActiveProfiles("test")` activates the "test" profile, loading `application-test.properties`.
- Line 7: We can inject the real `BookRepository` (no `@MockBean`).
- Line 8: Clean the database before each test for isolation.
- Line 9: `jsonPath("$.id").exists()` checks that the database generated an ID.
- Line 10: `.andReturn()` captures the response so we can extract data from it.
- Line 11: Extract the generated ID to use in subsequent requests.

```
@WebMvcTest vs @SpringBootTest:

@WebMvcTest                          @SpringBootTest
+----------+                         +----------+
| Controller| (real)                 | Controller| (real)
+-----+----+                         +-----+----+
      |                                    |
+-----+----+                         +-----+----+
| Service  | (@MockBean = fake)       | Service  | (real)
+----------+                         +-----+----+
                                           |
                                     +-----+----+
                                     | Repository| (real)
                                     +-----+----+
                                           |
                                     +-----+----+
                                     | H2 DB   | (real)
                                     +----------+

Faster, isolated                     Slower, full integration
```

26.7 Testing Repositories with @DataJpaTest

`@DataJpaTest` tests only the database layer. It configures an in-memory H2 database, scans for `@Entity` classes, and configures Spring Data JPA repositories.

```java
// src/test/java/com/example/bookstore/repository/BookRepositoryTest.java

package com.example.bookstore.repository;

import com.example.bookstore.entity.Book;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa
    .DataJpaTest;                                          // 1

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest                                                // 2
@DisplayName("BookRepository Tests")
class BookRepositoryTest {
```

```
    @Autowired
    private BookRepository bookRepository;                  // 3

    @BeforeEach
    void setUp() {
        bookRepository.deleteAll();
    }

    @Test
    @DisplayName("should save and find a book by ID")
    void shouldSaveAndFindBook() {
        // Arrange
        Book book = new Book("JPA in Action",
                             "John Doe", 34.99);

        // Act
        Book saved = bookRepository.save(book);             // 4
        Optional<Book> found =
            bookRepository.findById(saved.getId());         // 5

        // Assert
        assertTrue(found.isPresent());                      // 6
        assertEquals("JPA in Action",
                     found.get().getTitle());
    }

    @Test
    @DisplayName("should find books by author")
    void shouldFindBooksByAuthor() {
        // Arrange
        bookRepository.save(
            new Book("Book 1", "Craig Walls", 29.99));
        bookRepository.save(
            new Book("Book 2", "Craig Walls", 39.99));
        bookRepository.save(
            new Book("Book 3", "Other Author", 49.99));

        // Act
        List<Book> books =
            bookRepository.findByAuthor("Craig Walls");     // 7

        // Assert
        assertEquals(2, books.size());
        assertTrue(books.stream()
            .allMatch(b ->
                b.getAuthor().equals("Craig Walls")));
    }

    @Test
    @DisplayName("should return empty when book not found")
    void shouldReturnEmptyForNonExistentBook() {
        Optional<Book> result =
            bookRepository.findById(999L);
```

```
        assertTrue(result.isEmpty());
    }

    @Test
    @DisplayName("should delete a book")
    void shouldDeleteBook() {
        // Arrange
        Book book = bookRepository.save(
            new Book("To Delete", "Author", 19.99));

        // Act
        bookRepository.deleteById(book.getId());

        // Assert
        Optional<Book> result =
            bookRepository.findById(book.getId());
        assertTrue(result.isEmpty());
    }

    @Test
    @DisplayName("should count books")
    void shouldCountBooks() {
        bookRepository.save(
            new Book("Book 1", "Author 1", 10.0));
        bookRepository.save(
            new Book("Book 2", "Author 2", 20.0));

        long count = bookRepository.count();

        assertEquals(2, count);
    }
}
```

- Line 1-2: `@DataJpaTest` configures only the JPA layer: entities, repositories, and an H2 database. No controllers or services.
- Line 3: Inject the real repository. It connects to an in-memory H2 database.
- Line 4: `save()` inserts the book into the H2 database.
- Line 5: `findById()` queries the H2 database.
- Line 6: Assert the book was found.
- Line 7: Test the custom query method `findByAuthor()`.

```
What @DataJpaTest Loads:

+--------------------+
|   @DataJpaTest     |
|                    |
|  +------------+    |
|  | Entities   |    |    NOT loaded:
|  +------------+    |    - Controllers
|  | Repositories|   |    - Services
```

```
|  +------------+    |    - Security
|  | H2 Database|    |    - Web Layer
|  +------------+    |
+--------------------+
```

Output:

```
BookRepository Tests
  should save and find a book by ID           PASSED
  should find books by author                 PASSED
  should return empty when book not found     PASSED
  should delete a book                        PASSED
  should count books                          PASSED

5 tests completed, 5 passed
```

26.8 Using Test Properties

You often need different settings for tests. Maybe you want a different database, or you want to disable certain features.

Option 1: application-test.properties

```properties
# src/test/resources/application-test.properties

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false

# Disable features that interfere with tests
spring.mail.host=localhost
spring.cache.type=none
```

Activate with:

```java
@SpringBootTest
@ActiveProfiles("test")
class MyTest { ... }
```

Option 2: @TestPropertySource

```java
@SpringBootTest
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:customdb",
    "app.feature.email-enabled=false"
})
class MyTest { ... }
```

Option 3: application.properties in test/resources

```properties
# src/test/resources/application.properties
# This automatically overrides main application.properties

spring.datasource.url=jdbc:h2:mem:testdb
```

Priority Order

```
Highest Priority:
  1. @TestPropertySource (in the test class)
  2. src/test/resources/application-test.properties
  3. src/test/resources/application.properties
  4. src/main/resources/application.properties
Lowest Priority
```

26.9 Complete CRUD API Test Suite

Here is a complete, production-ready test suite that tests all CRUD operations with various scenarios:

```java
// src/test/java/com/example/bookstore/BookCrudApiTest.java

package com.example.bookstore;

import com.example.bookstore.entity.Book;
import com.example.bookstore.repository.BookRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
```

```
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;

import static org.hamcrest.Matchers.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.springframework.test.web.servlet
    .request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet
    .result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@DisplayName("Book CRUD API Tests")
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class BookCrudApiTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private BookRepository bookRepository;
    @Autowired private ObjectMapper objectMapper;

    @BeforeEach
    void cleanDatabase() {
        bookRepository.deleteAll();
    }

    // ======= Helper Methods =======

    private Book createTestBook(String title,
                                String author,
                                double price) {
        return new Book(title, author, price);
    }

    private Long createBookAndReturnId(Book book)
            throws Exception {

        MvcResult result = mockMvc.perform(
            post("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(book))
        )
        .andExpect(status().isCreated())
        .andReturn();

        return objectMapper
            .readTree(result.getResponse()
```

```
                .getContentAsString())
            .get("id").asLong();
    }

    // ======= CREATE Tests =======

    @Test
    @Order(1)
    @DisplayName("CREATE: should create a new book")
    void shouldCreateNewBook() throws Exception {
        Book book = createTestBook(
            "Effective Java", "Joshua Bloch", 45.99);

        mockMvc.perform(
            post("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(book))
        )
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").isNumber())
        .andExpect(jsonPath("$.title")
            .value("Effective Java"))
        .andExpect(jsonPath("$.author")
            .value("Joshua Bloch"))
        .andExpect(jsonPath("$.price").value(45.99));

        // Verify it was saved to the database
        assertEquals(1, bookRepository.count());
    }

    @Test
    @Order(2)
    @DisplayName("CREATE: should persist book in database")
    void shouldPersistBookInDatabase() throws Exception {
        Book book = createTestBook(
            "Clean Code", "Robert Martin", 39.99);

        Long id = createBookAndReturnId(book);

        // Verify directly in the database
        Book saved = bookRepository.findById(id)
            .orElseThrow();
        assertEquals("Clean Code", saved.getTitle());
        assertEquals("Robert Martin", saved.getAuthor());
        assertEquals(39.99, saved.getPrice());
    }

    // ======= READ Tests =======

    @Test
    @Order(3)
    @DisplayName("READ: should return book by ID")
    void shouldReturnBookById() throws Exception {
        Book book = createTestBook(
```

```
            "Head First Java", "Kathy Sierra", 49.99);
        Long id = createBookAndReturnId(book);

        mockMvc.perform(get("/api/books/" + id))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(id))
            .andExpect(jsonPath("$.title")
                .value("Head First Java"));
    }

    @Test
    @Order(4)
    @DisplayName("READ: should return all books")
    void shouldReturnAllBooks() throws Exception {
        createBookAndReturnId(
            createTestBook("Book A", "Author A", 10.0));
        createBookAndReturnId(
            createTestBook("Book B", "Author B", 20.0));
        createBookAndReturnId(
            createTestBook("Book C", "Author C", 30.0));

        mockMvc.perform(get("/api/books"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(3)))
            .andExpect(jsonPath("$[*].title",
                containsInAnyOrder(
                    "Book A", "Book B", "Book C")));
    }

    @Test
    @Order(5)
    @DisplayName("READ: should return empty list when no books")
    void shouldReturnEmptyList() throws Exception {
        mockMvc.perform(get("/api/books"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(0)));
    }

    // ======= UPDATE Tests =======

    @Test
    @Order(6)
    @DisplayName("UPDATE: should update existing book")
    void shouldUpdateExistingBook() throws Exception {
        Long id = createBookAndReturnId(
            createTestBook("Old Title", "Old Author", 10.0));

        Book updated = createTestBook(
            "New Title", "New Author", 99.99);

        mockMvc.perform(
            put("/api/books/" + id)
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(updated))
```

```
        )
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.title").value("New Title"))
        .andExpect(jsonPath("$.author").value("New Author"))
        .andExpect(jsonPath("$.price").value(99.99));

        // Verify the update persisted
        Book fromDb = bookRepository.findById(id)
            .orElseThrow();
        assertEquals("New Title", fromDb.getTitle());
    }

    // ======= DELETE Tests =======

    @Test
    @Order(7)
    @DisplayName("DELETE: should delete existing book")
    void shouldDeleteExistingBook() throws Exception {
        Long id = createBookAndReturnId(
            createTestBook("To Delete", "Author", 10.0));

        // Verify book exists
        assertEquals(1, bookRepository.count());

        // Delete
        mockMvc.perform(delete("/api/books/" + id))
            .andExpect(status().isNoContent());

        // Verify book is gone
        assertEquals(0, bookRepository.count());
        assertTrue(bookRepository.findById(id).isEmpty());
    }

    @Test
    @Order(8)
    @DisplayName("DELETE: should not affect other books")
    void shouldNotAffectOtherBooks() throws Exception {
        Long id1 = createBookAndReturnId(
            createTestBook("Keep This", "Author", 10.0));
        Long id2 = createBookAndReturnId(
            createTestBook("Delete This", "Author", 20.0));

        assertEquals(2, bookRepository.count());

        mockMvc.perform(delete("/api/books/" + id2))
            .andExpect(status().isNoContent());

        assertEquals(1, bookRepository.count());
        assertTrue(bookRepository.findById(id1).isPresent());
        assertTrue(bookRepository.findById(id2).isEmpty());
    }
}
```

Output:

```
Book CRUD API Tests
  CREATE: should create a new book               PASSED
  CREATE: should persist book in database         PASSED
  READ: should return book by ID                  PASSED
  READ: should return all books                   PASSED
  READ: should return empty list when no books    PASSED
  UPDATE: should update existing book             PASSED
  DELETE: should delete existing book             PASSED
  DELETE: should not affect other books            PASSED

8 tests completed, 8 passed
```

26.10 Running Tests

From the Command Line

```bash
# Run all tests
./mvnw test

# Run a specific test class
./mvnw test -Dtest=BookServiceTest

# Run a specific test method
./mvnw test -Dtest=BookServiceTest#shouldCreateBook

# Run tests with verbose output
./mvnw test -Dtest=BookControllerTest -Dsurefire.useFile=false
```

Understanding Test Output

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.bookstore.service.BookServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.example.bookstore.controller.BookControllerTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.example.bookstore.repository.BookRepositoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] Results:
[INFO]
```

```
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

Common Mistakes

Mistake 1: Not Cleaning the Database Between Tests

```java
// WRONG: Tests affect each other
@Test
void testCreateBook() {
    bookRepository.save(new Book("A", "B", 10.0));
    // If this runs first, the next test sees this book
}

@Test
void testEmptyList() {
    // FAILS because previous test left data!
    assertEquals(0, bookRepository.count());
}
```

```java
// CORRECT: Clean before each test
@BeforeEach
void setUp() {
    bookRepository.deleteAll();
}
```

Mistake 2: Using @MockBean When You Need Real Beans

```java
// WRONG: @MockBean replaces the real service
// Your test never actually tests the real service logic
@SpringBootTest
class IntegrationTest {
    @MockBean  // This defeats the purpose of integration testing!
    private BookService bookService;
}
```

```java
// CORRECT: Use real beans in integration tests
@SpringBootTest
class IntegrationTest {
    @Autowired  // Use the real service
    private BookService bookService;
}
```

Mistake 3: Testing Framework Code Instead of Your Code

```java
// WRONG: Testing that Spring Data JPA's save() works
// Spring already tests this. You do not need to.
@Test
void testSave() {
    Book book = new Book("A", "B", 10.0);
    Book saved = bookRepository.save(book);
    assertNotNull(saved.getId());  // This just tests JPA
}
```

```java
// CORRECT: Test YOUR custom logic
@Test
void shouldRejectNegativePrice() {
    Book book = new Book("A", "B", -10.0);
    assertThrows(IllegalArgumentException.class,
        () -> bookService.createBook(book));
}
```

Mistake 4: Using @SpringBootTest for Unit Tests

```java
// WRONG: Loads the entire application just to test one method
@SpringBootTest
class BookServiceTest {
    @Autowired
    private BookService bookService;
    // Slow! Takes seconds to start the entire application.
}
```

```java
// CORRECT: Use Mockito for unit tests (much faster)
@ExtendWith(MockitoExtension.class)
class BookServiceTest {
    @Mock
    private BookRepository bookRepository;
    @InjectMocks
    private BookService bookService;
```

```
    // Fast! No Spring context needed.
}
```

Mistake 5: Not Testing Error Scenarios

```java
// WRONG: Only testing the happy path
@Test
void testGetBook() {
    // Only tests when the book exists
    when(repo.findById(1L)).thenReturn(Optional.of(book));
    Book result = service.getBookById(1L);
    assertNotNull(result);
}
```

```java
// CORRECT: Test both success AND failure paths
@Test
void shouldReturnBookWhenFound() { ... }

@Test
void shouldThrowWhenBookNotFound() {
    when(repo.findById(99L)).thenReturn(Optional.empty());
    assertThrows(RuntimeException.class,
        () -> service.getBookById(99L));
}
```

Best Practices

1. Follow the test pyramid. Write many unit tests, some integration tests, and few end-to-end tests.
2. Use descriptive test names. A test name should explain what is being tested and the expected outcome. Use `@DisplayName` for readability.
3. Follow the AAA pattern. Every test should have three sections: Arrange (set up), Act (execute), Assert (verify).
4. Keep tests independent. Each test should be able to run alone. Clean the database between tests with `@BeforeEach`.
5. Use `@WebMvcTest` for controller tests. It is faster than `@SpringBootTest` because it only loads the web layer.
6. Use `@DataJpaTest` for repository tests. It only loads the JPA layer and is much faster than a full integration test.

7. Test error scenarios. Do not just test the happy path. Test what happens when things go wrong (invalid input, missing data, timeouts).
8. Use `@MockBean` only in slice tests. In integration tests (`@SpringBootTest`), use real beans to verify the full flow.

Quick Summary

In this chapter, you learned how to test Spring Boot applications from top to bottom. You started with the test pyramid, which guides you on how many of each type of test to write. You learned JUnit 5 basics: `@Test`, assertions, and lifecycle annotations. You used Mockito to mock dependencies in unit tests, isolating the class under test. You tested controllers with `@WebMvcTest` and `MockMvc`, sending fake HTTP requests without a real server. You wrote integration tests with `@SpringBootTest` that exercise the full application. You tested repositories with `@DataJpaTest` and an in-memory H2 database. Finally, you built a complete CRUD test suite that covers all operations and edge cases.

Key Points

Concept	Description
Test Pyramid	Strategy: many unit tests, some integration, few E2E
JUnit 5 `@Test`	Marks a method as a test
`assertEquals()`	Checks that expected and actual values match
`assertThrows()`	Checks that a specific exception is thrown
`@BeforeEach`	Runs before each test (setup)
Mockito `@Mock`	Creates a fake object that pretends to be a dependency
`@InjectMocks`	Creates a real object and injects mocks into it
`when().thenReturn()`	Defines what a mock should return when called
`verify()`	Checks that a mock method was called

`@WebMvcTest`	Tests only the web layer (controller tests)
`MockMvc`	Sends fake HTTP requests to controllers
`@MockBean`	Replaces a real bean with a mock in the Spring context
`@SpringBootTest`	Loads the full application for integration tests
`@DataJpaTest`	Tests only the JPA/database layer
`@ActiveProfiles`	Activates a specific configuration profile for tests
`jsonPath()`	Asserts values in JSON responses

Practice Questions

1. What is the test pyramid and why does it recommend more unit tests than integration tests?
2. What is the difference between `@Mock` and `@MockBean`? When would you use each one?
3. Explain the difference between `@WebMvcTest` and `@SpringBootTest`. When is each appropriate?
4. What does the `when().thenReturn()` pattern do in Mockito? Write an example.
5. Why should you clean the database before each test in integration tests? What problems occur if you do not?

Exercises

Exercise 1: Test a Calculator Service

Create a `CalculatorService` with methods for `add`, `subtract`, `multiply`, and `divide`. Write unit tests for all four operations, including a test that verifies `divide` throws an `ArithmeticException` when dividing by zero.

Exercise 2: Test an Employee API

Create an `Employee` entity with fields `id`, `name`, `department`, and `salary`. Build a complete REST API (controller, service, repository) and write:

- Unit tests for the service layer using Mockito.
- Controller tests using `@WebMvcTest`.
- Repository tests using `@DataJpaTest`.
- An integration test that covers the full CRUD flow.

Exercise 3: Test Edge Cases

Add the following edge case tests to the Book CRUD test suite:

- Test creating a book with a very long title (255 characters).
- Test creating a book with an empty title (should fail validation).
- Test updating a book that does not exist (should return 404).
- Test deleting a book that does not exist (should return 404).

What Is Next?

You now know how to test your Spring Boot applications thoroughly. But how do your API consumers know what endpoints are available and how to use them? In the next chapter, we will learn about API Documentation with Swagger and OpenAPI. You will create interactive, auto-generated documentation that lets anyone explore and test your API directly from a browser.

Chapter 27: API Documentation with OpenAPI and Swagger

What You Will Learn

- What API documentation is and why it matters.
- How to add OpenAPI documentation to your Spring Boot app with one dependency.
- How to access the Swagger UI at `/swagger-ui.html`.
- How to describe your endpoints with `@Operation` and `@ApiResponse`.
- How to document your data models with `@Schema`.
- How to group endpoints with `@Tag`.
- How to hide internal endpoints from the documentation.
- How to customize the API info (title, version, description).
- How to test your API directly from the Swagger UI.

Why This Chapter Matters

Imagine you just built a beautiful restaurant. The food is amazing. But there is no menu. Customers walk in, sit down, and have no idea what they can order. They have to keep asking the waiter, "What do you have? How much does it cost? What ingredients are in it?"

That is what it is like when your API has no documentation.

API documentation is your menu. It tells developers:

- What endpoints are available.
- What data to send.
- What data they will get back.
- What errors might occur.

Without documentation, other developers (and even your future self) will waste hours guessing how your API works. With good documentation, they can start using your API in minutes.

In this chapter, you will create interactive documentation that not only describes your API but also lets people test it directly from the browser.

27.1 What Are OpenAPI and Swagger?

These two terms are related but different. Let us clear up the confusion.

OpenAPI is a specification (a set of rules) for describing REST APIs. It defines a standard format for documenting your endpoints, request bodies, responses, and more. Think of it as a universal template for writing API menus.

Swagger is a set of tools that work with the OpenAPI specification. The most popular tool is Swagger UI, which turns your OpenAPI specification into a beautiful, interactive web page.

```
+------------------+       +------------------+       +------------------+
|  Your Spring     |       |  OpenAPI         |       |  Swagger UI      |
|  Boot App        |------>|  Specification   |------>|  (Interactive    |
|  (Java code)     |       |  (JSON/YAML)     |       |   web page)     |
+------------------+       +------------------+       +------------------+

Your code gets           A machine-readable         A human-readable
scanned automatically    description of your API    web page for exploring
                                                    and testing your API
```

Term	What It Is	Analogy
OpenAPI	The specification standard	The template for a menu
OpenAPI Spec	The actual JSON/YAML document	The completed menu
Swagger UI	The interactive web interface	The beautifully printed menu card
springdoc-openapi	The library that generates the spec	The printer that creates the menu

---

27.2 Adding OpenAPI to Your Spring Boot App

Adding API documentation to Spring Boot is incredibly easy. One dependency and you are done.

Step 1: Add the Dependency

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.4</version>
</dependency>
```

That is it. Seriously. Just one dependency.

Step 2: Start Your Application

```bash
./mvnw spring-boot:run
```

Step 3: Open Swagger UI

Open your browser and go to:

```
http://localhost:8080/swagger-ui.html
```

You will see a beautiful interactive page listing all your API endpoints. Spring Boot scanned your controllers and generated the documentation automatically.

What Happened Behind the Scenes?

```
+-------------------+
| Spring Boot Starts |
+--------+----------+
         |
         v
+--------+----------+
| springdoc-openapi  |
| scans your app     |
| for @RestController|
| @GetMapping, etc.  |
+--------+----------+
         |
```

```
         v
+--------+----------+
| Generates OpenAPI  |
| spec at:           |
| /v3/api-docs       |
+--------+----------+
         |
         v
+--------+----------+
| Swagger UI reads   |
| the spec and       |
| renders it at:     |
| /swagger-ui.html   |
+--------------------+
```

Useful URLs

URL	What It Shows
`/swagger-ui.html`	The interactive Swagger UI page
`/v3/api-docs`	The raw OpenAPI spec in JSON format
`/v3/api-docs.yaml`	The raw OpenAPI spec in YAML format

27.3 Customizing API Information

The default documentation shows "OpenAPI definition" as the title. Let us make it more professional.

Creating an OpenAPI Configuration

```java
// src/main/java/com/example/bookstore/config/OpenApiConfig.java

package com.example.bookstore.config;

import io.swagger.v3.oas.models.OpenAPI;                    // 1
import io.swagger.v3.oas.models.info.Contact;                // 2
import io.swagger.v3.oas.models.info.Info;                   // 3
import io.swagger.v3.oas.models.info.License;                // 4
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {                        // 5

        return new OpenAPI()
            .info(new Info()                                // 6
                .title("BookStore API")                     // 7
                .version("1.0.0")                           // 8
                .description(                               // 9
                    "REST API for managing a bookstore. "
                    + "Supports CRUD operations for books, "
                    + "authors, and orders.")
                .contact(new Contact()                      // 10
                    .name("BookStore Team")
                    .email("support@bookstore.com")
                    .url("https://bookstore.com"))
                .license(new License()                      // 11
                    .name("Apache 2.0")
                    .url("https://www.apache.org/licenses/"
                         + "LICENSE-2.0"))
            );
    }
}
```

Line-by-line explanation:

- Line 1: Import the `OpenAPI` class from the Swagger models library.
- Line 2-4: Import classes for contact, info, and license details.
- Line 5: Create a `@Bean` that returns a customized `OpenAPI` object.
- Line 6: Set the API information section.
- Line 7: The title shown at the top of Swagger UI.
- Line 8: The API version number.
- Line 9: A description of what the API does.
- Line 10: Contact information for the API maintainers.
- Line 11: License information.

Now Swagger UI shows your custom title, version, and description instead of the defaults.

Using application.properties for Basic Customization

```properties
# application.properties
```

```
# Change the Swagger UI path (default: /swagger-ui.html)
springdoc.swagger-ui.path=/api-docs

# Change the OpenAPI spec path (default: /v3/api-docs)
springdoc.api-docs.path=/api-spec

# Sort endpoints alphabetically
springdoc.swagger-ui.operationsSorter=alpha

# Expand/collapse sections by default
springdoc.swagger-ui.docExpansion=none

# Disable "Try it out" by default
springdoc.swagger-ui.tryItOutEnabled=false
```

27.4 Documenting Endpoints with @Operation

The auto-generated documentation shows your endpoints, but the descriptions are generic. `@Operation` lets you add detailed descriptions to each endpoint.

Before: Generic Documentation

Without annotations, Swagger shows method names like `getBook` and `createBook` with no explanation. Users have to guess what each endpoint does.

After: Rich Documentation

```java
// src/main/java/com/example/bookstore/controller/BookController.java

package com.example.bookstore.controller;

import com.example.bookstore.entity.Book;
import com.example.bookstore.service.BookService;
import io.swagger.v3.oas.annotations.Operation;             // 1
import io.swagger.v3.oas.annotations.Parameter;             // 2
import io.swagger.v3.oas.annotations.responses.ApiResponse;  // 3
import io.swagger.v3.oas.annotations.responses.ApiResponses; // 4
import io.swagger.v3.oas.annotations.tags.Tag;              // 5
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```
@RestController
@RequestMapping("/api/books")
@Tag(name = "Books",                                        // 6
     description = "Endpoints for managing books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    @Operation(                                              // 7
        summary = "Create a new book",                       // 8
        description = "Creates a new book in the bookstore. "
            + "The book ID is generated automatically."      // 9
    )
    @ApiResponses({                                          // 10
        @ApiResponse(
            responseCode = "201",                            // 11
            description = "Book created successfully"
        ),
        @ApiResponse(
            responseCode = "400",                            // 12
            description = "Invalid book data provided"
        )
    })
    public ResponseEntity<Book> createBook(
            @RequestBody Book book) {

        Book created = bookService.createBook(book);
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(created);
    }

    @GetMapping("/{id}")
    @Operation(
        summary = "Get a book by ID",
        description = "Returns a single book by its unique ID."
    )
    @ApiResponses({
        @ApiResponse(
            responseCode = "200",
            description = "Book found"
        ),
        @ApiResponse(
            responseCode = "404",
            description = "Book not found"
        )
    })
    public ResponseEntity<Book> getBook(
            @Parameter(description = "The book ID",          // 13
                       example = "1")                        // 14
```

```
            @PathVariable Long id) {

        Book book = bookService.getBookById(id);
        return ResponseEntity.ok(book);
    }

    @GetMapping
    @Operation(
        summary = "Get all books",
        description = "Returns a list of all books in the "
            + "bookstore. Returns an empty list if no "
            + "books exist."
    )
    @ApiResponse(
        responseCode = "200",
        description = "List of books retrieved successfully"
    )
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @PutMapping("/{id}")
    @Operation(
        summary = "Update an existing book",
        description = "Updates all fields of an existing book. "
            + "The book must exist or a 404 is returned."
    )
    @ApiResponses({
        @ApiResponse(
            responseCode = "200",
            description = "Book updated successfully"
        ),
        @ApiResponse(
            responseCode = "404",
            description = "Book not found"
        )
    })
    public ResponseEntity<Book> updateBook(
            @Parameter(description = "The book ID to update")
            @PathVariable Long id,
            @RequestBody Book book) {

        Book updated = bookService.updateBook(id, book);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    @Operation(
        summary = "Delete a book",
        description = "Permanently removes a book from "
            + "the bookstore."
    )
    @ApiResponses({
        @ApiResponse(
            responseCode = "204",
```

```
            description = "Book deleted successfully"
        ),
        @ApiResponse(
            responseCode = "404",
            description = "Book not found"
        )
    })
    public ResponseEntity<Void> deleteBook(
            @Parameter(description = "The book ID to delete")
            @PathVariable Long id) {

        bookService.deleteBook(id);
        return ResponseEntity.noContent().build();
    }
}
```

Key annotations explained:

- Line 1/7: `@Operation` describes what a single endpoint does.
- Line 8: `summary` is a short one-line description shown in the endpoint list.
- Line 9: `description` is a longer explanation shown when you expand the endpoint.
- Line 2/13: `@Parameter` describes a path variable or query parameter.
- Line 14: `example` provides a sample value shown in Swagger UI.
- Line 3-4/10: `@ApiResponses` and `@ApiResponse` describe the possible responses.
- Line 11: Response code `"201"` with its description.
- Line 12: Response code `"400"` for error cases.
- Line 5/6: `@Tag` groups related endpoints together (more on this next).

27.5 Documenting Data Models with @Schema

Your API returns and accepts JSON objects. `@Schema` describes the fields in those objects.

```java
// src/main/java/com/example/bookstore/entity/Book.java

package com.example.bookstore.entity;

import io.swagger.v3.oas.annotations.media.Schema;          // 1
import jakarta.persistence.*;

@Entity
@Table(name = "books")
@Schema(description = "Represents a book in the bookstore") // 2
```

```
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(                                                // 3
        description = "Unique identifier of the book",
        example = "1",
        accessMode = Schema.AccessMode.READ_ONLY            // 4
    )
    private Long id;

    @Column(nullable = false)
    @Schema(                                                // 5
        description = "Title of the book",
        example = "Spring Boot in Action",
        requiredMode = Schema.RequiredMode.REQUIRED,         // 6
        maxLength = 255
    )
    private String title;

    @Column(nullable = false)
    @Schema(
        description = "Author of the book",
        example = "Craig Walls",
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    private String author;

    @Column(nullable = false)
    @Schema(
        description = "Price of the book in USD",
        example = "49.99",
        minimum = "0",                                      // 7
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    private double price;

    // Constructors, getters, and setters...
    public Book() {}

    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

Key annotations explained:

- Line 1/3: `@Schema` describes a field in the data model.
- Line 2: `@Schema` on the class describes the entire model.
- Line 4: `accessMode = READ_ONLY` means this field is only in responses, not requests. Users do not send the `id` when creating a book.
- Line 5: Description and example help users understand what value to provide.
- Line 6: `requiredMode = REQUIRED` marks the field as mandatory.
- Line 7: `minimum = "0"` tells users the price must be zero or positive.

Now when users look at the Schemas section of Swagger UI, they see a detailed description of every field with examples.

```
In Swagger UI, the Book schema looks like:

Book {
  id*        integer($int64)    readOnly: true
             Unique identifier of the book
             Example: 1

  title*     string
             Title of the book
             Example: Spring Boot in Action

  author*    string
             Author of the book
             Example: Craig Walls

  price*     number($double)    minimum: 0
             Price of the book in USD
             Example: 49.99
}

* = required
```

27.6 Grouping Endpoints with @Tag

As your API grows, you will have many endpoints. `@Tag` groups related endpoints under a common heading in Swagger UI.

```java
// BookController.java
@RestController
@RequestMapping("/api/books")
@Tag(name = "Books",
     description = "Endpoints for managing books")
public class BookController { ... }

// AuthorController.java
@RestController
@RequestMapping("/api/authors")
@Tag(name = "Authors",
     description = "Endpoints for managing authors")
public class AuthorController { ... }

// OrderController.java
@RestController
@RequestMapping("/api/orders")
@Tag(name = "Orders",
     description = "Endpoints for placing and managing orders")
public class OrderController { ... }
```

In Swagger UI, endpoints are now grouped:

```
+-----------------------------------------+
|  BookStore API  v1.0.0                  |
+-----------------------------------------+
|                                         |
|  Books                                  |
|  Endpoints for managing books           |
|  +-----------------------------------+  |
|  | POST   /api/books                 |  |
|  | GET    /api/books                 |  |
|  | GET    /api/books/{id}            |  |
|  | PUT    /api/books/{id}            |  |
|  | DELETE /api/books/{id}            |  |
|  +-----------------------------------+  |
|                                         |
|  Authors                                |
|  Endpoints for managing authors         |
|  +-----------------------------------+  |
|  | POST   /api/authors               |  |
|  | GET    /api/authors               |  |
|  | GET    /api/authors/{id}          |  |
|  +-----------------------------------+  |
|                                         |
|  Orders                                 |
|  Endpoints for placing and managing...  |
|  +-----------------------------------+  |
|  | POST   /api/orders                |  |
|  | GET    /api/orders                |  |
|  | GET    /api/orders/{id}           |  |
|  +-----------------------------------+  |
+-----------------------------------------+
```

Ordering Tags

You can control the order of tag groups:

```properties
# application.properties
springdoc.swagger-ui.tagsSorter=alpha
```

Or define the order in your OpenAPI config:

```java
@Bean
public OpenAPI customOpenAPI() {
    return new OpenAPI()
        .info(new Info().title("BookStore API").version("1.0"))
        .tags(List.of(
            new io.swagger.v3.oas.models.tags.Tag()
                .name("Books").description("Book management"),
            new io.swagger.v3.oas.models.tags.Tag()
                .name("Authors").description("Author management"),
            new io.swagger.v3.oas.models.tags.Tag()
                .name("Orders").description("Order management")
        ));
}
```

27.7 Hiding Endpoints

Not all endpoints should be visible in the documentation. Internal health checks, admin endpoints, or debug endpoints should be hidden from the public API docs.

Option 1: Hide a Single Endpoint

```java
import io.swagger.v3.oas.annotations.Hidden;

@RestController
@RequestMapping("/api/internal")
public class InternalController {

    @Hidden                                          // 1
    @GetMapping("/debug")
    public String debugInfo() {
```

```
        return "Internal debug information";
    }
}
```

- Line 1: `@Hidden` removes this endpoint from the Swagger UI and OpenAPI spec.

Option 2: Hide an Entire Controller

```java
@Hidden                                                 // Hides all endpoints
@RestController
@RequestMapping("/api/admin")
public class AdminController {

    @GetMapping("/stats")
    public String stats() { ... }

    @PostMapping("/reset")
    public String reset() { ... }
}
```

Option 3: Hide Using Properties

```properties
# application.properties

# Only include specific paths in the documentation
springdoc.paths-to-match=/api/books/**, /api/authors/**

# Exclude specific paths
springdoc.paths-to-exclude=/api/internal/**, /api/admin/**
```

Option 4: Disable Swagger UI in Production

```properties
# application-prod.properties
# Completely disable Swagger UI in production
springdoc.api-docs.enabled=false
springdoc.swagger-ui.enabled=false
```

```properties
# application-dev.properties
# Enable Swagger UI in development
springdoc.api-docs.enabled=true
springdoc.swagger-ui.enabled=true
```

```
Development:                          Production:
+-------------------+                +-------------------+
| /swagger-ui.html  |                | /swagger-ui.html  |
| AVAILABLE         |                | 404 NOT FOUND     |
| (for developers)  |                | (hidden)          |
+-------------------+                +-------------------+
```

27.8 Testing Your API from Swagger UI

One of the best features of Swagger UI is the "Try it out" button. You can test your API directly from the browser without curl or Postman.

Step-by-Step: Testing the Create Endpoint

1. Open `http://localhost:8080/swagger-ui.html` in your browser.
2. Find the `POST /api/books` endpoint under the "Books" section.
3. Click on it to expand the details.
4. Click the "Try it out" button.
5. Edit the request body:

```json
{
  "title": "Spring Boot in Action",
  "author": "Craig Walls",
  "price": 49.99
}
```

6. Click "Execute".
7. See the response:

```
Response Code: 201

Response Body:
{
  "id": 1,
  "title": "Spring Boot in Action",
  "author": "Craig Walls",
  "price": 49.99
}

Response Headers:
content-type: application/json
```

Testing the Get Endpoint

1. Expand `GET /api/books/{id}`.
2. Click "Try it out".
3. Enter `1` in the `id` field.
4. Click "Execute".
5. See the book you just created in the response.

```
Swagger UI Test Flow:

+--------+     +-----------+     +--------+     +----------+
| Click  |     | Enter     |     | Click  |     | See      |
| "Try   |---->| Request   |---->|"Execute|---->| Response |
|  it    |     | Data      |     |"       |     | (JSON)   |
|  out"  |     |           |     |        |     |          |
+--------+     +-----------+     +--------+     +----------+
```

Configuring Default Values for Testing

You can set default values that appear in the "Try it out" form:

```java
@PostMapping
@Operation(summary = "Create a new book")
public ResponseEntity<Book> createBook(
    @io.swagger.v3.oas.annotations.parameters.RequestBody(
        description = "Book data to create",
        required = true,
        content = @io.swagger.v3.oas.annotations.media.Content(
            examples = @io.swagger.v3.oas.annotations.media
                .ExampleObject(
                    name = "Sample Book",
                    value = """
                        {
                            "title": "Spring Boot in Action",
                            "author": "Craig Walls",
                            "price": 49.99
                        }
                        """
                )
        )
    )
    @RequestBody Book book) {

    Book created = bookService.createBook(book);
    return ResponseEntity.status(HttpStatus.CREATED)
                         .body(created);
}
```

27.9 Complete Example: Fully Documented API

Here is a complete example that brings together everything we have learned:

```java
// src/main/java/com/example/bookstore/config/OpenApiConfig.java

package com.example.bookstore.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI bookstoreOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("BookStore API")
                .version("1.0.0")
                .description(
                    "A complete REST API for managing a "
                    + "bookstore. Supports creating, reading, "
                    + "updating, and deleting books.")
                .contact(new Contact()
                    .name("BookStore Support")
                    .email("support@bookstore.com"))
                .license(new License()
                    .name("MIT License")
                    .url("https://opensource.org/licenses/MIT"))
            )
            .servers(List.of(
                new Server()
                    .url("http://localhost:8080")
                    .description("Development Server"),
                new Server()
                    .url("https://api.bookstore.com")
                    .description("Production Server")
            ));
    }
}
```

```properties
# application.properties

# Swagger UI customization
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=method
springdoc.swagger-ui.tagsSorter=alpha
springdoc.swagger-ui.tryItOutEnabled=true
springdoc.swagger-ui.filter=true
springdoc.swagger-ui.docExpansion=list

# Include only public API paths
springdoc.paths-to-match=/api/**
```

Common Mistakes

Mistake 1: Forgetting to Add the Dependency

```xml
<!-- WRONG: Using the old Springfox library -->
<dependency>
    <groupId>io.springfox</groupId>
    <artifactId>springfox-boot-starter</artifactId>
    <version>3.0.0</version>
</dependency>
```

```xml
<!-- CORRECT: Use springdoc-openapi for Spring Boot 3.x -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.4</version>
</dependency>
```

Springfox is no longer maintained and does not work with Spring Boot 3. Always use springdoc-openapi.

Mistake 2: Leaving Swagger UI Enabled in Production

```properties
# WRONG: Swagger UI accessible in production
# (no profile-specific config)
springdoc.swagger-ui.enabled=true
```

```properties
# CORRECT: Disable in production
# application-prod.properties
springdoc.api-docs.enabled=false
springdoc.swagger-ui.enabled=false
```

Exposing your API documentation in production can reveal internal endpoints, data structures, and potential attack vectors.

Mistake 3: Over-Documenting with Redundant Information

```java
// WRONG: The summary just repeats the URL
@GetMapping("/{id}")
@Operation(summary = "GET /api/books/{id}")  // Redundant!
public ResponseEntity<Book> getBook(...) { }
```

```java
// CORRECT: Add meaningful information
@GetMapping("/{id}")
@Operation(summary = "Get a book by its unique ID")
public ResponseEntity<Book> getBook(...) { }
```

Mistake 4: Not Documenting Error Responses

```java
// WRONG: Only documenting the happy path
@Operation(summary = "Get a book")
@ApiResponse(responseCode = "200", description = "Success")
public ResponseEntity<Book> getBook(...) { }
```

```java
// CORRECT: Document all possible responses
@Operation(summary = "Get a book")
@ApiResponses({
    @ApiResponse(responseCode = "200",
                 description = "Book found"),
    @ApiResponse(responseCode = "404",
                 description = "Book not found"),
    @ApiResponse(responseCode = "500",
                 description = "Internal server error")
})
public ResponseEntity<Book> getBook(...) { }
```

Best Practices

1. Add meaningful summaries. Every endpoint should have a clear `summary` that explains what it does in plain English.
2. Document all response codes. Show both success and error responses so API consumers know what to expect.
3. Use @Schema on your models. Add descriptions and examples to every field. This helps users understand what data to send and what they will receive.
4. Group endpoints with @Tag. Organize related endpoints together so users can find what they need quickly.
5. Disable Swagger in production. Use profile-specific properties to hide the documentation in production environments.
6. Keep documentation up to date. If you change an endpoint, update the annotations immediately. Outdated documentation is worse than no documentation.
7. Use examples. Add `example` values to `@Schema` and `@Parameter`. This makes "Try it out" immediately useful.
8. Hide internal endpoints. Use `@Hidden` or path filters to keep internal, admin, and debug endpoints out of the public documentation.

Quick Summary

In this chapter, you learned how to create professional API documentation for your Spring Boot application. You added the `springdoc-openapi-starter-webmvc-ui` dependency to auto-generate an OpenAPI specification and Swagger UI. You customized the API information with titles, descriptions, and contact details. You used `@Operation` and `@ApiResponse` to describe each endpoint's purpose and possible responses. You documented your data models with `@Schema`, adding descriptions and examples for every field. You organized endpoints into logical groups with `@Tag`. You learned how to hide internal endpoints and disable Swagger in production. Finally, you tested your API directly from the browser using the Swagger UI's "Try it out" feature.

Key Points

Concept	Description
OpenAPI	A standard specification for describing REST APIs
Swagger UI	An interactive web page for exploring and testing APIs
`springdoc-openapi-starter-webmvc-ui`	The library that generates OpenAPI docs for Spring Boot 3
`/swagger-ui.html`	The URL to access Swagger UI
`/v3/api-docs`	The URL for the raw OpenAPI specification
`@Operation`	Describes a single endpoint (summary and description)
`@ApiResponse`	Describes a possible response (code and description)
`@Schema`	Describes a field in a data model (description, example, constraints)
`@Tag`	Groups related endpoints under a common heading
`@Hidden`	Hides an endpoint or controller from the documentation
`@Parameter`	Describes a path variable or query parameter

Practice Questions

1. What is the difference between OpenAPI and Swagger? How do they relate to each other?
2. What dependency do you need to add API documentation to a Spring Boot 3 application? Why should you not use Springfox?
3. What do `@Operation`, `@ApiResponse`, and `@Schema` do? Give an example of when you would use each one.
4. Why should you disable Swagger UI in production? How do you do it?
5. How does `@Tag` improve the organization of your API documentation? Give an example.

Exercises

Exercise 1: Document a Student API

Create a `StudentController` with CRUD endpoints for students (id, name, email, grade). Add complete OpenAPI documentation:

- Custom API info (title, version, description).
- `@Operation` and `@ApiResponse` on every endpoint.
- `@Schema` on all Student fields with examples.
- `@Tag` to group the endpoints.

Exercise 2: Multiple Tags

Create two controllers: `ProductController` and `CategoryController`. Use `@Tag` to group them separately. Add an `@Operation` that assigns a single endpoint to two tags (hint: use the `tags` property of `@Operation`).

Exercise 3: Environment-Specific Documentation

Configure your application so that:

- In the `dev` profile, Swagger UI is accessible and shows all endpoints.
- In the `prod` profile, Swagger UI is completely disabled.
- In the `staging` profile, Swagger UI is accessible but internal endpoints are hidden.

Test each profile and verify the behavior.

What Is Next?

Your API is well-documented and easy to use. But how do you know if your application is healthy in production? Is the database connected? How much memory is being used? Are there any performance bottlenecks? In the next chapter, we will learn about Actuator and Monitoring. Spring Boot Actuator gives you production-ready monitoring endpoints right out of the box.

Chapter 28: Actuator and Monitoring

What You Will Learn

- What Spring Boot Actuator is and why you need it.
- How to add Actuator to your application with one dependency.
- How to use the `/actuator/health` endpoint to check application health.
- How to expose and configure Actuator endpoints.
- How to use built-in endpoints: health, info, metrics, env, and loggers.
- How to create a custom `HealthIndicator` for your own checks.
- How to create a custom `InfoContributor` to expose application details.
- How to secure Actuator endpoints so only authorized users can access them.

Why This Chapter Matters

Imagine you own a car. You drive it every day. How do you know if something is wrong? You look at the dashboard. The dashboard shows you fuel level, engine temperature, speed, and warning lights. Without a dashboard, you would drive blind until the engine overheats or the fuel runs out.

Spring Boot Actuator is the dashboard for your application. It tells you:

- Is the application running? (Health)
- Is the database connected? (Health)
- How much memory is being used? (Metrics)
- What configuration is active? (Environment)
- What is happening right now? (Loggers)

In production, you need this information to keep your application running smoothly. When something goes wrong at 3 AM, Actuator endpoints help you diagnose the problem without digging through code.

28.1 What Is Spring Boot Actuator?

Spring Boot Actuator is a set of built-in features that help you monitor and manage your application in production.

Think of it as adding health sensors to your application. Just like a hospital patient has monitors for heart rate, blood pressure, and oxygen levels, Actuator adds monitors for your application's health, performance, and configuration.

```
Your Application Without Actuator:
+------------------+
|  Spring Boot     |
|  Application     |    "Is everything OK?"
|                  |    "I have no idea."
|  [No monitors]   |
+------------------+

Your Application With Actuator:
+------------------+
|  Spring Boot     |
|  Application     |    "Is everything OK?"
|                  |    "Database: UP
|  [Health: UP]    |     Memory: 78% used
|  [Metrics: OK]   |     Requests: 142/min
|  [Info: v1.2.3]  |     Version: 1.2.3"
+------------------+
```

What Actuator Provides

Feature	What It Tells You
Health checks	Is the app running? Is the database connected?
Metrics	How many requests per minute? How much memory?
Environment	What properties and profiles are active?
Loggers	What log levels are set? Can change them at runtime.
Info	What version is deployed? When was it built?
Beans	What Spring beans are loaded?
Mappings	What URL endpoints are available?

28.2 Adding Actuator to Your Application

Adding Actuator is as simple as adding one dependency.

Step 1: Add the Dependency

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Step 2: Start Your Application

```bash
./mvnw spring-boot:run
```

Step 3: Check the Health Endpoint

```bash
curl http://localhost:8080/actuator/health
```

Output:

```json
{
  "status": "UP"
}
```

That is it. Your application now has a health endpoint. "UP" means everything is working.

What You Get Out of the Box

After adding the dependency, these endpoints are available by default:

Endpoint	URL	Description

health	`/actuator/health`	Shows application health status
info	`/actuator/info`	Shows application information

Only `health` and `info` are exposed over HTTP by default. This is a security feature. You do not want to accidentally expose sensitive configuration to the internet.

```
Default Actuator Endpoints:

+-------------------------------------------+
| http://localhost:8080/actuator            |
+-------------------------------------------+
| /actuator/health     -> { "status": "UP" }|
| /actuator/info       -> {}                |
|                                           |
| All other endpoints: HIDDEN (for safety)  |
+-------------------------------------------+
```

28.3 Exposing More Endpoints

To see more endpoints, you need to explicitly expose them in `application.properties`.

Expose Specific Endpoints

```properties
# application.properties

# Expose specific endpoints over HTTP
management.endpoints.web.exposure.include=health,info,metrics,env,loggers
```

Expose All Endpoints

```properties
# application.properties

# Expose ALL endpoints over HTTP (use with caution!)
management.endpoints.web.exposure.include=*
```

> Warning: Exposing all endpoints is fine for development, but never do this in production without proper security. Some endpoints reveal sensitive information.

Exclude Specific Endpoints

```properties
# Expose all EXCEPT env and beans
management.endpoints.web.exposure.include=*
management.endpoints.web.exposure.exclude=env,beans
```

List All Available Endpoints

After exposing endpoints, visit:

```bash
curl http://localhost:8080/actuator
```

Output:

```json
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator"
    },
    "health": {
      "href": "http://localhost:8080/actuator/health"
    },
    "info": {
      "href": "http://localhost:8080/actuator/info"
    },
    "metrics": {
      "href": "http://localhost:8080/actuator/metrics"
    },
    "loggers": {
      "href": "http://localhost:8080/actuator/loggers"
    }
  }
}
```

This is a directory of all available endpoints with their URLs.

28.4 The Health Endpoint

The health endpoint is the most important Actuator endpoint. It tells you whether your application is healthy.

Basic Health Check

```bash
curl http://localhost:8080/actuator/health
```

Output (healthy):

```json
{
  "status": "UP"
}
```

Output (unhealthy):

```json
{
  "status": "DOWN"
}
```

Showing Detailed Health Information

By default, the health endpoint only shows "UP" or "DOWN". To see details about each component (database, disk space, etc.), add:

```properties
# application.properties
management.endpoint.health.show-details=always
```

Now the output includes component-level health:

```bash
curl http://localhost:8080/actuator/health
```

Output:

```json
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "H2",
        "validationQuery": "isValid()"
      }
```

```
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 499963174912,
        "free": 250148225024,
        "threshold": 10485760,
        "path": "/Users/dev/bookstore/.",
        "exists": true
      }
    },
    "ping": {
      "status": "UP"
    }
  }
}
```

```
Health Endpoint with Details:

+-------------------+
|  Application      |
|  Status: UP       |
+-------------------+
        |
   +----+----+----+
   |         |    |
+--+--+  +---+--+ +-----+
| DB  |  | Disk | | Ping|
| UP  |  | UP   | | UP  |
|     |  |      | |     |
| H2  |  | 250GB| |     |
| OK  |  | free | |     |
+-----+  +------+ +-----+

If ANY component is DOWN, the overall status becomes DOWN.
```

Health Status Options

```properties
# Show details options:
management.endpoint.health.show-details=never     # Only UP/DOWN
management.endpoint.health.show-details=always     # Always show details
management.endpoint.health.show-details=when-authorized  # Only for authenticated users
```

Health Indicators Spring Boot Provides

Spring Boot automatically adds health checks for components it detects:

Component	Health Indicator	What It Checks
Database	`DataSourceHealthIndicator`	Can connect to the database?
Disk Space	`DiskSpaceHealthIndicator`	Is there enough free disk space?
Mail Server	`MailHealthIndicator`	Can connect to the SMTP server?
Redis	`RedisHealthIndicator`	Can connect to Redis?
RabbitMQ	`RabbitHealthIndicator`	Can connect to RabbitMQ?

These are added automatically when the corresponding starter is in your `pom.xml`.

28.5 The Info Endpoint

The info endpoint shows information about your application. By default, it returns an empty object `{}`. You need to add information to it.

Adding Info via Properties

```properties
# application.properties

management.info.env.enabled=true

info.app.name=BookStore API
info.app.description=A REST API for managing books
info.app.version=1.0.0
info.app.java-version=${java.version}
```

```bash
curl http://localhost:8080/actuator/info
```

Output:

```json
{
  "app": {
    "name": "BookStore API",
    "description": "A REST API for managing books",
    "version": "1.0.0",
    "java-version": "17.0.9"
```

```
  }
}
```

Adding Build Info from Maven

Spring Boot can automatically include build information from Maven.

Add this plugin to your `pom.xml`:

```xml
<!-- pom.xml -->
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <executions>
                <execution>
                    <goals>
                        <goal>build-info</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Now the info endpoint includes build details:

```json
{
  "build": {
    "artifact": "bookstore",
    "name": "bookstore",
    "time": "2024-01-15T10:30:00Z",
    "version": "1.0.0",
    "group": "com.example"
  }
}
```

28.6 The Metrics Endpoint

The metrics endpoint provides numerical measurements about your application.

Listing Available Metrics

```bash
curl http://localhost:8080/actuator/metrics
```

Output:

```json
{
  "names": [
    "application.ready.time",
    "application.started.time",
    "disk.free",
    "disk.total",
    "http.server.requests",
    "jvm.memory.used",
    "jvm.memory.max",
    "jvm.threads.live",
    "process.cpu.usage",
    "system.cpu.usage",
    "tomcat.sessions.active.current"
  ]
}
```

Getting a Specific Metric

```bash
curl http://localhost:8080/actuator/metrics/jvm.memory.used
```

Output:

```json
{
  "name": "jvm.memory.used",
  "description": "The amount of used memory",
  "baseUnit": "bytes",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 157286400
    }
  ],
  "availableTags": [
    {
      "tag": "area",
      "values": ["heap", "nonheap"]
    }
  ]
}
```

Useful Metrics

Metric	What It Measures	Command
`jvm.memory.used`	Memory usage	`/actuator/metrics/jvm.memory.used`
`jvm.memory.max`	Maximum memory	`/actuator/metrics/jvm.memory.max`
`process.cpu.usage`	CPU usage (0-1)	`/actuator/metrics/process.cpu.usage`
`http.server.requests`	HTTP request stats	`/actuator/metrics/http.server.requests`
`jvm.threads.live`	Active threads	`/actuator/metrics/jvm.threads.live`
`disk.free`	Free disk space	`/actuator/metrics/disk.free`

Filtering Metrics by Tag

```bash
# Get HTTP request metrics for GET requests only
curl "http://localhost:8080/actuator/metrics/http.server.requests?tag=method:GET"

# Get HTTP request metrics for a specific endpoint
curl "http://localhost:8080/actuator/metrics/http.server.requests?tag=uri:/api/books"

# Get metrics for a specific status code
curl "http://localhost:8080/actuator/metrics/http.server.requests?tag=status:200"
```

28.7 The Env Endpoint

The env endpoint shows all environment properties and their sources.

```bash
curl http://localhost:8080/actuator/env
```

Output (abbreviated):

```json
{
  "activeProfiles": ["dev"],
  "propertySources": [
    {
      "name": "systemProperties",
      "properties": {
        "java.version": {
          "value": "17.0.9"
        }
      }
    },
    {
      "name": "applicationConfig: application.properties",
      "properties": {
        "server.port": {
          "value": "8080"
        },
        "spring.datasource.url": {
          "value": "jdbc:h2:mem:bookstore"
        }
      }
    }
  ]
}
```

Looking Up a Specific Property

```bash
curl http://localhost:8080/actuator/env/server.port
```

Output:

```json
{
  "property": {
    "source": "applicationConfig: application.properties",
    "value": "8080"
  }
}
```

> Security Note: The env endpoint can reveal sensitive information like database passwords. Spring Boot automatically sanitizes known sensitive keys (replacing values with `******`), but always be careful about exposing this endpoint.

28.8 The Loggers Endpoint

The loggers endpoint lets you view and change log levels at runtime. This is incredibly useful for debugging production issues without restarting the application.

View All Loggers

```bash
curl http://localhost:8080/actuator/loggers
```

Output (abbreviated):

```json
{
  "levels": ["OFF", "ERROR", "WARN", "INFO", "DEBUG", "TRACE"],
  "loggers": {
    "ROOT": {
      "configuredLevel": "INFO",
      "effectiveLevel": "INFO"
    },
    "com.example.bookstore": {
      "configuredLevel": null,
      "effectiveLevel": "INFO"
    },
    "com.example.bookstore.service.BookService": {
      "configuredLevel": null,
      "effectiveLevel": "INFO"
    }
  }
}
```

View a Specific Logger

```bash
curl http://localhost:8080/actuator/loggers/com.example.bookstore
```

Output:

```json
{
  "configuredLevel": null,
  "effectiveLevel": "INFO"
}
```

Change Log Level at Runtime

This is the killer feature. You can change log levels without restarting your application.

```bash
# Change to DEBUG level
curl -X POST http://localhost:8080/actuator/loggers/com.example.bookstore \
  -H "Content-Type: application/json" \
  -d '{"configuredLevel": "DEBUG"}'
```

Now your application logs DEBUG messages for `com.example.bookstore`.

```bash
# Reset to default
curl -X POST http://localhost:8080/actuator/loggers/com.example.bookstore \
  -H "Content-Type: application/json" \
  -d '{"configuredLevel": null}'
```

```
Changing Log Level at Runtime:

Before:                                After POST:
+-----------------------+             +-----------------------+
| Logger: bookstore     |             | Logger: bookstore     |
| Level: INFO           |             | Level: DEBUG          |
|                       |             |                       |
| Shows: INFO, WARN,    |   POST -->  | Shows: DEBUG, INFO,   |
|        ERROR          |             |        WARN, ERROR    |
| Hides: DEBUG, TRACE   |             | Hides: TRACE          |
+-----------------------+             +-----------------------+

No restart needed! Change takes effect immediately.
```

28.9 Creating a Custom HealthIndicator

The built-in health indicators check standard components like databases and disk space. But what about your own dependencies? Maybe your app depends on an external payment API or a file storage service. You can create custom health indicators to check those.

Example: Checking an External API

```java
// src/main/java/com/example/bookstore/health/PaymentServiceHealthIndicator.java

package com.example.bookstore.health;
```

```
import org.springframework.boot.actuate.health.Health;           // 1
import org.springframework.boot.actuate.health.HealthIndicator;  // 2
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

@Component                                                       // 3
public class PaymentServiceHealthIndicator
    implements HealthIndicator {                                  // 4

    @Override
    public Health health() {                                     // 5

        try {
            // Try to reach the payment service
            RestTemplate restTemplate = new RestTemplate();
            restTemplate.getForObject(                           // 6
                "https://api.payment-service.com/health",
                String.class
            );

            return Health.up()                                   // 7
                .withDetail("service", "Payment API")            // 8
                .withDetail("status", "Reachable")
                .build();

        } catch (Exception e) {
            return Health.down()                                 // 9
                .withDetail("service", "Payment API")
                .withDetail("error", e.getMessage())             // 10
                .build();
        }
    }
}
```

Line-by-line explanation:

- Line 1: Import `Health`, the builder class for health status.
- Line 2: Import `HealthIndicator`, the interface you implement.
- Line 3: `@Component` registers this as a Spring bean so Actuator discovers it automatically.
- Line 4: Implement `HealthIndicator`. You must override one method: `health()`.
- Line 5: This method is called every time someone hits `/actuator/health`.
- Line 6: Try to call the external service.
- Line 7: If successful, return `Health.up()` (healthy).
- Line 8: `withDetail()` adds extra information to the health response.
- Line 9: If the call fails, return `Health.down()` (unhealthy).
- Line 10: Include the error message so operators know what went wrong.

Output when the payment service is healthy:

```json
{
  "status": "UP",
  "components": {
    "db": { "status": "UP" },
    "diskSpace": { "status": "UP" },
    "paymentService": {
      "status": "UP",
      "details": {
        "service": "Payment API",
        "status": "Reachable"
      }
    }
  }
}
```

Output when the payment service is down:

```json
{
  "status": "DOWN",
  "components": {
    "db": { "status": "UP" },
    "diskSpace": { "status": "UP" },
    "paymentService": {
      "status": "DOWN",
      "details": {
        "service": "Payment API",
        "error": "Connection refused"
      }
    }
  }
}
```

Notice: The overall status becomes "DOWN" because one component is down.

Example: Checking Disk Space Threshold

```java
@Component
public class StorageHealthIndicator implements HealthIndicator {

    private static final long MIN_FREE_SPACE =
        1024 * 1024 * 500;  // 500 MB

    @Override
    public Health health() {
        File root = new File("/");
        long freeSpace = root.getFreeSpace();

        if (freeSpace >= MIN_FREE_SPACE) {
```

```
            return Health.up()
                .withDetail("freeSpace",
                    freeSpace / (1024 * 1024) + " MB")
                .withDetail("threshold",
                    MIN_FREE_SPACE / (1024 * 1024) + " MB")
                .build();
        } else {
            return Health.down()
                .withDetail("freeSpace",
                    freeSpace / (1024 * 1024) + " MB")
                .withDetail("error",
                    "Free space below threshold")
                .build();
        }
    }
}
```

28.10 Creating a Custom InfoContributor

Just as you can create custom health indicators, you can add custom information to the info endpoint.

```java
// src/main/java/com/example/bookstore/info/BookStoreInfoContributor.java

package com.example.bookstore.info;

import com.example.bookstore.repository.BookRepository;
import org.springframework.boot.actuate.info.Info;            // 1
import org.springframework.boot.actuate.info.InfoContributor; // 2
import org.springframework.stereotype.Component;

@Component
public class BookStoreInfoContributor
    implements InfoContributor {                               // 3

    private final BookRepository bookRepository;

    public BookStoreInfoContributor(
            BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @Override
    public void contribute(Info.Builder builder) {            // 4

        builder.withDetail("bookstore",                       // 5
```

```
            java.util.Map.of(
                "totalBooks", bookRepository.count(),          // 6
                "status", "operational",
                "supportEmail", "support@bookstore.com"
            )
        );
    }
}
```

Line-by-line explanation:

- Line 1-2: Import the `Info` builder and `InfoContributor` interface.
- Line 3: Implement `InfoContributor`. You must override `contribute()`.
- Line 4: The `contribute` method receives an `Info.Builder` that you add data to.
- Line 5: `withDetail("bookstore", ...)` adds a "bookstore" section to the info output.
- Line 6: Query the actual database for the book count.

Output:

```bash
curl http://localhost:8080/actuator/info
```

```json
{
  "app": {
    "name": "BookStore API",
    "version": "1.0.0"
  },
  "bookstore": {
    "totalBooks": 42,
    "status": "operational",
    "supportEmail": "support@bookstore.com"
  }
}
```

28.11 Securing Actuator Endpoints

Actuator endpoints can expose sensitive information. In production, you must secure them.

Option 1: Change the Actuator Base Path

```properties
# application.properties

# Move actuator to a non-obvious path
management.endpoints.web.base-path=/internal/monitor
```

Now endpoints are at `/internal/monitor/health` instead of `/actuator/health`. This hides them from casual discovery but is not real security (security through obscurity).

Option 2: Use a Different Port

```properties
# application.properties

# Run actuator on a different port
management.server.port=9090
```

Now your API runs on port 8080 and Actuator runs on port 9090. You can configure your firewall to block port 9090 from the internet while allowing internal monitoring tools to access it.

```
Two-Port Setup:

Internet Users:
+--------+     +------------------+
| Users  |---->| Port 8080        |     Actuator endpoints
|        |     | /api/books       |     are NOT accessible
|        |     | /api/authors     |     from port 8080
+--------+     +------------------+

Internal Monitoring:
+-----------+     +------------------+
| Monitoring|---->| Port 9090        |
| System    |     | /actuator/health |
| (internal)|     | /actuator/metrics|
+-----------+     +------------------+
```

Option 3: Secure with Spring Security

If you have Spring Security in your project, you can restrict access to Actuator endpoints:

```java
// src/main/java/com/example/bookstore/config/SecurityConfig.java

package com.example.bookstore.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders
    .HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http) throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                // Public API endpoints
                .requestMatchers("/api/**").permitAll()

                // Health endpoint is public
                // (for load balancers)
                .requestMatchers(
                    "/actuator/health").permitAll()

                // All other actuator endpoints require
                // ADMIN role
                .requestMatchers(
                    "/actuator/**").hasRole("ADMIN")

                .anyRequest().authenticated()
            )
            .httpBasic(httpBasic -> {});  // Basic auth for actuator

        return http.build();
    }
}
```

```properties
# application.properties

# Admin user for actuator access
spring.security.user.name=admin
spring.security.user.password=secretPassword
spring.security.user.roles=ADMIN
```

Now:

- `/actuator/health` is accessible without authentication (needed by load balancers).
- All other `/actuator/**` endpoints require the ADMIN role.

Option 4: Only Expose What You Need

The simplest security measure is to only expose the endpoints you actually use:

```properties
# application.properties

# Only expose health and info (safest option)
management.endpoints.web.exposure.include=health,info

# Show health details only to authorized users
management.endpoint.health.show-details=when-authorized
```

28.12 Recommended Production Configuration

Here is a complete, production-ready Actuator configuration:

```properties
# application-prod.properties

# Expose only necessary endpoints
management.endpoints.web.exposure.include=health,info,metrics,loggers

# Health endpoint configuration
management.endpoint.health.show-details=when-authorized
management.endpoint.health.show-components=when-authorized

# Run on a separate port (optional, for firewall control)
management.server.port=9090

# Customize the base path
management.endpoints.web.base-path=/actuator

# Info endpoint
management.info.env.enabled=true
info.app.name=BookStore API
info.app.version=@project.version@
info.app.environment=production
```

```properties
# application-dev.properties

# In development, expose everything
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Common Mistakes

Mistake 1: Exposing All Endpoints in Production

```properties
# WRONG: Exposes sensitive data to the internet
management.endpoints.web.exposure.include=*
```

```properties
# CORRECT: Only expose what you need
management.endpoints.web.exposure.include=health,info,metrics
```

Mistake 2: Showing Health Details to Everyone

```properties
# WRONG: Anyone can see your database details
management.endpoint.health.show-details=always
```

```properties
# CORRECT: Only show to authorized users in production
management.endpoint.health.show-details=when-authorized
```

Mistake 3: Not Adding Actuator at All

Many developers skip Actuator. Then when their application goes down in production, they have no way to diagnose the problem. Always add Actuator. It costs nothing and provides invaluable monitoring.

Mistake 4: Custom Health Checks That Are Too Slow

```java
// WRONG: Health check takes 30 seconds
@Override
public Health health() {
    // This call takes 30 seconds if the service is slow
    restTemplate.getForObject("https://slow-api.com/check",
                              String.class);
    return Health.up().build();
}
```

```java
// CORRECT: Set a timeout on the health check
@Override
public Health health() {
    try {
        // 3-second timeout
```

```
        RequestConfig config = RequestConfig.custom()
            .setConnectTimeout(Timeout.ofSeconds(3))
            .build();
        // ... use the config with your HTTP client
        return Health.up().build();
    } catch (Exception e) {
        return Health.down()
            .withDetail("error", "Timeout after 3 seconds")
            .build();
    }
}
```

Mistake 5: Not Understanding When Health Returns DOWN

```
Overall Status Logic:

Component 1: UP    \
Component 2: UP     >  Overall: UP (all components up)
Component 3: UP    /

Component 1: UP    \
Component 2: DOWN   >  Overall: DOWN (one component down!)
Component 3: UP    /

If ANY component is DOWN, the overall status is DOWN.
This can trigger alerts and kill your load balancer routing.
Make sure your custom health checks are accurate!
```

Best Practices

1. Always add Actuator. It is free, lightweight, and essential for production monitoring. There is no good reason to skip it.
2. Expose only necessary endpoints in production. Use `management.endpoints.web.exposure.include` to whitelist specific endpoints.
3. Secure sensitive endpoints. Use Spring Security, a separate port, or both to protect endpoints like env, beans, and loggers.
4. Keep the health endpoint public. Load balancers and orchestrators (like Kubernetes) need `/actuator/health` to be accessible without authentication.
5. Create custom health indicators for external dependencies. If your app depends on an external service, add a health check for it.

6. Use the loggers endpoint for production debugging. When you need to debug an issue, change the log level to DEBUG at runtime instead of redeploying.
7. Add build info to the info endpoint. This tells you exactly which version is deployed, which is invaluable when debugging.
8. Set timeouts on custom health checks. A slow health check can make your entire health endpoint slow, affecting load balancer decisions.

Quick Summary

In this chapter, you learned how to monitor your Spring Boot application with Actuator. You added the `spring-boot-starter-actuator` dependency and immediately got a `/actuator/health` endpoint. You exposed additional endpoints like metrics, env, loggers, and info. You explored the health endpoint in detail, seeing how Spring Boot automatically checks your database and disk space. You used the metrics endpoint to monitor JVM memory, CPU usage, and HTTP request statistics. You changed log levels at runtime using the loggers endpoint. You created custom health indicators to monitor external dependencies and custom info contributors to expose application-specific data. Finally, you learned how to secure Actuator endpoints in production using separate ports, Spring Security, and selective endpoint exposure.

Key Points

Concept	Description
`spring-boot-starter-actuator`	Adds monitoring endpoints to your application
`/actuator/health`	Shows whether the application and its dependencies are healthy
`/actuator/info`	Shows application information (version, build time, etc.)
`/actuator/metrics`	Shows numerical measurements (memory, CPU, requests)
`/actuator/env`	Shows environment properties and their sources

`/actuator/loggers`	Shows and changes log levels at runtime
`management.endpoints.web.exposure.include`	Controls which endpoints are exposed over HTTP
`management.endpoint.health.show-details`	Controls whether health details are shown
`HealthIndicator`	Interface for creating custom health checks
`InfoContributor`	Interface for adding custom data to the info endpoint
`management.server.port`	Runs Actuator on a separate port

Practice Questions

1. What is Spring Boot Actuator and why is it important for production applications?
2. Why are only the health and info endpoints exposed by default? What security risk exists if you expose all endpoints?
3. How do you create a custom `HealthIndicator`? What happens to the overall health status when one component returns DOWN?
4. How can you change the log level of a specific package at runtime without restarting the application?
5. Describe three different strategies for securing Actuator endpoints in production.

Exercises

Exercise 1: Custom Health Indicator

Create a custom `HealthIndicator` called `DatabaseStorageHealthIndicator` that:

- Queries the `books` table and counts the total number of books.
- Returns UP if there are fewer than 10,000 books.
- Returns DOWN (with a warning) if there are 10,000 or more books, indicating the database storage might be running low.
- Returns DOWN if the count query fails.

Exercise 2: Application Dashboard

Create a custom `InfoContributor` that exposes the following information at `/actuator/info`:

- Total number of books in the database.
- Total number of authors.
- The most expensive book title and price.
- Application uptime.

Test it by adding books and verifying the info endpoint updates.

Exercise 3: Production Configuration

Create a complete Actuator configuration for three environments:

- dev: All endpoints exposed, health details always shown, same port as the application.
- staging: Health, info, metrics, and loggers exposed. Health details shown when authorized. Same port.
- prod: Only health and info exposed. Health details shown when authorized. Separate management port (9090). Endpoints secured with Spring Security.

Test each profile and verify the correct endpoints are accessible.

What Is Next?

You now know how to monitor your application in production. But how do you manage configuration across different environments? How do you handle secrets like database passwords? In the next chapter, we will learn about Externalized Configuration. You will discover how Spring Boot lets you configure your application differently for development, staging, and production without changing any code.

Chapter 29: Externalized Configuration

What You Will Learn

- What externalized configuration is and why it matters.
- The complete Spring Boot configuration hierarchy (which source wins).
- How to use environment variables with `${DB_PASSWORD}` syntax.
- How to set default values for missing properties.
- How to use profiles to configure different environments.
- How to create type-safe configuration with `@ConfigurationProperties`.
- How to validate configuration with `@Validated`.
- How to use `.env` files for local development.
- How to manage secrets safely in production.

Why This Chapter Matters

Imagine you are a chef who works at three restaurants. Each restaurant has a different kitchen, different ovens, and different ingredients. But you cook the same recipes. You do not rewrite your recipes for each restaurant. Instead, you adjust the settings: oven temperature, ingredient quantities, cooking times.

Software applications work the same way. Your code (the recipe) stays the same, but the configuration (the settings) changes for each environment:

- Development: Use a local H2 database, show SQL queries, log everything.
- Staging: Use a test database, hide SQL, log warnings and errors.
- Production: Use the real database, strict security, minimal logging.

Externalized configuration means keeping your settings outside your code. You change the settings without touching, recompiling, or redeploying your code.

This chapter teaches you how to manage configuration like a professional. You will learn the right way to handle database passwords, API keys, and environment-specific settings.

29.1 The Configuration Hierarchy

Spring Boot reads configuration from many sources. When the same property is defined in multiple places, the one with higher priority wins.

Priority Order (Highest to Lowest)

```
Priority (highest wins):

  1. Command-line arguments         --server.port=9090
  2. Java system properties         -Dserver.port=9090
  3. OS environment variables       SERVER_PORT=9090
  4. Profile-specific properties    application-prod.properties
  5. application.properties         server.port=8080
  6. @PropertySource annotations    Custom property files
  7. Default properties             SpringApplication defaults

If server.port is set in BOTH environment variables (9090)
AND application.properties (8080), the environment variable
wins because it has higher priority.
```

Think of it like a stack of transparent sheets. Each sheet can have values written on it. You look down through the stack. The first value you see wins.

```
+-------------------------------+  <-- Highest Priority
| Command-line arguments        |  --server.port=9090
+-------------------------------+
| OS environment variables      |  SERVER_PORT=9090
+-------------------------------+
| application-prod.properties   |  server.port=8085
+-------------------------------+
| application.properties        |  server.port=8080
+-------------------------------+  <-- Lowest Priority

Question: What is server.port?
Answer: 9090 (command-line wins!)
```

Why This Hierarchy Exists

This hierarchy lets you:

1. Set default values in `application.properties` (committed to Git).
2. Override for specific environments with profiles (committed to Git).
3. Override secrets with environment variables (never in Git).
4. Override anything from the command line (for quick fixes).

29.2 Environment Variables

Environment variables are the most common way to pass configuration in production. They are set outside your application and are available to any process on the system.

Setting Environment Variables

```bash
# Linux/Mac
export DB_HOST=localhost
export DB_PORT=5432
export DB_USERNAME=admin
export DB_PASSWORD=superSecret123

# Windows (Command Prompt)
set DB_HOST=localhost
set DB_PORT=5432

# Windows (PowerShell)
$env:DB_HOST = "localhost"
$env:DB_PORT = "5432"
```

Using Environment Variables in application.properties

```properties
# application.properties

spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/bookstore
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

The `${...}` syntax tells Spring Boot to look for an environment variable with that name.

```
How ${DB_HOST} Gets Resolved:
```

```
application.properties:
  spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/bookstore

Environment Variables:
  DB_HOST=prod-db.example.com
  DB_PORT=5432

Resolved Value:
  spring.datasource.url=jdbc:postgresql://prod-db.example.com:5432/bookstore
```

Default Values

What if an environment variable is not set? Your app will fail to start. To prevent this, use default values:

```properties
# Syntax: ${VARIABLE_NAME:defaultValue}

spring.datasource.url=jdbc:h2:mem:bookstore
spring.datasource.username=${DB_USERNAME:sa}
spring.datasource.password=${DB_PASSWORD:}

server.port=${SERVER_PORT:8080}

app.email.from=${EMAIL_FROM:noreply@bookstore.com}
app.max-upload-size=${MAX_UPLOAD_MB:10}
```

The colon (`:`) separates the variable name from the default value. If `DB_USERNAME` is not set, Spring uses `sa`.

```
Default Value Resolution:

${DB_USERNAME:sa}
       |       |
       |       +-- Default value: "sa"
       +---------- Variable name: DB_USERNAME

If DB_USERNAME is set    -> Use its value
If DB_USERNAME is NOT set -> Use "sa"
```

Environment Variable Naming Convention

Spring Boot automatically maps property names to environment variable names:

Property Name	Environment Variable

`server.port`	`SERVER_PORT`
`spring.datasource.url`	`SPRING_DATASOURCE_URL`
`app.email.from`	`APP_EMAIL_FROM`
`my.custom-property`	`MY_CUSTOM_PROPERTY` or `MY_CUSTOMPROPERTY`

The rules are simple:

1. Replace dots (`.`) with underscores (`_`).
2. Replace hyphens (`-`) with underscores (`_`).
3. Convert to uppercase.

29.3 Profiles

Profiles let you have different configurations for different environments, all within the same project.

How Profiles Work

```
application.properties            <- Always loaded (defaults)
application-dev.properties        <- Loaded when profile = "dev"
application-staging.properties    <- Loaded when profile = "staging"
application-prod.properties       <- Loaded when profile = "prod"

Profile-specific files OVERRIDE application.properties.
```

Creating Profile-Specific Files

```properties
# src/main/resources/application.properties (defaults)

spring.application.name=BookStore
server.port=8080

# Default: H2 database
spring.datasource.url=jdbc:h2:mem:bookstore
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

```
logging.level.root=INFO
```

```properties
# src/main/resources/application-dev.properties

# Development overrides
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.h2.console.enabled=true
logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG
```

```properties
# src/main/resources/application-staging.properties

# Staging overrides
spring.datasource.url=jdbc:postgresql://${DB_HOST:staging-db}:5432/bookstore
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
logging.level.com.example=INFO
```

```properties
# src/main/resources/application-prod.properties

# Production overrides
spring.datasource.url=jdbc:postgresql://${DB_HOST}:5432/bookstore
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=false
logging.level.com.example=WARN

# Actuator security
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=when-authorized
```

Activating a Profile

There are several ways to activate a profile:

```properties
# Option 1: In application.properties
spring.profiles.active=dev
```

```bash
# Option 2: Environment variable
export SPRING_PROFILES_ACTIVE=prod
```

```bash
# Option 3: Command-line argument
java -jar bookstore.jar --spring.profiles.active=prod
```

```bash
# Option 4: JVM system property
java -Dspring.profiles.active=prod -jar bookstore.jar
```

Multiple Active Profiles

You can activate multiple profiles at once:

```bash
# Activate both "prod" and "email" profiles
java -jar bookstore.jar --spring.profiles.active=prod,email
```

Using @Profile in Java Code

You can make beans available only for specific profiles:

```java
@Configuration
public class DataSourceConfig {

    @Bean
    @Profile("dev")                             // Only in dev
    public DataSource devDataSource() {
        // H2 in-memory database
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .build();
    }

    @Bean
    @Profile("prod")                            // Only in prod
    public DataSource prodDataSource() {
        // Real PostgreSQL database
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl(System.getenv("DB_URL"));
        ds.setUsername(System.getenv("DB_USERNAME"));
        ds.setPassword(System.getenv("DB_PASSWORD"));
        return ds;
    }
}
```

```
Profile-Based Bean Loading:

Profile = "dev":                    Profile = "prod":
+-------------------+              +-------------------+
| devDataSource     | LOADED       | devDataSource     | SKIPPED
| (H2 in-memory)    |              | (H2 in-memory)    |
+-------------------+              +-------------------+
| prodDataSource    | SKIPPED      | prodDataSource    | LOADED
| (PostgreSQL)      |              | (PostgreSQL)      |
+-------------------+              +-------------------+
```

29.4 Type-Safe Configuration with @ConfigurationProperties

Using `@Value` annotations everywhere is messy and error-prone. `@ConfigurationProperties` is a better approach that maps properties to a Java class.

The Problem with @Value

```java
// Messy: @Value scattered across multiple classes
@Service
public class EmailService {

    @Value("${app.email.from}")
    private String from;

    @Value("${app.email.host}")
    private String host;

    @Value("${app.email.port}")
    private int port;

    @Value("${app.email.username}")
    private String username;

    // What if you misspell a property name?
    // No error until runtime!
    @Value("${app.email.pasword}")  // Typo! Missing 's'
    private String password;         // Fails at runtime
}
```

The Solution: @ConfigurationProperties

```java
// src/main/java/com/example/bookstore/config/EmailProperties.java

package com.example.bookstore.config;

import org.springframework.boot.context.properties
    .ConfigurationProperties;                                // 1
import org.springframework.stereotype.Component;

@Component                                                   // 2
@ConfigurationProperties(prefix = "app.email")               // 3
public class EmailProperties {

    private String from;                                     // 4
    private String host;
    private int port;
    private String username;
    private String password;

    // Getters and Setters (required!)                       // 5

    public String getFrom() { return from; }
    public void setFrom(String from) { this.from = from; }

    public String getHost() { return host; }
    public void setHost(String host) { this.host = host; }

    public int getPort() { return port; }
    public void setPort(int port) { this.port = port; }

    public String getUsername() { return username; }
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() { return password; }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

Line-by-line explanation:

- Line 1: Import `@ConfigurationProperties`.
- Line 2: `@Component` registers this as a Spring bean.
- Line 3: `@ConfigurationProperties(prefix = "app.email")` maps all properties starting with `app.email` to fields in this class.
- Line 4: Each field maps to a property. `from` maps to `app.email.from`.
- Line 5: Getters and setters are required for `@ConfigurationProperties` to bind the values.

The Properties File

```properties
# application.properties

app.email.from=noreply@bookstore.com
app.email.host=smtp.gmail.com
app.email.port=587
app.email.username=bookstore@gmail.com
app.email.password=${EMAIL_PASSWORD:default}
```

Property to Field Mapping

```
application.properties:          EmailProperties.java:

app.email.from = noreply@...     private String from;
app.email.host = smtp.gmail...   private String host;
app.email.port = 587             private int port;
app.email.username = book...     private String username;
app.email.password = ${...}      private String password;

The prefix "app.email" is stripped.
The remaining part maps to the field name.
```

Using the Properties Class

```java
@Service
public class EmailService {

    private final EmailProperties emailProperties;       // 1

    public EmailService(EmailProperties emailProperties) {
        this.emailProperties = emailProperties;          // 2
    }

    public void sendEmail(String to, String subject,
                          String body) {
        System.out.println("Sending from: "
            + emailProperties.getFrom());                // 3
        System.out.println("Using host: "
            + emailProperties.getHost());
        System.out.println("On port: "
            + emailProperties.getPort());
    }
}
```

Nested Properties

You can nest property groups:

```properties
# application.properties

app.email.from=noreply@bookstore.com
app.email.host=smtp.gmail.com
app.email.retry.max-attempts=3
app.email.retry.delay-ms=1000
```

```java
@Component
@ConfigurationProperties(prefix = "app.email")
public class EmailProperties {

    private String from;
    private String host;
    private Retry retry = new Retry();    // Nested class

    // Getters and setters for from and host...

    public Retry getRetry() { return retry; }
    public void setRetry(Retry retry) { this.retry = retry; }

    public static class Retry {                        // 1
        private int maxAttempts = 3;                   // 2
        private long delayMs = 1000;

        public int getMaxAttempts() { return maxAttempts; }
        public void setMaxAttempts(int maxAttempts) {
            this.maxAttempts = maxAttempts;
        }

        public long getDelayMs() { return delayMs; }
        public void setDelayMs(long delayMs) {
            this.delayMs = delayMs;
        }
    }
}
```

- Line 1: A static inner class for the nested group `retry`.
- Line 2: Default values are set in the field declaration.

Usage:

```java
int maxRetries = emailProperties.getRetry().getMaxAttempts();
long delay = emailProperties.getRetry().getDelayMs();
```

List and Map Properties

```properties
# application.properties

# List of allowed email domains
app.email.allowed-domains[0]=gmail.com
app.email.allowed-domains[1]=yahoo.com
app.email.allowed-domains[2]=outlook.com

# Map of rate limits per domain
app.email.rate-limits.gmail.com=100
app.email.rate-limits.yahoo.com=50
app.email.rate-limits.outlook.com=75
```

```java
@Component
@ConfigurationProperties(prefix = "app.email")
public class EmailProperties {

    private List<String> allowedDomains = new ArrayList<>();
    private Map<String, Integer> rateLimits = new HashMap<>();

    public List<String> getAllowedDomains() {
        return allowedDomains;
    }
    public void setAllowedDomains(List<String> allowedDomains) {
        this.allowedDomains = allowedDomains;
    }

    public Map<String, Integer> getRateLimits() {
        return rateLimits;
    }
    public void setRateLimits(Map<String, Integer> rateLimits) {
        this.rateLimits = rateLimits;
    }
}
```

29.5 Validating Configuration with @Validated

What if someone forgets to set a required property? Without validation, your application starts but fails later with a confusing error. With `@Validated`, Spring checks your configuration at startup and gives a clear error message.

Add the Validation Dependency

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Add Validation Annotations

```java
// src/main/java/com/example/bookstore/config/EmailProperties.java

package com.example.bookstore.config;

import jakarta.validation.constraints.*;                    // 1
import org.springframework.boot.context.properties
    .ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;  // 2

@Component
@ConfigurationProperties(prefix = "app.email")
@Validated                                                  // 3
public class EmailProperties {

    @NotBlank(message = "Email 'from' address is required")  // 4
    private String from;

    @NotBlank(message = "SMTP host is required")
    private String host;

    @Min(value = 1, message = "Port must be at least 1")     // 5
    @Max(value = 65535,
         message = "Port must be at most 65535")
    private int port;

    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")              // 6
    private String password;

    // Getters and setters...
    public String getFrom() { return from; }
    public void setFrom(String from) { this.from = from; }

    public String getHost() { return host; }
    public void setHost(String host) { this.host = host; }

    public int getPort() { return port; }
    public void setPort(int port) { this.port = port; }
```

```
    public String getUsername() { return username; }
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() { return password; }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

Key annotations:

- Line 1: Import Jakarta Validation constraints.
- Line 2: Import `@Validated`.
- Line 3: `@Validated` enables validation on this properties class.
- Line 4: `@NotBlank` ensures the value is not null, not empty, and not just whitespace.
- Line 5: `@Min` and `@Max` set a valid range for numeric fields.
- Line 6: If `password` is blank, the app fails to start with a clear error message.

What Happens When Validation Fails

If `app.email.from` is missing:

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Binding to target org.springframework.boot.context.properties
.bind.BindException: Failed to bind properties under
'app.email' to com.example.bookstore.config.EmailProperties

    Property: app.email.from
    Value: null
    Reason: Email 'from' address is required

Action:

Update your application's configuration
```

This is much better than a `NullPointerException` ten minutes after startup.

```
Without @Validated:                With @Validated:
```

```
App starts successfully            App FAILS to start
         |                                 |
   10 minutes later...             Clear error message:
         |                         "Email 'from' is required"
   NullPointerException!                   |
   "What went wrong??"            Fix it immediately!
```

29.6 Using .env Files for Local Development

In development, you do not want to set environment variables manually every time. A `.env` file stores environment variables in a file that your application reads at startup.

Step 1: Create a .env File

```bash
# .env (in the project root)

DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=admin
DB_PASSWORD=localDevPassword123
EMAIL_PASSWORD=myAppPassword
API_KEY=dev-api-key-12345
```

Step 2: Add .env to .gitignore

This is critical. Never commit `.env` files to version control.

```gitignore
# .gitignore

.env
*.env
.env.local
```

Step 3: Load .env in Spring Boot

Spring Boot does not load `.env` files by default. You have a few options:

Option A: Use spring-dotenv library

```xml
<!-- pom.xml -->
<dependency>
    <groupId>me.paulschwarz</groupId>
    <artifactId>spring-dotenv</artifactId>
    <version>4.0.0</version>
</dependency>
```

This automatically loads `.env` files. No additional code needed.

Option B: Load .env with a shell script

```bash
# run-dev.sh
#!/bin/bash
export $(cat .env | grep -v '^#' | xargs)
./mvnw spring-boot:run
```

Option C: Use IntelliJ IDEA / VS Code

Most IDEs let you specify an `.env` file in the run configuration. Check your IDE's documentation for details.

Referencing .env Variables

```properties
# application.properties

spring.datasource.url=jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/bookstore
spring.datasource.username=${DB_USERNAME:sa}
spring.datasource.password=${DB_PASSWORD:}

app.api-key=${API_KEY:default-key}
```

```
How .env Works in Development:

+----------+     +-------------------+     +------------------+
| .env     |     | Environment       |     | application      |
| file     |---->| Variables         |---->| .properties      |
|          |     | (loaded into OS)  |     | ${DB_HOST} = ... |
| DB_HOST= |     |                   |     |                  |
| localhost|     | DB_HOST=localhost  |     | Resolved!        |
+----------+     +-------------------+     +------------------+
```

29.7 Secrets Management

Secrets are sensitive values like database passwords, API keys, and encryption keys. Managing them correctly is critical for security.

What NOT to Do

```
NEVER DO THESE:

1. Hardcode secrets in Java code
   private String password = "myPassword123";

2. Commit secrets to Git
   spring.datasource.password=productionPassword!

3. Share secrets in chat messages or emails

4. Use the same secrets in development and production

5. Store secrets in plain text on servers
```

The Secrets Hierarchy

```
+-----------------------------------------+
|  Most Secure (Production)               |
|                                         |
|  Secrets Manager (AWS, Azure, Vault)    |
|  - Encrypted at rest                    |
|  - Access control                       |
|  - Audit logging                        |
|  - Automatic rotation                   |
+-----------------------------------------+
|  Secure (Staging / CI/CD)               |
|                                         |
|  Environment Variables                  |
|  - Set in CI/CD pipeline               |
|  - Set in container orchestration       |
|  - Not in source code                   |
+-----------------------------------------+
|  Acceptable (Development)               |
|                                         |
|  .env files (in .gitignore)             |
|  - Local to developer machine          |
|  - Never committed to Git              |
+-----------------------------------------+
|  NEVER (Anywhere)                       |
|                                         |
|  Hardcoded in source code               |
|  Committed to Git                       |
+-----------------------------------------+
```

Strategy 1: Environment Variables (Simple)

```bash
# Set in your deployment environment
export DB_PASSWORD=superSecretProd123
export JWT_SECRET=myJwtSigningKey456
export API_KEY=prod-api-key-789
```

```properties
# application.properties
spring.datasource.password=${DB_PASSWORD}
app.jwt.secret=${JWT_SECRET}
app.api.key=${API_KEY}
```

Strategy 2: Using Spring Cloud Config

For larger applications, Spring Cloud Config provides a central configuration server:

```
+----------------+       +-----------+       +-----------+
| Config Server  |<----->| Git Repo  |       | App 1     |
| (centralized)  |       | (config   |<----->| reads     |
|                |       |  files)   |       | config    |
+----------------+       +-----------+       +-----------+
        |                                     +-----------+
        +<----------------------------------->| App 2     |
        |                                     | reads     |
        |                                     | config    |
        |                                     +-----------+
        +<----------------------------------->+-----------+
                                              | App 3     |
                                              | reads     |
                                              | config    |
                                              +-----------+
```

Strategy 3: Vault Integration

HashiCorp Vault is a popular tool for managing secrets:

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-vault-config</artifactId>
</dependency>
```

```properties
# application.properties
spring.cloud.vault.uri=https://vault.example.com:8200
spring.cloud.vault.token=${VAULT_TOKEN}
spring.cloud.vault.kv.backend=secret
```

This is an advanced topic, but knowing it exists is important for production applications.

29.8 A Complete Configuration Example

Here is a real-world example bringing together everything in this chapter.

The Configuration Properties Class

```java
// src/main/java/com/example/bookstore/config/AppProperties.java

package com.example.bookstore.config;

import jakarta.validation.constraints.*;
import org.springframework.boot.context.properties
    .ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import java.util.List;

@Component
@ConfigurationProperties(prefix = "app")
@Validated
public class AppProperties {

    @NotBlank
    private String name;

    @NotBlank
    private String version;

    private Security security = new Security();
    private Cors cors = new Cors();

    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getVersion() { return version; }
    public void setVersion(String version) {
        this.version = version;
```

```
    }
    public Security getSecurity() { return security; }
    public void setSecurity(Security security) {
        this.security = security;
    }
    public Cors getCors() { return cors; }
    public void setCors(Cors cors) { this.cors = cors; }

    public static class Security {
        @NotBlank
        private String jwtSecret;

        @Min(300)   // At least 5 minutes
        @Max(86400) // At most 24 hours
        private long jwtExpirationSeconds = 3600;

        public String getJwtSecret() { return jwtSecret; }
        public void setJwtSecret(String jwtSecret) {
            this.jwtSecret = jwtSecret;
        }
        public long getJwtExpirationSeconds() {
            return jwtExpirationSeconds;
        }
        public void setJwtExpirationSeconds(
                long jwtExpirationSeconds) {
            this.jwtExpirationSeconds = jwtExpirationSeconds;
        }
    }

    public static class Cors {
        private List<String> allowedOrigins =
            List.of("http://localhost:3000");
        private List<String> allowedMethods =
            List.of("GET", "POST", "PUT", "DELETE");

        public List<String> getAllowedOrigins() {
            return allowedOrigins;
        }
        public void setAllowedOrigins(
                List<String> allowedOrigins) {
            this.allowedOrigins = allowedOrigins;
        }
        public List<String> getAllowedMethods() {
            return allowedMethods;
        }
        public void setAllowedMethods(
                List<String> allowedMethods) {
            this.allowedMethods = allowedMethods;
        }
    }
}
```

The Properties Files

```properties
# application.properties (defaults)

app.name=BookStore
app.version=1.0.0
app.security.jwt-secret=${JWT_SECRET:dev-secret-key-for-local-only}
app.security.jwt-expiration-seconds=3600
app.cors.allowed-origins=http://localhost:3000

spring.datasource.url=jdbc:h2:mem:bookstore
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
```

```properties
# application-dev.properties

spring.jpa.show-sql=true
spring.h2.console.enabled=true
logging.level.com.example=DEBUG
```

```properties
# application-prod.properties

spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT:5432}/bookstore
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=none

app.security.jwt-secret=${JWT_SECRET}
app.security.jwt-expiration-seconds=1800
app.cors.allowed-origins=${CORS_ORIGINS:https://bookstore.com}

logging.level.com.example=WARN
```

The .env File (Development Only)

```bash
# .env (never committed to Git)

JWT_SECRET=local-dev-secret-key-abc123
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=dev_user
DB_PASSWORD=dev_password
```

Using the Properties

```java
@RestController
@RequestMapping("/api/config")
public class ConfigCheckController {

    private final AppProperties appProperties;

    public ConfigCheckController(AppProperties appProperties) {
        this.appProperties = appProperties;
    }

    @GetMapping("/info")
    public Map<String, Object> getInfo() {
        return Map.of(
            "name", appProperties.getName(),
            "version", appProperties.getVersion(),
            "jwtExpiration",
                appProperties.getSecurity()
                    .getJwtExpirationSeconds(),
            "corsOrigins",
                appProperties.getCors()
                    .getAllowedOrigins()
        );
    }
}
```

Output:

```json
{
  "name": "BookStore",
  "version": "1.0.0",
  "jwtExpiration": 3600,
  "corsOrigins": ["http://localhost:3000"]
}
```

Common Mistakes

Mistake 1: Committing Secrets to Git

```properties
# WRONG: Real password in version control
spring.datasource.password=prodPassword123!
```

```properties
# CORRECT: Use environment variables
spring.datasource.password=${DB_PASSWORD}
```

Even if you later remove the secret and commit, it is still in the Git history. If this happens, consider the secret compromised and rotate it immediately.

Mistake 2: No Default Values for Optional Properties

```properties
# WRONG: App crashes if SERVER_PORT is not set
server.port=${SERVER_PORT}
```

```properties
# CORRECT: Provide a sensible default
server.port=${SERVER_PORT:8080}
```

Mistake 3: Using @Value for Complex Configuration

```java
// WRONG: Scattered @Value annotations
@Value("${app.email.from}") private String from;
@Value("${app.email.host}") private String host;
@Value("${app.email.port}") private int port;
@Value("${app.email.username}") private String username;
@Value("${app.email.password}") private String password;
```

```java
// CORRECT: One @ConfigurationProperties class
private final EmailProperties emailProperties;
// All properties in one place, type-safe, validated
```

Mistake 4: Same Secrets in All Environments

```
WRONG:
  Dev password:  "password123"
  Staging password: "password123"    <- Same!
  Prod password: "password123"      <- Same!

CORRECT:
  Dev password:  "devLocalPass"
  Staging password: "stg_Xk9mP2vB"  <- Different
  Prod password: "prd_Qw7nR4jT"     <- Different
```

Mistake 5: Not Validating Required Configuration

```java
// WRONG: No validation, fails at runtime
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String apiKey;  // Could be null!
}
```

```java
// CORRECT: Fails at startup with clear error
@ConfigurationProperties(prefix = "app")
@Validated
public class AppProperties {
    @NotBlank(message = "API key is required")
    private String apiKey;
}
```

Best Practices

1. Use @ConfigurationProperties over @Value. It is type-safe, supports validation, and keeps configuration organized.
2. Always provide default values for properties that have sensible defaults. Use `${VAR:default}` syntax.
3. Never commit secrets to Git. Use environment variables, `.env` files (in `.gitignore`), or a secrets manager.
4. Validate configuration at startup. Use `@Validated` with Jakarta Validation annotations. Fail fast with a clear error.
5. Use profiles for environment-specific configuration. Keep defaults in `application.properties` and overrides in `application-{profile}.properties`.
6. Use different secrets for each environment. Development, staging, and production should each have unique passwords and keys.
7. Document required environment variables. Maintain a list of required environment variables (without values) so new developers know what to set up.
8. Use a secrets manager in production. For serious production applications, use AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault instead of plain environment variables.

Quick Summary

In this chapter, you learned how to manage configuration in Spring Boot applications like a professional. You explored the configuration hierarchy, understanding which source takes priority when the same property is defined in multiple places. You used environment variables with `${DB_PASSWORD}` syntax to keep secrets out of your code. You set default values with `${VAR:default}` to handle missing properties gracefully. You used profiles to create environment-specific configurations for dev, staging, and production. You created type-safe configuration classes with `@ConfigurationProperties` and validated them with `@Validated`. You learned about `.env` files for local development and secrets management strategies for production.

Key Points

Concept	Description
Configuration Hierarchy	Command-line > env vars > profile properties > application.properties
`${VAR_NAME}`	References an environment variable in properties files
`${VAR:default}`	Provides a default value if the variable is not set
Profiles	Environment-specific config (application-dev.properties, etc.)
`spring.profiles.active`	Activates a specific profile
`@ConfigurationProperties`	Maps a group of properties to a type-safe Java class
`@Validated`	Enables Jakarta Validation on a configuration class
`.env` file	Stores environment variables for local development
`.gitignore`	Must include `.env` to prevent committing secrets
Secrets Manager	Production tool for secure secret storage (Vault, AWS SM)

Practice Questions

1. Explain the Spring Boot configuration hierarchy. If `server.port` is set to `8080` in `application.properties` and to `9090` as an environment variable, which value does the application use? Why?
2. What is the difference between `${DB_PASSWORD}` and `${DB_PASSWORD:secret}` in `application.properties`? When would you use each syntax?
3. How does `@ConfigurationProperties` differ from `@Value`? List at least three advantages of using `@ConfigurationProperties`.
4. Why should you add `@Validated` to your `@ConfigurationProperties` class? What happens when a required property is missing?
5. Why should `.env` files never be committed to version control? What should you use instead in production?

Exercises

Exercise 1: Multi-Profile Application

Create a Spring Boot application with three profiles: `dev`, `staging`, and `prod`. Each profile should configure:

- A different database (H2 for dev, PostgreSQL for staging and prod).
- Different logging levels (DEBUG for dev, INFO for staging, WARN for prod).
- Different server ports (8080 for dev, 8081 for staging, 80 for prod).

Test each profile by running the application and verifying the correct settings.

Exercise 2: Type-Safe File Upload Configuration

Create a `@ConfigurationProperties` class called `FileUploadProperties` with:

- `maxFileSize` (validated: between 1 MB and 100 MB).
- `allowedTypes` (a list of strings like "jpg", "png", "pdf").
- `uploadDirectory` (validated: not blank).
- `maxFilesPerUpload` (validated: between 1 and 20).

Add `@Validated` and test that the application fails to start with clear error messages when invalid values are provided.

Exercise 3: Secrets Setup

Set up a local development environment with:

1. A `.env` file containing `DB_PASSWORD`, `JWT_SECRET`, and `API_KEY`.
2. `application.properties` referencing these variables with defaults.
3. `.gitignore` excluding the `.env` file.
4. A `README-env.md` file documenting all required environment variables (names only, no values).

Verify that the application starts correctly with and without the `.env` file (using defaults).

What Is Next?

You now know how to configure your application for any environment. But how do you actually deploy it? How do you package it so it runs on any server? In the next chapter, we will learn about Docker and Deployment. You will containerize your application with Docker, create production-ready builds, and set up automated deployment pipelines.

Chapter 30: Docker and Deployment

What You Will Learn

- What Docker is and why it matters (the shipping container analogy).
- How to write a Dockerfile with multi-stage builds.
- How to build and run Docker images with `docker build` and `docker run`.
- How to use Docker Compose to run your app with a database.
- How to pass environment variables to Docker containers.
- How to add health checks to your containers.
- How to package your Spring Boot app with `mvn package`.
- How Spring Boot layers work for faster Docker builds.
- What deployment options are available for Spring Boot apps.
- How to set up a basic CI/CD pipeline with GitHub Actions.

Why This Chapter Matters

You have built a Spring Boot application. It works perfectly on your laptop. But your laptop is not a production server. You need to get your application running on a server where real users can access it.

Here is the problem. You tell your operations team, "Just install Java 17, Maven, set these 12 environment variables, configure the database connection, and run this command." They respond, "It does not work on our server."

This is the classic "it works on my machine" problem. Docker solves it.

Docker packages your application and everything it needs (Java, libraries, configuration) into a single container that runs the same way everywhere. Your laptop, your colleague's laptop, a staging server, a production server. Same container, same behavior.

This chapter teaches you how to containerize your Spring Boot application and deploy it to the real world.

30.1 What Is Docker? The Shipping Container Analogy

Before shipping containers were invented, loading cargo onto a ship was chaos. Each item had a different shape and size. Barrels, crates, bags, furniture. Workers had to figure out how to pack everything together. It was slow, expensive, and things got damaged.

Then someone invented the shipping container. A standardized metal box. Any cargo goes inside the container. The container fits on any ship, any truck, any train. Nobody cares what is inside. They just move the container.

```
Before Containers (Shipping):        After Containers:

+--ship---------+                    +--ship---------+
| [barrel]      |                    | [container]   |
|   [crate]     |  Messy!            | [container]   |  Clean!
| [bag] [chair] |  Slow!             | [container]   |  Fast!
|  [box]        |  Fragile!          | [container]   |  Reliable!
+---------------+                    +---------------+
```

Docker works the same way for software:

```
Before Docker (Software):            After Docker:

Server needs:                        Server needs:
- Java 17                            - Docker
- Maven
- PostgreSQL driver                  That is it!
- 12 environment variables
- Specific OS libraries              Everything else is inside
- Correct file permissions           the container.
- Prayer

"It works on my machine!"            "It works everywhere."
```

Docker Vocabulary

Term	Analogy	Description
Image	Blueprint	A template containing your app and all its dependencies
Container	Running instance	A running copy of an image (like a process)
Dockerfile	Recipe	Instructions for building an image
Docker Hub	App Store	A public repository for sharing images
Volume	External storage	Persistent data storage outside the container

```
Dockerfile        docker build       Image          docker run        Container
(recipe)    ------>  (cook)   ------>  (dish)  ------>  (serve)  ------> (running)

You write          Docker reads       A packaged      Docker starts     Your app
instructions       instructions       snapshot        a new instance    is running!
                   and creates        of your app
                   the image
```

30.2 Installing Docker

Before we continue, make sure Docker is installed on your machine.

Verify Installation

```bash
docker --version
```

Output:

```
Docker version 24.0.7, build afdd53b
```

```bash
docker compose version
```

Output:

```
Docker Compose version v2.23.0
```

If Docker is not installed, download Docker Desktop from `https://docs.docker.com/get-docker/`.

30.3 Building Your Spring Boot App with Maven

Before dockerizing, let us package our application into a JAR file.

The mvn package Command

```bash
./mvnw clean package -DskipTests
```

This command:

1. clean: Deletes the `target/` directory (starts fresh).
2. package: Compiles your code, runs tests, and creates a JAR file.
3. -DskipTests: Skips tests (for faster builds during development).

Output:

```
[INFO] Building BookStore 1.0.0
[INFO] --- spring-boot-maven-plugin:3.4.0:repackage ---
[INFO] Replacing main artifact with repackaged archive
[INFO] BUILD SUCCESS
[INFO] Total time: 8.234 s
```

The JAR file is created at:

```
target/bookstore-1.0.0.jar
```

Running the JAR

```bash
java -jar target/bookstore-1.0.0.jar
```

This starts your application without Maven or an IDE. The JAR contains everything: your code, all dependencies, and an embedded Tomcat server.

```
What is inside the JAR?

bookstore-1.0.0.jar
  |
  +-- BOOT-INF/
  |     +-- classes/          Your compiled code
  |     +-- lib/              All dependencies (100+ JARs)
  |
  +-- META-INF/
  |     +-- MANIFEST.MF       Points to the main class
  |
  +-- org/springframework/boot/loader/
        +-- JarLauncher.class  Starts the application
```

30.4 Writing a Dockerfile

A Dockerfile is a text file with instructions for building a Docker image. Think of it as a recipe.

Simple Dockerfile

```dockerfile
# Dockerfile

# Step 1: Start with a Java 17 base image
FROM eclipse-temurin:17-jre                          # 1

# Step 2: Set the working directory inside the container
WORKDIR /app                                          # 2

# Step 3: Copy the JAR file into the container
COPY target/bookstore-1.0.0.jar app.jar              # 3

# Step 4: Expose the port your app uses
EXPOSE 8080                                           # 4

# Step 5: Define the command to run the application
ENTRYPOINT ["java", "-jar", "app.jar"]               # 5
```

Line-by-line explanation:

- Line 1: `FROM eclipse-temurin:17-jre` starts with a pre-built image that has Java 17 installed. Think of it as "start with a computer that already has Java."

- Line 2: `WORKDIR /app` creates and switches to the `/app` directory inside the container.
- Line 3: `COPY target/bookstore-1.0.0.jar app.jar` copies your JAR from your computer into the container.
- Line 4: `EXPOSE 8080` documents that the app listens on port 8080. This is informational; it does not actually open the port.
- Line 5: `ENTRYPOINT` defines the command that runs when the container starts.

Multi-Stage Dockerfile (Recommended)

The simple Dockerfile requires you to build the JAR first. A multi-stage Dockerfile builds the JAR inside Docker, so the build is reproducible.

```dockerfile
# Dockerfile (Multi-Stage Build)

# ===== Stage 1: Build =====
FROM eclipse-temurin:17-jdk AS builder                # 1

WORKDIR /app

# Copy Maven wrapper and pom.xml first (for caching)
COPY mvnw .                                           # 2
COPY .mvn .mvn                                        # 3
COPY pom.xml .                                        # 4

# Download dependencies (cached if pom.xml unchanged)
RUN chmod +x mvnw && ./mvnw dependency:resolve        # 5

# Copy source code
COPY src src                                          # 6

# Build the application
RUN ./mvnw clean package -DskipTests                  # 7

# ===== Stage 2: Run =====
FROM eclipse-temurin:17-jre                           # 8

WORKDIR /app

# Copy only the JAR from the build stage
COPY --from=builder /app/target/*.jar app.jar          # 9

# Create a non-root user for security
RUN addgroup --system appgroup && \                   # 10
    adduser --system --ingroup appgroup appuser
USER appuser                                          # 11

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why two stages?

```
Stage 1 (Builder):                  Stage 2 (Final):
+-------------------------+        +-------------------------+
| JDK (large, 400MB)      |        | JRE (small, 200MB)      |
| Maven                   |        | app.jar                  |
| Source code              |        |                          |
| Dependencies             |        | That is it!              |
| Compiled classes         |        |                          |
| app.jar                  |        |                          |
+-------------------------+        +-------------------------+
   ~800MB                              ~250MB

The final image is MUCH smaller because it only
contains what is needed to RUN the app.
```

Key lines explained:

- Line 1: Use the JDK (Java Development Kit) for building. `AS builder` names this stage.
- Line 2-4: Copy Maven files first. Docker caches layers, so if `pom.xml` does not change, dependencies are not re-downloaded.
- Line 5: Download all dependencies. This layer is cached.
- Line 6: Copy your source code. This changes frequently, so it is a separate layer.
- Line 7: Build the JAR inside Docker.
- Line 8: Start a new stage with only the JRE (Java Runtime Environment). Much smaller.
- Line 9: `COPY --from=builder` copies the JAR from the first stage to the second stage.
- Line 10-11: Create a non-root user. Never run containers as root in production.

Docker Caching Strategy

```
Layers that change RARELY (cached):
  FROM eclipse-temurin:17-jdk
  COPY pom.xml .
  RUN ./mvnw dependency:resolve    <-- Only re-runs if pom.xml changes

Layers that change OFTEN (rebuilt):
  COPY src src                     <-- Changes with every code change
  RUN ./mvnw clean package         <-- Rebuilds the JAR

Result: Fast rebuilds! Only your code is recompiled.
```

30.5 Building and Running Docker Images

Building the Image

```bash
docker build -t bookstore:1.0 .
```

- `-t bookstore:1.0`: Tags the image with a name and version.
- `.`: Use the current directory (where the Dockerfile is).

Output:

```
[+] Building 45.2s (14/14) FINISHED
 => [builder 1/6] FROM eclipse-temurin:17-jdk
 => [builder 2/6] COPY mvnw .
 => [builder 3/6] COPY .mvn .mvn
 => [builder 4/6] COPY pom.xml .
 => [builder 5/6] RUN ./mvnw dependency:resolve
 => [builder 6/6] COPY src src
 => [builder 7/7] RUN ./mvnw clean package -DskipTests
 => [stage-1 1/3] FROM eclipse-temurin:17-jre
 => [stage-1 2/3] COPY --from=builder /app/target/*.jar app.jar
 => [stage-1 3/3] RUN addgroup --system appgroup ...
 => exporting to image
 => => naming to docker.io/library/bookstore:1.0
```

Running the Container

```bash
docker run -p 8080:8080 bookstore:1.0
```

- `-p 8080:8080`: Maps port 8080 on your machine to port 8080 in the container.

```
Port Mapping:

Your Machine                    Docker Container
+------------------+           +------------------+
| localhost:8080   |---------->| container:8080   |
| (you access      |   -p      | (app listens     |
|  this port)      | mapping   |  on this port)   |
+------------------+           +------------------+
```

Now visit `http://localhost:8080/api/books` and your app is running inside Docker.

Running in the Background

```bash
# Run in detached mode (background)
docker run -d -p 8080:8080 --name bookstore bookstore:1.0
```

- `-d`: Detached mode (runs in background).
- `--name bookstore`: Give the container a name.

Useful Docker Commands

```bash
# List running containers
docker ps

# View container logs
docker logs bookstore

# Follow logs in real-time
docker logs -f bookstore

# Stop a container
docker stop bookstore

# Remove a container
docker rm bookstore

# List all images
docker images

# Remove an image
docker rmi bookstore:1.0
```

30.6 Passing Environment Variables

In Chapter 29, you learned about externalized configuration. Docker makes passing environment variables easy.

Using -e Flag

```bash
docker run -d -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e DB_HOST=prod-database.example.com \
  -e DB_PORT=5432 \
```

```
  -e DB_USERNAME=bookstore_user \
  -e DB_PASSWORD=superSecretPassword \
  -e JWT_SECRET=myProductionJwtSecret \
  --name bookstore \
  bookstore:1.0
```

Using an Environment File

For many variables, use a file:

```bash
# docker.env
SPRING_PROFILES_ACTIVE=prod
DB_HOST=prod-database.example.com
DB_PORT=5432
DB_USERNAME=bookstore_user
DB_PASSWORD=superSecretPassword
JWT_SECRET=myProductionJwtSecret
```

```bash
docker run -d -p 8080:8080 \
  --env-file docker.env \
  --name bookstore \
  bookstore:1.0
```

> Important: Add `docker.env` to your `.gitignore`. Never commit environment files with secrets.

JVM Options

You can pass JVM options through environment variables:

```bash
docker run -d -p 8080:8080 \
  -e JAVA_OPTS="-Xms256m -Xmx512m" \
  --name bookstore \
  bookstore:1.0
```

Update your ENTRYPOINT to use JAVA_OPTS:

```dockerfile
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

30.7 Docker Compose

Docker Compose lets you define and run multiple containers together. Your application needs a database? Define both in one file.

docker-compose.yml

```yaml
# docker-compose.yml

version: '3.8'                                          # 1

services:
  # ===== Database Service =====
  database:                                              # 2
    image: postgres:16                                   # 3
    container_name: bookstore-db
    environment:                                         # 4
      POSTGRES_DB: bookstore
      POSTGRES_USER: bookstore_user
      POSTGRES_PASSWORD: bookstore_password
    ports:
      - "5432:5432"                                      # 5
    volumes:
      - db-data:/var/lib/postgresql/data                 # 6
    healthcheck:                                         # 7
      test: ["CMD-SHELL",
             "pg_isready -U bookstore_user"]
      interval: 10s
      timeout: 5s
      retries: 5

  # ===== Application Service =====
  app:                                                   # 8
    build: .                                             # 9
    container_name: bookstore-app
    ports:
      - "8080:8080"
    environment:                                         # 10
      SPRING_PROFILES_ACTIVE: prod
      DB_HOST: database                                  # 11
      DB_PORT: 5432
      DB_USERNAME: bookstore_user
      DB_PASSWORD: bookstore_password
      JWT_SECRET: myDockerComposeSecret
    depends_on:                                          # 12
      database:
        condition: service_healthy                       # 13
    healthcheck:                                         # 14
      test: ["CMD", "curl", "-f",
             "http://localhost:8080/actuator/health"]
```

```
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s                                # 15

volumes:                                               # 16
  db-data:
```

Line-by-line explanation:

- Line 1: Docker Compose file version.
- Line 2: Define a service called "database".
- Line 3: Use the official PostgreSQL 16 image.
- Line 4: Set environment variables for PostgreSQL (database name, user, password).
- Line 5: Map port 5432 so you can connect from your machine.
- Line 6: Mount a volume for persistent data. Without this, data is lost when the container stops.
- Line 7: Health check that runs `pg_isready` every 10 seconds.
- Line 8: Define the application service.
- Line 9: `build: .` means "build the image from the Dockerfile in the current directory."
- Line 10: Environment variables for the Spring Boot app.
- Line 11: `DB_HOST: database` uses the service name as the hostname. Docker networking resolves this automatically.
- Line 12: `depends_on` means the app waits for the database.
- Line 13: `condition: service_healthy` waits until the database health check passes.
- Line 14: Health check for the Spring Boot app using the Actuator endpoint.
- Line 15: `start_period` gives the app 40 seconds to start before health checks begin.
- Line 16: Named volume for database persistence.

```
Docker Compose Network:

+---docker-compose-network-------------------+
|                                             |
|  +----------+       +------------------+  |
|  | database |       | app              |  |
|  | :5432    |<------| :8080            |  |
|  |          |  DNS  |                  |  |
|  | Postgres |  name | Spring Boot      |  |
|  +----------+  =    +------------------+  |
|               "database"                   |
+---------------------------------------------+

External Access:
  localhost:8080 -> app container
```

```
  localhost:5432 -> database container
```

Docker Compose Commands

```bash
# Start all services
docker compose up

# Start in background
docker compose up -d

# Build and start (rebuild images)
docker compose up --build

# View logs
docker compose logs

# Follow logs for a specific service
docker compose logs -f app

# Stop all services
docker compose down

# Stop and remove volumes (deletes database data!)
docker compose down -v

# Check status
docker compose ps
```

Testing with Docker Compose

```bash
# Start everything
docker compose up -d --build

# Wait for the app to be healthy
docker compose ps

# Test the API
curl http://localhost:8080/api/books

# Check health
curl http://localhost:8080/actuator/health

# View app logs
docker compose logs -f app

# Shut down
docker compose down
```

30.8 Health Checks in Docker

Health checks tell Docker whether your container is working properly. Without health checks, Docker only knows if the process is running, not if it is actually healthy.

```
Without Health Check:                With Health Check:

Container Status:                   Container Status:
  "Running"                           "Healthy" or "Unhealthy"

Docker knows:                       Docker knows:
  "Process is alive"                  "App is responding to requests"

Problem: App might be frozen         Docker can restart unhealthy
but process is still running.        containers automatically.
```

Health Check in Dockerfile

```dockerfile
# Add to your Dockerfile
HEALTHCHECK --interval=30s \
            --timeout=10s \
            --retries=3 \
            --start-period=40s \
    CMD curl -f http://localhost:8080/actuator/health || exit 1
```

Make sure curl is available in your image. If not, use wget or a Java-based check:

```dockerfile
# Alternative: Use wget (available in most images)
HEALTHCHECK --interval=30s \
            --timeout=10s \
            --retries=3 \
    CMD wget -qO- http://localhost:8080/actuator/health || exit 1
```

Health Check Parameters

Parameter	Default	Description
`--interval`	30s	Time between health checks

`--timeout`	30s	Maximum time for a health check to complete
`--retries`	3	How many failures before marking "unhealthy"
`--start-period`	0s	Grace period for the app to start

30.9 Spring Boot Layers for Faster Docker Builds

Spring Boot 3 supports a layered JAR format that makes Docker builds much faster by taking advantage of Docker's layer caching.

The Problem

Every time you change one line of code, Docker rebuilds the entire JAR layer, even though your dependencies (100+ MB) have not changed.

The Solution: Layered JARs

Spring Boot splits the JAR into layers:

```
Layer 1: dependencies         (rarely changes)     ~100 MB
Layer 2: spring-boot-loader   (rarely changes)     ~1 MB
Layer 3: snapshot-dependencies (sometimes changes)  ~5 MB
Layer 4: application          (changes often)       ~1 MB

Docker caches layers 1-3. Only layer 4 is rebuilt.
Result: Builds go from 2 minutes to 5 seconds!
```

Dockerfile with Layers

```dockerfile
# Dockerfile with Spring Boot Layers

# ===== Stage 1: Build =====
FROM eclipse-temurin:17-jdk AS builder
WORKDIR /app
COPY mvnw .
COPY .mvn .mvn
```

```
COPY pom.xml .
RUN chmod +x mvnw && ./mvnw dependency:resolve
COPY src src
RUN ./mvnw clean package -DskipTests

# ===== Stage 2: Extract Layers =====
FROM eclipse-temurin:17-jdk AS extractor                # 1
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract      # 2

# ===== Stage 3: Run =====
FROM eclipse-temurin:17-jre
WORKDIR /app

# Copy layers in order (least to most frequently changing)
COPY --from=extractor /app/dependencies/ ./              # 3
COPY --from=extractor /app/spring-boot-loader/ ./        # 4
COPY --from=extractor /app/snapshot-dependencies/ ./     # 5
COPY --from=extractor /app/application/ ./               # 6

RUN addgroup --system appgroup && \
    adduser --system --ingroup appgroup appuser
USER appuser

EXPOSE 8080

ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]  # 7
```

- Line 1-2: Extract the JAR into layers using Spring Boot's built-in tool.
- Line 3-6: Copy each layer separately. Docker caches unchanged layers.
- Line 7: Use `JarLauncher` instead of `java -jar` since the app is no longer a single JAR.

30.10 Deployment Options

There are many ways to deploy a Spring Boot application. Here are the most common:

```
+-----------------------------------------------------+
|               Deployment Options                    |
+-----------------------------------------------------+
|                                                     |
|  Simple:                                            |
|    1. JAR on a VPS (DigitalOcean, Linode)          |
|    2. Docker on a VPS                               |
|                                                     |
|  Platform as a Service (PaaS):                      |
|    3. Heroku / Railway / Render                     |
```

```
|    4. AWS Elastic Beanstalk                      |
|    5. Google App Engine                          |
|    6. Azure App Service                          |
|                                                  |
|  Container Orchestration:                        |
|    7. AWS ECS / Fargate                          |
|    8. Google Cloud Run                           |
|    9. Kubernetes (EKS, GKE, AKS)                |
|                                                  |
|  Complexity ------>                               |
|  Simple                           Enterprise    |
+--------------------------------------------------+
```

Option 1: JAR on a VPS (Simplest)

```bash
# On your server
# 1. Install Java
sudo apt update && sudo apt install -y openjdk-17-jre-headless

# 2. Copy the JAR to the server
scp target/bookstore-1.0.0.jar user@server:/opt/bookstore/

# 3. Run the application
java -jar /opt/bookstore/bookstore-1.0.0.jar \
  --spring.profiles.active=prod
```

Option 2: Docker on a VPS

```bash
# On your server
# 1. Install Docker
curl -fsSL https://get.docker.com | sh

# 2. Copy docker-compose.yml and Dockerfile
scp docker-compose.yml Dockerfile user@server:/opt/bookstore/

# 3. Start the application
cd /opt/bookstore
docker compose up -d --build
```

Option 3: Cloud Run (Serverless Containers)

```bash
# Build and push to Google Container Registry
gcloud builds submit --tag gcr.io/my-project/bookstore

# Deploy to Cloud Run
gcloud run deploy bookstore \
  --image gcr.io/my-project/bookstore \
  --port 8080 \
  --allow-unauthenticated
```

30.11 CI/CD with GitHub Actions

CI/CD (Continuous Integration / Continuous Deployment) automates building, testing, and deploying your application every time you push code.

```
Developer pushes code
        |
        v
+---GitHub Actions-------------------+
|                                    |
|  1. Checkout code                  |
|  2. Set up Java 17                 |
|  3. Run tests                      |
|  4. Build Docker image             |
|  5. Push to registry               |
|  6. Deploy to server               |
|                                    |
+------------------------------------+
        |
        v
Application is live!
```

Basic CI Pipeline

```yaml
# .github/workflows/ci.yml

name: CI Pipeline                                       # 1

on:                                                     # 2
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

```
jobs:
  build-and-test:                                       # 3
    runs-on: ubuntu-latest                              # 4

    steps:
      - name: Checkout code                             # 5
        uses: actions/checkout@v4

      - name: Set up Java 17                            # 6
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'maven'                                # 7

      - name: Run tests                                 # 8
        run: ./mvnw test

      - name: Build application                         # 9
        run: ./mvnw clean package -DskipTests
```

Line-by-line explanation:

- Line 1: Name of the workflow (shown in GitHub UI).
- Line 2: Trigger on pushes to `main` and on pull requests targeting `main`.
- Line 3: Define a job called "build-and-test".
- Line 4: Run on the latest Ubuntu virtual machine.
- Line 5: Check out the repository code.
- Line 6: Install Java 17 (Temurin distribution).
- Line 7: Cache Maven dependencies for faster builds.
- Line 8: Run all tests.
- Line 9: Build the application JAR.

CI/CD Pipeline with Docker

```yaml
# .github/workflows/ci-cd.yml

name: CI/CD Pipeline

on:
  push:
    branches: [ main ]

jobs:
  test:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'maven'

      - name: Run tests
        run: ./mvnw test

  build-and-push:
    needs: test                                          # 1
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Log in to Docker Hub                       # 2
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}      # 3
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Build and push Docker image                # 4
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ${{ secrets.DOCKER_USERNAME }}/bookstore:latest
            ${{ secrets.DOCKER_USERNAME }}/bookstore:${{ github.sha }}

  deploy:
    needs: build-and-push                                # 5
    runs-on: ubuntu-latest

    steps:
      - name: Deploy to server                           # 6
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            docker pull ${{ secrets.DOCKER_USERNAME }}/bookstore:latest
            docker compose down
            docker compose up -d
```

- Line 1: `needs: test` means this job only runs if tests pass.
- Line 2-3: Log in to Docker Hub using secrets stored in GitHub.

- Line 4: Build the Docker image and push it to Docker Hub.
- Line 5: Deploy only after the image is built and pushed.
- Line 6: SSH into the server and pull the latest image.

Setting Up GitHub Secrets

In your GitHub repository, go to Settings > Secrets and variables > Actions, and add:

Secret	Value
`DOCKER_USERNAME`	Your Docker Hub username
`DOCKER_PASSWORD`	Your Docker Hub password or access token
`SERVER_HOST`	Your server's IP address
`SERVER_USER`	SSH username
`SERVER_SSH_KEY`	SSH private key

```
GitHub Actions Flow:

   Push to main
       |
       v
  +----+----+
  |  Test   |  Run ./mvnw test
  |  Job    |  All tests must pass
  +----+----+
       |
       v (only if tests pass)
  +----+----+
  |  Build  |  docker build
  |  & Push |  docker push
  +----+----+
       |
       v (only if push succeeds)
  +----+----+
  | Deploy  |  SSH to server
  |  Job    |  docker pull & restart
  +---------+
```

30.12 A .dockerignore File

Just like `.gitignore` keeps files out of Git, `.dockerignore` keeps files out of Docker builds.

```dockerignore
# .dockerignore

# Build artifacts
target/
!target/*.jar

# IDE files
.idea/
.vscode/
*.iml

# Git
.git/
.gitignore

# Docker
docker-compose.yml
Dockerfile

# Environment files (NEVER include in image!)
.env
*.env

# Documentation
*.md
docs/

# Test files
src/test/
```

Without `.dockerignore`, Docker copies everything into the build context, making builds slower and images larger.

Common Mistakes

Mistake 1: Running Containers as Root

```dockerfile
# WRONG: Running as root (security risk)
FROM eclipse-temurin:17-jre
COPY app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
# Runs as root by default!
```

```dockerfile
# CORRECT: Create and use a non-root user
FROM eclipse-temurin:17-jre
RUN addgroup --system app && adduser --system --ingroup app app
USER app
COPY app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Mistake 2: Using latest Tag in Production

```bash
# WRONG: "latest" can change without warning
docker pull bookstore:latest
```

```bash
# CORRECT: Use specific versions
docker pull bookstore:1.0.0
# OR use the Git commit SHA
docker pull bookstore:abc123f
```

Mistake 3: Storing Data Inside Containers

```yaml
# WRONG: Data is lost when container is removed
services:
  database:
    image: postgres:16
    # No volume! Data is gone when container stops.
```

```yaml
# CORRECT: Use volumes for persistent data
services:
  database:
    image: postgres:16
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:  # Named volume persists across restarts
```

Mistake 4: No Health Checks

```yaml
# WRONG: No health check
services:
```

```
  app:
    build: .
    ports:
      - "8080:8080"
    # Docker has no idea if the app is healthy
```

```yaml
# CORRECT: Add a health check
services:
  app:
    build: .
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "-f",
             "http://localhost:8080/actuator/health"]
      interval: 30s
      retries: 3
```

Mistake 5: Not Using Multi-Stage Builds

```dockerfile
# WRONG: JDK in final image (400MB larger than needed)
FROM eclipse-temurin:17-jdk
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
# Image: ~600MB (includes compiler, not needed at runtime)
```

```dockerfile
# CORRECT: Use JRE in final image
FROM eclipse-temurin:17-jre
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
# Image: ~250MB (only runtime, much smaller)
```

Best Practices

1. Use multi-stage builds. Keep your final image small by separating the build stage from the run stage.
2. Never run containers as root. Create a dedicated user in your Dockerfile.
3. Use specific version tags. Avoid `latest` in production. Use semantic versions or Git SHAs.
4. Add health checks. Docker and orchestrators need to know if your app is healthy.

5. Use Docker Compose for local development. Define your app, database, and other services in one file.
6. Use volumes for persistent data. Database data must survive container restarts.
7. Add a `.dockerignore` file. Keep builds fast and images clean.
8. Use CI/CD pipelines. Automate testing, building, and deployment with GitHub Actions.
9. Set memory limits. Use `-Xms` and `-Xmx` JVM flags to control memory usage inside containers.
10. Keep secrets out of images. Use environment variables or secrets managers. Never bake secrets into Docker images.

Quick Summary

In this chapter, you learned how to containerize and deploy your Spring Boot application. You started with the Docker shipping container analogy and understood why Docker solves the "it works on my machine" problem. You wrote Dockerfiles, starting with a simple version and progressing to optimized multi-stage builds. You built and ran Docker images with `docker build` and `docker run`. You used Docker Compose to run your application with a PostgreSQL database. You passed environment variables to configure your containerized app. You added health checks so Docker knows when your app is truly healthy. You learned about Spring Boot's layered JAR format for faster Docker builds. You explored deployment options from simple VPS hosting to cloud platforms. Finally, you set up a CI/CD pipeline with GitHub Actions to automate the entire build-test-deploy process.

Key Points

Concept	Description
Docker	Packages your app and dependencies into portable containers
Dockerfile	Instructions for building a Docker image
Multi-stage build	Two-phase build: compile in stage 1, run in stage 2 (smaller image)
`docker build`	Creates an image from a Dockerfile

`docker run`	Starts a container from an image
Docker Compose	Defines and runs multi-container applications
`-p 8080:8080`	Maps a host port to a container port
`-e VAR=value`	Passes environment variables to a container
Volume	Persistent storage that survives container restarts
Health check	Lets Docker know if your app is actually healthy
`mvn package`	Builds a runnable JAR file
Spring Boot layers	Splits the JAR for faster Docker caching
GitHub Actions	Automates CI/CD pipelines (test, build, deploy)
`.dockerignore`	Excludes files from the Docker build context

Practice Questions

1. Explain the Docker shipping container analogy. How does Docker solve the "it works on my machine" problem?
2. What is the difference between a Docker image and a Docker container? How do you create each one?
3. Why should you use multi-stage builds in your Dockerfile? What are the two stages, and what goes in each one?
4. In a `docker-compose.yml` file, how does the Spring Boot app connect to the database? Why can you use the service name as the hostname?
5. What is the purpose of a health check in Docker? What happens to a container that fails its health check?

Exercises

Exercise 1: Dockerize a Simple App

Create a simple Spring Boot application with a `/hello` endpoint. Write a multi-stage Dockerfile, build the image, and run it. Verify you can access the endpoint at `http://localhost:8080/hello`.

Exercise 2: Docker Compose with Database

Extend Exercise 1 by adding a PostgreSQL database using Docker Compose. Create a `Book` entity and REST API. Configure the app to connect to the PostgreSQL container. Verify that:

- The app starts after the database is healthy.
- You can create and retrieve books through the API.
- Data persists after stopping and restarting the containers (using volumes).

Exercise 3: CI/CD Pipeline

Set up a GitHub Actions workflow that:

1. Runs tests on every push to `main` and on pull requests.
2. Builds a Docker image when tests pass.
3. Tags the image with the Git commit SHA.
4. Pushes the image to Docker Hub (you will need a free Docker Hub account).

Verify by pushing a commit and checking that the workflow completes successfully.

What Is Next?

Congratulations! You have completed the main chapters of this book. You now know how to build, test, document, monitor, configure, and deploy Spring Boot applications. You have the skills to build production-ready applications from scratch.

Check out the appendices for additional topics like WebSockets, a complete project walkthrough, and a glossary of terms. Keep building, keep learning, and keep shipping great software.

Appendix A: Real-Time Communication with WebSocket

What You Will Learn

- What WebSocket is and how it differs from HTTP.
- Why WebSocket matters for real-time applications.
- What the STOMP protocol is and why Spring uses it.
- How to enable WebSocket support in Spring Boot.
- How to use @MessageMapping and @SendTo to handle messages.
- How to use SimpMessagingTemplate to send messages from anywhere in your code.
- How to build an HTML and JavaScript client that connects to your WebSocket server.
- How to build a live notification system.

Why This Chapter Matters

Imagine you are waiting for an important package. You have two choices. You could call the delivery company every five minutes and ask, "Is my package here yet?" That would be exhausting and wasteful. Or you could give them your phone number and say, "Call me when it arrives." That is much better.

Traditional HTTP works like the first approach. The browser keeps asking the server, "Any updates?" over and over. WebSocket works like the second approach. The server can call the browser whenever something happens. No repeated asking. No wasted effort.

If you want to build chat applications, live dashboards, real-time notifications, or collaborative tools, you need WebSocket. This appendix teaches you how to use WebSocket with Spring Boot so your applications can communicate in real time.

A.1 The Problem with HTTP for Real-Time Communication

In Chapters 9 and 10, you learned about REST APIs. The browser sends a request, and the server sends back a response. This is called the request-response model.

```
HTTP Request-Response Model:

Browser                          Server
  |                                |
  |--- GET /api/notifications ---->|
  |<--- Response: [] --------------|
  |                                |
  |   (5 seconds later)            |
  |--- GET /api/notifications ---->|
  |<--- Response: [] --------------|
  |                                |
  |   (5 seconds later)            |
  |--- GET /api/notifications ---->|
  |<--- Response: [1 new] ---------|
  |                                |
```

This approach is called polling. The browser asks the server repeatedly. There are two problems with polling:

1. Wasted resources. Most of the time, the server has nothing new. But the browser keeps asking anyway. This wastes bandwidth, server processing power, and battery life on mobile devices.
2. Delays. If you poll every 5 seconds, a notification that arrives 1 second after a poll will not reach the user for another 4 seconds. For a chat app, that delay is unacceptable.

Think of polling like a child in the back seat asking, "Are we there yet?" every 30 seconds. It is annoying for everyone, and the answer is usually "No."

A.2 What Is WebSocket?

WebSocket is a communication protocol that provides a persistent, two-way connection between the browser and the server.

> Protocol: A set of rules that two systems follow to communicate with each other. HTTP is a protocol. WebSocket is a different protocol.

With WebSocket, the browser and server open a connection once, and that connection stays open. Either side can send messages at any time without waiting for the other to ask.

```
WebSocket Model:

Browser                          Server
  |                                |
  |--- WebSocket Handshake ------->|
  |<--- Connection Established ----|
  |                                |
  |   (connection stays open)      |
  |                                |
  |<--- Message: "New order!" -----|
  |                                |
  |--- Message: "Got it" --------->|
  |                                |
  |<--- Message: "User joined" ----|
  |                                |
```

HTTP vs WebSocket: A Comparison

Feature	HTTP	WebSocket
Connection	Opens and closes for each request	Opens once, stays open
Direction	Client asks, server answers	Both can send anytime
Overhead	Headers sent with every request	Headers only during handshake
Best for	Loading pages, REST APIs	Chat, live updates, games
Analogy	Sending letters back and forth	Having a phone call

The Phone Call Analogy

HTTP is like sending letters. You write a letter (request), put it in the mailbox, wait for a reply, read the reply, and repeat. Every letter needs an envelope, a stamp, and an address. That is a lot of overhead.

WebSocket is like a phone call. You dial once (handshake), the connection opens, and then you can talk back and forth freely. No envelopes, no stamps, no waiting for the mail carrier. When you are done, you hang up.

```
Letters (HTTP):
  You ----[envelope + letter]----> Friend
  You <---[envelope + reply]------ Friend
  You ----[envelope + letter]----> Friend
  You <---[envelope + reply]------ Friend
  (each exchange needs a new envelope)

Phone Call (WebSocket):
  You ----[dial]----> Friend
  You: "Hey!"
  Friend: "Hi there!"
  You: "What's up?"
  Friend: "Not much."
  (one connection, many messages)
```

A.3 How WebSocket Works Under the Hood

WebSocket starts with an HTTP request. The browser sends a special HTTP request called an upgrade request that says, "I want to switch from HTTP to WebSocket." If the server supports WebSocket, it agrees, and the connection upgrades.

```
WebSocket Connection Lifecycle:

1. HANDSHAKE (uses HTTP)
   Browser: "Hey server, can we switch to WebSocket?"
   Server:  "Sure, switching now."

2. OPEN CONNECTION
   Both sides can send messages freely.
   Messages are small frames, not full HTTP requests.

3. COMMUNICATION
   Browser --> Server: messages
   Server --> Browser: messages
   (no headers, no overhead, just data)

4. CLOSE
   Either side: "I'm done. Closing connection."
   Connection closes cleanly.
```

The handshake uses HTTP, so WebSocket works with existing web infrastructure like firewalls and proxies. After the handshake, the protocol switches to WebSocket, which is much lighter than HTTP.

A.4 What Is STOMP?

Raw WebSocket is like a blank phone call. You can say anything, but there are no rules about the format of your messages. In a real application, you need structure.

STOMP stands for Simple Text Oriented Messaging Protocol. It adds structure to WebSocket messages, like adding a topic and format to a conversation.

> STOMP: A simple messaging protocol that works on top of WebSocket. It defines commands like CONNECT, SEND, SUBSCRIBE, and MESSAGE.

Think of STOMP like this. Raw WebSocket is like an open phone line where anyone can say anything. STOMP is like a walkie-talkie system with channels. You tune into channel 5 to hear weather updates, channel 7 for traffic updates, and channel 10 for emergency alerts. Each message has a clear destination and purpose.

```
Without STOMP (raw WebSocket):
  "new notification for user 42"
  (What does this mean? Who should get it? What format?)

With STOMP:
  DESTINATION: /topic/notifications
  CONTENT-TYPE: application/json
  BODY: {"message": "New order received", "userId": 42}
  (Clear destination, clear format, easy to handle)
```

Spring Boot uses STOMP because it provides:

1. Destinations - Messages go to specific topics (like channels).
2. Subscriptions - Clients subscribe to topics they care about.
3. Message format - Messages have headers and a body, just like HTTP.
4. Built-in support - Spring has excellent STOMP integration.

A.5 Setting Up WebSocket in Spring Boot

Let us build a real-time notification system step by step. First, create a new Spring Boot project or add WebSocket to an existing one.

Step 1: Add the Dependency

Open your `pom.xml` and add the WebSocket starter:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

This single dependency brings in everything you need for WebSocket support with STOMP.

Step 2: Configure WebSocket

Create a configuration class that enables WebSocket messaging:

```java
package com.example.websocketdemo.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").withSockJS();
    }
}
```

Output when the application starts:

```
INFO  --- WebSocket STOMP endpoint registered: /ws
INFO  --- Simple message broker started: [/topic]
```

Line-by-line explanation:

- `@Configuration` tells Spring this class contains configuration settings.
- `@EnableWebSocketMessageBroker` enables WebSocket message handling with STOMP support. This is the key annotation that activates the entire WebSocket infrastructure.

- `WebSocketMessageBrokerConfigurer` is an interface that provides methods to customize WebSocket behavior.
- `configureMessageBroker(MessageBrokerRegistry config)` sets up the message routing rules.
- `config.enableSimpleBroker("/topic")` creates an in-memory message broker. Any message sent to a destination starting with `/topic` will be forwarded to all subscribed clients. Think of this as creating a bulletin board that anyone can read.
- `config.setApplicationDestinationPrefixes("/app")` sets a prefix for messages that go to your controller methods. When a client sends a message to `/app/something`, Spring routes it to a `@MessageMapping("/something")` method.
- `registerStompEndpoints(StompEndpointRegistry registry)` registers the URL where clients connect.
- `registry.addEndpoint("/ws")` creates a WebSocket endpoint at `/ws`. Clients will connect to `ws://localhost:8080/ws`.
- `.withSockJS()` enables SockJS fallback. If a browser does not support WebSocket (rare today), SockJS uses HTTP polling as a backup.

```
Message Routing:

Client sends to /app/notify
       |
       v
  +--------------------+
  | @MessageMapping    |
  | ("/notify")        |
  | Controller method  |
  +--------------------+
       |
       v
  Sends to /topic/notifications
       |
       v
  +--------------------+
  | Simple Broker      |
  | /topic/*           |
  +--------------------+
       |
       v
  All subscribed clients receive the message
```

A.6 Creating a WebSocket Controller

Now create a controller that handles incoming WebSocket messages:

```java
package com.example.websocketdemo.controller;

import com.example.websocketdemo.model.Notification;
import com.example.websocketdemo.model.NotificationRequest;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.SendTo;
import org.springframework.stereotype.Controller;

import java.time.LocalDateTime;

@Controller
public class NotificationController {

    @MessageMapping("/notify")
    @SendTo("/topic/notifications")
    public Notification sendNotification(NotificationRequest request) {
        return new Notification(
            request.getMessage(),
            request.getSender(),
            LocalDateTime.now().toString()
        );
    }
}
```

Line-by-line explanation:

- `@Controller` marks this as a Spring controller. Note that we use `@Controller` here, not `@RestController`. WebSocket controllers use `@Controller`.
- `@MessageMapping("/notify")` maps this method to receive messages sent to `/app/notify`. Remember, the `/app` prefix was set in our configuration. The client sends to `/app/notify`, and Spring strips the prefix and matches `/notify`.
- `@SendTo("/topic/notifications")` sends the return value of this method to `/topic/notifications`. All clients subscribed to that topic will receive the message.
- `NotificationRequest request` is the incoming message from the client. Spring automatically converts the JSON message to this Java object using Jackson (just like REST controllers).
- The method creates a new `Notification` object with the message, sender, and current timestamp, then returns it. The returned object is automatically converted to JSON and broadcast to all subscribers.

The Model Classes

Create the request and response classes:

```java
package com.example.websocketdemo.model;

public class NotificationRequest {

    private String message;
    private String sender;

    public NotificationRequest() {
    }

    public NotificationRequest(String message, String sender) {
        this.message = message;
        this.sender = sender;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public String getSender() {
        return sender;
    }

    public void setSender(String sender) {
        this.sender = sender;
    }
}
```

```java
package com.example.websocketdemo.model;

public class Notification {

    private String message;
    private String sender;
    private String timestamp;

    public Notification() {
    }

    public Notification(String message, String sender, String timestamp) {
        this.message = message;
        this.sender = sender;
        this.timestamp = timestamp;
    }

    public String getMessage() {
        return message;
    }
```

```
    public void setMessage(String message) {
        this.message = message;
    }

    public String getSender() {
        return sender;
    }

    public void setSender(String sender) {
        this.sender = sender;
    }

    public String getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(String timestamp) {
        this.timestamp = timestamp;
    }
}
```

A.7 Using SimpMessagingTemplate

`@SendTo` is great when you want to broadcast the result of a message handler. But what if you want to send a message from a service class, a scheduled task, or a REST controller? That is where `SimpMessagingTemplate` comes in.

`SimpMessagingTemplate` lets you send WebSocket messages from anywhere in your application.

```java
package com.example.websocketdemo.service;

import com.example.websocketdemo.model.Notification;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;

@Service
public class NotificationService {

    private final SimpMessagingTemplate messagingTemplate;

    public NotificationService(SimpMessagingTemplate messagingTemplate) {
```

```
        this.messagingTemplate = messagingTemplate;
    }

    public void sendGlobalNotification(String message) {
        Notification notification = new Notification(
            message,
            "System",
            LocalDateTime.now().toString()
        );
        messagingTemplate.convertAndSend(
            "/topic/notifications", notification
        );
    }

    public void sendToUser(String username, String message) {
        Notification notification = new Notification(
            message,
            "System",
            LocalDateTime.now().toString()
        );
        messagingTemplate.convertAndSendToUser(
            username,
            "/queue/private",
            notification
        );
    }
}
```

Line-by-line explanation:

- `SimpMessagingTemplate messagingTemplate` is injected by Spring. It is automatically available when you enable WebSocket message broker support.
- `messagingTemplate.convertAndSend("/topic/notifications", notification)` converts the `Notification` object to JSON and sends it to all clients subscribed to `/topic/notifications`. Think of this as posting a message on the bulletin board.
- `messagingTemplate.convertAndSendToUser(username, "/queue/private", notification)` sends a message to a specific user only. The actual destination becomes `/user/{username}/queue/private`. Think of this as sending a private note instead of posting on the bulletin board.

Using SimpMessagingTemplate from a REST Controller

Here is how you might trigger a WebSocket notification from a regular REST endpoint:

```java
package com.example.websocketdemo.controller;

import com.example.websocketdemo.service.NotificationService;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/notifications")
public class NotificationRestController {

    private final NotificationService notificationService;

    public NotificationRestController(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @PostMapping
    public ResponseEntity<String> triggerNotification(
            @RequestBody Map<String, String> payload) {

        String message = payload.get("message");
        notificationService.sendGlobalNotification(message);

        return ResponseEntity.ok("Notification sent to all connected clients");
    }
}
```

Output when you call POST /api/notifications:

```
HTTP/1.1 200 OK
Content-Type: text/plain

Notification sent to all connected clients
```

And every connected WebSocket client receives:

```json
{
    "message": "Server maintenance at 10 PM",
    "sender": "System",
    "timestamp": "2024-01-15T14:30:00.123456"
}
```

```
How SimpMessagingTemplate Works:

  REST Controller          WebSocket Clients
  (POST /api/notifications)
        |
```

```
      v
NotificationService
      |
      | messagingTemplate.convertAndSend(...)
      v
+-------------------+
| Message Broker    |
| /topic/*          |
+-------------------+
      |         |         |
      v         v         v
Client A   Client B   Client C
(browser)  (browser)  (browser)
```

A.8 Building the HTML and JavaScript Client

Now let us build a web page that connects to our WebSocket server. Create this file at `src/main/resources/static/index.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebSocket Notification Demo</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 700px;
            margin: 40px auto;
            padding: 0 20px;
            background-color: #f5f5f5;
        }
        h1 {
            color: #333;
        }
        .status {
            padding: 10px;
            border-radius: 5px;
            margin-bottom: 20px;
            font-weight: bold;
        }
        .connected {
            background-color: #d4edda;
            color: #155724;
        }
```

```
        .disconnected {
            background-color: #f8d7da;
            color: #721c24;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
            display: block;
            margin-bottom: 5px;
            font-weight: bold;
        }
        input[type="text"] {
            width: 100%;
            padding: 8px;
            border: 1px solid #ccc;
            border-radius: 4px;
            box-sizing: border-box;
        }
        button {
            padding: 10px 20px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 14px;
            margin-right: 10px;
        }
        .btn-connect {
            background-color: #28a745;
            color: white;
        }
        .btn-disconnect {
            background-color: #dc3545;
            color: white;
        }
        .btn-send {
            background-color: #007bff;
            color: white;
        }
        .notification-list {
            list-style: none;
            padding: 0;
        }
        .notification-item {
            background: white;
            padding: 15px;
            margin-bottom: 10px;
            border-radius: 5px;
            border-left: 4px solid #007bff;
            box-shadow: 0 1px 3px rgba(0,0,0,0.1);
        }
        .notification-sender {
            font-weight: bold;
            color: #007bff;
        }
```

```
        .notification-time {
            color: #999;
            font-size: 12px;
        }
    </style>
</head>
<body>
    <h1>Live Notification System</h1>

    <div id="status" class="status disconnected">
        Disconnected
    </div>

    <div class="form-group">
        <label for="name">Your Name:</label>
        <input type="text" id="name" placeholder="Enter your name">
    </div>

    <div class="form-group">
        <label for="message">Message:</label>
        <input type="text" id="message" placeholder="Enter a notification message">
    </div>

    <div>
        <button class="btn-connect" onclick="connect()">Connect</button>
        <button class="btn-disconnect" onclick="disconnect()">Disconnect</button>
        <button class="btn-send" onclick="sendNotification()">Send</button>
    </div>

    <h2>Notifications</h2>
    <ul id="notifications" class="notification-list"></ul>

    <!-- SockJS and STOMP libraries -->
    <script src="https://cdn.jsdelivr.net/npm/sockjs-client@1/dist/sockjs.min.js"></scr...
    <script src="https://cdnjs.cloudflare.com/ajax/libs/stomp.js/2.3.3/stomp.min.js"></...

    <script>
        let stompClient = null;

        function connect() {
            // Step 1: Create a SockJS connection to our /ws endpoint
            const socket = new SockJS('/ws');

            // Step 2: Create a STOMP client over the SockJS connection
            stompClient = Stomp.over(socket);

            // Step 3: Connect to the server
            stompClient.connect({}, function (frame) {
                // This runs when the connection is established
                console.log('Connected: ' + frame);
                updateStatus(true);

                // Step 4: Subscribe to the /topic/notifications topic
                stompClient.subscribe(
                    '/topic/notifications',
```

```
                function (message) {
                    // This runs whenever a notification arrives
                    const notification = JSON.parse(message.body);
                    showNotification(notification);
                }
            );
        });
    }

    function disconnect() {
        if (stompClient !== null) {
            stompClient.disconnect();
        }
        updateStatus(false);
        console.log('Disconnected');
    }

    function sendNotification() {
        const name = document.getElementById('name').value;
        const message = document.getElementById('message').value;

        if (!name || !message) {
            alert('Please enter both name and message');
            return;
        }

        // Send a message to /app/notify
        stompClient.send(
            '/app/notify',
            {},
            JSON.stringify({
                'sender': name,
                'message': message
            })
        );

        // Clear the message input
        document.getElementById('message').value = '';
    }

    function showNotification(notification) {
        const list = document.getElementById('notifications');
        const item = document.createElement('li');
        item.className = 'notification-item';
        item.innerHTML =
            '<span class="notification-sender">'
            + notification.sender + '</span>: '
            + notification.message
            + '<br><span class="notification-time">'
            + notification.timestamp + '</span>';

        // Add new notifications at the top
        list.insertBefore(item, list.firstChild);
    }
```

```
        function updateStatus(connected) {
            const status = document.getElementById('status');
            if (connected) {
                status.textContent = 'Connected';
                status.className = 'status connected';
            } else {
                status.textContent = 'Disconnected';
                status.className = 'status disconnected';
            }
        }
    </script>
</body>
</html>
```

Line-by-line explanation of the JavaScript:

- `new SockJS('/ws')` creates a connection to our WebSocket endpoint. SockJS is a library that provides WebSocket-like behavior with fallbacks for older browsers.
- `Stomp.over(socket)` wraps the SockJS connection with the STOMP protocol. Now we can use STOMP commands like SUBSCRIBE and SEND.
- `stompClient.connect({}, callback)` connects to the STOMP server. The first argument `{}` is for headers (empty here). The second argument is a callback that runs when the connection succeeds.
- `stompClient.subscribe('/topic/notifications', callback)` subscribes to the notifications topic. Whenever a message arrives at this topic, the callback function runs.
- `JSON.parse(message.body)` converts the incoming JSON string to a JavaScript object.
- `stompClient.send('/app/notify', {}, JSON.stringify({...}))` sends a STOMP message to the server. The destination `/app/notify` maps to our `@MessageMapping("/notify")` controller method. The second argument `{}` is for headers. The third argument is the JSON message body.
- `stompClient.disconnect()` cleanly closes the WebSocket connection.

Running the Application

Start your Spring Boot application and open `http://localhost:8080` in two or more browser windows. Here is what happens:

1. Click "Connect" in both windows. The status changes to "Connected."
2. In Window 1, enter your name and a message, then click "Send."
3. Both Window 1 and Window 2 instantly display the notification.

```
How the Live Demo Works:

Window 1 (Browser)                Server                Window 2 (Browser)
```

```
      |                              |                        |
      |-- connect to /ws ----------->|                        |
      |<-- connected ----------------|                        |
      |-- subscribe /topic/notif --->|                        |
      |                              |                        |
      |                              |<-- connect to /ws -----|
      |                              |--- connected --------->|
      |                              |<-- subscribe /topic/ --|
      |                              |                        |
      |-- send /app/notify --------->|                        |
      |   {sender: "Alice",          |                        |
      |    message: "Hello!"}        |                        |
      |                              |                        |
      |   @MessageMapping runs       |                        |
      |   @SendTo broadcasts         |                        |
      |                              |                        |
      |<-- notification -------------|--- notification ------>|
      |   {sender: "Alice",          |   {sender: "Alice",    |
      |    message: "Hello!",        |    message: "Hello!",  |
      |    timestamp: "..."}         |    timestamp: "..."}   |
      |                              |                        |
```

A.9 Building a Live Notification Example with Scheduled Updates

Let us make the notification system more interesting. We will add a scheduled task that sends automatic notifications to all connected clients every 30 seconds.

```java
package com.example.websocketdemo.service;

import com.example.websocketdemo.model.Notification;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.atomic.AtomicInteger;

@Service
public class ScheduledNotificationService {

    private final SimpMessagingTemplate messagingTemplate;
    private final AtomicInteger activeUsers = new AtomicInteger(0);
```

```
    public ScheduledNotificationService(
            SimpMessagingTemplate messagingTemplate) {
        this.messagingTemplate = messagingTemplate;
    }

    @Scheduled(fixedRate = 30000)
    public void sendPeriodicUpdate() {
        String time = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("HH:mm:ss"));

        Notification notification = new Notification(
            "Server heartbeat at " + time
                + ". Active connections: " + activeUsers.get(),
            "System",
            LocalDateTime.now().toString()
        );

        messagingTemplate.convertAndSend(
            "/topic/notifications", notification
        );
    }

    public void userConnected() {
        activeUsers.incrementAndGet();
    }

    public void userDisconnected() {
        activeUsers.decrementAndGet();
    }
}
```

To track connections and disconnections, add an event listener:

```java
package com.example.websocketdemo.config;

import com.example.websocketdemo.service.ScheduledNotificationService;
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.messaging.SessionConnectedEvent;
import org.springframework.web.socket.messaging.SessionDisconnectEvent;

@Component
public class WebSocketEventListener {

    private final ScheduledNotificationService notificationService;

    public WebSocketEventListener(
            ScheduledNotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @EventListener
```

```
    public void handleWebSocketConnect(SessionConnectedEvent event) {
        System.out.println("New WebSocket connection established");
        notificationService.userConnected();
    }

    @EventListener
    public void handleWebSocketDisconnect(SessionDisconnectEvent event) {
        System.out.println("WebSocket connection closed");
        notificationService.userDisconnected();
    }
}
```

Output in the server console when clients connect and disconnect:

```
New WebSocket connection established
New WebSocket connection established
WebSocket connection closed
```

Line-by-line explanation:

- `@EventListener` tells Spring to call this method when a specific event occurs. Spring's WebSocket support publishes events when clients connect and disconnect.
- `SessionConnectedEvent` is fired when a WebSocket client successfully connects via STOMP.
- `SessionDisconnectEvent` is fired when a WebSocket client disconnects (closes browser, loses network, or calls disconnect).
- `AtomicInteger` is used for thread-safe counting because multiple clients can connect and disconnect at the same time.

Do not forget to enable scheduling in your main application class:

```java
package com.example.websocketdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class WebSocketDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(WebSocketDemoApplication.class, args);
    }
}
```

A.10 Complete Project Structure

Here is the final project structure:

```
websocket-demo/
+-- src/
|   +-- main/
|   |   +-- java/com/example/websocketdemo/
|   |   |   +-- WebSocketDemoApplication.java
|   |   |   +-- config/
|   |   |   |   +-- WebSocketConfig.java
|   |   |   |   +-- WebSocketEventListener.java
|   |   |   +-- controller/
|   |   |   |   +-- NotificationController.java
|   |   |   |   +-- NotificationRestController.java
|   |   |   +-- model/
|   |   |   |   +-- Notification.java
|   |   |   |   +-- NotificationRequest.java
|   |   |   +-- service/
|   |   |       +-- NotificationService.java
|   |   |       +-- ScheduledNotificationService.java
|   |   +-- resources/
|   |       +-- static/
|   |       |   +-- index.html
|   |       +-- application.properties
|   +-- test/
|       +-- java/com/example/websocketdemo/
|           +-- WebSocketDemoApplicationTests.java
+-- pom.xml
```

Your `application.properties` can remain simple:

```properties
spring.application.name=websocket-demo
```

Common Mistakes

1. Using @RestController instead of @Controller for WebSocket handlers. WebSocket message-handling methods with `@MessageMapping` should be in a class annotated with `@Controller`, not `@RestController`. The `@RestController` annotation adds `@ResponseBody`, which interferes with WebSocket message handling.

2. Forgetting the /app prefix when sending from the client. If your application destination prefix is `/app`, the client must send to `/app/notify`, not just `/notify`. But in `@MessageMapping`, you use `/notify` without the prefix.
3. Not including SockJS and STOMP JavaScript libraries. The HTML client needs both the SockJS and STOMP.js libraries to connect. Without them, the connection will fail.
4. Mixing up /topic and /queue. Use `/topic` for broadcasting messages to all subscribers (like a radio station). Use `/queue` for sending messages to a specific user (like a private message).
5. Forgetting @EnableWebSocketMessageBroker. Without this annotation, Spring will not set up the WebSocket infrastructure, and nothing will work.
6. Forgetting @EnableScheduling when using scheduled notifications. The `@Scheduled` annotation does nothing unless you add `@EnableScheduling` to your configuration or main application class.

Best Practices

1. Use SockJS fallback. Always add `.withSockJS()` to your endpoint registration. This ensures your application works even in environments where raw WebSocket connections are blocked.
2. Keep WebSocket messages small. WebSocket is designed for lightweight, frequent messages. If you need to transfer large files, use HTTP instead.
3. Handle disconnections gracefully. Clients can disconnect unexpectedly (browser crash, network loss). Use event listeners to clean up resources when clients disconnect.
4. Use SimpMessagingTemplate for server-initiated messages. When you need to push messages from background tasks, services, or scheduled jobs, use `SimpMessagingTemplate` instead of trying to use `@SendTo`.
5. Consider security. In production, secure your WebSocket endpoints with authentication. Spring Security integrates with WebSocket to protect your connections.
6. Use an external message broker for production. The simple in-memory broker is fine for development and small applications. For production with many users, consider using RabbitMQ or ActiveMQ as the message broker.

Quick Summary

WebSocket provides a persistent, two-way connection between the browser and server. Unlike HTTP (which is like sending letters), WebSocket is like a phone call where both sides can talk at any time. Spring Boot uses the STOMP protocol on top of WebSocket to add structure to messages with topics and subscriptions. You enable WebSocket with `@EnableWebSocketMessageBroker`, handle incoming messages with `@MessageMapping`, broadcast results with `@SendTo`, and send messages from anywhere using `SimpMessagingTemplate`. The JavaScript client uses the SockJS and STOMP.js libraries to connect, subscribe to topics, and send and receive messages.

Key Points

- HTTP is request-response (one-way at a time). WebSocket is full-duplex (both ways simultaneously).
- WebSocket starts with an HTTP handshake, then upgrades to the WebSocket protocol.
- STOMP adds structure (destinations, subscriptions, headers) on top of raw WebSocket.
- `@EnableWebSocketMessageBroker` activates WebSocket support in Spring Boot.
- `@MessageMapping` handles incoming WebSocket messages (like `@RequestMapping` for HTTP).
- `@SendTo` broadcasts the return value to a topic.
- `SimpMessagingTemplate` sends WebSocket messages from any part of your application.
- SockJS provides a fallback when WebSocket is not available.
- Use `/topic` for broadcasting and `/queue` for private messages.

Practice Questions

1. What is the main difference between HTTP and WebSocket communication? Give a real-life analogy.
2. Why does Spring Boot use the STOMP protocol on top of WebSocket instead of using raw WebSocket?
3. What is the purpose of `@EnableWebSocketMessageBroker` and what happens if you forget it?
4. Explain the difference between `@SendTo` and `SimpMessagingTemplate`. When would you use each one?

5. A developer sends a message from the JavaScript client to `/notify` but the server never receives it. The `@MessageMapping` method is mapped to `/notify` and the application prefix is `/app`. What is wrong?

Exercises

Exercise 1: Build a Chat Room

Modify the notification example to build a simple chat room. Add a "room" field to the message so users can join different rooms. Use `@DestinationVariable` to route messages to room-specific topics like `/topic/chat/{roomName}`. Test with two browser windows in the same room and two in a different room.

Exercise 2: Add a Typing Indicator

Add a "typing indicator" feature. When a user starts typing in the message field, send a WebSocket message to notify other users. Display "User X is typing..." below the notification list. The indicator should disappear after 3 seconds of no typing.

Exercise 3: Server-Sent Notifications from REST

Create a REST endpoint `POST /api/admin/broadcast` that accepts a JSON body with a `message` field. When called (for example, from Postman), it should send the message to all connected WebSocket clients. This simulates an admin sending a system-wide announcement.

What Is Next?

Now that you understand real-time communication with WebSocket, you are ready for Appendix B where you will build a complete project from scratch. The TaskTracker API combines everything you have learned in this book into one comprehensive application.

Appendix B: Complete Project — TaskTracker API

What You Will Learn

- How to build a complete REST API from scratch using everything you learned in this book.
- How to design User and Task entities with JPA relationships.
- How to implement repositories, services, and controllers following best practices.
- How to add input validation and global exception handling.
- How to secure the API with JWT authentication and role-based access.
- How to implement pagination, caching, and scheduled tasks.
- How to generate API documentation with OpenAPI and Swagger.
- How to write tests for your controllers and services.
- How to containerize the application with Docker.

Why This Chapter Matters

Imagine you have learned how to hammer nails, cut wood, install plumbing, and wire electricity. Each skill is useful on its own. But the real test is building an entire house. That is what this appendix is about.

Throughout this book, you learned each Spring Boot concept one at a time. Now you will combine all of those skills into one complete, production-ready application. This is your capstone project. By the end, you will have a TaskTracker API that you can put on your resume and show to potential employers.

Every line of code is included. You will type it, understand it, and run it.

B.1 Project Overview

TaskTracker is a task management API where users can create accounts, log in, and manage their tasks. Here is what the application does:

- Users can register and log in with JWT authentication.
- Users have roles (USER or ADMIN).
- Users can create, read, update, and delete their own tasks.
- Admins can view all tasks and all users.
- Tasks have a title, description, status, priority, and due date.
- Tasks can be filtered by status and priority with pagination.
- Overdue tasks are automatically flagged by a scheduled job.
- Frequently accessed data is cached for performance.
- The API is documented with OpenAPI (Swagger).

```
TaskTracker API Architecture:

+-----------------------------------------------------+
|                     Clients                         |
|  (Postman, Browser, Mobile App, Frontend App)       |
+-----------------------------------------------------+
                       |
                       | HTTP + JWT
                       v
+-----------------------------------------------------+
|              Spring Security Layer                  |
|  (JWT Filter, Authentication, Authorization)        |
+-----------------------------------------------------+
                       |
                       v
+-----------------------------------------------------+
|              REST Controllers                       |
|  (AuthController, TaskController, UserController)   |
+-----------------------------------------------------+
                       |
                       v
+-----------------------------------------------------+
|              Service Layer                          |
|  (AuthService, TaskService, UserService)            |
+-----------------------------------------------------+
                       |
                       v
+-----------------------------------------------------+
|              Repository Layer                       |
|  (UserRepository, TaskRepository)                   |
+-----------------------------------------------------+
                       |
                       v
+-----------------------------------------------------+
```

```
|              H2 Database                          |
|  (Users table, Tasks table)                       |
+---------------------------------------------------+
```

B.2 Project Setup

Step 1: Create the Project

Go to start.spring.io and create a new project with these settings:

Setting	Value
Project	Maven
Language	Java
Spring Boot	3.4.x (latest stable)
Group	com.example
Artifact	tasktracker
Name	tasktracker
Package name	com.example.tasktracker
Packaging	Jar
Java	17

Add these dependencies:

- Spring Web
- Spring Data JPA
- H2 Database
- Spring Security
- Spring Boot Starter Validation
- Spring Boot Starter Cache

Click "Generate" to download the project. Unzip it and open it in your IDE.

Step 2: Add Additional Dependencies

Open `pom.xml` and add these additional dependencies inside the `<dependencies>` section:

```xml
<!-- JWT Support -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>

<!-- OpenAPI / Swagger Documentation -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.4</version>
</dependency>
```

Step 3: Configure application.properties

Open `src/main/resources/application.properties` and add:

```properties
# Application name
spring.application.name=tasktracker

# H2 Database
spring.datasource.url=jdbc:h2:mem:tasktracker
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

```
# JWT
jwt.secret=mySecretKeyForJWTTokenGenerationThatIsAtLeast256BitsLong2024
jwt.expiration=86400000

# Swagger / OpenAPI
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.api-docs.path=/api-docs

# Cache
spring.cache.type=simple
```

Line-by-line explanation:

- `spring.datasource.url=jdbc:h2:mem:tasktracker` creates an in-memory H2 database named "tasktracker." Data is lost when the application stops, which is perfect for development.
- `spring.h2.console.enabled=true` enables the H2 web console at `/h2-console` so you can view the database tables.
- `spring.jpa.hibernate.ddl-auto=create-drop` tells Hibernate to create tables on startup and drop them on shutdown. In production, you would use `validate` or `none`.
- `spring.jpa.show-sql=true` prints SQL statements to the console so you can see what Hibernate is doing.
- `jwt.secret` is the secret key for signing JWT tokens. In production, use a secure, random key and store it in environment variables.
- `jwt.expiration=86400000` sets the JWT token expiration to 24 hours (in milliseconds).
- `spring.cache.type=simple` uses a simple in-memory cache based on `ConcurrentHashMap`.

B.3 Entity Classes

The User Entity

Create the file `src/main/java/com/example/tasktracker/entity/User.java`:

```java
package com.example.tasktracker.entity;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @JsonIgnore
    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    @JsonIgnore
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Task> tasks = new ArrayList<>();

    public User() {
    }

    public User(String username, String email, String password) {
        this.username = username;
        this.email = email;
        this.password = password;
    }

    // Getters and setters

    public Long getId() {
```

```
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public Role getRole() {
        return role;
    }

    public void setRole(Role role) {
        this.role = role;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public List<Task> getTasks() {
        return tasks;
    }

    public void setTasks(List<Task> tasks) {
        this.tasks = tasks;
    }
}
```

Line-by-line explanation:

- `@Entity` marks this class as a JPA entity that maps to a database table.
- `@Table(name = "users")` sets the table name to "users." We cannot use "user" because it is a reserved word in H2.
- `@Id` and `@GeneratedValue(strategy = GenerationType.IDENTITY)` make the `id` field the primary key with auto-increment.
- `@Column(nullable = false, unique = true, length = 50)` means the column cannot be null, must be unique, and has a max length of 50 characters.
- `@JsonIgnore` on the `password` field prevents the password from being included in JSON responses. You never want to send passwords to the client.
- `@Enumerated(EnumType.STRING)` stores the enum as a string in the database (like "USER" or "ADMIN") instead of a number.
- `@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)` creates a one-to-many relationship: one user has many tasks. `cascade = CascadeType.ALL` means operations on the user (like delete) cascade to their tasks. `orphanRemoval = true` means if a task is removed from the list, it is deleted from the database.
- `@JsonIgnore` on the `tasks` field prevents infinite recursion when converting to JSON (User has Tasks, Task has User, User has Tasks...).

The Role Enum

Create `src/main/java/com/example/tasktracker/entity/Role.java`:

```java
package com.example.tasktracker.entity;

public enum Role {
    USER,
    ADMIN
}
```

The Task Entity

Create `src/main/java/com/example/tasktracker/entity/Task.java`:

```java
package com.example.tasktracker.entity;

import jakarta.persistence.Column;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(length = 1000)
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TaskStatus status = TaskStatus.TODO;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TaskPriority priority = TaskPriority.MEDIUM;

    private LocalDate dueDate;

    @Column(nullable = false)
    private boolean overdue = false;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    private LocalDateTime updatedAt = LocalDateTime.now();

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    public Task() {
    }

    public Task(String title, String description,
                TaskPriority priority, LocalDate dueDate, User user) {
```

```
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.dueDate = dueDate;
        this.user = user;
    }

    // Getters and setters

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public TaskStatus getStatus() {
        return status;
    }

    public void setStatus(TaskStatus status) {
        this.status = status;
    }

    public TaskPriority getPriority() {
        return priority;
    }

    public void setPriority(TaskPriority priority) {
        this.priority = priority;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
```

```
    }

    public boolean isOverdue() {
        return overdue;
    }

    public void setOverdue(boolean overdue) {
        this.overdue = overdue;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }

    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }
}
```

Line-by-line explanation:

- `@ManyToOne(fetch = FetchType.LAZY)` creates the many-to-one side of the relationship. Many tasks belong to one user. `FetchType.LAZY` means the user data is only loaded from the database when you actually access it, which improves performance.
- `@JoinColumn(name = "user_id", nullable = false)` creates a `user_id` foreign key column in the tasks table. It cannot be null because every task must belong to a user.
- `boolean overdue` is a flag that gets set to `true` by a scheduled task when the due date has passed and the task is not completed.
- `updatedAt` tracks when the task was last modified.

The TaskStatus Enum

Create `src/main/java/com/example/tasktracker/entity/TaskStatus.java`:

```java
package com.example.tasktracker.entity;

public enum TaskStatus {
    TODO,
    IN_PROGRESS,
    COMPLETED
}
```

The TaskPriority Enum

Create `src/main/java/com/example/tasktracker/entity/TaskPriority.java`:

```java
package com.example.tasktracker.entity;

public enum TaskPriority {
    LOW,
    MEDIUM,
    HIGH
}
```

Entity Relationship Diagram

```
+-------------------+          +-------------------+
|      users        |          |      tasks        |
+-------------------+          +-------------------+
| id (PK)           |<---------| id (PK)           |
| username          |    1:N   | title             |
| email             |          | description       |
| password          |          | status            |
| role              |          | priority          |
| created_at        |          | due_date          |
+-------------------+          | overdue           |
                               | created_at        |
                               | updated_at        |
                               | user_id (FK)      |
                               +-------------------+
```

B.4 DTOs (Data Transfer Objects)

DTOs separate what the client sends and receives from the internal entity structure. This is important for security (you do not want to expose passwords) and flexibility (you can change

DTOs without changing entities).

Registration Request

Create `src/main/java/com/example/tasktracker/dto/RegisterRequest.java`:

```java
package com.example.tasktracker.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class RegisterRequest {

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be 3-50 characters")
    private String username;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 6, max = 100, message = "Password must be 6-100 characters")
    private String password;

    public RegisterRequest() {
    }

    public RegisterRequest(String username, String email, String password) {
        this.username = username;
        this.email = email;
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
```

```
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Login Request

Create `src/main/java/com/example/tasktracker/dto/LoginRequest.java`:

```java
package com.example.tasktracker.dto;

import jakarta.validation.constraints.NotBlank;

public class LoginRequest {

    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;

    public LoginRequest() {
    }

    public LoginRequest(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Auth Response

Create `src/main/java/com/example/tasktracker/dto/AuthResponse.java`:

```java
package com.example.tasktracker.dto;

public class AuthResponse {

    private String token;
    private String username;
    private String role;

    public AuthResponse() {
    }

    public AuthResponse(String token, String username, String role) {
        this.token = token;
        this.username = username;
        this.role = role;
    }

    public String getToken() {
        return token;
    }

    public void setToken(String token) {
        this.token = token;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

Task Request

Create `src/main/java/com/example/tasktracker/dto/TaskRequest.java`:

```java
package com.example.tasktracker.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;

public class TaskRequest {

    @NotBlank(message = "Title is required")
    @Size(max = 200, message = "Title cannot exceed 200 characters")
    private String title;

    @Size(max = 1000, message = "Description cannot exceed 1000 characters")
    private String description;

    private String priority;
    private LocalDate dueDate;

    public TaskRequest() {
    }

    public TaskRequest(String title, String description,
                       String priority, LocalDate dueDate) {
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.dueDate = dueDate;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getPriority() {
        return priority;
    }

    public void setPriority(String priority) {
        this.priority = priority;
    }
```

```
    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
    }
}
```

Task Response

Create `src/main/java/com/example/tasktracker/dto/TaskResponse.java`:

```java
package com.example.tasktracker.dto;

import com.example.tasktracker.entity.Task;

import java.time.LocalDate;
import java.time.LocalDateTime;

public class TaskResponse {

    private Long id;
    private String title;
    private String description;
    private String status;
    private String priority;
    private LocalDate dueDate;
    private boolean overdue;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private String username;

    public TaskResponse() {
    }

    public TaskResponse(Task task) {
        this.id = task.getId();
        this.title = task.getTitle();
        this.description = task.getDescription();
        this.status = task.getStatus().name();
        this.priority = task.getPriority().name();
        this.dueDate = task.getDueDate();
        this.overdue = task.isOverdue();
        this.createdAt = task.getCreatedAt();
        this.updatedAt = task.getUpdatedAt();
        this.username = task.getUser().getUsername();
    }

    // Getters and setters
```

```
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public String getPriority() {
        return priority;
    }

    public void setPriority(String priority) {
        this.priority = priority;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
    }

    public boolean isOverdue() {
        return overdue;
    }

    public void setOverdue(boolean overdue) {
        this.overdue = overdue;
```

```
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }

    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

B.5 Repositories

UserRepository

Create `src/main/java/com/example/tasktracker/repository/UserRepository.java`:

```java
package com.example.tasktracker.repository;

import com.example.tasktracker.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);
```

```
    Optional<User> findByEmail(String email);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);
}
```

Line-by-line explanation:

- `JpaRepository<User, Long>` gives us all standard CRUD operations (save, findById, findAll, delete) plus pagination and sorting. `User` is the entity type, and `Long` is the type of its primary key.
- `findByUsername` and `findByEmail` are query methods. Spring Data JPA automatically generates the SQL query from the method name. `findByUsername` becomes `SELECT * FROM users WHERE username = ?`.
- `existsByUsername` and `existsByEmail` return a boolean. These are useful for checking if a username or email is already taken during registration.

TaskRepository

Create `src/main/java/com/example/tasktracker/repository/TaskRepository.java`:

```java
package com.example.tasktracker.repository;

import com.example.tasktracker.entity.Task;
import com.example.tasktracker.entity.TaskPriority;
import com.example.tasktracker.entity.TaskStatus;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Repository
public interface TaskRepository extends JpaRepository<Task, Long> {

    Page<Task> findByUserId(Long userId, Pageable pageable);

    Page<Task> findByUserIdAndStatus(Long userId, TaskStatus status,
                                     Pageable pageable);
```

```
    Page<Task> findByUserIdAndPriority(Long userId, TaskPriority priority,
                                       Pageable pageable);

    Optional<Task> findByIdAndUserId(Long id, Long userId);

    @Query("SELECT t FROM Task t WHERE t.dueDate < :today "
         + "AND t.status <> 'COMPLETED' AND t.overdue = false")
    List<Task> findOverdueTasks(@Param("today") LocalDate today);

    long countByUserIdAndStatus(Long userId, TaskStatus status);
}
```

Line-by-line explanation:

- `Page<Task> findByUserId(Long userId, Pageable pageable)` returns a page of tasks for a specific user. `Pageable` contains the page number, page size, and sort order.
- `findByUserIdAndStatus` filters by both user and status, returning paginated results.
- `findByIdAndUserId` finds a task by its ID but only if it belongs to the specified user. This prevents users from accessing other users' tasks.
- `@Query` defines a custom JPQL query. This query finds tasks where the due date is before today, the status is not COMPLETED, and the `overdue` flag is not yet set.
- `countByUserIdAndStatus` counts how many tasks a user has with a specific status. Useful for dashboard statistics.

B.6 Exception Handling

Custom Exceptions

Create `src/main/java/com/example/tasktracker/exception/ResourceNotFoundException.java`:

```java
package com.example.tasktracker.exception;

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Create `src/main/java/com/example/tasktracker/exception/DuplicateResourceException.java`:

```java
package com.example.tasktracker.exception;

public class DuplicateResourceException extends RuntimeException {

    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

Create `src/main/java/com/example/tasktracker/exception/UnauthorizedException.java`:

```java
package com.example.tasktracker.exception;

public class UnauthorizedException extends RuntimeException {

    public UnauthorizedException(String message) {
        super(message);
    }
}
```

Error Response DTO

Create `src/main/java/com/example/tasktracker/dto/ErrorResponse.java`:

```java
package com.example.tasktracker.dto;

import java.time.LocalDateTime;
import java.util.Map;

public class ErrorResponse {

    private int status;
    private String message;
    private LocalDateTime timestamp;
    private Map<String, String> errors;

    public ErrorResponse(int status, String message) {
        this.status = status;
        this.message = message;
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponse(int status, String message,
                         Map<String, String> errors) {
        this.status = status;
        this.message = message;
        this.timestamp = LocalDateTime.now();
```

```
        this.errors = errors;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public Map<String, String> getErrors() {
        return errors;
    }

    public void setErrors(Map<String, String> errors) {
        this.errors = errors;
    }
}
```

Global Exception Handler

Create `src/main/java/com/example/tasktracker/exception/GlobalExceptionHandler.java`:

```java
package com.example.tasktracker.exception;

import com.example.tasktracker.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(), ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse> handleDuplicate(
            DuplicateResourceException ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.CONFLICT.value(), ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<ErrorResponse> handleUnauthorized(
            UnauthorizedException ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.UNAUTHORIZED.value(), ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(),
                       fieldError.getDefaultMessage());
        }

        ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            "Validation failed",
            errors
        );
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(Exception.class)
```

```
    public ResponseEntity<ErrorResponse> handleGeneral(Exception ex) {

        ErrorResponse error = new ErrorResponse(
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "An unexpected error occurred"
        );
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                             .body(error);
    }
}
```

Line-by-line explanation:

- `@RestControllerAdvice` makes this a global exception handler for all controllers. It combines `@ControllerAdvice` and `@ResponseBody`.
- Each `@ExceptionHandler` method catches a specific exception type and converts it to a meaningful JSON response.
- `MethodArgumentNotValidException` is thrown when `@Valid` validation fails. We extract each field error and create a map of field name to error message.
- The catch-all `Exception` handler catches any unexpected errors and returns a generic message. In production, you would also log the full stack trace.

B.7 JWT Security

JWT Utility Class

Create `src/main/java/com/example/tasktracker/security/JwtUtil.java`:

```java
package com.example.tasktracker.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtUtil {
```

```
    private final SecretKey key;
    private final long expiration;

    public JwtUtil(@Value("${jwt.secret}") String secret,
                   @Value("${jwt.expiration}") long expiration) {
        this.key = Keys.hmacShaKeyFor(
            secret.getBytes(StandardCharsets.UTF_8)
        );
        this.expiration = expiration;
    }

    public String generateToken(String username, String role) {
        return Jwts.builder()
            .subject(username)
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(key)
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaims(token).getSubject();
    }

    public String extractRole(String token) {
        return extractClaims(token).get("role", String.class);
    }

    public boolean isTokenValid(String token) {
        try {
            extractClaims(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    private Claims extractClaims(String token) {
        return Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}
```

Line-by-line explanation:

- `@Value("${jwt.secret}")` injects the secret key from `application.properties`.
- `Keys.hmacShaKeyFor(...)` creates a cryptographic key from the secret string.

- `generateToken` creates a JWT with the username as the subject, the role as a custom claim, and an expiration time.
- `extractUsername` and `extractRole` parse the token to read the stored values.
- `isTokenValid` attempts to parse the token. If parsing fails (expired, tampered, wrong key), it returns false.

JWT Authentication Filter

Create `src/main/java/com/example/tasktracker/security/JwtAuthenticationFilter.java`:

```java
package com.example.tasktracker.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);

            if (jwtUtil.isTokenValid(token)) {
                String username = jwtUtil.extractUsername(token);
                String role = jwtUtil.extractRole(token);

                SimpleGrantedAuthority authority =
```

```
                    new SimpleGrantedAuthority("ROLE_" + role);

                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                        username, null, List.of(authority)
                    );

                SecurityContextHolder.getContext()
                    .setAuthentication(authentication);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Line-by-line explanation:

- `OncePerRequestFilter` ensures this filter runs exactly once per HTTP request.
- `request.getHeader("Authorization")` reads the Authorization header from the incoming request.
- `authHeader.startsWith("Bearer ")` checks if the header follows the Bearer token format.
- `authHeader.substring(7)` extracts the token by removing the "Bearer " prefix (7 characters).
- `jwtUtil.isTokenValid(token)` verifies the token is not expired or tampered with.
- `SimpleGrantedAuthority("ROLE_" + role)` creates a Spring Security authority. The "ROLE_" prefix is required by Spring Security conventions.
- `UsernamePasswordAuthenticationToken` represents the authenticated user. We pass the username, null for credentials (we already verified the token), and the authorities.
- `SecurityContextHolder.getContext().setAuthentication(authentication)` stores the authentication in the security context. Now Spring Security knows who the user is for the rest of the request.
- `filterChain.doFilter(request, response)` continues to the next filter in the chain. If we did not call this, the request would stop here.

Security Configuration

Create `src/main/java/com/example/tasktracker/security/SecurityConfig.java`:

```java
package com.example.tasktracker.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi...

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/h2-console/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/api-docs/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .headers(headers -> headers
                .frameOptions(frame -> frame.disable())
            )
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Line-by-line explanation:

- `csrf(csrf -> csrf.disable())` disables CSRF protection. REST APIs that use JWT tokens do not need CSRF because the token itself prevents cross-site request forgery.

- `SessionCreationPolicy.STATELESS` tells Spring Security not to create HTTP sessions. We use JWT tokens instead of sessions.
- `requestMatchers("/api/auth/**").permitAll()` allows anyone to access the registration and login endpoints without authentication.
- `requestMatchers("/h2-console/**").permitAll()` allows access to the H2 database console during development.
- `requestMatchers("/swagger-ui/**", "/api-docs/**").permitAll()` allows access to the API documentation.
- `requestMatchers("/api/admin/**").hasRole("ADMIN")` restricts admin endpoints to users with the ADMIN role only.
- `anyRequest().authenticated()` requires authentication for all other endpoints.
- `headers(headers -> headers.frameOptions(frame -> frame.disable()))` disables X-Frame-Options so the H2 console (which uses iframes) can load.
- `addFilterBefore(...)` adds our JWT filter before Spring Security's default authentication filter. This means every request is checked for a JWT token first.

B.8 Service Layer

AuthService

Create `src/main/java/com/example/tasktracker/service/AuthService.java`:

```java
package com.example.tasktracker.service;

import com.example.tasktracker.dto.AuthResponse;
import com.example.tasktracker.dto.LoginRequest;
import com.example.tasktracker.dto.RegisterRequest;
import com.example.tasktracker.entity.User;
import com.example.tasktracker.exception.DuplicateResourceException;
import com.example.tasktracker.exception.UnauthorizedException;
import com.example.tasktracker.repository.UserRepository;
import com.example.tasktracker.security.JwtUtil;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
```

```
    private final JwtUtil jwtUtil;

    public AuthService(UserRepository userRepository,
                       PasswordEncoder passwordEncoder,
                       JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtil = jwtUtil;
    }

    public AuthResponse register(RegisterRequest request) {
        if (userRepository.existsByUsername(request.getUsername())) {
            throw new DuplicateResourceException(
                "Username already exists: " + request.getUsername()
            );
        }

        if (userRepository.existsByEmail(request.getEmail())) {
            throw new DuplicateResourceException(
                "Email already exists: " + request.getEmail()
            );
        }

        User user = new User(
            request.getUsername(),
            request.getEmail(),
            passwordEncoder.encode(request.getPassword())
        );

        userRepository.save(user);

        String token = jwtUtil.generateToken(
            user.getUsername(), user.getRole().name()
        );

        return new AuthResponse(
            token, user.getUsername(), user.getRole().name()
        );
    }

    public AuthResponse login(LoginRequest request) {
        User user = userRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new UnauthorizedException(
                "Invalid username or password"
            ));

        if (!passwordEncoder.matches(request.getPassword(),
                                     user.getPassword())) {
            throw new UnauthorizedException("Invalid username or password");
        }

        String token = jwtUtil.generateToken(
            user.getUsername(), user.getRole().name()
        );
```

```
        return new AuthResponse(
            token, user.getUsername(), user.getRole().name()
        );
    }
}
```

Line-by-line explanation:

- `passwordEncoder.encode(request.getPassword())` hashes the password using BCrypt before storing it. You should never store plain-text passwords.
- `passwordEncoder.matches(request.getPassword(), user.getPassword())` compares the raw password from the login request with the hashed password from the database.
- Notice that the same error message "Invalid username or password" is used for both a wrong username and a wrong password. This is a security best practice. If you said "User not found," an attacker would know which usernames exist.

TaskService

Create `src/main/java/com/example/tasktracker/service/TaskService.java`:

```java
package com.example.tasktracker.service;

import com.example.tasktracker.dto.TaskRequest;
import com.example.tasktracker.dto.TaskResponse;
import com.example.tasktracker.entity.Task;
import com.example.tasktracker.entity.TaskPriority;
import com.example.tasktracker.entity.TaskStatus;
import com.example.tasktracker.entity.User;
import com.example.tasktracker.exception.ResourceNotFoundException;
import com.example.tasktracker.repository.TaskRepository;
import com.example.tasktracker.repository.UserRepository;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

@Service
@Transactional
public class TaskService {

    private final TaskRepository taskRepository;
```

```
    private final UserRepository userRepository;

    public TaskService(TaskRepository taskRepository,
                       UserRepository userRepository) {
        this.taskRepository = taskRepository;
        this.userRepository = userRepository;
    }

    public TaskResponse createTask(TaskRequest request, String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        TaskPriority priority = TaskPriority.MEDIUM;
        if (request.getPriority() != null) {
            priority = TaskPriority.valueOf(
                request.getPriority().toUpperCase()
            );
        }

        Task task = new Task(
            request.getTitle(),
            request.getDescription(),
            priority,
            request.getDueDate(),
            user
        );

        Task savedTask = taskRepository.save(task);
        return new TaskResponse(savedTask);
    }

    @Cacheable(value = "tasks", key = "#username + '-' + #pageable.pageNumber")
    @Transactional(readOnly = true)
    public Page<TaskResponse> getTasksByUser(String username,
                                             Pageable pageable) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        return taskRepository.findByUserId(user.getId(), pageable)
            .map(TaskResponse::new);
    }

    @Transactional(readOnly = true)
    public Page<TaskResponse> getTasksByStatus(String username,
                                               TaskStatus status,
                                               Pageable pageable) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));
```

```
        return taskRepository
            .findByUserIdAndStatus(user.getId(), status, pageable)
            .map(TaskResponse::new);
    }

    @Transactional(readOnly = true)
    public Page<TaskResponse> getTasksByPriority(String username,
                                                 TaskPriority priority,
                                                 Pageable pageable) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        return taskRepository
            .findByUserIdAndPriority(user.getId(), priority, pageable)
            .map(TaskResponse::new);
    }

    @Transactional(readOnly = true)
    public TaskResponse getTaskById(Long taskId, String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        Task task = taskRepository.findByIdAndUserId(taskId, user.getId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Task not found with id: " + taskId
            ));

        return new TaskResponse(task);
    }

    @CacheEvict(value = "tasks", allEntries = true)
    public TaskResponse updateTask(Long taskId, TaskRequest request,
                                   String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        Task task = taskRepository.findByIdAndUserId(taskId, user.getId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Task not found with id: " + taskId
            ));

        task.setTitle(request.getTitle());
        task.setDescription(request.getDescription());
        task.setDueDate(request.getDueDate());
        task.setUpdatedAt(LocalDateTime.now());

        if (request.getPriority() != null) {
            task.setPriority(
                TaskPriority.valueOf(request.getPriority().toUpperCase())
```

```
            );
        }

        Task updatedTask = taskRepository.save(task);
        return new TaskResponse(updatedTask);
    }

    @CacheEvict(value = "tasks", allEntries = true)
    public TaskResponse updateTaskStatus(Long taskId, TaskStatus status,
                                         String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        Task task = taskRepository.findByIdAndUserId(taskId, user.getId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Task not found with id: " + taskId
            ));

        task.setStatus(status);
        task.setUpdatedAt(LocalDateTime.now());

        if (status == TaskStatus.COMPLETED) {
            task.setOverdue(false);
        }

        Task updatedTask = taskRepository.save(task);
        return new TaskResponse(updatedTask);
    }

    @CacheEvict(value = "tasks", allEntries = true)
    public void deleteTask(Long taskId, String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));

        Task task = taskRepository.findByIdAndUserId(taskId, user.getId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Task not found with id: " + taskId
            ));

        taskRepository.delete(task);
    }

    public int markOverdueTasks() {
        List<Task> overdueTasks = taskRepository
            .findOverdueTasks(LocalDate.now());

        for (Task task : overdueTasks) {
            task.setOverdue(true);
            task.setUpdatedAt(LocalDateTime.now());
        }
```

```
        taskRepository.saveAll(overdueTasks);
        return overdueTasks.size();
    }

    @Transactional(readOnly = true)
    public Page<TaskResponse> getAllTasks(Pageable pageable) {
        return taskRepository.findAll(pageable)
            .map(TaskResponse::new);
    }
}
```

Line-by-line explanation of key methods:

- `@Transactional` ensures that database operations within each method are wrapped in a transaction. If anything fails, the entire operation is rolled back.
- `@Transactional(readOnly = true)` on read-only methods tells Spring and the database driver that no writes will happen, which can improve performance.
- `@Cacheable(value = "tasks", key = "#username + '-' + #pageable.pageNumber")` caches the result. The next time the same user requests the same page, the result comes from the cache instead of the database.
- `@CacheEvict(value = "tasks", allEntries = true)` clears the cache when a task is created, updated, or deleted. This ensures the cache does not serve stale data.
- `findByIdAndUserId(taskId, user.getId())` ensures users can only access their own tasks. Without the `userId` check, a user could access any task by guessing its ID.
- `markOverdueTasks()` finds all tasks where the due date has passed, the task is not completed, and the overdue flag is not yet set. It then marks them as overdue. This method is called by a scheduled task.

UserService

Create `src/main/java/com/example/tasktracker/service/UserService.java`:

```java
package com.example.tasktracker.service;

import com.example.tasktracker.entity.User;
import com.example.tasktracker.exception.ResourceNotFoundException;
import com.example.tasktracker.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
```

```
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public Page<User> getAllUsers(Pageable pageable) {
        return userRepository.findAll(pageable);
    }

    public User getUserByUsername(String username) {
        return userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException(
                "User not found: " + username
            ));
    }
}
```

B.9 Scheduled Task for Overdue Check

Create `src/main/java/com/example/tasktracker/scheduler/OverdueTaskScheduler.java`:

```java
package com.example.tasktracker.scheduler;

import com.example.tasktracker.service.TaskService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class OverdueTaskScheduler {

    private static final Logger logger =
        LoggerFactory.getLogger(OverdueTaskScheduler.class);

    private final TaskService taskService;

    public OverdueTaskScheduler(TaskService taskService) {
        this.taskService = taskService;
    }

    @Scheduled(cron = "0 0 * * * *")
    public void checkOverdueTasks() {
```

```
        logger.info("Running overdue task check...");
        int count = taskService.markOverdueTasks();
        logger.info("Marked {} tasks as overdue", count);
    }
}
```

Line-by-line explanation:

- `@Scheduled(cron = "0 0 * * * *")` runs this method at the start of every hour. The cron expression means: second 0, minute 0, every hour, every day, every month, every day of the week.
- `taskService.markOverdueTasks()` calls the service method that finds and marks overdue tasks. It returns the count of newly marked tasks.
- We use SLF4J logging instead of `System.out.println` for proper log management.

Output in the console:

```
INFO  c.e.t.scheduler.OverdueTaskScheduler : Running overdue task check...
INFO  c.e.t.scheduler.OverdueTaskScheduler : Marked 3 tasks as overdue
```

B.10 REST Controllers

AuthController

Create `src/main/java/com/example/tasktracker/controller/AuthController.java`:

```java
package com.example.tasktracker.controller;

import com.example.tasktracker.dto.AuthResponse;
import com.example.tasktracker.dto.LoginRequest;
import com.example.tasktracker.dto.RegisterRequest;
import com.example.tasktracker.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication", description = "Register and login endpoints")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    @Operation(summary = "Register a new user")
    public ResponseEntity<AuthResponse> register(
            @Valid @RequestBody RegisterRequest request) {

        AuthResponse response = authService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/login")
    @Operation(summary = "Log in and receive a JWT token")
    public ResponseEntity<AuthResponse> login(
            @Valid @RequestBody LoginRequest request) {

        AuthResponse response = authService.login(request);
        return ResponseEntity.ok(response);
    }
}
```

Sample request and response for registration:

```
POST /api/auth/register
Content-Type: application/json

{
    "username": "alice",
    "email": "alice@example.com",
    "password": "password123"
}
```

Response (201 Created):

```json
{
    "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIs...",
    "username": "alice",
    "role": "USER"
}
```

Sample request and response for login:

```
POST /api/auth/login
Content-Type: application/json

{
    "username": "alice",
    "password": "password123"
}
```

Response (200 OK):

```json
{
    "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIs...",
    "username": "alice",
    "role": "USER"
}
```

TaskController

Create `src/main/java/com/example/tasktracker/controller/TaskController.java`:

```java
package com.example.tasktracker.controller;

import com.example.tasktracker.dto.TaskRequest;
import com.example.tasktracker.dto.TaskResponse;
import com.example.tasktracker.entity.TaskPriority;
import com.example.tasktracker.entity.TaskStatus;
import com.example.tasktracker.service.TaskService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/tasks")
@SecurityRequirement(name = "bearerAuth")
@Tag(name = "Tasks", description = "Task management endpoints")
public class TaskController {

    private final TaskService taskService;

    public TaskController(TaskService taskService) {
        this.taskService = taskService;
    }

    @PostMapping
    @Operation(summary = "Create a new task")
    public ResponseEntity<TaskResponse> createTask(
            @Valid @RequestBody TaskRequest request,
            Authentication authentication) {

        String username = authentication.getName();
        TaskResponse response = taskService.createTask(request, username);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping
    @Operation(summary = "Get all tasks for the current user")
    public ResponseEntity<Page<TaskResponse>> getMyTasks(
            @PageableDefault(size = 10, sort = "createdAt")
            Pageable pageable,
            Authentication authentication) {

        String username = authentication.getName();
        Page<TaskResponse> tasks =
            taskService.getTasksByUser(username, pageable);
        return ResponseEntity.ok(tasks);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get a specific task by ID")
    public ResponseEntity<TaskResponse> getTask(
            @PathVariable Long id,
            Authentication authentication) {

        String username = authentication.getName();
        TaskResponse response = taskService.getTaskById(id, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/status/{status}")
    @Operation(summary = "Get tasks filtered by status")
    public ResponseEntity<Page<TaskResponse>> getTasksByStatus(
```

```
            @PathVariable TaskStatus status,
            @PageableDefault(size = 10) Pageable pageable,
            Authentication authentication) {

        String username = authentication.getName();
        Page<TaskResponse> tasks =
            taskService.getTasksByStatus(username, status, pageable);
        return ResponseEntity.ok(tasks);
    }

    @GetMapping("/priority/{priority}")
    @Operation(summary = "Get tasks filtered by priority")
    public ResponseEntity<Page<TaskResponse>> getTasksByPriority(
            @PathVariable TaskPriority priority,
            @PageableDefault(size = 10) Pageable pageable,
            Authentication authentication) {

        String username = authentication.getName();
        Page<TaskResponse> tasks =
            taskService.getTasksByPriority(username, priority, pageable);
        return ResponseEntity.ok(tasks);
    }

    @PutMapping("/{id}")
    @Operation(summary = "Update a task")
    public ResponseEntity<TaskResponse> updateTask(
            @PathVariable Long id,
            @Valid @RequestBody TaskRequest request,
            Authentication authentication) {

        String username = authentication.getName();
        TaskResponse response =
            taskService.updateTask(id, request, username);
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/{id}/status")
    @Operation(summary = "Update task status")
    public ResponseEntity<TaskResponse> updateTaskStatus(
            @PathVariable Long id,
            @RequestBody Map<String, String> body,
            Authentication authentication) {

        String username = authentication.getName();
        TaskStatus status = TaskStatus.valueOf(
            body.get("status").toUpperCase()
        );
        TaskResponse response =
            taskService.updateTaskStatus(id, status, username);
        return ResponseEntity.ok(response);
    }

    @DeleteMapping("/{id}")
    @Operation(summary = "Delete a task")
    public ResponseEntity<Void> deleteTask(
```

```
            @PathVariable Long id,
            Authentication authentication) {

        String username = authentication.getName();
        taskService.deleteTask(id, username);
        return ResponseEntity.noContent().build();
    }
}
```

Line-by-line explanation of key features:

- `Authentication authentication` is injected by Spring Security. It contains the currently logged-in user's information (extracted from the JWT token by our filter).
- `authentication.getName()` returns the username stored in the JWT token.
- `@PageableDefault(size = 10, sort = "createdAt")` sets default pagination: 10 items per page, sorted by creation date. Clients can override with query parameters like `?page=0&size=20&sort=title`.
- `@SecurityRequirement(name = "bearerAuth")` tells Swagger that these endpoints require a Bearer token.
- `@PatchMapping("/{id}/status")` uses PATCH because we are partially updating the task (only the status field).

Sample request to create a task (with JWT token):

```
POST /api/tasks
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
    "title": "Complete Spring Boot book",
    "description": "Read all chapters and do the exercises",
    "priority": "HIGH",
    "dueDate": "2024-12-31"
}
```

Response (201 Created):

```json
{
    "id": 1,
    "title": "Complete Spring Boot book",
    "description": "Read all chapters and do the exercises",
    "status": "TODO",
    "priority": "HIGH",
    "dueDate": "2024-12-31",
    "overdue": false,
```

```
    "createdAt": "2024-01-15T10:30:00",
    "updatedAt": "2024-01-15T10:30:00",
    "username": "alice"
}
```

Sample request to get tasks with pagination:

```
GET /api/tasks?page=0&size=5&sort=priority,desc
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Response (200 OK):

```json
{
    "content": [
        {
            "id": 1,
            "title": "Complete Spring Boot book",
            "status": "TODO",
            "priority": "HIGH",
            "dueDate": "2024-12-31",
            "overdue": false,
            "username": "alice"
        }
    ],
    "pageable": {
        "pageNumber": 0,
        "pageSize": 5
    },
    "totalElements": 1,
    "totalPages": 1,
    "last": true,
    "first": true
}
```

AdminController

Create `src/main/java/com/example/tasktracker/controller/AdminController.java`:

```java
package com.example.tasktracker.controller;

import com.example.tasktracker.dto.TaskResponse;
import com.example.tasktracker.entity.User;
import com.example.tasktracker.service.TaskService;
import com.example.tasktracker.service.UserService;
import io.swagger.v3.oas.annotations.Operation;
```

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/admin")
@SecurityRequirement(name = "bearerAuth")
@Tag(name = "Admin", description = "Admin-only endpoints")
public class AdminController {

    private final UserService userService;
    private final TaskService taskService;

    public AdminController(UserService userService,
                           TaskService taskService) {
        this.userService = userService;
        this.taskService = taskService;
    }

    @GetMapping("/users")
    @Operation(summary = "Get all users (admin only)")
    public ResponseEntity<Page<User>> getAllUsers(
            @PageableDefault(size = 10) Pageable pageable) {

        return ResponseEntity.ok(userService.getAllUsers(pageable));
    }

    @GetMapping("/tasks")
    @Operation(summary = "Get all tasks from all users (admin only)")
    public ResponseEntity<Page<TaskResponse>> getAllTasks(
            @PageableDefault(size = 10) Pageable pageable) {

        return ResponseEntity.ok(taskService.getAllTasks(pageable));
    }
}
```

B.11 OpenAPI Configuration

Create `src/main/java/com/example/tasktracker/config/OpenApiConfig.java`:

```java
package com.example.tasktracker.config;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.info.Contact;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.security.SecurityScheme;
import org.springframework.context.annotation.Configuration;

@Configuration
@OpenAPIDefinition(
    info = @Info(
        title = "TaskTracker API",
        version = "1.0",
        description = "A task management REST API built with Spring Boot",
        contact = @Contact(name = "Developer", email = "dev@example.com")
    )
)
@SecurityScheme(
    name = "bearerAuth",
    type = SecuritySchemeType.HTTP,
    bearerFormat = "JWT",
    scheme = "bearer"
)
public class OpenApiConfig {
}
```

After starting the application, you can view the interactive API documentation at `http://localhost:8080/swagger-ui.html`.

B.12 Cache Configuration

Create `src/main/java/com/example/tasktracker/config/CacheConfig.java`:

```java
package com.example.tasktracker.config;

import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Configuration;

@Configuration
@EnableCaching
public class CacheConfig {
}
```

The `@EnableCaching` annotation activates Spring's cache infrastructure. Combined with `spring.cache.type=simple` in `application.properties`, this uses a `ConcurrentHashMap`-based

cache. The `@Cacheable` and `@CacheEvict` annotations in `TaskService` handle the caching logic.

B.13 Data Initializer

For testing purposes, let us create some sample data when the application starts.

Create `src/main/java/com/example/tasktracker/config/DataInitializer.java`:

```java
package com.example.tasktracker.config;

import com.example.tasktracker.entity.Role;
import com.example.tasktracker.entity.Task;
import com.example.tasktracker.entity.TaskPriority;
import com.example.tasktracker.entity.User;
import com.example.tasktracker.repository.TaskRepository;
import com.example.tasktracker.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.time.LocalDate;

@Configuration
public class DataInitializer {

    @Bean
    CommandLineRunner initData(UserRepository userRepository,
                               TaskRepository taskRepository,
                               PasswordEncoder passwordEncoder) {

        return args -> {
            // Create admin user
            User admin = new User(
                "admin",
                "admin@example.com",
                passwordEncoder.encode("admin123")
            );
            admin.setRole(Role.ADMIN);
            userRepository.save(admin);

            // Create regular user
            User alice = new User(
                "alice",
                "alice@example.com",
                passwordEncoder.encode("password123")
            );
```

```
            userRepository.save(alice);

            // Create sample tasks for alice
            taskRepository.save(new Task(
                "Read Spring Boot documentation",
                "Go through the official Spring Boot docs",
                TaskPriority.HIGH,
                LocalDate.now().plusDays(7),
                alice
            ));

            taskRepository.save(new Task(
                "Build a REST API",
                "Create a complete REST API with CRUD operations",
                TaskPriority.MEDIUM,
                LocalDate.now().plusDays(14),
                alice
            ));

            taskRepository.save(new Task(
                "Learn Docker basics",
                "Understand containers and images",
                TaskPriority.LOW,
                LocalDate.now().minusDays(2),
                alice
            ));

            System.out.println("Sample data initialized!");
            System.out.println("Admin: admin / admin123");
            System.out.println("User:  alice / password123");
        };
    }
}
```

Output when the application starts:

```
Sample data initialized!
Admin: admin / admin123
User:  alice / password123
```

B.14 Main Application Class

Update `src/main/java/com/example/tasktracker/TasktrackerApplication.java`:

```java
package com.example.tasktracker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class TasktrackerApplication {

    public static void main(String[] args) {
        SpringApplication.run(TasktrackerApplication.class, args);
    }
}
```

B.15 Writing Tests

Testing the AuthController

Create `src/test/java/com/example/tasktracker/controller/AuthControllerTest.java`:

```java
package com.example.tasktracker.controller;

import com.example.tasktracker.dto.AuthResponse;
import com.example.tasktracker.dto.RegisterRequest;
import com.example.tasktracker.security.JwtUtil;
import com.example.tasktracker.service.AuthService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class AuthControllerTest {

    @Autowired
    private MockMvc mockMvc;
```

```
    @Autowired
    private ObjectMapper objectMapper;

    @Test
    void registerShouldReturnToken() throws Exception {
        RegisterRequest request = new RegisterRequest(
            "testuser", "test@example.com", "password123"
        );

        mockMvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.token").isNotEmpty())
            .andExpect(jsonPath("$.username").value("testuser"))
            .andExpect(jsonPath("$.role").value("USER"));
    }

    @Test
    void registerWithInvalidDataShouldReturn400() throws Exception {
        RegisterRequest request = new RegisterRequest("", "", "12");

        mockMvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors").isNotEmpty());
    }

    @Test
    void loginWithValidCredentialsShouldReturnToken() throws Exception {
        String loginJson = """
            {
                "username": "alice",
                "password": "password123"
            }
            """;

        mockMvc.perform(post("/api/auth/login")
                .contentType(MediaType.APPLICATION_JSON)
                .content(loginJson))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.token").isNotEmpty())
            .andExpect(jsonPath("$.username").value("alice"));
    }

    @Test
    void loginWithInvalidPasswordShouldReturn401() throws Exception {
        String loginJson = """
            {
                "username": "alice",
                "password": "wrongpassword"
            }
            """;
```

```
        mockMvc.perform(post("/api/auth/login")
                .contentType(MediaType.APPLICATION_JSON)
                .content(loginJson))
            .andExpect(status().isUnauthorized());
    }
}
```

Testing the TaskController

Create `src/test/java/com/example/tasktracker/controller/TaskControllerTest.java`:

```java
package com.example.tasktracker.controller;

import com.example.tasktracker.dto.AuthResponse;
import com.example.tasktracker.dto.LoginRequest;
import com.example.tasktracker.service.AuthService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class TaskControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Autowired
    private AuthService authService;

    private String token;

    @BeforeEach
    void setUp() {
```

```
        AuthResponse response = authService.login(
            new LoginRequest("alice", "password123")
        );
        token = response.getToken();
    }

    @Test
    void createTaskShouldReturn201() throws Exception {
        String taskJson = """
            {
                "title": "Test task",
                "description": "A task for testing",
                "priority": "HIGH",
                "dueDate": "2024-12-31"
            }
            """;

        mockMvc.perform(post("/api/tasks")
                .header("Authorization", "Bearer " + token)
                .contentType(MediaType.APPLICATION_JSON)
                .content(taskJson))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.title").value("Test task"))
            .andExpect(jsonPath("$.priority").value("HIGH"))
            .andExpect(jsonPath("$.status").value("TODO"));
    }

    @Test
    void getTasksShouldReturnPaginatedResults() throws Exception {
        mockMvc.perform(get("/api/tasks")
                .header("Authorization", "Bearer " + token)
                .param("page", "0")
                .param("size", "5"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.content").isArray())
            .andExpect(jsonPath("$.totalElements").isNumber());
    }

    @Test
    void getTaskWithoutTokenShouldReturn403() throws Exception {
        mockMvc.perform(get("/api/tasks"))
            .andExpect(status().isForbidden());
    }

    @Test
    void deleteTaskShouldReturn204() throws Exception {
        // First, create a task
        String taskJson = """
            {
                "title": "Task to delete",
                "description": "Will be deleted",
                "priority": "LOW"
            }
            """;
```

```
        String responseBody = mockMvc.perform(post("/api/tasks")
                .header("Authorization", "Bearer " + token)
                .contentType(MediaType.APPLICATION_JSON)
                .content(taskJson))
            .andExpect(status().isCreated())
            .andReturn()
            .getResponse()
            .getContentAsString();

        Long taskId = objectMapper.readTree(responseBody).get("id").asLong();

        // Then, delete it
        mockMvc.perform(delete("/api/tasks/" + taskId)
                .header("Authorization", "Bearer " + token))
            .andExpect(status().isNoContent());
    }
}
```

Testing the TaskService

Create `src/test/java/com/example/tasktracker/service/TaskServiceTest.java`:

```java
package com.example.tasktracker.service;

import com.example.tasktracker.dto.TaskRequest;
import com.example.tasktracker.dto.TaskResponse;
import com.example.tasktracker.exception.ResourceNotFoundException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertThrows;

@SpringBootTest
class TaskServiceTest {

    @Autowired
    private TaskService taskService;

    @Test
    void createTaskShouldReturnTaskResponse() {
        TaskRequest request = new TaskRequest(
            "Service test task", "Testing the service", "HIGH", null
        );

        TaskResponse response = taskService.createTask(request, "alice");
```

```
        assertNotNull(response.getId());
        assertEquals("Service test task", response.getTitle());
        assertEquals("HIGH", response.getPriority());
        assertEquals("TODO", response.getStatus());
    }

    @Test
    void getTasksByUserShouldReturnPaginatedResults() {
        Page<TaskResponse> tasks = taskService.getTasksByUser(
            "alice", PageRequest.of(0, 10)
        );

        assertNotNull(tasks);
        assertEquals(0, tasks.getNumber());
    }

    @Test
    void getTaskByIdWithWrongUserShouldThrowException() {
        assertThrows(ResourceNotFoundException.class, () -> {
            taskService.getTaskById(999L, "alice");
        });
    }

    @Test
    void markOverdueTasksShouldReturnCount() {
        int count = taskService.markOverdueTasks();
        assertNotNull(count);
    }
}
```

Output when you run `mvn test`:

```
[INFO] Running com.example.tasktracker.controller.AuthControllerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.example.tasktracker.controller.TaskControllerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.example.tasktracker.service.TaskServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] Results:
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

B.16 Dockerfile

Create a `Dockerfile` in the project root directory:

```dockerfile
# Stage 1: Build the application
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
COPY .mvn ./.mvn
COPY mvnw .
RUN chmod +x mvnw
RUN ./mvnw clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Line-by-line explanation:

- `FROM eclipse-temurin:17-jdk-alpine AS build` uses a lightweight Alpine Linux image with Java 17 JDK for building the application. This is the "build stage."
- `COPY pom.xml .` and `COPY src ./src` copy the project files into the container.
- `RUN ./mvnw clean package -DskipTests` builds the JAR file inside the container, skipping tests (they should pass before you build the Docker image).
- `FROM eclipse-temurin:17-jre-alpine` starts a new stage with only the JRE (no JDK). This makes the final image smaller because you only need the runtime to run the application.
- `COPY --from=build /app/target/*.jar app.jar` copies the built JAR from the build stage.
- `EXPOSE 8080` documents that the application listens on port 8080.
- `ENTRYPOINT ["java", "-jar", "app.jar"]` starts the application.

Build and Run with Docker

```bash
# Build the Docker image
docker build -t tasktracker .

# Run the container
docker run -p 8080:8080 tasktracker
```

Output:

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.4.x)

INFO  --- Started TasktrackerApplication in 3.2 seconds
Sample data initialized!
Admin: admin / admin123
User:  alice / password123
```

B.17 Complete Project Structure

Here is the full project structure for reference:

```
tasktracker/
+-- src/
|   +-- main/
|   |   +-- java/com/example/tasktracker/
|   |   |   +-- TasktrackerApplication.java
|   |   |   +-- config/
|   |   |   |   +-- CacheConfig.java
|   |   |   |   +-- DataInitializer.java
|   |   |   |   +-- OpenApiConfig.java
|   |   |   +-- controller/
|   |   |   |   +-- AdminController.java
|   |   |   |   +-- AuthController.java
|   |   |   |   +-- TaskController.java
|   |   |   +-- dto/
|   |   |   |   +-- AuthResponse.java
|   |   |   |   +-- ErrorResponse.java
|   |   |   |   +-- LoginRequest.java
|   |   |   |   +-- RegisterRequest.java
|   |   |   |   +-- TaskRequest.java
|   |   |   |   +-- TaskResponse.java
|   |   |   +-- entity/
|   |   |   |   +-- Role.java
|   |   |   |   +-- Task.java
|   |   |   |   +-- TaskPriority.java
|   |   |   |   +-- TaskStatus.java
|   |   |   |   +-- User.java
|   |   |   +-- exception/
|   |   |   |   +-- DuplicateResourceException.java
|   |   |   |   +-- GlobalExceptionHandler.java
```

```
|   |   |   |   +-- ResourceNotFoundException.java
|   |   |   |   +-- UnauthorizedException.java
|   |   |   +-- repository/
|   |   |   |   +-- TaskRepository.java
|   |   |   |   +-- UserRepository.java
|   |   |   +-- scheduler/
|   |   |   |   +-- OverdueTaskScheduler.java
|   |   |   +-- security/
|   |   |   |   +-- JwtAuthenticationFilter.java
|   |   |   |   +-- JwtUtil.java
|   |   |   |   +-- SecurityConfig.java
|   |   |   +-- service/
|   |   |       +-- AuthService.java
|   |   |       +-- TaskService.java
|   |   |       +-- UserService.java
|   |   +-- resources/
|   |       +-- application.properties
|   +-- test/
|       +-- java/com/example/tasktracker/
|           +-- TasktrackerApplicationTests.java
|           +-- controller/
|           |   +-- AuthControllerTest.java
|           |   +-- TaskControllerTest.java
|           +-- service/
|               +-- TaskServiceTest.java
+-- Dockerfile
+-- pom.xml
```

B.18 API Endpoint Summary

Here is a quick reference of all the endpoints in the TaskTracker API:

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Log in and get JWT token
POST	/api/tasks	Authenticated	Create a new task
GET	/api/tasks	Authenticated	Get your tasks (paginated)
GET	/api/tasks/{id}	Authenticated	Get a specific task
GET	/api/tasks/status/{status}	Authenticated	Filter tasks by status

GET	/api/tasks/priority/{priority}	Authenticated	Filter tasks by priority
PUT	/api/tasks/{id}	Authenticated	Update a task
PATCH	/api/tasks/{id}/status	Authenticated	Update task status
DELETE	/api/tasks/{id}	Authenticated	Delete a task
GET	/api/admin/users	Admin only	Get all users
GET	/api/admin/tasks	Admin only	Get all tasks

```
Testing Flow:

1. Register: POST /api/auth/register
   -> Get JWT token

2. Create Task: POST /api/tasks (with token)
   -> Get task with ID

3. List Tasks: GET /api/tasks (with token)
   -> See paginated results

4. Update Status: PATCH /api/tasks/1/status (with token)
   -> Change TODO to IN_PROGRESS

5. Filter: GET /api/tasks/status/IN_PROGRESS (with token)
   -> See filtered results

6. Delete: DELETE /api/tasks/1 (with token)
   -> 204 No Content
```

Common Mistakes

1. Forgetting @EnableScheduling. The overdue task checker uses `@Scheduled`, which requires `@EnableScheduling` on the main application class or a configuration class.
2. Not adding @EnableCaching. The `@Cacheable` and `@CacheEvict` annotations are ignored unless caching is enabled.
3. Storing plain-text passwords. Always use `PasswordEncoder` to hash passwords before saving. Never store passwords in plain text.
4. Not checking task ownership. Always verify that the task belongs to the requesting user. Use `findByIdAndUserId` instead of just `findById`.

5. Using @RestController for WebSocket handlers. When you add WebSocket to this project later, remember to use `@Controller` for WebSocket message handlers.
6. Returning entities directly from controllers. Use DTOs (like `TaskResponse`) to control what data is sent to the client. Returning the `User` entity directly would expose the password hash.

Best Practices

1. Separate concerns into layers. Keep controllers thin. Put business logic in services. Let repositories handle data access.
2. Use DTOs for input and output. DTOs decouple your API contract from your database structure. You can change your entities without breaking the API.
3. Validate input at the controller level. Use `@Valid` and Jakarta Validation annotations to reject bad data before it reaches your service layer.
4. Handle exceptions globally. Use `@RestControllerAdvice` to return consistent error responses across all endpoints.
5. Write tests. Even basic tests catch many bugs. Test your controllers with MockMvc and your services with JUnit.
6. Use environment variables for secrets. Never hardcode JWT secrets or database passwords in `application.properties` for production. Use environment variables or a secrets manager.

Quick Summary

The TaskTracker API is a complete Spring Boot application that demonstrates entities and JPA relationships, repositories with custom queries and pagination, services with business logic and transactions, REST controllers with validation and authentication, JWT security with role-based access control, caching for performance, scheduled tasks for automated checks, global exception handling, OpenAPI documentation, integration and service tests, and Docker containerization. This project uses every major concept from the book and shows how they work together in a real application.

Key Points

- Start with entities and work your way up through repositories, services, and controllers.
- Use DTOs to separate your API contract from your internal data model.
- Use `@Valid` for input validation and `@RestControllerAdvice` for exception handling.
- JWT tokens provide stateless authentication. The server does not need to store sessions.
- Role-based access control (USER, ADMIN) restricts what users can do.
- `@Cacheable` improves read performance. `@CacheEvict` keeps the cache fresh.
- `@Scheduled` automates recurring tasks like checking for overdue items.
- OpenAPI (Swagger) generates interactive API documentation automatically.
- Docker packages your application into a portable container.

Practice Questions

1. Why do we use DTOs instead of returning the User entity directly from controllers?
2. Explain why the login method uses the same error message for both "user not found" and "wrong password." What security risk would a more specific message create?
3. What is the purpose of `@Transactional(readOnly = true)` on read-only service methods?
4. How does `findByIdAndUserId` in TaskRepository prevent users from accessing other users' tasks?
5. Why do we use a multi-stage Dockerfile? What is the benefit of separating build and runtime stages?

Exercises

Exercise 1: Add Task Search

Add a new endpoint `GET /api/tasks/search?keyword=spring` that searches tasks by title or description containing the keyword. Add the query method to `TaskRepository` and the logic to `TaskService`.

Exercise 2: Add User Profile

Create a `GET /api/profile` endpoint that returns the current user's information along with task statistics (total tasks, completed tasks, overdue tasks). Create a `ProfileResponse` DTO for this.

Exercise 3: Add Task Comments

Add a `Comment` entity with a many-to-one relationship to `Task`. Create endpoints to add comments to a task (`POST /api/tasks/{id}/comments`) and list comments for a task (`GET /api/tasks/{id}/comments`).

Congratulations!

You have built a complete, production-ready REST API from scratch. You started with empty files and ended with a fully functional application that includes authentication, authorization, validation, error handling, pagination, caching, scheduled tasks, documentation, tests, and Docker support.

This is a significant achievement. You now understand how all the pieces of a Spring Boot application fit together. You can confidently build similar applications for your own projects or for your employer.

What to Learn Next

Here are some topics to explore as you continue your Spring Boot journey:

1. Spring WebFlux - Build reactive, non-blocking applications for high-concurrency use cases.
2. Microservices with Spring Cloud - Break large applications into smaller, independently deployable services.
3. Message Queues (RabbitMQ, Apache Kafka) - Handle asynchronous communication between services.
4. PostgreSQL or MySQL - Replace H2 with a production-grade database.
5. CI/CD Pipelines - Automate testing and deployment with GitHub Actions, Jenkins, or GitLab CI.
6. Kubernetes - Orchestrate your Docker containers at scale.

7. Spring Boot Actuator - Add production-ready monitoring and health checks (covered in Chapter 28).
8. GraphQL with Spring - Build flexible APIs that let clients request exactly the data they need.

Keep building. Keep learning. The best way to master Spring Boot is to use it in real projects. Good luck!

Appendix C: Glossary of Spring Boot Terms

What You Will Learn

- The definitions of 70+ terms used throughout this book.
- How each term relates to Spring Boot development.
- Where in the book each concept is covered in detail.

Why This Chapter Matters

When you are learning a new technology, the hardest part is often the vocabulary. Spring Boot uses many terms that can be confusing when you first encounter them. This glossary gives you a single place to look up any term you come across in this book, in documentation, in tutorials, or in job interviews.

Think of this glossary as your Spring Boot dictionary. When you see a term you do not recognize, come here first. Each entry explains the term in plain English with a brief example where helpful.

Terms

@Autowired

An annotation that tells Spring to automatically inject (provide) a dependency. When Spring sees `@Autowired` on a constructor, field, or setter method, it looks for a matching bean in its container and provides it. In modern Spring Boot, constructor injection is preferred, and

`@Autowired` is optional on single-constructor classes.

```java
@Service
public class OrderService {
    private final EmailService emailService;

    // @Autowired is optional here because there is only one constructor
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

See: Chapter 6 (Dependency Injection).

@Bean

An annotation used on a method inside a `@Configuration` class. The method creates and returns an object, and Spring registers that object as a bean in the application context. Use `@Bean` when you need to configure a third-party library object that you cannot annotate with `@Component`.

```java
@Configuration
public class AppConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

See: Chapter 7 (Beans and Annotations).

BCrypt

A password hashing algorithm designed for security. It is intentionally slow, which makes it resistant to brute-force attacks. Spring Security provides `BCryptPasswordEncoder` to hash and verify passwords. You should never store plain-text passwords.

See: Chapter 19 (Password Encoding and Roles).

Bean

An object that is created and managed by the Spring IoC container. Beans are the building blocks of a Spring application. Spring creates them, injects their dependencies, and manages their lifecycle. Any class annotated with `@Component`, `@Service`, `@Repository`, or `@Controller` becomes a bean.

See: Chapter 7 (Beans and Annotations).

@Cacheable

An annotation that caches the return value of a method. The first time the method is called with specific arguments, the result is stored in the cache. Subsequent calls with the same arguments return the cached result instead of executing the method again. Requires `@EnableCaching` on a configuration class.

```java
@Cacheable(value = "products", key = "#id")
public Product getProduct(Long id) {
    return productRepository.findById(id).orElseThrow();
}
```

See: Chapter 24 (Caching).

@CacheEvict

An annotation that removes entries from the cache. Use this on methods that modify data to ensure the cache does not serve stale (outdated) values.

See: Chapter 24 (Caching).

CommandLineRunner

A functional interface in Spring Boot. If you create a bean of this type, Spring Boot runs it automatically after the application starts. Useful for initializing sample data, running startup checks, or printing diagnostic information.

See: Appendix B (Complete Project).

Component Scanning

The process by which Spring automatically discovers classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`, and other stereotype annotations. Spring scans the package of your main application class and all sub-packages. Any annotated class it finds becomes a bean in the application context.

See: Chapter 7 (Beans and Annotations).

@Component

A generic annotation that marks a class as a Spring-managed bean. Spring creates an instance of this class and manages it. More specific versions include `@Service`, `@Repository`, and `@Controller`, which add semantic meaning.

See: Chapter 7 (Beans and Annotations).

@Configuration

An annotation that marks a class as a source of bean definitions. Methods annotated with `@Bean` inside a `@Configuration` class produce beans managed by Spring. Configuration classes are where you set up third-party libraries, security rules, and custom components.

See: Chapter 7 (Beans and Annotations).

@ControllerAdvice

An annotation that makes a class a global handler for exceptions thrown by any controller. Combined with `@ExceptionHandler` methods, it provides centralized error handling. `@RestControllerAdvice` is a convenience variant that adds `@ResponseBody` automatically.

```java
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse(404, ex.getMessage()));
```

```
    }
}
```

See: Chapter 12 (Exception Handling).

CORS (Cross-Origin Resource Sharing)

A security mechanism that controls which websites can make requests to your API. By default, browsers block requests from a different domain. CORS headers tell the browser which origins are allowed. Spring Boot provides `@CrossOrigin` and `WebMvcConfigurer` to configure CORS.

See: Chapter 20 (CORS Configuration).

CRUD

An acronym for Create, Read, Update, Delete. These are the four basic operations for managing data. In Spring Boot, you implement CRUD through REST endpoints: POST (create), GET (read), PUT/PATCH (update), and DELETE (delete).

See: Chapter 15 (Repositories and CRUD).

CSRF (Cross-Site Request Forgery)

A type of web attack where a malicious website tricks a user's browser into making an unwanted request to your application. Spring Security enables CSRF protection by default for form-based applications. For stateless REST APIs using JWT tokens, CSRF is typically disabled because the token itself prevents this type of attack.

See: Chapter 17 (Spring Security Introduction).

DAO (Data Access Object)

A design pattern that provides an abstract interface to a database. In Spring Boot, the `@Repository` annotation marks a class as a DAO. Spring Data JPA repositories serve the same purpose with much less code.

See: Chapter 15 (Repositories and CRUD).

@DeleteMapping

A shortcut annotation for `@RequestMapping(method = RequestMethod.DELETE)`. It maps HTTP DELETE requests to a specific controller method. Used for endpoints that delete resources.

See: Chapter 10 (Request and Response Handling).

Dependency Injection (DI)

A design pattern where an object receives its dependencies from an external source instead of creating them itself. In Spring Boot, the framework creates objects (beans) and injects them where needed. This makes code easier to test and maintain because you can swap implementations without changing the dependent class.

See: Chapter 6 (Dependency Injection).

DispatcherServlet

The front controller in Spring MVC. It receives all incoming HTTP requests and routes them to the correct controller method based on the URL, HTTP method, and other factors. Spring Boot configures the DispatcherServlet automatically. You rarely interact with it directly.

```
Client Request --> DispatcherServlet --> Controller --> Service --> Repository
```

See: Chapter 9 (Your First REST Controller).

DTO (Data Transfer Object)

A simple Java object used to transfer data between layers of an application. DTOs separate the data sent to or received from clients from the internal entity structure. This improves security (you control what is exposed) and flexibility (you can change entities without breaking the API).

See: Chapter 10 (Request and Response Handling).

@EnableCaching

An annotation that activates Spring's cache abstraction. Without this annotation, `@Cacheable` and `@CacheEvict` are ignored. Place it on a configuration class or the main application class.

See: Chapter 24 (Caching).

@EnableScheduling

An annotation that enables Spring's scheduled task execution. Without this annotation, `@Scheduled` methods are ignored. Place it on a configuration class or the main application class.

See: Chapter 23 (Scheduling).

@EnableWebSocketMessageBroker

An annotation that enables WebSocket message handling with a STOMP message broker. It sets up the infrastructure for WebSocket communication, including message routing and subscription management.

See: Appendix A (WebSocket).

@Entity

A JPA annotation that marks a class as a database entity. Each entity maps to a table in the database. Each instance of the entity represents a row in that table. Fields in the entity class map to columns.

```java
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```
    private double price;
}
```

See: Chapter 14 (Spring Data JPA and H2).

@ExceptionHandler

An annotation on a method that handles a specific type of exception. When the specified exception is thrown, Spring calls this method instead of returning a default error page. Usually used inside a `@ControllerAdvice` class.

See: Chapter 12 (Exception Handling).

Filter

A component that intercepts HTTP requests and responses before they reach a controller or after the controller processes them. Filters operate at the servlet level and are useful for cross-cutting concerns like logging, authentication, and CORS. Spring Security uses a chain of filters to enforce security.

See: Chapter 13 (Interceptors and Filters).

@GetMapping

A shortcut annotation for `@RequestMapping(method = RequestMethod.GET)`. It maps HTTP GET requests to a specific controller method. Used for endpoints that retrieve data.

See: Chapter 9 (Your First REST Controller).

H2 Database

A lightweight, in-memory relational database written in Java. It is embedded in your application (no separate installation needed) and perfect for development and testing. Spring Boot provides auto-configuration for H2, and it includes a web console for viewing data at `/h2-console`.

See: Chapter 14 (Spring Data JPA and H2).

@Id

A JPA annotation that marks a field as the primary key of an entity. Every entity must have exactly one field annotated with `@Id`. Combined with `@GeneratedValue`, the database automatically assigns unique values.

See: Chapter 14 (Spring Data JPA and H2).

Interceptor

A Spring MVC component that intercepts requests before and after controller execution. Unlike filters (which operate at the servlet level), interceptors operate at the Spring MVC level and have access to the handler method information. Implement `HandlerInterceptor` to create one.

See: Chapter 13 (Interceptors and Filters).

IoC (Inversion of Control)

A design principle where the framework controls the creation and management of objects instead of the programmer. In traditional programming, your code creates objects. With IoC, the Spring container creates objects and gives them to your code. Dependency Injection is the most common way to implement IoC.

See: Chapter 6 (Dependency Injection).

Jackson

A Java library for converting between Java objects and JSON. Spring Boot uses Jackson automatically to serialize (Java to JSON) controller responses and deserialize (JSON to Java) request bodies. You can customize Jackson behavior with annotations like `@JsonIgnore`, `@JsonProperty`, and `@JsonFormat`.

See: Chapter 10 (Request and Response Handling).

Jakarta EE

The enterprise Java platform, formerly known as Java EE (Enterprise Edition). Spring Boot 3.x uses Jakarta EE APIs instead of the older javax packages. This means imports changed from `javax.persistence` to `jakarta.persistence` and from `javax.servlet` to `jakarta.servlet`.

See: Chapter 1 (What Is Spring Boot).

JAR (Java Archive)

A file format that packages Java classes, resources, and metadata into a single compressed file. Spring Boot creates "fat JARs" (also called "uber JARs") that include your application code, all dependencies, and an embedded web server. You run them with `java -jar myapp.jar`.

See: Chapter 3 (Your First Spring Boot Project).

JPA (Java Persistence API)

A specification for mapping Java objects to relational database tables. JPA defines annotations like `@Entity`, `@Table`, `@Column`, and `@Id`. Hibernate is the most popular implementation of JPA. Spring Data JPA builds on top of JPA to provide repositories with automatic query generation.

See: Chapter 14 (Spring Data JPA and H2).

@JoinColumn

A JPA annotation that specifies the foreign key column for a relationship. Used with `@ManyToOne` and `@OneToOne` to define which column in the database links two tables together.

See: Chapter 14 (Spring Data JPA and H2).

JSON (JavaScript Object Notation)

A lightweight data format used for exchanging data between a client and server. JSON uses key-value pairs and is easy for both humans and machines to read. Spring Boot automatically

converts Java objects to JSON and JSON to Java objects using Jackson.

```json
{
    "id": 1,
    "name": "Spring Boot",
    "version": "3.4"
}
```

See: Chapter 9 (Your First REST Controller).

JWT (JSON Web Token)

A compact, self-contained token used for authentication and information exchange. A JWT contains three parts: a header (algorithm), a payload (user data), and a signature (verification). In Spring Boot, JWT tokens are used for stateless authentication. The client includes the token in the Authorization header of each request.

See: Chapter 18 (JWT Authentication).

Logback

The default logging framework in Spring Boot. It is an improved version of Log4j and implements the SLF4J API. Configuration is done through `logback-spring.xml` or `application.properties`. Logback supports multiple log levels: TRACE, DEBUG, INFO, WARN, ERROR.

See: Chapter 22 (Logging).

@ManyToOne

A JPA annotation that defines a many-to-one relationship between entities. For example, many tasks belong to one user. The entity with `@ManyToOne` contains the foreign key column.

See: Chapter 14 (Spring Data JPA and H2).

Maven

A build automation tool for Java projects. Maven manages dependencies (libraries your project uses), compiles code, runs tests, and packages your application. It uses a `pom.xml` file to define project settings and dependencies. Spring Boot projects typically use Maven or Gradle.

See: Chapter 2 (Development Environment).

@MessageMapping

A WebSocket annotation similar to `@RequestMapping` for HTTP. It maps incoming STOMP messages to a handler method based on the destination. The application destination prefix (like `/app`) is stripped before matching.

See: Appendix A (WebSocket).

MockMvc

A Spring testing utility that simulates HTTP requests to your controllers without starting a real web server. It lets you test controllers in isolation, verify response status codes, check response body content, and validate headers. Use it with `@AutoConfigureMockMvc` in integration tests.

```java
mockMvc.perform(get("/api/products"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$[0].name").value("Laptop"));
```

See: Chapter 26 (Testing).

@OneToMany

A JPA annotation that defines a one-to-many relationship between entities. For example, one user has many tasks. The `mappedBy` attribute specifies the field in the child entity that owns the relationship.

See: Chapter 14 (Spring Data JPA and H2).

@OneToOne

A JPA annotation that defines a one-to-one relationship between entities. For example, one user has one profile. One side owns the relationship and contains the foreign key.

See: Chapter 14 (Spring Data JPA and H2).

OpenAPI

A specification for describing REST APIs. It defines endpoints, request and response formats, authentication methods, and more in a machine-readable format. Springdoc OpenAPI generates this specification automatically from your Spring Boot controllers. The Swagger UI provides a visual, interactive interface for the API documentation.

See: Chapter 27 (API Documentation).

@PatchMapping

A shortcut annotation for `@RequestMapping(method = RequestMethod.PATCH)`. It maps HTTP PATCH requests to a specific controller method. Used for endpoints that partially update a resource (only the fields that are provided are changed).

See: Chapter 10 (Request and Response Handling).

@PathVariable

An annotation that extracts a value from the URL path and binds it to a method parameter. For example, in `/api/users/42`, the `42` can be captured as a path variable.

```java
@GetMapping("/api/users/{id}")
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}
```

See: Chapter 10 (Request and Response Handling).

POJO (Plain Old Java Object)

A regular Java class with no special requirements. It does not extend framework-specific classes or implement framework-specific interfaces. Spring Boot encourages the use of POJOs for entities, DTOs, and other objects. The framework handles the rest through annotations.

@PostMapping

A shortcut annotation for `@RequestMapping(method = RequestMethod.POST)`. It maps HTTP POST requests to a specific controller method. Used for endpoints that create new resources.

See: Chapter 9 (Your First REST Controller).

Profile

A named set of configuration settings that can be activated in different environments. For example, you might have a `dev` profile for development (using H2) and a `prod` profile for production (using PostgreSQL). Activate a profile with `spring.profiles.active=dev` in `application.properties` or as a command-line argument.

See: Chapter 8 (Properties and Profiles).

@PutMapping

A shortcut annotation for `@RequestMapping(method = RequestMethod.PUT)`. It maps HTTP PUT requests to a specific controller method. Used for endpoints that fully replace a resource.

See: Chapter 10 (Request and Response Handling).

@Query

A Spring Data JPA annotation that defines a custom JPQL or native SQL query on a repository method. Use this when the automatically generated query method name would be too long or complex.

```java
@Query("SELECT t FROM Task t WHERE t.status = :status AND t.user.id = :userId")
List<Task> findTasksByStatusAndUser(@Param("status") TaskStatus status,
                                    @Param("userId") Long userId);
```

See: Chapter 15 (Repositories and CRUD).

@Repository

A stereotype annotation that marks a class as a data access component. It is a specialization of `@Component` and tells Spring to translate database-specific exceptions into Spring's `DataAccessException` hierarchy. In Spring Data JPA, you typically use interfaces that extend `JpaRepository` instead of writing repository classes by hand.

See: Chapter 15 (Repositories and CRUD).

@RequestBody

An annotation that tells Spring to convert the JSON body of an HTTP request into a Java object. Spring uses Jackson to perform this conversion automatically.

```java
@PostMapping("/api/users")
public User createUser(@RequestBody User user) {
    return userService.save(user);
}
```

See: Chapter 10 (Request and Response Handling).

@RequestMapping

An annotation that maps HTTP requests to controller methods. It can specify the URL path, HTTP method, content types, and more. Shortcut annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, and `@PatchMapping` are more commonly used.

See: Chapter 9 (Your First REST Controller).

@RequestParam

An annotation that extracts a query parameter from the URL and binds it to a method parameter. For example, in `/api/products?category=electronics`, the `category` value is extracted.

```java
@GetMapping("/api/products")
public List<Product> search(@RequestParam String category) {
    return productService.findByCategory(category);
}
```

See: Chapter 10 (Request and Response Handling).

REST (Representational State Transfer)

An architectural style for building web services. RESTful APIs use HTTP methods (GET, POST, PUT, DELETE) to perform operations on resources identified by URLs. Data is typically exchanged in JSON format. Spring Boot makes it easy to build REST APIs with `@RestController`.

See: Chapter 9 (Your First REST Controller).

@RestController

An annotation that combines `@Controller` and `@ResponseBody`. It marks a class as a REST controller where every method's return value is automatically converted to JSON and written to the HTTP response body. This is the most common annotation for building REST APIs.

See: Chapter 9 (Your First REST Controller).

@Scheduled

An annotation that marks a method to be executed on a schedule. You can specify a fixed rate (every N milliseconds), a fixed delay (N milliseconds after the previous execution finishes), or a cron expression for complex schedules. Requires `@EnableScheduling`.

```java
@Scheduled(fixedRate = 60000)  // Every 60 seconds
public void checkStatus() {
    // This runs automatically
}
```

See: Chapter 23 (Scheduling).

@Scope

An annotation that defines the lifecycle scope of a bean. The default scope is `singleton` (one instance per application). Other scopes include `prototype` (new instance every time), `request` (one per HTTP request), and `session` (one per HTTP session).

See: Chapter 7 (Beans and Annotations).

SecurityFilterChain

A Spring Security bean that defines the security rules for your application. It specifies which URLs require authentication, which roles can access which endpoints, how sessions are managed, and where custom filters are inserted. Configured as a `@Bean` method that takes `HttpSecurity` as a parameter.

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/public/**").permitAll()
        .anyRequest().authenticated()
    );
    return http.build();
}
```

See: Chapter 17 (Spring Security Introduction).

@SendTo

A WebSocket annotation that specifies the destination topic for the return value of a `@MessageMapping` method. All clients subscribed to that topic receive the message.

See: Appendix A (WebSocket).

@Service

A stereotype annotation that marks a class as a service component containing business logic. It is a specialization of `@Component`. Using `@Service` instead of `@Component` communicates the class's purpose and follows the layered architecture pattern.

See: Chapter 7 (Beans and Annotations).

SimpMessagingTemplate

A Spring class for sending WebSocket STOMP messages programmatically. Unlike `@SendTo` (which only works in `@MessageMapping` methods), `SimpMessagingTemplate` can send messages from any part of your application, including services, scheduled tasks, and REST controllers.

See: Appendix A (WebSocket).

SLF4J (Simple Logging Facade for Java)

A logging API that provides a common interface for various logging frameworks (Logback, Log4j2, etc.). Spring Boot uses SLF4J with Logback by default. You create a logger with `LoggerFactory.getLogger(MyClass.class)` and use methods like `logger.info()`, `logger.error()`, and `logger.debug()`.

```java
private static final Logger logger = LoggerFactory.getLogger(MyClass.class);
logger.info("Application started");
```

See: Chapter 22 (Logging).

Spring Boot Actuator

A module that adds production-ready features to your application. It exposes endpoints for health checks (`/actuator/health`), metrics (`/actuator/metrics`), environment info, and more. Useful for monitoring applications in production.

See: Chapter 28 (Actuator and Monitoring).

Spring Boot DevTools

A development-time tool that provides automatic application restart when code changes, live reload for browser resources, and sensible development defaults (like disabling template caching). Add the `spring-boot-devtools` dependency to use it. It is automatically disabled in production.

See: Chapter 2 (Development Environment).

@SpringBootApplication

The main annotation for a Spring Boot application. It combines three annotations: `@Configuration` (marks the class as a configuration source), `@EnableAutoConfiguration` (enables Spring Boot's auto-configuration), and `@ComponentScan` (enables component scanning in the current package and sub-packages).

See: Chapter 3 (Your First Spring Boot Project).

SpringBootTest

A test annotation that loads the full Spring application context for integration testing. It starts the entire Spring Boot application, including all beans, configurations, and auto-configurations. Use it when you need to test how components work together.

See: Chapter 26 (Testing).

Starter

A Spring Boot dependency that bundles together all the libraries you need for a specific feature. For example, `spring-boot-starter-web` includes Spring MVC, Jackson, Tomcat, and validation. Starters eliminate the need to figure out which individual libraries and versions are compatible.

See: Chapter 1 (What Is Spring Boot).

STOMP (Simple Text Oriented Messaging Protocol)

A messaging protocol that adds structure to WebSocket communication. It defines commands like CONNECT, SUBSCRIBE, SEND, and MESSAGE, along with destinations (topics) and

headers. Spring Boot uses STOMP for its WebSocket support.

See: Appendix A (WebSocket).

Swagger

A set of tools for designing, building, and documenting REST APIs. In the Spring Boot ecosystem, Springdoc OpenAPI generates a Swagger UI that provides an interactive, browser-based interface for exploring and testing your API endpoints. Access it at `/swagger-ui.html`.

See: Chapter 27 (API Documentation).

@Table

A JPA annotation that specifies the database table name for an entity. If not provided, JPA uses the class name as the table name. Use `@Table` when the class name conflicts with a database reserved word (like "user" in H2).

See: Chapter 14 (Spring Data JPA and H2).

Tomcat

An open-source web server and servlet container. Spring Boot embeds Tomcat by default, so you do not need to install it separately. Your application runs on Tomcat's built-in HTTP server (port 8080 by default). You can switch to Jetty or Undertow if preferred.

See: Chapter 1 (What Is Spring Boot).

@Transactional

An annotation that wraps a method's execution in a database transaction. If the method completes successfully, the transaction is committed. If an exception is thrown, the transaction is rolled back (all changes are undone). Use `@Transactional(readOnly = true)` for read-only operations to optimize performance.

See: Chapter 15 (Repositories and CRUD).

@Valid

An annotation that triggers Jakarta Bean Validation on a request body or method parameter. When placed before a `@RequestBody` parameter, Spring validates the object using annotations like `@NotBlank`, `@Size`, `@Email`, and `@Min`. If validation fails, Spring throws a `MethodArgumentNotValidException`.

```java
@PostMapping("/api/users")
public User createUser(@Valid @RequestBody CreateUserRequest request) {
    return userService.create(request);
}
```

See: Chapter 11 (Input Validation).

@Value

An annotation that injects a value from `application.properties` or an environment variable into a field or constructor parameter. It uses the `${property.name}` syntax.

```java
@Value("${app.name}")
private String appName;

@Value("${server.port:8080}")  // with default value
private int serverPort;
```

See: Chapter 8 (Properties and Profiles).

WAR (Web Application Archive)

A file format for packaging web applications to be deployed on an external servlet container like Tomcat or Jetty. Spring Boot defaults to JAR packaging with an embedded server, but WAR packaging is available if you need to deploy to an existing application server.

See: Chapter 30 (Docker and Deployment).

WebSocket

A communication protocol that provides persistent, full-duplex (two-way) communication between a client and server over a single TCP connection. Unlike HTTP, where the client must initiate every exchange, WebSocket allows the server to push messages to the client at any time without a prior request.

See: Appendix A (WebSocket).

YAML (YAML Ain't Markup Language)

A human-readable data format used for configuration files. Spring Boot supports YAML as an alternative to `.properties` files. YAML uses indentation to show hierarchy, which some developers find more readable than dot-separated property names.

```yaml
# application.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
server:
  port: 8080
```

See: Chapter 8 (Properties and Profiles).

Additional Terms

Application Context

The central container in Spring that holds all beans and their configurations. It is the implementation of the IoC container. When your Spring Boot application starts, it creates an application context, scans for components, creates beans, and injects dependencies.

Auto-Configuration

A Spring Boot feature that automatically configures beans based on the dependencies on your classpath and the properties you set. For example, if `spring-boot-starter-data-jpa` and

an H2 driver are on the classpath, Spring Boot automatically configures a DataSource, EntityManagerFactory, and transaction manager.

See: Chapter 5 (Auto-Configuration).

Classpath

The set of directories and JAR files where Java looks for classes and resources. In a Maven project, the classpath includes your compiled classes and all dependencies defined in `pom.xml`. Spring Boot uses the classpath to detect which auto-configurations to activate.

Connection Pool

A cache of database connections that are reused across requests. Creating a new database connection for every request is slow. A connection pool (like HikariCP, which Spring Boot uses by default) keeps a set of connections open and lends them to requests as needed.

Embedded Server

A web server (like Tomcat, Jetty, or Undertow) that is bundled inside your Spring Boot application. You do not need to install or configure it separately. When you run your JAR file, the embedded server starts automatically.

Hibernate

The most popular implementation of the JPA specification. It handles the mapping between Java objects and database tables, generates SQL queries, and manages database sessions. Spring Data JPA uses Hibernate under the hood by default.

HikariCP

The default JDBC connection pool in Spring Boot. It is known for being fast and lightweight. Spring Boot configures HikariCP automatically when you add a database dependency.

JPQL (Java Persistence Query Language)

A query language similar to SQL but operates on JPA entities instead of database tables. JPQL uses entity class names and field names instead of table and column names. Used with the `@Query` annotation in Spring Data JPA repositories.

Lazy Loading

A JPA strategy where related entities are loaded from the database only when they are accessed for the first time, not when the parent entity is loaded. For example, with `@ManyToOne(fetch = FetchType.LAZY)`, the related entity is loaded only when you call the getter method.

Migration (Database)

The process of evolving your database schema over time. Tools like Flyway and Liquibase manage database migrations by tracking which changes have been applied. In production, you use migrations instead of `ddl-auto=create-drop`.

Pageable

A Spring Data interface that contains pagination and sorting information: page number, page size, and sort order. Spring MVC automatically creates a `Pageable` object from query parameters like `?page=0&size=10&sort=name`.

See: Chapter 16 (Pagination and Sorting).

ResponseEntity

A class that represents an HTTP response, including the status code, headers, and body. It gives you full control over the response returned by a controller method.

```java
return ResponseEntity.status(HttpStatus.CREATED).body(newUser);
return ResponseEntity.ok(users);
return ResponseEntity.noContent().build();
```

See: Chapter 10 (Request and Response Handling).

Singleton

The default scope for Spring beans. A singleton bean has exactly one instance in the entire application context. Every time you inject that bean, you get the same instance.

See: Chapter 7 (Beans and Annotations).

Stereotype Annotation

An annotation that marks a class for a specific role in the application: `@Component` (generic), `@Service` (business logic), `@Repository` (data access), and `@Controller` (web handler). They all result in the class being registered as a bean, but they communicate the class's purpose.

See: Chapter 7 (Beans and Annotations).

Transaction

A sequence of database operations that are treated as a single unit. Either all operations succeed (commit) or all are undone (rollback). Spring's `@Transactional` annotation manages transactions automatically.

Quick Reference Table

Term	One-Line Definition
@Autowired	Injects a dependency automatically
Bean	Object managed by Spring's container

@Cacheable	Caches a method's return value
Component Scanning	Spring's auto-discovery of annotated classes
@ControllerAdvice	Global exception handler for controllers
CORS	Controls cross-domain API access
CSRF	Protection against cross-site request attacks
DAO	Pattern for abstracting database access
DI	Objects receive their dependencies externally
DispatcherServlet	Routes all HTTP requests to controllers
DTO	Object for transferring data between layers
@Entity	Maps a Java class to a database table
Filter	Intercepts HTTP requests at the servlet level
H2	Lightweight, in-memory Java database
Interceptor	Intercepts requests at the Spring MVC level
IoC	Framework controls object creation
Jackson	Converts between Java objects and JSON
JAR	Packaged Java application file
Jakarta EE	Enterprise Java platform (formerly Java EE)
JPA	Specification for object-relational mapping
JWT	Self-contained authentication token
Logback	Default logging framework in Spring Boot
Maven	Build tool and dependency manager
MockMvc	Tests controllers without a running server
@PathVariable	Extracts a value from the URL path
POJO	Plain Java class with no framework dependencies
Profile	Named configuration for different environments
@Repository	Marks a data access class
REST	Architectural style for web APIs

@RestController	Controller that returns JSON responses
SecurityFilterChain	Defines security rules for the application
@Service	Marks a business logic class
SLF4J	Logging API used by Spring Boot
Starter	Bundled dependency for a specific feature
Swagger	Interactive API documentation tool
Tomcat	Default embedded web server
@Transactional	Wraps method execution in a transaction
@Valid	Triggers input validation on a parameter
WebSocket	Protocol for real-time, two-way communication
YAML	Human-readable configuration file format

Common Mistakes

1. Confusing @Component with @Bean. Use `@Component` on your own classes. Use `@Bean` on methods inside `@Configuration` classes when you need to configure third-party objects.
2. Confusing REST and HTTP. REST is an architectural style. HTTP is a protocol. REST uses HTTP methods to define operations, but not all HTTP APIs follow REST principles.
3. Confusing JPA and Hibernate. JPA is a specification (a set of rules). Hibernate is an implementation (the code that follows those rules). Spring Data JPA provides a layer on top of both.
4. Confusing Filter and Interceptor. Filters work at the servlet level (before Spring MVC). Interceptors work at the Spring MVC level (after the DispatcherServlet). Use filters for security, CORS, and logging. Use interceptors for request-specific logic.
5. Using @Autowired on fields. Field injection with `@Autowired` makes testing harder. Prefer constructor injection, which is the recommended approach in modern Spring.

Best Practices

1. Learn the vocabulary. Understanding terms helps you read documentation faster and communicate with other developers more effectively.
2. Use the correct stereotype annotation. Use `@Service` for services, `@Repository` for data access, and `@Controller`/`@RestController` for web handlers. Do not use `@Component` for everything.
3. Understand the layers. Controller handles HTTP, Service handles business logic, Repository handles data. Keep them separate.
4. Read the official documentation. The Spring Boot reference documentation at `docs.spring.io` is thorough and well-organized. This glossary is a starting point. The docs go deeper.
5. Practice with real projects. Build the TaskTracker project in Appendix B. Then build your own projects. Terms become natural when you use them in context.

Quick Summary

This glossary covers more than 70 terms that you will encounter when working with Spring Boot. Terms range from core concepts like Dependency Injection and IoC to specific annotations like @RestController and @Transactional. Each entry explains the term in plain English and points you to the chapter in this book where you can learn more. Use this glossary as a reference whenever you encounter an unfamiliar term.

Key Points

- Spring Boot uses many terms from Java, JPA, Spring Framework, and web development.
- Annotations like @Component, @Service, @Repository, and @Controller are stereotype annotations that mark classes for specific roles.
- JPA is a specification. Hibernate is an implementation. Spring Data JPA provides a convenience layer on top.
- DTOs separate what clients see from how your database is structured.
- Dependency Injection and IoC are the core principles that make Spring Boot work.
- Understanding the vocabulary makes reading documentation and learning new features much faster.

Practice Questions

1. What is the difference between @Component, @Service, @Repository, and @Controller? When would you use each one?
2. Explain the difference between a Filter and an Interceptor. At what level does each one operate?
3. What is the relationship between JPA, Hibernate, and Spring Data JPA? How do they relate to each other?
4. What is the purpose of a DTO? Why would you use a DTO instead of returning an entity directly from a controller?
5. Explain the difference between @Cacheable and @CacheEvict. When would you use each one?

Exercises

Exercise 1: Build a Flashcard Set

Pick 20 terms from this glossary. Create a flashcard for each one with the term on the front and the definition on the back. Quiz yourself until you can define each term from memory.

Exercise 2: Map the Annotations

Draw a diagram showing where each major annotation is used in a Spring Boot application. Start with `@SpringBootApplication` at the top and show how `@RestController`, `@Service`, `@Repository`, `@Entity`, and other annotations fit into the layered architecture.

Exercise 3: Write Example Code

Choose five annotations you find most confusing. For each one, write a small code example (5 to 10 lines) that demonstrates how it works. Run the code to verify your understanding.